Reference textbook: Introduction to Algorithms, 3rd Edition
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

# Algorithms

## Full course handout

A4 · Two slides per page · Lin Cing

---

## Contents

1. Algorithm Analysis and Correctness

01 · Algorithm Analysis and Correctness 3

2. Recursion, Recurrences, and Divide-and-Conquer

02 · Recursion, Recurrences, and Divide-and-Conquer 29

3. Sorting, Selection, and Randomized Analysis

03 · Sorting, Selection, and Randomized Analysis 61

4. Data Structures and Amortized Analysis

04 · Data Structures and Amortized Analysis 138

21 · Disjoint sets: merge groups, never split them 253

5. Dynamic Programming

05 · Dynamic programming 273

6. Greedy Algorithms

06 · Greedy methods 312

7. Graph Traversal and Connectivity

07 · Graph traversal and connectivity 334

8. Minimum Spanning Trees and Shortest Paths

08 · Minimum spanning trees and shortest paths 367

9. Network Flow

09 · Network flows 404

10. Computational Geometry

10 · Computational geometry 434

11. Complexity and Approximation

11 · Complexity and approximation 456

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Algorithms

## Full course handout

A4 · Two slides per page · Lin Cing

---

## Contents

12. String Matching

12 · String matching 505

13. Matrix Operations and FFT

13 · Matrix operations and FFT 528

14. Number-Theoretic Algorithms

14 · Number-theoretic algorithms 578

15. Linear Programming

15 · Linear programming: models and certificates 616

16. Parallel Algorithms

16 · Multithreaded algorithms 635

Reference Appendices

A · Summations 657

B · Sets, relations, functions, graphs, and trees 671

C · Counting and probability 689

D · Matrices 713

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Algorithm Analysis and Correctness

Running a program tells us what happened on one execution. Analyzing an algorithm asks more: **Does it return a correct answer for every legal input? How does its cost grow with the input?**

We develop specifications and proofs, exact operation counts, asymptotic comparisons, and loop sums. Insertion sort provides one continuous example, from individual moves to a general cost formula.

Prerequisites: arrays, loops, elementary algebra and logarithms. Area and limit arguments also use basic calculus.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 1–3; summations in Appendix A.

## Reading route

1. Specifications and termination → linear-search invariants → a complete insertion-sort analysis.
2. Asymptotic quantifiers → growth comparisons → sums and loop counts.
3. Integer boundaries and space → interpreting costs → mathematical extensions.

Read the main sequence before the extensions; revisit definitions when a symbol is unfamiliar.



# Specify the answer before reading the code

A problem has a legal input domain $D$ and an output relation $R(x)$. One input $x \in D$ may admit several answers. An algorithm is a precise procedure for producing one of them.

A sorting input is a sequence of records with a key and a fixed identity. In $(3, a), (1, b), (3, c)$, the letters distinguish records; they are not secondary sorting keys.

| Output requirement | How to check it |
|---|---|
| Nondecreasing keys | Adjacent keys satisfy $A[i].key \leq A[i+1].key$ |
| No lost or duplicated records | The output is a permutation, including multiplicities |
| Stability, when required | Equal keys retain their original relative order |

Sortedness alone is insufficient: deleting every record leaves a sorted empty sequence, but does not sort the original input.

## Identities distinguish equal keys

Input [3a, 1b, 3c] → sort by key; letters identify records

| Sorted, preserved, stable | Sorted and preserved | Sorted, but missing a record |
|---|---|---|
| 1b 3a 3c | 1b 3c 3a | 1b 3a |
| All three requirements hold | Equal-key a and c are reversed | 3c is lost: invalid sorting |

Stability is an additional requirement. Swapping equal-key records can preserve sortedness and the input multiset while violating stability. Stability also requires the original order of $a$ and $c$.

Empty and singleton inputs are already sorted. A correct implementation must handle them without reading a nonexistent first record.



## A correct result and termination are separate obligations

**Partial correctness** means that if execution terminates, its output satisfies the specification. Total correctness additionally requires termination on every legal input.

```
while true
    skip
```

This program never returns an incorrect answer, but never returns an answer at all. A proof about its result after termination does not establish that it completes the task.

For a loop, a common termination argument uses a nonnegative integer that strictly decreases in every iteration. A finite program listing or a finite collection of tests does not replace that argument.

## Linear search: specify which occurrence to return

Given $A[1..n]$ and target $x$, return the **first** index with $A[i] = x$, or NIL if none exists. Allow $n = 0$; one record read or comparison costs a constant.

```
FIRST-MATCH(A, x)
    i = 1
    while i <= n
        if A[i] == x: return i
        i = i + 1
    return NIL
```

Arrays are indexed from 1. An interval $A[a..b]$ with $a > b$ is empty; it does not access an element. Here and in the sorting pseudocode, $n$ denotes the input array length.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Array references are different from record identities

Our pseudocode accesses an array through a reference. Updating $A[i]$ changes the array visible to the caller. Rebinding the procedure's local variable to another array does not rebind the caller's variable.

| Operations | Result |
|---|---|
| $A = [3, 1]$; $B = A$; then $B[1] = 8$ | Both names refer to one array; $A$ becomes $[8, 1]$ |
| $A = [3, 1]$; $B = \mathit{copy}(A)$; then $B[1] = 8$ | The arrays have separate storage; $A$ stays $[3, 1]$ |

A shallow array copy may still contain references to the same objects. Independent array storage does not imply a deep copy of every object. The identities $a, b, c$ used in sorting identify records, not necessarily their memory addresses.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## The search invariant records what has been ruled out

Before each `i <= n` test: $1 \le i \le n+1$, and no position in $A[1..i-1]$ matches $x$.

| Stage | Reason |
|---|---|
| Initialization | $i = 1$; the excluded prefix is empty |
| Maintenance | A failed comparison followed by $i \leftarrow i + 1$ extends that prefix by one |
| Match found | $A[i] = x$, and every earlier position has been excluded |
| Loop exhausted | $i = n + 1$; every position has been excluded, so NIL is correct |

Every iteration that does not return decreases the nonnegative integer $n - i + 1$ by 1. Thus execution stops. For $n = 0$, it returns NIL immediately without reading $A[1]$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Before i is excluded; after i is still unknown

A = [6, 2, 6, 5]; target x = 5

| | 1 | 2 | 3 | 4 | |
|---|---|---|---|---|---|
| i = 1 | 6 | 2 | 6 | 5 | Excluded: empty<br>Current: no match |
| i = 3 | 6 | 2 | 6 | 5 | Excluded: A[1..2]<br>Current: no match |
| i = 4 | 6 | 2 | 6 | 5 | Excluded: A[1..3]<br>Current: match; return 4 |

Green: excluded. Teal border: current. White: not yet tested.

For $A = [6, 2, 6, 5]$, search for 5. Before $i = 3$, only positions 1–2 are excluded. A failed comparison extends that prefix; at $i = 4$, the first match returns. **Untested does not mean excluded.**

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Three complete executions of the same code

| Input and target | Positions compared | Return value | Key comparisons |
|---|---|---|---|
| $[4, 2, 4]$, target 4 | 1 | 1 | 1 |
| $[4, 2, 4]$, target 7 | 1, 2, 3 | NIL | 3 |
| Empty array, target 4 | None | NIL | 0 |

The cases test first occurrence, exhaustion, and empty input; the invariant supplies the general proof.

If the first matching position, conditional on success, is uniform over $1..n$, the expected comparison count conditional on success is $(n + 1)/2$. Another distribution requires another calculation.



## State the cost model before measuring cost

Initially use a word-RAM model: indices and records occupy a constant number of machine words; reads, writes, comparisons, and supported basic arithmetic operations have constant cost. This is a model, not a claim that every real operation takes the same time.

| Object of analysis | Possible cost unit |
|---|---|
| Array search | Key comparisons or reads |
| Sorting | Comparisons, moves, auxiliary space |
| Large-integer algorithms | Bit lengths and multiword arithmetic |
| External storage | Block transfers |

For integers with $b$ bits, arbitrary-precision multiplication is not automatically one constant-time operation. Read every bound together with its model.

## Input length is not necessarily the numerical value

A positive integer $N$ has binary length $b = \lfloor \lg N \rfloor + 1$, where $\lg = \log_2$. A loop with $N$ iterations may be exponential in the input length $b$.

For example, $N = 13$ is `1101`, only four bits long. Enumerating every integer up to $N$ takes $\Theta(N)$ iterations. Since $2^{b-1} \le N < 2^b$, that is $\Theta(2^b)$ iterations as a function of $b$.

Array size usually means the number of records $n$. Graph bounds often need both the vertex count $V$ and edge count $E$. Writing $O(V + E)$ preserves that distinction.

### Choose the cost function: best, worst, or average

For inputs of length $n$ in $D_n$, let $C(x)$ be the cost on input $x$.

$$B(n) = \min_{x \in D_n} C(x), \qquad W(n) = \max_{x \in D_n} C(x).$$

Average cost additionally needs a distribution $P_n$:

$$A(n) = \mathbb{E}_{x \sim P_n}[C(x)] = \sum_{x \in D_n} P_n(x) C(x).$$

**Big-O does not mean worst case.** Each of $B, W, A$ is a function with its own upper and lower bounds. For a randomized algorithm, also distinguish randomness in the input from the algorithm's own random choices.



## Insertion sort extends a sorted prefix

Use $[(7, a), (2, b), (5, c), (2, d), (1, e)]$, abbreviated $[7a, 2b, 5c, 2d, 1e]$. Save the next record as `key` and shift larger records on its left one position right.

```
INSERTION-SORT(A)
    for j = 2 to n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
```

`and` short-circuits left to right, so $A[i]$ is not read when $i = 0$. For $n \le 1$ the outer loop is empty; strict comparison keeps older equal-key records before the new one.

## Every iteration leaves a checkable result

| $j$ / saved record | Key comparisons, right to left | Shifts $h_j$ | Array after insertion |
|---|---|---|---|
| 2 / $2b$ | $7 > 2$; reach boundary | 1 | $2b, 7a, 5c, 2d, 1e$ |
| 3 / $5c$ | $7 > 5$; $2 > 5$ is false | 1 | $2b, 5c, 7a, 2d, 1e$ |
| 4 / $2d$ | $7 > 2, 5 > 2$; $2 > 2$ is false | 2 | $2b, 2d, 5c, 7a, 1e$ |
| 5 / $1e$ | $7, 5, 2d, 2b$ all exceed 1 | 4 | $1e, 2b, 2d, 5c, 7a$ |

$2b$ remains before $2d$. There are eight shifts but ten key comparisons: iterations 3 and 4 each have a final failed comparison; iterations 2 and 5 reach $i = 0$ without another key comparison.



## A logical hole explains temporary duplicates

Fixed j = 4; saved key = 2d

| | 1 | 2 | 3 | 4 | |
|---|---|---|---|---|---|
| 1 | 2b | 5c | 7a | □ | 7 > 2: shift 7a right |
| 2 | 2b | 5c | □ | 7a | 5 > 2: shift 5c right |
| 3 | 2b | □ | 5c | 7a | 2 > 2 is false: stop shifting |
| 4 | 2b | 2d | 5c | 7a | Insert key; 2b stays before 2d |

□ is a logical hole, not a written value. The saved key remains outside the array.

The box is a **logical hole**, not a written empty marker. The positions outside it plus saved key retain the input. After the first shift, physical memory holds two copies of $7a$; the whole physical prefix is not a permutation.

## Outer invariant: order, preservation, and stability

Let $S$ be a snapshot of the original input. Before each iteration, $A[1..j-1]$ is a stable sorting of $S[1..j-1]$, and $A[j..n] = S[j..n]$.

| Stage | How the invariant holds |
|---|---|
| Start, $j = 2$ | One record is already sorted; the suffix is unchanged |
| Maintenance | Shift the larger suffix and insert key; equal keys do not cross; the unprocessed suffix is unchanged |
| Stop | At $j = n + 1$, the sorted prefix contains the whole original input |

The inner index decreases at every shift, with at most $j - 1$ shifts; there are at most $n - 1$ outer iterations. Thus the algorithm terminates. Inputs with $n \leq 1$ are handled separately by the empty loop.



## Inner invariant: include the position of the hole

Fix $j$. Let $B = A[1..j]$ at the start of this iteration, with $B[1..j-1]$ sorted. Save $key = B[j]$ and place the logical hole at $i + 1$.

Before each inner test:

1. $A[1..i] = B[1..i]$.
2. $A[i + 2..j] = B[i + 1..j - 1]$; every key in this right segment exceeds key.
3. Excluding the hole and adding the saved key preserves exactly the records in $B$.

Initially the right segment is empty. Copying $A[i]$ into the hole and decrementing $i$ preserves all three properties. On stopping, the left segment is at most key and the right segment exceeds it; filling the hole restores a sorted, record-preserving prefix.

## Count comparisons, moves, and guard tests separately

For $n \geq 2$, let $h_j$ count shifts in iteration $j$. Let $q_j = 1$ if it stops at a failed key comparison with $i \geq 1$, and $q_j = 0$ otherwise.

| Operation | Iteration $j$ | Entire execution |
|---|---|---|
| Shift write | $h_j$ | $\sum_{j=2}^{n} h_j$ |
| Final key write | 1 | $n-1$ |
| Complete inner guard test | $h_j + 1$ | $\sum h_j + n - 1$ |
| Key comparison | $h_j + q_j$ | $\sum h_j + \sum q_j$ |

Here $\sum h_j = 8$ and $\sum q_j = 2$: ten key comparisons, twelve guard tests, and twelve array writes. Reading key into a local variable is not an array write.



## Why does the shift count equal the inversion count?

An inversion is a pair of original positions $p < q$ with $S[p].key > S[q].key$. Denote the total by $I(S)$.

When inserting the original record $S[j]$, the earlier $j-1$ records are sorted. Exactly those with keys larger than $S[j]$ shift right. Thus $h_j$ counts precisely the inversions whose right endpoint is $j$.

Every inversion has a unique right endpoint, giving

$$\sum_{j=2}^{n} h_j = I(S), \qquad T(S) = \Theta(n + I(S)).$$

Equal keys do not form inversions. The example has $1, 1, 2, 4$ inversions by right endpoint, totaling eight, exactly as in the shift table.

## Best, worst, and average for one algorithm

| Input assumption | Inversions | Time |
|---|---|---|
| Already nondecreasing | 0 | $\Theta(n)$ |
| Distinct keys in decreasing order | $n(n-1)/2$ | $\Theta(n^2)$ |
| All permutations of distinct keys equally likely | Expected $n(n-1)/4$ | Expected $\Theta(n^2)$ |

For the last row, let $X_{pq}$ indicate whether positions $p < q$ form an inversion. A uniform permutation gives $\mathbb{E}[X_{pq}] = 1/2$. By linearity of expectation,

$$\mathbb{E}[I] = \sum_{p<q} \mathbb{E}[X_{pq}] = \binom{n}{2}\frac{1}{2}.$$

Independence between these indicators is unnecessary. Repeated keys or a nonuniform input model require a new distribution argument.



## What asymptotic analysis ignores, and what it retains

We study growth as $n \to \infty$, allowing fixed multiplicative constants and finitely many small inputs to be disregarded for the bound. Small inputs still require correct behavior.

Here $n$ is a positive integer, $f$ is eventually nonnegative, and $g$ is eventually strictly positive. $O(g), \Omega(g), \Theta(g)$ are **sets of functions**. The conventional notation $f = O(g)$ means $f \in O(g)$.

For example, $n \in O(n^2)$ is true but loose. Use $\Theta$ when both sides have been established; Big-O alone does not mean an exact growth order.

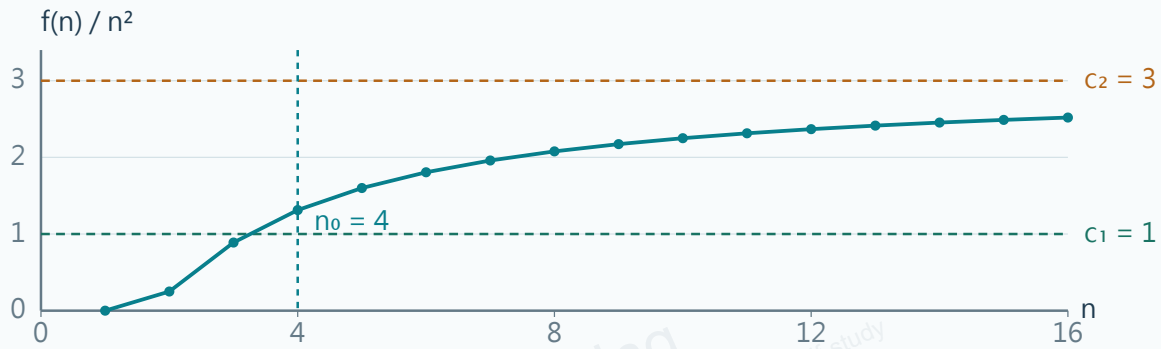
### The quantifiers in upper, lower, and tight bounds

| Notation | Formal requirement: for every integer $n \geq n_0$ |
|---|---|
| $f \in O(g)$ | There exist $c > 0, n_0$ with $0 \leq f(n) \leq cg(n)$ |
| $f \in \Omega(g)$ | There exist $c > 0, n_0$ with $0 \leq cg(n) \leq f(n)$ |
| $f \in \Theta(g)$ | There exist $c_1, c_2 > 0, n_0$ with $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ |

Constants may depend on the fixed functions, **not on input size** $n$. The threshold need not be the smallest possible one.

To refute $f \in O(g)$, every candidate $c, n_0$ must admit an $n \geq n_0$ with $f(n) > cg(n)$. Defeating only one proposed pair is insufficient.

## Plot the constants and the threshold together




For every n ≥ 4, the ratio lies between 1 and 3; earlier values need not.

Let $f(n) = 3n^2 - 8n + 5$. For $n \geq 4$, $f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1$; for $n \geq 1$, $f(n) \leq 3n^2$.

Thus $c_1 = 1, c_2 = 3, n_0 = 4$ prove $f \in \Theta(n^2)$. The plot illustrates the ratio. The inequalities, rather than finitely many plotted points, cover every $n \geq 4$.



## Prove that the same function has no linear upper bound

We have $f(n) = 3n^2 - 8n + 5 \geq n^2$ for $n \geq 4$. Suppose $f \in O(n)$. Then some fixed $c, n_0$ satisfy $f(n) \leq cn$ for every $n \geq n_0$.

Choose an integer $n > \max\{4, n_0, c\}$. It follows that

$$n^2 \leq f(n) \leq cn \quad \Longrightarrow \quad n \leq c,$$

contradicting $n > c$. Therefore $f \notin O(n)$.

The order of choices matters: allow arbitrary candidate constants first, then choose a large enough input. One fixed test input cannot refute every candidate bound.

## Use the larger of two thresholds

Suppose $(c_U, N_U)$ witnesses $f \in O(g)$ and $(c_L, N_L)$ witnesses $f \in \Omega(g)$. Both inequalities hold for $n \geq \max(N_U, N_L)$, so

$$\Theta(g) = O(g) \cap \Omega(g).$$

Other familiar rules also have assumptions:

| Property | Reason or condition |
|---|---|
| Transitivity of $O$ | Multiply the constants; take the larger threshold |
| $f \in O(g) \Leftrightarrow g \in \Omega(f)$ | Read the same inequality in reverse with reciprocal constant |
| $f + g = \Theta(\max(f, g))$ | For nonnegative functions, $\max(f, g) \leq f + g \leq 2\max(f, g)$ |

If cancellation is allowed, the last rule cannot be used without adjustment: $n + (-n) = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## little-o works for every positive multiplier

$$f \in o(g) \iff \forall c > 0\ \exists n_0(c)\ \forall n \geq n_0(c) :\ 0 \leq f(n) < cg(n).$$

Big-O asks for one multiplier; little-o asks for **every positive multiplier**. For $7n \in o(n^2)$, given $c > 0$, choose an integer $n_0 > 7/c$.

Likewise $f \in \omega(g)$ requires $0 \leq cg(n) < f(n)$ eventually for every $c > 0$. Hence $n^2 \in \omega(7n)$.

For eventually positive functions, little-o is equivalent to $f/g \to 0$, and little-omega to $f/g \to \infty$. $7n^2$ and $n^2$ have the same order; their ratio does not tend to zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A ratio limit is a tool, not the only criterion

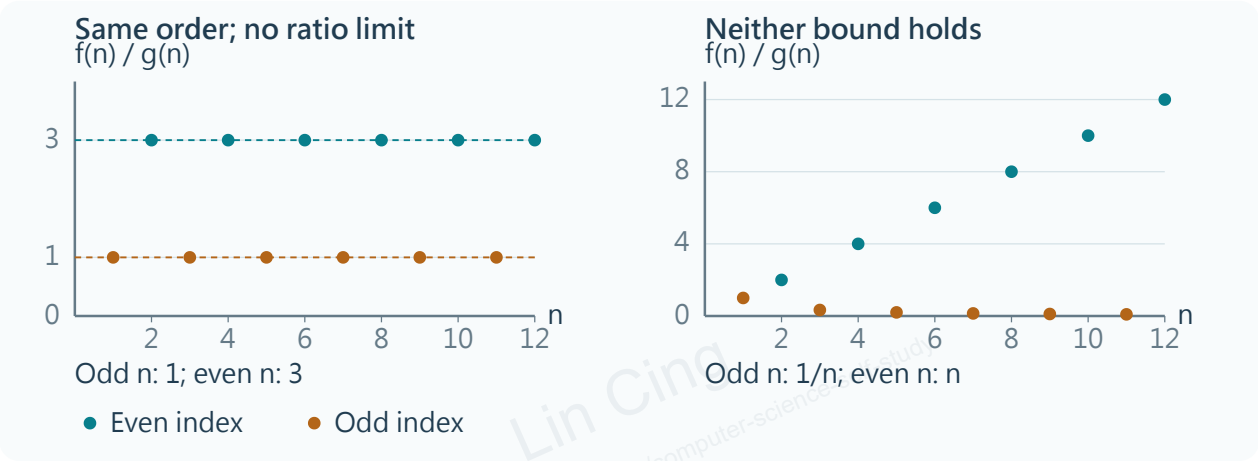
| $\lim f(n)/g(n)$ | Conclusion |
|---|---|
| 0 | $f \in o(g)$ |
| Finite positive $L$ | $f \in \Theta(g)$ |
| $+\infty$ | $f \in \omega(g)$ |
| Does not exist | Continue the analysis; incomparability does not follow |

For finite positive $L$, choose $\varepsilon = L/2$. Eventually $L/2 < f/g < 3L/2$, providing two fixed positive multipliers.

The first three rows require the stated limit to exist. Logarithmic comparison also requires positive arguments: $\lg f / \lg g \to 1$ alone does **not** imply $f = \Theta(g)$.



## An oscillating ratio can still be tightly bounded




Left: $f = n(2 + (-1)^n), g = n$ satisfy $n \le f \le 3n$ despite oscillation. Right: $g = n$; use $f = n^2$ for even $n$ and $f = 1$ for odd $n$. Even ratios are unbounded, refuting $O(g)$; odd ratios tend to zero, refuting $\Omega(g)$.

## Do not exchange function bounds and input quantifiers

Let $C(x)$ be input cost and $W(n) = \max_{x \in D_n} C(x)$. If $W(n) \in O(n^2)$, there are **shared** constants $c, n_0$ such that every $n \geq n_0$ and every $x \in D_n$ satisfy $C(x) \leq cn^2$.

If a separate constant may grow with each input, the statement does not provide that uniform upper bound.

Insertion sort has $W(n) = \Theta(n^2)$, yet a sorted input takes $\Theta(n)$. A tight worst-case bound does not say that every input requires quadratic time.

### Three practical approaches to comparing growth

| Approach | Useful for | Common mistake |
|---|---|---|
| Definition and inequalities | Polynomials, oscillation, explicit constants | Only sampling values or proving one side |
| Ratio limit | Positive functions with a manageable limit | Treating nonexistence as incomparability |
| Take logarithms | Positive products, powers, exponentials | Inferring original tight bounds from tight logarithms |

For $f = n \lg n, g = n$, $\lg f / \lg g = 1 + \lg\lg n / \lg n \to 1$, but $f/g = \lg n \to \infty$. Thus $f \in \omega(g)$.

Logs can simplify comparison. A fixed multiplicative bound still requires the original ratio, or a sufficiently precise bound on the difference of logarithms.



## Growth hierarchies require fixed constants

For fixed $a > 1, b > 0, \varepsilon > 0$, and $k > 0$,

$$1 \prec \lg\lg n \prec (\lg n)^b \prec n^\varepsilon \prec a^n \prec n! \prec n^n.$$

Here $f \prec g$ means $f = o(g)$, for sufficiently large positive integers.

With $x = \ln n$, $(\lg n)^b / n^\varepsilon$ is a fixed constant times $x^b / e^{\varepsilon x}$, which tends to zero. Also $\ln(n^k / a^n) = k \ln n - n \ln a \to -\infty$, proving $n^k = o(a^n)$.

The following panel justifies the basic limit $\ln n / n \to 0$. If a base or exponent varies with $n$, these fixed-constant conclusions need not apply.

### Why does a positive power dominate a logarithm?

For real $x \geq 4$, the derivative of $\sqrt{x} - \ln x$ is $(\sqrt{x} - 2)/(2x) \geq 0$, and its value at 4 is positive. Hence $0 \leq \ln x / x \leq 1/\sqrt{x} \to 0$.

Now fix $b, \varepsilon > 0$ and set $t = \ln n$. Taking natural logarithms gives

$$\ln \frac{(\ln n)^b}{n^\varepsilon} = b \ln t - \varepsilon t = t \left( b \frac{\ln t}{t} - \varepsilon \right).$$

Since $\ln t / t \to 0$, the parenthesis is eventually at most $-\varepsilon/2$. The logarithm tends to $-\infty$, so the original ratio tends to zero. Thus $(\ln n)^b = o(n^\varepsilon)$; a fixed change of log base changes only a constant.

## Logarithms, powers, and changes of base

| Expression | Interpretation |
|---|---|
| $\log_a n = \lg n / \lg a$ | Fixed $a > 1$: log bases differ by a constant |
| $\lg(n^k) = k \lg n$ | Take a power, then a logarithm |
| $\lg^k n = (\lg n)^k$ | Take a logarithm, then a power |
| $2^{\lg n} = n$ | Matching exponential and logarithm bases |
| $2^{2 \lg n} = n^2$ | A constant in an exponent cannot simply be discarded |

$\lg n$ and $2 \lg n$ have the same order; $2^{\lg n}$ and $2^{2 \lg n}$ do not. A varying log base is another issue: $\log_n n = 1$ does not behave like a fixed-base logarithm.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## From a loop to a sum: recheck each iteration's range

```
count = 0
for i = 1 to n
    for j = 1 to i
        count = count + 1
```

Iteration $i$ performs $i$ increments, so

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

Saying the inner loop averages about half the maximum is not a proof. The exact sum supplies a formula. Alternatively, $n^2$ is an upper bound, while the final half of the iterations each perform at least $n/2$ operations, giving a lower bound $n^2/4$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# A triangular iteration space is still quadratic

n = 6; one cell is one visit(i, j)

Upper: at most 6 × 6 = 36 cells. Lower: three cells in each of rounds 4–6.

Each cell is one inner operation. For $n = 6$, $1 + 2 + 3 + 4 + 5 + 6 = 21$ cells. Taking three cells from each of rows 4–6 gives $3 \times 3 = 9$: a concrete instance of the preceding general quadratic lower bound.



# A doubling loop counts how often 2 is multiplied

```
i = 1
while i <= n
    visit(i)
    i = 2*i
```

With unbounded integer arithmetic and $n \geq 1$, it visits $1, 2, 4, \ldots, 2^{\lfloor \lg n \rfloor}$, exactly $\lfloor \lg n \rfloor + 1$ values. For example, input 13 visits $1, 2, 4, 8$: four values.

A fixed-width implementation must avoid overflow in `2*i`. After visiting, stop if $i > \lfloor n/2 \rfloor$; otherwise doubling is safe. A mathematical termination proof does not automatically cover machine overflow.

## A geometric sum has the order of its last term

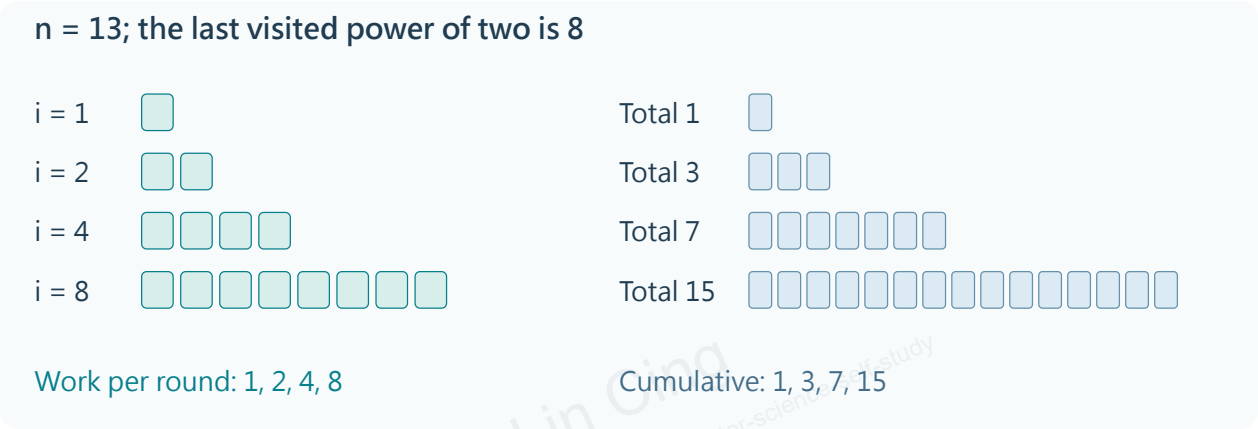
If the outer values are $i = 1, 2, 4, \ldots, 2^k \leq n$ and each iteration performs $i$ operations, the total is

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

Since $2^k \leq n < 2^{k+1}$, we have $n/2 < 2^k \leq n$. Thus the total is $\Theta(n)$, not $\Theta(n \lg n)$.

For $n = 13$, the four iterations perform $1, 2, 4, 8$ operations, totaling 15. Replacing every iteration's work by $n$ gives an upper bound, but it does not establish a tight bound.

## Sum the changing work, not the final work times the rounds




Rounds cost $1, 2, 4, 8$; cumulative work is $1, 3, 7, 15$. Generally, $2^k \le \sum_{r=0}^{k} 2^r < 2 \cdot 2^k$. The total is within twice the last round, with no extra factor for the number of rounds.



## Bound a harmonic sum by areas on both sides

Let $H_n = \sum_{i=1}^{n} 1/i$. The function $1/x$ decreases on the positive real axis, so

$$\int_1^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_1^{n} \frac{dx}{x}.$$

Thus $\ln(n+1) \le H_n \le 1 + \ln n$. For $n \ge 3$, $1 + \ln n \le 2 \ln n$, giving $H_n = \Theta(\ln n) = \Theta(\lg n)$.

For the upper bound, separate the first term 1 and place later terms below the curve on the preceding intervals. For the lower bound, left-endpoint rectangles cover the integral. The following panel separates the two constructions.

### Power sums need not have a closed form to be bounded

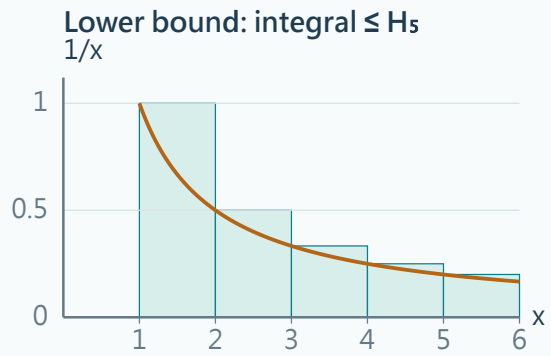
For fixed real $p \ge 0$, let $S_p(n) = \sum_{i=1}^{n} i^p$. There are $n$ terms, each at most $n^p$, so $S_p(n) \le n^{p+1}$.

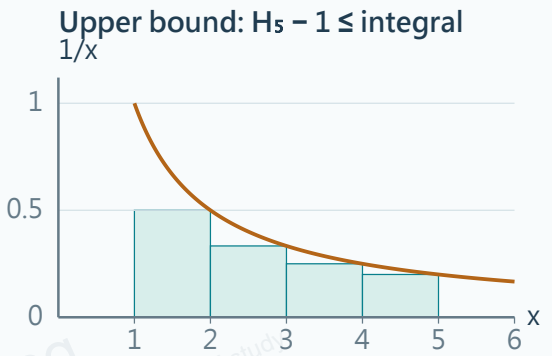
For a lower bound, retain $i = \lceil n/2 \rceil .. n$. There are at least $n/2$ such terms, each at least $(n/2)^p$, so

$$S_p(n) \ge \frac{n}{2} \left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}.$$

Thus $S_p(n) = \Theta(n^{p+1})$. The factors are fixed because $p$ is fixed. The case $p = -1$ is harmonic and does not satisfy this conclusion.

## One curve, two rectangle constructions




Left: height $1/i$ on $[i, i+1]$ covers the curve. Right: terms $i = 2..5$ lie below it on $[i-1, i]$. The common-scale panels give $1.7918 \approx \ln 6 \leq H_5 = 137/60 \approx 2.2833 \leq 1 + \ln 5 \approx 2.6094$.



## Dependent nested loops can take n log n time

```
for i = 1 to n
    j = i
    while j <= n
        visit(i, j)
        j = j + i
```

The inner loop visits $i, 2i, \ldots, \lfloor n/i \rfloor i$, giving

$$S(n) = \sum_{i=1}^{n} \lfloor n/i \rfloor .$$

For $x \geq 1$, $x/2 \leq \lfloor x \rfloor \leq x$. Hence $nH_n/2 \leq S(n) \leq nH_n$, proving $\Theta(n \lg n)$. At $n = 6$, the counts are $6, 3, 2, 1, 1, 1$, totaling 14.

## Take logarithms before comparing factorial growth

For $n \geq 4$, $n! \leq n^n$, so $\lg(n!) \leq n \lg n$.

At least $n/2$ factors in the latter half are at least $n/2$, giving

$$\lg(n!) \geq \frac{n}{2} \lg(n/2) \geq \frac{n}{4} \lg n.$$

Hence $\lg(n!) = \Theta(n \lg n)$. This does not imply $n! = \Theta(n^n)$: that ratio tends to zero.

For fixed $a > 1$, $\ln(n!/a^n) \geq (n/2)\ln(n/2) - n \ln a \to \infty$, proving $a^n = o(n!)$. Use Stirling's formula for a more precise factor; an error inside a logarithm cannot be exponentiated and discarded indiscriminately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Boundaries for floor, ceiling, and remainder

For real $x$, $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$. For integer $n$, this implies

$$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n.$$

For integer $a$ and positive integer $m$, use the mathematical remainder

$$a \bmod m = a - m\lfloor a/m \rfloor, \qquad 0 \leq a \bmod m < m.$$

For example, $-7 \bmod 3 = 2$, since $-7 = 3(-3) + 2$. A language's `%` operator may instead use a quotient truncated toward zero and produce a negative remainder. Check the semantics before translating a formula into code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## When can successive integer divisions be combined?

For $x \geq 0$ and positive integers $a, b$,

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor, \qquad \left\lceil \frac{\lceil x/a \rceil}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil.$$

For floor, set $q = \lfloor x/(ab) \rfloor$, so $abq \leq x < ab(q+1)$. Divide by $a$ and take floor: $bq \leq \lfloor x/a \rfloor \leq bq + b - 1$. Dividing by $b$ and taking floor now gives $q$.

For $x = 17, a = 3, b = 2$, successive floors give $17 \to 5 \to 2$, matching $\lfloor 17/6 \rfloor = 2$. Positivity and integrality of the divisors support the integer-interval argument.

### The ceiling proof uses a strict lower endpoint

Set $q = \lceil x/(ab) \rceil$. Then $ab(q-1) < x \leq abq$. Divide by $a$ and take ceiling:

$$b(q-1) + 1 \leq \lceil x/a \rceil \leq bq.$$

Dividing by $b$ and taking ceiling gives $q$. For $x = 17, a = 3, b = 2$, the steps $17 \to 6 \to 3$ match $\lceil 17/6 \rceil = 3$.

For positive integers $u, v$, $\lceil u/v \rceil \leq (u + v - 1)/v$ and $\lfloor u/v \rfloor \geq (u - v + 1)/v$. Write $u = qv + r$, $0 \leq r < v$, and check $r = 0$ and $r > 0$ separately. The implementation `(u+v-1)/v` still needs an overflow check for the addition.



## Peak space counts what is alive simultaneously

Auxiliary space excludes the input but normally includes temporary arrays and the call stack. If output space is also excluded, state that convention.

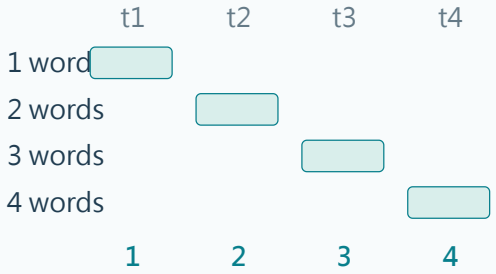
| Situation | Cumulative allocation | Peak live storage |
|---|---|---|
| Allocate and free $n$ single-word blocks in turn | $\Theta(n)$ | $\Theta(1)$ |
| Retain all $n$ single-word blocks | $\Theta(n)$ | $\Theta(n)$ |
| $K$ calls; peak of $d$ constant-size live frames | $\Theta(K)$ | $\Theta(d)$ |

Insertion sort keeps a constant number of indices and temporary records. Its auxiliary space is $\Theta(1)$ when each record occupies a constant number of words. For large objects, clarify whether the algorithm moves objects or references.

## Draw allocation lifetimes to find the peak

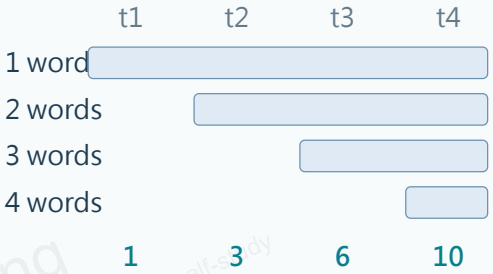
Allocate 1, 2, 3, 4 words; horizontal positions are processing intervals

Free after processing




Peak 4; allocated total 10

Retain all blocks




Peak 10; allocated total 10

Allocate blocks of 1, 2, 3, and 4 words. Immediate release gives peak 4; retaining all gives peak 10. Both cumulatively allocate ten words. Count these live blocks only, excluding allocator fragmentation, caches, and operating-system resident memory.



## Constants and crossover points still matter

In the same cost unit, let $T_A(n) = 8n^2$ and $T_B(n) = 64n \lg n$, $n \geq 2$. A is faster exactly when $n < 8 \lg n$.

| $n$ | $8n^2$ | $64n \lg n$ | Comparison |
|---|---|---|---|
| 16 | 2048 | 4096 | A faster |
| 32 | 8192 | 10240 | A faster |
| 64 | 32768 | 24576 | B faster |

For real $n \geq 4$, $n / \lg n$ is strictly increasing. Checking 43 and 44 on opposite sides, together with 2 and 3, locates the integer crossover between 43 and 44. If these formulas came from measurements, the result would be conditional on that model and environment.

## Hardware speed has units too

If A executes $10^9$ model operations per second and B executes $10^8$, compare

$$t_A(n) = T_A(n)/10^9, \qquad t_B(n) = T_B(n)/10^8.$$

Do not compare an operation count directly with a time in seconds. Hardware changes constants; an algorithmic change may change the growth order. Both comparisons need an explicit model.

Also check space: an $n^2$-entry table may exceed memory even when its running time is acceptable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Put the analysis together

For a new program, build the argument in this order rather than guessing a Big-O first:

1. Fix legal inputs, input size, and the comparison/arithmetic model.
2. Trace a small input completely to identify what each iteration does.
3. State invariants, boundary cases, and a decreasing quantity.
4. Write a general sum or recurrence for the cost.
5. Establish upper and lower bounds; identify the best/worst/average function.
6. Find peak space and check overflow, aliasing, and distribution assumptions.

For insertion sort, $T(S) = \Theta(n + I(S))$ explains how input structure affects work, beyond the looser statement of worst-case $O(n^2)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Extension: why does a polynomial's leading term dominate?

Let $p(n) = a_d n^d + \sum_{i<d} a_i n^i$, with fixed $d \geq 1$, $a_d > 0$, and $M = \sum_{i<d} |a_i|$.

For $n \geq 1$, the lower terms have total absolute value at most $Mn^{d-1}$. If also $n \geq 2M/a_d$, then

$$\frac{a_d}{2} n^d \leq p(n) \leq \frac{3a_d}{2} n^d.$$

Thus $p(n) = \Theta(n^d)$ even with negative lower coefficients. A negative leading coefficient violates our eventually nonnegative cost convention. If $d = 0, a_0 > 0$, the result is $\Theta(1)$ instead.

### Extension: an asymptotic equality need not be reversible

The expression $n^2 + n = n^2 + O(n)$ means that a function in $O(n)$ makes the equality true; here it is the function $n$.

It implies $n^2 + n = \Theta(n^2)$, but $f = \Theta(n^2)$ does not imply $f = n^2 + O(n)$. Take $f = 2n^2$: the difference $n^2$ is not in $O(n)$.

To calculate with anonymous functions, name them first. For $u, v \in O(n)$, the triangle inequality gives $|u + v| \leq (c_u + c_v)n$ eventually. Subtraction requires a convention for signed error bounds; do not silently ignore the nonnegative-function convention used in a definition.



## Extension: e, ln, and approximation error

For $x > -1$, $\ln(1 + x) \leq x$. Therefore $(1 + 1/n)^n \leq e$ for positive integers $n$.

For $x \geq 0$, a more precise pair of bounds is

$$x - \frac{x^2}{2} \leq \ln(1 + x) \leq x.$$

For the lower bound, set $h(x) = \ln(1 + x) - x + x^2/2$. Then $h(0) = 0$ and $h'(x) = x^2/(1 + x) \geq 0$. Substituting $x = 1/n$ gives

$$e^{1-1/(2n)} \leq (1 + 1/n)^n \leq e.$$

This bounds finite-$n$ error, not just the limit. Approximations require a domain and an error bound; higher-order terms cannot always be discarded.

## Extension: an exponential remainder with explicit bounds

Taylor's theorem gives $e^x = 1 + x + (e^\xi/2)x^2$, with $\xi$ between 0 and $x$. For $|x| \le 1$,

$$\frac{e^{-1}}{2}x^2 \le e^x - 1 - x \le \frac{e}{2}x^2.$$

Thus $e^x = 1 + x + \Theta(x^2)$ as $x \to 0$, rather than as $x \to \infty$. Convexity gives $e^x \ge 1 + x$ for every real $x$.

The standard power series $e^x = \sum_{k=0}^{\infty} x^k/k!$ converges for every real $x$. Using only a few terms requires a remainder estimate such as the one above. This chapter uses the basic calculus theorem, rather than deriving Taylor's theorem itself.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Extension: logarithmic bounds also work for negative x

For $x > -1$, the general pair is

$$\frac{x}{1+x} \le \ln(1+x) \le x.$$

For the upper bound, $h = x - \ln(1+x)$ has derivative $x/(1+x)$ and its minimum at 0 is zero. For the lower bound, $q = \ln(1+x) - x/(1+x)$ has derivative $x/(1+x)^2$ and the same minimum. Both arguments cover negative and positive $x$.

For fixed real $z$, substitute $x = z/n$ for sufficiently large $n$: $z/(1+z/n) \le n\ln(1+z/n) \le z$. Both sides tend to $z$, proving $(1+z/n)^n \to e^z$.

For $|x| < 1$, the standard series is $\ln(1+x) = \sum_{k=1}^{\infty}(-1)^{k+1}x^k/k$; its stated domain matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Extension: how much precision does Stirling supply?

For positive integers $n$, Stirling's formula is

$$n! = \sqrt{2\pi n}(n/e)^n\big(1 + O(1/n)\big).$$

Stirling's theorem controls the relative error of the leading approximation. Taking natural logarithms gives

$$\ln(n!) = n\ln n - n + \tfrac{1}{2}\ln(2\pi n) + O(1/n).$$

The relative error $O(1/n)$ becomes a logarithmic error $O(1/n)$ since $\ln(1+u) = O(u)$ for sufficiently small $u$. Conversely, a difference $O(n)$ between logarithms can become an exponential factor, not a constant relative error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Extension: turn Stirling's error into a numerical interval

A sharper form of Stirling's theorem is

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n} \quad (n \geq 1).$$

For $n = 6$, the leading approximation $L = \sqrt{12\pi}(6/e)^6 \approx 710.078$ is below $6! = 720$. The bounds give

$$Le^{1/73} < 720 < Le^{1/72}.$$

The correction is bounded, not an arbitrary rounding choice. Since $\alpha_n = \Theta(1/n)$ and $e^u - 1 = \Theta(u)$ near zero, $e^{\alpha_n} = 1 + \Theta(1/n)$. Coarse product bounds establish the order of $\lg(n!)$; this theorem additionally controls the leading approximation's relative error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Extension: iterated logs need a stopping rule

Set $\lg^{(0)} n = n$ and $\lg^{(k+1)} n = \lg(\lg^{(k)} n)$, taking another log only when the previous value exceeds 1. Define

$$\lg^* n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}.$$

| Input | Successive values | $\lg^* n$ |
|---|---|---|
| 1 | $1$ | 0 |
| 2 | $2 \to 1$ | 1 |
| 16 | $16 \to 4 \to 2 \to 1$ | 3 |
| 65536 | $65536 \to 16 \to 4 \to 2 \to 1$ | 4 |

$\lg^k n$ raises a logarithm to a power; $\lg^{(k)} n$ repeatedly applies a function. The notation and growth behavior differ.



## Extension: verify the Fibonacci closed form

Define $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Set $\varphi = (1 + \sqrt{5})/2$ and $\psi = (1 - \sqrt{5})/2$; both satisfy $r^2 = r + 1$.

Therefore $G_n = (\varphi^n - \psi^n)/\sqrt{5}$ satisfies the same recurrence, with $G_0 = 0, G_1 = 1$. Induction gives $G_n = F_n$ for every nonnegative integer.

Since $|\psi| < 1 < \varphi$, $F_n = \Theta(\varphi^n)$. But the **size of the answer** is not the time needed to compute it. Naive recursion, a table, and fast doubling require separate analyses in the recurrence chapter.

### References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

# Recursion, Recurrences, and Divide-and-Conquer

A recursive call has a contract. We must explain what it returns, why its subcalls finish, and how their work combines with the work of the current call.

Read in this order: **one recursive path → merge sort → tools for proving recurrence bounds → maximum subarrays and matrix multiplication → unequal-split extensions**.

Prerequisites: Chapter 1's invariants, induction, asymptotic notation, and sums. Matrix examples use elementary matrix multiplication; the extension uses integration.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 2 and 4; supplementary teaching reference: MIT 6.046J (2005), Lectures 2–3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# A subcall contract is the interface for induction

Let $P(n)$ mean: for every legal input of size $n$, the procedure terminates and returns a result satisfying its specification.

1. **Base cases:** check the smallest inputs, including whether empty input is legal.
2. **Shrinking:** every subproblem is legal and strictly smaller than $n$.
3. **Combination:** assuming the smaller calls are correct, prove the current result correct.

This is strong induction. Assuming the current call is correct would be circular. Shrinking proves termination, not the result specification by itself.

Analyze cost separately: $T(n) = \sum_i T(n_i) + f(n)$. Charge only the current call's comparisons, partitioning, copies, and combination to $f$; **do not charge a subtree twice**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Binary search continues down one branch

Preconditions: $A[1..N]$ is nondecreasing over totally ordered keys. Return **any** matching index, or NIL if none exists. Call with $(1, N)$; $N = 0$ returns immediately.

```
BINARY-SEARCH(A, low, high, x)
    if low > high: return NIL
    mid = low + floor((high-low)/2)
    if A[mid] == x: return mid
    if A[mid] < x:
        return BINARY-SEARCH(A, mid+1, high, x)
    return BINARY-SEARCH(A, low, mid-1, x)
```

Intervals include both endpoints. A nonempty call has $1 \le low \le high \le N$; an empty interval reads no element. This version does not promise the first duplicate.



## Why may an entire interval be discarded?

If $A[mid] < x$, then $A[i] \le A[mid] < x$ for every $i \le mid$. Any match must lie on the right. If $A[mid] > x$, keep only the left. Equality already satisfies the contract.

For current length $n = high - low + 1 \ge 1$, a recursive interval has length at most $\lfloor n/2 \rfloor < n$. The empty interval is the failure base case. Strong induction on length proves termination and correctness.

With constant-cost comparisons and index-only arguments, worst-case cost satisfies

$$W(0) = \Theta(1), \qquad W(n) = W(\lfloor n/2 \rfloor) + \Theta(1).$$

The longest path has $\Theta(\lg(n+1))$ calls; a match at the root costs $\Theta(1)$. Recursive execution has peak stack space $\Theta(\lg(n+1))$.

## Trace both success and failure on the same array

For $A = [3, 5, 7, 8, 9, 12, 15]$, searching for 9 checks $(low, mid, high) = (1, 4, 7), (5, 6, 7), (5, 5, 5)$ and returns 5.

Searching for 10 makes the same first three probes. Since $9 < 10$, it then calls $(6, 5)$ and returns NIL on the empty interval. Gray cells are excluded candidates; the array itself is unchanged.



## Copying can change the cost of the same search logic

Keep original length $N$ distinct from current interval length $n$. On a worst-case search path:

| Argument representation | Local work per call | Total worst-case cost |
|---|---|---|
| Indices or a view | $\Theta(1)$ | $\Theta(\lg(N+1))$ |
| Copy the current interval | $\Theta(n)$ | $\Theta(N)$ |
| Copy the original whole array | $\Theta(N)$ | $\Theta(N \lg(N+1))$ |

The second row sums $N + N/2 + N/4 + \cdots = \Theta(N)$. In the third row, $N$ stays fixed throughout recursion; replacing it by $n$ changes the model.

**A recursion tree may have only one child per node.** It then becomes a path. Multiple recursive calls are not a prerequisite for the method.

## Repeated squaring: compute the subproblem once

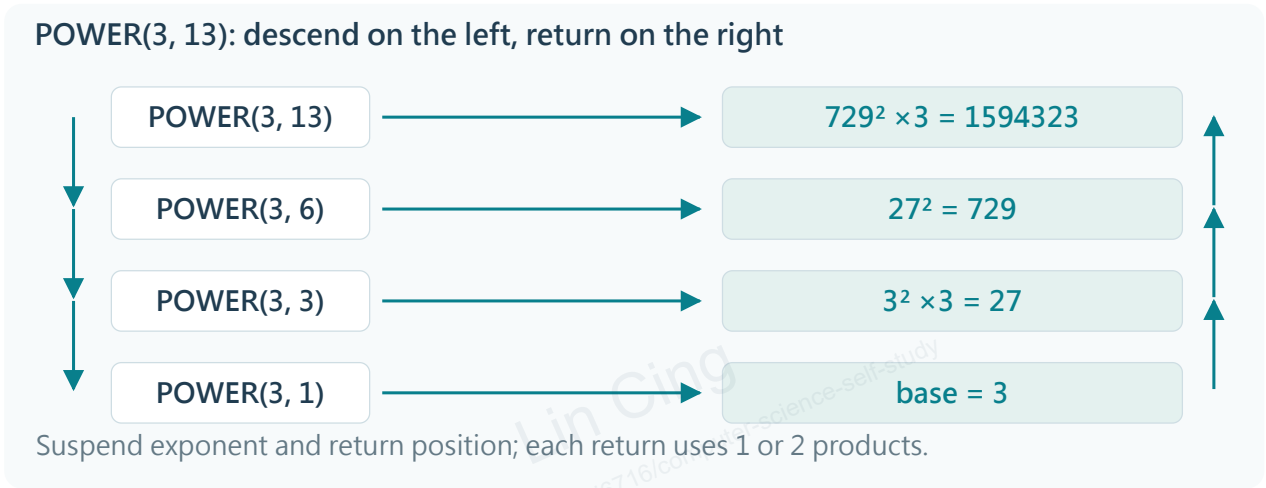
Input is a nonnegative integer $n$; output is $a^n$, with $a^0 = 1$. We first count multiplications; this does not make arbitrary-precision multiplication constant-time.

```
POWER(a, n)
    if n == 0: return 1
    if n == 1: return a
    y = POWER(a, floor(n/2))
    z = y*y
    if n is odd: z = z*a
    return z
```

If $n = 2k$, the subcall returns $y = a^k$, so $y^2 = a^n$. If $n = 2k + 1$, multiply once more by $a$. The two bases and $\lfloor n/2 \rfloor < n$ complete strong induction.



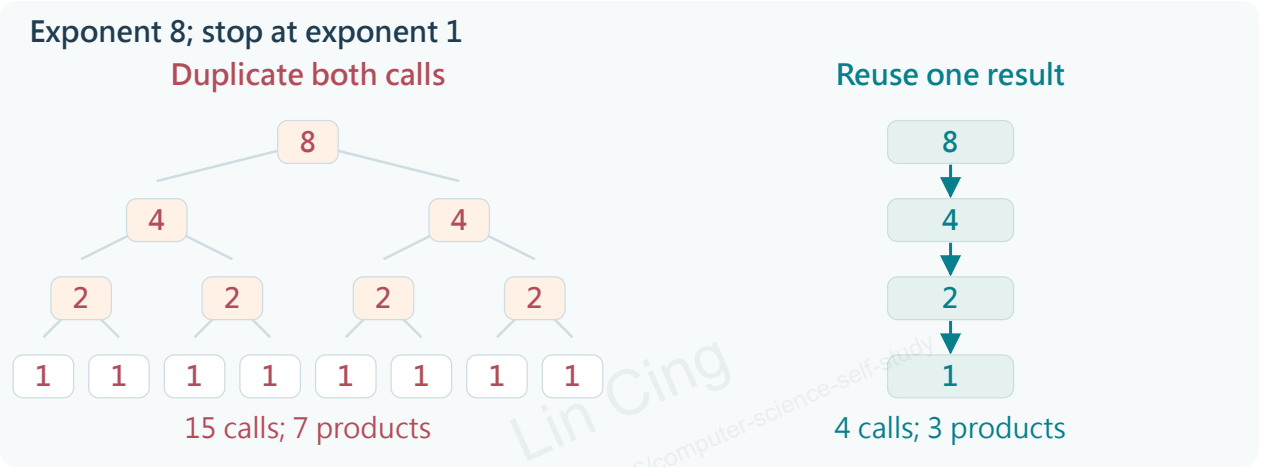
## Descend with suspended calls, then compute on return




For $3^{13}$, exponents descend as $13 \to 6 \to 3 \to 1$. Returned values are $3 \to 27 \to 729 \to 1594323$, using 0, 2, 1, and 2 multiplications: five in total.

For $n \geq 1$, there are $\lfloor \lg n \rfloor$ squarings and $\operatorname{popcount}(n) - 1$ extra multiplications by $a$. Total multiplications are their sum. At $n = 0$, the count is zero.

## Two occurrences in an identity need not mean two calls




Writing two separate POWER calls for $a^{n/2}a^{n/2}$ gives $D(n) = 2D(n/2) + \Theta(1) = \Theta(n)$ on powers of two, despite equal returned values.

Saving one returned value before squaring gives $R(n) = R(\lfloor n/2 \rfloor) + \Theta(1) = \Theta(\lg n)$ under unit-cost multiplication. Count **executed calls**, not occurrences left after algebraic simplification.

Fixed-width code must handle multiplication overflow. Modular exponentiation may reduce after each product, but the intermediate product must itself be computed safely.



## Fibonacci: repeated calls and the size of the answer

Define $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Naive recursion repeats work. Its call count obeys $C_0 = C_1 = 1$ and $C_n = C_{n-1} + C_{n-2} + 1$.

Setting $D_n = C_n + 1$ proves $C_n = 2F_{n+1} - 1$ from the recurrence and bases. Computing $F_5 = 5$ takes 15 calls. Time is $\Theta(\varphi^n)$ but stack depth is only $\Theta(n)$, where $\varphi = (1 + \sqrt{5})/2$.

Computing in increasing order while retaining the previous two values takes $n - 1$ additions for $n \geq 2$. This introduces reuse; full dynamic-programming design comes later.

**Separate arithmetic steps from bit complexity.** The answer has $\Theta(n)$ bits. Two retained integers need not occupy a constant number of machine words. Sequential ordinary integer additions take $\Theta(n^2)$ bit operations.

## Compute Fibonacci exactly through a matrix power

Let $Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$. For $n \geq 1$,

$$Q^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}.$$

The identity holds at $n = 1$. Right multiplication by $Q$ adds the two old columns to form the first new column and copies the first old column to the second. The Fibonacci recurrence proves the next case.

For example, $Q^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ and $Q^4 = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$, so $Q^5 = \begin{bmatrix} 8 & 5 \\ 5 & 3 \end{bmatrix}$ gives $F_5 = 5$. Return 0 directly at $n = 0$.

Repeated squaring needs $O(\lg(n+1))$ exact $2 \times 2$ matrix multiplications. This counts **arithmetic operations**. Writing the $\Theta(n)$-bit answer already rules out logarithmic bit time. Floating-point evaluation of the closed form is not an unconditional exact algorithm.



## Merge Sort and Divide-and-Conquer

Divide and conquer solves strictly smaller instances and combines their results. State boundaries, base cases, merge preconditions, and results. For $A[p..r]$, length is $m = r - p + 1$, with $p > r$ denoting an empty interval.

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

A legal nonempty call has $1 \leq p \leq r \leq n$. The full call is $(1, n)$, giving immediate return at $(1, 0)$ for empty input. Child lengths are $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, both smaller for $m \geq 2$. Computing the midpoint from a difference avoids the usual $p + r$ overflow, provided the index range itself is representable.

## MERGE without an extra key value

Assume $1 \le p \le q < r \le n$ and two stably sorted halves. L and R are **independent copies**, not views affected by overwriting A. Their lengths are $q - p + 1$ and $r - q$.

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key: A[k] = L[i]; i = i+1
        else: A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Choose L on equality. Every original left-half record precedes every right-half record, so stability is preserved. Length checks require no key-domain value named infinity.



## Proof of merge preservation, order, and termination

Before the main loop,

$$k - p = (i - 1) + (j - 1).$$

The output $A[p..k-1]$ is exactly the stable merge of consumed prefixes $L[1..i-1], R[1..j-1]$. It is sorted, and every emitted key is at most every unconsumed key. Remaining records still exist in independent buffers.

The empty output establishes initialization. When both inputs remain nonempty, their heads are their smallest remaining records. Choosing the smaller head, with left priority on ties, gives the next record of the stable order. Consuming one record and incrementing $k$ preserves lengths, data, order, and stability. Remaining record count decreases by one, proving termination.

At main-loop exit, at least one buffer is exhausted. The other is sorted with keys no smaller than the output tail, so appending it preserves the invariant. Finally $i = |L| + 1, j = |R| + 1, k = r + 1$, and all $m$ records occur once. Without copies, an output write could destroy unread input and invalidate the proof.

## Complete merge with duplicate keys

Let $L = [1a, 4b, 4c]$, $R = [2d, 4e, 6f]$, with L originally before R. Letters label records; compare only numeric keys.

| i,j before test | Comparison | Output prefix |
|---|---|---|
| 1,1 | $1 \leq 2$; take 1a | 1a |
| 2,1 | $4 > 2$; take 2d | 1a,2d |
| 2,2 | $4 \leq 4$; take left 4b | 1a,2d,4b |
| 3,2 | $4 \leq 4$; take left 4c | 1a,2d,4b,4c |
| 4,2 | L exhausted; no key comparison | append 4e,6f |

There are 4 key comparisons, 6 output writes, and 6 copied buffer records. Interleaving until the last two records can require $m - 1 = 5$ comparisons. $\Theta(m)$ work does not mean exactly m key comparisons.



## Complete merge with duplicate keys — diagram

**Take left on equality:4b and 4c must precede 4e**

| | L | | | R | | | Output prefix |
|---|---|---|---|---|---|---|---|
| start | 1a | 4b | 4c | 2d | 4e | 6f | ø |
| 1≤2 | 1a | 4b | 4c | 2d | 4e | 6f | 1a |
| 4>2 | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d |
| 4≤4 | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d, 4b |
| 4≤4 | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d, 4b, 4c |
| tail | 1a | 4b | 4c | 2d | 4e | 6f | 1a, 2d, 4b, 4c, 4e, 6f |

Tinted cells are consumed; unconsumed records remain in independent copies.

## Sentinel preconditions

A sentinel implementation can append an infinity record to each buffer and select the smaller head exactly m times, avoiding repeated exhaustion tests. The sentinel key must strictly exceed every legal key, and its record must not be legal input. The maximum legal integer is unsafe as a sentinel.

If legal keys include $+\infty$, take $L = [+\infty]$, $R = [+\infty, +\infty]$. Left priority can select the real left record, then its equal-valued sentinel, then read beyond L on the third output. This is not an out-of-domain marker.

Length checks work for any totally ordered key domain. A tagged sentinel is another option, but it must rank strictly above every real record and never be emitted as data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## From merge to complete sorting

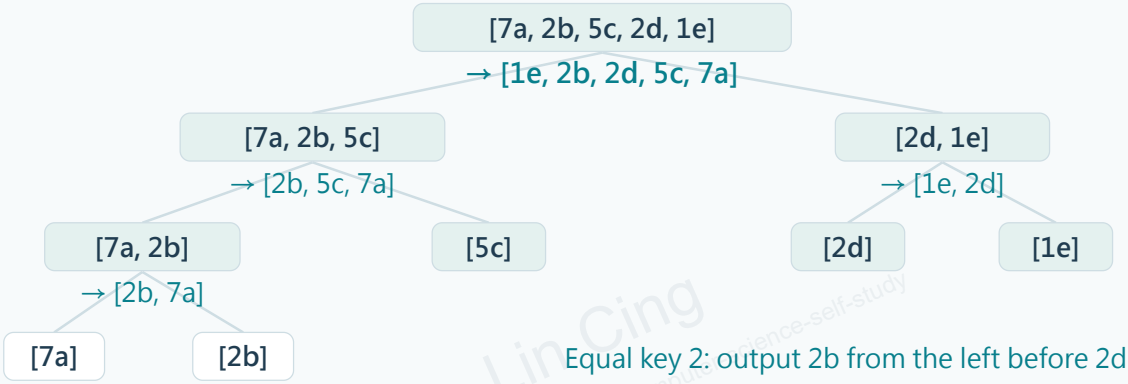
Use strong induction on interval length m. At $m \leq 1$, input is sorted and unchanged. For $m \geq 2$, both smaller calls stably sort and preserve their records by induction. MERGE's preconditions hold, and its proof gives a stable sorting of the whole interval, leaving outside storage unchanged. Child sizes decrease to 0 or 1, proving termination without a power-of-two assumption.

For [7a,2b,5c,2d,1e], splits have lengths 3 and 2, then 2/1 and 1/1. The figure shows every merged node. At the root tie, left 2b still precedes right 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## From merge to complete sorting — diagram

Divide downward; labels show each returned sorted result




Equal key 2: output 2b from the left before 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Include merging in the recurrence

Midpoint computation costs $\Theta(1)$. Copying and outputting m records costs $\Theta(m)$, giving

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

For $m = 2^h$, first solve the exact surrogate $U(1) = d$, $U(m) = 2U(m/2) + cm$, with positive c,d. Depth $\ell < h$ has $2^\ell$ nodes costing $cm/2^\ell$ each, hence cm per level. There are h internal levels and m leaves of cost d, so $U(m) = cm \lg m + dm$.

Actual internal costs lie between fixed positive multiples of size, and leaves have constant lower and upper costs. Apply the expansion to both bounds to prove Θ(m log m), rather than assuming one exact c describes every instruction and base case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Arbitrary lengths and a six-record cost table

At depth $\ell$, each surviving interval has size at most $\lceil m/2^\ell \rceil$, so height is at most $\lceil \lg m \rceil$. Intervals at one depth are disjoint with total size at most m, proving O(m log m).

At depths $\ell < \lfloor \lg m \rfloor$, each interval has size at least $\lfloor m/2^\ell \rfloor \geq 2$; none has become a leaf. Total size is exactly m, giving Ω(m) at each of $\lfloor \lg m \rfloor$ levels. For $m \geq 2$, this is at least $(\lg m)/2$, proving the lower bound for arbitrary m.

| Depth for m=6 | Sizes | Total internal size |
|---|---|---|
| 0 | 6 | 6 |
| 1 | 3,3 | 6 |
| 2 | 2,1,2,1 | 4 |
| 3 | 1,1,1,1; two earlier leaves | 0 |

Buffer copies and output writes each total 16, not 6 at every depth. There are six base cases.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Peak space differs from cumulative allocation

MERGE allocates L/R only after both recursive calls return and releases them after merging. A length-m merge holds m buffer records. Child buffers do not coexist with parent buffers, giving $\Theta(n)$ peak auxiliary record space plus $\Theta(\lg n)$ stack space, hence $\Theta(n)$ overall for $n \geq 2$ .

Cumulative allocation may be $\Theta(n \lg n)$, but it is not simultaneous demand. Retaining completed buffers changes the lifetime model and invalidates this peak bound. A reusable n-record scratch buffer is also possible with the same nonaliasing requirement.

Insertion sort needs only a saved record and constant index words. This merge sort guarantees $\Theta(n \lg n)$ time but is not in-place. Both are stable under their respective strict-shift and left-tie rules.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Substitution: prove one fixed bound

Guess a bound, then use strong induction with the same fixed constants at every size.

Let $T(1) = 1$ and $T(n) = 2T(\lfloor n/2 \rfloor) + n$ for $n \geq 2$. Try $T(n) \leq cn \lg n$. For $n \geq 4$, put $m = \lfloor n/2 \rfloor \geq 2$. Then

$$\begin{aligned} T(n) &\leq 2cm \lg m + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - (c-1)n \leq cn \lg n \end{aligned}$$

for fixed $c \geq 1$. At $n = 1$ the candidate right side is zero, so that input cannot establish this induction. Instead, compute $T(2) = 4, T(3) = 5$ from the recurrence. Fixed $c = 2$ covers both, and every child at $n \geq 4$ is at least 2, closing induction.

A recurrence base and an induction base may differ. The proof must still cover every reached small input.



## Lower bounds and stronger hypotheses

First restrict to powers of two, with $T(1) = 1$ and $T(n) = 2T(n/2) + n$. Assuming $T(n/2) \geq d(n/2) \lg(n/2)$ gives

$$T(n) \geq dn \lg n + (1-d)n \geq dn \lg n, \qquad 0 < d \leq 1.$$

The base holds. To extend this to every integer, first prove $T$ is nondecreasing: $T(2) > T(1)$; for $n \geq 2$, strong induction gives $T(n+1) - T(n) = 2[T(\lfloor (n+1)/2 \rfloor) - T(\lfloor n/2 \rfloor)] + 1 \geq 1$. Set $p = 2^{\lfloor \lg n \rfloor}$; then $p > n/2$ and $1 + \lg p > \lg n$. Since $T(p) = p(1 + \lg p)$, monotonicity yields $T(n) \geq T(p) > \frac{1}{2} n \lg n$.

Now consider $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$, with $T(1) = 1$. The guess $T(n) \leq cn$ yields only $cn + 1$. Strengthen it to

$$T(n) \leq cn - d.$$

Substitution yields $T(n) \leq cn - 2d + 1 \leq cn - d$ for $d \geq 1$. Choosing $c = 2, d = 1$ covers the base; induction in fact gives the exact answer $2n - 1$.

The lower-order deductions at children pay the new work at the parent. Saying that $cn + n$ is still $O(n)$ does not establish the same fixed bound $cn$ and cannot close induction.

## Variable changes and strict shrinking

Start on $n = 2^{2^k}$ with $T(n) = 2T(\sqrt{n}) + \lg n$ and $T(2) = \Theta(1)$. Set $m = \lg n$ and $S(m) = T(2^m)$. Then

$$S(m) = 2S(m/2) + m = \Theta(m \lg m).$$

Restoring the variable gives $T(n) = \Theta(\lg n \lg \lg n)$, not a bound with the new variable mistaken for the original input. The next argument treats an integer version with a floor.

A fixed shift also needs a shrinking check. In $T(n) = 2T(n/2 + 17) + n$, children shrink only for $n > 34$. Put $n = 34 + x$ and $U(x) = T(34 + x)$, obtaining

$$U(x) = 2U(x/2) + x + 34.$$

With a sufficiently large fixed base interval, level sums give $\Theta(x \lg x) = \Theta(n \lg n)$. This argument first uses real arguments; integer versions must also bound rounding. Giving only $T(1)$ would leave $n = 34$ calling itself, so the recurrence would not be well-defined.



## How to justify a floor inside a square root

Define $T(n) = 1$ for integers $1 \le n \le 3$, and for $n \ge 4$ let

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n.$$

Here $2 \le \lfloor \sqrt{n} \rfloor < n$, so the recurrence is well-defined and terminates.

Set $a_k = 2^{2^k}$ and $U_k = T(a_k)$. Since $\sqrt{a_k} = a_{k-1}$, these points have no rounding error. We have $U_0 = 1$ and $U_k = 2U_{k-1} + 2^k$.

Dividing by $2^k$ and summing yields $U_k = (k+1)2^k$. Thus at these special inputs, $T(n) = \lg n(1 + \lg \lg n)$.

This establishes a result only on the special sequence. Extending it to every integer requires the monotonicity and sandwich argument next.

## Prove monotonicity before sandwiching inputs

The bases satisfy $T(1) = T(2) = T(3) = 1 < T(4)$. Assume smaller-input values are nondecreasing. Since $\lfloor \sqrt{n+1} \rfloor \geq \lfloor \sqrt{n} \rfloor$, both child arguments are below $n$, and $\lg(n+1) > \lg n$, we get $T(n+1) \geq T(n)$. Strong induction proves monotonicity.

Choose $k$ with $a_k \leq n < a_{k+1}$, equivalently $2^k \leq \lg n < 2^{k+1}$. Monotonicity gives

$$(k+1)2^k \leq T(n) \leq (k+2)2^{k+1}.$$

Within this interval, $\lg n$ is within a factor two of $2^k$, and $1 + \lg\lg n$ is within fixed factors of $k+1$. Therefore

$$T(n) = \Theta\big(\lg n(1 + \lg\lg n)\big), \qquad n \geq 2.$$

For sufficiently large $n$, this is $\Theta(\lg n \lg\lg n)$. The sandwich uses proved monotonicity, not an assumption that the floor can be discarded.



## A recursion tree charges local work once

Each internal node stores only its current nonrecursive work $f(\text{size})$; its descendants account for recursive work. Storing the full $T(\text{size})$ at a node and also summing its descendants would double-count.

Take $T(n) = 3T(n/4) + n^2$, $T(1) = 1$, initially on $n = 4^L$. Depth $j$ has $3^j$ nodes, each costing $(n/4^j)^2$, so its total is $n^2(3/16)^j$. There are $3^L = n^{\log_4 3}$ leaves. Thus

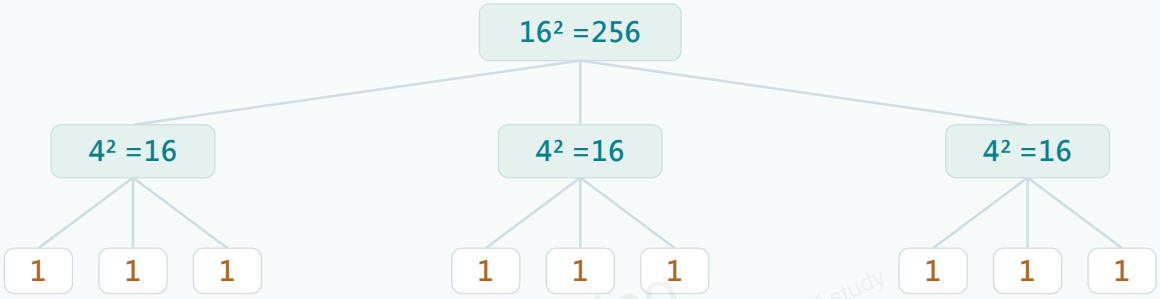
$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3}.$$

The geometric sum is at most $16/13$, and leaf work is at most $n^2$, proving $O(n^2)$. Root work supplies the matching $\Omega(n^2)$ lower bound.

At $n = 16$, level costs are 256,48,9, totaling 313. The final 9 already pays for nine base cases; do not add $f(1)$ again at each leaf.

## A recursion tree charges local work once — diagram

T(n)=3T(n/4)+n², T(1)=1; n=16




Level costs:256 →48 →9 (nine base calls)
Total 313; do not add f(1) again at leaves.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A contracting level sum still needs leaf costs

For integer $n \geq 8$, define

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + n,$$

with $T(n) = 1$ for $0 \leq n \leq 7$. Child sizes sum to at most $7n/8$. Internal work at depth $j$ is at most $n(7/8)^j$, totaling at most $8n$.

Let $I$ and $L$ count internal nodes and leaves. Every internal node has three children, so counting edges gives $3I = I + L - 1$ and $L = 2I + 1$. Each internal node costs at least 8, hence $8I \leq 8n$ and $L \leq 2n + 1$.

Thus $T(n) \leq 8n + 2n + 1 = O(n)$. The root supplies $T(n) \geq n$, proving $\Theta(n)$. **The geometric sum pays for internal work; a separate node count pays for leaves.**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Unequal splits: depth and mass per level

Let $a = \lfloor n/3 \rfloor$, $b = n - a$, and $T(n) = T(a) + T(b) + cn$ for fixed $c > 0$, with fixed positive bounds on small-input costs.

Along any path, the depth-$j$ size $q_j$ satisfies

$$\frac{n}{3^j} - \frac{3}{2} \le q_j \le n(2/3)^j + 3.$$

Induct from $q/3 - 1 \le q_{\text{next}} \le 2q/3 + 1$ and sum the geometric rounding errors. Absorb all work at sizes up to fixed $K \ge 6$ into base cases; this changes only constants. The upper bound gives $O(\lg n)$ height. Disjoint intervals at a depth have total length at most $n$ and cost at most $cn$. Positive integer leaf lengths sum to $n$, so there are at most $n$ leaves. Total cost is $O(n \lg n)$.

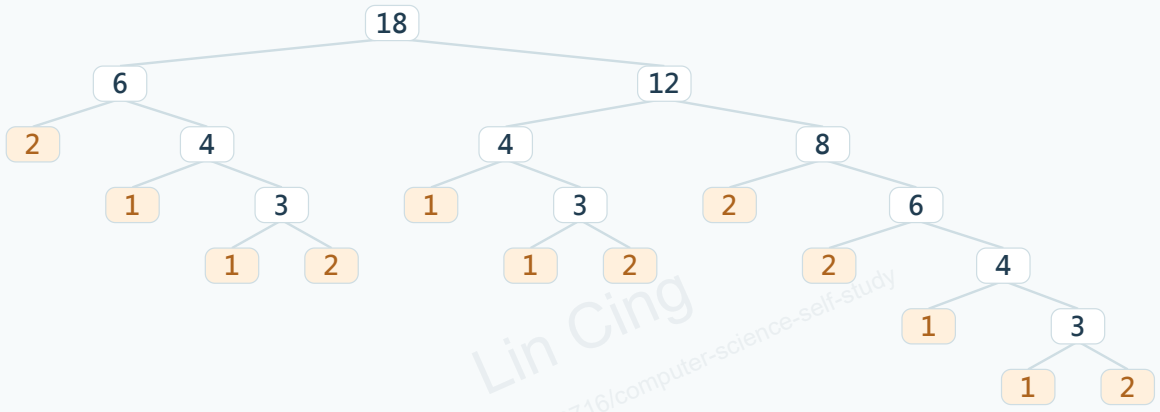
For $0 \le j \le \lfloor \log_3(n/(K+2)) \rfloor$, the lower bound keeps every $q_j > K$. No path has stopped: each complete level retains mass $n$ and costs $cn$. There are $\Omega(\lg n)$ such levels, proving the matching lower bound.

With only $O(n)$ local work, only the upper bound follows. Zero combination work can leave linear leaf cost alone.



## Unequal splits: the complete call tree

n=18; white nodes split, orange nodes stop at n≤2

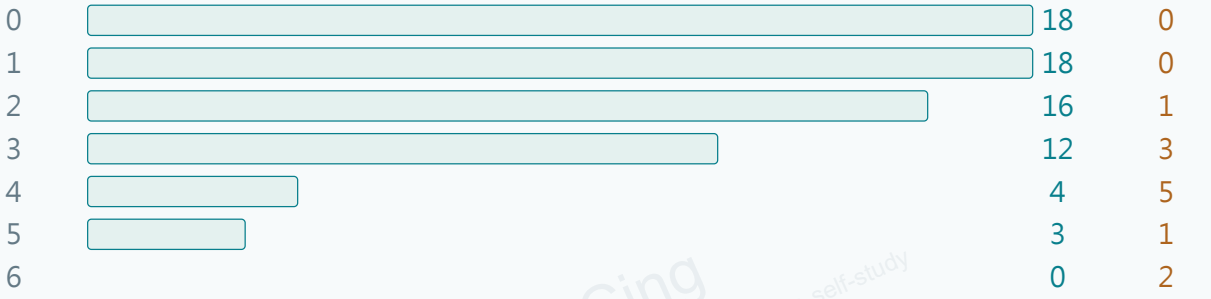



11 internal nodes, 12 leaves; leaf depths range from 2 to 6.

Each edge is one executed call. Orange leaves stop at different depths; the deepest path is 18 →12 →8 →6 →4 →3 →2.

## Unequal splits: distinguish complete and truncated levels

n=18; stop at n≤2; bars measure internal mass per level




Charge leaves separately; stopped nodes produce no descendants.

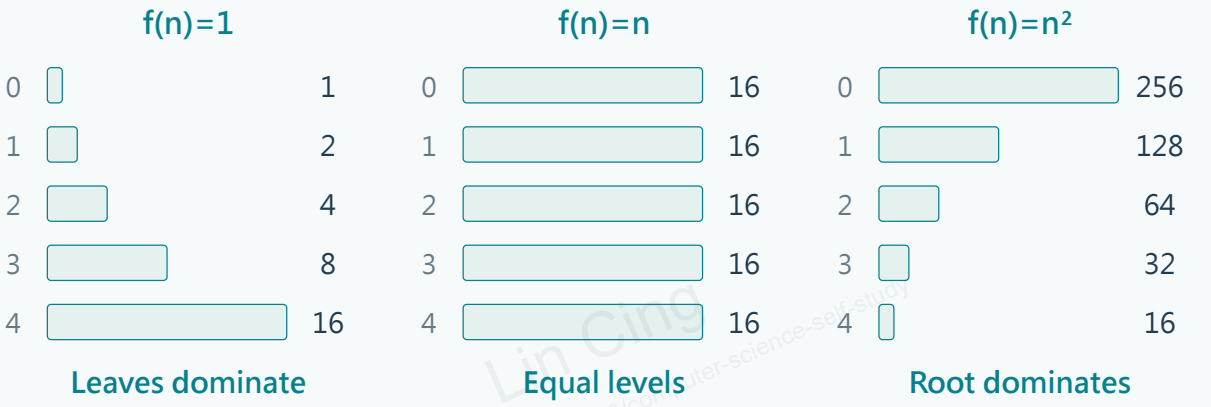
The figure stops at $n \leq 2$; larger nodes split into $\lfloor n/3 \rfloor$ and $n - \lfloor n/3 \rfloor$. Complete levels initially have mass 18. After leaves appear, **only internal nodes** pay combination work.

The shortest path limits the complete levels usable for a lower bound. The longest path controls the depth needed for an upper bound. Multiplying the deepest height by $n$ does not establish a lower bound for truncated levels.



## Read the level work behind the three cases — diagram

Independent scale per panel; labels are total cost per level



## Read the level work behind the three cases

All three use $n = 16, a = b = 2, T(1) = 1$. Nonrecursive terms are $1, n, n^2$. Costs by level, including leaves:

| $f(n)$ | Depths 0,1,2,3,4 | Total |
|---|---|---|
| $1$ | 1,2,4,8,16 | 31 |
| $n$ | 16,16,16,16,16 | 80 |
| $n^2$ | 256,128,64,32,16 | 496 |

These illustrate leaf, equal-level, and root domination. The theorem still requires fixed parameters, polynomial gaps, and case 3 regularity. A drawing alone does not verify those conditions.



## Master theorem: check the hypotheses

Let fixed $a \geq 1, b > 1$ and nonnegative local work $f$ define

$$T(n) = aT(n/b) + f(n), \qquad p = \log_b a.$$

Base costs on a constant-size interval have fixed positive upper and lower bounds. Start on exact powers; rounding conditions follow later. An actual call count $a$ is integral; a real $a$ can instead be a recurrence weight.

| Case | Required hypotheses | Result |
|---|---|---|
| 1 | Fixed $\epsilon > 0$; $f(n) = O(n^{p-\epsilon})$ | $\Theta(n^p)$ |
| 2 | $f(n) = \Theta(n^p)$ | $\Theta(n^p \lg n)$ |
| 3 | Fixed $\epsilon > 0$; $f(n) = \Omega(n^{p+\epsilon})$; eventually $af(n/b) \leq cf(n)$ for fixed $c < 1$ | $\Theta(f(n))$ |

Merely smaller or larger than $n^p$ is insufficient. Cases 1 and 3 require a fixed polynomial gap; case 3 also requires regularity.

## Complete case classification examples

Every row assumes fixed positive base costs. Compute $p = \log_b a$ rather than guessing it from a plot.

| $(a, b, f(n))$ | Concrete hypothesis check | Tight bound |
|---|---|---|
| $(9, 3, n)$ | $p = 2$; choose $\epsilon = 1$ | $\Theta(n^2)$ |
| $(1, 3/2, 1)$ | $p = 0$; $f(n) = \Theta(n^0)$ | $\Theta(\lg n)$ |
| $(3, 4, n \lg n)$ | $p \approx 0.7925$; choose $\epsilon = 0.1$; regularity ratio at most $3/4$ | $\Theta(n \lg n)$ |
| $(8, 2, n^2)$ | $p = 3$; choose $\epsilon = 1$ | $\Theta(n^3)$ |
| $(7, 2, n^2)$ | $p = \lg 7$; choose $\epsilon = \lg 7 - 2 > 0$ | $\Theta(n^{\lg 7})$ |

For the third row,

$$\frac{3f(n/4)}{f(n)} = \frac{3}{4}\frac{\lg n - 2}{\lg n} \le \frac{3}{4} \quad (n \ge 4).$$

By contrast, $2T(n/2) + n \lg n$ has no fixed polynomial gap. None of the classical three cases applies directly, but the level sums next still solve it.



## Sum logarithmic boundary cases

Let $n = b^L, p = \log_b a$, and $f(n) = \Theta(n^p (\lg n)^k)$ for fixed real $k$. Above a fixed base interval, renumber depth $j$ by distance $r = L - j$ from the leaf end. Level cost is $\Theta(n^p r^k)$.

Add $\Theta(n^p)$ leaf work to $n^p$ times the resulting power sum:

$$T(n) = \begin{cases} \Theta(n^p (\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg \lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

For $k > -1$, integral comparison gives $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$. At $k = -1$ it is harmonic; for $k < -1$ it converges to a positive constant. For negative $k$, apply the formula only above a fixed base interval larger than 1, avoiding negative powers of $\lg 1$.

Thus $2T(n/2) + n/\lg n$ gives $\Theta(n \lg \lg n)$; replacing its local term by $n/(\lg n)^2$ gives $\Theta(n)$. These follow from level sums, not classical case 1.

## A counterexample without regularity

Restrict to $n = 2^L$, with $T(1) = 1$ and

$$f(n) = \begin{cases} n^2 & L \text{ even}, \\ n^3 & L \text{ odd}, \end{cases} \qquad T(n) = 2T(n/2) + f(n).$$

Always $f(n) \geq n^2$, giving a fixed polynomial gap above $n^p = n$. For even $L \geq 2$, however, the child exponent is odd:

$$T(n) \geq 2f(n/2) = n^3/4, \qquad \frac{T(n)}{f(n)} \geq n/4 \to \infty.$$

Hence $T(n) \notin \Theta(f(n))$. The regularity ratio itself equals $n/4$, so no common $c < 1$ works eventually.

This illustrates omission of a hypothesis, not failure of the theorem. For an oscillating function, a bounded change in its argument need not produce comparable values.



## Why the master theorem holds

Start with $n = b^L, T(1) = t > 0$. Expanding $L$ times gives

$$T(n) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

Depth $j$ has total weight $a^j$; when $a$ counts actual calls, it is also the node count. The leaf term $tn^p$ gives a positive lower bound in every case.

In case 1, eventually $f(x) \leq Cx^{p-\epsilon}$. Large-size levels contribute at most

$$Cn^{p-\epsilon} \sum_{j=0}^{L-1} (b^\epsilon)^j = O(n^{p-\epsilon} b^{\epsilon L}) = O(n^p).$$

The ratio $b^\epsilon > 1$ makes this sum increase toward the leaves. Extending the bound across finitely many small arguments adds at most $O(n^p)$: their total weight has that order, and their fixed function values are absorbed into constants.

Adding leaves gives $O(n^p)$; their positive cost supplies $\Omega(n^p)$, proving $\Theta(n^p)$.

## Equal-level and root-dominated proofs

**Case 2.** Since $a = b^p$, every sufficiently large-size level costs

$$a^j \Theta((n/b^j)^p) = \Theta(n^p).$$

There are $L - O(1) = \Theta(\lg n)$ such levels. Summing their upper and lower bounds gives $\Theta(n^p \lg n)$ and covers leaf work.

**Case 3.** Repeated regularity gives, while arguments remain sufficiently large,

$$a^j f(n/b^j) \le c^j f(n), \qquad 0 < c < 1.$$

Their sum is at most $f(n)/(1-c)$. Remaining fixed-size levels and leaves cost $O(n^p)$, which is $O(f(n))$ by polynomial separation. The root's local work $f(n)$ supplies the matching lower bound, proving $\Theta(f(n))$.

Small-input work is not constant across the entire tree. There can be $\Theta(n^p)$ small nodes; their total must be included.



## Rounding errors remain bounded

Along an actual call path, let $q_0 = n$ and let $q_{j+1}$ be either $\lfloor q_j/b \rfloor$ or $\lceil q_j/b \rceil$. Both satisfy $|q_{j+1} - q_j/b| \le 1$. For fixed $C \ge b/(b-1)$, induction gives

$$|q_j - n/b^j| \le C,$$

because the next error is at most $C/b + 1 \le C$. It does not accumulate linearly with depth.

Choose a fixed base interval $q \le n_0$, with $n_0 > C + 2$ and large enough for strict shrinking. The two error bounds place every leaf at depth $\log_b n + O(1)$. Where $n/b^j \ge 2C$, actual sizes and unrounded sizes are within fixed factors.

With an integral number $a$ of actual children, leaf depths differ by only a constant, giving $\Theta(a^{\log_b n}) = \Theta(n^p)$ leaves and $O(n^p)$ total fixed-size work. For $a = 1$, the tree is one path with constant bottom work.

## Conditions for all integer sizes

Here $a$ is an integral number of actual subcalls. Real weights on exact powers were handled by the earlier expansion.

**Cases 1 and 2:** Actual sizes $q_j$ and $n/b^j$ are within fixed factors on large-size levels. Even for a negative fixed exponent,

$$q_j^{p-\epsilon} = \Theta((n/b^j)^{p-\epsilon}).$$

Case 1 therefore retains its polynomial bound and geometric sum. In case 2, each complete large-size level still costs $\Theta(n^p)$, with $\Theta(\lg n)$ levels. Charge incomplete fixed-size levels separately as $O(n^p)$. The logarithmic extension also survives: away from the fixed base interval, $\log q_j$ is comparable to distance from the bottom.

**Case 3:** Apply regularity to actual rounded arguments. A sufficient condition is

$$a \max\{f(\lfloor x/b \rfloor), f(\lceil x/b \rceil)\} \le cf(x), \qquad c < 1.$$

Then level work contracts by $c$, and polynomial separation absorbs bottom work. Bounded argument error alone does not justify dropping this condition for arbitrary $f$.



## Regularity can imply a polynomial gap

On exact powers, suppose $af(x/b) \le cf(x)$ for fixed $0 < c < 1$, and $f(x_0) > 0$ at a fixed bottom point. Reversing the inequality through $k$ levels gives

$$f(b^k x_0) \ge (a/c)^k f(x_0).$$

Since $\log_b(a/c) = p + \log_b(1/c)$, set $\epsilon = \log_b(1/c) > 0$. Along this sequence, $f(n) = \Omega(n^{p+\epsilon})$.

This explains why some versions omit a separate polynomial-gap hypothesis under suitable positive bottom conditions. We retain the complete three-case assumptions for direct checking.

The argument covers $n = b^k x_0$. Extending it to every real or integer input requires control of the final base interval and a positive lower bound for $f$ there. A proof on one sequence alone does not cover all inputs.

## Maximum nonempty contiguous subarray

Given $n \geq 1$ finite real values $A[1..n]$, find a nonempty contiguous interval $[i, j]$ maximizing

$$\sum_{k=i}^{j} A[k],$$

and return $(i, j, \text{sum})$. Any maximizing interval is acceptable. We count exact additions and comparisons; floating-point error, NaN, and overflow are outside this model.

For prices $P_0, \ldots, P_n$, let $A_i = P_i - P_{i-1}$. Then $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$: interval $i..j$ means buying on day $i - 1$ and selling on day $j$.

Allowing no transaction adds an empty interval with value 0 and changes all-negative cases. Enumerating each start and extending its end with a running sum takes $\Theta(n^2)$ time and $O(1)$ auxiliary space. Re-summing every interval instead takes $\Theta(n^3)$.



## Crossing means suffix plus prefix

For $low < high$, set $mid = low + \lfloor (high - low)/2 \rfloor$. Every nonempty interval belongs to exactly one class:

1. Entirely in $low..mid$.
2. Entirely in $mid + 1..high$.
3. Crossing the boundary, with $i \leq mid < j$.

A crossing interval contains both $mid$ and $mid + 1$. It splits uniquely into a left suffix $i..mid$ and a right prefix $mid + 1..j$.

The two endpoints can be chosen independently. Thus its best sum is **the best nonempty left suffix plus the best nonempty right prefix**. Scan both sides separately rather than enumerating endpoint pairs. The best subarray within each half need not touch the boundary, so adding those two answers is invalid.

## Crossing means suffix plus prefix — diagram

mid=5: a crossing interval includes positions 5 and 6

Left sums:−1, 3, 0, 1,−1 → best 3, start 4

Right sums:2, 3,−2, 2 → best 3, end 7

**Combine A[4..7]=[4,−1, 2, 1], sum 6.**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Complete crossing scan

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

After each left-loop update, `total` is the sum of $A[i..mid]$, and `left` is the maximum among all left suffixes examined so far. The initial $-\infty$ is a comparison marker, not input data; the first finite candidate updates it and sets an endpoint.

Each iteration adds exactly one candidate and takes the maximum, preserving the invariant. All left suffixes are covered at termination; the right prefixes are analogous. Adding the two maxima gives the best crossing sum. Total scan iterations are

$$(mid - low + 1) + (high - mid) = high - low + 1.$$

## Recursive maximum subarray and proof

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = low + floor((high-low)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

Use strong induction on interval length. A singleton is its only answer. For a longer interval, both children are nonempty and smaller. Their contracts solve the first two classes; CROSS solves the third. The classes are disjoint and exhaustive, so taking their maximum is correct. Strict shrinking also guarantees termination.

Operation cost is

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n).$$

Sequential subcalls with index arguments retain only constant many indices and sums per frame, giving $O(\lg n)$ peak stack space. Copying subarrays changes allocation and lifetime costs and must be analyzed separately.



## Nine-element crossing trace

For $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$, use $low = 1, high = 9, mid = 5$.

| Left index $i$ | Sum of $A[i..5]$ | Best sum and start so far |
|---|---|---|
| 5 | $-1$ | $-1$,5 |
| 4 | 3 | 3,4 |
| 3 | 0 | 3,4 |
| 2 | 1 | 3,4 |
| 1 | $-1$ | 3,4 |

Right-prefix sums at $j = 6, 7, 8, 9$ are $2, 3, -2, 2$, with best interval $6..7$ of sum 3. The crossing answer is $4..7$ of sum 6. Left-only and right-only maxima are $4..4$ and $9..9$, both of sum 4. Return $(4, 7, 6)$.

For all-negative input $[-8, -2, -5]$, the nonempty contract returns $(2, 2, -2)$, not 0.

## A linear algorithm's state and invariant

Let $E_i$ be the best nonempty sum ending exactly at $i$, and $B_i$ the best nonempty sum within $1..i$:

$$E_1 = B_1 = A_1, \qquad E_i = \max(A_i, E_{i-1} + A_i), \quad B_i = \max(B_{i-1}, E_i).$$

An interval ending at $i$ either consists only of $A_i$ or extends one ending at $i-1$. In the latter case, the best previous ending sum suffices. This classification and the bases prove both state meanings by induction.

Choosing $A_i$ resets the start to $i$; extending preserves the old start. Save that start and endpoint $i$ when the global best improves. Constant work per iteration gives $\Theta(n)$ time and $O(1)$ auxiliary space.

For the nine-value example, $E_i = [-2, 1, -2, 4, 3, 5, 6, 1, 5]$ and $B_i = [-2, 1, 1, 4, 4, 5, 6, 6, 6]$. Divide-and-conquer is not the fastest solution here; it illustrates a different classification and combination proof.



## From scalar products to block multiplication

Standard multiplication computes $C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$. After step $k$, the accumulator contains the first $k$ products. The $n^2$ outputs each take $n$ multiply-add steps: $\Theta(n^3)$ operations for this algorithm, not a universal lower bound.

For even $n$, use $n/2$-order blocks:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}.$$

Their product satisfies

$$C_{11} = ae + bg, \quad C_{12} = af + bh, \quad C_{21} = ce + dg, \quad C_{22} = cf + dh.$$

Preserve multiplication order; matrix blocks generally do not commute.

## Full cost of eight recursive products

Pad to a power-of-two order, then extract the original-size result. Each call reads its own view's order rather than reusing the original order.

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

An index view retains an origin, stride, and order, permitting constant-time splitting. Explicit block copies cost $\Theta(n^2)$, the same order as fixed many block additions. Hence $T(n) = 8T(n/2) + \Theta(n^2)$ with a $\Theta(1)$ base.

Eight recursive products give $8^j$ calls at depth $j$ and $8^{\lg n} = n^3$ leaves. The branch count changes the exponent; it cannot be hidden in one child's constant.



## All seven Strassen products

Compute each product recursively with STRASSEN. Additions and subtractions inside parentheses are elementwise block operations.

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Assemble the four output blocks:

$$\begin{aligned} C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\ C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6. \end{aligned}$$

For $n = 1$, multiply the two scalars. For $n > 1$, split, compute these seven products, and assemble the four blocks. Together with the base case, these steps specify the recursive procedure.

## Correctness of all four output blocks

Use distributivity and additive cancellation, preserving every product's factor order:

$$\begin{aligned}
C_{11} &= (ae + ah + de + dh) + (dg - de) \\
&\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\
C_{12} &= (af - ah) + (ah + bh) = af + bh, \\
C_{21} &= (ce + de) + (dg - de) = ce + dg, \\
C_{22} &= (ae + ah + de + dh) - (ce + de) \\
&\quad + (af - ah) + (ce + cf - ae - af) = cf + dh.
\end{aligned}$$

Scalar multiplication is a correct base. If the seven smaller products are correct by induction, these identities assemble all four blocks of $AB$, completing the proof.

The identities require addition, subtraction, and distributivity. They do not directly apply to a general semiring without subtraction.



## Numerical products and reconstruction

Take $A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 2 \\ -2 & 1 \end{bmatrix}$.

| Product | Substitution | Value |
|---|---|---|
| $M_1$ | $(2 + 4)(5 + 1)$ | 36 |
| $M_2$ | $(3 + 4)5$ | 35 |
| $M_3$ | $2(2 - 1)$ | 2 |
| $M_4$ | $4(-2 - 5)$ | $-28$ |
| $M_5$ | $(2 - 1)1$ | 1 |
| $M_6$ | $(3 - 2)(5 + 2)$ | 7 |
| $M_7$ | $(-1 - 4)(-2 + 1)$ | 5 |

The four blocks are $36 - 28 - 1 + 5 = 12$, $2 + 1 = 3$, $35 - 28 = 7$, and $36 - 35 + 2 + 7 = 10$. Direct multiplication verifies:

$$AB = \begin{bmatrix} 10 + 2 & 4 - 1 \\ 15 - 8 & 6 + 4 \end{bmatrix} = \begin{bmatrix} 12 & 3 \\ 7 & 10 \end{bmatrix}.$$

## Numerical products and reconstruction — diagram

Keep C11,C12,C21,C22 in their correct block positions

| A | | | B | | | C | |
|---|---|---|---|---|---|---|---|
| 2 | -1 | × | 5 | 2 | = | 12 | 3 |
| 3 | 4 | | -2 | 1 | | 7 | 10 |

C11: 36−28−1+5=12

C12: 2+1=3

C21: 35−28=7

C22: 36−35+2+7=10

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Time, padding, and memory

The seven Strassen subproducts need ten input-block additions/subtractions and eight more to assemble the outputs. View splitting is $O(1)$, giving

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \qquad \lg 7 \approx 2.80735.$$

Larger addition constants may make small matrices slower. A fixed cutoff to standard multiplication preserves the exponent. Algebraic correctness alone gives no floating-point error guarantee.

For arbitrary $n \geq 1$, pad to $m = 2^{\lceil \lg n \rceil}$, where $n \leq m < 2n$. The top-left $n \times n$ result is the original product, and $m^{\lg 7} < 7n^{\lg 7}$ preserves the order. Return an empty matrix directly at zero size.

Execute subcalls sequentially and release temporaries promptly. Retaining only fixed many current-level blocks gives peak space

$$S(n) \leq S(n/2) + O(n^2) = O(n^2),$$

plus $O(\lg n)$ frames. Simultaneously retaining every child workspace requires a different space recurrence.

## Extension: conditions for Akra–Bazzi

We cite the theorem in MIT's *Mathematics for Computer Science* (2010), § 10.4.3, using positive bounded base costs as sufficient conditions. **Applications below are not a full proof of this theorem.**

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x), \qquad x > x_0.$$

- Fixed finite $k$, fixed $a_i > 0$, and fixed $0 < b_i < 1$.
- Fixed $0 < d_1 \le T(x) \le d_2$ on $0 \le x \le x_0$.
- Nonnegative differentiable $g$, with $|g'(x)|$ polynomially bounded; $|h_i(x)| = O(x/\log^2 x)$.
- Choose $x_0$ so every child lies in $[0, \rho x]$ for a fixed $\rho < 1$, making recursion well-defined and terminating.

These are the conditions used here. Zero base costs and identically zero $g$ would give identically zero cost, invalidating a conclusion with a positive leaf term.



## Find the unique exponent before integrating

Let $p$ satisfy $\sum_i a_i b_i^p = 1$. The cited theorem gives

$$T(x) = \Theta\left(x^p \left[1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du\right]\right).$$

Since $0 < b_i < 1$, $F(p) = \sum_i a_i b_i^p$ is continuous and strictly decreasing, tending to infinity as $p \to -\infty$ and to zero as $p \to +\infty$. The real root is unique.

For $T(x) = T(x/3) + T(2x/3) + x$, with positive constant base costs, the equation $(1/3)^p + (2/3)^p = 1$ gives $p = 1$. Therefore

$$T(x) = \Theta\left(x \left[1 + \int_1^x \frac{du}{u}\right]\right) = \Theta(x \log x).$$

This agrees with the earlier complete-level and maximum-depth bounds. Keep the assumptions of both arguments explicit.

## Rounding and a changed ratio are different

Floor, ceiling, and fixed shifts have bounded additive error, satisfying $O(x/\log^2 x)$. Choose a sufficiently large base interval to ensure shrinking.

In $T(n) = T(n/2 + n/4) + T(n/2) + n$, however, the first child is $3n/4$. The term $n/4$ is not a small perturbation; two one-half ratios give the wrong equation.

The correct equation is $(3/4)^p + (1/2)^p = 1$, with unique root $p \approx 1.507126591639 > 1$. Since

$$\int_1^n u^{-p}\,du = \frac{1 - n^{1-p}}{p-1} = \Theta(1),$$

we obtain $T(n) = \Theta(n^p)$. The equation defines the exact exponent; the decimal is approximate.

If another source writes child sizes as $n/c_i$, its parameters correspond to $b_i = 1/c_i$ here. Convert the equation together with the notation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Further reading and theorem sources

- Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, 3rd Edition: Chapter 2 for merge sort; Chapter 4 for divide-and-conquer, substitution, recursion trees, and the master theorem with its proof.
- Demaine and Leiserson, MIT 6.046J (Fall 2005), Lectures 2–3: recurrences and their connection to binary search, powering, Fibonacci, and matrix multiplication.
- Lehman, Leighton, Meyer, *Mathematics for Computer Science* (2010), § 10.4.3, Theorem 10.4.1: official MIT notes. We use positive bounded base costs as sufficient conditions.

The chapter proves the three master-theorem cases. Akra–Bazzi is a cited extension; its applications verify hypotheses, solve the exponent equation, and evaluate the integral.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Sorting, Selection, and Randomized Analysis

Sorting determines the order of every record; selection asks for one rank. They can share partitioning and heaps without sharing the same time bound.

Read in this order: **comparison-model limits → heaps and sorting → probability tools → partitioning and quicksort → counting, radix, and bucket sorting → order statistics**. Further examples use collisions, loads, waiting times, and online selection to practice probabilistic modeling.

Prerequisites: specifications, invariants, and sums from Chapter 1; divide-and-conquer and recurrences from Chapter 2. Probability tools are developed before their use in expected-time analysis.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 5–9; supplementary reference: MIT 6.046J (2005), Lectures 4–6.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



# Specify the records and the sorting guarantee

A record contains a **key** and fields that move with it. Identity labels $a, b, c$ distinguish equal-key records; comparisons inspect only the key unless stated otherwise.

A correct sort produces nondecreasing keys and preserves the input multiset of records. **Stability** additionally preserves the original order of equal-key records.

For input $[2_a, 1_b, 2_c]$, output $[1_b, 2_a, 2_c]$ is stable. Output $[1_b, 2_c, 2_a]$ is sorted but unstable. Output $[1_b, 2_a, 2_a]$ loses $2_c$ and violates the sorting contract itself.

Use **1-based closed intervals** $A[p..r]$, with $p > r$ denoting an empty interval. Key-indexed counting tables may start at $0$. Selection rank $k$ is always 1-based and local to the current subarray.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Count comparisons, time, and space separately

Unless stated otherwise, comparing or moving a fixed-size record and accessing one index take constant time. Long strings, large integers, and external objects require explicit comparison, digit-access, and movement costs.

| Question | Conditions that must be fixed |
| --- | --- |
| Worst-case time $\max_A T(A)$ | Length and legal input set |
| Average-case time $\mathbb{E}_A[T(A)]$ | Joint input distribution |
| Randomized time $\mathbb{E}_R[T(A,R)]$ | Fixed input $A$; average over internal randomness $R$ |
| Auxiliary space | Whether output, buffers, and live call frames are counted |

An in-place partition does not remove quicksort's recursion stack. A small expected time does not exclude an expensive random execution.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## A comparison-tree leaf must identify an ordering

First restrict attention to $n$ distinct keys. A comparison sort learns order only through comparisons between keys, without using key values as array indices. Fix $n$ and represent all executions by a binary tree.

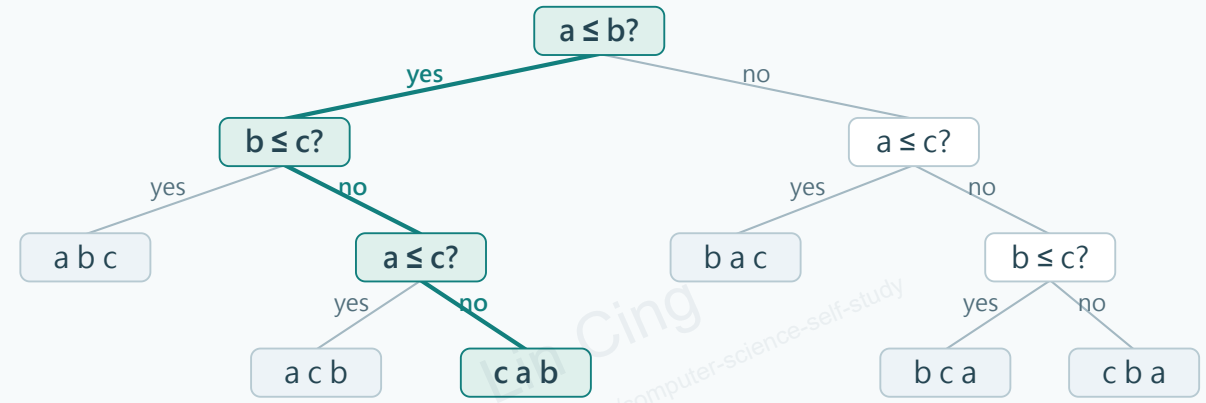
Each internal node identifies two original records. Its branches record $\leq$ or $>$. A root-to-leaf path records the observations, and its leaf must determine the complete identity order.

The $n!$ relative orders require distinct reachable leaves. If two orders reached the same leaf, the algorithm would observe the same outcomes and output the same identity permutation; at least one would be incorrect. Thus at least $n!$ leaves are reachable.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Follow the comparisons to the identity order

Six orders, six reachable leaves




a=4, b=7, c=1: yes → no → no → c a b. Leaves name record identities.

Leaf $c, a, b$ means $c < a < b$. For $a = 4, b = 7, c = 1$, follow the comparison outcomes to that leaf to check one complete execution.



## A worst-case lower bound requires some hard input

Here height $h$ counts edges on a longest root-to-leaf path; if the root is level 1, the number of levels is $h + 1$. A binary tree of height $h$ has at most $2^h$ leaves, so

$$n! \le 2^h, \qquad h \ge \lceil \lg(n!) \rceil.$$

For $n \ge 2$, at least $\lfloor n/2 \rfloor$ final factors of $n!$ are at least $n/2$, while every factor is at most $n$. Hence

$$\lfloor n/2 \rfloor \lg(n/2) \le \lg(n!) \le n \lg n.$$

These give $\lg(n!) = \Theta(n \lg n)$ for sufficiently large $n$; finitely many smaller sizes are covered separately.

Thus **every correct comparison sort has a worst-case input requiring $\Omega(n \lg n)$ comparisons**. It need not spend that much on every input: insertion sort uses $n - 1$ key comparisons on sorted data.

Allowing duplicates still includes the hard distinct-key subset. Counting and radix sorts inspect key representations and leave this model, introducing range and bit-length costs instead.

## The deepest leaf alone does not bound the average

Let the $N = n!$ relative orders be uniform, and select one representative leaf for each, at depths $d_1, \ldots, d_N$. Their root paths are prefix-free. Associate a depth-$d_i$ leaf with its binary interval of length $2^{-d_i}$ in $[0, 1)$. The intervals are disjoint, so

$$\sum_{i=1}^{N} 2^{-d_i} \leq 1.$$

The geometric mean is at most the arithmetic mean:

$$2^{-\frac{1}{N}\sum_i d_i} \leq \frac{1}{N}\sum_i 2^{-d_i} \leq \frac{1}{N}.$$

Taking logarithms yields $\frac{1}{N}\sum_i d_i \geq \lg N = \Omega(n \lg n)$. This bounds average depth under uniform inputs; the deepest path alone would not suffice.

For a randomized sort correct for every random choice $r$, this bound holds for each fixed $r$. Averaging over $r$ gives $\max_A E_r[C(A, r)] \geq \lg(n!)$, since a maximum is at least the uniform-input average.



## Three keys: distinguish the two averaging domains

Example: choose fairly between the earlier three-key tree L and its identity-renamed version R (input order $a, c, b$).

| Identity order | Tree L | Tree R | Fair-choice mean |
|---|---|---|---|
| $a, b, c$ | 2 | 3 | $5/2$ |
| $a, c, b$ | 3 | 2 | $5/2$ |
| $b, a, c$ | 2 | 3 | $5/2$ |
| $b, c, a$ | 3 | 3 | 3 |
| $c, a, b$ | 3 | 2 | $5/2$ |
| $c, b, a$ | 3 | 3 | 3 |

Uniform-input column means equal $16/6 = 8/3 \geq \lg 6$; the maximum fixed-input row mean is three. The averaging domains differ.

## A heap has both a shape rule and a key-order rule

A binary heap is a **complete binary tree**: all levels before the last are full, and the last fills from left to right. In a 1-based array,

$$parent(i) = \lfloor i/2 \rfloor, \quad left(i) = 2i, \quad right(i) = 2i + 1.$$

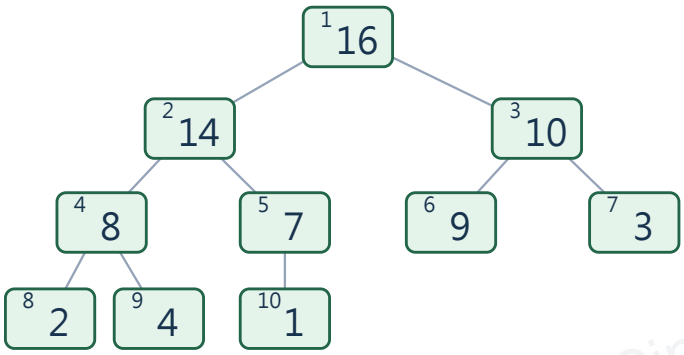
Only positions $1 \le i \le s$ belong to the current heap, where $s$ is its heap-size. Array capacity, array length, and $s$ are separate quantities.

A max-heap requires $A[parent(i)] \ge A[i]$ for every nonroot position $i$. Here $A[i]$ abbreviates its key; swaps move entire records. Following ancestors proves the root is a maximum. Levels and different subtrees need not be sorted.

Height counts **edges** on a longest downward path: a singleton has height 0, and a nonempty $n$-node heap has height $\lfloor \lg n \rfloor$. Counting levels instead gives $\lfloor \lg n \rfloor + 1 = \lceil \lg(n+1) \rceil$. Handle an empty heap separately, without evaluating $\lg 0$.



## One heap, two representations




Node: small index, large key

Fixed indices; moving keys

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 4 | 1 |

Every edge: parent key ≥ child key
One level need not be sorted

Node labels are array positions. Swaps move keys between positions; a parent index stays fixed.

## State the precondition for sifting down

Precondition: $1 \le i \le s$, both child subtrees are already max-heaps, and only edges from $i$ to its children may violate order. Missing children are not read; and uses short-circuit evaluation.

```
MAX-HEAPIFY(A,i,s):
    while true:
        largest = i
        if 2*i <= s and A[2*i] > A[largest]:
            largest = 2*i
        if 2*i+1 <= s and A[2*i+1] > A[largest]:
            largest = 2*i+1
        if largest == i: return
        swap(A[i],A[largest])
        i = largest
```

Each iteration finds a maximum among the current node and its existing children before deciding whether to swap. Comparing the two children alone cannot determine whether their parent needs repair.



## Why sifting down is correct and terminates

If the current node is a maximum, its child edges satisfy heap order. The two subtrees already satisfy it, so the entire subtree is repaired.

Otherwise promote the larger child root. It dominates its descendants, the other child root, and the old parent key. Both new edges from the parent are valid. Only the old parent's new position may violate order; the other subtree is unchanged.

Induction on remaining height proves correctness. Each swap changes $i$ to $2i$ or $2i+1$, moving down one level. At height $h$, there are at most $h$ descents, giving $O(h+1)$ time and $O(1)$ auxiliary space for this iterative version.

For $A = [16, 4, 10, 8, 7, 9, 3]$, $i = 2$, $s = 7$, choose child key $8$ and swap positions $2, 4$, producing $[16, 8, 10, 4, 7, 9, 3]$. Child indices $8, 9$ of position $4$ are out of range, so stop. Key $3$ at index $7$ belongs to the other subtree and is not compared in this descent.

## Build bottom-up so the child subtrees are ready

```
BUILD-MAX-HEAP(A):
    s = length(A)
    for i = floor(s/2) downto 1:
        MAX-HEAPIFY(A,i,s)
```

Invariant: at each iteration's start, every node whose index exceeds $i$ roots a valid max-heap.

Initially $i = \lfloor n/2 \rfloor$, and all larger indices are leaves. Child indices exceed $i$, so MAX-HEAPIFY's precondition holds. It repairs $i$ while preserving all descendant heaps. Decrementing $i$ restores the invariant for the next iteration.

At $i = 0$, root $1$ is valid and the entire heap is complete. For $n = 0, 1$, the loop is skipped and the empty or singleton heap is already valid.

Index $\lfloor n/2 \rfloor$ is the last possible internal node, **not the first leaf**. For $n = 10$, position $5$ still has left child $10$ and must be processed.



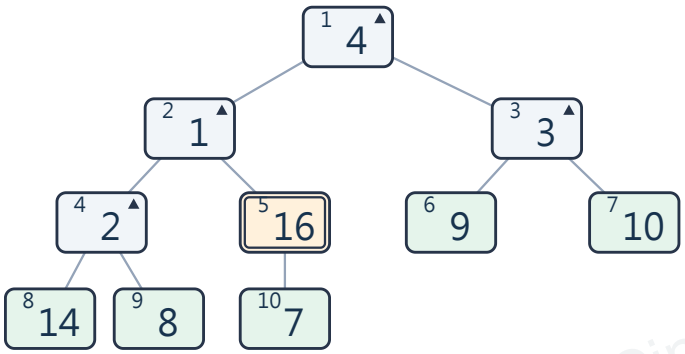
## Build backward from position five: complete states

Start with $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, active size $s = 10$. All indices are **one-based**. Each root is processed only after both child subtrees have been repaired.

| Root $i$ | Child choice and exchanged indices | Repaired $A[1..10]$ |
|---|---|---|
| $5$ | Only child key $7$; $16 \geq 7$, no swap | $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$ |
| $4$ | Choose $14$ over $8$; swap $4 \leftrightarrow 8$ | $[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$ |
| $3$ | Choose $10$ over $9$; swap $3 \leftrightarrow 7$ | $[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$ |
| $2$ | Choose $16$, then $7$; swap $2 \leftrightarrow 5$, $5 \leftrightarrow 10$ | $[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| $1$ | Choose $16, 14, 8$; swap $1 \leftrightarrow 2$, $2 \leftrightarrow 4$, $4 \leftrightarrow 9$ | $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ |

The following tree-and-array figures use these same records. Orange marks the root being processed; green marks certified heap roots. Position relationships stay fixed while swaps move keys and their associated records.

## Position five has only a left child




Node: small index, large key

Fixed indices; moving keys

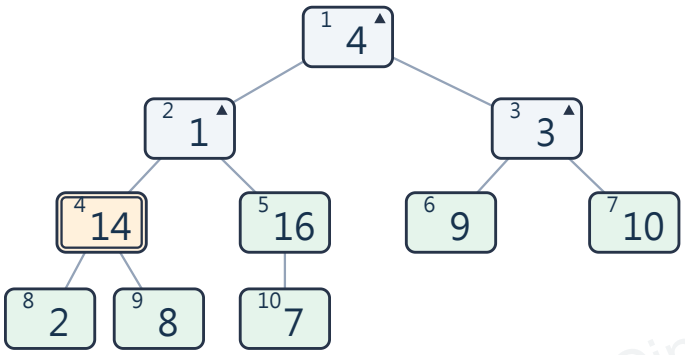
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 1 | 3 | 2 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 10 | 14 | 8 | 7 |

Double frame: root just repaired
No mark: legal; triangle: pending

Key $16$ at position $5$ already dominates key $7$ at left-child position $10$, so no swap occurs. Right-child index $11 > s$ must not be read. Positions $6..10$ are leaves; the repaired subtree at five extends the bottom-up invariant.



## Position four promotes the larger child, fourteen




Node: small index, large key

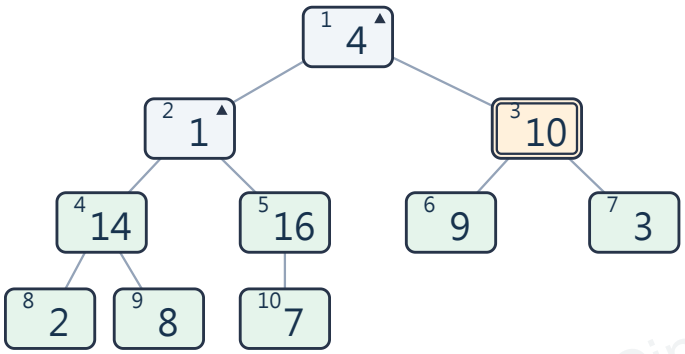
Fixed indices; moving keys

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 1 | 3 | 14 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 10 | 2 | 8 | 7 |

Double frame: root just repaired
No mark: legal; triangle: pending

Position four contains key two; children at eight and nine contain fourteen and eight. Choose fourteen, compare it with two, then swap positions four and eight. The old key reaches leaf eight and stops; key eight at position nine is unchanged.

## Position three chooses ten over nine




Node: small index, large key

Fixed indices; moving keys

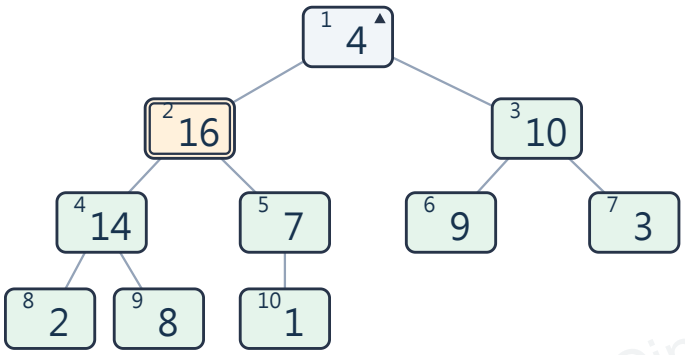
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 1 | 10 | 14 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 8 | 7 |

Double frame: root just repaired
No mark: legal; triangle: pending

Position three has key three; children six and seven contain nine and ten. Choose ten and swap positions three and seven. The old key stops at leaf seven; key nine at six is unchanged. Previously valid child heaps now combine into a valid heap rooted at three.



## Position two sends the old key through five and ten




Node: small index, large key

Fixed indices; moving keys

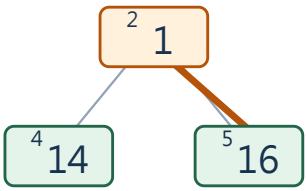
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 16 | 10 | 14 | 7 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 8 | 1 |

Double frame: root just repaired
No mark: legal; triangle: pending

Position two has child keys fourteen and sixteen; swap with sixteen first. The old key one reaches position five, remains below its sole child seven, and swaps again with position ten. The path is $2 \to 5 \to 10$; key fourteen at position four does not move.

## Follow the same old key through local child choices

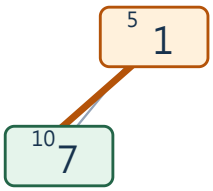
Comparison 1




16 > 1 → swap(2,5)

A[2]=16; A[5]=1

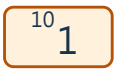
Comparison 2




7 > 1 → swap(5,10)

A[5]=7; A[10]=1

Leaf: stop




20, 21 > heap-size 10

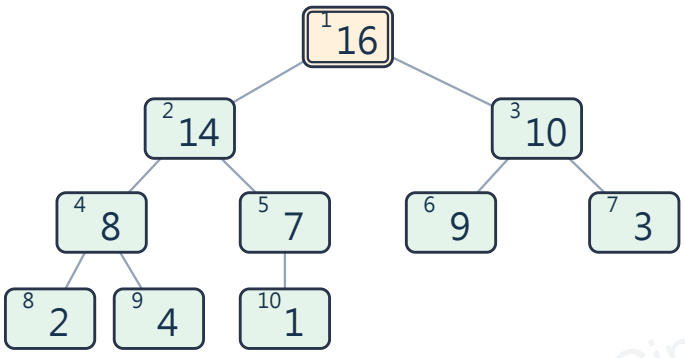
No child access

Old key 1 ends at index 10

Orange follows the old key one; the heavy edge identifies the chosen larger child. Index labels and key values are separate. First choose child five over four; at position five, only child ten exists. Unshown subtrees remain valid and need no rebuilding.

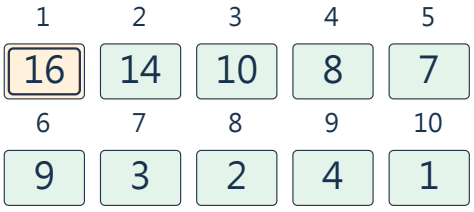


## Position one sends the old root through two, four, and nine




Node: small index, large key

Fixed indices; moving keys

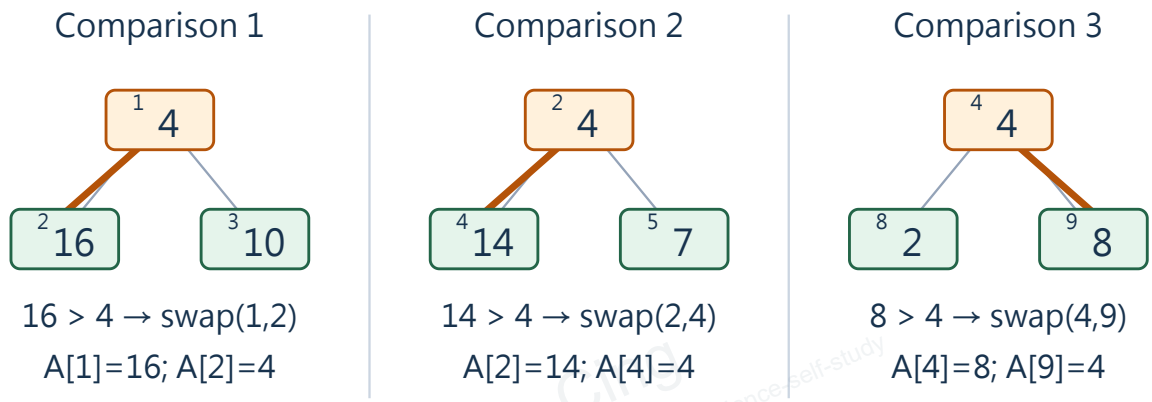



Double frame: root just repaired

No mark: legal; triangle: pending

The old root key four swaps with sixteen, fourteen, then eight, stopping at leaf nine. Final parent-child checks are $16 \geq 14, 10$, $14 \geq 8, 7$, $10 \geq 9, 3$, $8 \geq 2, 4$, and $7 \geq 1$. They cover all nine tree edges, certifying the heap while preserving all ten records.

## Three downward steps: distinguish positions from keys




Each panel shows local state before a swap and the resulting keys below it. Positions follow $1 \to 2 \to 4 \to 9$, while the same old key remains four; chosen child keys are sixteen, fourteen, and eight. At position nine, child indices eighteen and nineteen exceed $s = 10$, so the repair stops.



## The complete array after every exchange

Continue after repairing position three. All array indices stay fixed at $1..10$:

| Root being repaired | Exchanged indices | Resulting $A[1..10]$ |
|---|---|---|
| 2 | $2 \leftrightarrow 5$ | $[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$ |
| 2 | $5 \leftrightarrow 10$ | $[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| 1 | $1 \leftrightarrow 2$ | $[16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| 1 | $2 \leftrightarrow 4$ | $[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$ |
| 1 | $4 \leftrightarrow 9$ | $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ |

At each step, choose the largest valid child before comparing it with the parent. One swap changes only two cells; eight other cells and the record multiset stay unchanged. Keep intermediate states to reconstruct a downward path: the final array alone does not record its history.

## Linear-time building by summing node heights

For $n \geq 1$, position $i$ has height at least $h$ exactly when its leftmost depth-$h$ descendant exists: $i2^h \leq n$. Hence **exactly** $\lfloor n/2^h \rfloor$ nodes have height at least $h$.

Write each height as a sum of unit layers and exchange the finite sums:

$$\sum_{i=1}^{n} height(i) = \sum_{h=1}^{\lfloor \lg n \rfloor} \#\{i : height(i) \geq h\} = \sum_{h=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

One repair costs $O(height(i) + 1)$ and there are $\lfloor n/2 \rfloor$ calls, giving $O(n)$ total time. For $n \geq 2$, the loop itself performs at least $\lfloor n/2 \rfloor$ constant-work iterations, supplying $\Omega(n)$ and therefore $\Theta(n)$.

For $n = 15$, the height sum is $7 + 3 + 1 = 11$. Most nodes are not as high as the root. For an empty heap, the height sum is zero and no repair calls occur; return immediately without evaluating $\lg 0$ or asserting $0 < 0$. Building instead by repeated insertion can require $\Theta(n \lg n)$ because new keys may bubble to the root; that is a different construction algorithm.



## Heapsort shrinks the active prefix

```
HEAPSORT(A):
    BUILD-MAX-HEAP(A)
    for end = length(A) downto 2:
        swap(A[1],A[end])
        MAX-HEAPIFY(A,1,end-1)
```

At each iteration's start, $A[1..end]$ is a max-heap. Suffix $A[end + 1..n]$ is sorted increasingly and every suffix key is at least every heap key.

The swap moves the active maximum to $A[end]$, extending the sorted suffix. With heap-size $end - 1$, the old root's two subtrees remain valid, so only the new root needs repair. Swaps preserve the record multiset. At termination, the remaining singleton prefix completes the sorted array.

Building costs $O(n)$, followed by at most $n - 1$ repairs of $O(\lg(n + 1))$ time each. Total time is $O(n \lg(n + 1))$, with worst case $\Theta(n \lg n)$ from the comparison-sort lower bound. This implementation stops on equal keys and can take only $\Theta(n)$ on all-equal input; a worst-case bound need not describe every input.

Iterative repair uses $O(1)$ auxiliary space. Recursive repair requires additional stack space.

## Ten-record heapsort: nine maximum extractions

Start from $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$. Each round first exchanges position one with the old last position $s + 1$, moving the maximum to the sorted suffix, then repairs the prefix of new size $s$. Paths below record only repair swaps, excluding the root-to-end exchange.

| New size $s$ | Repair path | Repaired heap prefix | Sorted suffix |
|---|---|---|---|
| $9$ | 1→2→4→9 | $[14, 8, 10, 4, 7, 9, 3, 2, 1]$ | $[16]$ |
| $8$ | 1→3→6 | $[10, 8, 9, 4, 7, 1, 3, 2]$ | $[14, 16]$ |
| $7$ | 1→3→7 | $[9, 8, 3, 4, 7, 1, 2]$ | $[10, 14, 16]$ |
| $6$ | 1→2→5 | $[8, 7, 3, 4, 2, 1]$ | $[9, 10, 14, 16]$ |

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Finish the remaining five extractions

| New size $s$ | Repair path | Repaired heap prefix | Sorted suffix |
|---|---|---|---|
| $5$ | 1→2→4 | $[7, 4, 3, 1, 2]$ | $[8, 9, 10, 14, 16]$ |
| $4$ | 1→2 | $[4, 2, 3, 1]$ | $[7, 8, 9, 10, 14, 16]$ |
| $3$ | 1→3 | $[3, 2, 1]$ | $[4, 7, 8, 9, 10, 14, 16]$ |
| $2$ | 1→2 | $[2, 1]$ | $[3, 4, 7, 8, 9, 10, 14, 16]$ |
| $1$ | — | $[1]$ | $[2, 3, 4, 7, 8, 9, 10, 14, 16]$ |

In round two, the old final key one moves to the root and descends through $1 \to 3 \to 6$, swapping with ten and then nine. Every suffix is ascending and dominates every active heap key. The final prefix and suffix together form the complete ascending sequence.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Priority queues: maximum, extraction, increase, and insertion

$Q.A[1..Q.s]$ is a max-heap with writable capacity `Q.capacity`. The fixed-capacity code checks empty/full conditions first. Swaps move whole records; keys determine priority.

```
MAXIMUM(Q):
    if Q.s == 0: return EMPTY
    return Q.A[1]

EXTRACT-MAX(Q):
    if Q.s == 0: return EMPTY
    result = Q.A[1]; Q.A[1] = Q.A[Q.s]; Q.s -= 1
    if Q.s > 0: MAX-HEAPIFY(Q.A,1,Q.s)
    return result
```

`MAXIMUM` costs $O(1)$. `EXTRACT-MAX` saves the maximum record, replaces the root with the last record, decreases active size, and repairs downward in worst-case $O(\lg(s+1))$. Extracting a singleton leaves size zero and does not inspect a new root.



## Increase a key: repair only toward ancestors

```
SIFT-UP(Q,i):
    while i > 1 and key(Q.A[floor(i/2)]) < key(Q.A[i]):
        swap(Q.A[i],Q.A[floor(i/2)])
        i = floor(i/2)

INCREASE-KEY(Q,i,k):
    require 1 <= i <= Q.s and k >= key(Q.A[i])
    set-key(Q.A[i],k)
    SIFT-UP(Q,i)
```

An increase can violate only ancestor edges: the new key still dominates its old children. Each exchange repairs the current parent edge and moves the candidate upward. Stop at the root or a dominating parent, in worst-case $O(\lg(s+1))$. Evaluate `and` left to right with short-circuiting, checking $i > 1$ first.

## Insert directly without a sentinel value

```
INSERT(Q,record):
    if Q.s == Q.capacity: return FULL
    Q.s += 1
    Q.A[Q.s] = record
    SIFT-UP(Q,Q.s)
```

Write the new record directly into the new final position and sift upward; no negative-infinity sentinel is needed. Nonfull fixed-capacity insertion costs worst-case $O(\lg(s+1))$. Dynamic-array reallocation can move $\Theta(s)$ records in one operation and needs a separate amortized analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Trace the increase from position eight

Increase-key follows parent indices 8 → 4 → 2

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Original heap | 20 | 15 | 18 | 7 | 12 | 9 | 17 | 2 |
| Write 16 at 8 | 20 | 15 | 18 | 7 | 12 | 9 | 17 | **16** |
| Swap 8 ↔ 4 | 20 | 15 | 18 | **16** | 12 | 9 | 17 | 7 |
| Swap 4 ↔ 2 | 20 | **16** | 18 | 15 | 12 | 9 | 17 | 7 |

Compare 16 with parents 7, 15, then 20; stop because 20 ≥ 16.

After writing sixteen at eight, compare parent keys seven, fifteen, and twenty. Exchanges $8 \leftrightarrow 4$ and $4 \leftrightarrow 2$ suffice. Indices identify positions; record identities move during swaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Decrease, indexed deletion, and equal priorities

Decreasing a max-heap key can violate only child edges, so repair downward. For deletion at position $i$, replace it with the last record and decrease size; deleting the old last position simply decreases size.

Let replacement key be $z$. If $i > 1$ and $z > A[parent(i)]$, the old parent dominated the deleted key, which dominated its children; consequently $z$ dominates those children too and needs only upward repair. Otherwise the parent edge is valid and only downward repair is needed. Each operation costs worst-case $O(\lg(s+1))$ given a position; searching for an arbitrary key can still cost $\Theta(s)$.

A handle identifies a record. Every swap must update the handle-to-index map, and deletion removes the deleted handle. An array index does not permanently identify a record.



## Sequence numbers have different roles in sorting and queues

Heapsort is unstable: $[2a, 2b, 1c]$ can become $[1c, 2b, 2a]$. Stable ascending sorting can compare $(key, original_index)$, placing later equal records into the suffix first. FIFO ties in a max-priority queue instead compare $(priority, -arrival_index)$. The sequence-number direction differs, and both choices change the comparison and storage specification.

Let $a, b$ arrive in that order, with sequence numbers $1, 2$ and equal keys $5$:

| Purpose | Key for $a$ | Key for $b$ | First maximum extracted | Required order |
|---|---|---|---|---|
| Stable ascending sort | $(5, 1)$ | $(5, 2)$ | $b$, placed in the suffix | Output $a, b$ |
| FIFO priority ties | $(5, -1)$ | $(5, -2)$ | $a$, served immediately | Service $a, b$ |

Ascending sorting places extracted maxima from right to left; a queue serves each extracted maximum immediately. Determine how extraction order maps to the desired order before choosing the sequence-number sign.

## Hiring: distinguish interview and replacement costs

There are $n$ candidates with distinct, comparable ranks; larger is better. After each interview, replace the incumbent exactly when the new candidate is better. Dummy candidate `dummy` ranks below everyone.

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]
            hire best; hires += 1
    return (best, hires)
```

After iteration $i$, `best` is the best of the first $i$ candidates: initially none have arrived; a better arrival replaces the incumbent, otherwise the current best remains. Thus the final result is the overall best.



## One procedure, three different cost questions

With $X$ hires, interview cost $c_I$ and hiring cost $c_H$, total cost is $nc_I + Xc_H$. Empty input returns `dummy` with $X = 0$.

For $n \geq 1$, increasing ranks give $X = n$ and reverse order gives $X = 1$. Now distinguish three questions:

| Question | Random source or quantifier | Quantity |
|---|---|---|
| Worst case | Maximize over legal inputs | $\max_A X(A) = n$ |
| Average case | Specify an input distribution, e.g. uniform over $n!$ orders | $\mathbb{E}_A[X(A)]$ |
| Randomized procedure | Fix any input and uniformly shuffle it internally | $\mathbb{E}_R[X(A, R)]$ |

A uniform shuffle makes every fixed distinct-rank input a uniform permutation; the processing order must be controllable. Expected bounds permit bad runs: increasing order still has probability $1/n!$. Fewer hires reduce hiring cost, not the number of interviews.

## Indicators turn a count into event probabilities

Fix a finite sample space $\Omega$ with nonnegative outcome probabilities summing to $1$. A random variable has expectation $\mathbb{E}[X] = \sum_{\omega \in \Omega} \Pr(\omega) X(\omega)$.

The indicator $I_E$ is $1$ when event $E$ occurs and $0$ otherwise, so

$$\mathbb{E}[I_E] = 1 \Pr(E) + 0 \Pr(E^c) = \Pr(E).$$

If a target count equals $X = \sum_{i=1}^{m} X_i$, exchanging finite sums gives

$$\mathbb{E}[X] = \sum_{\omega} \Pr(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} \Pr(\omega) X_i(\omega) = \sum_{i} \mathbb{E}[X_i].$$

This is **linearity of expectation**; the $X_i$ need not be independent. Independence may instead be needed to calculate a probability or factor an intersection probability.

For example, let $X_1 = X_2$ both indicate heads on the same fair toss. Then $\mathbb{E}[X_1 + X_2] = 1$, but $\Pr(X_1 = X_2 = 1) = 1/2$, not $1/4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Three tools: union bound, Markov bound, and tail sums

First identify whether the target is an event probability, an expected count, or the expected maximum. Indicators yield three basic tools.

Since $I_{\cup_i E_i} \le \sum_i I_{E_i}$, taking expectations gives

$$\Pr\left(\bigcup_i E_i\right) \le \sum_i \Pr(E_i).$$

For $X \ge 0$ and $t > 0$, inequality $X \ge t I_{\{X \ge t\}}$ gives Markov's bound

$$\Pr(X \ge t) \le \frac{\mathbb{E}[X]}{t}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Tail sums: count the levels reached by each outcome

For nonnegative integer $L$, every outcome satisfies $L = \sum_{j \geq 1} I_{\{L \geq j\}}$, hence

$$\mathbb{E}[L] = \sum_{j \geq 1} \Pr(L \geq j).$$

Finite bounds allow finite-sum exchange. Infinite tails follow by monotone limits of nonnegative partial sums, allowing both sides to be infinite. None of these three tools requires independence.

Concrete probability products may still require independent trials. Do not drop those model assumptions merely because the general tool needs no independence.

Example: $L = 0, 1, 3$ with probabilities $1/2, 1/4, 1/4$. Direct expectation gives $0/2 + 1/4 + 3/4 = 1$. The tails are $\Pr(L \geq 1) = 1/2$, $\Pr(L \geq 2) = 1/4$, $\Pr(L \geq 3) = 1/4$; their sum is also $1$.



## Hiring count: the probability of a record at each position

Assume uniformity over all $n!$ distinct-rank permutations, and let $X_i$ indicate hiring at position $i$. Condition on the set of the first $i$ candidates: all $i!$ relative orders remain equally likely, so their best candidate is last with probability $1/i$.

Hiring at $i$ is exactly the event that this candidate is best among the first $i$. Thus

$$\mathbb{E}[X] = \sum_{i=1}^{n} \Pr(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

Since $1/x$ decreases, comparing rectangles with integrals gives

$$\int_1^{n+1} \frac{dx}{x} \leq H_n \leq 1 + \int_1^n \frac{dx}{x}, \qquad n \geq 1.$$

Consequently $\mathbb{E}[X] = \Theta(\ln n)$, but total expected cost remains $nc_I + c_H H_n$: interview cost does not become logarithmic.

## A six-candidate run and its expected-cost comparison

| Position $i$ | Rank | Best after interview | Hire? | Total hires |
|---|---|---|---|---|
| 1 | 2 | 2 | Yes | 1 |
| 2 | 5 | 5 | Yes | 2 |
| 3 | 1 | 5 | No | 2 |
| 4 | 4 | 5 | No | 2 |
| 5 | 6 | 6 | Yes | 3 |
| 6 | 3 | 6 | No | 3 |

Input $[2, 5, 1, 4, 6, 3]$ sets records at positions $1, 2, 5$, making three hires; the uniform mean at this size is $H_6 = 49/20 = 2.45$. Exactly one hire has probability $1/n$; hiring everyone requires the unique increasing permutation, with probability $1/n!$. For empty input define $H_0 = 0$ separately.



## The same method counts inversions and fixed points

Uniformly permute $n$ distinct keys. For positions $i < j$, let $I_{ij}$ indicate $A[i] > A[j]$. Exchanging the two entries bijectively pairs inversion and noninversion outcomes, so $\Pr(I_{ij} = 1) = 1/2$.

$$V = \sum_{i<j} I_{ij}, \qquad \mathbb{E}[V] = \binom{n}{2}\frac{1}{2} = \frac{n(n-1)}{4}.$$

Inversion events sharing positions can be dependent; linearity still applies. When $n = 2$, the count is either $0$ or $1$, although its expectation is $1/2$.

Now uniformly return $n \geq 1$ hats one-to-one to their original owners. Each person receives their own hat with probability $1/n$. The fixed-point count satisfies

$$F = \sum_{i=1}^{n} I_i, \qquad \mathbb{E}[F] = n\frac{1}{n} = 1.$$

For $n = 2$, both hats are correct or both wrong, each with probability $1/2$. Mean one does not imply exactly one correct hat in every outcome. For $n = 0$, directly take $F = 0$.

## Uniform integers by rejecting out-of-range bit patterns

The contract of `RANDOM(a,b)` is an inclusive integer with probability $1/(b-a+1)$ per value, conditionally uniform given prior history. Assume integer $a \le b$ and access to independent fair bits.

Set $m = b - a + 1$. If $m = 1$, return $a$. Otherwise let $k = \lceil \lg m \rceil$, repeatedly read $k$ bits as $z \in \{0, \ldots, 2^k - 1\}$, and return $a + z$ when $z < m$. Reject other patterns and redraw the whole block.

Acceptance probability per round is $p = m/2^k > 1/2$. For any legal value $v$, its eventual return probability is

$$\sum_{t=0}^{\infty} (1-p)^t \frac{1}{2^k} = \frac{1}{p2^k} = \frac{1}{m}.$$

Expected rounds are $1/p < 2$, giving $O(\lg m)$ expected bit cost. Termination is almost sure; there is no fixed worst-case retry limit.

Taking a remainder directly can bias the result: eight three-bit patterns reduced modulo $3$ give preimage counts $3, 3, 2$ for remainders $0, 1, 2$. The pseudorandom generator and bounded-integer sampling procedure are distinct parts of an implementation.



## In-place shuffling fixes one more position per round

Select uniformly from the unfixed suffix and swap the selected record into position $i$. Distinguish record identities even when keys coincide.

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap(A[i],A[j])
```

After round $i$, later swaps never touch $A[1..i]$. Empty input has one empty output; a singleton swaps only with itself. Under unit-cost integer sampling, time is $\Theta(n)$ and extra space is $O(1)$; bit-sampling cost is separate.

## Uniform shuffling: extend a prescribed prefix

Claim: any prescribed sequence of $i-1$ distinct identities occupies $A[1..i-1]$ with probability $(n-i+1)!/n!$. For $i=1$, the empty prefix has probability one.

Conditioned on that prefix, the next desired identity occupies one of the $n-i+1$ remaining positions. Conditional uniform sampling selects it with probability $1/(n-i+1)$, so

$$\frac{(n-i+1)!}{n!}\frac{1}{n-i+1}=\frac{(n-i)!}{n!}.$$

Induction gives probability $0!/n!=1/n!$ for every complete permutation. For $n=4$ and target $[C,D,A,B]$:

| Fixed length | $0$ | $1$ | $2$ | $3$ | $4$ |
|---|---|---|---|---|---|
| Prescribed prefix | empty | $C$ | $C,D$ | $C,D,A$ | $C,D,A,B$ |
| Probability | $1$ | $1/4$ | $1/12$ | $1/24$ | $1/24$ |

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Four-record shuffle: track the fixed prefix at every round

Start with $[A,B,C,D]$ and draw $j=3,4,3,4$ in successive rounds. Letters identify records rather than comparison keys.

| Round $i$ | Legal indices | Drawn $j$ | Full array after swap | Fixed prefix |
|---|---|---|---|---|
| 1 | $1..4$ | 3 | $[C,B,A,D]$ | $C$ |
| 2 | $2..4$ | 4 | $[C,D,A,B]$ | $C,D$ |
| 3 | $3..4$ | 3 | $[C,D,A,B]$ | $C,D,A$ |
| 4 | $4$ | 4 | $[C,D,A,B]$ | $C,D,A,B$ |

This path has probability $(1/4)(1/3)(1/2)(1)=1/24$. Every prescribed output uniquely determines a legal path: locate its next identity in the remaining suffix and swap it into position $i$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## The fixed prefix never moves again

Start [A,B,C,D]: select only from the unfixed suffix

| | Pos 1 | Pos 2 | Pos 3 | Pos 4 | Round probability |
|---|---|---|---|---|---|
| i=1, j=3 | C | B | A | D | 1/4 |
| i=2, j=4 | C | D | A | B | 1/3 |
| i=3, j=3 | C | D | A | B | 1/2 |
| i=4, j=4 | C | D | A | B | 1 |

Green cells stay fixed. This path has probability 1/4·1/3·1/2·1=1/24.

Later rounds touch only unfixed positions, so the prefix remains fixed. A trace checks the procedure; the preceding joint invariant proves general uniformity.



## Uniform positions do not guarantee a uniform permutation

A uniform cyclic shift puts each record in every position with probability $1/n$, yet for $n \geq 3$ it produces only $n$ outputs rather than $n!$. Uniform marginals do not imply a uniform joint permutation.

Another incorrect procedure runs $i = 1..n$, drawing $j$ from all of $1..n$ each round before swapping. For $n = 3$, its $3^3 = 27$ random paths are equally likely. Uniformity over six permutations would require $27/6$ paths per output, impossible for an integer path count.

This procedure can move earlier positions again, so the fixed-prefix proof no longer applies. A demonstrated bias is stronger than saying the shuffle looks insufficiently random.

Running only the first $m$ rounds of the correct shuffle, $0 \leq m \leq n$, gives each ordered sample probability $(n-m)!/n!$. Each unordered $m$-subset has $m!$ orders, hence probability $1/\binom{n}{m}$. This saves later swaps, but constructing an indexable array can still cost $\Theta(n)$.

## Random priorities: collision handling is part of uniformity

Give $n \geq 1$ records independent uniform priorities $P_i \in \{1, \ldots, M\}$ for integer $M \geq n$, and sort by priority. Conditioned on all priorities being distinct, exchanging record labels bijectively maps orders while preserving probability. Thus all $n!$ relative orders are equally likely.

Each pair collides with probability $1/M$. The union bound gives

$$\Pr(\exists i < j : P_i = P_j) \leq \frac{\binom{n}{2}}{M}.$$

With $M = n^3$ and $n \geq 2$, this is $(n-1)/(2n^2) < 1/(2n)$. Rare collisions are still possible.

Keeping original order on ties biases the unconditional output. For $n = 2$, the first record precedes the second with probability $\Pr(P_1 \leq P_2) = (M+1)/(2M) > 1/2$.

One repair redraws the entire set after any collision and accepts only distinct priorities. For this $M$, acceptance probability has a positive constant lower bound, so expected rounds are constant. Alternatively, uniformly shuffle each tie group independently. Comparison-based priority sorting takes $\Theta(n \lg n)$ time and $O(n)$ priority storage; redraws must also detect collisions, which is not free.



## After randomization, specify correctness and what is controllable

RANDOMIZED-HIRE uniformly shuffles, then runs HIRE. Every fixed distinct-rank input becomes a uniform permutation, giving $H_n$ expected hires and expected cost $nc_I + c_H H_n$. Another $\Theta(n)$ shuffle operations must be accounted for separately from monetary hiring costs.

If candidates arrive externally and cannot be reordered, the algorithm lacks this control. The $H_n$ analysis still requires a uniform-arrival assumption; an unavailable shuffle cannot remove it.

For the shuffle and randomized-pivot sort in this chapter, cost is random while every returned result satisfies its specification. These are Las Vegas algorithms. Monte Carlo algorithms allow a small error probability, which must be stated with its input quantifiers. Expected time alone says nothing about correctness probability.

The later online-selection strategy has a different objective: it follows its selection rule even when the chosen candidate is not globally best. Its success probability needs a separate model and proof.

## Quicksort partitions, then sorts both sides

Use the 1-based closed interval $A[p..r]$. Partitioning around a pivot establishes order between regions; it does not sort either region internally.

```
QUICKSORT(A,p,r):
    if p >= r: return
    q = PARTITION-LAST(A,p,r)
    QUICKSORT(A,p,q-1)
    QUICKSORT(A,q+1,r)
```

An empty interval ($p > r$) or singleton ($p = r$) returns immediately. Partition returns the pivot position $q$, and both recursive intervals **exclude** $q$, so their lengths strictly decrease. Correctness and termination depend on this return contract.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Last-position partition: save the key, move whole records

The last-position partition is given below. Save pivot key $x$ first. Swaps move complete records, keeping every associated field with its key.

```
PARTITION-LAST(A,p,r):
    x = key(A[r]); i = p-1
    for j = p to r-1:
        if key(A[j]) <= x:
            i += 1
            swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Loop boundaries: i ends the small region; j is next

The pivot stays at A[r]. Use **1-based indices and the inclusive interval A[p..r]**. The boundary i=p−1 means that the ≤ region is empty.

| Region | Known relation | Initially i=p−1, j=p |
|---|---|---|
| A[p..i] | ≤ pivot | Empty |
| A[i+1..j−1] | > pivot | Empty |
| A[j..r−1] | Unclassified | All non-pivot keys |
| A[r] | Pivot | Fixed until the final exchange |

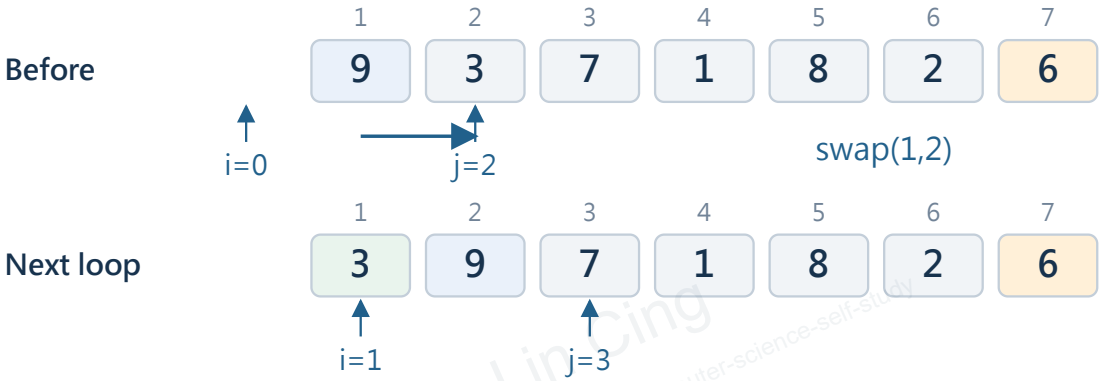
**Diagram key:** green ≤, blue >, gray unknown, orange pivot. Both transition diagrams use [9,3,7,1,8,2,6] with pivot 6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A small key: exchange, then advance both boundaries

### 3 ≤ 6: advance i, then exchange A[1] and A[2]



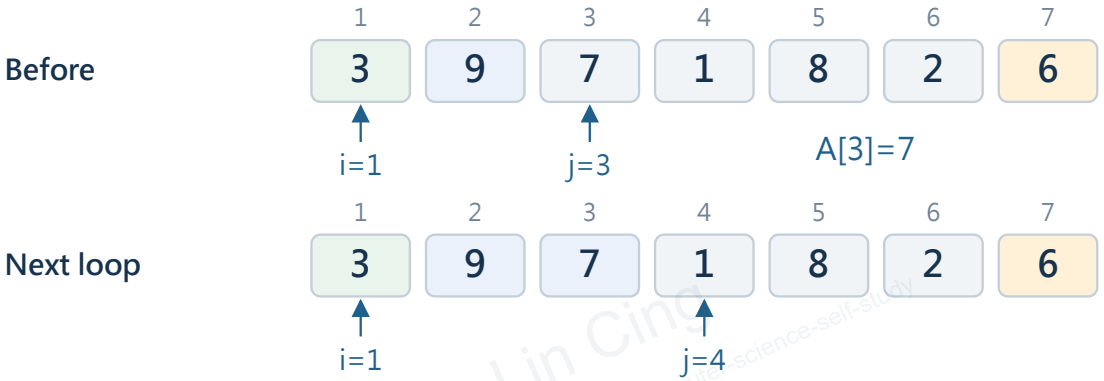

3 joins ≤; displaced 9 stays in >. Pivot 6 remains at index 7.

For A[2]=3, advance i to 1 and exchange A[1],A[2]. The key 3 joins ≤; displaced 9 stays in >. Finally advance j. In general i may equal j, giving a harmless self-exchange.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A large key: leave values fixed and advance j

### 7 > 6: no exchange; keep i and advance j



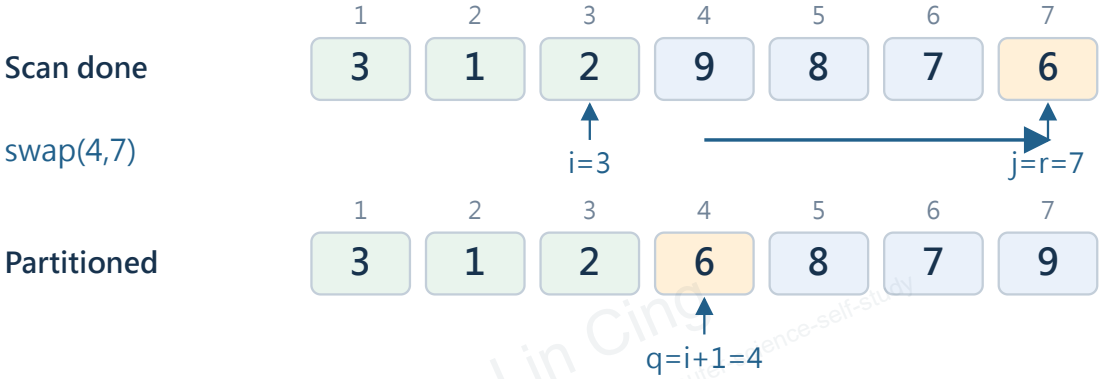

7 moves from unknown to >. Values stay fixed; the boundary moves.

Since A[3]=7>6, leave i and the array unchanged. Advancing j adds 7 to the > region. **The values stay fixed; our classification changes.**



## The final exchange establishes the pivot position q

### Only after scanning: place the pivot between the two regions




≤6: [3,1,2] | pivot 6 | >6: [8,7,9]; the sides remain unsorted.

The unknown length r−j decreases once per loop, proving termination. At j=r, exchange A[i+1],A[r] and return q=i+1. The left region is ≤pivot and the right region >pivot; neither side is necessarily sorted.

## From the partition invariant to sorting correctness

1. **Initialization:** i=p−1,j=p makes both classified regions empty.
2. **Maintenance:** a small key expands $\leq$ after an exchange; a large key expands >. Either branch removes one unknown key and leaves the pivot fixed.
3. **Partition termination:** no unknown keys remain. The final exchange places the pivot at q. Exchanges preserve the input multiset.
4. **Recursive induction:** empty and singleton intervals are sorted. Both children exclude q and are strictly shorter. Sorting them and adjoining the pivot therefore sorts the whole interval.

The ordering relation alone is insufficient: the proof also needs **multiset preservation, strictly smaller recursive intervals**, and sorted children upon return.



## Complete partition trace: every test and exchange

Input is $A = [9, 3, 7, 1, 8, 2, 6]$ with $p = 1, r = 7$. Save last key $x = 6$ and initialize $i = 0$. The table gives $i$ after each test.

| Scan index $j$ | Test | $i$ | Full array after action |
|---|---|---|---|
| 1 | $9 > 6$, no swap | 0 | $[9, 3, 7, 1, 8, 2, 6]$ |
| 2 | $3 \leq 6$, swap $1, 2$ | 1 | $[3, 9, 7, 1, 8, 2, 6]$ |
| 3 | $7 > 6$, no swap | 1 | $[3, 9, 7, 1, 8, 2, 6]$ |
| 4 | $1 \leq 6$, swap $2, 4$ | 2 | $[3, 1, 7, 9, 8, 2, 6]$ |
| 5 | $8 > 6$, no swap | 2 | $[3, 1, 7, 9, 8, 2, 6]$ |
| 6 | $2 \leq 6$, swap $3, 6$ | 3 | $[3, 1, 2, 9, 8, 7, 6]$ |
| End of scan | Swap $4, 7$ | 3 | $[3, 1, 2, 6, 8, 7, 9]$ |

Return $q = 4$; both sides remain unsorted. Six key comparisons; $\Theta(r - p + 1)$ time and $O(1)$ extra space.

## Worst-case time: a family attaining quadratic work

Sorted distinct input with a fixed final pivot always chooses the maximum. The two child lengths are n−1 and 0.

| Current length | Key comparisons | Next nonempty length |
|---|---|---|
| n | n−1 | n−1 |
| n−1 | n−2 | n−2 |
| … | … | … |
| 2 | 1 | 1 |

The sum is $n(n-1)/2$, proving a worst-case Ω(n²) lower bound for an entire input family, rather than extrapolating from one measurement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The quadratic upper bound must cover every split

Let C(n) be the worst comparison count. Child sizes satisfy a+b=n−1. Induction on smaller sizes gives

$$C(a)+C(b)+(n-1)\le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1=\frac{n(n-1)}{2}-ab\le \frac{n(n-1)}{2}.$$

With C(0)=C(1)=0, this covers all splits. Equality holds when a=0 or b=0. Each pivot leaves recursion, so there are O(n) calls. Total time is O(n+C); the matching family proves **worst-case Θ(n²)**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Balanced splits: count the comparison work by level

Take n=15 and always choose the true median. Count only Lomuto key comparisons; singletons do not call partition.

| Depth d | Subproblems | Size each | Comparisons at level |
|---|---|---|---|
| 0 | 1 | 15 | 14 |
| 1 | 2 | 7 | 2×6=12 |
| 2 | 4 | 3 | 4×2=8 |
| 3 | 8 | 1 | 0 |

Total: 34 comparisons, versus 105 for endpoint splits on 15 keys. Partition still scans linearly; the difference is how much work survives at each depth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A fixed split ratio also gives logarithmic depth

If the larger child always has size at most ρ times the parent size, for a fixed ρ<1, the longest branch has O(log n) levels. Intervals at one depth are disjoint, giving at most n scanned keys per level and O(n log n) total work. A fixed 1:9 split suffices.

For perfectly balanced $n = 2^k - 1$, depth d has $2^d$ subproblems of length $2^{k-d} - 1$. Its comparison work is

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

The first roughly k/2 levels each require Θ(n) comparisons, proving Ω(n log n) for that execution and hence Θ(n log n) overall. This lower bound counts actual recursive work; a comparison-sorting worst-input lower bound does not apply to every individual input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Duplicates: a random pivot cannot repair all-equal input

For $[5, 5, 5, 5, 5]$, every `<= x` test succeeds, so every partition returns $q = r$. Successive sizes $5, 4, 3, 2$ cost $4 + 3 + 2 + 1 = 10$ comparisons; for $n$ records the total is $n(n-1)/2$.

A randomly chosen pivot is still $5$ and cannot change this path. The later expected $\Theta(n \lg n)$ analysis of the two-way version therefore assumes **distinct keys**. The order-statistics section gives three-way partitioning into keys less than, equal to, and greater than the pivot. Only the strict regions recurse. All-equal input then finishes in one $\Theta(n)$ partition.

Stability is a separate guarantee. Input $[2_a, 2_b, 1_c]$ with last pivot $1$ becomes $[1_c, 2_b, 2_a]$ after the final swap. The two key-$2$ records have reversed order. Random pivot selection does not make this swapping implementation stable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Recurse on the smaller side to bound stack space

The original two-recursion procedure can retain $\Theta(n)$ stack frames. In-place partitioning alone does not give constant-space quicksort. Recurse only on the smaller side and handle the larger side in the current frame's loop.

```
QUICKSORT-SMALL-STACK(A,p,r):
    while p < r:
        q = PARTITION-LAST(A,p,r)
        if q-p < r-q:
            QUICKSORT-SMALL-STACK(A,p,q-1)
            p = q+1
        else:
            QUICKSORT-SMALL-STACK(A,q+1,r)
            r = q-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Smaller-side recursion bounds the live call chain

Side lengths satisfy $a + b = n - 1$, so the actual recursive call has size at most $\lfloor (n-1)/2 \rfloor$. Sizes at least halve along the chain of unreturned calls, yielding worst-case $O(\lg(n+1))$ stack space.

If every pivot is largest, the recursive side is empty but the loop still partitions sizes $n, n-1, \ldots, 2$. Stack space is then constant while time remains $\Theta(n^2)$. Analyze these two resources separately.

Median pivots: a chain of live calls

Nested frames

15 → 7 → 3 → 1: four simultaneous frames

Largest pivots: successive loop iterations

One working frame

Each empty recursive call returns immediately; the larger side stays in this frame.



## Randomization bounds expectation for each fixed input

At each partition, draw integer $k$ uniformly from closed interval $p..r$, swap $A[k]$ with $A[r]$, then use the same last-position partition. Future choices must remain uniform conditional on their current states.

For distinct keys, every local rank has pivot probability $1/n$, whatever the current arrangement. Expectation is over **internal algorithm choices**; no uniform-input assumption is required.

Every fixed input can still encounter a path that repeatedly chooses an endpoint, costing $\Theta(n^2)$. An expectation is neither a per-execution upper bound nor an unproved tail-probability bound.

Separate mathematical uniform sampling from its implementation. A fixed seed reproduces a pseudorandom sequence; merely calling random does not prove uniformity. Bounded-integer sampling can use the rejection procedure developed earlier in this chapter.

## A pair compares exactly when an interval endpoint is first

Name distinct keys in increasing order $z_1 < \cdots < z_n$. Let $X_{ij}$ indicate whether $z_i, z_j$ are directly compared. They compare at most once: one must be the pivot, which then leaves recursion.

Consider $Z_{ij} = \{z_i, \ldots, z_j\}$. An outside pivot keeps this entire interval on one side. If the first pivot inside it is endpoint $z_i$ or $z_j$, the pair compares immediately. If it is an interior key, the endpoints enter different recursive regions and never meet again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Outside pivots preserve the entire rank interval

**Sorted ranks for analysis: $Z_{2,5}$ = {2,3,4,5}; not an array snapshot**

Pivot 7 first: ranks 2..5 remain together in its left child; $X_{2,5}$ is undecided.

**First within interval: endpoint 2 (or 5)**

This partition compares 2 and 5 → $X_{2,5}$=1

**First within interval: interior 3 (or 4)**

2 left, 5 right → never compare, $X_{2,5}$=0

Four interval keys are equally likely to be first; two are endpoints. P($X_{2,5}$=1)=2/4=1/2.

Thus $X_{ij} = 1$ exactly when the first pivot selected from $Z_{ij}$ is one of its endpoints. It need not be the first pivot of the entire execution.

## From four first choices to the general comparison probability

For $z_2, z_5$, inspect only the first pivot from $Z_{2,5} = \{z_2, z_3, z_4, z_5\}$.

| First pivot inside the interval | What happens to $z_2, z_5$ | $X_{2,5}$ |
|---|---|---|
| $z_2$ | Compare immediately | 1 |
| $z_3$ | Separate forever | 0 |
| $z_4$ | Separate forever | 0 |
| $z_5$ | Compare immediately | 1 |

Each key has probability $1/4$ of being first inside the interval, so the comparison probability is $1/2$. Generally, for interval length $m = j - i + 1$,

$$\Pr(X_{ij} = 1) = \frac{2}{m} = \frac{2}{j - i + 1}.$$



## Why the first pivot inside an interval is uniform

Prove uniformity rather than assume it. Fix an interval $Z$ of $m$ keys and a designated $z \in Z$. Induct on the size $N$ of the current subproblem containing $Z$. If $N = m$, the first pivot is uniform, giving probability $1/m$. For $N > m$, choosing $z$ immediately has probability $1/N$; choosing another key of $Z$ fails.

An outside pivot leaves all of $Z$ in a strictly smaller subproblem. Induction gives designated-first probability $1/m$ there. Summing over the $N - m$ outside keys yields

$$\frac{1}{N} + \frac{N - m}{N}\frac{1}{m} = \frac{1}{m}.$$

The first pivot inside the interval is therefore uniform; the denominator is $m$, not original input size $n$. Other recursive branches may execute in between but do not move or compare these keys.

## Expected comparisons: sum over rank-interval lengths

Total key comparisons are $C = \sum_{1 \le i < j \le n} X_{ij}$. Linearity of expectation requires no independence. There are $n + 1 - k$ rank intervals of length $k = j - i + 1$, so

$$\begin{aligned}\mathbb{E}[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\ &= 2(n+1)\sum_{k=2}^{n} \frac{1}{k} - 2(n-1) \\ &= 2(n+1)H_n - 4n, \qquad H_n = \sum_{k=1}^{n} \frac{1}{k}.\end{aligned}$$

Bounds $\ln(n+1) \le H_n \le 1 + \ln n$ give expected $\Theta(n \lg n)$ comparisons. Scanning work is proportional to these comparisons, with $O(n)$ additional call overhead, so expected time is also $\Theta(n \lg n)$.

The exact formula assumes distinct keys, conditionally uniform pivots, and one `<=` test per nonpivot key. It excludes index tests, swaps, and random-number generation. Comparison events are dependent; independent-variable variance or tail formulas cannot be imported without further proof.



## A first-position pivot has its own invariant

This single-scan version keeps pivot $x = A[p]$ at the first position until cleanup. The interval is still the closed interval $A[p..r]$.

```
PARTITION-FIRST(A,p,r):
    x = A[p]; i = p
    for j = p+1 to r:
        if key(A[j]) <= key(x):
            i = i+1
            swap(A[i],A[j])
    swap(A[p],A[i])
    return i
```

Before each test, $A[p] = x$, $A[p+1..i] \le x$, $A[i+1..j-1] > x$, and $A[j..r]$ is unknown. Small keys expand the left region; large keys only advance $j$. The unknown length decreases each round. Cleanup places the pivot at $i$; exclude $i$ from both recursive sorts.

## Check both partitions using the same pivot key

Use first-position input $[6, 10, 13, 5, 8, 3, 2, 11]$, pivot $x = 6$, and initial $i = 1$.

| $j$ | Test and action | Array after action |
|---|---|---|
| 2 | $10 > 6$, $i = 1$ | $[6, 10, 13, 5, 8, 3, 2, 11]$ |
| 3 | $13 > 6$, $i = 1$ | Unchanged |
| 4 | $5 \leq 6$, $i = 2$, swap $2, 4$ | $[6, 5, 13, 10, 8, 3, 2, 11]$ |
| 5 | $8 > 6$, $i = 2$ | Unchanged |
| 6 | $3 \leq 6$, $i = 3$, swap $3, 6$ | $[6, 5, 3, 10, 8, 13, 2, 11]$ |
| 7 | $2 \leq 6$, $i = 4$, swap $4, 7$ | $[6, 5, 3, 2, 8, 13, 10, 11]$ |
| 8 | $11 > 6$, $i = 4$ | Unchanged |
| End | Swap $1, 4$, return $4$ | $[2, 5, 3, 6, 8, 13, 10, 11]$ |



## Check the same pivot in both variants

### Same pivot key, different internal orders

Both: q=4, pivot=6; left <=6, right >6. Regions need not be sorted.

To choose the same key $6$ in the last-position version, first exchange the endpoints, then partition $[11, 10, 13, 5, 8, 3, 2, 6]$. The result can be $[5, 3, 2, 6, 8, 10, 13, 11]$. Both place $6$ at position $4$ and make $7$ scan-key comparisons; their internal region orders may differ.

## Compare contracts, boundaries, and return values

| Version | Pivot and initial boundaries | Return value and recursion |
|---|---|---|
| CLRS last-position single scan | $x = A[r], i = p - 1, j = p$ | One pivot index $q$; recurse on $[p, q - 1], [q + 1, r]$ |
| MIT 2005 first-position single scan | $x = A[p], i = p, j = p + 1$ | One pivot index $q$; exclude $q$ again |
| First-position inward scan with final placement | $x = A[p]$; both scans stop on equality | Exchange pivot after crossing; return its position $q$ |
| This chapter's three-way partition | Save pivot key $x$; $lt = i = p$, $gt = r$ | Equal closed interval $[lt, gt]$; exclude that whole interval |

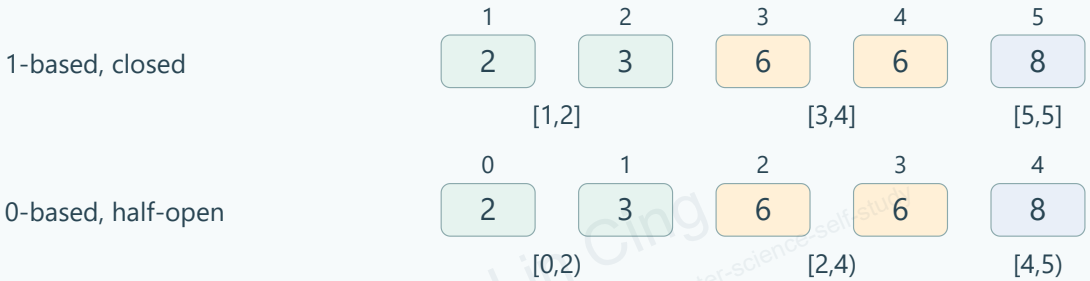
Inward scanning still leaves two return contracts to distinguish. Classical Hoare partition returns a split boundary without guaranteeing the pivot's final position. Another inward-scan variant finally exchanges the pivot and returns its position. Their recursive intervals differ; the name Hoare alone is insufficient. The inward-placement code below specifies the latter procedure.



## Two interval contracts for the same records

The two highlighted keys are the same records: closed $[3, 4]$ in 1-based indexing, half-open $[2, 4)$ in 0-based indexing.

Same records; different interval contracts




Changing notation moves no records. Exclude both equal keys from recursion.

Convert closed $[p, r]$ to half-open $[p - 1, r)$: exclude the right endpoint. Likewise $[p, q - 1]$ becomes $[p - 1, q - 1)$. Empty intervals have length zero.

## Inward scanning with final placement stops on equal keys

Use closed interval $p \le r$ with the first record as pivot. Evaluate and left to right with short-circuiting: check the boundary before reading an array entry.

```
PARTITION-INWARD-PLACE(A,p,r):
    x = key(A[p]); i = p+1; j = r
    while i <= j:
        while i <= j and key(A[i]) < x: i += 1
        while i <= j and key(A[j]) > x: j -= 1
        if i >= j: break
        swap(A[i],A[j])
        i += 1; j -= 1
    swap(A[p],A[j])
    return j
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Inward scanning: invariant and complete exchanges

Classified regions satisfy $A[p+1..i-1] \le x$ and $A[j+1..r] \ge x$. Scans skip strictly smaller or strictly larger keys. Before crossing, exchange the stopped records and advance both indices, so equal keys cannot stall progress. The unknown region shrinks; finally exchange the pivot into $j$. Recurse on $[p, j-1]$ and $[j+1, r]$.

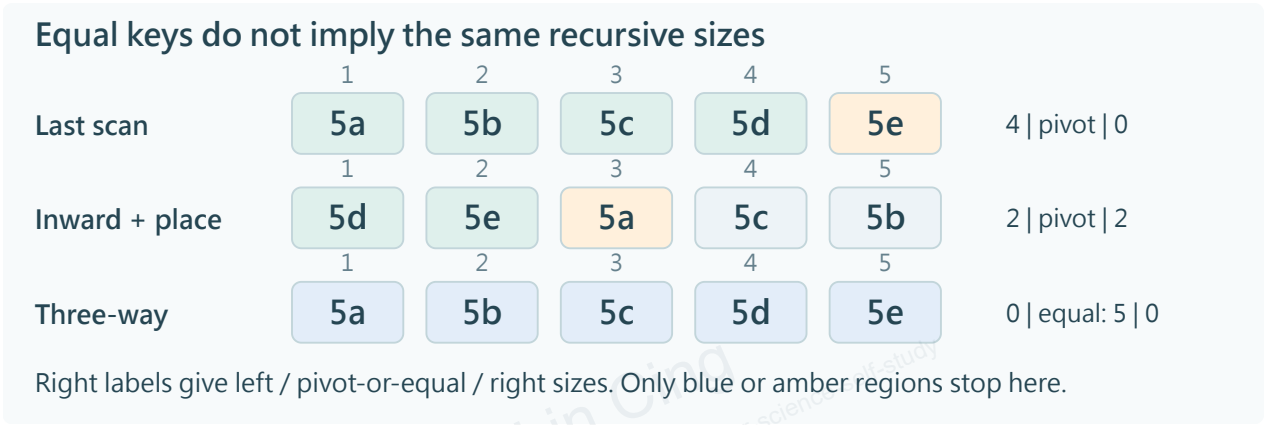
Input $[6, 10, 13, 5, 8, 3, 2, 11]$, $x = 6$. Scan stops are recorded before the exchange.

| Stopped $(i, j)$ | Exchange | Complete array afterward |
|---|---|---|
| $(2, 7)$ | $2, 7$ | $[6, 2, 13, 5, 8, 3, 10, 11]$ |
| $(3, 6)$ | $3, 6$ | $[6, 2, 3, 5, 8, 13, 10, 11]$ |
| $(5, 4)$ crossed | Final $1, 4$ | $[5, 2, 3, 6, 8, 13, 10, 11]$ |

Return $q = 4$. Left keys are $\le 6$ and right keys are $\ge 6$; neither side needs to be internally sorted.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Five equal keys produce three different partition outcomes




Letters identify records. Inward placement swaps $2, 5$, then $3, 4$, then $1, 3$. For all-equal inputs, its nearly halved sides give $\Theta(n \lg n)$ time, versus $\Theta(n^2)$ for last-position scanning and $\Theta(n)$ for three-way partitioning. These are input-family bounds; inward scanning can still be quadratic on other inputs.



# One classification can require two key comparisons

The exact formula counts executions of `key(A[j]) <= x` in the last-position single scan: one per nonpivot key. A three-way version that tests `< x` and then, on failure, `> x` can use two Boolean comparisons per classification, including for an equal key.

For keys $[1, 4, 3]$ classified against pivot value $x = 3$, key $1$ needs one test; keys $4, 3$ need two each, totaling five rather than three. In-place three-way swapping may change processing order, but its branch-test rule must still be counted.

The data-structures chapter uses 0-based half-open $[l, r)$ and equal region $[lt, gt)$. This chapter uses 1-based closed $[p, r]$ and equal region $[lt, gt]$. Both exclude the entire equal region, but boundaries and exact comparison constants require separate verification.

With distinct keys and conditionally uniform pivots, three-way quicksort still has expected $\Theta(n \lg n)$ time: each level performs at most constant-factor linear classification work, with the same recursive sizes. This asymptotic conclusion does not make its Boolean count exactly $2(n + 1)H_n - 4n$.

## Four ranks: pair probabilities differ from run costs

Name four distinct keys $z_1, z_2, z_3, z_4$ by increasing rank; their input order is arbitrary.

| Rank pairs | Interval length | Direct-comparison probability |
|---|---|---|
| $(1,2), (2,3), (3,4)$ | 2 | $1$ each |
| $(1,3), (2,4)$ | 3 | $2/3$ each |
| $(1,4)$ | 4 | $1/2$ |

Thus $\mathbb{E}[C] = 3 + 4/3 + 1/2 = 29/6$. Substituting $H_4 = 25/12$ gives the same $2 \cdot 5 \cdot 25/12 - 16 = 29/6$.

A run uses an integer number of comparisons. A first pivot of rank $2$ or $3$ gives four comparisons with probability $1/2$. An endpoint first, then the median of the remaining three, gives five with probability $(1/2)(1/3) = 1/6$. Remaining runs use six, with probability $1/3$. Hence $4/2 + 5/6 + 6/3 = 29/6$, independently checking the expected run cost.

Adjacent ranks always compare, but not every pair does. Average $29/6$ does not mean any run performs a fractional comparison.



## Counting sort assigns an output block to each key

Preconditions: integer $K \geq 0$ and keys in $0..K$. Input/output positions are $1..n$. Key-indexed storage must support $K + 1$ entries, and each counter must represent values $0..n$.

```
COUNTING-SORT(A,K):
    C[0..K] = 0
    for j = 1 to n: C[key(A[j])] += 1
    for v = 1 to K: C[v] += C[v-1]
    for j = n downto 1:
        v = key(A[j])
        B[C[v]] = A[j]
        C[v] -= 1
    return B
```

After counting, $C[v]$ is the frequency of key $v$. After prefix accumulation, it counts records with keys $\leq v$: the **last position** of that key's output block. Placement consumes the counters; copy the unconsumed prefix table to $P$ first if range queries will be needed.

## Ending-position placement: trace every write

Take $A = [2_a, 0_b, 2_c, 1_d, 2_e]$ and $K = 2$. Frequencies of keys $0, 1, 2$ are $[1, 1, 3]$, with prefix ending positions $[1, 2, 5]$. Their output blocks are position $1$, position $2$, and positions $3..5$.

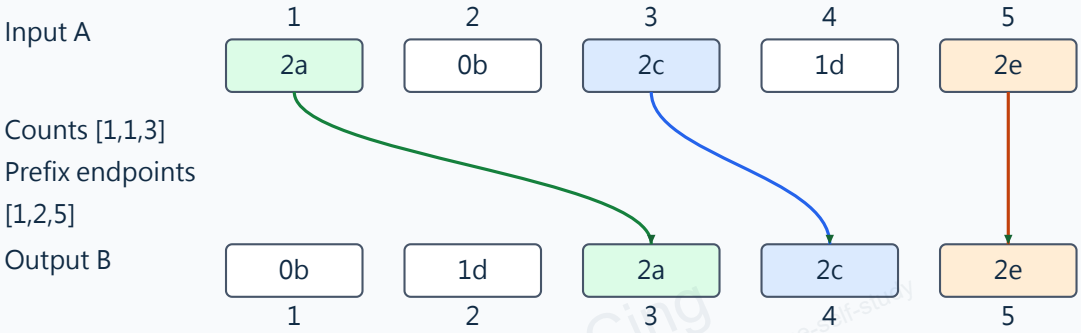
| Scan index $j$ | Record | Position written in $B$ | $C$ after placement |
|---|---|---|---|
| 5 | $2_e$ | 5 | $[1, 2, 4]$ |
| 4 | $1_d$ | 2 | $[1, 1, 4]$ |
| 3 | $2_c$ | 4 | $[1, 1, 3]$ |
| 2 | $0_b$ | 1 | $[0, 1, 3]$ |
| 1 | $2_a$ | 3 | $[0, 1, 2]$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Equal keys retain their identity order

### Counting sort: track destinations of equal-key identities




Process e→d→c→b→a backward; positions are 5→2→4→1→3.

Green a, blue c, orange e all have key 2. Output keeps a→c→e.

Output is $[0_b, 1_d, 2_a, 2_c, 2_e]$. Key-2 identities still occur in order $a, c, e$, establishing stability. Bare keys $[0, 1, 2, 2, 2]$ cannot show whether stability holds.

## Placement invariant, stability, and cost

Fix key $v$ and let $L = P[v-1]$ (use $L = 0$ for $v = 0$), $R = P[v]$. Key-$v$ records in the processed input suffix occupy exactly the right end $[C[v]+1, R]$ of their block. Counter $C[v]$ identifies its rightmost unfilled position.

Another key-$v$ record is still unplaced, so $C[v] > L$. Its write is legal and cannot overwrite a placed record; decrementing the counter preserves the invariant. At termination $C[v] = L$, every block contains exactly its frequency of records. Increasing block order proves sortedness and multiset preservation.

If equal-key record $x$ precedes $y$ in input, reverse scanning places $y$ to the right before placing $x$ to its left. This proves stability. Forward scanning with the same ending-position decrement reverses ties; the next section's starting-position increment is the matching forward rule.

Initialization, counting, prefix accumulation, and placement give $\Theta(n + K + 1)$ time and auxiliary space including output. This is linear when $K = O(n)$ and $n \geq 1$; a huge key domain still incurs table cost for a small input.



## Save prefix counts before consuming positions

For $0 \leq a \leq b \leq K$, subtract prefix counts to count the keys in $[a, b]$:

$$Q(a,b) = \begin{cases} P[b], & a = 0, \\ P[b] - P[a-1], & a > 0. \end{cases}$$

Copy $P$ before placement. For the earlier five records:

| Key $v$ | 0 | 1 | 2 |
|---|---|---|---|
| Frequency $F[v]$ | 1 | 1 | 3 |
| Saved prefix $P[v]$ | 1 | 2 | 5 |
| Consumed counter $C[v]$ | 0 | 1 | 2 |

$Q(1,2) = 5 - 1 = 4$; consumed counters give the wrong answer $2 - 0 = 2$. For $a = 0$, use $Q(0,0) = P[0] = 1$, with no negative index.

Preprocessing: $\Theta(n + K + 1)$. Each query: $O(1)$.

## Stable counting sort can also fill from left to right

The ending-position version scans right to left and decrements its counter. Another stable version starts from each key's **first output position** $S[v]$, scans left to right, and increments that position.

Let $F[v]$ be the frequency of key $v$ in domain $0..K$:

```
S[0] = 1
for v = 1 to K:
    S[v] = S[v-1] + F[v-1]
for j = 1 to n:
    v = key(A[j])
    B[S[v]] = A[j]
    S[v] = S[v] + 1
```

Initially $S[v] = 1 + \sum_{u<v} F[u]$, the first position of that key's block. Empty key blocks may share a start position because no records are written for them.



## Trace all eight records through starting positions

Take $A = [3_a, 1_b, 4_c, 2_d, 6_e, 1_f, 2_g, 2_h]$ and $K = 6$. Frequencies for keys $0..6$ are $[0, 2, 3, 1, 1, 0, 1]$, giving initial $S = [1, 1, 3, 6, 7, 8, 8]$.

| $j$ | Record | Output position | Next position for its key |
|---|---|---|---|
| 1 | $3_a$ | 6 | $S[3] = 7$ |
| 2 | $1_b$ | 1 | $S[1] = 2$ |
| 3 | $4_c$ | 7 | $S[4] = 8$ |
| 4 | $2_d$ | 3 | $S[2] = 4$ |

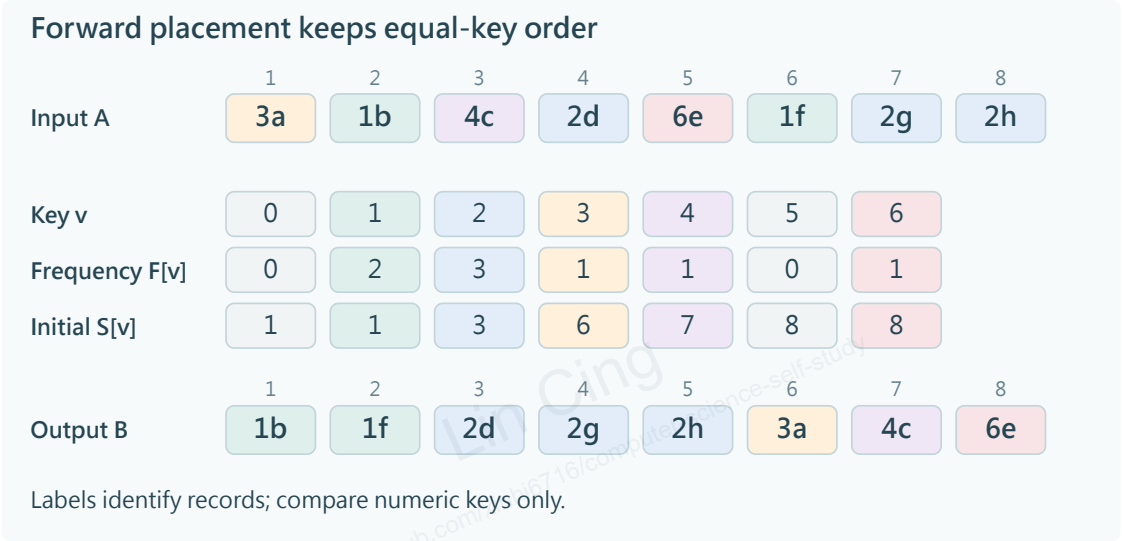
## Finish the eight writes and interpret the final pointers

| $j$ | Record | Output position | Next position for its key |
|---|---|---|---|
| 5 | $6_e$ | 8 | $S[6] = 9$ |
| 6 | $1_f$ | 2 | $S[1] = 3$ |
| 7 | $2_g$ | 4 | $S[2] = 5$ |
| 8 | $2_h$ | 5 | $S[2] = 6$ |

The result is $[1_b, 1_f, 2_d, 2_g, 2_h, 3_a, 4_c, 6_e]$. A final pointer may be one past its key block: it denotes the next unused position, not a position that will necessarily be accessed.



## Equal-key records fill their block in input order




$1_b$ takes position 1 before $1_f$ takes position 2. Records $2_d, 2_g, 2_h$ take positions 3,4,5 in that order. Color connects each equal-key block; identity labels check stability.

## The filled prefix and next-free-position invariant

Let $S_0[v]$ be the unconsumed start and let $c_j(v)$ count key-$v$ records in the already-scanned first $j-1$ inputs. At each iteration's start,

$$S[v] = S_0[v] + c_j(v).$$

Block $[S_0[v], S[v]-1]$ contains exactly the scanned key-$v$ records in input order. Different key blocks do not overlap.

Initially $c_1(v) = 0$ and all filled prefixes are empty. On reading another key-$v$ record, $c_j(v) < F[v]$ because that record is still unplaced; the write is inside its legal block. Writing its next free position and incrementing $S[v]$ preserves records and order, without changing other blocks.

At termination $c_{n+1}(v) = F[v]$, so every key block is full and blocks occur in increasing key order. The result is correct and stable. Counting, start construction, and scanning take $\Theta(n+K+1)$ time and $\Theta(n+K+1)$ auxiliary space including output.

**Pair scan direction with position meaning:** forward/start/increment, or backward/end/decrement. Changing only the scan direction may preserve sorted keys while reversing equal-key identities.



## LSD radix sorting preserves lower-digit order within ties

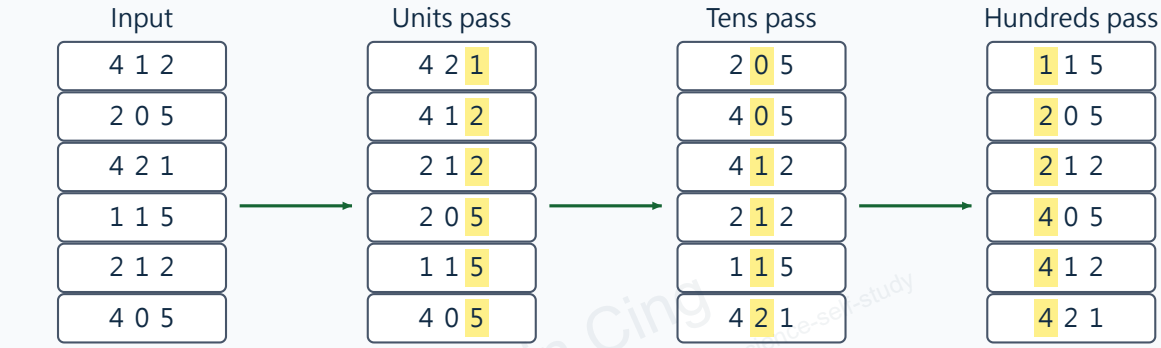
Let every key be a fixed $d \geq 1$-digit nonnegative integer in base $r \geq 2$, padded with leading zeros. LSD processes digit positions $1, 2, \ldots, d$ from least to most significant, using a **stable** sort at every pass.

Invariant: after pass $t$, the sequence is sorted by its lowest $t$ digits. The $t = 0$ condition is empty. Pass $t$ groups records by that digit. It determines order between different groups, while stability retains the previous $t-1$-digit order inside a group. Thus the lowest $t$ digits are sorted; $t = d$ gives complete order.

Lower-digit order is retained **within an equal higher-digit group**, not globally across all records. For $[12, 11]$, units sorting gives $[11, 12]$. Tens digits are both $1$; an unstable reversal would incorrectly restore $[12, 11]$.

Sorting the entire sequence by hundreds, tens, then units makes units the final primary key. MSD must recurse within equal higher-digit groups; it is not LSD with its loop simply reversed.

# Three radix passes with every full sequence




Yellow marks the current digit. Equal current digits retain the previous column order.

In the hundreds pass, $205$ precedes $212$ because their shared hundreds group retains lower-two-digit order $05 < 12$. Each pass inspects the current digit; the invariant builds full-key order progressively.



# Verify every radix pass

| Digits processed | Complete sequence |
|---|---|
| None | $[412, 205, 421, 115, 212, 405]$ |
| Units | $[421, 412, 212, 205, 115, 405]$ |
| Units and tens | $[205, 405, 412, 212, 115, 421]$ |
| All three | $[115, 205, 212, 405, 412, 421]$ |

In the tens pass, $205, 405$ both have tens digit $0$ and retain their previous order. Records $412, 212, 115$ all have tens digit $1$ and also retain their prior order; their lowest two digits are $12, 12, 15$.

## Account jointly for bit length, radix, and passes

Stable counting sort takes $\Theta(n + r)$ per pass, giving $\Theta(d(n + r))$ over $d$ passes and $\Theta(n + r)$ reusable auxiliary space. Linear time follows directly only for fixed $d$ and $r = O(n)$.

For $b \geq 1$-bit keys and $s$ bits per pass, $1 \leq s \leq b$,

$$r = 2^s, \qquad d = \lceil b/s \rceil, \qquad T = \Theta(\lceil b/s \rceil (n + 2^s)) .$$

Pad a short highest group with zeros. This assumes constant-time key/digit access; integers longer than a machine word require explicit extraction and movement costs.

For $n \geq 2$, choose $s = \min(b, \lfloor \lg n \rfloor)$, so $2^s \leq n$ and time is $O(n\lceil b/s \rceil)$. When $b = O(\lg n)$ this is $O(n)$.

Under a cost of $n + r$ per pass, 32-bit keys with $s = 8$ cost $4(n + 256)$, while $s = 16$ costs $2(n + 65536)$. The latter is smaller exactly when $n > 65024$, with equality at that threshold. Actual allocation, cache, and bandwidth costs lie outside this specified model.



## Distribution is a shared technique, not one algorithm

| Method | Distribution rule | What follows distribution |
|---|---|---|
| LSD radix sort | Digits from least to most significant | Stable full-sequence sort at each pass |
| MSD radix sort | Start with the most significant digit | Recurse on the next digit inside each equal-prefix group |
| Interval bucket sort | Map $x \in [0, 1)$ to $\lfloor nx \rfloor$ | Fully sort each bucket, then concatenate buckets |

All can use buckets, but their **digits, recursion rules, and cost assumptions differ**. Ten digit buckets are not the same as $n$ equal-width intervals. Fully sorting a bucket is not the same procedure as recursively distributing its next digit.

Grouping $[312, 305, 121, 118, 214, 207]$ by hundreds yields $[121, 118]$, $[214, 207]$, $[312, 305]$. This is not yet sorted. MSD processes lower digits within each group; interval bucket sorting uses its specified local sorting algorithm. Concatenating the unsorted groups is incorrect.

# Interval bucket sorting distributes, sorts, and concatenates

For $n \geq 1$ keys $x \in [0, 1)$, create $n$ buckets $B[0..n-1]$ and append each key to $B[\lfloor nx \rfloor]$ in input order. Insertion-sort each bucket, then concatenate by bucket index. Empty input returns immediately. Key $x = 1$ is outside the domain and cannot index $B[n]$.

Correctness needs no distribution assumption: $x \leq y$ implies $\lfloor nx \rfloor \leq \lfloor ny \rfloor$. Different buckets are ordered by disjoint half-open intervals; local sorting orders records within each bucket. Input-order appending plus stable insertion sort also guarantees stability; head insertion may change equal-key order first.

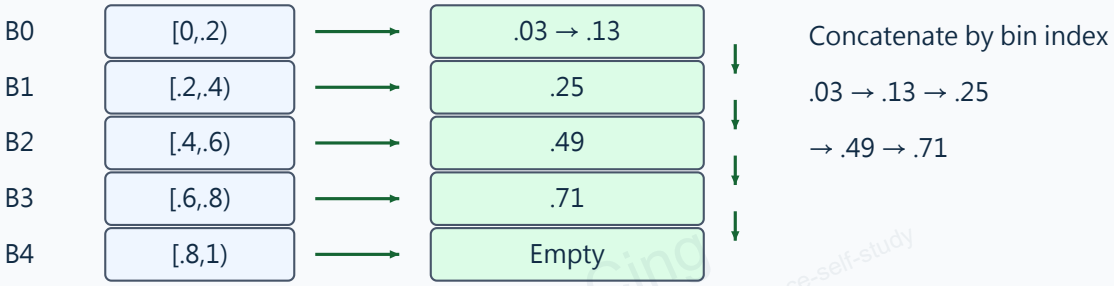
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Five buckets: interval, local order, and output

## Bucket intervals guarantee cross-bin order

Input .13,.25,.03,.71,.49 → bins 0,1,0,3,2




Blue: half-open bin intervals. Green: sorted contents. No further cross-bin comparisons.

For $n = 5$, input $[.13, .25, .03, .71, .49]$ maps to bucket indices $[0, 1, 0, 3, 2]$. Bucket $0$ becomes $[.03, .13]$; buckets $1, 2, 3$ contain $.25, .49, .71$ respectively; bucket $4$ is empty. Output is $[.03, .13, .25, .49, .71]$.

## Expected linear time uses squared bucket sizes

Let $N_i$ be bucket $i$'s size. Initialization, distribution, and concatenation cost $\Theta(n)$; insertion sorting costs at most a constant times $\sum_i N_i^2$. Thus

$$T \le c_0 n + c_1 \sum_{i=0}^{n-1} N_i^2.$$

Now introduce the input assumption: keys are **mutually independent and uniform** on $[0, 1)$. Indicator $X_{ij}$ records whether input $j$ enters bucket $i$. Then $N_i = \sum_j X_{ij}$, $X_{ij}^2 = X_{ij}$, and

$$\begin{aligned}\mathbb{E}[N_i^2] &= \sum_j \mathbb{E}[X_{ij}] + \sum_{j \ne k} \mathbb{E}[X_{ij} X_{ik}] \\ &= n\frac{1}{n} + n(n-1)\frac{1}{n^2} = 2 - \frac{1}{n}.\end{aligned}$$

Hence $\mathbb{E}[\sum_i N_i^2] = 2n - 1$, giving $\mathbb{E}[T] = O(n)$. Reading the input supplies $\Omega(n)$, so expected time is $\Theta(n)$.

Independence is used for the destinations of different records $j, k$. Bucket sizes are dependent because they sum to $n$, but expectation linearity still applies. Knowing $\mathbb{E}[N_i] = 1$ does not imply $\mathbb{E}[N_i^2] = 1$.



## Concentration, dependence, and a genuine worst-case input

If records remain independent but bucket probabilities are $p_i$, with $\sum_i p_i = 1$, the same expansion gives

$$\mathbb{E}\left[\sum_i N_i^2\right] = n + n(n-1)\sum_i p_i^2.$$

The expected upper bound remains linear when $\sum_i p_i^2 = O(1/n)$. Uniform probabilities give $1/n$; concentrating all probability in one bucket makes the squared-size sum $n^2$.

Uniform marginals alone do not suffice. Choose uniform $J \in \{0, \dots, n-1\}$ and set every $x_j = (J + 1/2)/n$. Every record has uniform bucket marginals, yet all enter one bucket, so $\sum_i N_i^2 = n^2$. This refutes a squared-cost estimate; equal-key insertion sorting itself may still be linear.

For an actual quadratic-time family, take $n \ge 2$ and $x_j = (n - j + 1)/(2n^2)$. These keys are distinct, strictly decreasing, and all in bucket $0$. Insertion sort performs $n(n-1)/2$ shifts, establishing worst-case $\Theta(n^2)$ time.

Replacing each local sort by a worst-case $O(m \lg(m+1))$ method gives total worst-case $O(n \lg(n+1))$. Independent uniform inputs retain linear expected time because $m \lg(m+1) = O(m^2)$.

## Order statistics and extrema: specify the target

Inputs contain $n \geq 1$ totally ordered keys. The $k$th smallest value occupies position $k$ in the sorted multiset, for $1 \leq k \leq n$. Repeated occurrences take separate positions but can yield the same answer: ranks $1, 2$ of $[2, 5, 2, 8]$ both return $2$.

Our lower median has rank $\lceil n/2 \rceil$. For even $n$, it is the smaller central value, unlike the statistical convention that averages the two central values. Reject empty inputs and invalid ranks at entry.

A minimum scan initializes from the first record and compares each remaining key with the current minimum. Its invariant is that the current value is the minimum of the processed prefix. It uses $n - 1$ key comparisons; maximum is symmetric.

Distinct-key inputs give a matching comparison lower bound: every nonminimum record must be shown larger than something before it can be eliminated. A comparison can eliminate at most one new minimum candidate. Thus at least $n - 1$ comparisons are needed. Using distinct keys for a worst-case lower bound does not restrict the algorithm to distinct-key inputs.



## Paired extrema: eight records, ten comparisons

Compare each pair first: only its smaller record challenges the minimum, its larger the maximum. Initialize from the first pair; later pairs preserve both extrema of the processed prefix.

Input: $[14, 3, 9, 1, 8, 5, 12, 6]$.

| Pair | Smaller, larger | Minimum | Maximum | Comparisons |
|---|---|---|---|---|
| $14, 3$ | $3, 14$ | 3 | 14 | 1 |
| $9, 1$ | $1, 9$ | 1 | 14 | 4 |
| $8, 5$ | $5, 8$ | 1 | 14 | 7 |
| $12, 6$ | $6, 12$ | 1 | 14 | 10 |

Even $n$: $1 + 3(n - 2)/2$. Odd $n$: initialize from one record, then use $3(n - 1)/2$. Both give $\lceil 3n/2 \rceil - 2$ comparisons (zero for $n = 1$). Ties preserve correct extremal values.

## Why paired extrema achieve the worst-case lower bound

For distinct keys, every record initially qualifies as both a minimum and a maximum candidate. Only one qualification of each type survives, so $2n - 2$ qualifications must be eliminated.

An adversary answers comparisons as follows. Let $U$ mean untested, $H$ maximum-only, $L$ minimum-only, and $M$ neither.

| Compared states | Answer and new eliminations |
|---|---|
| $U, U$ | Choose a larger and a smaller key; eliminate 2 qualifications |
| $H, H$ or $L, L$ | Eliminate one maximum or minimum candidate |
| $H, L$ | Answer $H > L$; eliminate none |
| $U$ versus a classified key | Against $H$, answer $U < H$; against $L$, answer $U > L$; against $M$, choose either consistent direction. Eliminate one $U$ qualification |
| Involving $M$, with no $U$ | Maintain $H > M > L$; for $M, M$ use a linear extension of the existing order. At most one new elimination |

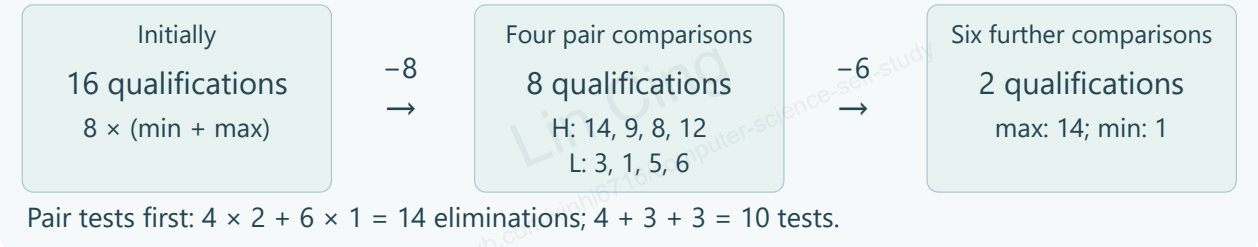


## Count the eliminated candidate qualifications

The answers stay consistent: $H$ keys have no known larger key, $L$ keys no known smaller key, and $U$ keys no constraints. Place $H$ above $M$ above $L$, extending the comparison order within each class; no cycle is introduced.

Only $U, U$ removes two qualifications, at most $\lfloor n/2 \rfloor$ times. Other comparisons remove at most one. Thus

$$2n - 2 \le c + \lfloor n/2 \rfloor, \qquad c \ge \lceil 3n/2 \rceil - 2.$$




Pair tests first: 4 × 2 + 6 × 1 = 14 eliminations; 4 + 3 + 3 = 10 tests.

## A tournament preserves the second-minimum candidates

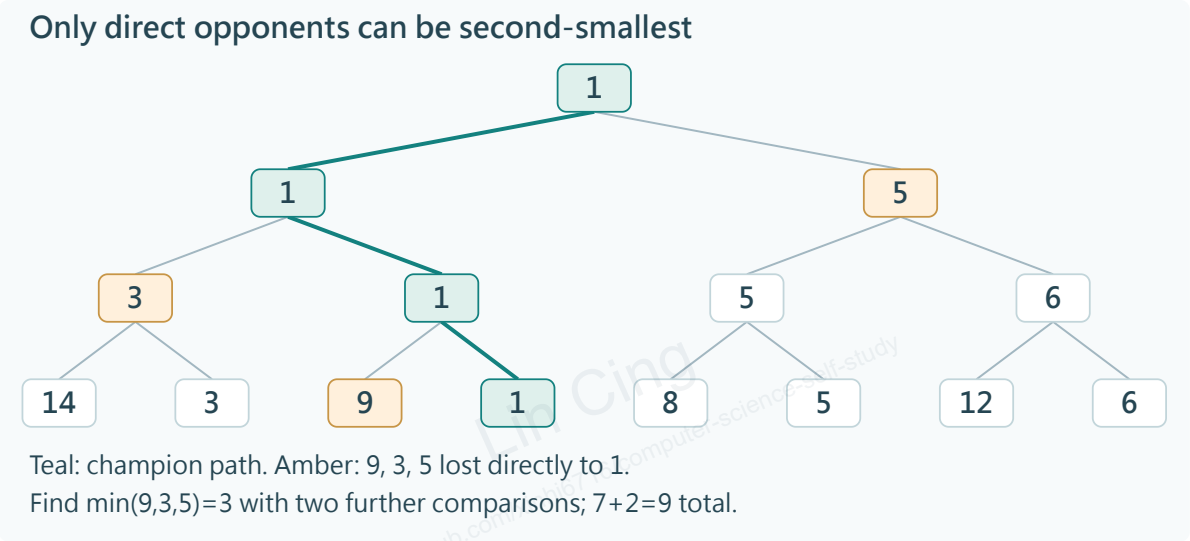
Advance the smaller key and retain each winner's directly defeated opponents. The second-smallest key must lose directly to the minimum: losing to a different key would put two keys below it. Assume distinct keys initially.

For $n \geq 2$, a balanced bracket can use byes rather than fictional keys. Eliminating one record per comparison costs $n - 1$. The champion faces $d \leq \lceil \lg n \rceil$ actual opponents; searching that list uses $d - 1$ more. The worst-case upper bound is $n + \lceil \lg n \rceil - 2$. With duplicate keys, break ties by $(key, identity)$; the result then means the second record, not necessarily the second distinct value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Eight keys: whom did the champion defeat?




Champion $1$ directly defeated $9, 3, 5$, so the second minimum is their minimum, $3$. The tournament uses $7$ comparisons and the opponent list uses $2$, totaling $9$.

## Three-way selection removes the whole equal-key region

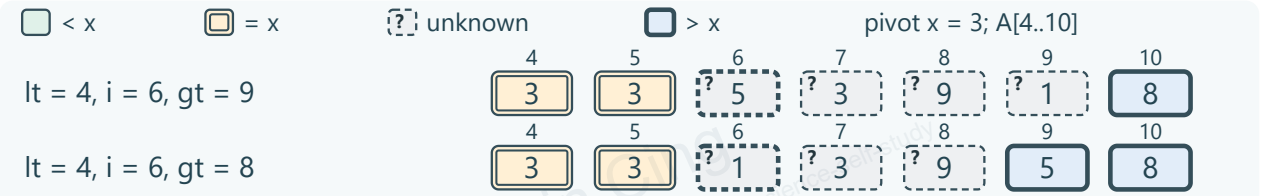
Sorting processes both sides; selection retains only the side containing the answer. Array indices use **one-based closed intervals** $A[p..r]$. Target $k$ is the local rank in this subarray, with $1 \le k \le r - p + 1$. Compare keys and exchange whole records; $x$ is a key present in the subarray.

```
PARTITION3(A,p,r,x):
    lt = p; i = p; gt = r
    while i <= gt:
        if key(A[i]) < x:
            swap(A[lt],A[i]); lt += 1; i += 1
        elif key(A[i]) > x:
            swap(A[i],A[gt]); gt -= 1
        else:
            i += 1
    return (lt,gt)
```



## Three-way partition: invariant and progress

At loop entry, $A[p..lt-1] < x$, $A[lt..i-1] = x$, $A[i..gt]$ is unknown, and $A[gt+1..r] > x$; empty ranges are permitted. A smaller key extends the less-than and equal regions; an equal key extends the equal region. A larger key swaps in an unclassified right-end record, so $i$ must stay. Each iteration decreases unknown length by one, and swaps preserve the multiset. Termination removes the unknown region.




Swap A[6] = 5 with A[9] = 1; gt becomes 8, but i stays 6.
Next classify the new A[6] = 1. Since p = lt = 4, the < x region is empty.

## Randomized selection changes the local rank only on the right

```
RANDOM-SELECT(A,p,r,k):
    require 1 <= k <= r-p+1
    while p < r:
        x = key(A[uniform-integer(p,r)])
        (lt,gt) = PARTITION3(A,p,r,x)
        L = lt-p; E = gt-lt+1
        if k <= L: r = lt-1
        elif k <= L+E: return x
        else: k -= L+E; p = gt+1
    return key(A[p])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



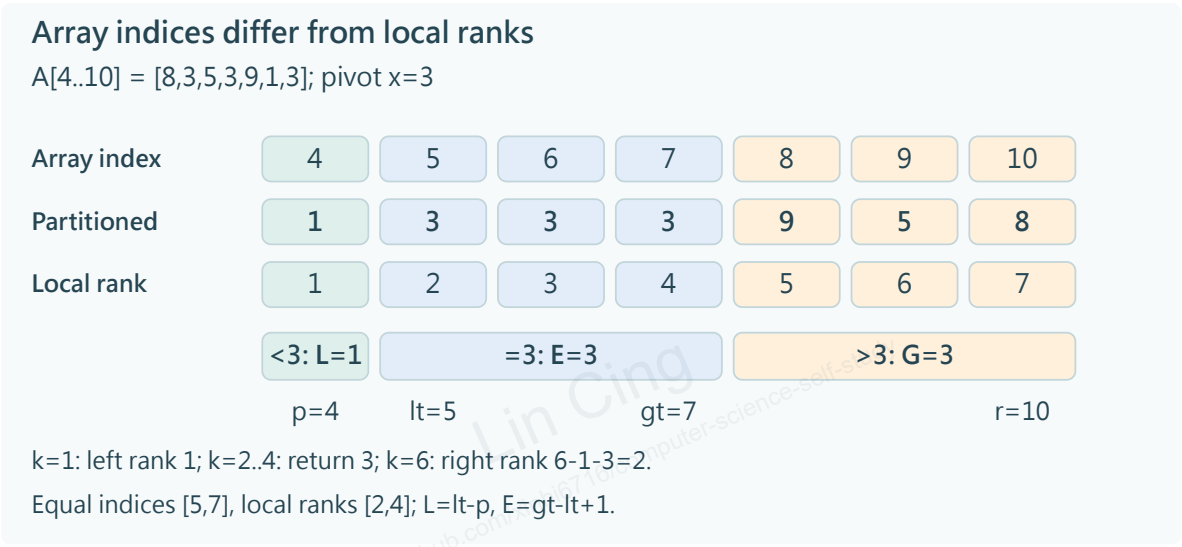
## Selection: preserve the requested value and shrink

Because $x$ is drawn from current records, $E \geq 1$. Local ranks $L+1..L+E$ all return $x$. A left move preserves $k$; a right move subtracts the $L$ smaller and $E$ equal occurrences.

Loop invariant: the current subarray's $k$th smallest key equals the original requested key. The left branch has $1 \leq k \leq L$; the right branch has $1 \leq k - L - E \leq r - gt$, so neither retains an empty range. Every nonreturning step reduces size by at least $E$. A singleton returns its sole key, proving correctness and termination even with duplicates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Separate local ranks, region lengths, and absolute indices




For $A[4..10] = [8, 3, 5, 3, 9, 1, 3]$ with $x = 3$, partition returns $lt = 5, gt = 7$, hence $L = 1, E = 3, G = 3$. Equal-region local ranks are $2..4$.



# Convert each target using region lengths

| Original local rank | Test | Next state or answer |
|---|---|---|
| 1 | $1 \le L$ | Rank 1 in $A[4..4]$ |
| 4 | $L < 4 \le L + E$ | Return 3 |
| 6 | $6 > L + E$ | Rank $6 - 1 - 3 = 2$ in $A[8..10]$ |

Comparing $k$ directly with $lt$ mixes origins: $k$ starts at the current subarray, whereas $lt$ starts at the full array's index origin. An alternative absolute target $t = p + k - 1$ stays unchanged through iterations; do not also decrement a local rank under that convention.

## Randomized selection: strong induction over all states

Conditioned on the current array, choose the pivot uniformly among its $n$ records. Future pivots are freshly uniform within their then-current subarrays. For distinct keys, local pivot rank $R$ is uniform on $1..n$. Whatever the target rank, the next size is at most

$$m_R = \max(R-1, n-R).$$

Let $F(A,k)$ be the expected remaining cost from that state. Bound current scanning and overhead by $cn$. Assume $F(B,j) \le a|B|$ for **every shorter state**. Conditioning on the chosen pivot and applying induction gives

$$F(A,k) \le cn + \frac{a}{n}\sum_{r=1}^{n} \max(r-1, n-r).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Average the larger-side sizes explicitly

For $n = 2h$, larger-side sizes $h, \dots, 2h-1$ each occur twice: the sum is $2\sum_{t=h}^{2h-1} t = 3h^2 - h$. For $n = 2h+1$, the middle rank contributes $h$ and all other sizes occur in symmetric pairs: the sum is $h + 2\sum_{t=h+1}^{2h} t = 3h^2 + 2h$. Dividing by $n$ gives at most $3n/4$, so

$$F(A,k) \le (c + 3a/4)n \le an \qquad (a \ge 4c).$$

Choose $a$ large enough for the $n = 1$ base case. This proves $O(n)$; the initial partition scans $n > 1$ records, giving expected $\Theta(n)$ time.

No monotonicity of an unknown cost function or product of dependent expectations was assumed. The $3n/4$ bound concerns the expected larger side, not a deterministic upper bound on every partition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Duplicates preserve expected linear time with three-way selection

For analysis only, break equal-key ties by a fixed identity order to assign unique ranks $R \in \{1, \ldots, n\}$. Actual partitioning still compares keys, not identities. Uniformly sampling a record makes this analytical rank uniform.

If the answer is in $< x$, its region has length at most $R - 1$; if in $> x$, at most $n - R$; an equal-region answer returns immediately. Every pivot identity therefore admits the same bound $m_R = \max(R - 1, n - R)$. The preceding strong induction over all smaller states applies, yielding expected $\Theta(n)$ time.

Worst-case execution remains $\Theta(n^2)$: with distinct keys, seek the minimum while choosing the current maximum every time. Scan sizes are $n, n - 1, \ldots, 2$, a possible positive-probability path. Expected and worst-case bounds differ. The iterative version with in-place three-way partition needs $O(1)$ extra space and changes its input.

All-equal input returns after one scan. Switching back to a two-way Lomuto partition that sends every equal key to one side loses this guarantee, even with random pivots.



## Deterministic selection: align code with the worked trace

All lists and sorted results below are **one-based**. SORT-COPY sorts a copy by key. STABLE-SPLIT-COPY scans the input into three fresh lists for keys less than, equal to, and greater than $x$, retaining their input order within each region. This trace-friendly version uses $O(n)$ extra space; it is not an in-place implementation.

Groups contain at most five records; take each lower median. With $g = \lceil n/5 \rceil$ medians, recursively select rank $\lceil g/2 \rceil$ as the pivot.

```
MEDIAN-PIVOT(A):
    M = empty list
    for each consecutive group G of at most 5 records in A:
        B = SORT-COPY(G)
        append key(B[ceil(length(B)/2)]) to M
    return DETERMINISTIC-SELECT(M,ceil(length(M)/2))
```

## Deterministic selection: return or recurse on one region

```
DETERMINISTIC-SELECT(A,k):
    n = length(A); require 1 <= k <= n
    if n <= 5: return key(SORT-COPY(A)[k])
    x = MEDIAN-PIVOT(A)
    (L,E,G) = STABLE-SPLIT-COPY(A,x)
    if k <= length(L):
        return DETERMINISTIC-SELECT(L,k)
    elif k <= length(L)+length(E): return x
    else:
        return DETERMINISTIC-SELECT(G,k-length(L)-length(E))
```

Correctness follows from region order and induction on size. Pivot recursion has size $g < n$; answer recursion has size at most $n - 1$, since $x$ is present and the equal region is nonempty. Both terminate, and both must be charged in the time analysis.



## Groups of five certify exclusions, not an exact median

### Median-of-medians: three witnesses per full group

One group per column, larger above. Middle row: group medians. Pivot x=12.

| | G1 | G2 | G3 | G4 | G5 |
|---|---|---|---|---|---|
| | 25 | 24 | ▲ 23 | ▲ 22 | ▲ 21 |
| | 17 | 18 | ▲ 19 | ▲ 20 | ▲ 16 |
| Median→ | ▼ 4 | ▼ 8 | ◆ 12 | ▲ 14 | ▲ 15 |
| | ▼ 2 | ▼ 5 | ▼ 9 | 10 | 13 |
| | ▼ 1 | ▼ 3 | ▼ 6 | 7 | 11 |

Medians [4,8,12,14,15] → third smallest 12. The overall median is 13.

▼ + ◆: 9 values ≤12 ▲ + ◆: 9 values ≥12 ◆: shared pivot

These $25$ records contain exactly keys $1..25$. Group medians are $[4, 8, 12, 14, 15]$, giving pivot $x = 12$, while the true lower median is rank $13$, namely $13$.

## List the actual witnesses on both sides

For the three groups whose medians are $\leq 12$, their smallest triples are $[1, 2, 4]$, $[3, 5, 8]$, and $[6, 9, 12]$: nine certified values $\leq 12$. For the three groups with medians $\geq 12$, their largest triples are $[12, 19, 23]$, $[14, 20, 22]$, and $[15, 16, 21]$: nine values $\geq 12$.

Overlap at $12$ is harmless. A strict greater-than recursion excludes all $\leq x$; a strict less-than recursion excludes all $\geq x$. These are alternative branches, not simultaneous claims about disjoint witnesses. With duplicates, the non-strict certificates still hold and three-way partitioning removes the entire equal region.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rounding and incomplete groups: finish the linear-time induction

Among $g = \lceil n/5 \rceil$ group medians, their lower median $x$ has at least $\lceil g/2 \rceil$ medians on each non-strict side. Conservatively discard the possible incomplete group and the group containing the selected pivot identity. At least $\lceil g/2 \rceil - 2$ complete groups remain on either side.

Each supplies three witnesses, so each non-strict side has at least

$$3(\lceil g/2 \rceil - 2) \geq 3n/10 - 6$$

records. A strict recursive side therefore has size at most $7n/10 + 6$. A negative small-input lower bound is merely uninformative; the actual nonempty equal region still ensures strict shrinkage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pay for both recursive calls in the induction

Let nonrecursive work be at most $cn$. Inductively bound every shorter state by $a$ times its length. For $n \geq 140$,

$$\begin{aligned} T(n) &\leq a\lceil n/5 \rceil + a(7n/10 + 6) + cn \\ &\leq (9a/10 + c)n + 7a \leq an. \end{aligned}$$

Choose $a \geq 20c$, so $a/10 - c \geq a/20$ and $n \geq 140$ gives $an/20 \geq 7a$. Increase $a$ to cover all finite sizes $1 \leq n < 140$; this proof threshold does not change the code's size-five base case. Initial grouping costs $\Omega(n)$, establishing worst-case $\Theta(n)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rank sixteen: trace groups, region sizes, and the next rank

Concatenate these five groups as the input, without first sorting the whole collection:

$[1, 2, 4, 17, 25]$, $[3, 5, 8, 18, 24]$, $[6, 9, 12, 19, 23]$, $[7, 10, 14, 20, 22]$, $[11, 13, 15, 16, 21]$.

| Current $(n, k)$ | Group medians $M$ and pivot $x$ | $(\lvert L\rvert, \lvert E\rvert, \lvert G\rvert)$ | Next step |
|---|---|---|---|
| $(25, 16)$ | $[4, 8, 12, 14, 15] \to 12$ | $(11, 1, 13)$ | Greater side, rank $4$ |
| $(13, 4)$ | $[19, 20, 16] \to 19$ | $(6, 1, 6)$ | Less side, rank $4$ |
| $(6, 4)$ | $[15, 16] \to 15$ | $(2, 1, 3)$ | Greater side, rank $1$ |
| $(3, 1)$ | Base sort $[16, 17, 18]$ | No partition | Return $16$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Reconstruct the next groups from the retained list

The first greater side retains scan order: $[17, 25, 18, 24, 19, 23, 14, 20, 22, 13, 15, 16, 21]$. Groups of five, five, and three have sorted copies $[17, 18, 19, 24, 25]$, $[13, 14, 20, 22, 23]$, and $[15, 16, 21]$. Their medians are $19, 20, 16$; a size-three base call selects $19$.

The next less side is $[17, 18, 14, 13, 15, 16]$. Its first five have median $15$; the singleton contributes $16$, so their lower median is $15$. The greater side $[17, 18, 16]$ returns rank one, $16$. This trace follows the stable-list code. In-place swaps may change region order and subsequent groups, so they need not produce the same intermediate path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Changing group size changes the proof obligations

For fixed odd group size $s = 2t + 1$, each full group median certifies $t + 1$ records on a non-strict side. Ignoring rounding and additive constants, the two recursive-size fractions sum to

$$\frac{1}{s} + 1 - \frac{t+1}{2s} = 1 + \frac{1-t}{2s}.$$

Groups of five give $9/10 < 1$, and seven give $6/7 < 1$, leaving room to pay linear work. Incomplete groups and finite base cases still require justification. Groups of three give sum one and only the coarse bound

$$T(n) \le T(\lceil n/3 \rceil) + T(2n/3 + O(1)) + O(n).$$

This proves $O(n \lg n)$, not an $\Omega(n \lg n)$ lower bound for the actual algorithm. A lower bound needs additional evidence that such bad splits persist across levels.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Account for copied lists and the active call chain

The stable-list code uses $O(n)$ extra space: each call retains $O(n)$ copied records and executes its two recursive calls sequentially. For sufficiently large sizes, every active call chain shrinks by a fixed ratio below one, giving a geometric $O(n)$ sum; the finite small-size tail is constant. Worst-case call depth is $O(\lg n)$. In-place partitioning alone does not establish $O(1)$ total space: medians, copies, and the recursive stack also count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Birthday collisions: build the no-collision complement

Model $k \geq 0$ people as independent uniform choices among $n \geq 1$ dates. This simplified model is not an assertion about real birthday distributions.

Conditioned on $j$ distinct dates so far, the next person has $n - j$ noncolliding choices, with conditional probability $(n - j)/n$. The chain rule gives, for $k \leq n$,

$$\Pr(\text{all dates distinct}) = \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right),$$

$$\Pr(\text{at least one collision}) = 1 - \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right).$$

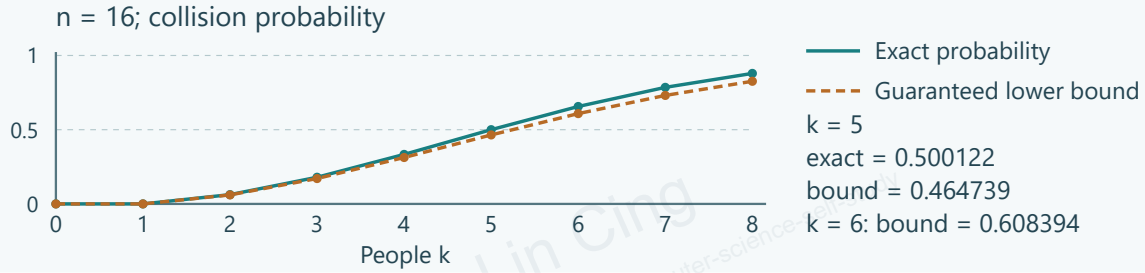
For $k = 0, 1$ there is no collision; the empty product is $1$. For $k > n$, the pigeonhole principle gives probability $1$ directly. Do not extend the product into negative factors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A sufficient threshold can be conservative

For $2 \le k \le n$, apply $1 - x \le e^{-x}$ to each factor and take the complement:

$$\Pr(\text{collision}) \ge 1 - \exp\left(-\sum_{j=0}^{k-1} \frac{j}{n}\right) = 1 - e^{-k(k-1)/(2n)}.$$




The bound reaches $1/2$ when $k(k-1) \ge 2n \ln 2$: a **sufficient condition**, not an exact threshold. For $n = 16$, the true probability first exceeds $1/2$ at $k = 5$; the bound certifies it only at $k = 6$.



## Expected collision pairs are not collision probability

Let $I_{ij}$ indicate that people $i, j$ share a date. Independence and uniformity give

$$\Pr(I_{ij} = 1) = \sum_{d=1}^{n} \Pr(B_i = d, B_j = d) = n\frac{1}{n^2} = \frac{1}{n}.$$

The collision-pair count is $X = \sum_{i<j} I_{ij}$, hence

$$\mathbb{E}[X] = \frac{\binom{k}{2}}{n}.$$

A collision is event $X \ge 1$. Markov only yields $\Pr(X \ge 1) \le \mathbb{E}[X]$, not a reverse implication. For example, let $X = M^2$ with probability $1/M$ and zero otherwise. Its mean is $M$, but its positive-value probability is only $1/M$.

Under the birthday model, $k = o(\sqrt{n})$ makes collision probability vanish by this upper bound. For $k \ge C\sqrt{n}$ with sufficiently large fixed $C$, the exponential complement bound gives a fixed positive collision probability. Both directions establish the $\sqrt{n}$ transition scale.

## Sixteen dates: multiply every conditional probability

Take $n = 16, k = 5$ and accumulate no-collision probabilities person by person.

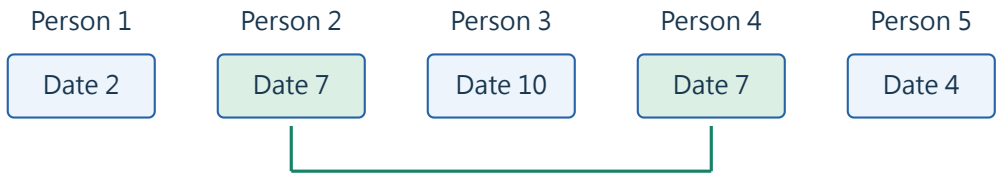
| Person added | Conditional noncollision probability | Probability all dates remain distinct |
|---|---|---|
| 1 | $1$ | $1$ |
| 2 | $15/16$ | $15/16$ |
| 3 | $14/16$ | $105/128$ |
| 4 | $13/16$ | $1365/2048$ |
| 5 | $12/16$ | $4095/8192$ |

Collision probability is $4097/8192 \approx 0.500122$, whereas expected collision pairs are $\binom{5}{2}/16 = 5/8$. At $k = 5$, the exponential complement bound $e^{-10/16} > 1/2$ cannot prove a probability above one half. At $k = 6$, $e^{-15/16} < 1/2$ can. A sufficient condition can be conservative without contradicting the exact result.



## A collision in one sample is not a model average

One date sample and its model: distinguish counts from expectations




Only colliding pair in this sample: (2,4)

Model: 16 dates, 5 people. P(collision)=4097/8192≈0.500122.

E[colliding pairs]=10/16=5/8, distinct from the event probability above.

The illustrated sample is $[2, 7, 10, 7, 4]$. Only people $2, 4$ collide, so $X = 1$ and $I_{\{X \geq 1\}} = 1$. These are values in one outcome, unlike averages $5/8$ and $4097/8192$ over all outcomes.

## Pairwise independence does not justify a joint product

Let $U, V$ be independent uniform elements of $\{0, 1, 2\}$ and define

$$B_1 = U, \qquad B_2 = V, \qquad B_3 = (2U - V) \bmod 3.$$

Every pair of dates uniquely determines $(U, V)$. The first pair is immediate; $B_1, B_3$ give $V = 2B_1 - B_3$, and $B_2, B_3$ give $U = 2(B_2 + B_3)$, modulo $3$. Hence every date pair is uniform over nine ordered pairs and is independent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Enumerate all nine equally likely outcomes

| $U$ | $V$ | $(B_1, B_2, B_3)$ | Collision pairs $X$ |
|---|---|---|---|
| 0 | 0 | $(0, 0, 0)$ | 3 |
| 0 | 1 | $(0, 1, 2)$ | 0 |
| 0 | 2 | $(0, 2, 1)$ | 0 |
| 1 | 0 | $(1, 0, 2)$ | 0 |
| 1 | 1 | $(1, 1, 1)$ | 3 |
| 1 | 2 | $(1, 2, 0)$ | 0 |
| 2 | 0 | $(2, 0, 1)$ | 0 |
| 2 | 1 | $(2, 1, 0)$ | 0 |
| 2 | 2 | $(2, 2, 2)$ | 3 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Equal means can hide different collision probabilities

| Model | $\Pr(X=0)$ | $\Pr(X=1)$ | $\Pr(X=3)$ | $\Pr(X>0)$ | $\mathbb{E}[X]$ |
|---|---|---|---|---|---|
| Pairwise independent construction | $2/3$ | $0$ | $1/3$ | $1/3$ | $1$ |
| Three fully independent dates | $2/9$ | $2/3$ | $1/9$ | $7/9$ | $1$ |

With full independence, the $27$ equally likely triples include $6$ with all dates distinct, $3$ with all dates equal, and $3 \cdot 3 \cdot 2 = 18$ with exactly one equal pair: choose the pair, its date, then the different date. Exactly two equal pairs are impossible: equality is transitive.

Here collision probability is $3/9 = 1/3$. Three fully independent dates give $1 - 6/27 = 7/9$, yet both models have expected collision-pair count $1$. Equal expectations do not determine distributions; the exact complement product needs stronger joint assumptions.



## Balls into bins: fixed toss count, random loads

Toss $t \geq 0$ balls independently and uniformly into $b \geq 1$ bins. A fixed bin's load $N$ is the sum of $t$ indicators, so $\mathbb{E}[N] = t/b$. Choosing which $r$ tosses hit that bin gives

$$\Pr(N = r) = \binom{t}{r}(1/b)^r(1-1/b)^{t-r}, \quad 0 \leq r \leq t.$$

Let $Z_0, Z_1$ count empty bins and singleton bins. Applying indicators to each bin gives

$$\mathbb{E}[Z_0] = b(1-1/b)^t,$$

$$\mathbb{E}[Z_1] = b\left[t\frac{1}{b}(1-1/b)^{t-1}\right] = t(1-1/b)^{t-1}, \quad t \geq 1.$$

For $t = 0$, directly take $Z_0 = b, Z_1 = 0$. For $b = 1$, evaluate the events directly or interpret boundary formulas with empty-product convention $0^0 = 1$.

For $t = b = 4$, a fixed bin has mean load $1$, yet $\mathbb{E}[Z_0] = 81/64$ and $\mathbb{E}[Z_1] = 27/16$. Loads sum to four and are dependent. Mean one does not mean exactly one ball in every bin.

## Collecting every bin: wait for one new bin at each stage

Tosses are independent and uniform. For a fixed bin, let $T$ count tosses through its first hit, including the successful toss. Since $\Pr(T > t) = (1 - 1/b)^t$ for $t \geq 0$, tail summation gives

$$\mathbb{E}[T] = \sum_{t=0}^{\infty} (1 - 1/b)^t = b.$$

Four bins; wait for the first hit on bin 3

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 2 | 2 | 4 | 1 | 4 | 3 |
| miss | miss | miss | miss | miss | hit |

T = 6

E[T] = 4

P(T = 6) = $(3/4)^5 \times (1/4) = 243/4096$

Five misses, then one hit; a single waiting time need not equal its mean.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Add the waiting times for successive new bins

For complete collection, after $i$ distinct bins have appeared, new-bin probability is $(b - i)/b$. Conditioned on the currently seen set, the waiting count $T_i$ has expectation $b/(b - i)$. Future tosses remain independent and uniform; elapsed time does not change this value.

Total stopping time is $C = \sum_{i=0}^{b-1} T_i$, so linearity gives

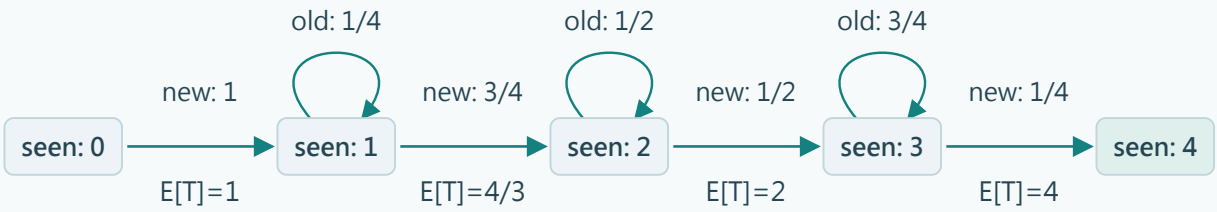
$$\mathbb{E}[C] = \sum_{i=0}^{b-1} \frac{b}{b - i} = bH_b.$$

This is $\Theta(b \ln b)$ for $b \geq 2$; for $b = 1$ it is exactly one toss. Adding expectations does not require a separate proof of independence between stages.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Four bins: repeated hits stay in the same state

## A new bin advances; an old bin repeats the state




Tosses 2,2,4,1,4,3: new bins at tosses 1,3,4,6; actual stage lengths 1,2,1,2.

Stage means sum to 1+4/3+2+4=25/3; a particular run may stop at toss 6.

Four bins have stage means $1, 4/3, 2, 4$, summing to $25/3$. The illustrated tosses are $2, 2, 4, 1, 4, 3$, discovering new bins at tosses $1, 3, 4, 6$. Actual stage lengths $1, 2, 1, 2$ sum to six. A run's stopping time need not equal its expectation.



# First collision versus complete collection: different stopping times

First bound complete-collection time $C$. After $t$ tosses, a designated bin is empty with probability $(1 - 1/b)^t \le e^{-t/b}$. Incomplete collection means at least one empty bin, so

$$\Pr(C > t) \le \min\{1, be^{-t/b}\}.$$

For integer $t \ge b(\ln b + c)$ and $c \ge 0$, failure probability is at most $e^{-c}$. Empty-bin events may be dependent; the union bound is sufficient.

## Same tosses; the stopping rule changes the answer

| t = 1 | t = 2 | t = 3 | t = 4 | t = 5 | t = 6 |
|---|---|---|---|---|---|
| 2 | 2 | 4 | 1 | 4 | 3 |
| {2} | {2} | {2,4} | {1,2,4} | {1,2,4} | {1,2,3,4} |

D = 2: first repeat

C = 6: all four bins seen

Sets show distinct bins seen so far; compare D with C, not two different samples.

## First-collision time has square-root expectation

Let $D$ be the toss index of the first repeated bin. Event $D > k$ means the first $k$ tosses are distinct. For $k > b$ its probability is zero; otherwise use the birthday complement. Since $k(k-1) \geq k^2/2$ for $k \geq 2$,

$$\begin{aligned}\mathbb{E}[D] &= \sum_{k \geq 0} \Pr(D > k) \\ &\leq 2 + \sum_{k \geq 2} e^{-k^2/(4b)} \\ &\leq 2 + \int_0^\infty e^{-x^2/(4b)} dx = 2 + \sqrt{\pi b}.\end{aligned}$$

For a lower bound, integer $k \leq \sqrt{b}/2$ has collision probability at most $k(k-1)/(2b) \leq 1/8$. Thus the first $\Theta(\sqrt{b})$ tail terms are at least $7/8$, giving $\mathbb{E}[D] = \Omega(\sqrt{b})$.

Consequently $\mathbb{E}[D] = \Theta(\sqrt{b})$, whereas $\mathbb{E}[C] = \Theta(b \ln b)$. The same balls-and-bins model has different answers for first repetition and complete coverage.



## Heads streaks: window count and longest length differ

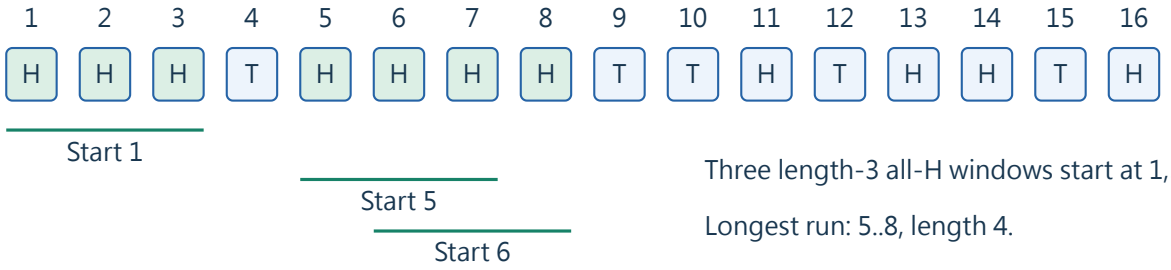
Flip a fair coin independently $n$ times. Let $L$ be the longest consecutive-heads length, with $L = 0$ when no head occurs. For $1 \leq k \leq n$, event $A_{i,k}$ means positions $i$ through $i + k - 1$ are all heads; valid starts satisfy $1 \leq i \leq n - k + 1$. Thus

$$X_k = \sum_{i=1}^{n-k+1} \mathbf{1}_{A_{i,k}}, \qquad \mathbb{E}[X_k] = (n - k + 1)2^{-k}.$$

Overlapping windows need not be independent. For example, $\Pr(A_{1,3} \cap A_{2,3}) = 2^{-4} = 1/16$ because only four heads are needed, whereas the product of marginals is $2^{-6} = 1/64$. Linearity of expectation remains valid; exact union probabilities require accounting for overlaps.

## Count windows without confusing them with runs

Four consecutive Hs contain two dependent length-3 windows




Three length-3 all-H windows start at 1,5,6.

Longest run: 5..8, length 4.

Overlapping windows can both occur; starts 5 and 6 share two flips.

The illustrated length-three windows start at $1, 5, 6$, giving $X_3 = 3$. The heads run at positions $5..8$ contributes two of these windows. Its longest length is $L = 4$, distinct from both the window count and the number of heads runs.



## Upper bounds: from streak tails to expected length

Event $\{L \geq k\}$ is the union of all all-heads windows of length $k$. For $1 \leq k \leq n$, the union bound gives

$$\Pr(L \geq k) \leq \min\{1, (n - k + 1)2^{-k}\} \leq \min\{1, n2^{-k}\}.$$

For $n \geq 1$, put $k_0 = \lceil \lg n \rceil$. Integer tail sums and a geometric series yield

$$\begin{aligned} \mathbb{E}[L] &= \sum_{k=1}^{n} \Pr(L \geq k) \\ &\leq k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \leq \lceil \lg n \rceil + 1. \end{aligned}$$

Here $\lg$ denotes base-two logarithm. For $k > n$ the event is impossible; for $n = 0$, set $L = 0$ directly without taking a logarithm.

For $r \geq 1$, choosing integer threshold $k = \lceil r \lg n \rceil$ gives $\Pr(L \geq k) \leq n^{1-r}$. At $n = 1000, k = 20$, the sharper bound is $981/2^{20} < 10^{-3}$; at $k = 30$, even $1000/2^{30} < 10^{-6}$ suffices. These are probability guarantees for an unusually long streak in one run.

## Lower bound: disjoint blocks supply independence

For sufficiently large $n$, let $s = \lfloor (\lg n)/2 \rfloor \geq 1$ and $m = \lfloor n/s \rfloor$. Partition the first $ms$ flips into $m$ blocks of length $s$, discarding the incomplete final block. Each block is all heads with probability $2^{-s} \geq n^{-1/2}$. Disjoint blocks use independent flips, so their success events are mutually independent.

If $L < s$, every block must fail. The converse need not hold: a long run can cross a block boundary. Consequently,

$$\Pr(L < s) \leq (1 - 2^{-s})^m \leq e^{-m2^{-s}} \leq e^{-m/\sqrt{n}} \longrightarrow 0.$$

Since $m = \Theta(n/\lg n)$, we have $m/\sqrt{n} \to \infty$. Nonnegativity then gives

$$\mathbb{E}[L] \geq s \Pr(L \geq s) = \Omega(\lg n).$$

Combining with the upper bound proves $\mathbb{E}[L] = \Theta(\lg n)$. Many overlapping windows in expectation alone would not justify the high-probability existence step; independence of the selected blocks does.

For $n = 16, s = 2, m = 8$, at least one designated block is HH with probability $1 - (3/4)^8 = 58975/65536$. Hence $\mathbb{E}[L] \geq 117950/65536$, a checkable lower bound rather than an exact mean.



## Online selection: specify information and decisions

There are $n \geq 2$ candidates with distinct abilities, arriving in a uniformly random permutation. Interviews permit comparisons only with candidates already seen; rejected candidates cannot be recalled, and at most one hire is allowed. The objective is to select the globally best candidate, rather than minimize interview cost.

Choose integer $1 \leq k < n$. Reject the first $k$ candidates while recording their highest score $M$, then accept the first later score exceeding $M$. If none does, accept the last candidate.

The second loop need not update $M$: before the first threshold crossing every observed score is at most $M$, and a crossing returns immediately. The final fallback accepts the current last candidate, without recalling anyone. Handle $n = 1$ by selecting its sole candidate; define $k = 0$ separately as selecting the first candidate.

## Online selection: code and stopping invariant

```
ONLINE-SELECT(n,k):
    require n >= 2 and 1 <= k < n
    M = -infinity
    for i = 1 to k:
        M = max(M, score(i))      // interview, then reject
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

Before inspecting each later candidate, no earlier candidate has crossed the sample maximum $M$. A crossing ends the search immediately; if none occurs, the fallback accepts the last candidate.



## Success probability: condition on the best arrival

Condition on the globally best candidate arriving at position $i$. If $i \le k$, rejection during observation makes success impossible. If $i > k$, success occurs exactly when the best of the first $i-1$ candidates lies among the first $k$: otherwise a later candidate would already have crossed $M$, while if it does, candidate $i$ is the first crossing.

The best position is uniform, with probability $1/n$. Conditional on that position, the first $i-1$ candidates still have a uniform relative order. Their maximum lies among the first $k$ with conditional probability $k/(i-1)$. Summing disjoint position events gives

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

This uses conditional probabilities, without assuming independence of the two maximum-position events.

Define boundaries separately: for $n = 1$ success is certain; for $k = 0$ choosing the first candidate succeeds with probability $1/n$. The harmonic sum formula applies only to $k \ge 1$.

## Optimal threshold: finite differences before limits

Put $q_k = \sum_{j=k}^{n-1} 1/j$, with empty sum $q_n = 0$. Since $q_k = 1/k + q_{k+1}$, for $1 \le k \le n-2$,

$$P_{k+1} - P_k = \frac{(k+1)q_{k+1} - kq_k}{n} = \frac{q_{k+1} - 1}{n}.$$

The strictly decreasing sequence $q_k$ makes success unimodal. A threshold satisfying $q_k \ge 1 \ge q_{k+1}$ maximizes it; equality can produce adjacent tied optima. For $n = 2$, the sole positive threshold $k = 1$ ties $k = 0$ at success $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why the optimal success rate tends to 1/e

Integral bounds for decreasing $1/x$ yield

$$\ln(n/k) \le q_k \le \ln(n/k) + 1/k.$$

Thus $f(k/n) \le P_k \le f(k/n) + 1/n$, where $f(x) = x\ln(1/x)$. Derivatives $f'(x) = \ln(1/x) - 1$ and $f''(x) = -1/x < 0$ give maximum $1/e$ at $x = 1/e$.

Choose the nearest legal integer to $n/e$, making $k/n$ differ from $1/e$ by $O(1/n)$. Continuous differentiability gives $f(k/n) = 1/e - O(1/n)$, while every threshold satisfies $P_k \le 1/e + 1/n$. Optimal success therefore converges to $1/e$. At finite $n$, use the exact difference rather than treating $n/e$ as an exact integer solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Six candidates: verify thresholds and individual decisions

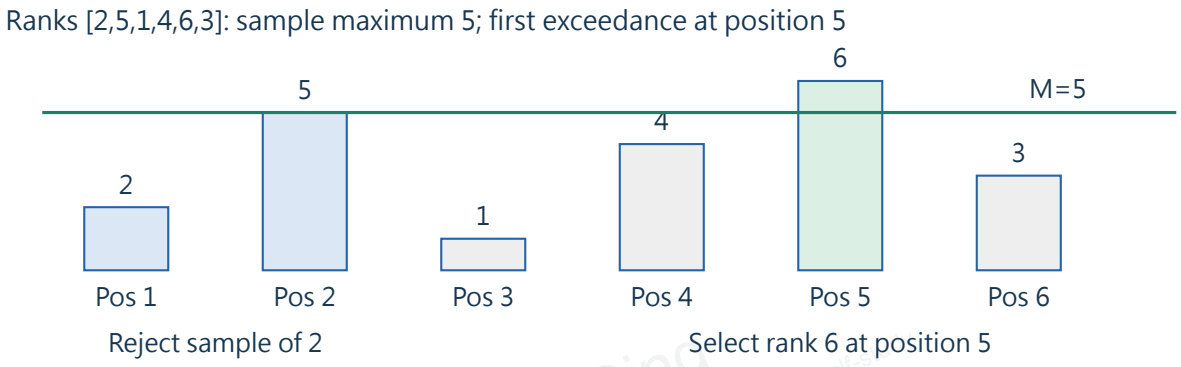
For $n = 6$, substitute each threshold into $P_k = (k/6)q_k$:

| $k$ | $q_k$ | Exact success probability $P_k$ |
|---|---|---|
| 1 | $137/60$ | $137/360$ |
| 2 | $77/60$ | $77/180$ |
| 3 | $47/60$ | $47/120$ |
| 4 | $9/20$ | $3/10$ |
| 5 | $1/5$ | $1/6$ |

Since $q_2 > 1 > q_3$, the best positive threshold is $k = 2$; threshold $0$ succeeds with probability only $1/6$.



## Follow the threshold through every candidate




For ranks $[2, 5, 1, 4, 6, 3]$, reject the first two and set $M = 5$. Reject candidate three because $1 \le 5$, and four because $4 \le 5$. Candidate five has $6 > 5$ and is accepted immediately; candidate six need not be interviewed. For failure example $[5, 6, 1, 2, 3, 4]$, the best candidate is rejected during observation, nobody later exceeds $6$, and the fallback accepts rank $4$.

## Random recursive splitting: conditioning and leaf conservation

Consider a separate cost model. A call of size $n \geq 2$ prints $n$ stars, chooses $p$ uniformly from $\{1, \ldots, n-1\}$, and processes sizes $p, n-p$. A size-one call prints one exclamation mark and stops. Every draw is uniform conditional on prior history.

Let $L_n, S_n$ denote expected exclamation marks and stars, with $L_1 = 1, S_1 = 0$. Condition on $p$, add the two expected costs, then average over $p$. Symmetry under $j \leftrightarrow n-j$ gives

$$L_n = \frac{2}{n-1} \sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1} \sum_{j=1}^{n-1} S_j.$$

Linearity needs no independence between the two subtree costs. Every branch strictly decreases its positive integer size, ensuring termination.

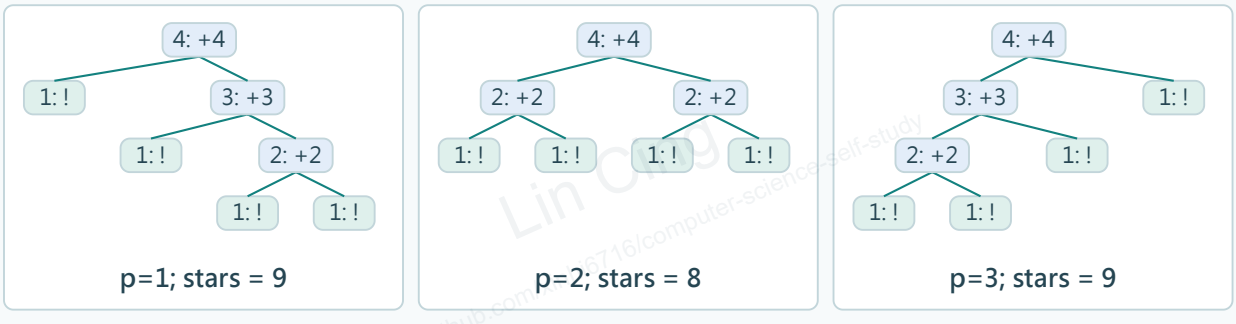
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Four items: the same leaf count, different work

Splitting conserves total size and all final leaves have size one, so **every execution has exactly $n$ leaves**, not merely $n$ in expectation. Star count varies. At $n = 4$, a balanced split costs $4 + 2 + 2 = 8$. A $1 + 3$ split costs $4 + (3 + 2) = 9$. Averaging the three equally likely root splits gives $S_4 = (9 + 8 + 9)/3 = 26/3$.

Three equally likely root splits; four leaves in every tree




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Solving split cost: subtract adjacent sizes

Strong induction verifies the leaf mean: if $L_j = j$ for $j < n$, then $L_n = \frac{2}{n-1}\sum_{j<n} j = n$.

For stars, clear the denominators:

$$(n-1)S_n = n(n-1) + 2\sum_{j<n} S_j,$$

$$nS_{n+1} = n(n+1) + 2\sum_{j<n} S_j + 2S_n.$$

Subtract to obtain $nS_{n+1} - (n-1)S_n = 2n + 2S_n$, hence $S_{n+1} = 2 + \frac{n+1}{n}S_n$. The subtraction applies for $n \geq 2$; at $n = 1$, check the difference directly from $S_1 = 0, S_2 = 2$.

With $Q_n = S_n/n$, we have $Q_{n+1} = Q_n + 2/(n+1)$. Sum from $Q_1 = 0$ to get

$$Q_n = 2(H_n - 1), \qquad S_n = 2n(H_n - 1) = \Theta(n \ln n).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



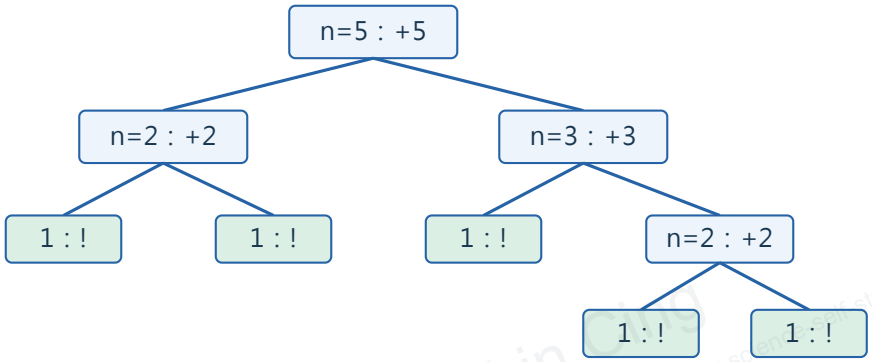
## Verify the closed form on small inputs

| $n$ | Leaves in every execution | $\mathbb{E}[\text{stars}]$ |
|---|---|---|
| 2 | 2 | 2 |
| 3 | 3 | 5 |
| 4 | 4 | $26/3$ |
| 5 | 5 | $77/6$ |
| 6 | 6 | $87/5$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Expected cost and worst execution are different

Five unit leaves: leaf count is conserved; star cost varies




Leaves here=5

Stars here=12

E[stars]=77/6

+number: stars at this call

!: one exclamation mark

Shown: p=2, then 3=1+2. All splits give 5 leaves; total stars depend on splits.

The worst execution splits off one each time, printing $2 + 3 + \cdots + n = n(n+1)/2 - 1$ stars. The expected linearithmic bound is not a bound on every execution.



## References and attribution

Reference textbook: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition, Chapters 5–9. Supplementary notes: MIT 6.046J/18.401J, Fall 2005. Lecture 4: Quicksort (Charles E. Leiserson, 2005-09-21); Lecture 5: Sorting Lower Bounds / Linear-Time Sorting (Erik Demaine, 2005-09-26); Lecture 6: Order Statistics (Erik Demaine, 2005-09-28).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

# Data Structures and Amortized Analysis

A data structure turns abstract operations into maintained states. First specify legal states and operation guarantees; then ask how much work an entire operation sequence requires.

Read **representations and contracts → amortized analysis → hashing → search trees and augmentation → B-trees → Fibonacci heaps → integer universes**.

Prerequisites are specifications and invariants from Chapter 1, recurrence analysis from Chapter 2, and probability and heaps from Chapter 3. Two stacks and a resizable array introduce the cost tools needed later.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# The same operation name can hide different contracts

An **abstract data type (ADT)** specifies observable behavior; arrays, links, and trees implement it. Complexity depends on the contract.

| Operation | Question to settle first |
|---|---|
| DELETE | Is the argument a key, an array index, or an existing node handle? |
| SEARCH | Are duplicate keys allowed? Which record is returned? |
| POP / DEQUEUE | Is an empty call rejected, given a special result, or forbidden? |
| Pointer update | Must node identity and external handles remain valid? |

Deleting a known doubly linked-list node can take constant time. Finding a node by key and then deleting it also incurs search cost. Every later bound uses its own argument and representation contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Circular queue: physical position is not logical order

Fix capacity $C \geq 1$ and use array $A[0..C-1]$, front index $h$, and size $s$. A valid state satisfies $0 \leq h < C$ and $0 \leq s \leq C$. Logical item $j$ is

$$Q[j] = A[(h + j) \bmod C], \qquad 0 \leq j < s.$$

**Empty means $s = 0$; full means $s = C$.** Do not infer occupancy from equal indices. The next insertion position is $(h + s) \bmod C$, writable only when not full. All physical slots are usable.

Old values left in the array are not necessarily queue members. Clearing a removed slot may release resources, but $h, s$ determine membership. Fixed-capacity ENQUEUE and DEQUEUE take worst-case $O(1)$, excluding allocation and nonconstant element-copy costs.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Check boundaries before changing queue state

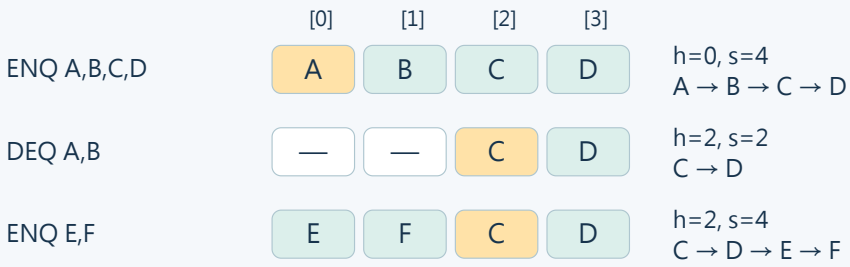
```
ENQUEUE(x):
    if s == C: return overflow
    A[(h+s) mod C] = x
    s = s + 1

DEQUEUE():
    if s == 0: return underflow
    x = A[h]
    h = (h+1) mod C
    s = s - 1
    return x
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Assume successful element reads/writes and sequential execution. A rejected call changes neither metadata nor live records. Distinguish error status from a returned data value. This is not a concurrent lock-free queue; sharing it requires synchronization.

## After wrapping, dequeue still returns C, D, E, F

### Circular queue: physical slots and logical order




Amber = front; — = not live, even if old bits remain

Enqueue A–D, dequeue A and B, then enqueue E and F. Storage is [E,F,C,D], but $h = 2, s = 4$: read $2 \to 3 \to 0 \to 1$. Reject G; the next-insertion index equals h because the queue is full.



## Why circular indexing preserves FIFO

Before insertion, $s < C$. Position $(h + s) \bmod C$ differs from every live position $(h + j) \bmod C$, $0 \le j < s$: equality would imply $C \mid (s - j)$, contradicting $1 \le s - j < C$. Thus insertion overwrites no live record and appends exactly at the logical rear.

Dequeue returns $A[h] = Q[0]$. With $h' = (h + 1) \bmod C$ and $s' = s - 1$, each $0 \le j < s'$ satisfies

$$A[(h' + j) \bmod C] = A[(h + j + 1) \bmod C] = Q[j + 1].$$

The remaining sequence is precisely the old sequence without its first item. The empty initial state is valid; successful calls preserve the invariant and failed calls preserve state. Induction covers every legal call sequence.

## Another representation: reserve one slot instead of size

CLRS § 10.1 uses head for the first item and tail for the next insertion slot, reserving one slot. In zero-based notation, empty is $h = t$ and full is $(t + 1) \bmod C = h$, so usable capacity is $C - 1$.

Another convention places $f$ immediately before the first item and $r$ at the last item. Empty is $f = r$ and full is $(r + 1) \bmod C = f$. Convert by $h = (f + 1) \bmod C$ and $t = (r + 1) \bmod C$. Renaming variables alone does not preserve read/write order.

This chapter uses $h, s$ and all C slots. The derived next-insertion index is $t = (h + s) \bmod C$; $t = h$ can mean empty or full, distinguished by $s$. A reserved slot, a count, and a full flag are different designs with different tests.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Singly linked insertion: preserve the old successor first

Given a valid node $p$, insert a detached new node $x$ after it. Node identity is fixed and keys may repeat. Node $x$ must not already belong to the list.

```
q = p.next
x.next = q
p.next = x
```

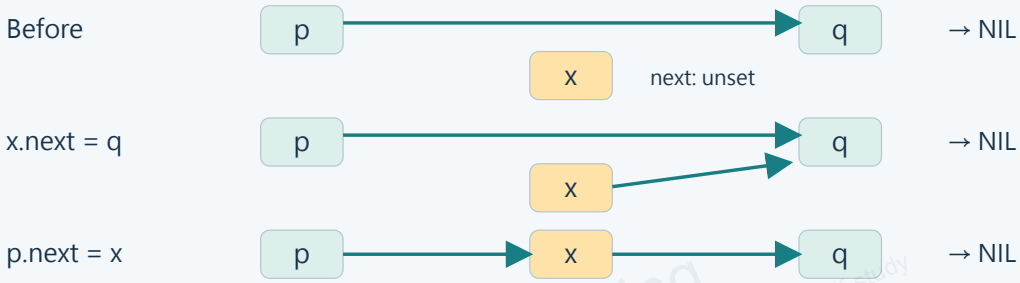
The path $p \to q$ becomes $p \to x \to q$; all other next links stay unchanged. The same code handles q=NIL. If a tail pointer is maintained and p was the tail, update tail=x.

Writing `p.next=x` before `x.next=p.next` creates a self-loop at x and can lose reachability to q. The $O(1)$ bound covers rewiring with known p, excluding search, allocation, and nonconstant data-copy costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three states: connecting a new node to the old path

### Initialize the new node before changing the old path




Connect x.next to q before connecting p.next to x. In the intermediate state, x is not yet a member reachable from head. Node labels denote identity rather than key values.



## Singly linked deletion: a node is not its predecessor

To delete the node after p, save `x=p.next`; reject x=NIL. Otherwise save `q=x.next`, then set `p.next=q`. If x was the tail, set tail=p. Reclaim x last. Given x alone without its predecessor, deleting a tail generally requires searching from head, taking worst-case $\Theta(n)$.

Copying the successor's data into x and deleting that successor changes which node owns which record and fails at the tail. It does not implement deletion with the node-identity contract used here.

## In-place reversal preserves access to the untouched suffix

Reverse an acyclic, NIL-terminated singly linked list:

```
prev = NIL; cur = head
while cur != NIL:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
head = prev
```

If a tail is stored, save old_head first and set tail=old_head at the end. During the loop, prev is the reversed processed prefix and cur begins the untouched suffix. Saving nxt preserves access after rewiring. Each iteration consumes one node: $\Theta(n)$ time and $O(1)$ extra space.



## Circular doubly linked list: a sentinel joins the endpoints

Allocate a permanent dummy node z, the sentinel, excluded from the data count. Empty means `z.next=z.prev=z`; otherwise z.next is the first item and z.prev the last.

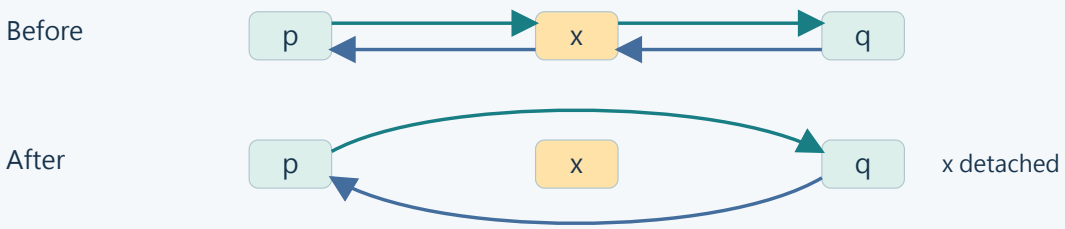
For every node v in the ring, including z, maintain `v.next.prev=v` and `v.prev.next=v`. Require a valid member handle and forbid deleting z. Validating an arbitrary foreign handle is not included in constant-time rewiring of a known member.

```
DELETE(x):                // x != z, valid member
    p = x.prev; q = x.next
    p.next = q
    q.prev = p
```

Either neighbor may be z, so the code handles first, last, and sole data nodes. Reclaim x only after rewiring; external handles to x then become invalid. These are sequential operations, not intermediate states safe for unsynchronized concurrent readers.

## Doubly linked deletion changes two neighbor links

**Delete x: green next, blue prev**




Replace p.next=x and q.prev=x with p.next=q and q.prev=p. Other members keep their adjacencies. Old fields inside x do not make it a member after removal.

For a sole data item, p=q=z. The two writes restore z.next=z and z.prev=z, exactly the empty state. Two pointer writes do not mean deletion by an arbitrary key takes two steps: finding that key can still scan the entire list.



## Doubly linked insertion must establish four links

Insert a detached x after a known p; p may be z. First save q=p.next:

```
x.prev = p
x.next = q
q.prev = x
p.next = x
```

The result is p↔x↔q, with all other links unchanged. For an empty list p=q=z, both z.next and z.prev become x. If p was last, q=z and z.prev automatically becomes the new last node.

Each new adjacency has its matching backlink, restoring the invariant. This order establishes correctness after a sequential call, not concurrent publication safety. A sentinel simplifies boundaries at the cost of one extra node per list; it does not improve search asymptotically.

## Array indices as node handles: location and order differ

Parallel arrays key, next, and prev represent nodes without native pointers. Three fields at the same index belong to one object. Reserve index 0 for NIL and use indices 1–5 for objects.

| Index | State | key | next | prev |
|---|---|---|---|---|
| 1 | free | — | 4 | — |
| 2 | live | B | 3 | 5 |
| 3 | live | C | 0 | 2 |
| 4 | free | — | 0 | — |
| 5 | live | A | 2 | 0 |

With head=5, follow $5 \to 2 \to 3 \to 0$, giving A→B→C. With free=1, unused slots follow $1 \to 4 \to 0$. Adjacent physical indices need not be logical neighbors.



## Allocation and reclamation keep live and free lists disjoint

Pool slots have three disjoint ownership classes: live, free, and caller-owned. ALLOCATE transfers a free slot to the caller; initialization and linking make it live. The free list uses only next and behaves as a stack.

```
ALLOCATE():
    if free == 0: return exhausted
    x = free; free = next[x]
    return x                    // initialize before linking

FREE(x):                        // detached, not already free
    next[x] = free
    free = x
```

Both operations change constantly many fields and take $O(1)$ in a fixed pool. Growing the pool and allocating variable-length records are outside this bound.

## Object-pool trace: allocation through reclamation

Use the same five slots; head remains 5:

| Completed operation | Live chain (NIL=0) | Free chain | Caller-owned |
|---|---|---|---|
| Initial | 5(A)→2(B)→3(C)→0 | 1→4→0 | none |
| ALLOCATE returns 1 | 5→2→3→0 | 4→0 | 1 |
| Append D after C | 5→2→3→1(D)→0 | 4→0 | none |
| Unlink and FREE(2) | 5→3→1→0 | 2→4→0 | none |

Append: key[1]=D, prev[1]=3, next[1]=0, next[3]=1. Remove B: set next[5]=3 and prev[3]=5, then next[2]=4 and free=2.

Double-free corrupts the free list; a stale handle can refer to an unrelated reused object. An in-range index does not establish a valid lifetime.



## How two stacks preserve FIFO

Use $S$ for incoming records and $T$ for outgoing records. Read each sequence in the stated direction:

$$Q = \mathrm{reverse}(T_{\mathrm{bottom}\to\mathrm{top}}) \parallel S_{\mathrm{bottom}\to\mathrm{top}}.$$

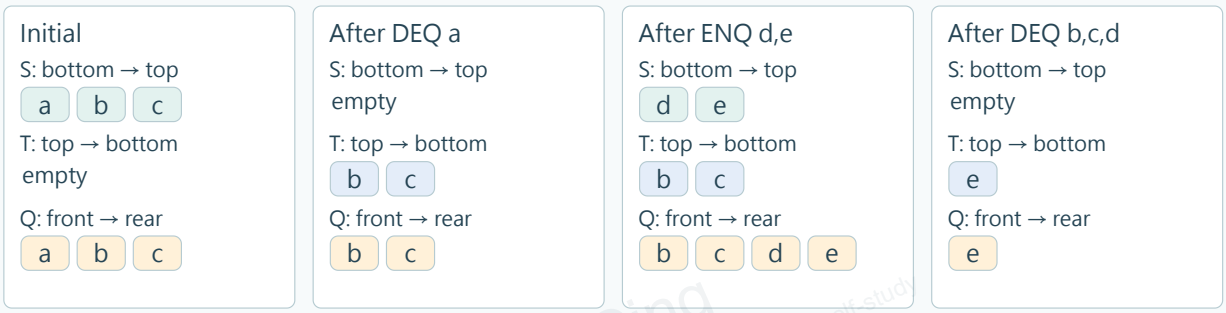
Here $\parallel$ denotes concatenation. **The top of $T$ is the queue front.** Transfer all of $S$ to $T$ only when $T$ is empty.

ENQUEUE pushes onto $S$, appending to $Q$. If $T$ is nonempty, DEQUEUE pops its top, removing the first record of $Q$. If $T$ is empty, transferring reverses $S$, making its oldest record the top of $T$ while preserving queue order.

If both are empty, return underflow without changing state. Transferring before every dequeue would put new records ahead of older records still waiting in $T$.

## Follow the order before counting the expensive transfer

One queue, two stacks: older records leave first




Read $S$ bottom to top and $T$ top to bottom. Enqueuing $d, e$ leaves $b, c$ waiting in $T$. Transfer $S$ only after both $b, c$ have left; then dequeue $d$.



## One expensive call does not make every call expensive

Count one stack PUSH or POP as one unit, temporarily omitting fixed queue-call overhead. ENQUEUE costs 1. A DEQUEUE transferring $k$ records costs $2k + 1$: pop and push each transferred record, then pop $T$.

Starting empty, each record can be **pushed onto $S$ → popped from $S$ → pushed onto $T$ → popped from $T$** with each of these four primitives occurring at most once for that record. Thus $e$ enqueues and $d$ successful dequeues cost at most $3e + d$.

Any legal sequence of $m$ operations has $e + d \leq m$, giving at most $3m$ primitive operations. Restoring fixed per-call overhead preserves $O(m)$. No distribution over sequences is assumed.

A single DEQUEUE can still cost $\Theta(n)$. A sequence guarantee is not a bound on each call's latency.

## Potential: attach deferred work to the state

Operation $i$ changes $D_{i-1}$ into $D_i$ at actual cost $c_i$. Choose a state function $\Phi$ and define

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

A positive $\Delta\Phi$ stores credit for future work; a negative change releases previously stored credit. Amortized cost is not a second measurement of execution time.

Summing a prefix of $m$ operations cancels intermediate potentials:

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

If all reachable states satisfy $\Phi(D_i) \geq \Phi(D_0)$, total amortized cost bounds total actual cost. A common sufficient condition is $\Phi(D_0) = 0$ and $\Phi \geq 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Initial state and negative amortized charges matter

If only $\Phi \geq 0$ is known, without $\Phi(D_0) = 0$, the correct conclusion is

$$\sum c_i \leq \sum \hat{c}_i + \Phi(D_0).$$

A queue starting with $n$ records in $S$ may transfer all of them on its first dequeue. Either charge for constructing that initial state or retain an $O(n)$ initial term.

An individual $\hat{c}_i$ may be negative. The telescoping identity remains valid under the endpoint condition: the operation releases more credit than the actual work it performs.

An **amortized** bound controls total cost over legal sequences. An **expected** bound needs a probability model. An expected amortized bound can combine both; it must state both quantifiers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Queue potential counts records not yet transferred

Let $\Phi(S, T) = 2|S|$. Both stacks start empty, so $\Phi_0 = 0$.

| Operation | Actual $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| ENQUEUE | 1 | 2 | 3 |
| DEQUEUE with nonempty $T$ | 1 | 0 | 1 |
| Transfer $k \geq 1$ records, then dequeue | $2k + 1$ | $-2k$ | 1 |
| Report underflow on empty queue | 0 | 0 | 0 |

The last row counts stack primitives only. Including queue-call overhead adds a fixed constant to each row. Records in $T$ have already paid their transfer cost.

Every prefix has total cost at most $3e + d$, agreeing with the aggregate argument that each record incurs at most four stack primitives.



## A complete ledger for interleaved operations

Both stacks run **bottom to top**; $\Phi = 2|S|$. Both totals are 19 because endpoint potentials are zero.

| Operation | $S$ | $T$ | $c$ | $\Phi$ | $\hat{c}$ |
|---|---|---|---|---|---|
| ENQ a | a | empty | 1 | 2 | 3 |
| ENQ b | a,b | empty | 1 | 4 | 3 |
| ENQ c | a,b,c | empty | 1 | 6 | 3 |
| DEQ → a | empty | c,b | 7 | 0 | 1 |
| ENQ d | d | c,b | 1 | 2 | 3 |
| ENQ e | d,e | c,b | 1 | 4 | 3 |
| DEQ → b | d,e | c | 1 | 4 | 1 |
| DEQ → c | d,e | empty | 1 | 4 | 1 |
| DEQ → d | empty | e | 5 | 0 | 1 |

## Accounting: every prefix must pay its own bill

Operation $i$ costs $c_i$ but is charged $a_i$. The bank balance is

$$B_t = B_0 + \sum_{i=1}^{t} (a_i - c_i).$$

Start empty with $B_0 = 0$. Proving $B_t \geq 0$ for every prefix of every legal sequence gives $\sum_{i=1}^{t} c_i \leq \sum_{i=1}^{t} a_i$.

**Borrowing against future operations is insufficient.** The caller may stop while the bank is overdrawn. An initial credit $B_0 > 0$ must appear in the total-cost bound; it is not free work.

Attach credit to objects to show that it cannot be spent twice. If total credit depends only on the current state, setting $\Phi(D) = B(D)$ recovers the potential method.



## MULTIPOP: who pays for a batch of removals?

An initially empty stack supports PUSH, POP, and MULTIPOP($k$), with $k \geq 0$. The last operation removes at most $k$ items and stops when empty. Each successful push or pop costs one unit.

Charge PUSH 2: pay 1 now and leave 1 on the new item. Charge POP and MULTIPOP 0: removed items pay with their stored credit. **Every item still on the stack carries exactly one unit.** Induction proves the bank never goes negative.

Among $m$ calls, let $p$ be the number of pushes and $q$ the number of successful removals. Since $q \leq p$, actual cost is $p + q \leq 2p \leq 2m$. Pushing the same object again is a new insertion requiring new payment.

This model counts successful item operations. Adding one unit of checking/dispatch per API call changes the bound to $m + p + q \leq 3m$; an empty MULTIPOP does not literally take zero time.

# Binary counter: fix width and overflow semantics

Use a fixed width $k \geq 1$. Bit $A[0]$ is least significant; the value is $\sum_{j=0}^{k-1} A[j]2^j$. Start at zero. INCREMENT adds one modulo $2^k$, so all ones wraps to all zeros.

```
INCREMENT(A, k):
    j = 0
    while j < k and A[j] == 1:
        A[j] = 0
        j = j + 1
    if j < k:
        A[j] = 1
```

Cost $c_i$ counts flipped bits. Tests and loop control take $\Theta(1 + c_i)$ actual time, so a constant overhead per call preserves the asymptotic sequence bound. Arbitrary-precision integers that increase their width require a separate representation contract.



# A carry does not flip every bit on every call

### Carry: trailing ones and the next zero

Amber = flipped on this call; most significant bit first

| | A[3] | A[2] | A[1] | A[0] | |
|---|---|---|---|---|---|
| 0110 → 0111 | 0 | 1 | 1 | 1 | c = 1 ΔΦ = +1 ĉ = 2 |
| 0111 → 1000 | 1 | 0 | 0 | 0 | c = 4 ΔΦ = -2 ĉ = 2 |
| 1000 → 1001 | 1 | 0 | 0 | 1 | c = 1 ΔΦ = +1 ĉ = 2 |

The transition 0111→1000 flips four bits but also spends stored potential. Next, distinguish an ordinary carry from overflow.

## Analyze ordinary carry and overflow separately

For a nonoverflowing carry through $t$ trailing ones, clear those ones and set the next zero: $c = t + 1$. Let $\Phi$ be the number of set bits. Then $\Delta\Phi = 1 - t$ and $\hat{c} = 2$.

Overflow is a separate branch: $t = k$, every bit clears and no bit is set. Thus $c = k$, $\Delta\Phi = -k$, and $\hat{c} = 0$. The formula $t + 1$ does not apply to this branch.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## One increment sequence, three consistent analyses

Starting at zero, perform $m \geq 1$ increments. Bit $j$ flips once every $2^j$ calls, including wraps. The exact total is

$$T(m, k) = \sum_{j=0}^{k-1} \left\lfloor \frac{m}{2^j} \right\rfloor < 2m.$$

**Aggregate analysis** sums flips per position. **Accounting** charges 2 per call: setting a zero pays 1 now and leaves 1 for its later clearing. Every set bit owns its credit. **Potential analysis** counts those set bits, charging 2 for an ordinary increment and 0 for a wrap.

For $k = 4, m = 8$, positions flip $8, 4, 2, 1$ times, totaling 15 rather than $8 \times 4 = 32$. The proof covers every $m$; the numerical example checks the count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Complete ledger: from 0000 through wraparound

Four bits, initially $\Phi_0 = 0$. Bit strings are most significant first. The first eight charges total 16; subtract final potential 1 to get actual cost 15.

| Call | Result | $c_i$ | $\Phi_i$ | Cumulative cost |
|---|---|---|---|---|
| 1 | 0001 | 1 | 1 | 1 |
| 2 | 0010 | 2 | 1 | 3 |
| 3 | 0011 | 1 | 2 | 4 |
| 4 | 0100 | 3 | 1 | 7 |
| 5 | 0101 | 1 | 2 | 8 |
| 6 | 0110 | 2 | 2 | 10 |
| 7 | 0111 | 1 | 3 | 11 |
| 8 | 1000 | 4 | 1 | 15 |

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Complete ledger: paying for the highest bit

Continue the same sequence. Calls 9–15 each charge 2; call 16 wraps and charges 0. Total actual and amortized costs both equal 30, with no credit remaining.

| Call | Result | $c_i$ | $\Phi_i$ | Cumulative cost |
|---|---|---|---|---|
| 9 | 1001 | 1 | 2 | 16 |
| 10 | 1010 | 2 | 2 | 18 |
| 11 | 1011 | 1 | 3 | 19 |
| 12 | 1100 | 3 | 2 | 22 |
| 13 | 1101 | 1 | 3 | 23 |
| 14 | 1110 | 2 | 3 | 25 |
| 15 | 1111 | 1 | 4 | 26 |
| 16 | 0000 | 4 | 0 | 30 |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Which changes invalidate the constant amortized bound?

**A nonzero initial state:** starting with $k$ ones makes the first call cost $k$. The same potential only yields $\sum c_i \le 2m + \Phi_0 \le 2m + k$. Initial credit cannot be discarded.

**Adding DECREMENT:** for $k \ge 2$, increment from zero to $2^{k-1} - 1$, then alternate INCREMENT and DECREMENT. The states alternate between $0\underbrace{11\cdots1}_{k-1}$ and $1\underbrace{00\cdots0}_{k-1}$. Every alternating call flips all $k$ bits.

Let preparation take $r = 2^{k-1} - 1$ calls, followed by $2r$ alternating calls. There are $m = 3r$ calls, and the alternating portion alone costs $2rk = \Omega(mk)$. The upper bound is $O(mk)$ since each call flips at most $k$ bits, giving $\Theta(mk)$ on this family.

An expensive carry can now be immediately undone by a borrow. The previous credit attached to set bits is insufficient. Changing the supported operations requires a new cost proof.



## Resizable arrays: specify what moving costs

State $(n, M)$ records the number of live elements and capacity. Start at $(0, 0)$; allocate one slot on the first insertion, and double a full table before inserting.

Count **one new-record write plus the number of old records moved**. Temporarily omit fixed allocation, release, and test overhead. Initializing every newly allocated slot adds work and changes exact constants.

| Insertion | Result $(n, M)$ | Actual cost | Cumulative cost |
|---|---|---|---|
| 1 | (1,1) | 1 | 1 |
| 2 | (2,2) | 2 | 3 |
| 3 | (3,4) | 3 | 6 |
| 4 | (4,4) | 1 | 7 |
| 5 | (5,8) | 5 | 12 |

Five insertions move $1 + 2 + 4 = 7$ old records. The moved-record count need not be smaller than the insertion count; the full geometric sum proves the linear total.

## For growth alone, aggregate and potential bounds agree

After $m \geq 2$ insertions, let $L$ be the largest power of two below $m$. Moves total $1 + 2 + \cdots + L = 2L - 1$, giving $T(m) = m + 2L - 1 < 3m$. Costs for $m = 0, 1$ are 0 and 1.

Choose $\Phi(0, 0) = 0$ and $\Phi(n, M) = 2n - M$ for nonempty tables. Reachable insertion-only states satisfy $M/2 < n \leq M$, so potential is nonnegative.

| Branch | $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| First insertion $(0, 0) \to (1, 1)$ | 1 | 1 | 2 |
| Insert without moving | 1 | 2 | 3 |
| Full $(M, M) \to (M + 1, 2M)$ | $M + 1$ | $2 - M$ | 3 |

Hence $T(m) \leq 3m$. The first insertion has amortized cost 2; a formula involving $\log 0$ cannot be applied to the initial state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Shrinking too eagerly repeatedly moves the same records

Consider the bad rule: double when full, and halve as soon as deletion leaves the table half full.

Let $k \geq 2$ be a power of two. Start full at $(k, k)$. An insertion grows to $(k + 1, 2k)$ and moves $k$ records. Deleting that record shrinks back to $(k, k)$ and moves $k$ again. Each repeated insert/delete pair moves $2k$ records and costs exactly $2k + 2$.

Building the initial state costs $O(k)$. Another $2r$ alternating operations cost $\Omega(kr)$. Taking $r = k$ gives $O(k)$ operations but $\Omega(k^2)$ total work.

Separate the thresholds: **grow only when full; shrink only when deletion leaves at most one-quarter occupancy**. Shrinking restores half occupancy, leaving room to accumulate credit before the opposite resize.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Resize contract: thresholds, empty tables, and record order

This example supports **APPEND and DELETE-LAST**. Deleting from the middle may shift a suffix; that cost is separate.

- The first APPEND allocates one slot. A full table doubles before writing the new record.
- DELETE-LAST removes the last record. If $M \geq 2$ and the resulting $n \leq M/4$, halve capacity and move the $n$ survivors; otherwise retain capacity.
- Capacities are powers of two. Once allocated, minimum retained capacity is 1. An empty delete reports underflow without changing state.
- Each successful operation costs $1+$ records moved. Moving preserves survivor order but can invalidate external raw pointers.

**Threshold convention:** this example uses $n \leq M/4$. Textbook § 17.4 uses $n < M/4$, delaying contraction by one deletion when $M \geq 4$. Both allow constant amortized bounds, but their exact ledgers differ.

At most one resize occurs per operation. For $M \geq 4$, a shrink is triggered at exactly $n = M/4$: the prior state was above the threshold and one operation removes one record.



## Follow the same records through growth and contraction

Separate resize thresholds leave time to save credit

| | | | |
|---|---|---|---|
| Full | n=4, M=4 | a b c d | cost — |
| APPEND e | n=5, M=8 | a b c d e · · · | cost 5 = 1 + 4 |
| DELETE e | n=4, M=8 | a b c d · · · · | cost 1 |
| DELETE d | n=3, M=8 | a b c · · · · · | cost 1 |
| DELETE c | n=2, M=4 | a b · · | cost 3 = 1 + 2 |

Half full: retain capacity. Quarter full: halve capacity.

After growing to eight slots, retain capacity when four records remain. Shrink to four slots only when two remain. This separation between thresholds is **hysteresis**: it prevents immediate resize reversals.

## Dense and sparse states store credit for different resizes

For $M \geq 2$, define

$$\Phi(n, M) = \begin{cases} 2n - M, & n \geq M/2, \\ M/2 - n, & n < M/2. \end{cases}$$

Also set $\Phi(0, 0) = \Phi(0, 1) = 0$ and $\Phi(1, 1) = 1$. Both branches equal zero at half occupancy; every reachable state has nonnegative potential.

Dense states accumulate credit during insertions to pay for growth. Sparse states accumulate credit during deletions to pay for contraction. A state uses exactly one branch, so credit is not counted twice.

The next page checks each branch. Empty-table cases at capacities 1 and 2 are separate because $M/4$ need not be an attainable integer count.



## The general-capacity branches all have constant amortized bounds

Here $M$ is capacity before the operation. Substitute into both branches when a nonresizing operation crosses half occupancy.

| Case | $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| Insert without growth | 1 | $-1$ or 2 | 0 or 3 |
| Grow a full table | $M + 1$ | $2 - M$ | 3 |
| Delete without shrinking | 1 | $-2$ or 1 | $-1$ or 2 |
| Shrink after reaching $M/4$, $M \geq 4$ | $1 + M/4$ | $1 - M/4$ | 2 |

For the last row, before deletion $n = M/4 + 1$ and potential is $M/4 - 1$. Afterwards capacity is $M/2$ and count is $M/4$: exactly half full, with potential zero.

Example: $(3, 8) \to (2, 4)$ moves two records, so $c = 3$. Potential falls from 1 to 0, giving $\hat{c} = 2$.

## Minimum-capacity cases and the total-cost conclusion

Check the remaining small-capacity boundaries separately:

| State change | $c$ | $\Delta\Phi$ | $\hat{c}$ |
|---|---|---|---|
| First $(0,0) \to (1,1)$ | 1 | 1 | 2 |
| Reuse $(0,1) \to (1,1)$ | 1 | 1 | 2 |
| Delete last $(1,1) \to (0,1)$ | 1 | $-1$ | 0 |
| Smallest shrink $(1,2) \to (0,1)$ | 1 | 0 | 1 |

Every successful operation has $\hat{c} \leq 3$. Initial potential is zero and final potential is nonnegative, so any $m$ successful APPEND / DELETE-LAST operations cost at most $3m$ in this model.

Charging one unit for an underflow test adds an operation with unchanged potential and amortized cost 1. The bound does not cover arbitrary-position deletion, arbitrary shrinking policies, or unmodeled record-moving costs.



## Dictionary: decide what inserting an existing key means

Maintain a key→record mapping with at most one record per key. INSERT updates the value of an existing key. DELETE takes a key and reports absence if missing. Equal hash values do not imply equal keys.

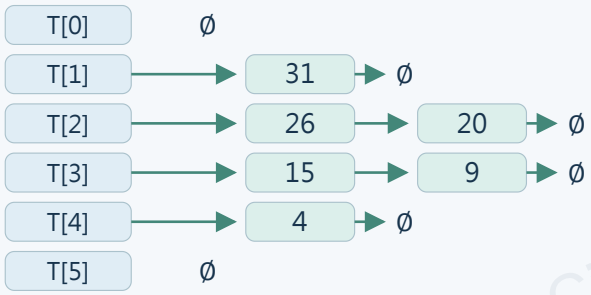
For universe $\{0, \ldots, M-1\}$, direct addressing uses the key as an index. Operations take worst-case $O(1)$, with $\Theta(M)$ space and ordinary full-table initialization. Hashing becomes useful when only $n \ll M$ records are present.

Use $h : U \to \{0, \ldots, m-1\}$. If $|U| > m$, every fixed h has collisions by the pigeonhole principle, so retain full keys for verification. Bounds assume keys, hashing, and equality take constantly many word operations. Long strings and arbitrary-precision integers require their own reading and arithmetic costs.

# Chaining keeps colliding records in a list

m = 6; h(k) = k mod 6

Insert at heads: 4, 9, 15, 20, 26, 31




Find 20

26 → 20 ✓

Find 14

26 → 20 → ø

Same bucket: compare full keys

With $m = 6, h(k) = k \bmod 6$, insert 4,9,15,20,26,31 at chain heads. Slot 2 contains 26→20 and slot 3 contains 15→9. Searching for 20 checks two records; absent key 14 also checks both records in slot 2.



# Constant-time head insertion does not cover all of INSERT

Let $\alpha = n/m$; chaining allows $\alpha > 1$. Search scans the designated chain until finding the key or reaching its end. Worst-case time is $\Theta(n)$.

Head insertion rewires links in $O(1)$ when the key is known to be new. Our update-on-existing-key INSERT must first search for duplicates. DELETE(key) also pays for search; only deletion through a valid doubly linked node handle supports direct $O(1)$ unlinking.

CLRS § 11.2 gives constant-time insertion and deletion under these different input contracts: insertion assumes a new object and deletion receives the object itself. The bounds are compatible once the contracts are distinguished.

## Unsuccessful search: add the cost of each collision

First assume simple uniform hashing: fixed distinct keys have independent uniform bucket locations. For a fixed absent query q, define $I_x = \mathbf{1}[h(x) = h(q)]$. Then

$$L_q = \sum_{x \in K} I_x, \qquad E[L_q] = \sum_{x \in K} \frac{1}{m} = \frac{n}{m} = \alpha.$$

Search checks every record in that chain. Adding constant hashing and indexing work gives expected time $\Theta(1 + \alpha)$. At $\alpha = 0$, inspecting an empty bucket still costs time.

Linearity of expectation itself needs no independence between collision events. The full uniform model is sufficient; a universal family will later give an upper bound using only pairwise collision bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Successful search averages records rather than buckets

Fix distinct insertion order $x_1, \ldots, x_n$, insert at heads, perform no deletions, and choose a successful query uniformly among stored records. Finding $x_i$ checks itself and later inserted records in its bucket.

$$E[C_i] = 1 + \frac{n - i}{m}, \qquad E[C] = \frac{1}{n} \sum_{i=1}^{n} E[C_i] = 1 + \frac{n - 1}{2m}.$$

Adding constant hashing work gives $\Theta(1 + \alpha)$. Larger buckets contain more keys that may be queried; successful queries do not select buckets uniformly.

Our six-key table has two length-two chains and two singletons. Summed successful checks are $(1 + 2) + (1 + 2) + 1 + 1 = 8$, averaging $4/3$. This fixed-table average need not equal the random-model expectation $1 + 5/12$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Division and string hashing: a formula is not a distribution guarantee

Division hashing is $h(k) = k \bmod m$. For $m = 2^p$, it retains only p low bits: 0 and 8 both map to 0 when m=8. A prime modulus can avoid some input patterns, but does not prove uniformity for arbitrary key sets.

Horner evaluation uses $H_0 = 0$ and $H_{i+1} = (BH_i + c_i) \bmod m$. Induction shows this equals the full base-B integer modulo m. A length-$\ell$ string requires $\Theta(\ell)$ constant-cost steps.

With B=10, m=7, codes [1,2,3] yield intermediate values $1, 5, 4$, matching $123 \bmod 7 = 4$. If $B \equiv 1 \pmod m$, the result is just the code sum modulo m, unchanged by permutation. Computing the formula correctly does not establish low collision probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Multiplication hashing: take high bits of the low word

Use $h(k) = \lfloor m \operatorname{frac}(kA) \rfloor$. For $A = s/2^w, m = 2^p$, compute $r_0 = ks \bmod 2^w$, then $h(k) = \lfloor r_0 / 2^{w-p} \rfloor$, with $1 \le p \le w$.

Write $ks = q2^w + r_0$. Then $kA = q + r_0/2^w$, so its fractional part is $r_0/2^w$. Multiplying by $2^p$ and flooring discards the low w−p bits of the low word, leaving its p high bits.

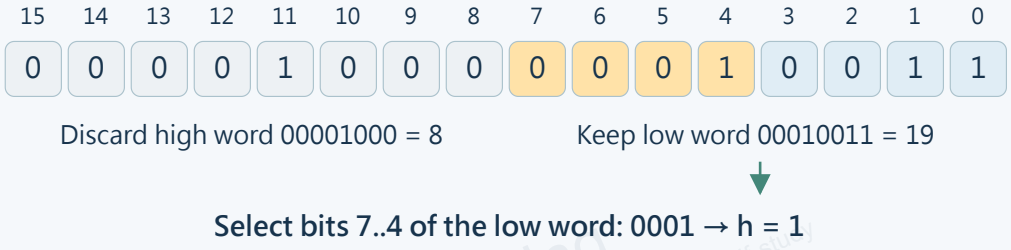
Next use w=8, s=159, k=13, p=4 to trace every bit; the result should be 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Multiplication example: from full product to slot index

w = 8; s = 159; k = 13; p = 4

ks = 2067 = 8 × 256 + 19




$2067 = 8 \cdot 256 + 19$. The low word is 00010011; its four high bits are 0001. Do not take the highest bits of the full product. Respect the stated word width.



## Universal families randomize the choice of function

A finite family H is universal if every fixed pair of distinct keys x,y satisfies

$$\Pr_h[h(x) = h(y)] \leq \frac{1}{m}$$

when h is chosen uniformly from H. Keys and queries must be independent of that draw. No uniform input distribution or mutual independence of all collisions is required. An individual sampled table may still have a long chain.

**Source convention:** we use the “at most” definition of CLRS § 11.3.3. MIT 2005 Lecture 8 defines an exact collision probability and demonstrates a prime-modulus dot-product family. Its equality implies our inequality, but the reverse implication is invalid.

## Universal chaining: the complete expected upper bound

Fix set K and query q. Let $Y_q = \sum_{x \in K, x \neq q} \mathbf{1}[h(x) = h(q)]$. Linearity and universality give

$$E[Y_q] \leq \frac{|K \setminus \{q\}|}{m}.$$

An absent query scans at most $n/m$ records in expectation. A present query's entire chain has expected length at most $1 + (n-1)/m$, and search costs no more than scanning it. Thus each fixed query has expected time $O(1+\alpha)$ without requiring uniform successful queries.

For a fixed operation sequence with at most $O(m)$ live keys, update-style INSERT and DELETE(key) search first. Total expected time for t calls is $O(t)$ plus $\Theta(m)$ initialization. Queries chosen adaptively from h or earlier observations are not covered by this fixed-pair proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Affine family: prove the prime-field mapping first

Choose prime $P > \max U$, independently uniform $a \in \{1, \ldots, P-1\}$ and $b \in \{0, \ldots, P-1\}$, and define

$$h_{a,b}(k) = ((ak + b) \bmod P) \bmod m.$$

For fixed x≠y, let $r = (ax + b) \bmod P$ and $s = (ay + b) \bmod P$. Since $a(x-y) \not\equiv 0 \pmod P$, r≠s.

Conversely, each r≠s uniquely determines $a = (r-s)(x-y)^{-1} \bmod P$ and $b = r - ax \bmod P$. Uniform $(a, b)$ therefore produces uniform ordered distinct $(r, s)$ among $P(P-1)$ possibilities. Primality guarantees the inverse of nonzero x−y.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The final reduction still bounds collisions by 1/m

Condition on r. Each of the other P−1 residues s is equally likely. At most $\lceil P/m \rceil - 1 \leq (P-1)/m$ share r's residue modulo m. Hence

$$\Pr[h(x) = h(y) \mid r] \leq \frac{\lceil P/m \rceil - 1}{P-1} \leq \frac{1}{m}.$$

Averaging over r proves universality. For m≥P, reducing modulo m merges no distinct r,s, so collision probability is zero.

Write $P = qm + d, 0 \leq d < m$. If $r \bmod m < d$, its bucket has $q+1$ elements and $q$ others after excluding $r$; otherwise a nonempty bucket has $q$ elements and $q-1$ others. Thus the count is at most $q$; for $d = 0$ it is exactly $q-1$. Table size $m$ need not be prime; $P$ must be prime. Modular arithmetic must fit the assumed word model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why the MIT dot-product family also works

Let m be prime and represent keys by fixed-length vectors $x = (x_0, \ldots, x_r) \in \mathbb{F}_m^{r+1}$. Draw uniform coefficient vector a and use $h_a(x) = \sum_i a_i x_i \bmod m$.

For x≠y, choose j with $x_j - y_j \neq 0$. After fixing all other coefficients, collision requires

$$a_j(x_j - y_j) = -\sum_{i \neq j} a_i(x_i - y_i) \pmod m.$$

The difference is invertible, so exactly one of m values of $a_j$ satisfies the equation. Collision probability is exactly 1/m. Here table modulus m must be prime, unlike the arbitrary table size of the affine family. Variable vector length also changes hashing cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Open addressing uses three distinct slot states

Store one record per slot, so n≤m. Each key has a reproducible fixed sequence $h(k,0),\ldots,h(k,m-1)$. To find space whenever available, these m positions must form a permutation of the table.

| State | Search action | Deletion effect |
|---|---|---|
| OCCUPIED | Compare full keys | May become DELETED |
| DELETED (tombstone) | Continue probing | Not a stopping point |
| EMPTY | Stop: absent | Never created by ordinary deletion |

EMPTY means unused since this table was built. An inserted key could not have skipped an EMPTY earlier in its sequence. Deletion leaves DELETED rather than breaking that path. A rebuild establishes a new table state.



## Search and deletion probe at most m slots

```
FIND(k):
    for i = 0 .. m-1:
        j = h(k,i)
        if T[j] is EMPTY: return absent
        if T[j] is OCCUPIED and T[j].key == k:
            return j
    return absent
```

DELETE(k) runs FIND, changing a found slot to DELETED and otherwise preserving state. EMPTY and DELETED require separate tags rather than reserved values that could also be legal keys.

The loop bound matters: a table may contain tombstones but no EMPTY slots. An unsuccessful search must return after m probes rather than wait indefinitely for an empty slot.

## Update-style INSERT remembers a tombstone but keeps searching

```
first = NONE
for i = 0 .. m-1:
    j = h(k,i)
    if T[j] is OCCUPIED and T[j].key == k:
        T[j].value = value; return j
    if T[j] is DELETED and first == NONE: first = j
    if T[j] is EMPTY:
        target = first if first != NONE else j
        T[target] = OCCUPIED(k,value); return target
if first != NONE: T[first] = OCCUPIED(k,value); return first
return full
```

An existing key may lie beyond the first DELETED. Inserting immediately could create duplicates. Establish absence at EMPTY or after all m probes, then reuse the saved slot. This assumes a full-table probe permutation.



## Tombstones preserve search paths while allowing reuse

m = 7; home(k) = k mod 7

∅ = EMPTY; D = DELETED

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | |
|---|---|---|---|---|---|---|---|---|
| Insert 1, 8, 15 | ∅ | 1 | 8 | 15 | ∅ | ∅ | ∅ | Probes: 1 → 2 → 3 |
| Delete 1 | ∅ | D | 8 | 15 | ∅ | ∅ | ∅ | Find 15: 1 → 2 → 3 |
| Update 8 | ∅ | D | 8 | 15 | ∅ | ∅ | ∅ | Update value at slot 2 |
| Insert 22 | ∅ | 22 | 8 | 15 | ∅ | ∅ | ∅ | Probe to 4; write slot 1 |

Continue past D; rule out an existing key before reusing it.

Use $m = 7, h(k,i) = (k \bmod 7 + i) \bmod 7$. Keys 1,8,15 occupy slots 1,2,3. After deleting 1, finding 15 still follows $1 \to 2 \to 3$; a tombstone does not stop search.

## A complete probe ledger for the same records

Three records remain. INSERT(22) confirms absence at slot 4, then writes slot 1.

| Call | Slots probed | Write / result | Probes |
|---|---|---|---|
| INSERT(1,a) | 1 | store (1,a) at 1 | 1 |
| INSERT(8,b) | 1,2 | store (8,b) at 2 | 2 |
| INSERT(15,c) | 1,2,3 | store (15,c) at 3 | 3 |
| DELETE(1) | 1 | slot 1=DELETED | 1 |
| FIND(15) | 1,2,3 | return slot 3 | 3 |
| INSERT(8,b′) | 1,2 | update slot 2 only | 2 |
| INSERT(22,d) | 1,2,3,4 | reuse slot 1 | 4 |
| FIND(29) | 1,2,3,4 | absent | 4 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Probe formulas: first establish table coverage

Linear probing uses $(h_0(k) + i) \bmod m$ and covers every slot. Contiguous occupied runs attract further insertions, producing primary clustering.

Quadratic probing uses $(h_0(k) + c_1 i + c_2 i^2) \bmod m$ with constrained parameters. For m=7 and offset $i^2$, the first seven offsets are $0, 1, 4, 2, 2, 4, 1$, visiting only four slots. Insertion can fail despite unused slots elsewhere.

Equal home positions give identical sequences in these two schemes, the source of secondary clustering. Double hashing varies the step as well, but coverage still requires a step coprime to m; adding a function alone is not a proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Two provable full-coverage probe schemes

**Double hashing:** use $h(k,i) = (h_1(k) + id) \bmod m$. Returning home requires $m \mid id$. With $g = \gcd(m,d)$, cancellation gives minimal positive period $m/g$. Full coverage therefore requires g=1. Use 1≤d<m for prime m, or odd d for power-of-two m.

**Triangular offsets:** for $m = 2^w$, use $f(i) = i(i+1)/2 \bmod m$, dividing as an integer first. For m=8 the offsets are $0, 1, 3, 6, 2, 7, 5, 4$.

If f(i)=f(j), then $2^{w+1} \mid (i-j)(i+j+1)$. The factors have opposite parity, so the even factor is divisible by $2^{w+1}$. But $|i-j| < 2^w$ and $1 \le i+j+1 < 2^{w+1}$. The only possibility is i−j=0. Thus f is injective and covers all m slots.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Uniform probing is a stronger idealized model

The following bounds assume each new key has a uniformly random permutation relative to the current occupied state, with no deletions or tombstones and n<m. This is sampling without replacement, not selecting a fresh random slot independently on every probe.

Linear, quadratic, and double hashing generate restricted sequence families. Uniform home locations or a universal family do not automatically imply the ideal distribution over m! permutations.

Separate collision probabilities for chaining, full-table probe coverage, and the distribution of probe permutations. The first two do not imply the expected formulas that follow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Unsuccessful-search bound from tail probabilities

Let X count probes through the first EMPTY and $\alpha = n/m < 1$. The probability that the first i−1 positions are occupied is

$$\Pr(X \geq i) = \prod_{j=0}^{i-2} \frac{n-j}{m-j} \leq \alpha^{i-1}, \quad 1 \leq i \leq n+1.$$

For i=1 the empty product is 1; for i>n+1 the probability is zero. Each ratio is at most n/m since $m \geq n$. Using $X = \sum_{i \geq 1} \mathbf{1}[X \geq i]$,

$$E[X] = \sum_{i \geq 1} \Pr(X \geq i) \leq \sum_{i \geq 1} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

The same bound covers insertion of a new key. At m=10,n=8 it is 5, not an exact count per query or an unconditional guarantee for linear probing.



## Successful-search bound replays the insertion path

Fix n distinct insertions with no deletions or relocations. Choose successful queries uniformly among records. Insertion i+1 sees i occupied slots and expects at most $m/(m-i)$ probes; later searches replay that path.

$$E[C] \leq \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^{m} \frac{1}{k}$$
$$\leq \frac{1}{\alpha} \int_{m-n}^{m} \frac{dx}{x} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}.$$

Since 1/x decreases, its integral over [k−1,k] is at least 1/k. No successful query exists at α=0; do not substitute zero into 1/α. The right side tends to 1 as α→0. Both the ideal probe model and uniform successful queries are needed for this formula.

## Low live load can still require a full-table search

For m=7, fill all slots with seven distinct home positions, then delete every key. Now n=0 and α=0, but every slot is DELETED. Any absent query probes all seven positions.

The tombstone-free $1/(1-\alpha)$ bound is inapplicable. Tracking $\beta = (n+d)/m$, where d counts tombstones, exposes β=1 and the absence of an early EMPTY stopping point. β alone is not a performance proof for every implementation.

Rebuilding removes tombstones but costs allocation, rehashing, and reinsertion; open-address reinsertion itself requires probes. An expected amortized rebuild claim needs explicit thresholds, a random model, and rebuild rules, not just a promise to rehash periodically.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Perfect hashing moves random retries into construction

FKS stores a fixed set of n distinct keys. After construction, queries take worst-case $O(1)$ word operations with $O(n)$ space. Random retries give expected linear construction time, not worst-case linear time.

For n=0, return an empty dictionary without modulo zero. For n≥1 use m=n, draw a universal first-level h, and let bucket j contain $n_j$ keys. Allocate $n_j^2$ secondary slots.

Let Y count unordered colliding pairs. Applying $n_j^2 = n_j + 2\binom{n_j}{2}$ to each bucket gives

$$\sum_j n_j^2 = n + 2Y, \qquad E\left[\sum_j n_j^2\right] \le n + 2\binom{n}{2}\frac{1}{n} = 2n - 1.$$

This controls total squared space through collision pairs; it does not require every bucket to be small.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## First level: check space before accepting the draw

Let $S = \sum_j n_j^2$. Markov gives $\Pr(S \geq 4n) \leq E[S]/(4n) < 1/2$. Independently redraw until the measured value satisfies S<4n.

Each trial clears n bucket counts and processes n keys in O(n). **Check S before allocating quadratic secondary tables for a rejected draw.** Acceptance probability exceeds 1/2, so expected trials are below 2 and first-level expected work is O(n).

Acceptance establishes a deterministic S<4n for the built structure. Merely knowing E[S]<2n would not establish that every output has linear size. Rare runs can require many retries; expected trial count is not a hard limit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Second level: squared space buys collision-free placement

Condition on an accepted first-level partition. A bucket of s≥2 keys gets $s^2$ slots and an independent universal $h_j$. Its collision-pair count Z satisfies

$$E[Z] \leq \binom{s}{2}\frac{1}{s^2} = \frac{s-1}{2s} < \frac{1}{2}, \qquad \Pr(Z \geq 1) \leq E[Z] < \frac{1}{2}.$$

Accept only after verifying zero collisions. Each trial clears $s^2$ slots and places s keys in $O(s^2)$, with fewer than two expected trials. Summing conditionally over the fixed accepted partition gives $O(\sum_j s_j^2) = O(n)$ expected work; averaging over the first level preserves O(n) total construction.

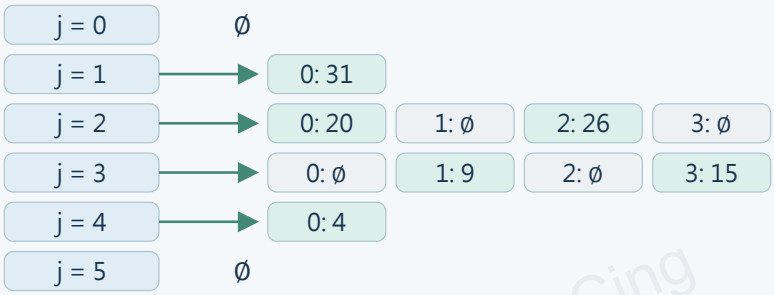
Empty buckets allocate nothing; singleton buckets store their record directly. Sizes, parameters, and pointers add O(n) space. Worst-case queries assume constant-cost hashing and key comparison; inserting new keys is outside this static contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Two-level layout: place the same keys in actual slots

**P = 37; h(k) = (k mod 37) mod 6**

Two-key buckets: $h_j(k) = (k \bmod 37) \bmod 4$




26, 14 → j = 2 → slot 2

Stored key = 26

26 = 26 ✓

14 ≠ 26 ✗

Total secondary slots: 1 + 4 + 4 + 1 = 10

Use $K = \{4, 9, 15, 20, 26, 31\}$, P=37, and $h(k) = (k \bmod 37) \bmod 6$. Bucket sizes [0,1,2,2,1,0] have square sum 10<4n=24. Both two-key buckets use $h_j(k) = (k \bmod 37) \bmod 4$, occupying (0,2) and (1,3), respectively, with verified zero collisions.



## An absent key can still reach an occupied perfect-hash slot

Query 26 follows first-level bucket 2 and secondary slot 2, reads key 26, and succeeds after equality comparison. Query 14 follows the same route but reads 26, so it is absent.

Query 5 reaches empty first-level bucket 5 and stops without evaluating a nonexistent secondary hash. Query 10 reaches singleton bucket 4 but reads 4≠10, so it also fails.

**Perfect hashing separates stored keys; it does not certify membership for absent keys.** Handle empty buckets, check slot occupancy, and compare full keys. After construction, present and absent queries both traverse constantly many levels, establishing the worst-case query bound.

## BST constraints come from every ancestor

A binary search tree turns comparisons into branches. Continue the unique-key dictionary contract: a node stores key, value, left, right, and parent; missing links are NIL and root.parent=NIL. Inserting an existing key updates its value.

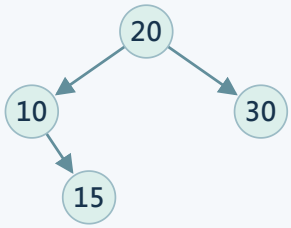
**Every key in a node's entire left subtree is smaller; every key in its entire right subtree is larger.** Parent-child comparisons alone are insufficient. A left turn tightens the upper bound; a right turn tightens the lower bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



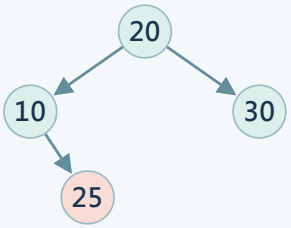
## Ancestor intervals detect globally invalid placement

### Every ancestor contributes a bound




15 ∈ (10, 20)

All bounds hold




25 > 10 ✓

25 < 20 ✗

Ancestor bound (10, 20)

CLRS § 12.1 also permits duplicates under non-strict inequalities. Here order is strict. Duplicates could instead use a count or a (key,unique-ID) ordering; these require an explicit contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Search rules out an entire subtree at each comparison

```
FIND(T,k):
    x = T.root
    while x != NIL:
        c = compare(k,x.key)   // one three-way comparison
        if c == 0: return x
        x = x.left if c < 0 else x.right
    return absent
```

Invariant: if k exists, it lies in the subtree rooted at x. Global BST order rules out the other subtree after an unequal comparison. Each iteration descends one level; reaching NIL proves absence.

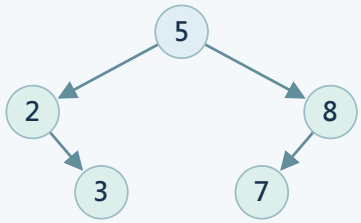
Depth d counts **edges** from the root, whose depth is 0. Successful search at depth d visits d+1 records. Failed search counts only visited non-NIL nodes. Count one three-way key comparison per visit; NIL tests and comparator instructions are separate costs.

Height h counts edges on a longest path; the empty tree has height −1. For nonempty trees search costs O(h+1); an empty tree costs O(1). A uniform bound is O(h+2).



## Trace successful and unsuccessful searches in one tree

**Root depth is 0; successful comparisons = depth + 1**




d = 0

Find 7: 5 → 8 → 7; 3 comparisons

d = 1

Find 6: 5 → 8 → 7 → NIL

Failure also uses 3 comparisons

d = 2

Queries 9 and 10 fail to the right of 8

Finding 7 visits 5→8→7, using three key comparisons. Finding 6 visits those same three nodes before reaching NIL. Queries 9 and 10 visit 5→8→NIL, using two comparisons.

Count visited records for either outcome. The final NIL pointer test is not another key comparison.

## Average cost depends on how queries are drawn

Uniform queries among the five stored keys give total successful comparisons 1+2+2+3+3=11, hence mean **11/5**. Uniform integer queries from 1 through 10 also include five failures.

| query | 1 | 4 | 6 | 9 | 10 |
|---|---|---|---|---|---|
| keys visited | 5,2 | 5,2,3 | 5,8,7 | 5,8 | 5,8 |
| comparisons | 2 | 3 | 3 | 2 | 2 |

The mixed mean is (11+12)/10=**2.3**. Gaps are not equally likely: $(8,+\infty)$ contains queries 9 and 10 here, with weight 2/10. Equal weights on NIL links would model a different experiment.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Inorder traversal and extrema

Inorder traversal lists left subtree, current node, then right subtree. Inductively, both subtrees produce increasing sequences. Global BST order places every left key below the root and every right key above it, so concatenation is sorted. Each node is processed once, giving $\Theta(n+1)$ time and $O(h+2)$ recursive stack space.

MINIMUM(x) requires $x \neq \text{NIL}$ and follows left links. If a left subtree exists it contains the minimum; otherwise x is smaller than everything to its right. MAXIMUM is symmetric. A dictionary-level minimum on an empty tree should return absent.

**BST sorting includes construction.** Inserting 1,2,…,n produces a right chain and costs $0+1+\dots+(n-1)=n(n-1)/2$ comparisons. Adding linear inorder traversal still gives $\Theta(n^2)$ total time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Successor: descend right or climb to the first left turn

```
SUCCESSOR(x):                   // valid node handle
    if x.right != NIL: return MINIMUM(x.right)
    p = x.parent
    while p != NIL and x == p.right:
        x = p; p = p.parent
    return p                   // NIL means no successor
```

With a right subtree, its minimum is the next key. Without one, each climb from a right child passes a smaller ancestor; the first ancestor reached from its left side is the smallest larger key.

For root 5 with children 2,8, plus 2.right=3 and 8.left=7: succ(5)=7 via 8→7; succ(3)=5 via 3→2→5; succ(8)=NIL. PREDECESSOR swaps left and right.

A call follows one path and costs O(h+1). Starting at the minimum and repeatedly finding successors traverses each edge at most once downward and once upward, giving O(n) for the full sequence. Multiplying individual worst-case bounds does not prove Θ(nh).



## Insertion finds a missing link, not an arbitrary leaf

```
INSERT(T,k,v):
    p = NIL; x = T.root
    while x != NIL:
        p = x; c = compare(k,x.key)
        if c == 0: x.value = v; return x
        x = x.left if c < 0 else x.right
    z = Node(k,v,NIL,NIL,p)
    if p == NIL: T.root = z
    elif k < p.key: p.left = z
    else: p.right = z
    return z
```

The search maintains the interval imposed by ancestors. At NIL, the new key satisfies every bound. The duplicate branch changes only the value, preserving key and identity. Arbitrarily changing a key in place can violate order.

Worst-case time is O(h+2), including empty-tree insertion, with O(1) new-node space.

## Transplant changes the incoming parent link

```
TRANSPLANT(T,u,v):              // u is a valid member node
    p = u.parent
    if p == NIL: T.root = v
    elif u == p.left: p.left = v
    else: p.right = v
    if v != NIL: v.parent = p
```

This redirects the incoming link to u and updates a nonempty v's parent. **It does not reconnect v.left or v.right.** NIL is allowed as v but is not a writable parent-bearing node in this section.

If z has no left child, delete it with TRANSPLANT(T,z,z.right), including a leaf. If it has only a left child, transplant that child. Every retained subtree remains within the old ancestor bounds.

Deleting root 20 with sole right child 40 makes root=40 and 40.parent=NIL. Deleting the last node makes root=NIL. These cases change a constant number of links; DELETE(key) must also pay for finding z.

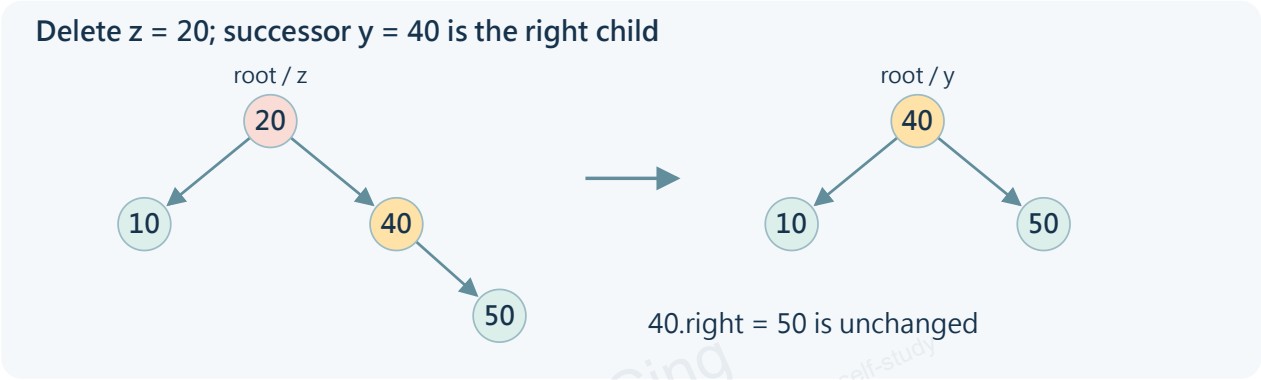


## Two children: detach the successor before moving it

```
DELETE-NODE(T,z):
    if z.left == NIL: TRANSPLANT(T,z,z.right)
    elif z.right == NIL: TRANSPLANT(T,z,z.left)
    else:
        y = MINIMUM(z.right)
        if y.parent != z:
            TRANSPLANT(T,y,y.right)
            y.right = z.right; y.right.parent = y
        TRANSPLANT(T,z,y)
        y.left = z.left; y.left.parent = y
```

The right-subtree minimum y has no left child but **may have a right child**. A deeper y must first be replaced by y.right before receiving z's old right subtree. If y=z.right, leave its right link unchanged: assigning y.right=z.right would create y.right=y.

# An immediate successor keeps its existing right child

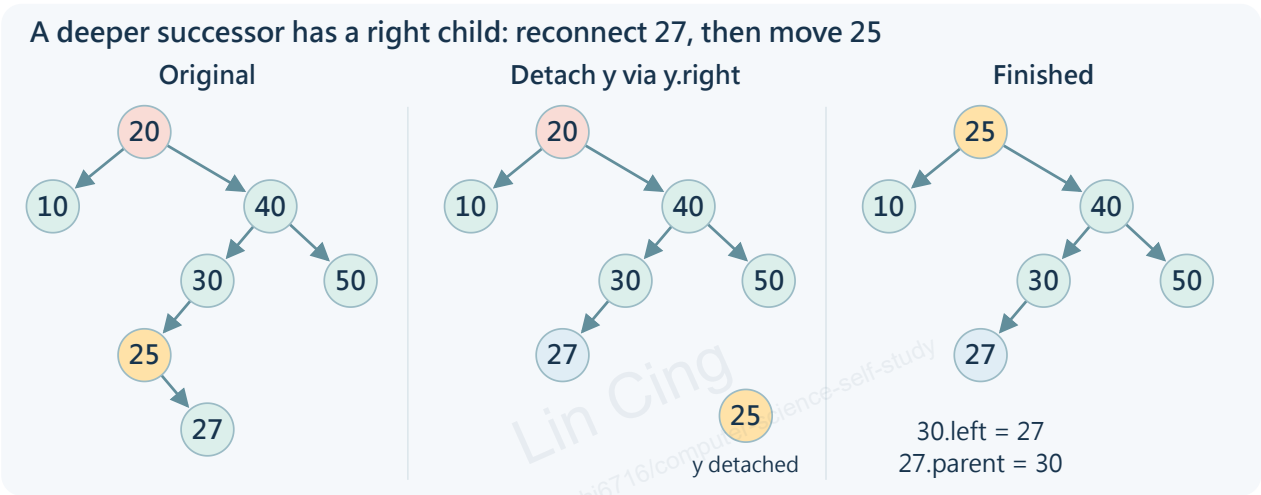



Delete z=20: y=40 and y.parent=z. Redirect root to y and set y.left=10 and 10.parent=y. **The link y.right=50 never changes.**

Inorder changes from [10,20,40,50] to [10,40,50]. Every retained key, value, and object identity is unchanged; only z leaves the tree. For nonroot z, redirect its old parent's left or right link instead of root.



# A deeper successor must not lose its right child




Delete root z=20. Its successor y=25 has right child 27. Writes are: ① 30.left=27 and 27.parent=30; ② y.right=40 and 40.parent=y; ③ root=y and y.parent=NIL; ④ y.left=10 and 10.parent=y.

## Check order and reachability after a deep transplant

The middle panel shows y temporarily detached. Its stale links must be cleared or overwritten; the whole object pool is not a valid single BST during the update. Finally root=25,25.right=40,40.left=30,30.left=27.

Inorder [10,20,25,27,30,40,50] loses only 20. Writing 30.left=NIL would lose 27. Forgetting parent updates would break later upward successor searches.

Why is order preserved? Successor 25 is the minimum of the old right subtree. Every old left-subtree key is below 25, and every remaining right-subtree key is above 25. Key 27 lies between 25 and 30, so reconnecting it as 30.left is valid.

Every original object except z remains reachable exactly once. Finally check root.parent=NIL, matching child/parent links, and absence of cycles. Finding the successor costs O(h+1); rewiring is O(1). Reclaim z only after reconnection.



## Deleting a key and deleting an object are different contracts

Let z.key=20 and its successor y.key=25. Both methods can preserve the same key set, but handles behave differently.

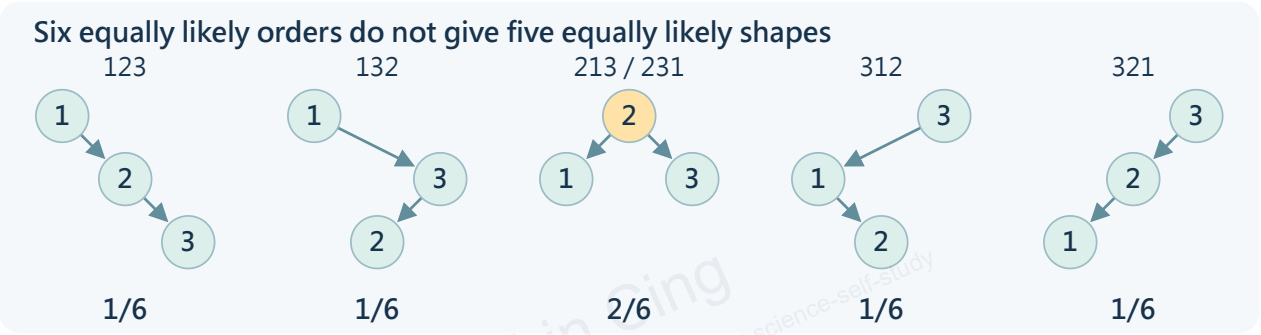
| Method | Object occupying z's old position | Removed object | Handle effect |
|---|---|---|---|
| Copy y's payload, then delete y | Original z, now representing 25 | y | z changes meaning; y is invalidated |
| Transplant node y | Original y in a new position | z | z is invalidated; y retains identity |

**Source convention:** the handwritten notes illustrate copying; CLRS third edition § 12.3 transplants nodes, as we do here. Copying a dictionary record must move the matching value, not just its key. Extra fields, resource ownership, and external indices also need maintenance.

Either method may use the predecessor instead, with every direction mirrored consistently. Do not combine a predecessor diagram with successor code.

# Uniform insertion orders do not give uniform tree shapes

Draw uniformly from n! orders on n distinct keys. The first key becomes root; given root rank r, each side retains uniform relative order. Analyze insertion-only construction.




Orders 213 and 231 share the balanced tree (2/6). Each other order gives a different shape (1/6). Sampling five shapes with probability 1/5 each is a different model.



# Expected depth of a fixed rank: count ancestors

Write keys in order $x_1<\ldots<x_n$ and let $D_i$ be the depth of $x_i$. For $j\neq i$, $x_j$ is an ancestor of $x_i$ **iff $x_j$ is inserted first among the keys in the closed rank interval between i and j**.

Earlier keys outside the interval route both to the same side. If $x_j$ appears first inside, it lies on the search path to $x_i$. Any other first key either is $x_i$ or separates them. Thus

$$\Pr(x_j \text{ is an ancestor of } x_i) = \frac{1}{|j-i|+1}.$$

Depth counts ancestors. Linearity of expectation, without independent events, gives

$$E[D_i] = \sum_{j<i} \frac{1}{i-j+1} + \sum_{j>i} \frac{1}{j-i+1} = H_i + H_{n+1-i} - 2.$$

Here $H_t = \sum_{k=1}^{t} 1/k$. For n=5,i=3, expected depth is $2H_3 - 2 = 5/3$. A successful search adds one comparison, giving 8/3.

## Expected construction cost sums all node depths

The nodes compared during insertion become exactly that node's ancestors in the finished tree; later insertions do not change its depth. Hence total construction comparisons are $C = \sum_i D_i$. Using

$$\sum_{i=1}^{n} H_i = \sum_{k=1}^{n} \frac{n-k+1}{k} = (n+1)H_n - n,$$

we obtain

$$E[C] = 2(n+1)H_n - 4n = \Theta(n \log n), \qquad n \geq 2.$$

Allocation and linking add Θ(n). Uniform successful search has expected mean comparisons E[C]/n+1. For n=3 these are 8/3 and 17/9 respectively; n=1 uses zero construction comparisons and one successful-search comparison.

This connects to quicksort using the first item as pivot recursively: comparison of a key pair depends on which key in their rank interval appears first. However, **logarithmic expected average depth does not yet bound expected maximum depth**.



## Height analysis handles the maximum before expectation

Let $X_n$ be edge height and $Y_n = 2^{X_n}$ for n≥1. Set **$Y_0$=0 as an auxiliary value**, not $2^{X_0} = 2^{-1}$. A singleton has $Y_1$=1. For n≥2, at least one subtree is nonempty; conditional on root rank R=r,

$$Y_n = 2 \max(Y_{r-1}^{L}, Y_{n-r}^{R}).$$

Write $a_j$=E[$Y_j$]. Conditional subtree orders are uniform. Since max(u,v)≤u+v,

$$a_n \leq \frac{2}{n} \sum_{r=1}^{n} (a_{r-1} + a_{n-r}) = \frac{4}{n} \sum_{j=0}^{n-1} a_j, \qquad n \geq 2.$$

This uses **conditional expectation**, not unconditional independence of random subtree sizes and R. It also avoids the false equality E[max(U,V)]=max(E[U],E[V]).

For example, $a_1 = 1$ and $a_2 = 2$. Among the six orders on three keys, two have height 1 and four have height 2, so $a_3$=(2 • 2+4 • 4)/6=10/3.

## Bound exponential height, then return to height

Strong induction gives $a_n \le n^3$ for $n \ge 1$. The base is $a_1 = 1$. For $n \ge 2$, use $a_0 = 0$ and $\sum_{j=1}^{n-1} j^3 = [n(n-1)/2]^2$:

$$a_n \le \frac{4}{n} \sum_{j=1}^{n-1} j^3 = n(n-1)^2 \le n^3.$$

Let $\mu = E[X_n]$. Inequality $e^u \ge 1 + u$ implies $2^x \ge 2^\mu [1 + (\ln 2)(x - \mu)]$. Taking expectation cancels the linear term, yielding the needed Jensen inequality:

$$2^{E[X_n]} \le E[2^{X_n}] = a_n \le n^3 \quad \Longrightarrow \quad E[X_n] \le 3 \log_2 n.$$

The derivative of $e^u - 1 - u$ is $e^u - 1$, so its minimum is 0 at $u = 0$. This proves the tangent bound. Height has finite support, so these expectations exist.

A binary tree of edge height $h$ contains at most $2^{h+1} - 1$ nodes; thus every tree has $h \ge \log_2(n+1) - 1$. Together the bounds give $\Theta(\log n)$ expected height. A single run may still have worst-case height $n - 1$.



## Red-black trees control height through path invariants

A red-black tree is still a strictly ordered BST, with one extra color field. B means black and R means red. Figures include these labels as well as colors.

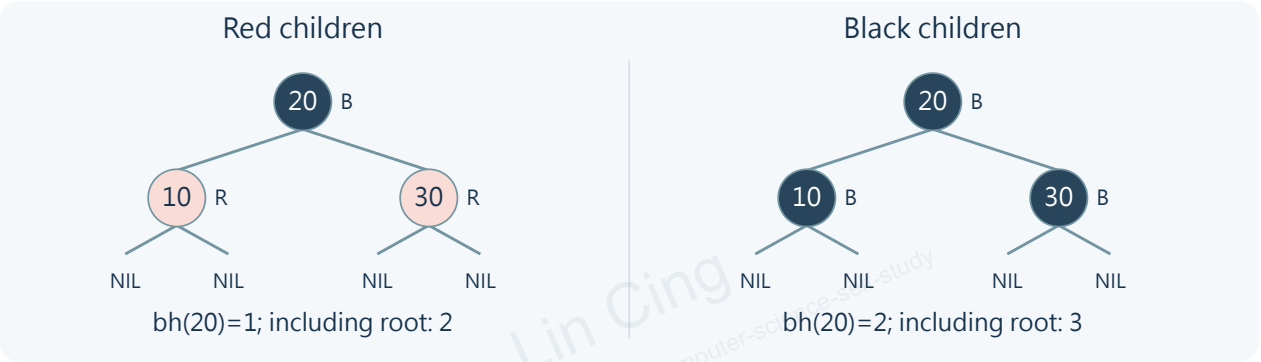
1. Each node is red or black, and the root is black.
2. Every external NIL position is black. A data node with no child still links to NIL.
3. Both children of a red node are black: consecutive red nodes are forbidden.
4. **From every node, all downward paths to NIL contain the same number of black nodes.** Checking only selected root–NIL paths is insufficient.

Use one shared sentinel with NIL.left=NIL.right=NIL, NIL.color=B, and root.parent=NIL. Require matching parent links only for non-NIL children. A duplicate-key insertion updates the value without changing shape or colors.

Earlier BST operations did not write NIL.parent. Deletion repair will temporarily use it for one deficient position. It cannot represent all missing-child parents simultaneously. Rotations still skip the parent write when $\beta$=NIL.

## Fix exactly which nodes black-height counts

Define $bh(x)$ as the black nodes on a downward path from x to NIL, **excluding x and including the terminal NIL**; set $bh(NIL) = 0$. The path invariant makes this well-defined.




For a non-NIL child c, a red c has $bh(c) = bh(x)$; a black c has $bh(c) = bh(x) - 1$. The parent's count includes c, whereas c's own black-height excludes c. For c=NIL the relation is $bh(x) = 1 + bh(NIL)$.



## Prove the height bound by counting required nodes

Retain internal edge height h: a singleton has h=0 and an empty tree h=−1. Separately let H count edges from root to external NIL. Then H=h+1 for a nonempty tree, and H=0 for an empty tree. Use H in the height proof without silently changing the meaning of h.

**Lemma:** The subtree at x contains at least $2^{bh(x)} - 1$ internal nodes. Induct on subtree height. For x=NIL, $0 = 2^0 - 1$. Otherwise each child has black-height at least bh(x)−1. Apply induction and count x:

$$n_x \geq 2\left(2^{bh(x)-1} - 1\right) + 1 = 2^{bh(x)} - 1.$$

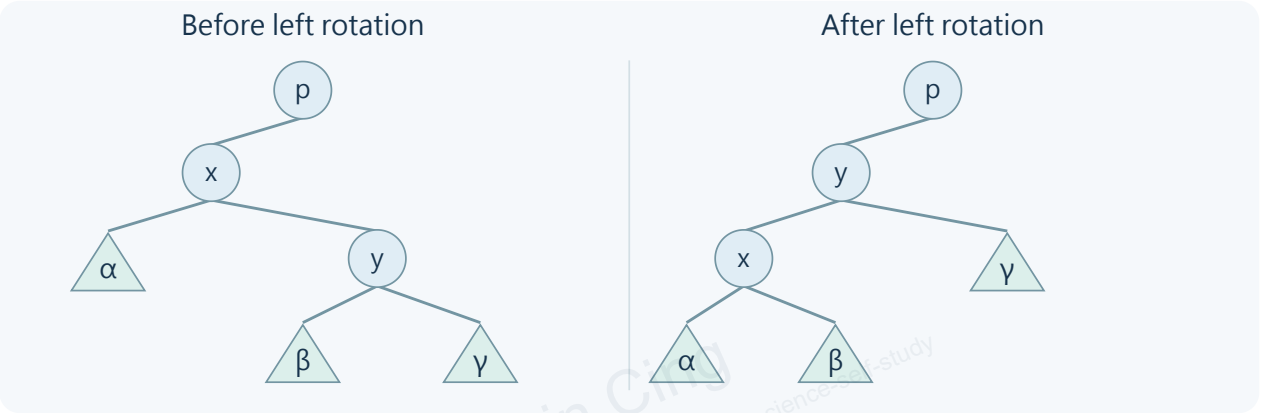
On a longest root–NIL path, let B count black nodes including both endpoints, and R count red nodes. Black endpoints and no adjacent reds imply $R \leq B - 1$. Its edge length is

$$H = B + R - 1 \leq 2(B - 1) = 2bh(root).$$

Hence $n \geq 2^{bh(root)} - 1 \geq 2^{H/2} - 1$ and $H \leq 2\log_2(n + 1)$. The empty tree has H=0. Searching and its terminating test cost O(H+1), or uniformly O(log(n+2)), including the empty-tree constant cost.

This bounds search height; update repair still needs a termination and cost argument. Red-black trees do not require child heights to differ by at most one: that is an AVL condition.

## Left rotation changes the parent of β and the child of p




Require x≠NIL and y=x.right≠NIL. The symbols α, β, γ denote entire subtrees, possibly NIL; p is x's **old parent**. Node y becomes the local root, and β moves from y.left to x.right.



## Complete, symmetric pointer updates for both rotations

```
LEFT-ROTATE(T,x):                RIGHT-ROTATE(T,y):
    p=x.parent; y=x.right            p=y.parent; x=y.left
    beta=y.left                      beta=x.right
    x.right=beta                     y.left=beta
    if beta!=NIL: beta.parent=x      if beta!=NIL: beta.parent=y
    y.parent=p                       x.parent=p
    if p==NIL: T.root=y              if p==NIL: T.root=x
    elif p.left==x: p.left=y          elif p.left==y: p.left=x
    else: p.right=y                  else: p.right=x
    y.left=x                         x.right=y
    x.parent=y                       y.parent=x
```

Save the old parent p. Skip the sentinel.parent write for β=NIL. Require x.right≠NIL or y.left≠NIL, respectively.

## Check each rotation write against a concrete tree

Take p=30, x=10, y=20, β=15, with p.left=x. Leave all other subtree contents unchanged.

| Step | Write | Purpose |
|---|---|---|
| 1 | 10.right=15 | Reconnect β |
| 2 | 15.parent=10 | Update β's parent |
| 3 | 20.parent=30 | Connect the new local root upward |
| 4 | 30.left=20 | Replace the old outside child slot |
| 5 | 20.left=10 | Make the old root a left child |
| 6 | 10.parent=20 | Update the old root's parent |

For a root rotation, step 3 becomes 20.parent=NIL and step 4 becomes T.root=20.

Intervals α<x<β<y<γ remain valid, preserving BST order and node identities. Keys and values do not change. **Rotation alone does not ensure red-black validity.**



## Insert a red leaf and identify the only possible violation

Use the previous INSERT search. An existing key only updates its value. Insert a new node z as R with left=right=NIL and a correct parent link.

Replacing a black NIL by a red z with black NIL children preserves every affected path's black count. Inserting a black node would add one black only to the paths through that position.

At loop entry maintain: **z is red; if its parent is the root, that parent is black; the only possible violation is a red root or the red pair z and its parent.** BST order and equal black counts remain valid.

A black parent needs no repair. A new root is simply blackened at the end. A red parent p enters the loop and cannot be the root, so grandparent g exists and is black. The uncle u is g's other child; it may be NIL, whose color is safe to read.

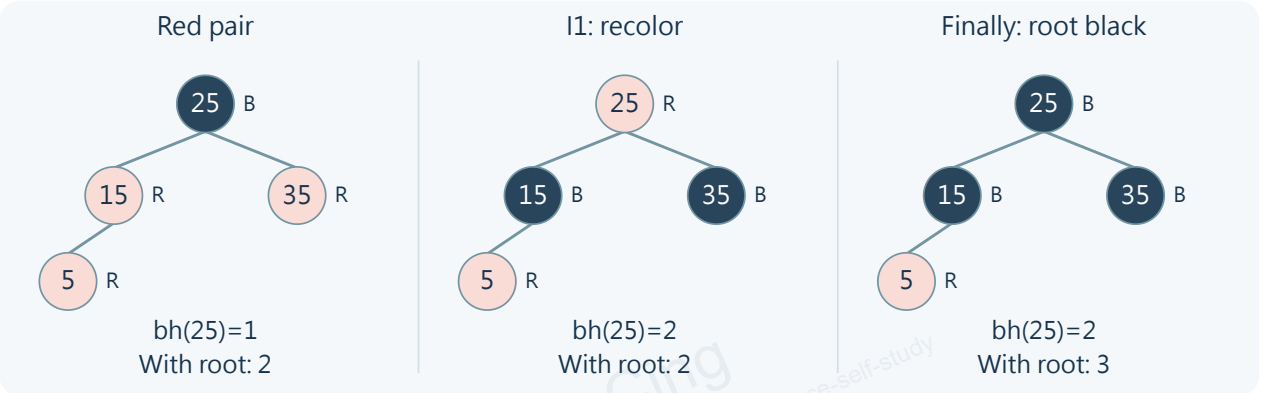
I2 is not an alternative terminating case: it falls through into I3. Rotations change parent links. I3 uses the current z.parent and grandparent, not cached pre-rotation p and g.

## Insertion repair when the parent is a left child

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:               // I2
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                    // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: apply the explicitly mirrored branch
T.root.color=B
```



## I1 preserves the black count seen by ancestors




Insert 25,15,35, then 5. The new 5R conflicts with parent 15R; uncle 35R is red too. Color 15 and 35 black, color 25 red, and move z to 25.

## I1: which black count changes?

Let every path below p/u, excluding p/u, contribute b black nodes. A complete path through g originally contributes 1+b, and still contributes 1+b after g becomes red and p/u become black. **The ancestor-to-NIL black count through g stays unchanged**, although bh(g), which excludes g, rises from b to b+1.

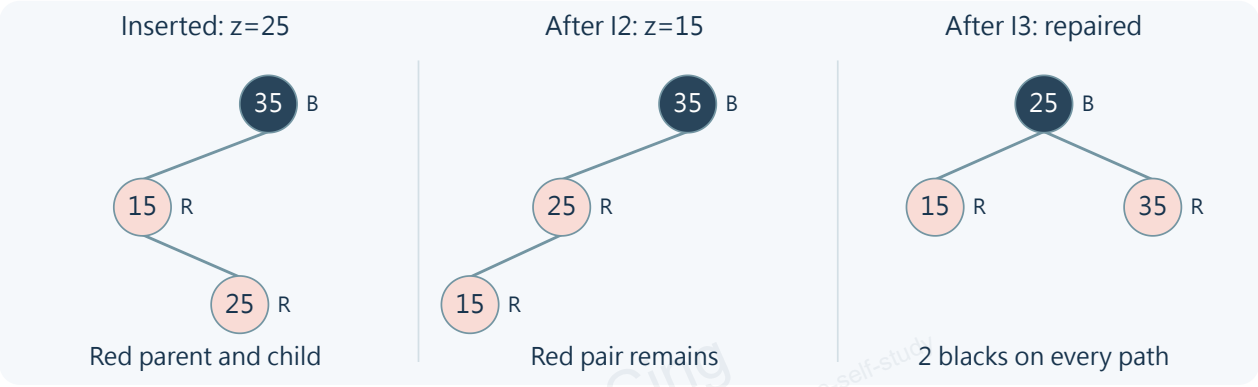
Only g and its parent can now form a red pair. If g is the root, the final root-blackening step adds one black to all root–NIL paths. That final step must not be applied to a non-root g.

| Counting range | Before I1 | After I1 |
|---|---|---|
| bh(g), excluding g | b | b+1 |
| g through NIL, including g | 1+b | 1+b |
| Ancestor through g to NIL | outside count+1+b | outside count+1+b |

For root g=25 and b=1, I1 leaves bh(25)=2 and a full-root count of 2. Final root blackening raises only the latter to 3.



## I2 followed by I3: straighten the bend, then repair it

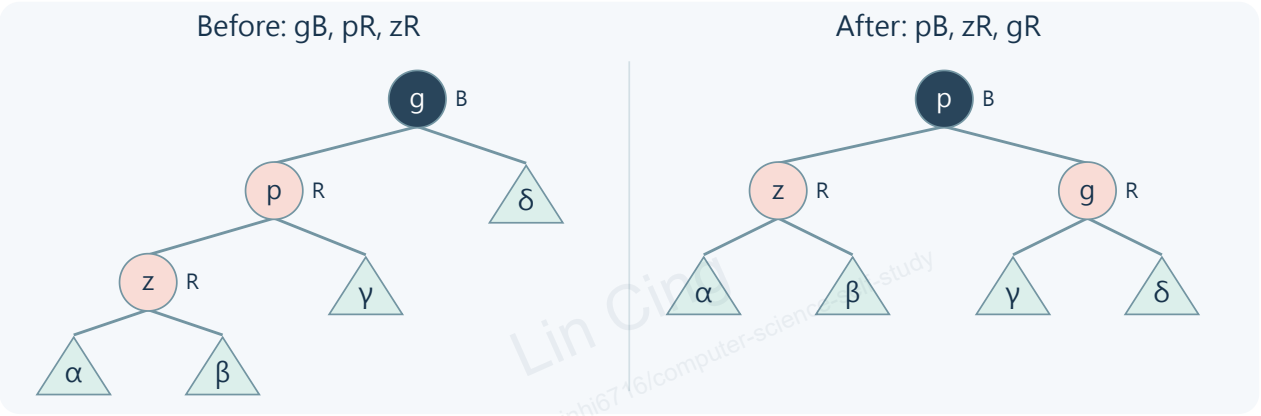



Insert 35,15,25. The uncle is black NIL; parent 15 is a left child and the new 25 is a right child. I2 first sets z to the old parent 15 and left-rotates 15. Now 25R is the parent of 15R: **the red pair still exists**.

I3 blackens the current parent 25, reddens grandparent 35, and right-rotates 35. Result: 25B with 15R,35R, and two blacks per path. Do not return after I2.

## Why I3 preserves black counts and removes the red pair

Start with gB-left pR-left zR and a black uncle. Let α,β be z's children, γ be p's old right subtree, and δ be g's old right subtree. Blacken p, redden g, and right-rotate g.






## I3: verify every repaired invariant

Red z forces α,β to have black roots; red p forces γ black; δ is the black uncle. Equal path counts make every boundary subtree contribute b blacks, including its root.

| Destination subtree | Local black nodes before | Local black nodes after |
|---|---|---|
| α,β | gB, then b | pB, then b |
| γ | gB, then b | pB; gR adds zero; then b |
| δ | gB, then b | pB; gR adds zero; then b |

All four destinations retain 1+b blacks. Each boundary subtree keeps its internal links and colors, preserving its own path equalities.

New local root p is black, so no outside red pair is possible. Red z has black children α,β; red g has black children γ,δ. An outer insertion chain skips I2; the same proof applies.

## Write out the mirrored right-parent branch

This replaces the preceding code's else branch when z.parent is the grandparent's right child. The uncle is now on the left, and both rotation directions reverse.

```
u=z.parent.parent.left
if u.color==R:                              // I1 mirrored
    z.parent.color=B; u.color=B
    z.parent.parent.color=R
    z=z.parent.parent
else:
    if z==z.parent.left:                     // I2 mirrored
        z=z.parent
        RIGHT-ROTATE(T,z)
    z.parent.color=B                        // I3 mirrored
    z.parent.parent.color=R
    LEFT-ROTATE(T,z.parent.parent)
```



## Why insertion needs at most two rotations

I1 moves z upward by two levels without rotating, at most O(H) times. I2 falls through to I3, which blackens the local root and terminates. A black parent also terminates the loop; final root blackening resolves the remaining root-color case.

| Branch | Primitive rotations | What follows |
|---|---|---|
| I1: red uncle | 0 | Move z up two levels; may repeat |
| I2: black uncle, inner child | 1 | Immediately perform I3 |
| I3: black uncle, outer child | 1 | Black local root; terminate |
| Final root blackening | 0 | Done |

The rotating suffix is either I3 or I2→I3. Repeated I1 steps add recolorings, not rotations.

Therefore there are at most two primitive rotations in total. Search plus recoloring costs O(log(n+2)); a new node uses O(1) space.

## Two repair schedules must not be mixed

| Comparison | Bottom-up here | Top-down 2-3-4 splitting |
|---|---|---|
| Two red children | Insert a red leaf, then repair upward | Split/recolor a 4-node during descent |
| Possible rotation | Black-uncle I2/I3 | A red pair created during downward repair |
| Shared invariants | BST order, no red pair, equal black counts | Same goals, different repair-entry conditions |

These algorithms may produce different valid shapes. Do not mix their recoloring schedules while retaining one algorithm's proof. A “double-rotation adjustment” may count as two primitive rotations; fix the unit and algorithm before comparing counts.

Coloring the edge to each red child gives a 2-3-4 interpretation after contracting red edges. General red-black trees allow either red-edge direction. Left-leaning red-black trees impose an additional convention.



## Distinguish z, y, x, and y's original color

z is the record to remove. If z has fewer than two children, y=z; otherwise y is the minimum in z.right and moves as a whole object into z's position. x is the sole possible child filling **y's old position**, namely y.right for a successor with no left child; when y=z, use z's only child or NIL. Save yOriginalColor before giving moved y z.color. Missing black depends on yOriginalColor, not y's new color or merely z.color.

```
RB-TRANSPLANT(T,u,v):
    if u.parent==NIL: T.root=v
    elif u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent              // write even when v==NIL
```

NIL.parent now records the parent of the current empty x position so repair can find its sibling. It is not a universal parent for all empty links. The root's parent is NIL. Rotations skip beta.parent for beta=NIL to preserve this context. After a stable operation, NIL.parent may be reset to NIL; NIL.color must always remain black.

## RB-DELETE: relocate first, then inspect the original color

```
RB-DELETE(T,z):                  // z is an existing non-NIL record
    original=z.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(T,z,x)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(T,z,x)
    else:
        x,original=RELOCATE-SUCCESSOR(T,z)
    if original==B: DELETE-FIXUP(T,x)
    NIL.parent=NIL                // only after repair finishes
    reclaim z
```

Only object z is removed; all other identities and key/value pairs survive. A red original color removes no black; a black original color starts repair at replacement x. A key-based API must first search and return absent on failure. Never call this procedure on NIL.



## Two children: preserve the successor's original color

```
RELOCATE-SUCCESSOR(T,z):          // requires two non-NIL children
    y=MINIMUM(z.right); original=y.color; x=y.right
    if y.parent==z:
        x.parent=y                // even when x==NIL
    else:
        RB-TRANSPLANT(T,y,x)
        y.right=z.right; y.right.parent=y
    RB-TRANSPLANT(T,z,y)
    y.left=z.left; y.left.parent=y
    y.color=z.color
    return x,original
```

## Successor relocation: trace every changed link

Insert 40,20,60,10,30,50,70,45,55. The tree has root 40B, left 20B(10R,30R), and right 60R with left 50B(45R,55R) and right 70B. Deleting z40 chooses y45R and x=NIL, with y.parent 50≠z.

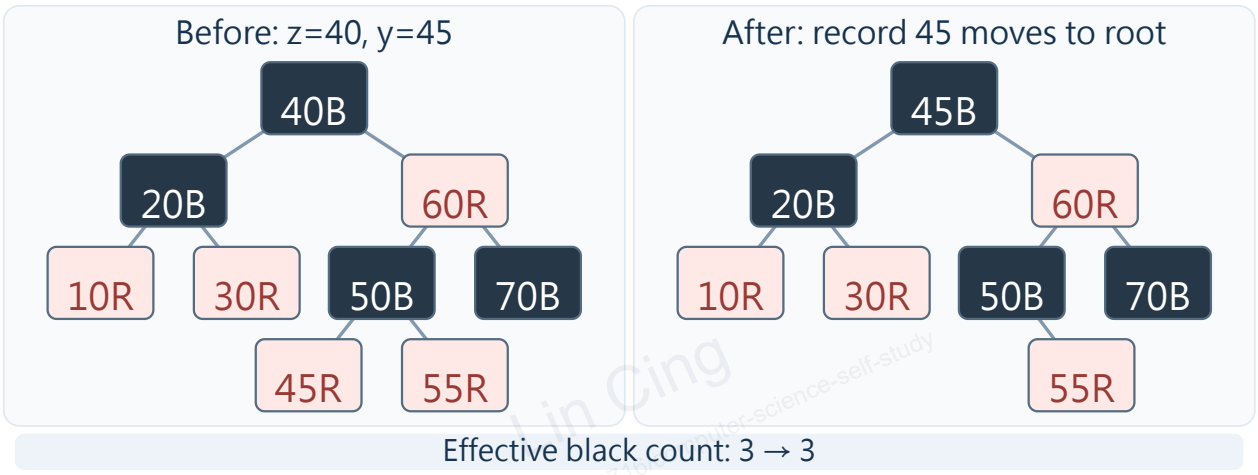
Remove 45 from 50.left, giving 50.left=NIL and NIL.parent=50. Attach 60 as y.right and set 60.parent=45. Replace root 40 with y, then attach 20 as y.left and set 20.parent=45. Give y color B. Since y's original color was R, no black repair runs. Root is 45B, original 45's data moves with 45, and only record 40 is removed.

Successor y has no left child, so only y.right can fill its old position. If y is z's immediate right child, x's new parent is y, not the soon-reclaimed z. For a deeper y, the first transplant sets x.parent; moving y afterward must preserve that context.

Assigning y.color=z.color preserves the contribution at z's old position. The repair decision depends on **y's color before relocation**, a separate quantity.



## A deeper successor: before and after relocation




Both panels show the same deletion: node 45 moves from the left of 50 to the root. Its old position is filled by x=NIL. The first transplant sets NIL.parent=50; moving 45 must preserve that context until repair or final cleanup resets NIL.parent=NIL. Omitted empty children are black NIL nodes.

## Track a missing black at x without a third color

Removing an originally red y reduces no path black count. Giving moved y z's old color preserves the other local relationships, so no fix is needed. If y was black, temporarily assign its missing black to x's position. Count one extra black on each path through x to match the pre-deletion effective counts.

This proof-only extra black does not change the actual R/B color domain. A red x absorbs it by becoming black. At the root, drop it from all tree paths together. Otherwise the loop has a black nonroot x. Its sibling w cannot be NIL: w would contribute only one ordinary NIL black while x's side contributes at least NIL plus extra, contradicting equal effective black counts.

Below, x=p.left, w=p.right, near=w.left, and far=w.right. A right-side deficit exchanges left and right, giving near=w.right and far=w.left, and reverses rotations.

Figure legend: x +1 marks the proof-only extra black, not a third stored color. Unshown missing children are black NIL. Complete-path counts include the starting node and NIL.



## The repair cursor tracks the unique extra-black position

```
DELETE-FIXUP(T,x):
    while x!=T.root and x.color==B:
        if x==x.parent.left: x=REPAIR-LEFT(T,x)
        else: x=REPAIR-RIGHT(T,x)
    x.color=B
```

The helpers return the next repair position: D2 returns the parent; D4 returns the root to terminate. Final blackening absorbs the extra black when x is red. When x is already the black root, remove the proof's extra contribution from every path without changing its stored color.

## Left-side deficit: the full D1–D4 control flow

```
REPAIR-LEFT(T,x):
    p=x.parent; w=p.right
    if w.color==R:                          // D1
        w.color=B; p.color=R; LEFT-ROTATE(T,p)
        w=p.right
    if w.left.color==B and w.right.color==B:  // D2
        w.color=R
        return p
    if w.right.color==B:                    // D3
        w.left.color=B; w.color=R; RIGHT-ROTATE(T,w)
        w=p.right
    w.color=p.color                         // D4
    p.color=B; w.right.color=B; LEFT-ROTATE(T,p)
    return T.root
```



## Right-side deficit: write the mirror explicitly

```
REPAIR-RIGHT(T,x):
    p=x.parent; w=p.left
    if w.color==R:                          // D1 mirrored
        w.color=B; p.color=R; RIGHT-ROTATE(T,p)
        w=p.left
    if w.left.color==B and w.right.color==B:  // D2 mirrored
        w.color=R
        return p
    if w.left.color==B:                     // D3 mirrored
        w.right.color=B; w.color=R; LEFT-ROTATE(T,w)
        w=p.left
    w.color=p.color                         // D4 mirrored
    p.color=B; w.left.color=B; RIGHT-ROTATE(T,p)
    return T.root
```

## Select cases by sibling and near/far colors

| Case | Condition, after handling a red sibling | Next state |
|---|---|---|
| D1 | Red sibling | Black sibling; continue into D2/D3/D4 |
| D2 | Black sibling, both children black | Move the deficit to the parent |
| D3 | Black sibling, red near/black far child | Convert to D4 |
| D4 | Black sibling, red far child | Remove extra black and terminate |

For x on the left, near=w.left and far=w.right. Refresh w after D1 and D3. D3 falls through into D4. Returning p in D2 hands control back to the outer loop; it does not declare deletion complete.

For x on the right, near=w.right and far=w.left. Define these geometrically relative to x; near does not always mean left. Neither helper changes NIL.parent: the transplant establishes that context until repair finishes.

Shared NIL does not make the side test ambiguous here: the loop invariant requires a non-NIL sibling, so the parent cannot have NIL on both sides.



## D1 converts a red sibling to a black sibling — black-count proof

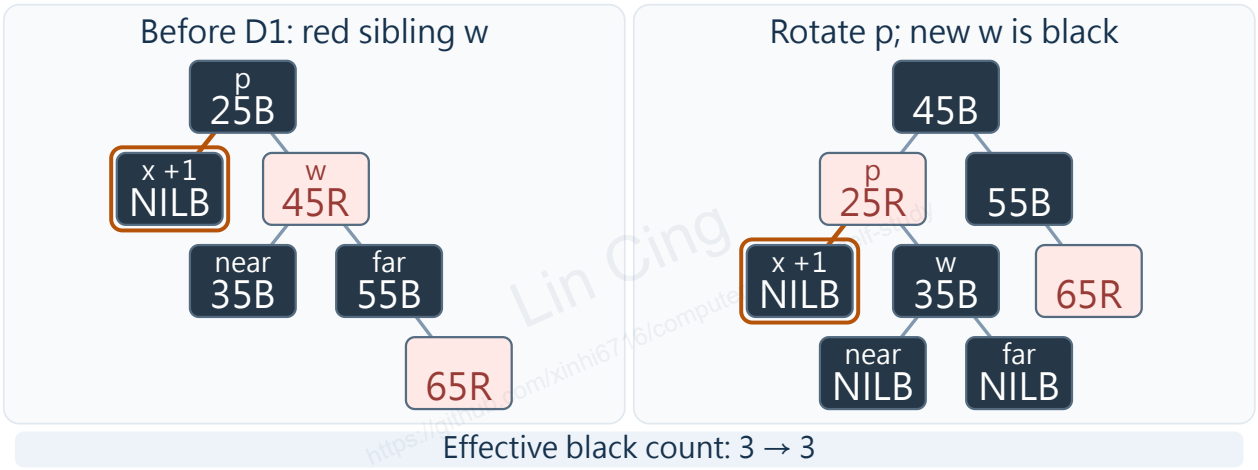
Let x's **actual** complete black count be b, including x, or NIL if x=NIL. Extra makes it b+1. Red w forces p and both w children black. Each w-child path contributes b+1, so every effective complete path including p contributes b+2.

Make w black and p red, then left-rotate p. New black local root w has a red left child p whose x-side effective count b+1 equals the old near-child count b+1. Its right side, the old far child, also contributes b+1. Total remains b+2. Extra remains at x; its new sibling is w's old black near child. Proceed to D2, D3, or D4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## D1 converts a red sibling to a black sibling — worked trace

Insert 15,25,35,45,55,65 and delete 15. After removal x=NIL is 25B.left and w45R. D1 makes 45 black and 25 red, then left-rotates 25. New root 45B has left 25R(right 35B) and right 55B(right 65R); x remains 25.left. Complete black counts through ancestors are unchanged.





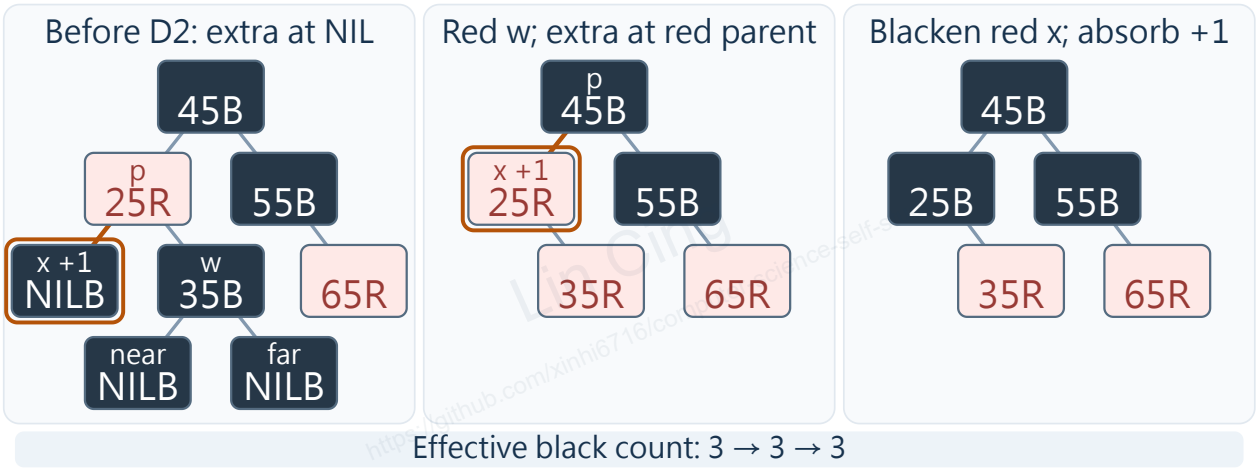

## D2 moves the deficit upward or absorbs it at a red parent — black-count proof

Now w and both its children are black. x contributes actual b and effective b+1; w contributes b+1 and each child b. Making w red reduces its actual contribution to b. Both branches now have equal **actual** counts, but the whole p-subtree is one black short relative to its ancestors. Move extra to p by setting x=p.

If p was red, final blackening absorbs the extra and stops. If p was black and nonroot, iterate upward. At the root, drop extra consistently from all paths. D1 just made p red, so D1 followed by D2 absorbs in this round and cannot climb to another D1.

## D2 moves the deficit upward or absorbs it at a red parent — worked trace

Continue the example: w35B has NIL children. Make 35 red and move x from NIL to 25R. The loop exits and 25 becomes black. Finally 45B has left 25B(right 35R) and right 55B(right 65R).

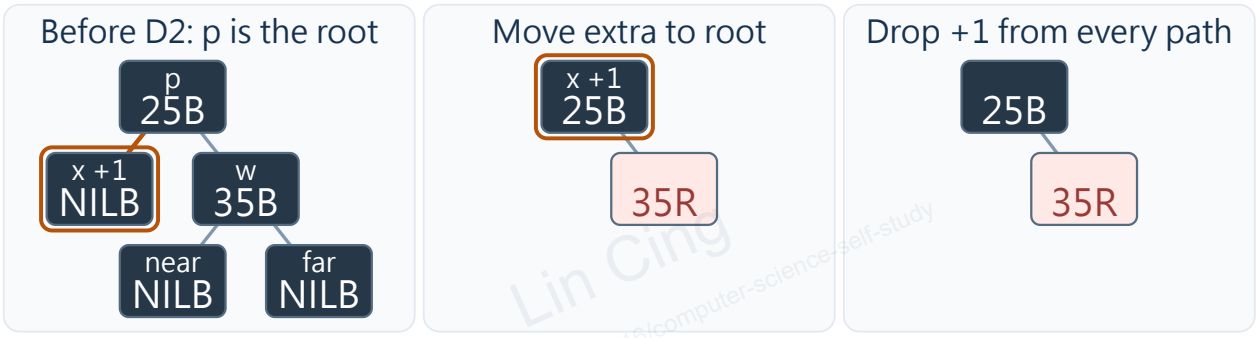



Effective black count: 3 → 3 → 3



## D2 at the root reduces every path by one black

Start with 25B and black leaves 15B,35B. Deleting 15 leads D2 to make 35 red and move x to root 25. Dropping extra from all root-leaf paths leaves 25B(right 35R). Root bh decreases from 2 to 1, with equal path counts. This differs from absorption by blackening a red parent.




Effective black count: 3 → 3 → 2

## D3 moves an available red node to the far side — black-count proof

w is black, near N is red, and far is black. Let x's actual count be b. Both w children contribute b; red N's children each contribute b, as does far. Make N black and w red, then right-rotate w. New black sibling N has old N.left contributing b and old red w whose old N.right and far also contribute b. Its total remains b+1, preserving everything above p.

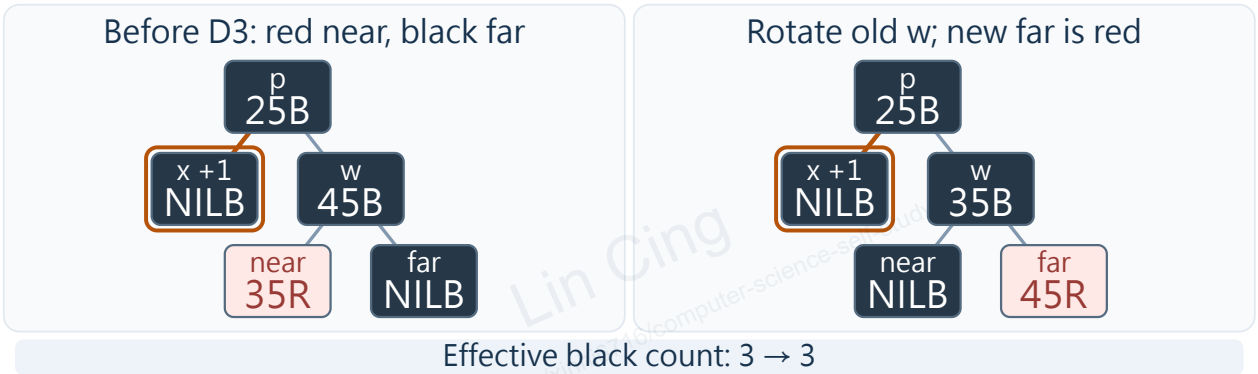
New sibling's far child is old red w, so execute D4 immediately. Extra remains at x; this case neither moves the deficit upward nor terminates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## D3 moves an available red node to the far side — worked trace

Insert 15,25,45,35 to obtain 25B with left 15B and right 45B(left 35R). Delete 15, leaving x=NIL, w45B, near 35R, and black far=NIL. Make 35 black and 45 red, right-rotate 45, and obtain 25B with right 35B(right 45R). D4 must follow.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## D4 repairs both sides by recoloring and rotation — black-count proof

Let p's original black contribution be c∈{0,1}, x's actual count b, and both children of black w contribute b; far is red. Give w p's old color, make p and far black, and left-rotate p. New root w has black left child p with equal actual b on x and near, so its complete left count is b+1. Far changes red to black and also contributes b+1. New local total is c+b+1, equal to original p's effective c+(b+1).

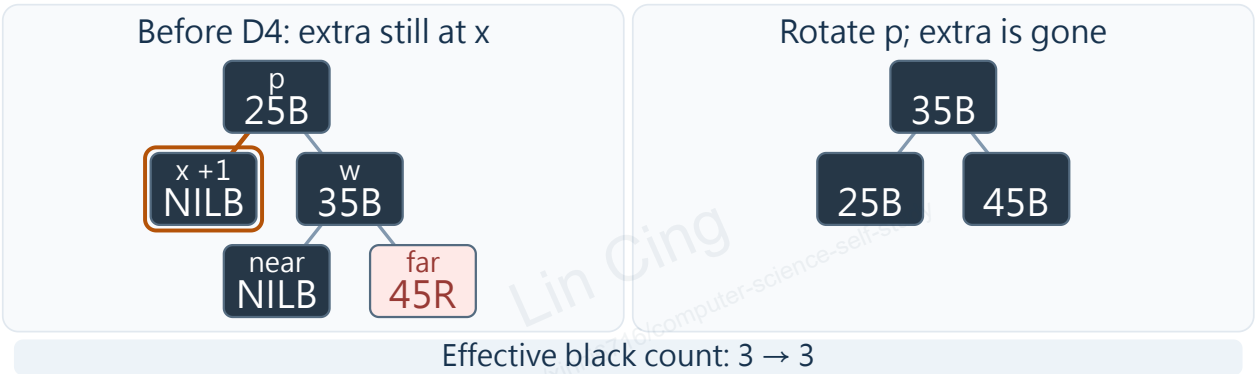
If new w is red, its children p and far are black. Making p black creates no red-red conflict, and w retains p's external color relationship. Extra is eliminated. Set x=root to stop and ensure the root is black.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



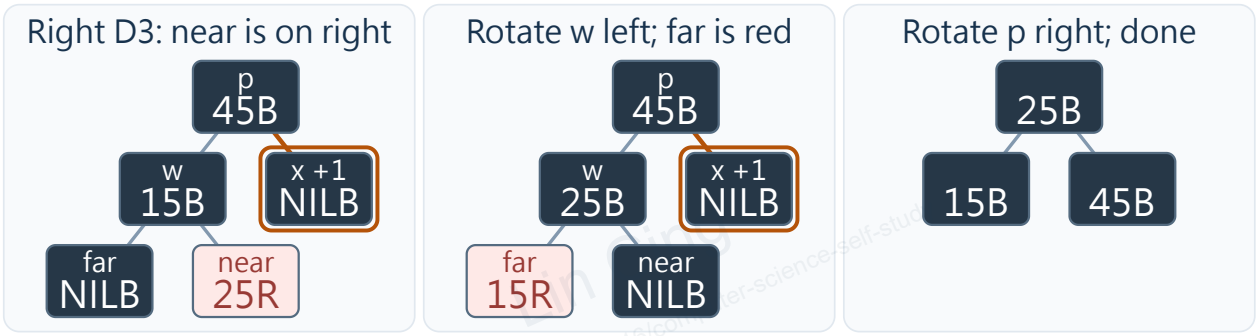
## D4: finish the numeric example

Continue D3:35 takes 25's old black color;25 and far 45 become black. Left-rotate 25 to obtain 35B with 25B,45B children. Inorder is 25,35,45, and every root-NIL path contains 3 blacks with no extra.



## Right-side D3 → D4

Mirror example: insert 15,35,45,25, then delete 35. Direct successor 45B becomes root; x=NIL is 45.right, so NIL.parent must be 45. Sibling 15B has near 25R and far=NIL. Right-side D3 left-rotates 15, then right-side D4 right-rotates 45, yielding 25B with 15B,45B children.






## Count black nodes in the earlier left-side D3→D4 example

Every row **includes its starting node and NIL**. D3 leaves x=NIL in place; D4 removes the extra black.

| Stage and path | Actual black | Extra | Effective total |
|---|---|---|---|
| Before D3: 25B → NIL(x) | 2 | 1 | 3 |
| Before D3: 25B → 45B → 35R → NIL | 3 | 0 | 3 |
| After D3: 25B → 35B → 45R → NIL | 3 | 0 | 3 |
| After D4: 35B → 25B → NIL | 3 | 0 | 3 |
| After D4: 35B → 45B → NIL | 3 | 0 | 3 |

Each black node adds 1 and each red node adds 0. Add the extra only on paths through x. The other NIL paths also total 3 by the same rule; check them in the figures. After D4 both sides have the needed black without an extra.

## From local conservation to the general proof and cost

BST relocation preserves order and every undeleted record. Removing red directly leaves a valid tree. Removing black assigns extra to x so effective ancestor-to-leaf counts remain balanced. D1 and D3 preserve its position and effective counts; D2 moves the sole extra upward; D4 eliminates it. A red x is repaired by final blackening. In D2, newly red w has black children; rotations and recolorings satisfy the checked local color conditions. At termination all five properties hold.

D2 ascends one level with no rotation, at most O(log(n+2)) times. After D1, p is red, so D2 absorbs, or D3/D4 terminates. Therefore D1, D3, and D4 occur at most once each, for at most 3 rotations. Searching for z and its successor plus relocation and repair totals O(log(n+2)). NIL, root, and red-child absorption are valid base cases; deleting the last node leaves black root=NIL.

The rotation bound limits structural rearrangements. D2 propagation and recoloring may still traverse an ancestor path, so complete deletion takes O(log(n+2)), not O(1).



## Dynamic order statistics from subtree sizes

Repeated updates and rank/select queries motivate an **order-statistic tree**: a balanced search tree augmented with subtree size, including the node itself. NIL.size=0 and

$$\text{size}(x) = 1 + \text{size}(x.left) + \text{size}(x.right).$$

Ranks are 1-based inorder positions of node identities. Examples use distinct keys; duplicates require a declared order such as (key,identity). A value alone does not identify one duplicate's rank.

An augmentation must specify the summary, its maintenance after every modification, and the correctness of new queries. Here summaries use local data and child summaries in O(1) and do not depend on colors; color-dependent summaries require separate recoloring costs. First bound costs by height h: O(h+1). Red-black invariants then give O(log(n+2)); adding a field does not balance an ordinary BST.

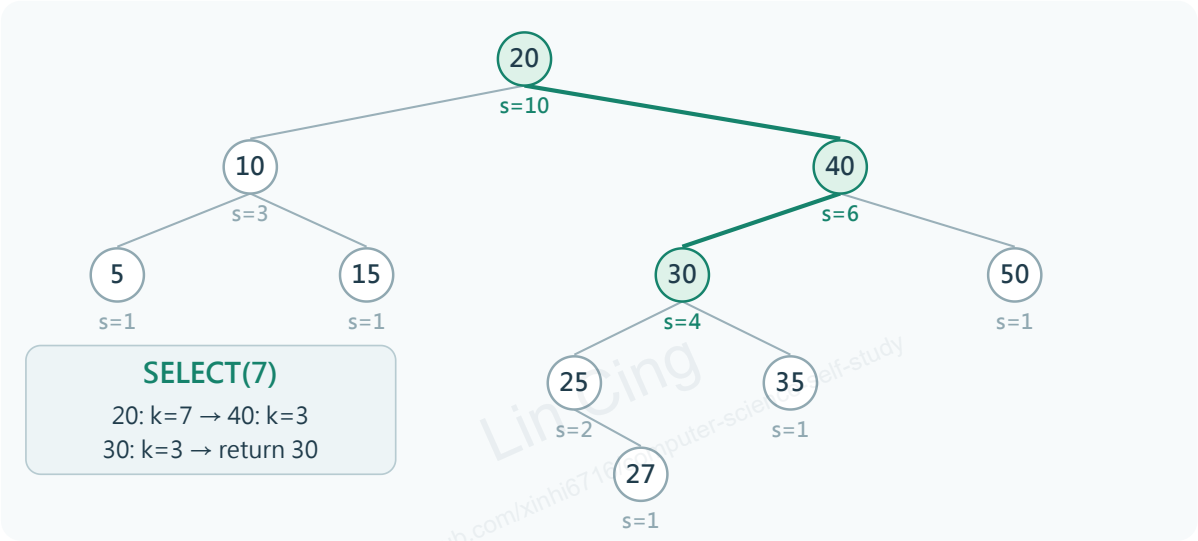
## Select uses a local rank at every level

```
select(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

Invariant: the answer is local rank k in the current x-subtree. The left subtree has r−1 nodes and x is position r. Going left retains k; going right subtracts the entire left subtree plus x, r nodes. A valid k guarantees the chosen side is nonempty and its new k valid, so descent never incorrectly enters NIL. Each iteration descends one level and returns the correct node.



## One tree: keys, subtree sizes, and the selection path




Keys appear inside circles; s below each circle is subtree size. Green traces SELECT(7): 20→40→30. Rank and deletion examples start from the same tree.

## Select(7): subtract only the skipped prefix

| Current key | k | left.size | r=left.size+1 | Action and new k |
|---|---|---|---|---|
| 20 | 7 | 3 | 4 | Right to 40; 7−4=3 |
| 40 | 3 | 4 | 5 | Left to 30; still 3 |
| 30 | 3 | 2 | 3 | Return this node |

Inorder is 5,10,15,20,25,27,30,35,40,50, confirming key 30 at rank 7. **Subtract the left subtree and current node only when moving right.** Reject k=0 or k>root.size before querying. An empty tree has no valid k.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rank adds disjoint prefixes while climbing

```
rank(T,x):
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

Invariant: r is original x's inorder rank inside current u's subtree. If u is a left child, the parent and right side follow x, so add nothing. If u is a right child, its parent's left subtree and parent precede x and lie outside the previously counted u-subtree. Add exactly left.size+1 without duplication. At root, r is the global rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rank(27): which ancestors add a prefix?

Find the rank of node 27, starting with r=0+1=1.

| u → parent | u's side | Added prefix | New r |
|---|---|---|---|
| 27 → 25 | right | size(NIL)+1=1 | 2 |
| 25 → 30 | left | 0 | 2 |
| 30 → 40 | left | 0 | 2 |
| 40 → 20 | right | size(10)+1=4 | 6 |

Each added set lies outside the old u subtree and precedes 27. The sets are disjoint and exhaustive, confirming inorder rank 6. This API requires a valid node belonging to the tree; NIL and removed objects are invalid arguments.



## Repair summaries bottom-up, including during rotations

```
PULL(v):                      # v != NIL
    v.size=1+v.left.size+v.right.size
    v.max=max(v.high,v.left.max,v.right.max)
REPAIR-UP(v):
    while v!=NIL:
        PULL(v); v=v.parent
```

This version stores both size and the largest interval high; omit max in a size-only tree. NIL.size=0 and NIL.max=−∞. A new leaf has size 1; after attaching it, repair upward from its parent.

For deletion, finish BST relocation, repair along final parent links, and then run red-black fixup. After changing rotation pointers, pull the lowered node before the promoted node. Recoloring does not affect size/max. Deletion or a shorter interval may decrease max; keeping a historical upper bound is unsafe.

## Where does summary repair start after deletion?

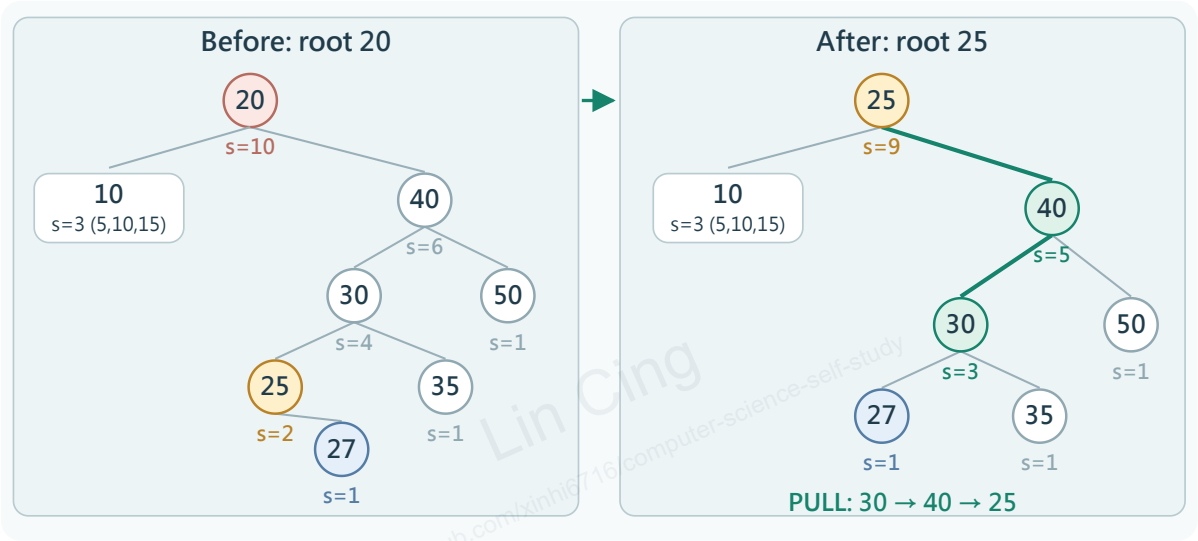
In the preceding RB-DELETE/RELOCATE-SUCCESSOR, save a before relocation according to the table. After all transplants, run REPAIR-UP(a), then repair colors.

| Branch | Saved a | Why this final chain suffices |
|---|---|---|
| z has at most one child | z's old parent | The child's own subtree is unchanged; ancestors lose one record |
| Successor y is z.right | y | y has a new left subtree; repair upward from y |
| Successor y is deeper | y's old parent | Its final chain visits the old right subtree, moved y, and z's old ancestors |

Never read z.parent after freeing z. NIL.parent is not a universal parent for all empty links. Save a separately as a real node or NIL; NIL.size=0 and NIL.max=−∞ never change.



## Delete 20 and relocate 25: track links and sizes




Node 25 keeps its identity;27 replaces it under 30, and 25 takes the old root position. Colors distinguish transplant roles, not red-black colors. Complete BST relocation and summary repair before the preceding red-black fixup.

## Pull 30, then 40, then the new root 25

Successor 25 has right child 27. Replace 30.left by 27, then move 25 to the root with old left subtree 10 and right subtree 40.

| Repair order | Calculation | New size |
|---|---|---|
| 30 | 1+size(27)+size(35)=1+1+1 | 3 |
| 40 | 1+size(30)+size(50)=1+3+1 | 5 |
| 25 | 1+size(10)+size(40)=1+3+5 | 9 |

The old size of 25, namely 2, is obsolete. Pulling 25 first reads stale 40.size=6 and wrongly produces 10. Merely decrementing z's old ancestors updates nothing here because z was the root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## When can a rotation update only two summaries?

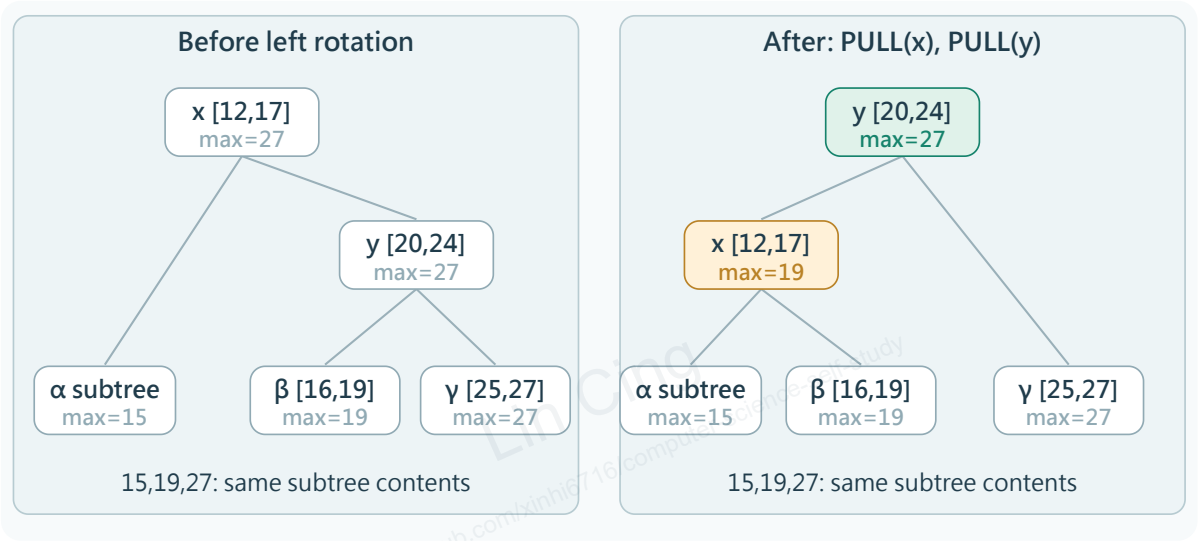
In left-rotate(x), y=x.right rises and β=y.left moves to x. Finish pointer changes, then pull(x) before pull(y), because new y depends on new x. Reversing the order reads stale x information.

Size and maximum interval high depend only on subtree contents. New y contains exactly old x's entire subtree after rotation, so its summary equals old x's summary and other ancestors do not change. These two summaries need O(1) local recomputation.

A local recurrence alone does not imply a two-node update. Height can change at the local root after rotation, requiring ancestor propagation. A general constant-time child-based summary costs O(h+1) per rotation; the constant rotation bounds of red-black updates retain O(log(n+2)) overall. Storing global ranks instead would require Θ(n) explicit writes when a new minimum is inserted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The same interval set before and after rotation




First x.max=max(17,15,19)=19; then y.max=max(24,19,27)=27. The new local root still represents the entire old set.



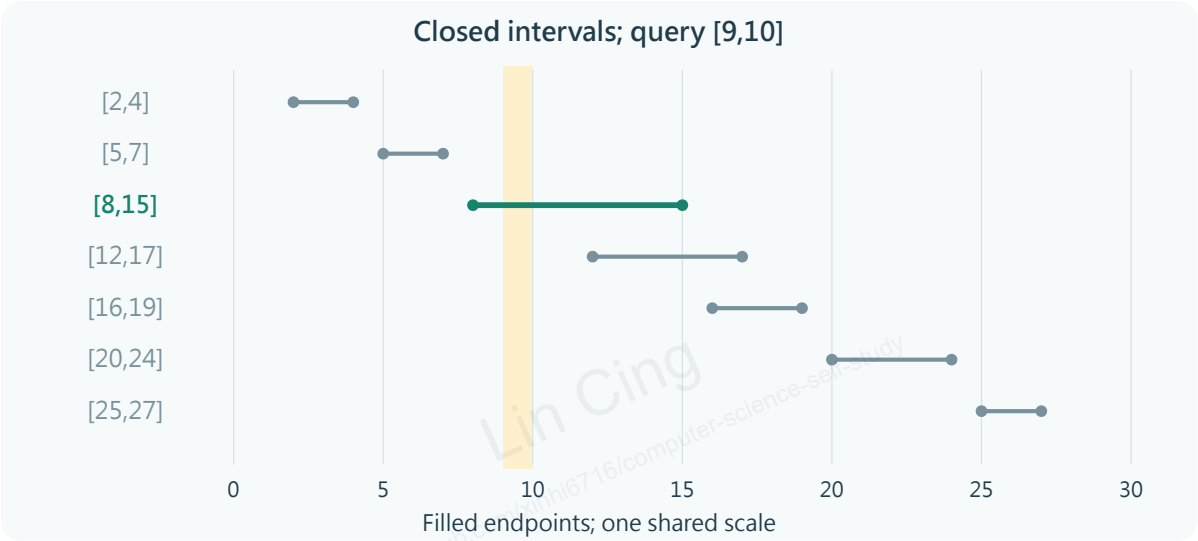
## Interval trees: order by low, summarize high

Intervals here are closed [l,h], with l≤h, including singletons [l,l]. Intervals i,q overlap exactly when i.low≤q.high and q.low≤i.high. Equivalently, neither lies completely left of the other: i.high<q.low and q.high<i.low are the two rejection cases.

Order the BST primarily by low; use identity to break equal-low ties, retaining left.low≤x.low≤right.low. Store max(x)=max(x.high,left.max,right.max), with NIL.max=−∞. This is the highest high in the **whole subtree**, not the node's high or the largest low.

Root [12,17] has high 17 but max 27. Left[5,7] has max 15 from right descendant [8,15]. Looking only at the current interval can miss a farther-reaching descendant.

## All seven intervals share one coordinate scale




The shaded query is [9,10]; only [8,15] overlaps it. The tree uses exactly these intervals.

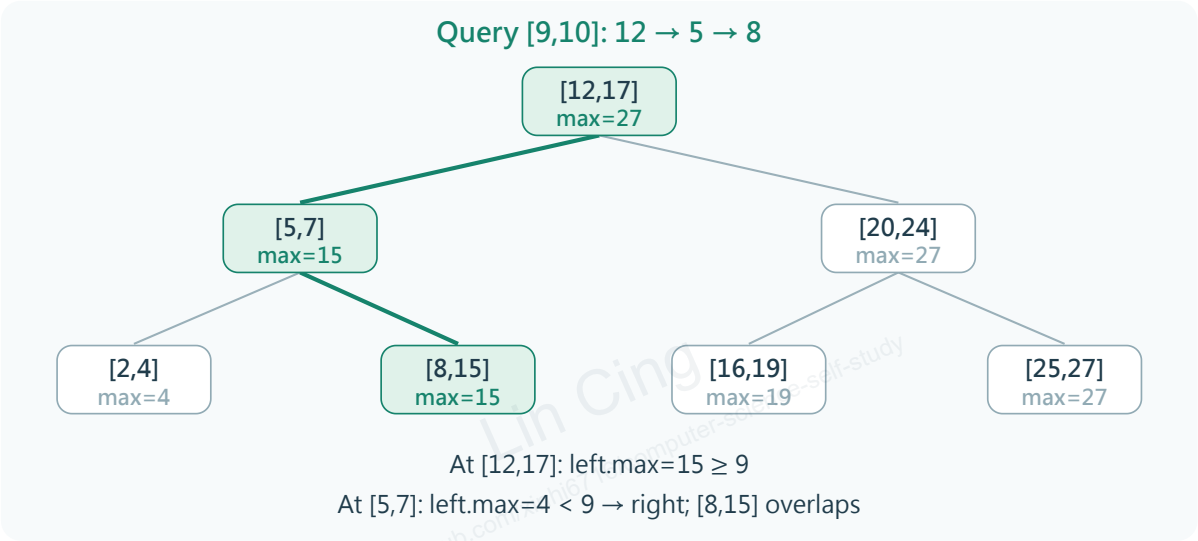


## Overlap search chooses one subtree at a time

```
interval-search(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

This returns any one overlap, not the leftmost or all matches. Prove that every chosen subtree retains an answer whenever the current subtree has one.

## One interval tree and the query [9,10] path




Read left.max at the current node: 15≥9 chooses left; then 4<9 chooses right and reaches [8,15].



## Three queries with traceable branch conditions

Query [9,10] does not overlap root [12,17], but left.max 15≥9 leads to [5,7]. That also misses; left [2,4].max 4<9 leads right to overlapping [8,15].

Query [18,18] goes from nonoverlapping root right because left.max 15<18. At [20,24], left.max 19≥18 leads to matching [16,19]. Query [28,29] follows [12,17]→[20,24]→[25,27]→NIL and fails.

## Going right: rule out the entire left subtree

Maintain: if an overlap exists anywhere in the tree, the current x-subtree contains one. This holds initially at root. Return if x overlaps. Otherwise prove that the chosen direction cannot discard all answers.

If left=NIL, it has no candidate. If left.max<q.low, every left interval j has j.high≤left.max<q.low and lies completely left of the query. Exclude the whole left side. Since x also misses, any answer must be right, preserving the invariant.

This uses a bound on every left descendant's high. The single condition x.high<q.low does not imply every left interval lies too far left. Change only the example root to [12,13] and query [14,14]: root.high 13<14, yet left descendant [8,15] overlaps, so the left side cannot be pruned on that basis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Going left: why no right-side backtracking is needed

When left.max≥q.low, choose a left interval j attaining that maximum. If left has any overlap, descending left is safe. If it has none, j.high≥q.low while j misses, so j.low>q.high; it cannot have j.high<q.low.

For every right descendant k, BST order gives `k.low≥x.low≥j.low>q.high`. The whole right side also lies too far right. With x itself nonoverlapping, this means the x-subtree had no answer. Thus either way, if an answer existed, the chosen left side retains one.

This proves the one-path algorithm. It does not assert that left.max≥q.low guarantees a left overlap. At NIL the chosen subtree is empty, so the invariant implies global absence. Each step descends once, giving O(height+1), or O(log(n+2)) with a red-black foundation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## An overestimated stale max can miss an answer

Change [8,15] to [8,27], then delete it. The left subtree now contains only [5,7] and [2,4], so its exact max is 7. If it incorrectly remains 27, query [22,22] behaves as follows.

| Position | Comparison used | Incorrect choice |
|---|---|---|
| [12,17] | Stale left.max=27≥22 | Left, abandoning the real answer on the right |
| [5,7] | left.max=4<22 | Into empty right child |
| NIL | Loop ends | Incorrect failure |

The actual answer is [20,24] in the right subtree. With exact left.max=7 the first step goes right and finds it. **This one-path algorithm requires an exact maximum, not just an upper bound.** Underestimates can prune true answers directly.

Recompute max bottom-up after modifications, allowing decreases.



## Endpoint and update conventions are part of the contract

| Convention | Nonempty condition | overlap(i,q) | Left may contain an answer |
|---|---|---|---|
| Closed [l,h] | l≤h | i.low≤q.high and q.low≤i.high | left.max≥q.low |
| Half-open [l,h) | l<h | i.low<q.high and q.low<i.high | left.max>q.low |

[4,5] and [5,7] meet at 5; [4,5) and [5,7) do not overlap. Half-open [5,5) is empty, unlike a singleton closed interval.

Changing low changes the BST ordering key: delete and reinsert the record instead of editing the field in place. Changing only high preserves order, provided the interval stays nonempty, but requires max repair upward from that node.

## One match, leftmost match, and all matches differ

The algorithm promises any overlapping record, not minimum low or all matches. To enumerate all, recurse left only if left.max≥q.low, test x, and recurse right only if x.low≤q.high and right.max≥q.low. Each visited node costs O(1), with worst-case O(n). The one-path O(log(n+2)) bound cannot be transferred to enumeration.

Another provable enumeration method repeats overlap-search, temporarily deletes each returned record, and records it in a list. k matches plus one final failure cost O((k+1)log(n+2)); reinsertion has the same order. This temporarily mutates the tree and must fit the caller's contract. Output alone costs Ω(k). Achieving exactly O(log n+k) requires another analysis or representation; saying “output-sensitive” is no proof.

Size and max can be stored together, each pulled in O(1), with the same maintenance order. Rank then follows BST order on (low,identity), not high order or number of overlaps, which need other summaries or algorithms.



## B-trees: use a page to reduce tree height

A node stores several sorted keys and child pointers separating their ranges. Search chooses a range within the current page, then reads another page. Separate **page accesses (I/O)** from **within-page comparisons and movement (CPU)**.

We use a set of distinct keys. Each key travels with its payload or record pointer; internal nodes can store actual records. Duplicate insertion returns EXISTS without adding a record. A B+ tree uses a different representation: records reside in leaves and internal nodes store separators.

Whether one node fits a page depends on page size, key/payload sizes, child pointers, and headers. The I/O analysis assumes this layout requirement holds.

## Minimum degree t: fix the occupancy contract

Fix an integer $t \geq 2$. Key and child arrays are zero-based.

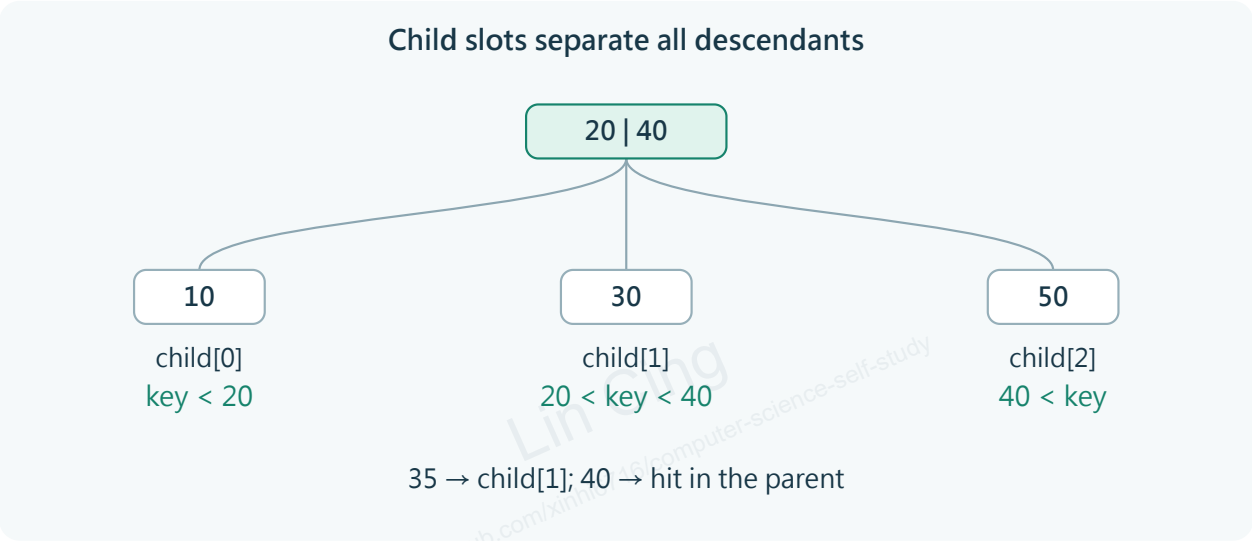
| Node type | Key count k | Child count |
|---|---|---|
| Nonroot | $t-1 \leq k \leq 2t-1$ | 0 for a leaf; exactly k+1 otherwise |
| Nonempty root | $1 \leq k \leq 2t-1$ | 0 for a leaf; at least 2 otherwise |
| Empty tree | Zero-key leaf root | 0 |

All leaves have the same depth. The root has depth 0 and leaf depth is height h. A nonempty height-zero tree stores everything in its root. We also assign height 0 to the empty tree, but the nonempty height theorem does not apply at n=0.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Every child pointer owns a whole key range




Keys $a_0 < \ldots < a_{k-1}$ divide the subtree into k+1 ranges. Each restriction applies to **every descendant**, not merely the first key of a child. Search for 35 chooses the middle child; search for 40 stops at the parent.

## Do not mix order m with minimum degree t

| Convention used here | Nonroot internal children | Nonroot keys |
|---|---|---|
| Order m means at most m children | ⌈m/2⌉…m | ⌈m/2⌉−1…m−1 |
| Minimum degree t | t…2t | t−1…2t−1 |

**Capacities match exactly when m=2t.** Order 3 gives a 2-3 tree. It is neither t=1.5, which is invalid, nor t=2, which permits four children and gives a 2-3-4 tree.

Books do not use “order” uniformly. Check whether it means maximum children or another capacity before applying a formula. The table explicitly fixes the maximum-child convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Preemptive and overflow splitting have different premises

Our insertion splits a full node with 2t−1 keys before descending: promote one key, leaving t−1 keys on each side. Both sides are immediately legal.

An order-m overflow algorithm first permits m temporary keys, then promotes a median. With m=3, inserting 5 into [2,8] temporarily gives [2,5,8]; promote 5 and retain [2] and [8].

Preemptively splitting [2,8] at order 3 leaves one side empty regardless of which key rises, violating the one-key minimum. **Capacity, repair timing, and proof must agree.** Deletion likewise has both repair-before-descent and repair-after-underflow schedules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Height upper bound: count the sparsest legal tree

Assume $n \geq 1$. The root has at least one key and, if $h \geq 1$, at least two children. Depth $i \geq 1$ contains at least $2t^{i-1}$ nodes, each with at least $t-1$ keys. Thus

$$n \geq 1 + \sum_{i=1}^{h} 2t^{i-1}(t-1) = 2t^h - 1.$$

Rearranging and taking base-$t$ logarithms gives

$$h \leq \log_t \frac{n+1}{2}.$$

For $h=0$ the sum is empty, giving $n \geq 1$ as required. The root minimum is one key; do not count it as another node with $t-1$ keys.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Height lower bound and a billion-key example

Each node has at most $2t-1$ keys and $2t$ children, so

$$n \leq (2t-1)\sum_{i=0}^{h}(2t)^i = (2t)^{h+1} - 1.$$

For $t=100$ and $n=10^9$, height 2 holds at most 7,999,999 keys, too few; height 3 holds at most 1,599,999,999. Height 4 requires at least 199,999,999 keys, while height 5 requires 19,999,999,999, too many. A legal tree therefore has $h=3$ or 4.

Search visits 4 or 5 nodes. With a resident root and no other path pages cached, it performs **3 or 4 page reads**. A nonresident root adds one more read.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Search: locate a slot, then choose the only possible subtree

```
SEARCH(x,k):
    i=lower_bound(x.keys,k)       // first key >= k, or len(keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x.children is empty: return NOT_FOUND
    read x.children[i] if not cached
    return SEARCH(x.children[i],k)
```

Without an equal key, global range separation leaves only child i as a possible location. Every step descends one level and eventually succeeds or fails at a leaf. This proves termination and correctness. Linear scanning and binary search implement the same slot choice and page path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



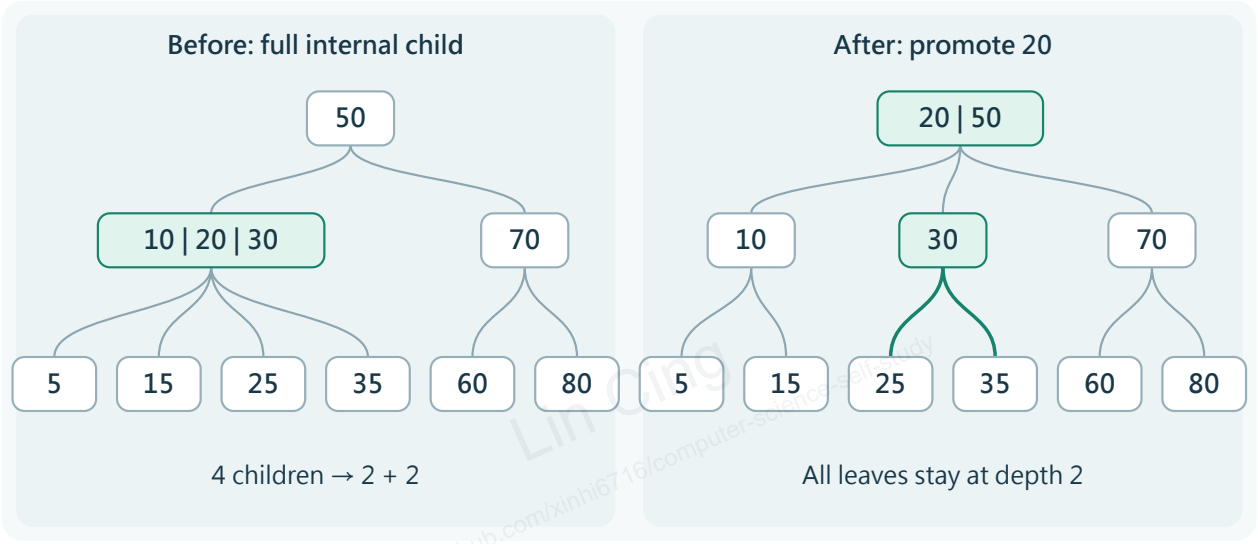
## SPLIT: move the median and divide the child pointers

```
SPLIT(p,i):                       // p nonfull; child i full
    y=p.children[i]; old=y.keys; kids=y.children
    median=old[t-1]
    z=Node(keys=old[t:], children=kids[t:] if kids else [])
    y.keys=old[:t-1]
    y.children=kids[:t] if kids else []
    p.keys.insert(i,median); p.children.insert(i+1,z)
    write y,z,p
```

Slices include the left endpoint and exclude the right. old and kids refer to the pre-change arrays. Both halves have t−1 keys and, if internal, t children. The median moves to the parent, with **no copy retained in either child**. A nonfull parent can safely gain one key.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Splitting an internal node must preserve all four child subtrees



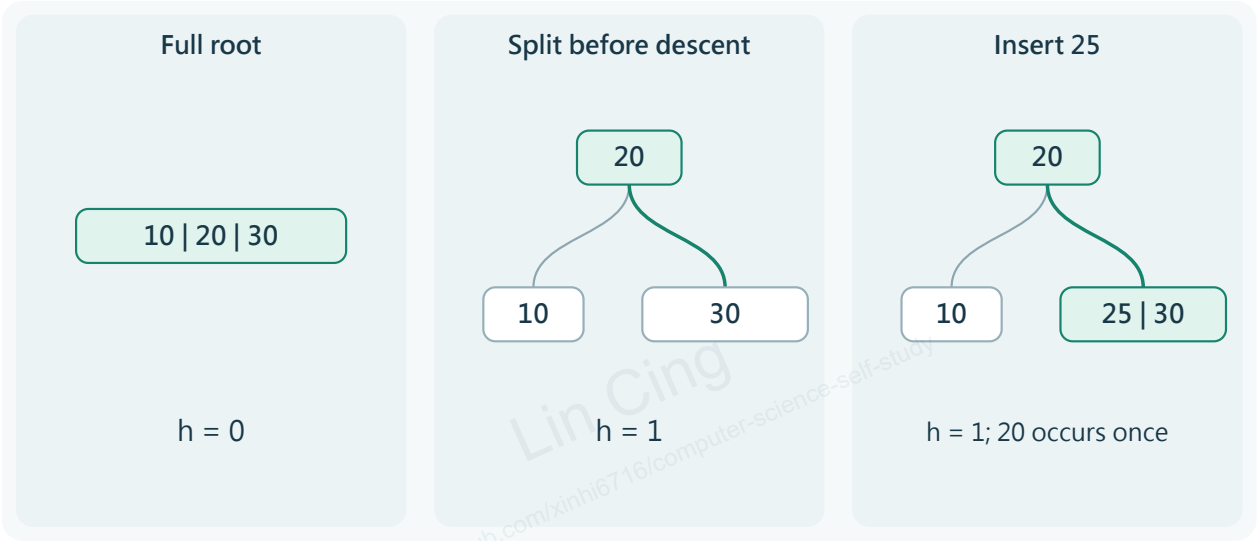

For t=2, parent [50] has full left child [10,20,30]. Promoting 20 gives parent [20,50]. Node [10] keeps [5],[15]; node [30] receives [25],[35]. Old right subtree [70] is unchanged. All leaves remain at depth 2.



## Insertion: always descend into a nonfull node

```
INSERT(k):
    if SEARCH(root,k) succeeds: return EXISTS
    if len(root.keys)==2*t-1:
        root=Node([], [root]); SPLIT(root,0)
    x=root
    while x.children is not empty:
        i=lower_bound(x.keys,k)
        if len(x.children[i].keys)==2*t-1:
            SPLIT(x,i)
            if k>x.keys[i]: i+=1
        x=x.children[i]
    insert k in sorted order into x.keys; write x
    return INSERTED
```

## Root splitting increases height at the top




For t=2, insert 25 into full leaf root [10,20,30]. Promote 20 first, then choose the right leaf since 25>20, yielding [25,30]. Only root splitting adds a level; all old leaves deepen together and remain at equal depth.



## Insertion correctness: why no upward repair is needed

At loop entry, x is nonfull. A full original root is split under a new root, establishing the invariant initially. If the next child is full, nonfull x can accept its promoted key. Both split children have t−1 keys, so the chosen child is again nonfull.

Each iteration descends one level. A leaf gains just one key, at most from 2t−2 to 2t−1, without overflowing. SPLIT preserves keys and subtree ranges; the final new key lies in the range chosen by search. The represented set gains exactly k.

Repair precedes descent, unlike an overflow-based schedule, so insertion need not return to an earlier parent to fix overflow. The separate duplicate check adds another search path but does not change the asymptotic bound.

## Deletion: give the next subproblem one key of slack

Ordinary nonroot occupancy is $t-1$ keys. Deletion strengthens the entry condition: **before recursing into a nonroot node, ensure it has at least $t$ keys**. Losing one key at that level then leaves the legal $t-1$ minimum.

| Situation | Action |
|---|---|
| k found in a leaf | Remove it |
| k found internally; left child has $\geq t$ keys | Replace by predecessor; delete it on the left |
| Left child lacks $t$; right has $\geq t$ | Replace by successor; delete it on the right |
| Both adjacent children have $t-1$ | Merge both with the parent key, then delete k |

Even when k is not found locally, repair its target child before descending. The root is exempt from the stronger minimum; a final zero-key internal root is replaced by its only child.



## Borrow left: lower the separator and raise the sibling maximum

```
BORROW-LEFT(p,i):                // i>0
    y=p.children[i]; s=p.children[i-1]
    y.keys.insert(0,p.keys[i-1])
    p.keys[i-1]=s.keys.pop()
    if s.children: y.children.insert(0,s.children.pop())
    write s,y,p
```

Require $t-1$ keys in y and at least $t$ in s, at the same depth. The rightmost subtree of s lies between the new and old separators and must become the leftmost subtree of y. Afterward y has $t$ keys, s retains at least $t-1$, and the parent key count is unchanged.

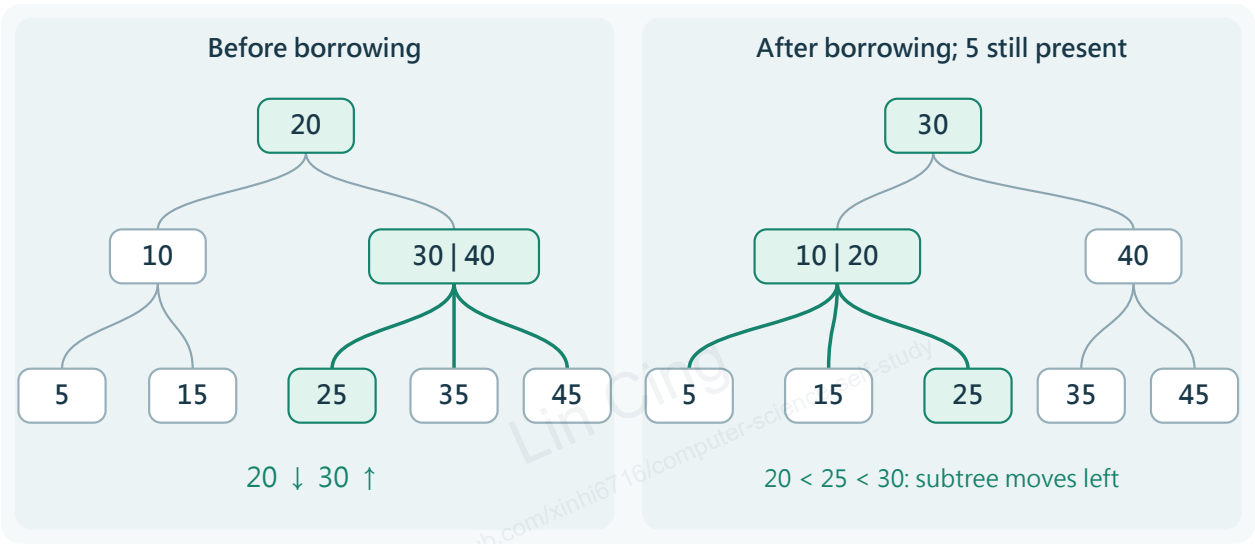
## Borrow right: move the child pointer to the correct side

```
BORROW-RIGHT(p,i):                // i<len(p.keys)
    y=p.children[i]; s=p.children[i+1]
    y.keys.append(p.keys[i])
    p.keys[i]=s.keys.pop(0)
    if s.children: y.children.append(s.children.pop(0))
    write s,y,p
```

Again require t−1 keys in y and at least t in s. The leftmost child of s becomes the rightmost child of y. Moving keys without moving that pointer assigns an entire key range to the wrong subtree, even if every individual node remains sorted.



## Borrowing example: why the 25 subtree must move left




For t=2, delete 5. Left internal node [10] has only one key; right sibling [30,40] has two. First borrow: 20 descends and 30 rises. Subtree [25] lies between 20 and 30 and must become the last child of [10,20]. The figure shows pre-descent repair; 5 has not yet been deleted.

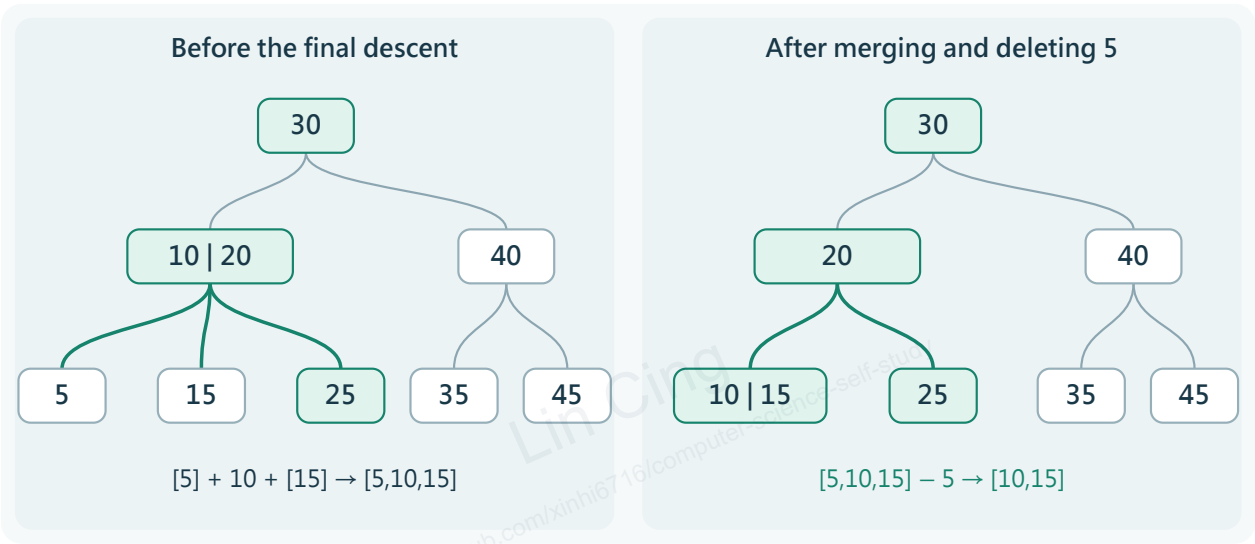
## MERGE includes the parent separator

```
MERGE(p,i):                     // adjacent children both t-1 keys
    y=p.children[i]; z=p.children[i+1]
    y.keys.append(p.keys.pop(i))
    y.keys.extend(z.keys)
    y.children.extend(z.children)
    p.children.pop(i+1)
    write y,p; free z
    return y
```

New y has (t−1)+1+(t−1)=2t−1 keys: full but not overflowing. If internal, it has t+t=2t children. The parent loses one key and one child. A nonroot parent entered with at least t keys and retains t−1. An empty root is contracted after the operation.



## Continue deleting 5: merge two minimal leaves first




Inside [10,20], target leaf [5] and sibling [15] each have one key. Include parent separator 10 to form [5,10,15], then remove 5, leaving [10,15]. The left internal node is [20] with children [10,15],[25]. Root [30] and right node [40] are unchanged. Inorder loses exactly 5 and all leaves remain at depth 2.

## An internal match moves a whole predecessor or successor record

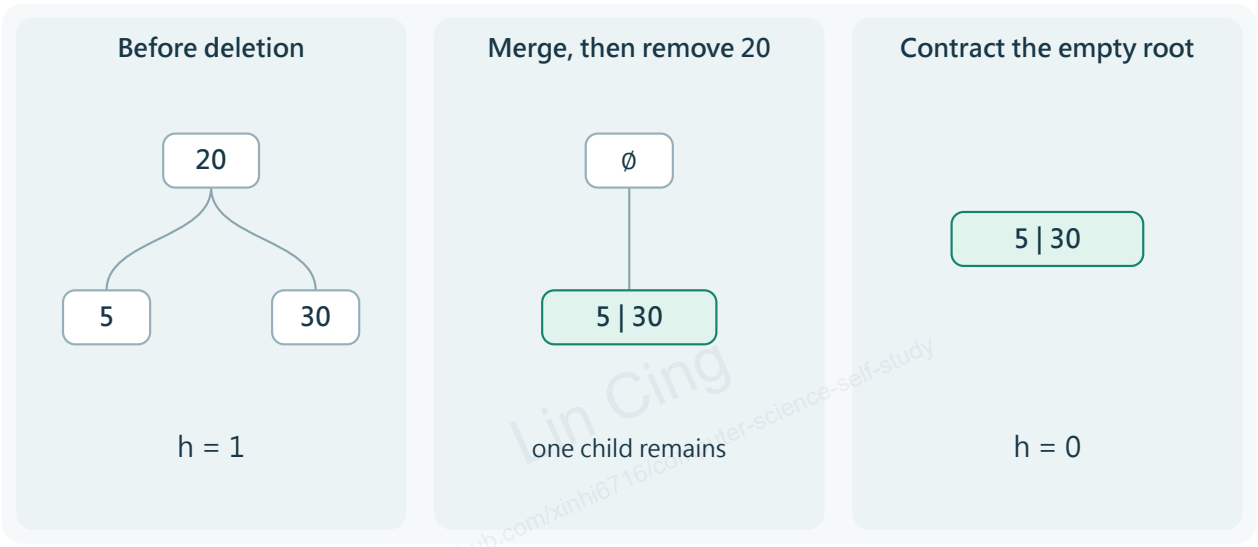
| Initial t=2 tree: root and two leaves | Delete | Result |
|---|---|---|
| Root [20]; left [5,10], right [30] | 20 | Predecessor 10 rises: root[10], leaves[5],[30] |
| Root [20]; left [5], right [30,40] | 20 | Successor 30 rises: root[30], leaves[5],[40] |

Find a predecessor by following the left subtree's rightmost children to a leaf and taking its last key. A successor follows the right subtree's leftmost children and takes the first key. Delete that record from its original subtree; do not retain duplicate parent/leaf copies.

The record pointer must move with its key. Changing only the key while leaving the old payload makes later queries return the wrong record.



## Root contraction: only the root may remove a level




Merge root separator [20] with leaves [5],[30] into [5,20,30], then delete 20, leaving [5,30]. The old root has zero keys and one child; replace it by that child, reducing h from 1 to 0. Doing this to an ordinary internal node would shorten only one branch and violate equal leaf depth.

## Deletion procedure: the three internal-match cases

```
DELETE-NODE(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if not x.children: erase x.keys[i]; write x; return FOUND
        L=x.children[i]; R=x.children[i+1]
        if len(L.keys)>=t:
            q=MAX-RECORD(L); DELETE-NODE(L,q.key); x.keys[i]=q
        elif len(R.keys)>=t:
            q=MIN-RECORD(R); DELETE-NODE(R,q.key); x.keys[i]=q
        else:
            y=MERGE(x,i); DELETE-NODE(y,k)
        write x; return FOUND
    return DELETE-DESCEND(x,i,k)  // next card
```

Here the keys array contains records; comparisons use their key fields. q must be a saved record value or stable pointer, not an object dereferenced after its original storage is freed.



## Deletion descent: borrow first; adjust the index after merging

```
DELETE-DESCEND(x,i,k):
    if not x.children: return NOT_FOUND
    if len(x.children[i].keys)==t-1:
        if i>0 and len(x.children[i-1].keys)>=t:
            BORROW-LEFT(x,i)
        elif i<len(x.keys) and len(x.children[i+1].keys)>=t:
            BORROW-RIGHT(x,i)
        elif i<len(x.keys): MERGE(x,i)
        else: MERGE(x,i-1); i-=1
    return DELETE-NODE(x.children[i],k)
```

Merging left absorbs old child i into child i−1, so decrement i. Public DELETE(k) calls DELETE-NODE(root,k), then **contracts a zero-key internal root on either FOUND or NOT_FOUND**, and returns the original search result.

## An absent-key deletion can still change tree shape

Let t=2, root [20], and leaves [5],[30]. Delete 99: choose the right child. It has only t−1=1 key and its sibling cannot lend, so merge into [5,20,30] and change the child index from 1 to 0.

The resulting leaf lacks 99 and returns NOT_FOUND. The wrapper must still contract the empty internal root. The final tree is leaf root [5,20,30]: the set is unchanged, while height falls from 1 to 0.

**NOT_FOUND promises no element removal, not an unchanged shape.** An API requiring shape preservation on failure can first search for existence and account for that extra path. Empty trees and deletion of the last key retain a zero-key leaf root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## General deletion proof and two separate cost measures

Induct on remaining height. Leaf removal is correct. Internal replacement deletes the predecessor or successor from its old subtree, removing exactly k overall. Borrowing and merging preserve global order, child counts, and equal leaf depth, while giving the next subproblem at least t keys. A nonroot parent losing a separator retains t−1. Root contraction handles the exception. Every recursive call descends, proving termination.

Only constantly many neighboring pages are touched per level: I/O is $O(h+1)=O(1+\log_t(n+1))$. Array movement costs up to O(t) per level, giving O(t(h+1)) insertion/deletion CPU time. Separate predecessor/successor lookup adds paths totaling O(h+1), not a fresh full-height traversal at every level.

Binary search inside a page gives O((h+1)log t) search comparisons. It does not reduce insertion/deletion array movement to O(log t).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fibonacci heaps: defer linking until extract-min

A min-heap requires each parent key to be no greater than its child keys; different subtrees need not be ordered. A Fibonacci heap keeps a list of such trees. Insertion and meld concatenate roots; EXTRACT-MIN later links roots of equal degree.

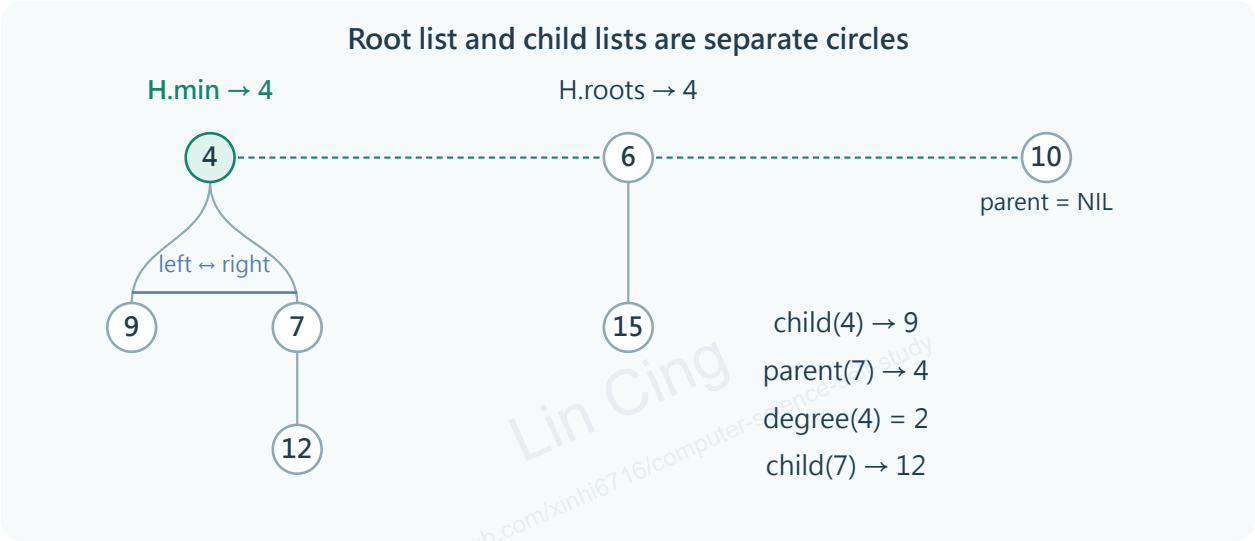
Degree is the **number of immediate children**, not height. Cuts destroy the exact binomial-tree shape. We will prove degree O(log n); this does not assert logarithmic root-to-leaf paths.

DECREASE-KEY(H,x,k) and DELETE(H,x) receive a valid node handle. Finding x by an arbitrary key may require scanning the whole heap and is excluded from those operation bounds. Equal keys may coexist as distinct nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Separate the root list, child lists, and minimum pointer




Roots are 4,6,10. Node 4 owns 9 and 7; 7 owns 12. Root-list entry and minimum pointer have different roles. Each sibling group has its own circular list.

## Node fields and the exact meaning of mark

A node stores key, parent, child, left, right, degree, and mark. The child pointer selects any child; siblings form a circular doubly linked list. A lone child points left and right to itself. H.roots enters the root list, H.min selects a minimum root, and H.n counts all nodes. Both entries are NIL in an empty heap.

A nonroot mark records one child loss since its current attachment. A second loss cuts the node. Our convention clears marks whenever nodes become roots or are linked as children; every root is unmarked.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Circular lists: the mechanism behind constant-time meld

To remove a known node x, connect x.left.right=x.right and x.right.left=x.left. Replace the list entry if it names x; a singleton leaves NIL. Finally isolate x by setting both sibling pointers to itself. Only O(1) pointers change.

For nonempty circles entered at a and b, save al=a.left and bl=b.left. Set al.right=b, b.left=al, bl.right=a, and a.left=bl. Empty-list cases simply choose the other entry.

Below ATTACH-ROOT(H,x) takes an isolated node, splices it into the root list, and sets parent=NIL and mark=false. REMOVE-CHILD(p,x) repairs p.child, sibling links, and degree. These primitives neither search by key nor traverse every sibling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## INSERT, MINIMUM, and destructive UNION

| Operation | Complete action | Actual cost |
|---|---|---|
| MAKE | roots=min=NIL; n=0 | O(1) |
| INSERT(x) | New singleton root; update min; increment n | O(1) |
| MINIMUM | Return min, or NIL if empty | O(1) |
| UNION(H1,H2) | Splice root circles, choose the smaller non-NIL min, add counts | O(1) |

UNION consumes two disjoint heaps. The result owns all their nodes; the old entries must not be used independently. H1=H2 is forbidden. Two empty inputs produce an empty result. This bound excludes rewriting a per-node heap-owner field; an implementation demanding that work cannot claim constant-time meld.



## EXTRACT-MIN: promote children, then consolidate

```
EXTRACT-MIN(H):
    z=H.min
    if z==NIL: return NIL
    while z.child!=NIL:
        c=z.child
        REMOVE-CHILD(z,c)
        ATTACH-ROOT(H,c)
    REMOVE-ROOT(H,z)
    H.n-=1
    if H.roots==NIL: H.min=NIL
    else: CONSOLIDATE(H)
    return z
```

Repeatedly remove the current child rather than following sibling pointers invalidated by mutation. Promotion clears parent and mark; isolate z before returning it. One original root does not imply an empty result: its promoted children may remain.

## CONSOLIDATE: leave at most one root of each degree

```
CONSOLIDATE(H):
    A=empty degree table; H.min=NIL
    while H.roots!=NIL:
        x=POP-ROOT(H)              // isolate one root
        d=x.degree
        while A[d]!=NIL:           // absent slot means NIL
            y=A[d]; A[d]=NIL
            if y.key<x.key: swap(x,y)
            LINK-ISOLATED(y,x)      // y.parent=x; y.mark=false
            d=x.degree
        A[d]=x
    for each non-NIL x in A:
        ATTACH-ROOT(H,x)
        if H.min==NIL or x.key<H.min.key: H.min=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LINK and the degree-table contract

LINK-ISOLATED joins equal-degree isolated roots: the larger-key y becomes a child of x, increasing x.degree by one. Ties choose either parent. A can be a growing array using O(D(n)+1) slots. Draining the root list avoids revisiting old roots that have already been linked.

```
LINK-ISOLATED(y,x):
    require x!=y; x.degree==y.degree
    require x.key<=y.key; both roots isolated
    y.parent=x; y.mark=false
    splice y into x.child circular list
    x.degree+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Consolidation invariant and termination

At each outer-loop entry, unprocessed roots remain in H.roots; processed nodes lie in disjoint subtrees named by A. Each occupied slot stores one tree whose degree equals the index. Every original node belongs to exactly one of these parts.

The inner loop maintains d=x.degree. Linking an equal-degree y preserves the node set and heap order, replaces two roots by one, and increments d. Every link decreases the finite forest root count, so the loop terminates.

Each outer iteration removes an unprocessed root. At termination A represents all remaining trees, with distinct root degrees. Rebuilding the root list and comparing its roots gives the correct minimum. Distinct degrees are guaranteed only immediately after consolidation; INSERT, UNION, and CUT may create duplicates again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



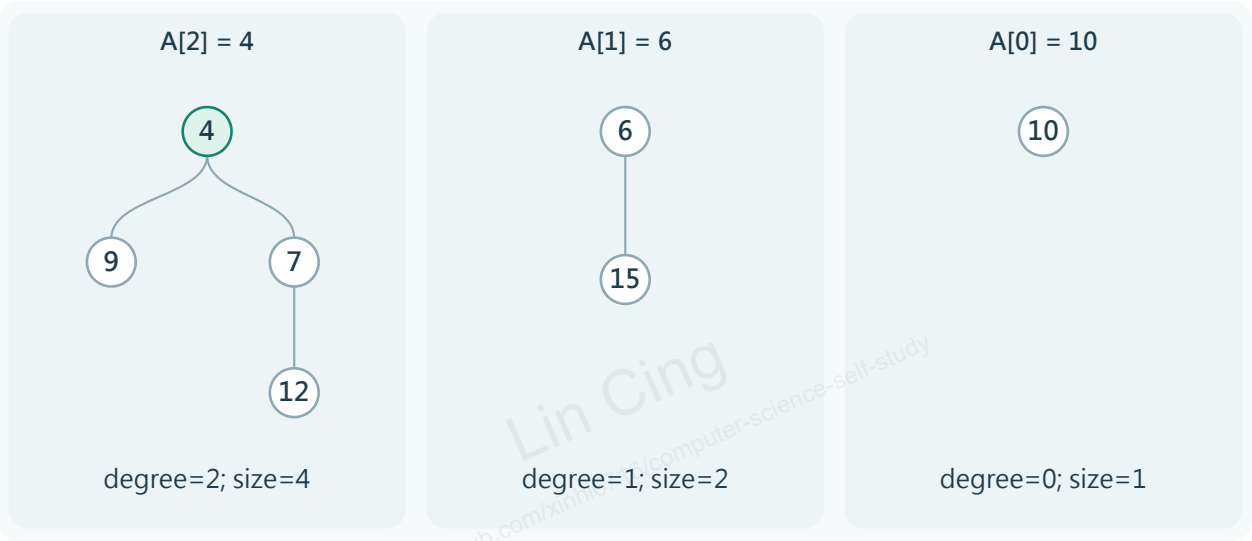
## A worked consolidation: follow every degree collision

Start with eight singleton roots 1,4,9,7,12,6,15,10. Extract 1, then process the remaining roots in order 4,9,7,12,6,15,10; each starts with degree 0.

| Root taken | Links in this iteration | Occupied A slots (degree:root) |
|---|---|---|
| 4 | None | 0:4 |
| 9 | Attach 9 under 4 | 1:4 |
| 7 | None | 0:7, 1:4 |
| 12 | Attach 12 under 7; then 7 under 4 | 2:4 |
| 6 | None | 0:6, 2:4 |
| 15 | Attach 15 under 6 | 1:6, 2:4 |
| 10 | None | 0:10, 1:6, 2:4 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Verify the final forest, degrees, and root count




Root degrees are 2,1,0. Subtree sizes total 4+2+1=7 and min=4. Other roots need not be ordered by key.



## The same extraction: fewer roots release stored potential

Root 4 has children 9 and 7; node 7 retains child 12. Root 6 has child 15, and root 10 has no children. Their degrees are 2,1,0, with 4+2+1=7 nodes, and every parent key is no greater than its child key.

| Stage | Roots r | Marks m | Potential K(r+2 m) |
|---|---|---|---|
| Before extracting 1: eight singleton roots | 8 | 0 | 8 K |
| After removing 1 and completing four links | 3 | 0 | 3 K |

Potential falls by 5 K: removing the minimum eliminates one root and the four links eliminate four more. This connects the example to the amortized formula. **5 K is released potential, not an exact count of five machine instructions.**

These happen to be uncut binomial trees. After CUT, degree d no longer guarantees $2^d$ nodes; the general lower bound will be $F_{d+2}$.

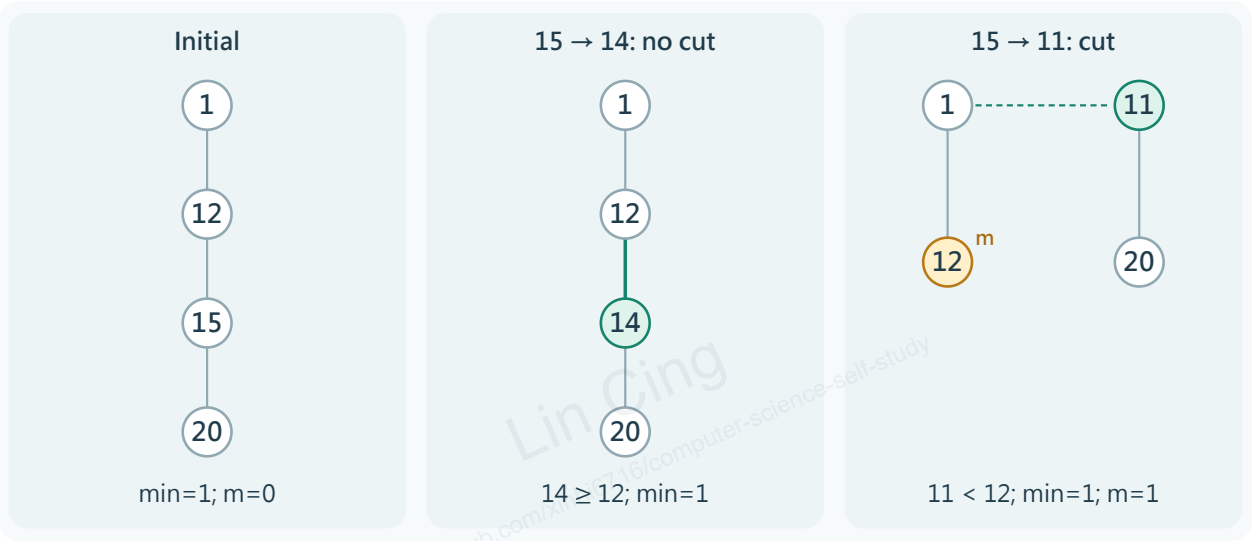
## DECREASE-KEY: cut only when parent order is violated

```
DECREASE-KEY(H,x,k):
    if k>x.key: error
    x.key=k; p=x.parent
    if p!=NIL and x.key<p.key:
        CUT(H,x,p)
        CASCADING-CUT(H,p)
    if x.key<H.min.key: H.min=x

CUT(H,x,p):
    REMOVE-CHILD(p,x)       // degree decreases by one
    ATTACH-ROOT(H,x)       // parent=NIL; mark=false
```

Lowering x cannot violate its order relative to its children; only parent→x may fail. A root or a new key still at least its parent key needs no cut. The new minimum is either the old minimum or x, since no other key changed.



## Worked decrease: 15→14 stays; 15→11 cuts the subtree




Start both cases from the left tree. Key 14 stays; key 11 is cut with child 20 still attached. Nonroot 12 records its first child loss.

## Check both decreases field by field

Root 1 has child 12, which has child 15, which has child 20. Start each case from that same tree. Decreasing 15→14 keeps 14≥12 and changes no links. Decreasing 15→11 violates 11<12 and cuts the **entire subtree**: 20 remains a child of the node now keyed 11.

In the second case, an initially unmarked nonroot 12 becomes marked after its first child loss. Root 1 is unaffected. New root 11 exceeds the old minimum 1, so min stays. Decreasing an existing root, such as 3→0, updates min without a cut.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## CASCADING-CUT: mark the first loss, cut on the second

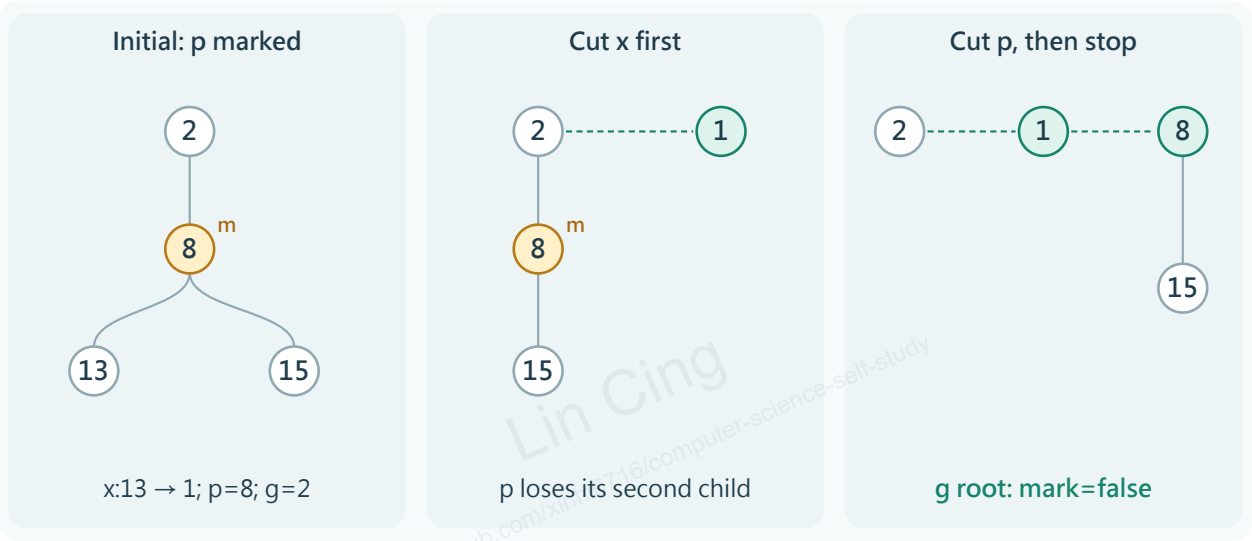
```
CASCADING-CUT(H,y):
    while y.parent!=NIL:
        p=y.parent
        if y.mark==false:
            y.mark=true
            return
        CUT(H,y,p)
        y=p
    return                      // root: stop, do not mark
```

At entry y has just lost a child. If unmarked, record its first loss since this attachment and stop. If marked, this is its second loss: cut y, clear its mark, and continue at p, which has now lost y.

Deleting edges preserves heap order on all remaining edges. Cascading cuts limit how many children a nonroot can lose, preserving the relation between degree and subtree size; they do more than repair key order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Cascading branch one: stop at a root grandparent




Cut x, then marked p. Stop at root g without marking it. Child 15 remains attached to p.



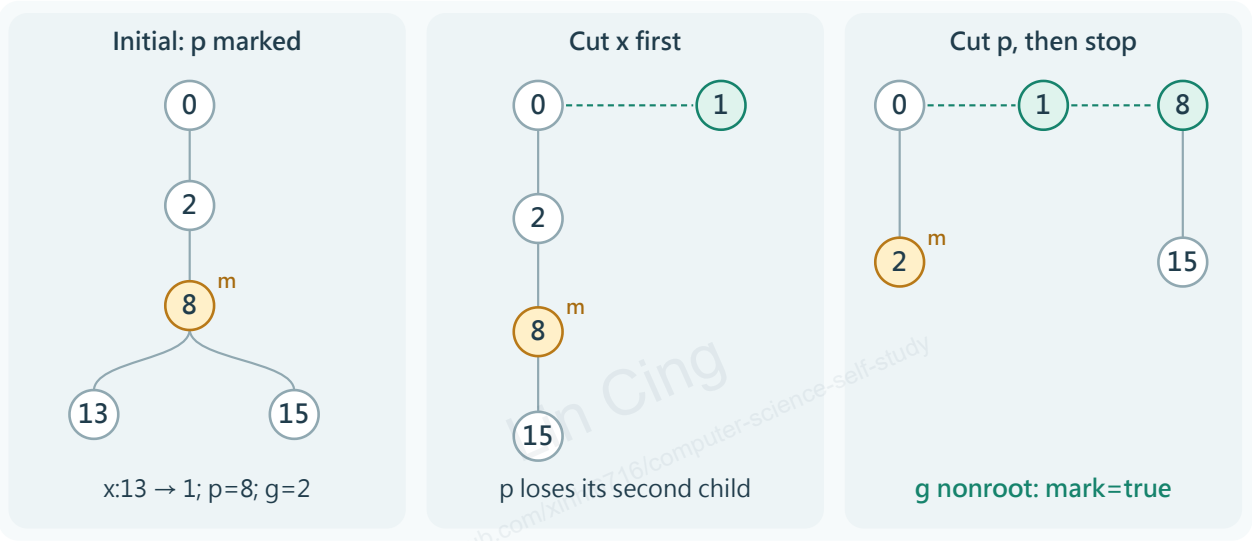
## Root branch: root list and potential ledger

Initially the sole root is g=2. Its marked child p=8 has children x=13 and s=15. The mark records a previous child loss. Now decrease x from 13 to 1.

Cut x, then cut already-marked p. At g, parent=NIL, so stop and **do not mark root g**. Final roots are 2,1,8; node 8 still owns child 15. The minimum is 1.

With r roots and m marked nodes, (r,m) changes from (1,1) to (3,0). Unscaled potential r+2m changes from 3 to 3. Clearing p's mark offsets the two additional roots.

## Cascading branch two: mark an unmarked nonroot grandparent




Here g has parent q and starts unmarked. After cutting x and p, mark g and stop. Root q=0 remains the minimum.



## Nonroot branch: clear p, then mark g

Add root q=0 above the previous local structure, making g=2 an unmarked nonroot; p=8 remains marked. Decrease x=13 to 1. Cut x, then p. Now g.parent=q and g.mark=false, so mark g and stop.

Final roots are q=0, x=1, p=8. Node q retains g=2, and p retains s=15. The minimum stays 0. Roots rise 1→3, while marks stay 1→1: p is cleared and g is newly marked. Potential rises 3→5.

If g had already been marked, cut g as well and continue at q. **Parent and mark determine the endpoint**; marking the grandparent is not an unconditional outcome.

## One potential pays for every operation

Choose one fixed constant K>0 and sum over all live, pairwise disjoint heaps:

$$\Phi = K \sum_{H} \big(r(H) + 2m(H)\big), \qquad \hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

Choose K once, sufficiently large to cover the constant-time pointer primitives below. Do not change it between operations. Starting empty gives $\Phi_0=0$ and nonnegative final potential, so $\Sigma c_i \le \Sigma \hat{c}_i$. A preexisting forest adds initial $\Phi_0$ to the sequence bound.

INSERT adds one root and no mark, giving $\Delta\Phi=K$ and O(1) amortized cost. MINIMUM and MAKE leave potential unchanged. UNION replaces two heaps by their disjoint union; root and mark counts add, so global potential is unchanged and both actual and amortized meld costs are O(1).



## Why a long cascade has constant amortized cost

With no cut, actual work and potential change are O(1). Otherwise let $c \ge 1$ count every cut, including the first cut of x. Each of the remaining $c-1$ cuts clears a marked nonroot. At most one final nonroot becomes newly marked; any old mark on x is also cleared. Hence

$$\Delta r = c, \qquad \Delta m \le 1 - (c-1) = 2 - c,$$
$$\Delta\Phi \le K(4-c).$$

For a fixed constant a, actual work is at most $a(c+1)$. Choose $K \ge a$:

$$\hat{c} \le a(c+1) + K(4-c) \le a + 4K = O(1).$$

This holds for every legal operation sequence without a random-input assumption. One actual cascade can still follow a long path and take linear time; O(1) is the amortized bound.

## Extract-min analysis: many roots release stored potential

Before extraction let n be the node count, r the root count, and D=D(n) a maximum-degree bound. Promoting the removed root's children leaves R≤r−1+D roots. Every link removes a root, so there are at most R links. Child promotion, root processing, and degree-table initialization/rebuilding cost at most a(r+D+1).

After consolidation at most D+1 roots remain. Promotion and linking only clear marks, so

$$\Delta\Phi \le K(D+1-r).$$

$$\hat{c} \le a(r+D+1) + K(D+1-r) \le (a+K)(D+1).$$

Use the same K≥a; empty and singleton heaps cost O(1). This proves O(D+1), not yet logarithmic time: the degree theorem must supply D=O(log(n+1)). After n singleton insertions, the first extraction can take Θ(n) actual time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The degree lemma: each retained child loses at most one child

Let x currently have d children $y_1, \dots, y_d$, ordered by their **most recent attachment times** to x. For i≥2, all earlier children that still survive were already attached when $y_i$ joined. Thus x then had degree at least i−1.

LINK joins equal-degree roots, so $y_i$ also had degree at least i−1 at attachment. Losing a second child would have cut it from x. Since $y_i$ still remains, it has lost at most one child:

$$\deg(y_1) \ge 0, \qquad \deg(y_i) \ge i-2 \quad (2 \le i \le d).$$

Current sibling degrees need not be sorted. Nor is a nonroot limited to one child loss over its entire lifetime: cutting and later relinking resets the relevant history.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## From child degrees to a Fibonacci subtree-size bound

Use strong induction on subtree node count to prove size(x)≥$F_{d+2}$ for every node of degree d. Child subtrees are smaller, so their induction hypotheses apply. Combine the preceding degree lemma with monotonicity of Fibonacci numbers.

$$F_0 = 0, \quad F_1 = 1, \quad F_j = F_{j-1} + F_{j-2}\ (j \geq 2).$$

For d=0, size≥1=$F_2$. For d=1, x and one child give size≥2=$F_3$. For d≥2, the first child contributes at least one node and the remaining children give

$$\text{size}(x) \geq 2 + \sum_{i=2}^{d} F_i = F_{d+2}.$$

The identity follows from $\sum_{i=0}^{d} F_i$=$F_{d+2}$−1: at d=0 it is 0=$F_2$−1; adding $F_{d+1}$ and using the recurrence proves the next case. Inducting on subtree size avoids assuming an unproved monotonicity of minimum subtree size by degree.



## Finish the degree bound: exponential size implies logarithmic degree

Let ϕ=(1+$\sqrt{5}$)/2, so $\phi^2$=ϕ+1. Prove $F_{d+2}$≥$\varphi^d$: the bases are 1≥1 and 2≥ϕ. For d≥2,

$$F_{d+2} = F_{d+1} + F_d \geq \varphi^{d-1} + \varphi^{d-2} = \varphi^d.$$

For any node in a nonempty n-node heap, n≥size(x)≥$F_{d+2}$≥$\varphi^d$, giving $d \leq \lfloor \log_\varphi n \rfloor$. Maximum degree, degree-table size, the post-consolidation root count, and amortized EXTRACT-MIN cost are therefore O(log(n+1)).

| Degree d | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| Guaranteed minimum size $F_{d+2}$ | 1 | 2 | 3 | 5 | 8 | 13 | 21 |

This is a guaranteed lower bound, not a claim that a specified operation sequence attains equality for every d. Handle the empty heap separately rather than taking log 0.

## DELETE: make the designated node uniquely minimum

DELETE(H,x) may first decrease x to a special sentinel below every ordinary comparison key, then call EXTRACT-MIN. For a nonroot, DECREASE-KEY correctly handles the parent's child loss and any cascade. Extraction promotes x's children and consolidates.

Writing “minus infinity” while allowing another node with the same comparison value is insufficient: min could still name the wrong node. Extend comparison to (tag,key), normally with tag=1; only the deletion target temporarily has tag=0. It becomes uniquely minimum. Restore its original key after returning the removed node.

Adding both operations under the same potential cancels the intermediate potential, giving amortized O(1)+O(log(n+1)). Direct internal-node removal is another possible implementation, but it must specify child promotion, parent marks, and cascading repair; promotion alone is incomplete.



## Operation bounds and their conditions

| Operation | Actual per-operation upper bound | Amortized upper bound |
|---|---|---|
| MAKE, MINIMUM, INSERT, UNION | O(1) | O(1) |
| DECREASE-KEY with a handle | O(n+1) | O(1) |
| EXTRACT-MIN, DELETE with a handle | O(n+1) | O(log(n+2)) |
| SEARCH by an arbitrary key | O(n+1) | Heap order gives no automatic improvement |

The logarithmic bound uses log(n+2) to retain constant cost when n=0. Valid-handle DELETE and DECREASE-KEY require a nonempty heap; empty EXTRACT-MIN is legal and returns NIL. Here n is the relevant heap size; UNION consumes disjoint input heaps. Bounds exclude allocator costs, per-node ownership relabeling, and locating a handle.

## Turn operation counts into a graph-algorithm bound

A graph algorithm using O(V) insertions/extractions and O(E) handle-based decreases spends O(E+V log(V+1)) total heap time. Constants, memory usage, and access patterns still affect implementation choices; asymptotic bounds are not measured speed.

For V=1000 and E=8000, suppose each vertex is inserted/extracted at most once and each edge triggers at most one decrease:

| Work | Maximum calls | Total Fibonacci-heap amortized bound |
|---|---|---|
| INSERT | 1000 | O(V) |
| EXTRACT-MIN | 1000 | O(V log(V+1)) |
| DECREASE-KEY | 8000 | O(E) |

Adding gives O(E+V log(V+1)); the corresponding binary-heap bound is O((E+V)log(V+1)). Counts are call limits, not milliseconds or exact costs: constants differ between representations.



## Use integer bits to change the search model

A red-black tree compares keys; a **van Emde Boas tree (vEB)** also uses their bit representation. Fix $S \subseteq \{0, \ldots, u-1\}$ with $u = 2^m$, $m \geq 1$, and distinct keys. **n=|S| and universe size u are different parameters.**

Use a word RAM with keys and addresses fitting in one word. Shifts, masks, comparisons, and array indexing cost O(1); arbitrary-precision bit complexity is a different model.

Membership, updates, predecessor, and successor take worst-case $O(1 + \log\log u)$; extrema take O(1). Comparison-sorting lower bounds do not prohibit this: integer operations and initialization/storage costs are additional assumptions.

## What direct addressing and naive recursion miss

A bit vector supports membership and updates in O(1), but scanning for the successor of 0 in $S = \{0, u-1\}$ can take Θ(u). A binary summary tree improves this query to O(log u).

Recursively storing roughly $\sqrt{u}$ clusters and their nonempty indices is not sufficient. Finding a cluster and then recursively finding its minimum gives

$$T(u) = 2T(\sqrt{u}) + O(1), \quad R(m) = 2R(m/2) + O(1) = \Theta(m).$$

The result is still Θ(log u). The crucial requirement is **at most one nonconstant recursive call per level**. Cached extrema make that possible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Odd bit widths need unequal high and low parts

Let $\ell = \lfloor m/2 \rfloor$, $h = \lceil m/2 \rceil$, $L = 2^{\ell}$, and $H = 2^{h}$, so u=HL. For u>2:

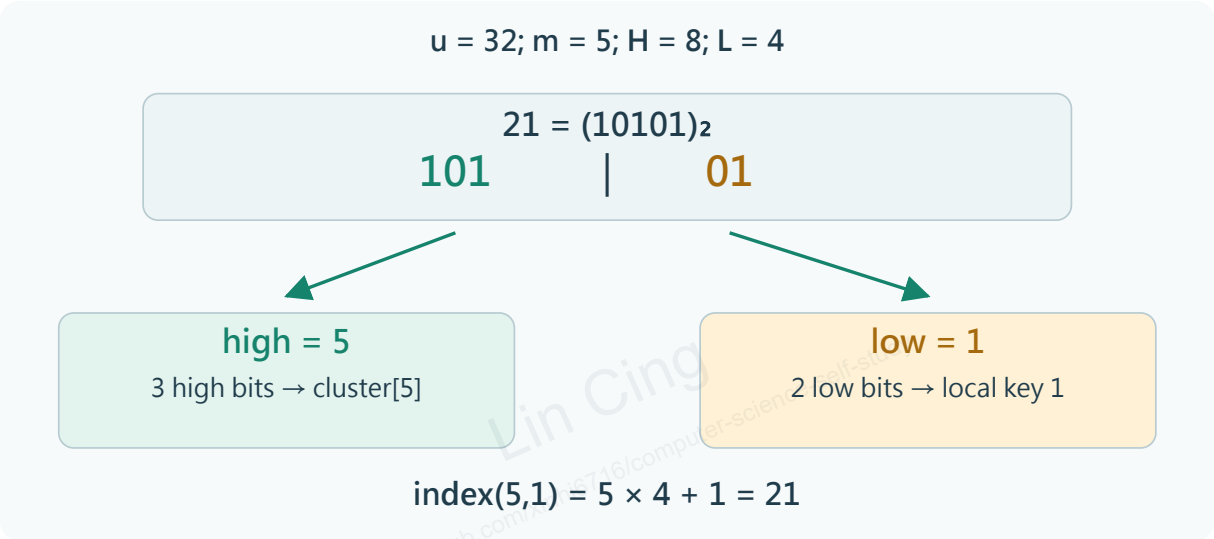
$$\text{high}(x) = \lfloor x/L \rfloor, \quad \text{low}(x) = x \bmod L, \quad \text{index}(i, j) = iL + j.$$

There are H clusters, each with universe L. The summary stores cluster indices and has universe H. **Recompute the split from the local u at every level.**

For u=32: m=5, h=3, ℓ=2, H=8, L=4. Split $21 = (10101)_2$ as $101 \mid 01$: high=5, low=1, and 5×4+1=21. The u=2 base has no recursive structures.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Two coordinates of the same key




high selects a cluster; low is the key passed into it. A 5 in the summary means cluster 5 is nonempty, not that the original set contains key 5.



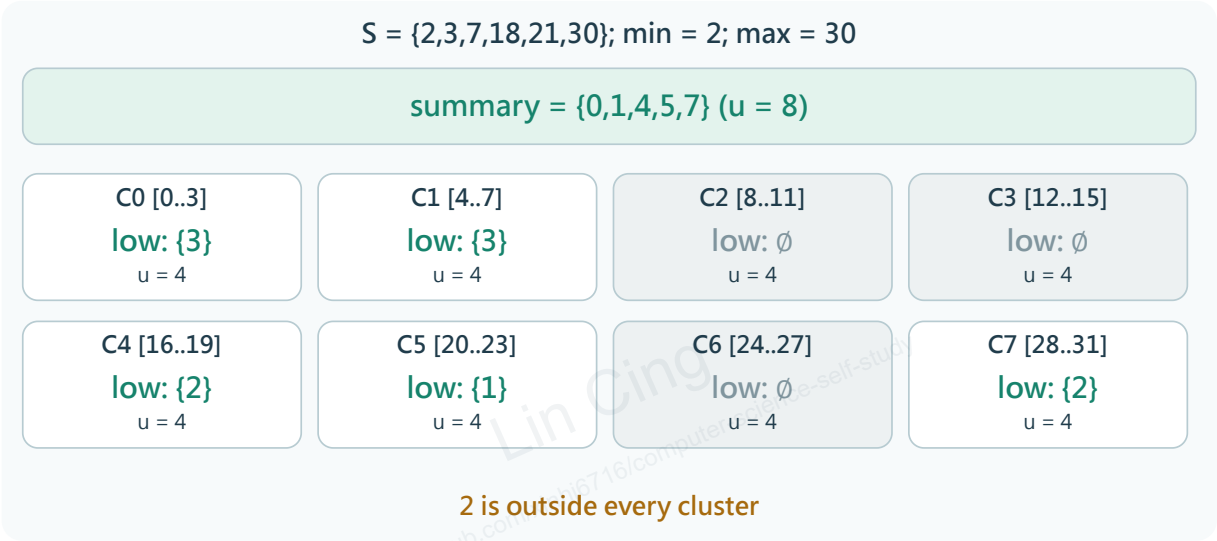
## Representation invariant: the minimum stays outside clusters

Each node stores u, min, and max; for u>2 it also owns a summary and cluster[0..H−1]. Use a fully preallocated structure. Empty nodes have min=max=NIL.

For a nonempty set, min is stored **outside all clusters**. Every other key is represented by its low part in the cluster selected by its high part. A singleton has empty clusters and summary. With at least two keys, max also appears in a cluster.

The summary contains exactly the nonempty cluster indices: $i \in \text{summary} \iff C_i.\min \neq \text{NIL}$. Thus min=max detects a singleton in O(1), and inserting into an empty node sets min=max=x without descending.

## A u=32 set: distinguish original keys, offsets, and summary




The set is {2,3,7,18,21,30}; top-level min=2 and max=30. $C_0$ stores only offset 3, not 2. The summary is {0,1,4,5,7}. Only the top level is expanded; every child obeys the same invariant.



## Membership and valid arguments

```
MEMBER(V,x):                 # require 0<=x<V.u
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return MEMBER(V.cluster[high(x)],low(x))
MINIMUM(V): return V.min
MAXIMUM(V): return V.max
```

high/low use V's local dimensions. Membership follows one cluster; extrema can stop the search early. NIL is a distinct state, never the valid key 0.

Successor is the smallest stored key **strictly greater than x**; predecessor is the largest strictly smaller key. Queries need not be members but must be inside the universe. Public interfaces reject invalid arguments before array access.

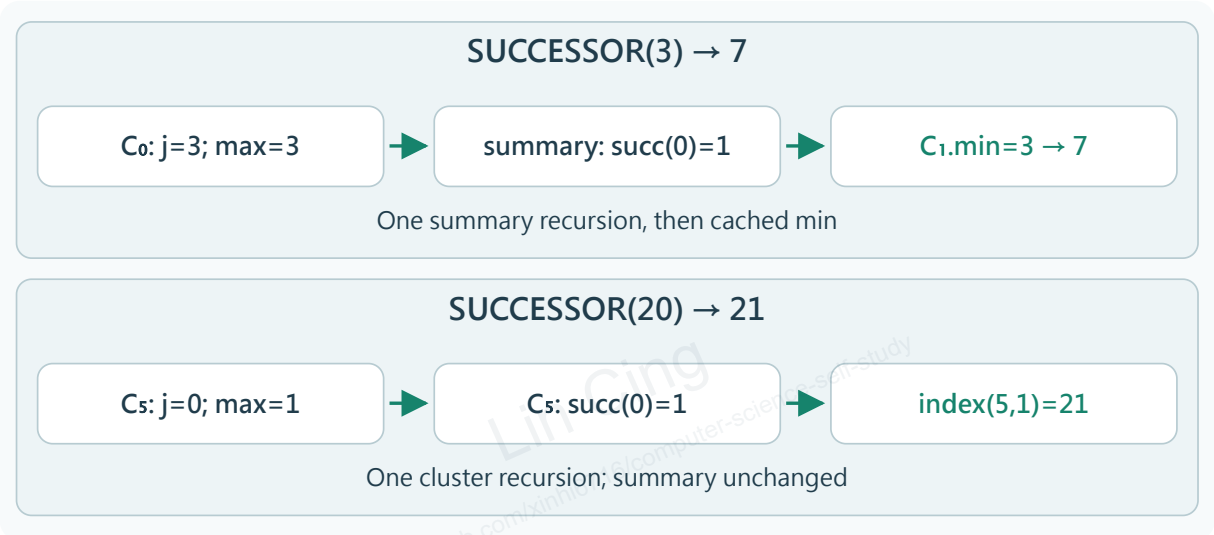
## Successor: max chooses the recursive branch

```
SUCCESSOR(V,x):
    if V.min==NIL: return NIL
    if x<V.min: return V.min
    if x>=V.max: return NIL
    if V.u==2: return 1
    i=high(x); j=low(x); C=V.cluster[i]
    if C.max!=NIL and j<C.max:
        return index(i,SUCCESSOR(C,j))
    k=SUCCESSOR(V.summary,i)
    if k==NIL: return NIL
    return index(k,V.cluster[k].min)
```

If the current cluster's max exceeds j, recurse there. Otherwise skip directly to the summary. **Do not perform a failed recursive search first.** Reading the selected cluster's min then costs O(1).



## Successor traces from one fixed state




Cached max chooses the cluster or summary branch; never both.

## Check every successor coordinate

Use {2,3,7,18,21,30}. SUCCESSOR(3): i=0,j=3 and $C_0$.max=3, so query the summary. Its successor index is 1; $C_1$.min=3 gives $1\times4+3=7$.

SUCCESSOR(20): i=5,j=0 and $C_5$.max=1>0, so recurse only into $C_5$. Offset 1 reconstructs 21. Query 20 need not be stored. SUCCESSOR(30)=NIL because the order is strict.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Predecessor must include the detached minimum

```
PREDECESSOR(V,x):
    if V.min==NIL: return NIL
    if x>V.max: return V.max
    if x<=V.min: return NIL
    if V.u==2: return 0
    i=high(x); j=low(x); C=V.cluster[i]
    if C.min!=NIL and j>C.min:
        return index(i,PREDECESSOR(C,j))
    k=PREDECESSOR(V.summary,i)
    if k==NIL: return V.min
    return index(k,V.cluster[k].max)
```

When k=NIL, return the detached min: earlier guards already established x>V.min, and no lower cluster exists. For example PREDECESSOR(3)=2 even though $C_0$ has no smaller offset. A symmetric copy of successor would miss 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Insert: swap out the old min and update only necessary summaries

```
INSERT-ABSENT(V,x):          # require x not in V
    if V.min==NIL: V.min=V.max=x; return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        i=high(x); j=low(x); C=V.cluster[i]
        if C.min==NIL:
            INSERT-ABSENT(V.summary,i)
            C.min=C.max=j
        else: INSERT-ABSENT(C,j)
    if x>V.max: V.max=x
```

A smaller new key becomes min; **the old min** is then inserted below. An empty cluster requires a recursive summary insertion but only O(1) singleton initialization. A nonempty cluster already has its index in the summary, so recurse only into that cluster. The expensive branches are exclusive.



## Insert 1: keep the new min outside and place the old min in $C_0$

Insert 1 into the original {2,3,7,18,21,30}:

| Step | State | Invariant |
|---|---|---|
| Swap | min=1; pending x=2 | New min stays outside |
| Split | high(2)=0, low(2)=2 | Top-level L=4 |
| $C_0$ | {3} becomes {2,3} | Local min=2, max=3 |
| Summary | Still {0,1,4,5,7} | $C_0$ was already nonempty |
| Finish | max remains 30 | Largest key unchanged |

$C_0$ has u=4, so its own split uses L=2. Its old local min 3 moves to local cluster 1 as offset 1. Coordinate systems change at each recursive level.

## Delete: handle singletons and u=2 first

DELETE-PRESENT requires that x exists. Handle these two cases before the next page's general procedure:

```
DELETE-PRESENT(V,x):
    if V.min==V.max:
        V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=1-x; return
    DELETE-GENERAL(V,x)
```

The first branch is a nonempty singleton by contract; all its clusters were already empty. At u=2 with two keys, the set is {0,1}, so deleting x leaves 1−x.

Calling this procedure on an absent key can erase valid data. A public interface first checks MEMBER and returns NOT_FOUND on failure; the precondition must be explicit.



## General deletion: promote the next min, then remove its cluster copy

```
DELETE-GENERAL(V,x):         # u>2; at least two keys
    if x==V.min:
        k=V.summary.min
        x=index(k,V.cluster[k].min); V.min=x
    i=high(x); j=low(x); C=V.cluster[i]
    DELETE-PRESENT(C,j)
    if C.min==NIL:
        DELETE-PRESENT(V.summary,i)
        if x==V.max:
            k=V.summary.max
            if k==NIL: V.max=V.min
            else: V.max=index(k,V.cluster[k].max)
    elif x==V.max:
        V.max=index(i,C.max)
```

## Why deletion rebinds x to the promoted minimum

Promotion changes x to the new min. The recursive removal deletes its old cluster representation. Update the cluster and summary before reconstructing max; the previous largest cluster may no longer exist.

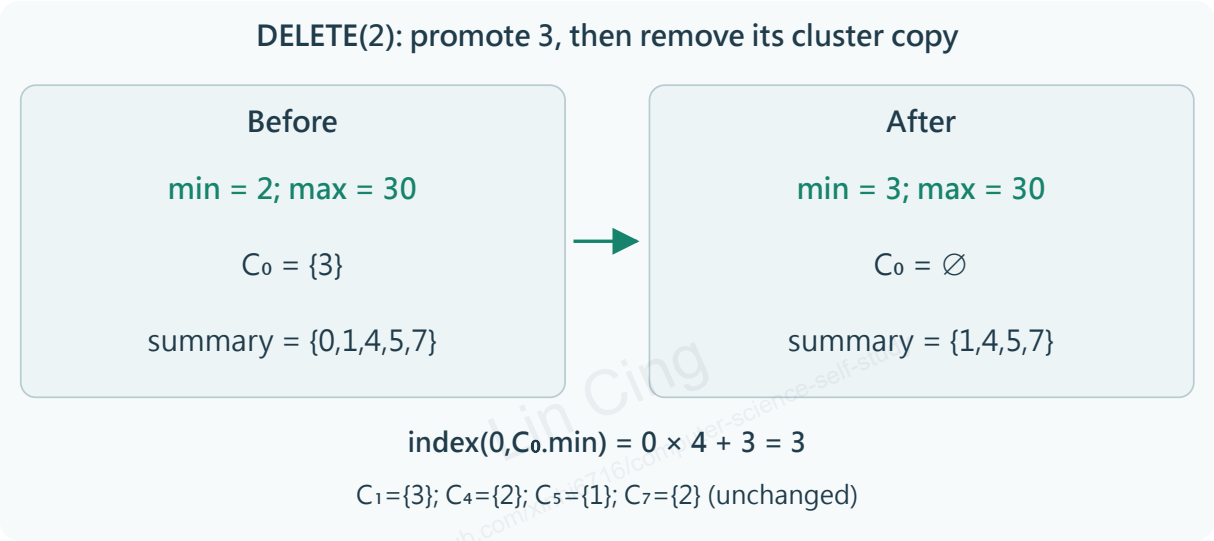
For {2,3}, delete 2: promotion sets x=3 and min=3; removing 3 empties $C_0$ and then the summary. Because the rebound x equals old max 3, set max=min=3, leaving the correct singleton.

With 7,18,21,30 also present, the rebound x=3 differs from max 30, so max stays unchanged. Every later comparison uses the current x, not the original argument 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Delete 2: promote 3 without shifting the other keys




Promote 3 to the top-level min, empty $C_0$, and remove summary index 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Check promotion and summary removal

Restart from the original set. summary.min=0 and $C_0$.min=3, so the new min is 3. Removing offset 3 empties $C_0$; deleting index 0 from the summary leaves {1,4,5,7}. max stays 30.

The promoted 3 now lives **only at the top level**. Keeping it in $C_0$ would violate the invariant. Here the cluster deletion removes a singleton and costs O(1); the summary is the only potentially expensive recursive path.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Deleting maxima and the final two elements

From the original set, delete 30,21,18,7,3,2 in order:

| Removed | Remaining min / max | Top-level summary |
|---|---|---|
| 30 | 2 / 21 | {0,1,4,5} |
| 21 | 2 / 18 | {0,1,4} |
| 18 | 2 / 7 | {0,1} |
| 7 | 2 / 3 | {0} |
| 3 | 2 / 2 | Empty |
| 2 | NIL / NIL | Empty |

After deleting 3, no cluster remains, so max becomes the detached min 2. Never use NIL from an empty summary as an array index. Deleting the final 2 clears extrema; all recursive structures are already empty.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Why two deletion calls still leave one expensive path

General deletion may call DELETE(cluster) and DELETE(summary), but the second occurs only if the cluster becomes empty. That means the cluster **was a singleton**, so its deletion took O(1).

| Previous cluster | Cluster deletion | Remove summary index? |
|---|---|---|
| Singleton | O(1) | Yes; may recurse |
| At least two keys | May recurse | No; remains nonempty |

Hence the cost is $\max\{T(L), T(H)\} + O(1)$, not $T(L) + T(H) + O(1)$. The representation and branch conditions prove a **worst-case** bound; neither probability nor amortization is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Solve the recurrence in bit width

Let T(u) bound an operation and set $R(m) = T(2^m)$. One nonconstant recursive call reaches a child with at most $\lceil m/2 \rceil$ bits:

$$R(m) \le R(\lceil m/2 \rceil) + c, \qquad R(1) = O(1).$$

For m≥2, $\lceil m/2 \rceil \le 2m/3$, so at most $\lceil \log_{3/2} m \rceil$ levels reach the base. Constant work per level gives $O(1 + \log m) = O(1 + \log \log u)$.

Starting at m=5, a longest width path is 5→3→2→1; a cluster path can be 5→2→1. Odd widths do not have identical square-root child universes. The additive 1 handles the constant-time u=2 base.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Correctness through the represented set

Induct on bit width m. At m=1, enumerate the empty set, {0}, {1}, and {0,1}. Larger sets are uniquely reconstructed from the detached min and all cluster offsets.

**Queries:** extrema handle boundaries. If a same-cluster answer exists, no higher cluster can beat it; otherwise the summary selects the next nonempty cluster. Predecessor also considers detached min.

**Insertion:** swapping a new minimum with the old one relocates a key without losing it. Insert the pending key into its cluster and add a summary index exactly when that cluster first becomes nonempty.

**Deletion:** promote the least cluster key when deleting min, then remove its cluster copy; otherwise remove the requested key. Remove indices of newly empty clusters and reconstruct max from updated state. The induction hypothesis makes child operations correct, establishing the exact set change.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## An empty structure still costs Θ(u) space and initialization

The fully allocated version pays allocation before operations. For u>2, there are H clusters of universe L and a summary of universe H. Local pointer/field space is at most cH:

$$P(u) \le HP(L) + P(H) + cH, \qquad P(2) \le p_0.$$

Prove $P(u) \le A(u-1) - B$ by strong induction. Choose B≥c and A≥B+$p_0$; substitution gives

$$P(u) \le A[H(L-1) + (H-1)] - B(H+1) + cH \le A(u-1) - B.$$

Thus space is O(u). Cluster-only base nodes already require Ω(u), giving Θ(u). Visiting and initializing all nodes also takes Θ(u). This proof permits unequal H and L.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Public interfaces, duplicate keys, and space tradeoffs

| Case | Public contract |
|---|---|
| Duplicate INSERT(x) | MEMBER first; return ALREADY_PRESENT |
| Absent DELETE(x) | MEMBER first; return NOT_FOUND |
| x<0 or x≥u | Reject before indexing |
| Multiplicity needed | Maintain counts; change key set only on 0↔1 |
| Satellite data | Specify key→record mapping and ownership |

A membership check plus an update still costs O(1+log log u). Multiplicity assumes constant-time, nonoverflowing counters; duplicate keys must not enter INSERT-ABSENT directly.

The full tree is appropriate only when u is affordable. Lazy allocation and hash-table clusters require a new space/allocation/expected-time analysis; they do not inherit this deterministic worst-case proof. A balanced O(n)-space tree may be preferable for a sparse, huge universe.



## Keep amortized, expected, and worst-case guarantees distinct

| Structure / operation | Guarantee | Essential assumptions |
|---|---|---|
| Red-black query/update | Worst-case O(log(n+2)) | Comparisons; O(n) space |
| Hash lookup | Expected O(1+α) | Stated hashing model; no ordered neighbors |
| Fibonacci decrease-key | Amortized O(1) | Valid handle; shared sequence potential |
| Full vEB predecessor/update | Worst-case O(1+log log u) | Integer word RAM; Θ(u) space/setup |

Building a full vEB and performing q operations costs $O(u + q(1 + \log\log u))$. Initialization cannot disappear from the total. Comparison keys, integer universes, and randomized models impose different contracts; a context-free ranking of structures is misleading.

# References and attribution

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing, personal handwritten data-structure notes: queues, search trees, red-black trees, multiway search trees, and heaps.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Disjoint sets: merge groups, never split them

The partition changes as groups merge. The query asks: **are these two elements in the same group now?**

| Question | Method |
|---|---|
| How do we identify a set? | A list's set record, or a forest root |
| How much must a merge change? | Move the shorter list; attach the lower-rank root |
| Why can later finds get cheaper? | Path compression and amortized sequence analysis |

Prerequisites: pointers, rooted trees, asymptotic notation, and the potential method. Rank, level, and iter are defined here.



## Three operations maintain one partition

Every created element belongs to exactly one nonempty set; distinct sets are disjoint.

| Operation | Precondition | Result |
|---|---|---|
| MAKE-SET(x) | x has not been created | Create singleton {x} |
| FIND(x) | x exists | Return its set's representative |
| UNION(x,y) | Both elements exist | Merge their sets; do nothing if already together |

**A representative identifies a set; it need not be its smallest key.** Finds on one set return the same representative; different sets have different representatives.

## Count calls separately from successful merges

Create n singletons first. Let m count external MAKE-SET, FIND, and UNION calls, so $m \geq n$. Do not count UNION's internal FIND calls again in m.

| Quantity | Contract |
|---|---|
| Successful merges | At most n−1, because each removes one set |
| UNION calls | Repeated calls are allowed; no n−1 bound |
| LINK(r,s) | Accepts two **distinct roots**, not arbitrary elements |

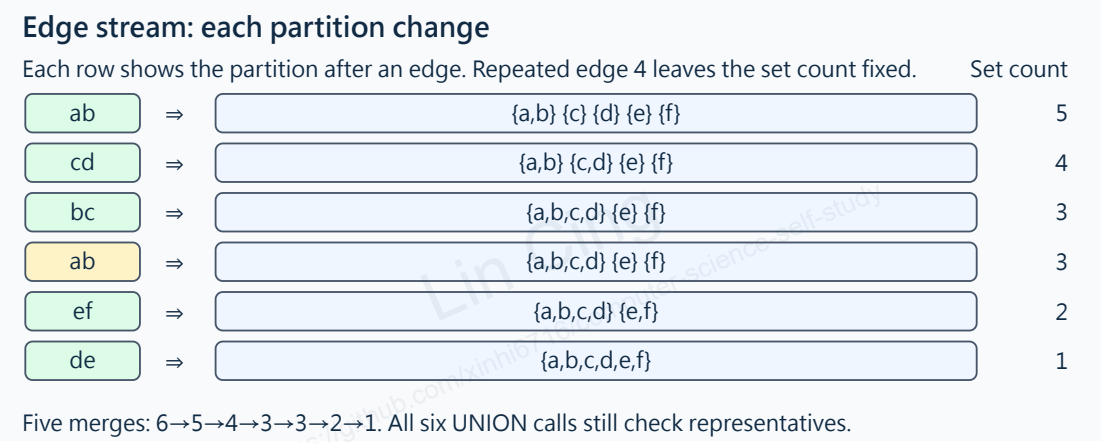
> UNION finds both roots and returns immediately if they agree. **Never LINK a root to itself or increase its rank for a repeated merge.**

FIND may change parents while preserving the partition. Deletion, splitting, and rollback require additional structures and analysis.



## Undirected connectivity: what does a new edge merge?

Create a set for each vertex, then UNION each edge. The diagram uses ab, cd, bc, ab, ef, de.




**The fourth edge ab is repeated: the call counts, but the partition does not change.** Six UNION calls cause five successful merges.

## Why are these exactly the connected components?

Invariant: the components of the processed-edge subgraph equal the maintained partition.

| Step | Connectivity | Partition |
|---|---|---|
| No edges yet | Each vertex is isolated | One singleton per vertex |
| New edge within a component | Already connected | UNION changes nothing |
| New edge between components | Joins exactly those two components | UNION merges exactly their two sets |

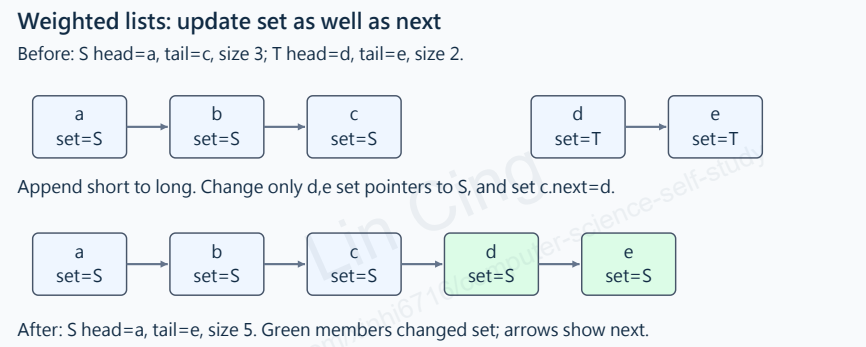
Induction gives FIND(u)=FIND(v) exactly when an undirected u–v path exists.

DFS/BFS finds static components in $\Theta(n+|E|)$. DSU also supports **connectivity queries as edges arrive**. Ignoring edge directions yields weak components, not SCCs.



## Lists: joining tails does not finish the merge

A set record stores head, tail, size. Each member stores next and a set pointer. FIND(x) returns x.set.head in O(1).




> Merging a→b→c with d→e changes tail c→e and size 3→5. **Both d.set and e.set must point to the retained record** before their finds return a.

## Finish every step of weighted-list UNION

Let S=x.set and T=y.set. Return for the same record; otherwise retain the longer list as S, with first-set ties.

| Order | Update | Purpose |
|---|---|---|
| 1 | S.tail.next=T.head | Join the chains |
| 2 | Preserve T's old head; update S.tail and S.size | Keep the traversal start and update metadata |
| 3 | Follow T's next links; redirect every member's set to S | Make their finds return S.head |
| 4 | Discard T's record only after all redirects | Leave no dangling set pointers |

Do not change T's internal next links during traversal. The merge costs $\Theta(1+\min(|S|,|T|))$: **a tail pointer makes joining constant-time, not the whole UNION.**



## Why can unweighted lists take quadratic work?

Always append the second set to the first. In UNION($x_2$,$x_1$), UNION($x_3$,$x_2$), …, each new singleton absorbs the existing long list.

| Merge | Members whose set pointers change |
|---|---|
| UNION($x_2$,$x_1$) | 1 |
| UNION($x_3$,$x_2$) | 2 |
| … | … |
| UNION($x_n$,$x_{n-1}$) | n−1 |

Total rewrites are $n(n-1)/2$. There are only n creations and n−1 unions, so $m=2n-1$, yet the cost is $\Theta(n^2)$.

## Moving the shorter list: how often can one member move?

Fix member x. Whenever its set pointer changes, it belonged to the shorter list; its new set is **at least twice as large**.

After k moves its set has at least $2^k$ members, at most n. Thus $k \leq \lfloor \log_2 n \rfloor$. Sum over members and add the m constant call overheads: **$O(m+n \log n)$**, or $O(m+n \log(n+1))$ including small n.

| Quantity | Bound or example |
|---|---|
| Whole sequence | $O(m+n \log n)$ |
| One merge of two n/2-member sets | Still $\Theta(n)$ |
| Balanced rounds with $n=2^k$ | n/2 rewrites per round; $(n/2)\log_2 n$ total |

> A sequence bound is not a per-call worst-case bound. Weighted union does not make every sequence linear.



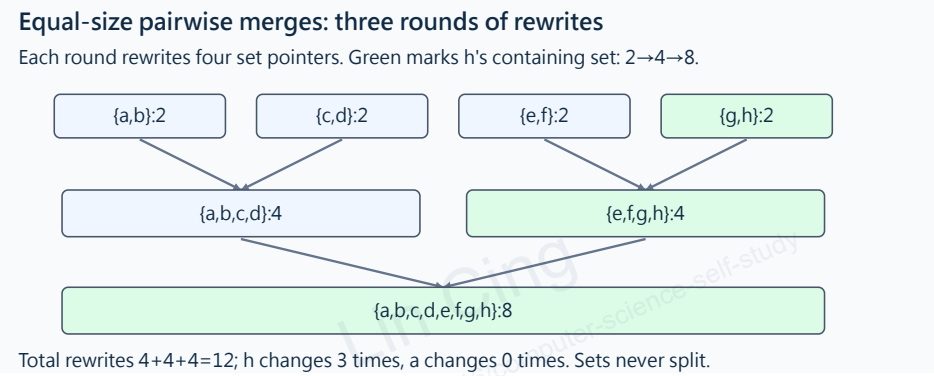
## Eight elements: exactly which members move?

Retain the first set on equal sizes. The same operations will later build a forest.

| UNION | Old sizes | Members rewritten | New representative / size | Cumulative |
|---|---|---|---|---|
| a,b | 1,1 | b | a / 2 | 1 |
| c,d | 1,1 | d | c / 2 | 2 |
| e,f | 1,1 | f | e / 2 | 3 |
| g,h | 1,1 | h | g / 2 | 4 |
| a,c | 2,2 | c,d | a / 4 | 6 |
| e,g | 2,2 | g,h | e / 4 | 8 |
| a,e | 4,4 | e,f,g,h | a / 8 | 12 |

Member h follows h→g→e→a as its set grows 1→2→4→8; a never moves. **Twelve total rewrites do not mean every member moves three times.**

## One diagram connects round totals to one member's growth




Round totals and one member's move count have different denominators. The table sums to 12 rewrites; h alone moves three times.

Attaching only new singletons to a large set can instead take n−1 rewrites. This meets the upper bound; it does not make every possible sequence linear.



## Forests: find the roots before linking

Each element stores parent and rank; a root points to itself. MAKE-SET sets parent[x]=x and rank[x]=0. UNION must first find both roots. Linking arbitrary x directly to y can split an existing set or create a cycle.

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    if r==s: return r
    return LINK(r,s)
LINK(r,s):                    // r and s are distinct roots
    if rank[r]<rank[s]: swap r,s
    parent[s]=r
    if rank[r]==rank[s]: rank[r]+=1
    return r
```

After linking roots, all old members still reach the new root. One root loses its self-parent, with no reverse path creating a cycle. Unequal ranks leave the retained rank unchanged; equal ranks increase it by one. Rank forests and size-weighted lists are different representations. Rank is not exact set size.

## Two-pass FIND: locate the root, then follow the old path

Starting at x, follow parent pointers until reaching a self-parent root. Pass one only finds root r. Pass two redirects each node on the **original search path** directly to r.

```
FIND(x):
    r=x
    while parent[r]!=r: r=parent[r]
    while parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    return r
```

Variable old preserves the next stop before mutation. Using x=parent[x] instead reads the root just written and skips intermediate nodes. Recursive parent[x]=FIND(parent[x]) redirects nodes as calls return.

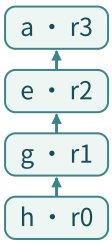
The two-pass version uses O(1) auxiliary space; the recursive stack scales with original path height. Both take Θ(s) work for s visited nodes, change only this path, and preserve ranks.



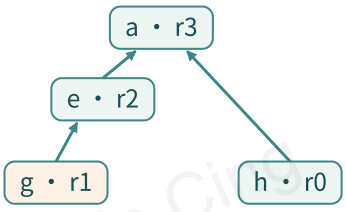
## Finding the right root is not full path compression

Arrows point **child→parent**. Only h→g→e→a is drawn; a is self-parent.

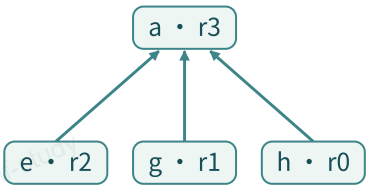
Original path




Jump to root after writing




Save old parent; finish path




| Current node in pass two | Save old | Assign parent | Continue at |
|---|---|---|---|
| h | g | a | g |
| g | e | a | e |
| e | a | a, unchanged | a, then stop |

## The same eight elements before and after compression

For the seven earlier unions, equal ranks retain the first root. The successive root ranks are a1,c1,e1,g1,a2,e2,a3. Final parents are b→a, c→a, d→c, e→a, f→e, g→e, h→g; a points to itself.

| Vertex | a | b | c | d | e | f | g | h |
|---|---|---|---|---|---|---|---|---|
| rank | 3 | 0 | 1 | 0 | 2 | 0 | 1 | 0 |
| Parent before FIND(h) | a | a | a | c | a | e | e | g |
| Parent after FIND(h) | a | a | a | c | a | e | a | a |

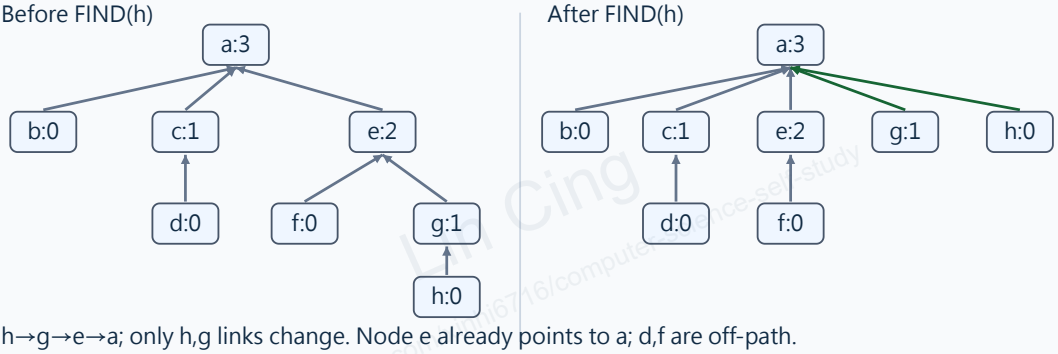
- Pass one follows h→g→e→a: three parent edges.
- Only **h and g change parent**; e already points to a. Off-path nodes do not change.
- **Every rank stays fixed.** Compression does not replace historical ranks with current heights.



## Read the tree: redirect the path, retain the branches

### The same eight-node tree: parents before and after compression

Nodes show key:rank; arrows show parent; root self-loops omitted; green links changed.




h→g→e→a; only h,g links change. Node e already points to a; d,f are off-path.

> A later FIND(h) takes one edge; FIND(d) still follows d→c→a. **One FIND compresses only its own search path.**

## Why does rank bound tree height?

Rank is an **upper bound**, rather than a measurement of current height. Induct on operations:

| Operation | Effect on height | Why height $\leq$ rank survives |
|---|---|---|
| Make a singleton | Height is 0 | Initial rank is also 0 |
| Merge unequal ranks | Attach the lower-rank root | The rank gap pays for its extra parent edge |
| Merge equal ranks | Height increases by at most 1 | The retained root's rank also increases by 1 |
| Compress a path | Redirect to ancestors, shortening paths | Ranks stay fixed |

> **Do not recompute rank after compression.** A rank-2 root may now have height 1.



## Rank remembers the members accumulated during merging

A root's rank rises only by merging two different sets of equal rank:

- Rank 0 requires at least one member.
- Two rank-(r−1) sets each contain at least $2^{r-1}$ members. Their merged rank-r root has at least $2^r$.
- Unequal-rank merges add members; FIND preserves set membership.

A nonroot's rank freezes when it loses root status. **Its set then had at least $2^r$ members.** Thus $2^r \leq n$, giving $r \leq \lfloor \log_2 n \rfloor$. This is not a bound on its current subtree size.

> Rank alone gives $O(\log n)$ per FIND. Compression improves the **amortized sequence bound**; a first deep-leaf query can still traverse $\Theta(\log n)$ edges.

## 21.4 Fine-grained analysis of rank and compression

### Ackermann iteration means composition

Fix the final number of elements $n$ and set $\alpha = \alpha(n)$ throughout the analysis, even while MAKE-SET calls are still creating elements. For $j \geq 1$, define

$$f^{(0)}(j) = j, \quad f^{(i+1)}(j) = f(f^{(i)}(j)), \quad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j) \ (k \geq 1).$$

The superscript counts **repeated applications to the previous output**. Since $A_0^{(i)}(j) = j + i$, we get $A_1(j) = 2j + 1$. Induction gives $A_1^{(i)}(j) = 2^i(j + 1) - 1$, hence

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

Inductively, each $A_k$ is strictly increasing in its input and satisfies $A_k(j) \geq j + 1$: a larger starting input undergoes more iterations, each increasing its input. Since $j + 1 \geq 2$, also $A_k(j) > A_{k-1}(j)$. Both monotonicities are used below.



## The chosen inverse-Ackermann convention

Use the single-parameter convention of CLRS third edition, Section 21.4:

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

| k | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $A_k(1)$ | 2 | 3 | 7 | 2047 | greater than $10^{80}$ |

Thus $\alpha(2) = 0$, $\alpha(8) = 3$, and $\alpha(2048) = 4$. Other texts may use a two-parameter convention; its numerical values cannot be substituted here.

**The complete bound is** $O(m(1 + \alpha(n)))$ for $m$ external operations, including $n$ MAKE-SET calls. Only for $n \geq 3$ can it be shortened to $O(m\alpha(n))$. Small instances still require constant work per operation.

## Which quantities can change?

Induction on operations shows that a root points to itself and every nonroot has smaller rank than its parent. Linking equal-rank roots raises the retained root's rank by one. Unequal ranks attach the smaller rank under the larger. Compression changes parents, never ranks.

**Once a node becomes a nonroot, its rank is fixed.** Its parent's rank may increase, or compression may replace the parent by a higher ancestor. Neither event decreases parent rank. This is why a node's level cannot decrease over time.

A rank-$r$ root contains at least $2^r$ elements. A nonroot retains that historical bound from losing root status. Hence every rank is at most $\lfloor \log_2 n \rfloor < n$, regardless of the current compressed subtree size.

Define level and iter only for positive-rank nonroots. Rank-zero nodes and roots receive separate potential formulas.



## Level selects the scale; iter measures progress within it

For a nonroot, write $r = \operatorname{rank}(x) \geq 1$, $p = \operatorname{rank}(\operatorname{parent}(x))$, and define

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

Here $\ell = \operatorname{level}$ and $i = \operatorname{iter}$. First find the highest scale reached by parent rank; then count iterations at that scale.

Since $p \geq r + 1 = A_0(r)$, we have $\ell \geq 0$. Also $A_\alpha(r) \geq A_\alpha(1) \geq n > p$, so $\ell < \alpha$. The definition gives $i \geq 1$. If $i \geq r + 1$, then

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

contradicting maximality of $\ell$. Therefore $0 \leq \ell < \alpha$ **and** $1 \leq i \leq r$.

## Nonnegative potential records work still available

Assign a potential to each node and sum over the forest:

$$\phi(x) = \begin{cases} \alpha r, & x \text{ is a root or } r = 0, \\ (\alpha - \ell(x))r - i(x), & x \text{ is a nonroot and } r \geq 1, \end{cases} \qquad \Phi = \sum_x \phi(x).$$

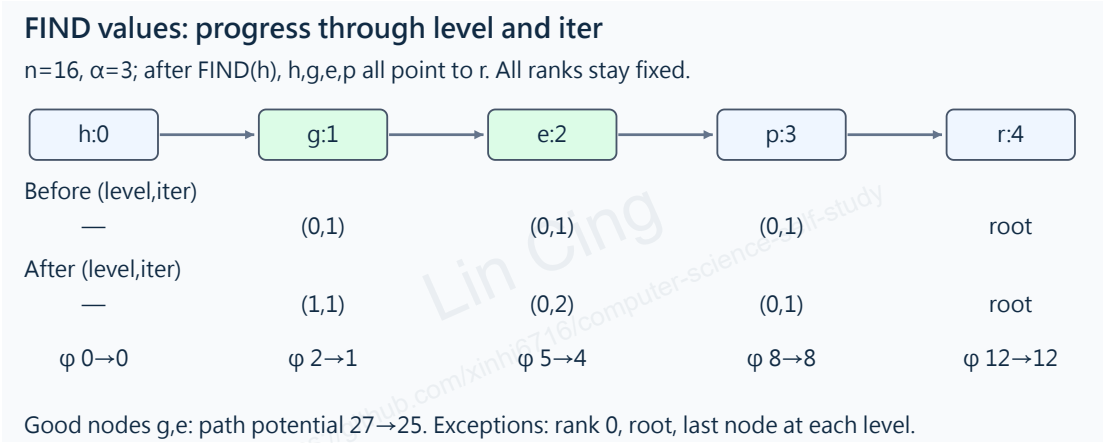
For nonroots, the lower bound is $(\alpha - (\alpha - 1))r - r = 0$, and the upper bound is $\alpha r - 1$. Roots have potential $\alpha r$; rank-zero nodes have zero potential. Thus $\Phi$ is always nonnegative.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Potential accounting on a five-node path

Take $n = 16$, $\alpha = 3$, and compress a path of ranks 0, 1, 2, 3, 4. The root keeps its self-parent.




Each good node releases one unit: g gains a level; e increases iter within one level. The other three potentials stay fixed, so the path total falls from 27 to 25.

## Why can a rising level reset iter?

Fix the rank $r$ of a nonroot. Within one level, increasing parent rank leaves iter unchanged or increases it. Each additional iteration decreases $\phi$ by one.

If the level rises from $k$ to $k' > k$, iter may reset from $i$ to a smaller $i'$. Nevertheless,

$$\Delta\phi = -(k' - k)r + (i - i') \leq -r + (r - 1) = -1.$$

This uses $k' - k \geq 1$, $i \leq r$, and $i' \geq 1$. **The gain from changing level pays for resetting iter.** The pair progresses in lexicographic order, comparing level first, then iter. Its two coordinates need not increase independently.

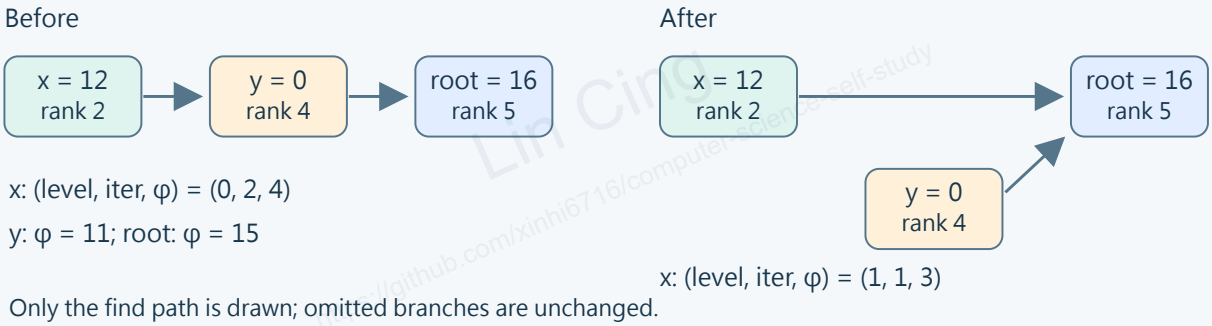


## A legal FIND that really decreases iter

Take $n = 32$ and $\alpha = 3$. The find path is $12 \to 0 \to 16$, with ranks $2, 4, 5$. Unvisited branches are omitted and remain unchanged.

### A higher level can reset iter from 2 to 1

n=32, α=3; FIND(12)




Only the find path is drawn; omitted branches are unchanged.

## Computing the reset example term by term

Node $x = 12$ keeps rank 2. Only its parent rank changes, from 4 to 5.

| Quantity | Before compression | After compression |
|---|---|---|
| level | $A_0(2) = 3 \le 4 < A_1(2) = 5$, so 0 | $A_1(2) = 5 \le 5 < A_2(2) = 23$, so 1 |
| iter | $A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$, so 2 | $A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$, so 1 |
| $\phi(x)$ | $(3-0)2 - 2 = 4$ | $(3-1)2 - 1 = 3$ |

Node 0 keeps potential 11, and the root keeps potential 15. All other branches are unchanged. Thus the forest has $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$. The displayed totals sum only the path; the other 29 nodes need not have zero potential.



## Constructing that path with legal operations

Create singleton keys 0 through 31. Equal-rank merges retain the first root.

| Operation | Checkable result |
|---|---|
| Merge adjacent equal-size blocks of 0..15 in rounds | Root 0 has rank 4; node 12 has rank 2 |
| FIND(12) | Parent of 12 becomes 0; ranks stay fixed |
| Build 16..31 by the same rounds | Root 16 has rank 4 |
| UNION(16,0) | Root 16 rises to rank 5; parent of 0 becomes 16 |
| FIND(12) again | Path $12 \to 0 \to 16$; parent of 12 becomes 16 |

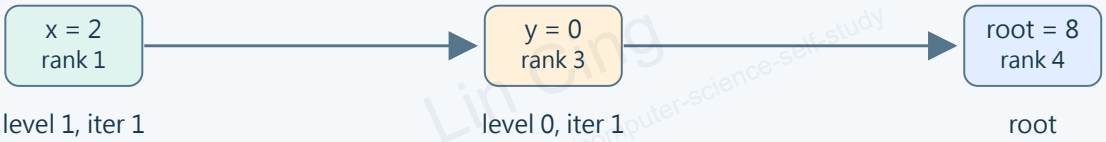
The first half starts with (0,1), (2,3), …, (14,15), then (0,2), (4,6), …, followed by (0,4), (8,12), and finally (0,8). These ranks come from a legal history rather than arbitrary assignments to a drawn chain.

## Levels need not increase along a path

Take $n = 16$. Merge 0..7 into root 0 of rank 3, then call FIND(2). Build 8..15 into root 8 of rank 3 and call UNION(8,0). This creates the following legal path.

### Increasing ranks do not imply increasing levels

n=16, α=3; FIND(2)




$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

Along this path: ranks 1 < 3 < 4, but levels 1 > 0.

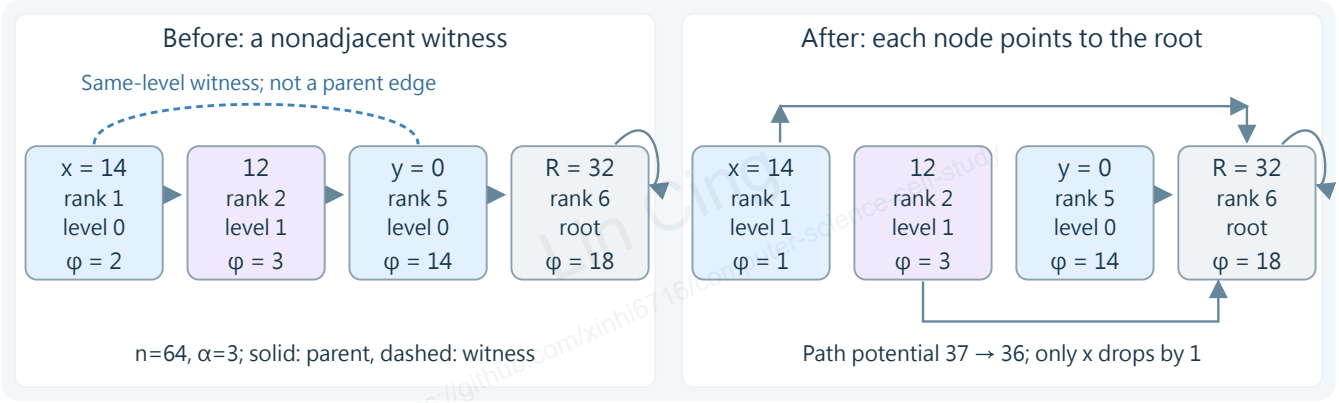
Count the last node of each level; no sorted-level assumption is needed.

Comparing different nodes changes the input rank too. Temporal monotonicity concerns one fixed node.



## A same-level witness need not be the next node

A positive-rank nonroot x is good if a later nonroot y on the old path has the same level. **The witness need not be x's parent.**




In 14→12→0→32, levels are 0,1,0. Node 0 witnesses that 14 is good; node 12 may have another level. The root has no level.

## Construct the path through legal operations

Start with 64 singletons. Equal-rank UNION retains its first root. Building an interval in rounds means merging adjacent singletons, then adjacent equal-sized blocks.

| Operation | Fact established by the history |
|---|---|
| Build 0..31 in rounds | Root 0 has rank 5; node 12 has rank 2 and node 14 rank 1 |
| FIND(12) | Old path is $12 \to 8 \to 0$; attach 12 to 0, while 14 still points to 12 |
| Build 32..63 in rounds | Root 32 has rank 5 |
| UNION(32,0) | Equal ranks: 32 rises to 6, and 0 points to 32 |
| Before FIND(14) | Path $14 \to 12 \to 0 \to 32$, with ranks 1,2,5,6 |

FIND(12) follows $12 \to 8 \to 0$ upward to the root, then attaches each visited nonroot directly to that root. It never enters a child subtree, so 14 still points to 12. The later union attaches root 0 to root 32. These ranks follow from legal operations; 32 retains its self-parent.



## Compute every level, iteration count, and potential

Fix $n = 64, \alpha = 3$. Level and iter apply only to positive-rank nonroots; the root uses $\phi = \alpha r$.

| Node | Rank | Parent rank: old→new | Level: old→new | Iter: old→new | $\phi$: old→new |
|---|---|---|---|---|---|
| 14 | 1 | 2→6 | 0→1 | 1→1 | 2→1 |
| 12 | 2 | 5→6 | 1→1 | 1→1 | 3→3 |
| 0 | 5 | 6→6 | 0→0 | 1→1 | 14→14 |
| 32, root | 6 | 6→6 | Not defined | Not defined | 18→18 |

For 14, initially $A_0(1) = 2 \le 2 < A_1(1) = 3$, so level=0,iter=1. Afterwards $A_1(1) = 3 \le 6 < A_2(1) = 7$, and $A_1^{(2)}(1) = 7 > 6$, so level=1,iter=1. Its potential drops from $(3-0)1 - 1 = 2$ to $(3-1)1 - 1 = 1$.

For 12, $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$, so level=1,iter=1. For 0, $A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$, so level=0,iter=1,potential=14. Path total changes $37 \to 36$. All other nodes keep rank,parent rank,root status; **the whole forest also loses exactly one unit**.

## The good-node inequality, with a reason for each step

Freeze all quantities before compression. Write $\rho(v) = \mathrm{rank}(v)$, $p_x = \mathrm{parent}(x)$, $p_y = \mathrm{parent}(y)$, and let $R$ be the root. Put $r = \rho(x), k = \ell(x) = \ell(y), i = i(x)$.

| Step | Reason |
|---|---|
| $\rho(R) \geq \rho(p_y)$ | $p_y$ is on the root path; rank strictly increases along nonroot parent edges |
| $\rho(p_y) \geq A_k(\rho(y))$ | Definition of $\ell(y) = k$ |
| $A_k(\rho(y)) \geq A_k(\rho(p_x))$ | $y$ equals or is above $p_x$, so $\rho(y) \geq \rho(p_x)$; $A_k$ increases |
| $A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$ | Iter gives $\rho(p_x) \geq A_k^{(i)}(r)$; use monotonicity again |
| $A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$ | One more function composition, not multiplication |

The new parent rank is therefore at least $A_k^{(i+1)}(r)$. No step assumes sorted path levels: the proof uses ancestor rank order and monotonicity at one fixed level $k$.



## Crossing a threshold pays for a good node

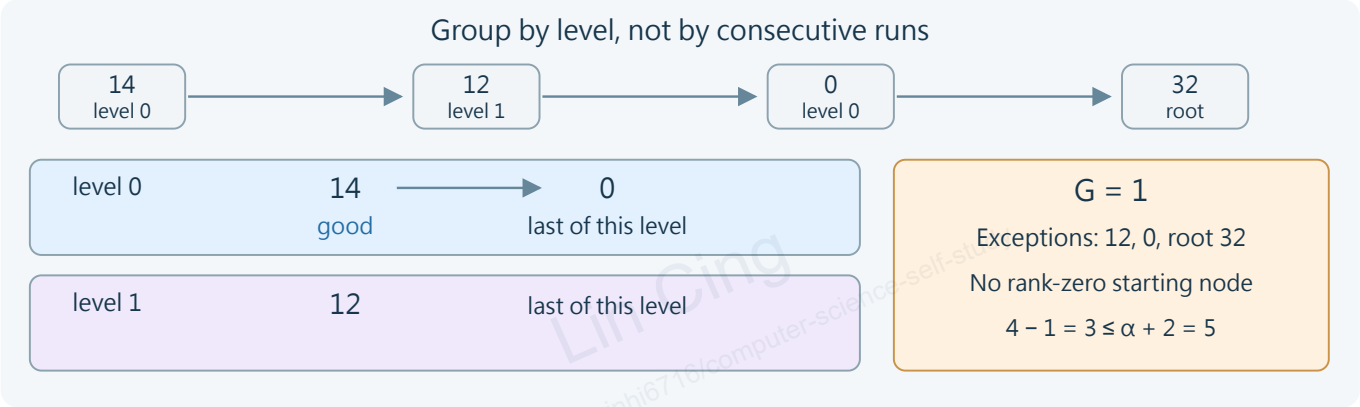
Compression makes $R$ the parent of $x$, keeping $r$ fixed. The proof crosses the **next iteration threshold of the old level**. It does not guarantee that the new iter exceeds the old one; distinguish two cases.

| After compression | Potential change | Why it decreases |
|---|---|---|
| Level stays $k$ | $\Delta\phi = -(i' - i) \leq -1$ | Iter is at least $i + 1$ |
| Level rises | $\Delta\phi = -(k' - k)r + i - i'$ | $k' - k \geq 1, i \leq r, i' \geq 1$, so $\Delta\phi \leq -r + (r - 1) = -1$ |

In the 64-node example, $x = 14, y = 0, k = 0, i = 1$; the inequalities become $6 \geq 6 \geq 6 \geq 3 \geq 3$. The next old-level threshold is $A_0^{(2)}(1) = 3$. New parent rank 6 also crosses $A_1(1) = 3$, so this is the second case and potential drops by one.

Charge the decrease to good node $x$, not again to witness $y$; $y$'s potential need not drop. Other nodes' potentials do not increase, so the good-node decreases may be summed.

## Only the last node of each level is an exception




Within one level, every node except the last has a later same-level witness. Here 14 is good; 12 and 0 are the last nodes of levels 1 and 0. Adding root 32 gives three exceptions. Treating consecutive runs 0/1/0 as three groups would miss good node 14.



## Convert group counts into total potential decrease

An $s$-node path has at most $\alpha$ positive-rank nonroot levels, numbered $0, \dots, \alpha - 1$. At most one last node per level, the root, and a possible rank-zero starting node give at most $\alpha + 2$ exceptions. Strictly increasing ranks allow at most one rank-zero node.

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

$G$ counts actual good nodes; $g$ is only a conservative lower bound. Here $s = 4, \alpha = 3$, hence $g = 0$, yet $G = 1$ and $\Delta\Phi = -1$. A short path need not attain the bound. The next argument uses the general lower bound to pay for long paths.

## Fixed constants pay for the entire scan

Suppose two-pass find costs at most $cs$. Fix $C > c$ and use scaled potential $\Psi = C\Phi$. The amortized cost satisfies

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

| Path length | Bound after substituting $g$ |
|---|---|
| $s \le \alpha + 2$ | $cs \le c(\alpha + 2)$ |
| $s > \alpha + 2$ | $(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$ |

Thus each FIND has amortized cost $O(1 + \alpha)$. For $s = 7$, $\alpha = 3$, $c = 2$, $C = 3$, the lower bound is $g = 2$, so $\hat{c} \le 14 - 6 = 8$. Keep the same $C$ throughout the sequence.



## Only the retained LINK root can gain potential

MAKE-SET adds a rank-zero node with zero potential and still has amortized cost $O(1)$. Suppose LINK attaches old root $u$ under retained root $v$.

| Possibly affected nodes | Reason for the potential change |
|---|---|
| $u$ | Loses root status without changing rank; potential does not rise |
| Old children of $v$ | Parent rank cannot fall; the earlier monotonicity argument applies |
| $v$ | Unequal ranks leave it fixed; equal ranks add one, increasing potential by $\alpha$ |

Every other node keeps its rank, parent rank and root status. Therefore $\Delta\Psi \le C\alpha$, and LINK costs $O(1) + C\alpha = O(1 + \alpha)$ amortized. UNION performs two FIND calls and links only distinct roots, expanding into at most three operations.

## Recovering actual sequence cost

Let $c_j$ be actual cost and $a_j = c_j + \Psi_j - \Psi_{j-1}$ amortized cost. Summing cancels intermediate potentials:

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \leq \sum_{j=1}^{m'} a_j.$$

The inequality uses the empty initial forest, $\Psi_0 = 0$, and nonnegative final potential. Expanding UNION gives $m' \leq 3m$, so the original $m$ operations cost $O(m(1 + \alpha(n)))$ in total.

This is a worst-case bound on the entire sequence. With rank and compression, a single FIND can still cost $\Theta(\log n)$: build a balanced rank tree and first query a deepest leaf. A linear chain requires removing the rank rule and does not satisfy this theorem's hypotheses.

Potential analysis reference: CLRS, third edition, Section 21.4, printed pp. 576–581.



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

# Dynamic programming

Recursion splits a problem into subproblems. Dynamic programming also identifies repeated subproblems and solves each state once.

Read **states and dependencies → rod cutting → knapsack → longest common subsequences and alignment → matrix chains and optimal binary search trees**.

Prerequisites are recursion, asymptotic analysis, arrays, and trees. Connect the problem definition, recurrence, evaluation order, and reconstruction into one algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Decide what a state means first

| Design question | What the answer must establish |
| --- | --- |
| Is the state sufficient? | Equal states have the same future choices and costs. |
| Is the recurrence complete? | Every legal optimal solution belongs to a candidate branch. |
| Which states come first? | Boundaries and dependencies determine a valid evaluation order. |
| What must be returned? | Values, stored choices, and reconstructed solutions serve different purposes. |

For each model, fix the legal solutions and cost before comparing time, space, and the ability to reconstruct an optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Dynamic programming starts with state meaning

Dynamic programming (DP) merges repeated subproblems into states, stores their answers, and combines them to solve the original instance.

- **State meaning:** Which feasible solutions does one entry optimize over?
- **Recurrence and proof:** Do candidates cover every optimum, and are all candidates feasible?
- **Evaluation and reconstruction:** Solve dependencies first; record a choice attaining each optimum.

Optimal substructure justifies composition. Overlap explains the benefit of caching. Neither replaces a precise state definition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A state must retain everything that affects the future

Two histories may share a state only if they have the same feasible continuations, future cost rules, and remaining objective.

Past value can be accounted for separately. Information that changes future choices belongs in the state.

For rod cutting, prices depend only on piece length, cuts are free, and there is no piece-count limit. Every remainder of length $j$ has the same options, so $r[j]$ suffices.

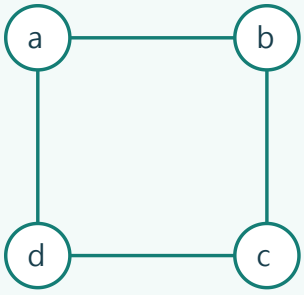
With an additional limit of $k$ pieces, remaining length alone is insufficient: retain the remaining piece allowance too.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Longest simple paths: endpoints omit a resource constraint

The undirected graph has edges $ab, bc, cd, da$, each of length 1. A simple path cannot repeat vertices.

### Same endpoints do not mean the same available vertices




Whole optimum: a → b → c → d

Suffix: c → d (1)

Independent longest: c → b → a → d (3)

Splice: a → b → c → b → a → d

Repeated a, b: infeasible

$a \to b \to c \to d$ is a longest simple path from $a$ to $d$, of length 3. Its suffix $c \to d$ is not longest between its endpoints: $c \to b \to a \to d$ has length 3. Substitution yields $a, b, c, b, a, d$, which is infeasible.

The endpoint state omits vertices consumed by the prefix; independently optimal subpaths need not compose.



## A richer state repairs correctness at exponential cost

Fix a start $s$. Define $D[S, v]$ as the longest simple path from $s$ to $v$ using exactly vertex set $S$; use $-\infty$ when none exists.

$$D[\{s\}, s] = 0, \qquad D[S, v] = \max_{\substack{u \in S \setminus \{v\} \\ uv \in E}} (D[S \setminus \{v\}, u] + 1).$$

Other base cases are $-\infty$. Evaluate nonbase states only for $s, v \in S$ and $v \neq s$. The answer is $\max_{S \ni s,t} D[S, t]$.

Set size strictly increases, giving a state DAG. Time is at most $O(2^{|V|}|V|^2)$: this is exponential DP. If the original graph is a DAG, longest paths can instead use endpoint states in topological order.

## Rod cutting: assumptions and state

Given integer length $n \geq 0$ and prices $p[1..n]$, cut the whole rod into positive integer lengths and sell every piece. Cuts are free; a length may occur repeatedly.

$$r[j] = \text{maximum revenue using exactly length } j, \qquad r[0] = 0.$$

Example prices, in revenue units:

| Length $i$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p[i]$ | 1 | 5 | 8 | 9 |

Find $r[4]$ and the piece lengths. Selling the entire rod uncut is feasible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Choose the first finished piece

Every nonempty decomposition has a first piece of length $i$, with $1 \leq i \leq j$. Sell that piece directly; only its remainder is subdivided further.

$$r[j] = \max_{1 \leq i \leq j} \{p[i] + r[j-i]\}, \qquad j \geq 1.$$

- $i = j$ represents no cut: $p[j] + r[0]$.
- Use $p[i]$, not $r[i]$: here $i$ denotes a finished piece.
- The remainder is strictly shorter, enabling induction on $j$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Prove both directions of the recurrence

Let $OPT(j)$ be the true optimum. Assume all shorter states are correct.

**Upper-bound direction:** An optimal decomposition starts with some $i$. Its remaining revenue is at most $r[j-i]$. Hence $OPT(j) \le p[i] + r[j-i] \le \max_i(p[i] + r[j-i])$.

**Lower-bound direction:** For every $i$, one piece of length $i$ followed by an optimal decomposition of length $j-i$ is feasible and uses exactly $j$. Thus $p[i] + r[j-i] \le OPT(j)$; taking a maximum preserves the inequality.

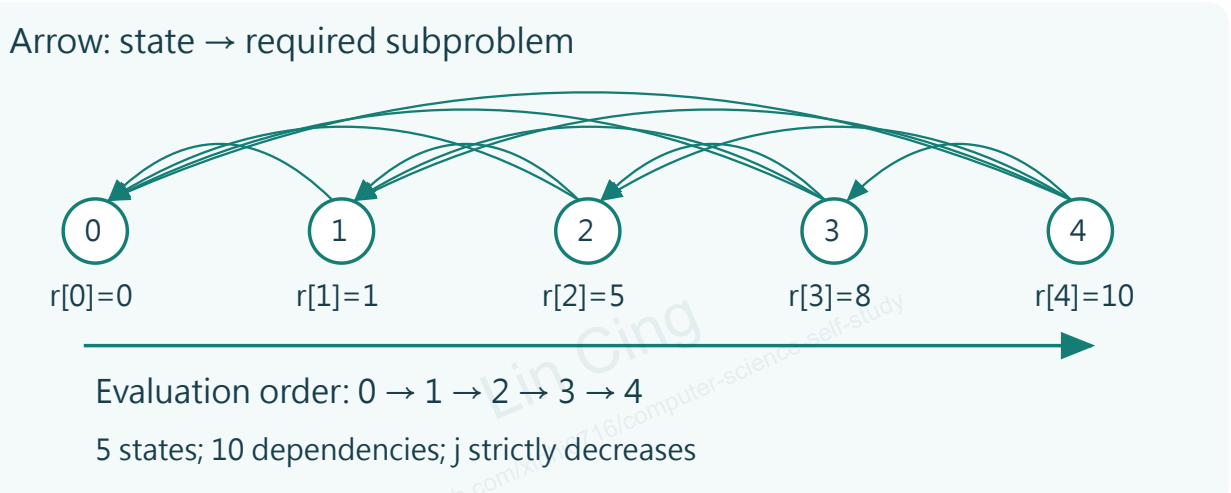
Together these give equality. Replacing the remainder preserves feasibility of the first piece: this is the essential optimal-substructure argument.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Overlap becomes a dependency DAG

An arrow $j \to k$ means that evaluating $r[j]$ requires $r[k]$.




Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Evaluate shorter dependencies before longer states

Every $k < j$ is a candidate dependency of $r[j]$. For example, both $r[3]$ and $r[4]$ require $r[2]$. Merging equal-length calls stores that answer once.

For length 4, the dependency DAG has 5 states, $0, 1, 2, 3, 4$. State $j$ has $j$ outgoing dependency edges, so the total is $1 + 2 + 3 + 4 = 10$.

Evaluate $0, 1, 2, 3, 4$, opposite to the dependency-arrow direction. When $r[j]$ is evaluated, every needed $r[k]$ is already complete. The strictly smaller dependency length prevents a cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Memoization expands each state once

```
memo[0] = 0; all other entries are UNKNOWN
solve(j):
    if memo[j] is not UNKNOWN: return memo[j]
    q = -∞
    for i = 1 .. j:
        q = max(q, p[i] + solve(j-i))
    memo[j] = q
    return q
```

The target requests its dependencies recursively. Repeated requests return cached values. Use a separate `UNKNOWN` marker; a negative revenue can be a valid answer.

Every length is reachable in this problem: time $\Theta(n^2)$, table space $\Theta(n)$, and at most $O(n)$ recursive stack space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Tabulation also records the first piece

```
r[0] = 0
for j = 1 .. n:
    r[j] = -∞
    for i = 1 .. j:           # increasing i
        q = p[i] + r[j-i]
        if q > r[j]:         # ties retain smaller first piece
            r[j] = q
            first[j] = i
```

Loop invariant: when length $j$ starts, $r[0..j-1]$ are correct. Every candidate reads only completed entries.

Time is $\sum_{j=1}^{n} j = n(n+1)/2 = \Theta(n^2)$. Value and choice arrays together take $\Theta(n)$ space.



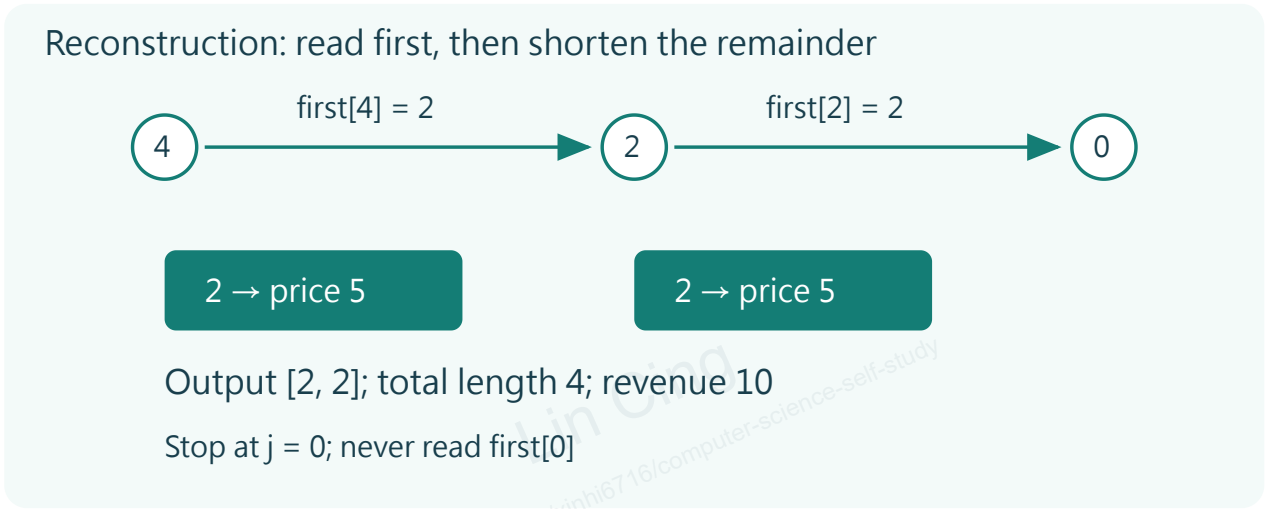
## A complete candidate trace

List $p[i] + r[j-i]$ in increasing order $i = 1, 2, \ldots, j$.

| $j$ | Candidate revenues, increasing $i$ | Running maximum | $r[j]$ | first[$j$] |
|---|---|---|---|---|
| 0 | Empty decomposition | — | 0 | — |
| 1 | $1+0=1$ | 1 | 1 | 1 |
| 2 | $1+1=2,\ 5+0=5$ | 2, 5 | 5 | 2 |
| 3 | $1+5=6,\ 5+1=6,\ 8+0=8$ | 6, 6, 8 | 8 | 3 |
| 4 | $1+8=9,\ 5+5=10,\ 8+1=9,\ 9+0=9$ | 9, 10, 10, 10 | 10 | 2 |

The highest individual selling price need not be the best first piece: $p[4] = 9$, while two pieces of length 2 earn 10.

## Follow recorded choices to recover the optimal cuts




Here $first[4] = 2$ and $first[2] = 2$. The remaining length follows $4 \to 2 \to 0$, outputting $[2, 2]$. Check length $2 + 2 = 4$ and revenue $5 + 5 = 10 = r[4]$.



## Why recorded cuts preserve the optimum

At remaining length $j$, the emitted pieces have total length $n - j$, and their revenue plus $r[j]$ equals $r[n]$. Initially $j = n$ and no pieces have been emitted, so both claims hold.

Let $i = first[j]$. Emitting $i$ and replacing $j$ by $j - i$ increases emitted length by $i$. Because $r[j] = p[i] + r[j - i]$, the emitted revenue rises by exactly the amount removed from the remaining optimal revenue. Both claims are preserved.

At $j = 0$, the emitted length is $n$ and $r[0] = 0$, so emitted revenue is $r[n]$. In the example, emitting 2 twice gives total length 4 and revenue 10.

## Reconstruct until the remaining length is zero

After tabulation, `first[j]` records an optimal first piece for every positive length $j$. Follow those choices to produce the complete list.

```
j = n; pieces = []
while j > 0:
    i = first[j]
    append i to pieces
    j = j - i
return pieces
```

Every choice satisfies $1 \leq first[j] \leq j$. Each step reduces $j$ by at least 1, so the loop stops at zero after at most $n$ iterations and takes $O(n)$ time. The invariant then gives total length $n$ and revenue $r[n]$.

For $n = 0$, the loop does not run and returns an empty list.



## Two schedules for the same dependency graph

| Aspect | Top-down memoization | Bottom-up tabulation |
|---|---|---|
| Start | Request dependencies from the target | Evaluate states in a valid order |
| States evaluated | Reachable states only | Usually a prescribed range |
| Extra overhead | Calls and cache lookups | Usually no recursive stack |
| This example: time / table | $\Theta(n^2)$ / $\Theta(n)$ | $\Theta(n^2)$ / $\Theta(n)$ |

Memoization is not automatically faster; tabulation need not use a rectangular table. Both need a correct recurrence and an acyclic dependency schedule.

## Boundary conditions change the problem

**Use the entire rod:** For $p[1] = -2, p[2] = -5, r[1] = -2$ and $r[2] = \max(-4, -5) = -4$. Initialize each maximum to $-\infty$, not zero.

**Allow discarded material:** Change the recurrence to $r[j] = \max(0, \max_i(p[i] + r[j - i]))$. The same negative-price example now has value 0. Reconstruction must also record "stop and discard."

**Empty input:** With $n = 0$, prices may be empty; revenue is 0 and the decomposition is empty. For $n > 0$, this chapter assumes a price for every length $1..n$; a missing price is not zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Recheck the recurrence when assumptions change

Suppose each actual cut costs $c \geq 0$. Selling the entire remaining rod involves no cut.

$$r[j] = \max_{1 \leq i \leq j} \left(p[i] + r[j - i] - c\,\mathbf{1}_{i<j}\right).$$

For the original prices with $c = 2$, two pieces of length 2 earn $5 + 5 - 2 = 8$; selling length 4 uncut earns 9 and is now optimal.

Length still suffices as a state. Piece-count constraints or history-dependent prices may require more dimensions: check whether state alone determines future feasibility and cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Five checks for a DP design

1. Define the feasible solution set of each state and its bases.
2. Choose a first or last decision and enumerate every feasible candidate.
3. Prove that candidates cover an optimum and compose into feasible solutions.
4. Find a strictly decreasing measure and a valid evaluation order.
5. Count states and candidate work; verify reconstruction separately.

For rod cutting: length states, a first-piece decision, a two-direction proof, decreasing length, and quadratic time. Validate the output by checking both total length and revenue.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 0/1 knapsack: one copy per item

There are $n$ items with positive integer weights $w_i$, values $v_i$, and a nonnegative integer capacity $W$.

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \le W, \quad x_i \in \{0, 1\}.$$

The empty set is allowed, so the optimum is at least zero; negative-value items can be omitted. Equal weights or values do not make two items the same item. Integer capacities support this array-indexed DP. Arbitrary real weights cannot directly index the table.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why the state includes an item prefix

Let $D[i, c]$ be the maximum value using only items $1, \ldots, i$ with total weight at most $c$. The answer is $D[n, W]$. Remaining capacity alone is insufficient: different available items can give different answers at the same capacity. A prefix identifies the available set and prevents reuse of an item. For a fixed $(i, c)$, the available choices depend only on that prefix and capacity. Of histories reaching the same state, an inferior value cannot improve an extension. This assumes additive values and no constraints linking items.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



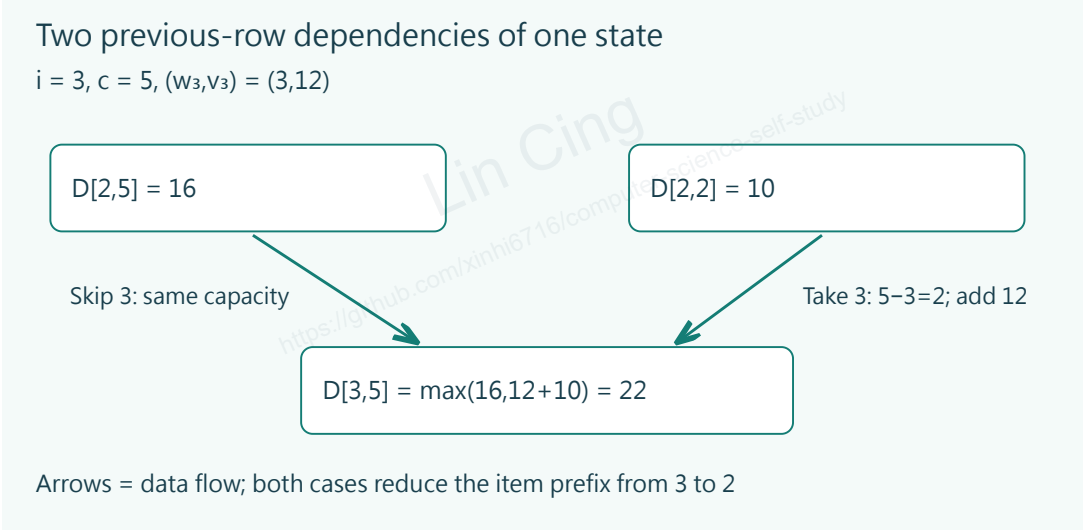
## Two disjoint cases give the recurrence

$$D[0, c] = 0, \qquad D[i, 0] = 0.$$

If $c < w_i$, then $D[i, c] = D[i-1, c]$. Otherwise,

$$D[i, c] = \max\{D[i-1, c],\ v_i + D[i-1, c - w_i]\}.$$

Skipping $i$ preserves capacity. Taking $i$ consumes $w_i$. Both cases leave only the first $i - 1$ items.



## Proving both inequalities

Consider any feasible set $S$ for $(i, c)$. If $i \notin S$, its value is at most $D[i-1, c]$. If $i \in S$, removing $i$ leaves a set among the first $i-1$ items of weight at most $c - w_i$, so its value is at most $v_i + D[i-1, c-w_i]$. Thus the optimum is at most the maximum candidate. Conversely, an optimal set for the skip case is feasible unchanged. An optimal set for the take case excludes $i$; adding $i$ preserves feasibility. Both candidates are attainable, so their maximum is at most the optimum. Together with the base case, induction on $i$ proves correctness. Equivalently, if the remainder after removing $i$ were inferior, replacing it and adding $i$ back would contradict optimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## One example: inputs and table order

Capacity $W = 5$; item pairs $(w_i, v_i)$ are $(2, 10), (1, 6), (3, 12)$.

Rows are available prefixes and columns are capacity limits, not required exact weights.

Each cell reads only the previous row. Fill rows in order $i = 1, 2, 3$; capacities within a row may be visited in any order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One example, the complete table

Complete table and traceback: W = 5

Items (w,v): 1=(2,10), 2=(1,6), 3=(3,12)

| Prefix i / capacity c | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| i = 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| i = 1 | 0 | 0 | 10 | 10 | 10 | 10 |
| i = 2 | 0 | 6 | 10 | 16 | 16 | 16 |
| i = 3 | 0 | 6 | 10 | 16 | 18 | 22 |

Arrows = traceback: take 3 → skip 2 → take 1; weight 5, value 22

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Reading the comparisons

The completed table gives $D[3,5] = 22$.

Item 1: $D[1,1] = 0$ (too heavy), and $D[1,2] = \max(0, 10+0) = 10$. Item 2: $D[2,3] = \max(D[1,3], 6 + D[1,2]) = \max(10, 16) = 16$. Item 3: $D[3,3] = \max(16, 12 + D[2,0]) = 16$: fitting does not force selection. Also $D[3,4] = \max(16, 12+6) = 18$ and $D[3,5] = \max(16, 12+10) = 22$. A take candidate adds the current item's value to a feasible optimum for the specified prefix and residual capacity, not to an arbitrary maximum in the same row.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Traceback recovers the chosen items

Start at $(i, c) = (n, W)$. If $D[i, c] = D[i-1, c]$, skip $i$ and decrement $i$. Otherwise take $i$, set $c \leftarrow c - w_i$, and decrement $i$. On equality, consistently skip to return one optimum. Here $(3, 5) : 22 > 16$, so take 3 and move to $(2, 2)$. Since $10 = 10$, skip 2 and move to $(1, 2)$. Since $10 > 0$, take 1 and reach $(0, 0)$. The result is $\{1, 3\}$, weight $2 + 3 = 5$, value $10 + 12 = 22$. Each step decreases $i$, giving $O(n)$ traceback. Capacity decrease alone is not the loop invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A complete two-dimensional algorithm

```
D[0..n][0..W] = 0
for i = 1..n:
    for c = 0..W:
        D[i,c] = D[i-1,c]
        if c >= w[i]:
            D[i,c] = max(D[i,c], v[i] + D[i-1,c-w[i]])
```

There are $(n + 1)(W + 1)$ cells and constant work per cell. Time is $O(n(W + 1))$ and full-table space is $O(n(W + 1))$, conventionally $O(nW)$ when $W \geq 1$. The retained table supports the preceding traceback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One dimension: descend to preserve the old row

```
dp[0..W] = 0
for i = 1..n:
    for c = W down to w[i]:
        dp[c] = max(dp[c], v[i] + dp[c-w[i]])
```

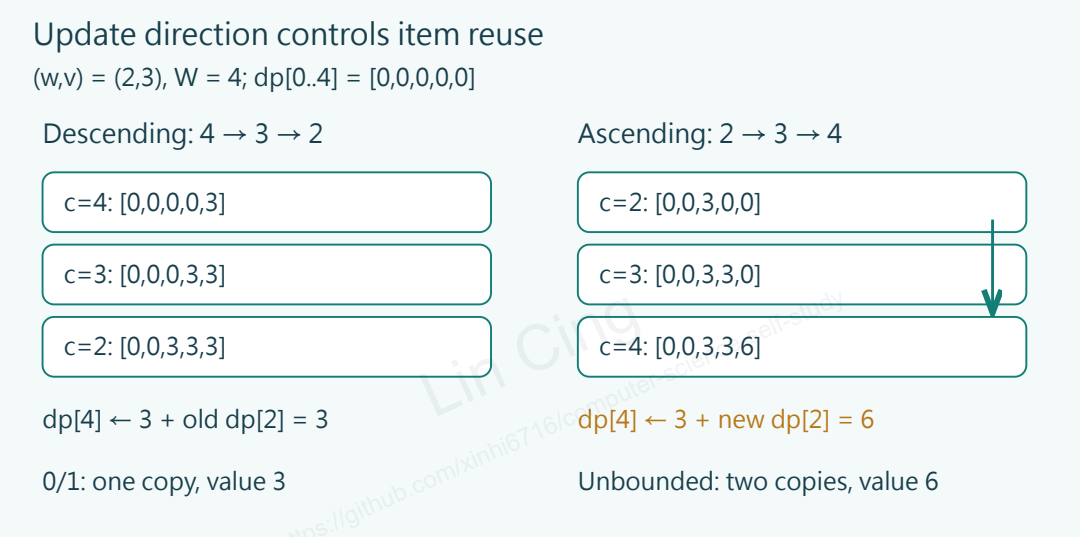
During item $i$, unmodified capacities hold $D[i-1,\cdot]$. Descending order and $w_i > 0$ ensure $c - w_i < c$ is still unmodified. The right side reads the previous row; afterward $dp[c] = D[i,c]$. Time remains $O(nW)$; value storage falls to $O(W)$. Overwritten history does not directly support two-dimensional traceback. To recover items, retain the full table or use versioned decision records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Ascending order reuses the same item

There is one item $(w,v) = (2,3)$ and capacity $W = 4$. The 0/1 optimum must be 3.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ascending updates first write $dp[2] = 3$. At $c = 4$, the new $dp[2]$ yields 6, equivalent to taking two copies of the single item. Loop direction selects the version of the state being read.

## Unbounded knapsack needs the current row

If each item type has arbitrarily many copies, let $U[i, c]$ be the maximum value using the first $i$ types within capacity $c$. When the item fits,

$$U[i, c] = \max\{U[i-1, c],\ v_i + U[i, c - w_i]\}.$$

After taking a copy, type $i$ remains available, so the second term uses the same row. An ascending capacity loop implements this dependency in one dimension. Since $w_i > 0$, the same-row dependency has strictly smaller capacity. For $(2, 3), W = 4$, two copies give 6, so the preceding ascending result matches this model. Zero-weight items are excluded here: unlimited copies of a zero-weight positive-value type make the objective unbounded.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exact weight: infeasible does not mean zero

Let $E[i, c]$ be the maximum value using the first $i$ items with total weight **exactly** $c$. Use the same 0/1 recurrence, but initialize

$$E[0, 0] = 0, \quad E[0, c] = -\infty\ (c > 0).$$

An infeasible candidate stays $-\infty$. Check reachability before addition to avoid overflow from integer sentinels. Positive weights imply $E[i, 0] = 0$. For a single item $(2, -3), W = 2$, the at-most optimum is 0 (empty), whereas the exact-weight optimum is $-3$. For $W = 1$, exact weight is infeasible. Initializing every cell to zero invents feasible weights.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why the bound is pseudopolynomial

$O(nW)$ is polynomial in the numerical capacity $W$, but not necessarily in binary input length. Encoding capacity uses about $\log_2(W+1)$ bits; if $W = 2^b$, the capacity loop takes $2^b$ steps. For 20 items, capacity $10^3$ means about 20,000 updates; capacity $10^9$ means about 20 billion. Indexable integer weights do not guarantee a practical table. This analysis treats each value addition and comparison as constant cost. Very large integer values also require arithmetic bit costs. The decision version of 0/1 knapsack is NP-complete; the optimization version is NP-hard.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why fractional knapsack admits greedy choice

With $0 \le x_i \le 1$, items may be split. Process positive-value items in decreasing density $v_i/w_i$; omit negative values. If lower-density weight is selected while higher-density supply remains, exchanging equal weight $\delta$ cannot decrease value. Repeated exchanges yield the density-greedy solution. Use the same items with capacity changed to $W = 4$. Densities are 6 for item 2, 5 for item 1, and 4 for item 3. The fractional optimum takes 2, 1, and $1/3$ of 3, giving $6 + 10 + 4 = 20$. The 0/1 density rule takes 2 and 1 for 16; the 0/1 optimum takes 2 and 3 for 18. Equal-weight exchanges may be impossible without splitting. Sorting gives $O(n \log n)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A branch-and-bound bound is not a DP value

For maximization, branch and bound explores a take/skip search tree while keeping an incumbent feasible value $L$. Let $V$ be the value already taken and $C$ the remaining capacity. Let $F(C)$ be the fractional optimum using only undecided items. The total upper bound at the node is $B = V + F(C)$. Relaxation enlarges the feasible set, so every 0/1 completion at the node has value at most $B$. If $B \leq L$, prune the node when only one optimum is required. In the previous example, the root has $B = 20$ while the integer optimum is 18: a bound need not be attainable. A DP entry $D[i, c]$ is the **exact** optimum of its defined subproblem. These quantities serve different purposes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Subsequences preserve order, not adjacency

A subsequence uses strictly increasing indices. A substring must also be contiguous.

Fix $X =$ `ABCBDAB` and $Y =$ `BDCABA`. `BCBA` uses indices $(2, 3, 4, 6)$ in $X$ and $(1, 3, 5, 6)$ in $Y$. Its length is 4. `BCA` is common but is not longest.

First compute the **length** of a longest common subsequence (LCS); then reconstruct one witness.

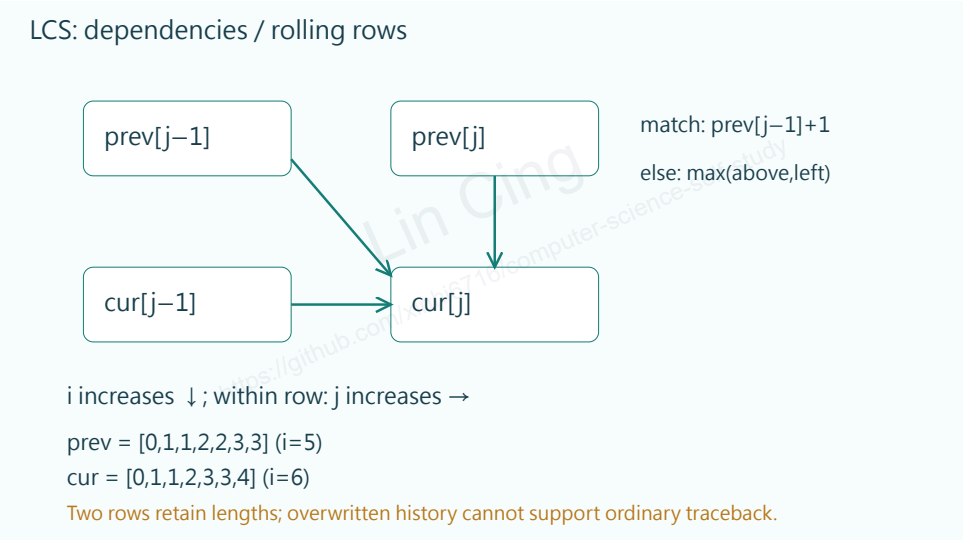
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A state is a pair of prefixes

Let $C[i, j]$ be the LCS length of $X[1..i]$ and $Y[1..j]$. The answer is $C[7, 6]$.

$C[0, j] = C[i, 0] = 0$: an empty prefix has only the empty subsequence.

Dependencies decrease $i + j$, forming a DAG. Fill rows $i = 1..m$, scanning $j = 1..n$ within each row. Above, left, and diagonal predecessors are then ready.






## Equal final symbols: an achievable lower bound

If $x_i = y_j = a$, append $a$ to an LCS of the two shorter prefixes. The resulting indices are legal, so

$$C[i, j] \geq C[i - 1, j - 1] + 1.$$

This constructs a feasible candidate. At $(6, 6)$ both final symbols are A and $C[5, 5] = 3$, so length 4 is achievable. A lower bound alone does not prove optimality.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Equal final symbols: the matching upper bound

Take any LCS $Z$ of length $k$. Its last symbol must be $a$: otherwise both final positions are unused, and appending $a$ would make a longer common subsequence.

Remove the last $a$ of $Z$. The remaining symbols embed in both shorter prefixes, hence $k - 1 \leq C[i-1, j-1]$. Combining both bounds gives

$$C[i,j] = C[i-1,j-1] + 1.$$

An optimal embedding using both final positions exists. This does not force every optimal index matching to use them: for A versus AA, either A position can be matched.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Unequal final symbols: prove both inequalities

If $x_i \neq y_j$, an embedding of a common subsequence cannot use both final positions: both would have to match its same last symbol.

Thus an optimal embedding omits $x_i$ or $y_j$, so its length is at most $\max(C[i-1,j], C[i,j-1])$.

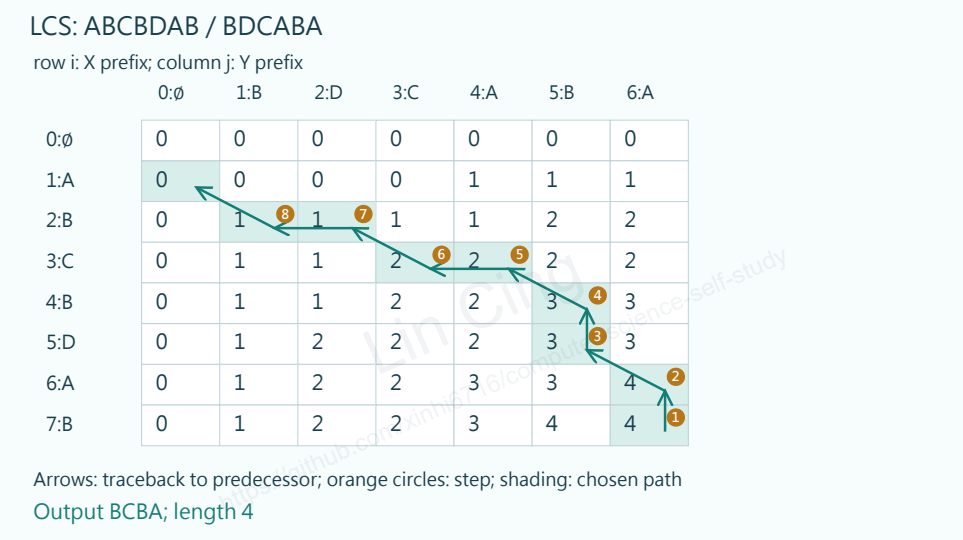
Conversely, every common subsequence of either shorter-prefix pair remains feasible in the full pair. Choosing the longer candidate attains that bound. Therefore

$$C[i,j] = \max\{C[i-1,j], C[i,j-1]\}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The complete length table

Equal symbols use diagonal +1; unequal symbols use max(above, left). Keep the zero boundary.




$C[3,3] = C[2,2] + 1 = 2$; $C[5,5] = \max(3,2) = 3$; $C[7,6] = \max(4,4) = 4$. Numbered arrows identify the exact traceback on the next card.



# Traceback: reverse ABCB to obtain BCBA

For unequal symbols, prefer above when above and left tie. For equal symbols, move diagonally and record the symbol.

| Step | Current → predecessor | Reason / record |
|---|---|---|
| 1 | $(7,6) \to (6,6)$ | both 4; prefer above |
| 2 | $(6,6) \to (5,5)$ | A=A; record A |
| 3 | $(5,5) \to (4,5)$ | above 3 > left 2 |
| 4 | $(4,5) \to (3,4)$ | B=B; record B |
| 5 | $(3,4) \to (3,3)$ | left 2 > above 1 |
| 6 | $(3,3) \to (2,2)$ | C=C; record C |
| 7 | $(2,2) \to (2,1)$ | left 1 > above 0 |
| 8 | $(2,1) \to (1,0)$ | B=B; record B; stop |

## Ties choose a witness, not its length

Recorded backward: ABCB. Reverse it to obtain BCBA.

BCBA, BCAB, and BDAB are all LCSs of length 4. A different tie rule may return another string without changing the optimum length.

At $(7, 6)$, moving left to $(7, 5)$ is also optimal. Preferring above simply makes this example deterministic.

One stored predecessor reconstructs one answer. Enumerating all distinct LCSs requires exploring all optimal branches and removing duplicates. The number of traceback paths need not equal the number of distinct strings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Computing a value and rebuilding a witness

The table has $(m + 1)(n + 1)$ states, each initialized or computed in constant time: both time and space are $\Theta((m + 1)(n + 1))$. When $m, n \geq 1$, this simplifies to $\Theta(mn)$; the full expression also covers empty strings.

Traceback decreases at least one index at every step, taking $O(m + n)$ time. Store a choice table, or keep the full value table and compare predecessors during traceback.

Brute-force index-subset enumeration has $2^m$ candidates on one side. Many recursive routes ask for the same prefix pair. DP merges those repeated states.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rolling rows preserve lengths, not the history

Use the shorter string for columns. `prev` is the completed preceding row; `cur` is the row being filled from left to right.

```
prev[0..n] = 0
cur[0..n] = 0
for i = 1..m:
    cur[0] = 0
    for j = 1..n:
        if X[i] == Y[j]: cur[j] = prev[j-1] + 1
        else: cur[j] = max(prev[j], cur[j-1])
    swap(prev, cur)
return prev[n]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rolling rows and witness reconstruction

Time is $\Theta((m+1)(n+1))$ and auxiliary space is $O(\min(m,n)+1)$. For $m, n \geq 1$, these simplify to $\Theta(mn)$ and $O(\min(m,n))$. This code returns only a length.

Each update reads the previous row and the current row's left neighbor. Once the rows swap, the old history can be overwritten because later length calculations no longer need it.

Overwritten rows no longer support ordinary full-table traceback. To output a sequence, retain the full value/choice table or use another reconstruction algorithm, such as divide and conquer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Global alignment accounts for every symbol

Fix $X = \texttt{ACGTA}$ and $Y = \texttt{AGCAC}$. Insert gaps `-` to make equally long rows. A `-/-` column is forbidden. Removing gaps must recover the original strings.

| Column type | Score |
|---|---|
| matching symbols | +3 |
| mismatching symbols | −2 |
| one symbol and one gap | −3 |

Maximize the sum of all column scores, including leading and trailing gaps. This is global alignment with a linear gap cost. Its score is not an LCS length.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Alignment state and complete initialization

$F[i, j]$ is the best global alignment score of prefixes $X[1..i]$ and $Y[1..j]$.

$$F[0, 0] = 0, \qquad F[i, 0] = -3i, \qquad F[0, j] = -3j.$$

If only one prefix is nonempty, each symbol must face a gap. Both boundaries are `0, -3, -6, -9, -12, -15`.

Zero boundaries would allow an unmatched prefix for free and change the problem. Adding a zero candidate also changes the specified global-alignment recurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The final column has three possible forms

Let $s(a,b) = 3$ for equal symbols and $-2$ otherwise. For $i, j > 0$:

$$F[i,j] = \max \begin{cases} F[i-1, j-1] + s(x_i, y_j) & \text{symbol / symbol} \\ F[i-1, j] - 3 & x_i \text{ / gap} \\ F[i, j-1] - 3 & \text{gap / } y_j. \end{cases}$$

Every legal alignment ends in one of these forms. Removing its last column leaves the corresponding prefix alignment, bounding its score by the maximum above. Conversely, append each listed column to an optimal predecessor alignment to attain that candidate. Both inequalities establish the recurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

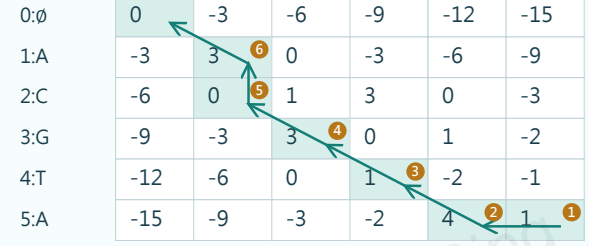


## A complete alignment table: optimum 1

Global alignment: ACGTA / AGCAC

row i: X prefix; column j: Y prefix

| | 0:ø | 1:A | 2:G | 3:C | 4:A | 5:C |
|---|---|---|---|---|---|---|
| 0:ø | 0 | -3 | -6 | -9 | -12 | -15 |
| 1:A | -3 | 3 | 0 | -3 | -6 | -9 |
| 2:C | -6 | 0 | 1 | 3 | 0 | -3 |
| 3:G | -9 | -3 | 3 | 0 | 1 | -2 |
| 4:T | -12 | -6 | 0 | 1 | -2 | -1 |
| 5:A | -15 | -9 | -3 | -2 | 4 | 1 |



Arrows: traceback to predecessor; orange circles: step; shading: chosen path

Output ACGTA- / A-GCAC; total score 1

$F[2,3] = \max(0+3, -3-3, 1-3) = 3.$

## Reconstruct every alignment column

Trace backward from $(5, 5)$:

| Current → predecessor | Column | Added score |
|---|---|---|
| $(5,5) \to (5,4)$ | `-/C` | −3 |
| $(5,4) \to (4,3)$ | `A/A` | +3 |
| $(4,3) \to (3,2)$ | `T/C` | −2 |
| $(3,2) \to (2,1)$ | `G/G` | +3 |
| $(2,1) \to (1,1)$ | `C/-` | −3 |
| $(1,1) \to (0,0)$ | `A/A` | +3 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Read the reconstructed alignment forward

Reverse the column order:

```
ACGTA-
A-GCAC
```

Total score: $3 - 3 + 3 - 2 + 3 - 3 = 1$. Both original sequences are fully represented.

The two gap columns account for C in $X$ and the final C in $Y$. The T/C mismatch contributes $-2$; the three matching columns contribute $3 + 3 + 3$.

$F[5, 5] = \max(-2 - 2, -1 - 3, 4 - 3) = 1$, so its final column is `-/C`. Interior scores may be negative even when the final score is positive; global alignment does not reset them to zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Similar grids, different meanings

| Problem | State value | Equal symbols | Unequal symbols | Boundary |
|---|---|---|---|---|
| LCS | maximum length | diagonal +1 | max(above,left) | 0 |
| Global alignment | maximum total score | compare all 3 candidates | compare all 3 candidates | accumulated gap cost |

LCS skips symbols for free. Global alignment must place every symbol in a scored column. Even when symbols match, the alignment algorithm compares its three specified branches; the LCS if/else is not its recurrence.

Both use a prefix DAG, and both require enough predecessor information to reconstruct a witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Interval DP: the final decision

The last matrix multiplication chooses a split $k$; a BST root $r$ determines the left and right key intervals. Both preserve the original order.

A state $[i, j]$ represents a contiguous interval. Fill increasing lengths because every decision depends on shorter intervals. The value table answers “how good”; the decision table answers “how to build”.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Matrix chains: dimensions determine merge cost

Let $p = [5, 10, 3, 12, 5]$. Matrix $A_i$ has dimensions $p_{i-1} \times p_i$.

| Matrix | $A_1$ | $A_2$ | $A_3$ | $A_4$ |
|---|---|---|---|---|
| Dimensions | $5 \times 10$ | $10 \times 3$ | $3 \times 12$ | $12 \times 5$ |

Under standard multiplication, $(a \times b)(b \times c)$ costs $abc$ scalar multiplications and produces $a \times c$. Change parentheses, preserving order. Planning does not perform the products.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

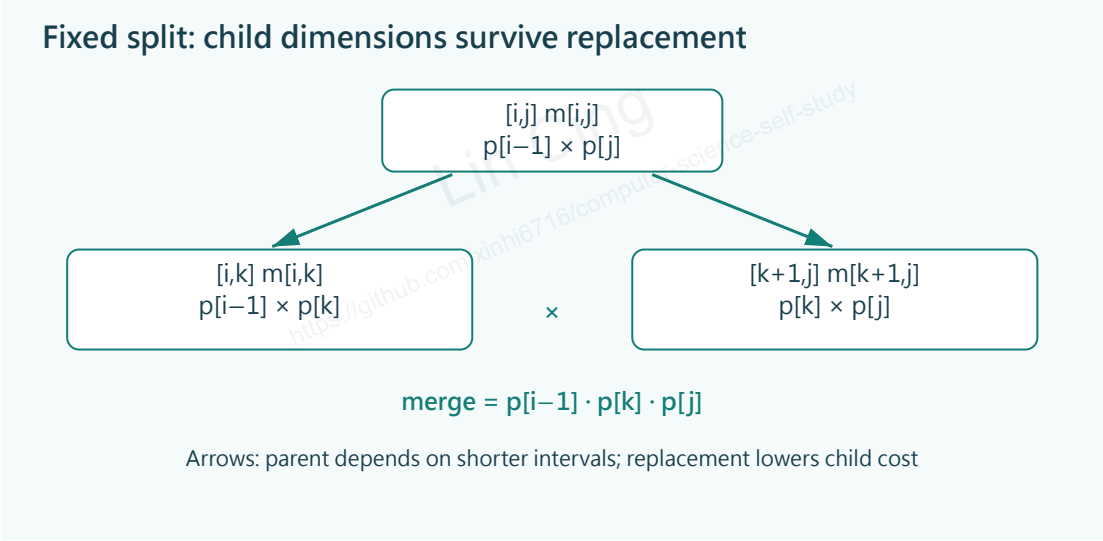


# Matrix-chain recurrence and sufficient state

Let $m[i, j]$ be the minimum multiplication count for $A_i \cdots A_j$, with $m[i, i] = 0$.

$$m[i, j] = \min_{i \le k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j\}.$$

Every parenthesization of this interval produces $p_{i-1} \times p_j$. The outside computation needs endpoints and minimum cost, without the internal parentheses. A fixed split fixes both child dimensions.



## Proof by splitting, replacing, and combining

Every full parenthesization has a final split $i \le k < j$. If its left child is suboptimal, replace it with a cheaper solution of the same output dimensions. The right cost and merge cost stay fixed, reducing the total: a contradiction. The right child follows identically.

Conversely, for every split, two feasible child products can be multiplied legally. Each candidate is a feasible complete solution. Candidates include an optimum and introduce no invalid solution; their minimum is optimal. Induction on interval length establishes every table entry.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fill by length: all short-interval candidates

Initialize every diagonal $m[i, i] = 0$. Each candidate is "left + right + merge".

| Length | Interval | Candidates and arithmetic | $m$; $s$ |
|---|---|---|---|
| 2 | $[1, 2]$ | $k = 1 : 0 + 0 + 5 \cdot 10 \cdot 3 = 150$ | 150; 1 |
| 2 | $[2, 3]$ | $k = 2 : 0 + 0 + 10 \cdot 3 \cdot 12 = 360$ | 360; 2 |
| 2 | $[3, 4]$ | $k = 3 : 0 + 0 + 3 \cdot 12 \cdot 5 = 180$ | 180; 3 |
| 3 | $[1, 3]$ | $k = 1 : 0 + 360 + 600 = 960$; $k = 2 : 150 + 0 + 180 = 330$ | 330; 2 |
| 3 | $[2, 4]$ | $k = 2 : 0 + 180 + 150 = 330$; $k = 3 : 360 + 0 + 600 = 960$ | 330; 2 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The full chain: compare all three splits

| Split for $[1, 4]$ | Left | Right | Final multiplication | Total |
|---|---|---|---|---|
| $k = 1$ | 0 | 330 | $5 \cdot 10 \cdot 5 = 250$ | 580 |
| $k = 2$ | 150 | 180 | $5 \cdot 3 \cdot 5 = 75$ | **405** |
| $k = 3$ | 330 | 0 | $5 \cdot 12 \cdot 5 = 300$ | 630 |

The merge cost alone is not a general greedy rule: both optimal child costs must be included. In this example, $k = 2$ happens to minimize both merge cost and total cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The full chain: compare all three splits (continued)

### Complete matrix-chain table: m[i,j] / s[i,j]

| | j=1 | j=2 | j=3 | j=4 |
|---|---|---|---|---|
| i=1 | 0 / – | 150 / 1 | 330 / 2 | 405 / 2 |
| i=2 | | 0 / – | 360 / 2 | 330 / 2 |
| i=3 | | | 0 / – | 180 / 3 |
| i=4 | | | | 0 / – |

Fill lengths L=1 → 2 → 3 → 4; read only shorter intervals

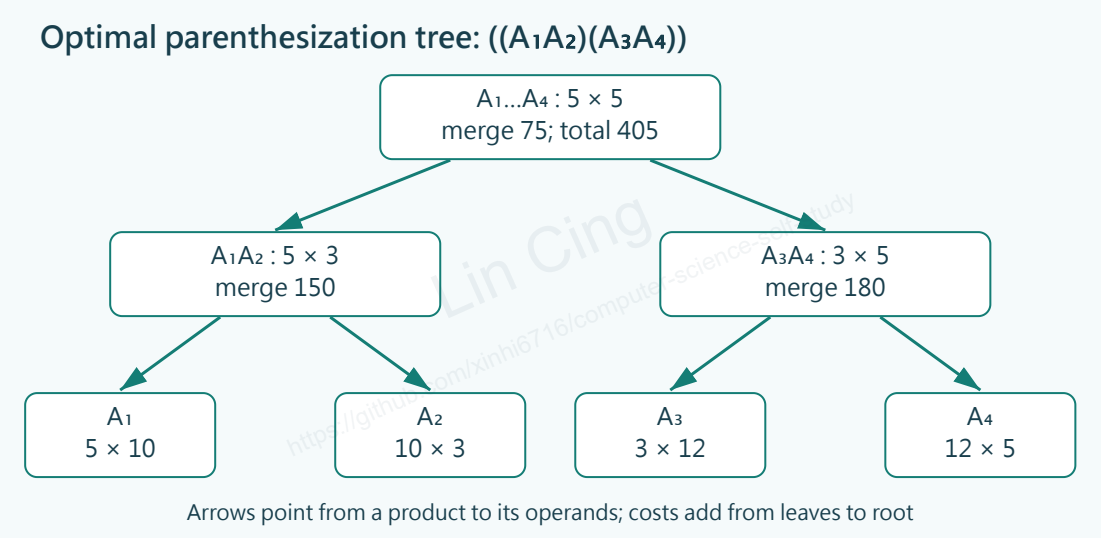
**L=4: 580, 405, 630 → min = 405**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Reconstruct the parenthesization tree

Use $s[1,4]=2$, then $s[1,2]=1$ and $s[3,4]=3$, yielding $((A_1A_2)(A_3A_4))$.

Compute $A_1A_2$ : $5\times 3$ at cost 150 and $A_3A_4$ : $3\times 5$ at cost 180. Their product is $5\times 5$ at cost 75; total 405.




Recursion: for $i=j$, output $A_i$. Otherwise output parentheses around `build(i,s[i,j])` and `build(s[i,j]+1,j)`. Each leaf and merge node is visited once: $\Theta(n)$ reconstruction.



## Matrix-chain algorithm and exact work

```
for i=1..n: m[i,i]=0
for L=2..n:
  for i=1..n-L+1:
    j=i+L-1; m[i,j]=infinity
    for k=i..j-1:
      t=m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
      if t<m[i,j]: m[i,j]=t; s[i,j]=k
```

Length $L$ has $n-L+1$ states and $L-1$ candidates per state:

$$\sum_{L=2}^{n}(n-L+1)(L-1)=\binom{n+1}{3}=\frac{n^3-n}{6}.$$

Time is $\Theta(n^3)$; value and split tables each occupy $\Theta(n^2)$ space. These bounds concern planning, not executing matrix products.

## OBST: model successful and unsuccessful searches

Sorted real keys are $k_1 < \cdots < k_n$, with successful-search weights $p_1, \ldots, p_n$. External leaves $d_0, \ldots, d_n$ represent unsuccessful-search gaps, with weights $q_0, \ldots, q_n$.

$d_0$ means below $k_1$; $d_i$ means $k_i < x < k_{i+1}$; $d_n$ means above $k_n$. An extended BST with $n$ real keys has $n + 1$ external leaves.

Probabilities satisfy $\sum p + \sum q = 1$. Nonnegative frequency weights also work directly; divide total cost by the global total weight to obtain expected cost per query.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Two cost conventions: a constant difference

The root has depth 0. $C(T)$ counts real-key comparisons; $E(T)$ also counts one terminal check at the external leaf.

$$C(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j \operatorname{depth}(d_j).$$

$$E(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j(\operatorname{depth}(d_j) + 1).$$

Thus $E(T) - C(T) = \sum_j q_j$, independent of shape, so both have the same optimal trees. An empty interval contains only $d_{i-1}$: $C[i, i-1] = 0$ and $E[i, i-1] = q_{i-1}$. Initialization and reported costs must use the same convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## OBST intervals retain global probability mass

Interval $[i, j]$ contains keys $k_i, \ldots, k_j$ and external leaves $d_{i-1}, \ldots, d_j$.

$$w[i, j] = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t.$$

Subproblems retain the original global weights, without dividing by interval mass. Lowering a subtree by one level adds one check for every query in it, increasing cost by global mass $w$.

If conditional expectations are stored instead, each must be multiplied by its subtree mass when combining. Conditional expectations with different denominators cannot simply be added. Empty intervals still have $w[i, i-1] = q_{i-1}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## OBST recurrence and replacement proof

$$E[i, j] = w[i, j] + \min_{i \le r \le j} \{E[i, r-1] + E[r+1, j]\}.$$

$C$ has the same recurrence shape, with zero empty-interval costs. Inorder order uniquely determines both child key ranges; merging adds $w[i, j]$.

Every optimum has some root. Replace any suboptimal child with a cheaper tree over the same keys and gaps. BST order and subtree mass stay unchanged, as does the cost of placing it at the same depth. The total decreases, a contradiction. Conversely, every candidate root and two feasible children form a feasible BST. Minimizing over all roots is therefore optimal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three-key example B: initialization and singletons

Fix $p = (.20, .30, .10)$ and $q = (.10, .05, .15, .10)$, total mass 1.

Empty intervals: $E[1, 0] = .10$, $E[2, 1] = .05$, $E[3, 2] = .15$, $E[4, 3] = .10$.

| Interval | $w$ | Only candidate $E_L + E_R + w$ | $E$ | $C$ |
|---|---|---|---|---|
| $[1, 1]$ | .35 | $.10 + .05 + .35$ | .50 | .35 |
| $[2, 2]$ | .50 | $.05 + .15 + .50$ | .70 | .50 |
| $[3, 3]$ | .35 | $.15 + .10 + .35$ | .60 | .35 |

A singleton's $C$ includes unsuccessful comparisons for both adjacent gaps, so it is not just $p_i$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Example B: every root for pairs and the full interval

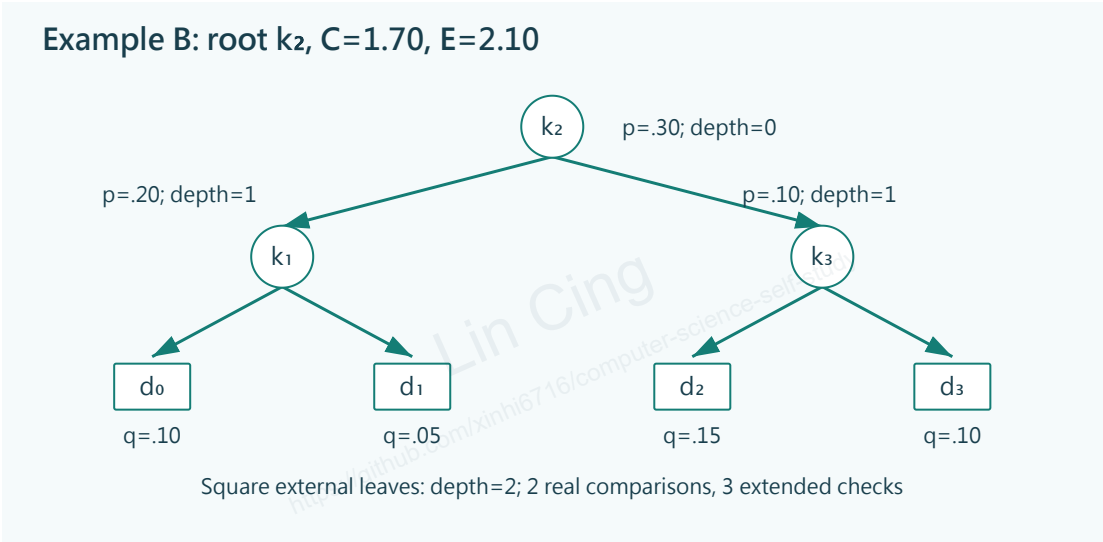
| Interval; mass | Candidate roots and complete arithmetic | Minimum $E$; $C$; root |
|---|---|---|
| $[1, 2]$; .80 | $r = 1 : .10 + .70 + .80 = 1.60$; $r = 2 : .50 + .15 + .80 = 1.45$ | 1.45; 1.15; 2 |
| $[2, 3]$; .70 | $r = 2 : .05 + .60 + .70 = 1.35$; $r = 3 : .70 + .10 + .70 = 1.50$ | 1.35; 1.05; 2 |
| $[1, 3]$; 1.00 | $r = 1 : .10 + 1.35 + 1 = 2.45$; $r = 2 : .50 + .60 + 1 = 2.10$; $r = 3 : 1.45 + .10 + 1 = 2.55$ | **2.10; 1.70; 2** |

Each interval has $C = E - \sum_{t=i-1}^{j} q_t$: subtract that interval's gap mass. The full table evaluates 10 root candidates, not just the final three.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Example B: reconstruct and audit the expectation

The root is $k_2$, with left child $k_1$ and right child $k_3$. Real-key depths are 1, 0, 1; all four external leaves have depth 2.






## Example B: account for each query outcome

The real-key comparison counts are 2, 1, and 2. Successful queries contribute $.20 \cdot 2 + .30 \cdot 1 + .10 \cdot 2 = .90$

The four unsuccessful gaps have total probability $.10 + .05 + .15 + .10 = .40$. Each unsuccessful query compares two real keys. Counting the external-leaf termination check raises its extended cost to three.

$C = .90 + (.10 + .05 + .15 + .10) \cdot 2 = 1.70$; $E = .90 + .40 \cdot 3 = 2.10$. The difference is .40. These are expectations per global query, not expectations conditional on success.

## OBST tabulation, reconstruction, and exact work

```
for i=1..n+1: E[i,i-1]=w[i,i-1]=q[i-1]
for L=1..n:
  for i=1..n-L+1:
    j=i+L-1; w[i,j]=w[i,j-1]+p[j]+q[j]
    E[i,j]=infinity
    for r=i..j:
      t=E[i,r-1]+E[r+1,j]+w[i,j]
      if t<E[i,j]: E[i,j]=t; root[i,j]=r
```

The candidate count is $\sum_{L=1}^{n}(n-L+1)L=\binom{n+2}{3}=n(n+1)(n+2)/6$. The basic algorithm takes $\Theta(n^3)$ time and $\Theta(n^2)$ space. `build(i,j)` returns $d_{i-1}$ when empty; otherwise attach both children to `root[i,j]`. Output contains $2n+1$ nodes and takes $\Theta(n)$ time.

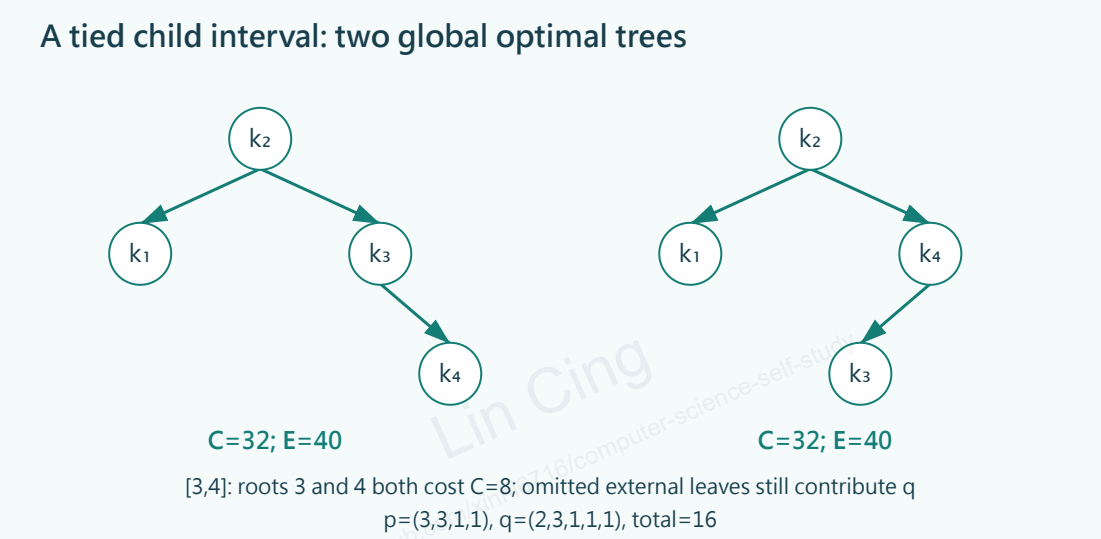


## Integer example C: two trees can share the optimum

Use frequency weights $p=(3,3,1,1)$, $q=(2,3,1,1,1)$, global total 16.

Interval $[3,4]$ has $w=5$. Its $C$ candidates are $0+3+5=8$ for root 3 and $3+0+5=8$ for root 4: a tie. The full root-2 candidate is $C[1,1]+C[3,4]+16=8+8+16=32$; roots 1, 3, 4 cost 35, 38, 41.

## Integer example C: two trees can share the optimum (continued)




Global $C = 32$, $E = 40$; dividing by 16 gives expectations 2 and 2.5. Storing the first minimizing root reconstructs one optimum. All optimal trees require every tied root, and output size may be exponential.



## Connecting successful-only and full models

| Model | Weights and boundaries | Objective |
|---|---|---|
| Data structures ch.10 successful-only version | Only $p$; empty cost 0; $W = \sum p$ | Weighted real-key comparisons for successful queries |
| Full model here | $p, q$; empty $C = 0$, empty $E = q$; $w = \sum p + \sum q$ | Global weighted cost of successful and unsuccessful queries |

The current data structures chapter also presents the full $p/q$ model and $E - C$ difference; successful-only is not its sole convention. The original data structures notes use real-key cost $C$; CLRS § 15.5 uses extended cost $E$.

Setting every $q$ to zero recovers successful-only. Deleting nonzero $q$ changes the problem.

## Checks that matter for interval DP

Matrix chains: endpoints fix output dimensions; use $i \leq k < j$; minimize the whole candidate cost; reconstruct recorded splits.

OBST: preserve key order and gap membership; align empty costs with the $C/E$ convention; retain global weights; compare every root; tie policy chooses among optimal trees.

Sharing an interval-DP pattern does not make merge costs or boundaries interchangeable. The analysis here concerns the basic cubic algorithms. Faster OBST optimization requires a separate root-monotonicity argument and does not automatically apply to arbitrary interval DP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 15, Dynamic Programming. MIT 6.046J, Fall 2004, Lecture 12, Dynamic Programming. Lecturer Charles E. Leiserson. MIT 6.046J, Fall 2005, Lecture 15, Dynamic Programming. Lecturer Charles E. Leiserson; slides credited to Erik D. Demaine and Charles E. Leiserson. Original scanned algorithms notes (演算法_wjungle), dynamic programming pages; original scanned data structures notes (資料結構), physical pages 166–170, for comparison of optimal search tree models. These notes cover algorithms and data structures and are not the full text of a single textbook.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

# Greedy methods

Can choosing the best immediate candidate guarantee a globally optimal solution? The key is proving that the choice can still extend to an optimum.

Study **activity selection → Huffman coding → matroids and job scheduling**, moving from concrete exchanges to reusable structural conditions.

Prerequisites are sorting, priority queues, trees, sets, and induction. Distinguish the local rule, feasibility, and optimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Turn a choice rule into a proof

| Design question | Required evidence |
| --- | --- |
| What is a legal solution? | Fix compatibility, prefix-code, or independence constraints. |
| Which candidate comes next? | Define the candidates, ordering, and tie handling. |
| Why can an optimum survive? | Give an exchange, contraction, or matroid argument. |
| Does a changed objective preserve the rule? | Test weighted variants and changed constraints with counterexamples. |

Activity selection, Huffman coding, and matroids require their own proofs. A plausible local rule alone proves nothing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Finish early to preserve later choices

One room serves activities with fixed times and equal value. Select as many mutually compatible activities as possible.

At each step, choose the compatible activity with the earliest finish time. An exchange argument proves this choice safe. Success on one example is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Touching endpoints are compatible

Activity i occupies the half-open interval $[s_i,f_i)$, with $s_i < f_i$.

Two activities are compatible exactly when $f_i \leq s_j$ or $f_j \leq s_i$. If the selected prefix ends at L, accept a candidate when $s_i \geq L$.

[0,3) and [3,5) are compatible. [0,3) and [2,5) overlap. A required cleanup time δ changes the test to $s_i \geq L+\delta$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Sort by finish time, then scan once

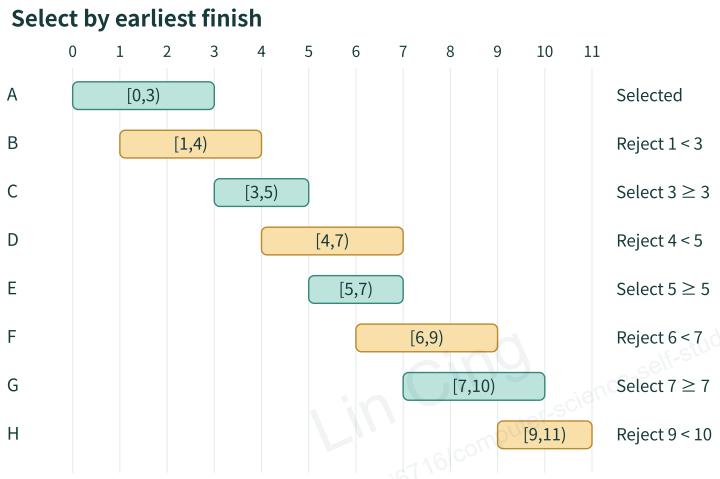
```
selected = []
last_finish = −∞
for (id, start, finish) in sort_by_finish(activities):
    if start >= last_finish:
        append id to selected
        last_finish = finish
return selected
```

Break finish-time ties in any fixed order. This example uses ID order. Empty input returns an empty set. Rejection leaves last_finish unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A timeline of eight activities




Time (hours). Teal selected; amber rejected for overlap.

Times are in hours. Input A[0,3), B[1,4), C[3,5), D[4,7), E[5,7), F[6,9), G[7,10), H[9,11). Teal marks selected A, C, E, G. Amber marks a conflict with the most recently selected activity.

## Compare each start against the last finish

| Candidate | L before | Test | Action | L after |
|---|---|---|---|---|
| A [0,3) | $-\infty$ | $0 \geq -\infty$ | Select A | 3 |
| B [1,4) | 3 | $1 < 3$ | Reject B | 3 |
| C [3,5) | 3 | $3 \geq 3$ | Select C | 5 |
| D [4,7) | 5 | $4 < 5$ | Reject D | 5 |
| E [5,7) | 5 | $5 \geq 5$ | Select E | 7 |
| F [6,9) | 7 | $6 < 7$ | Reject F | 7 |
| G [7,10) | 7 | $7 \geq 7$ | Select G | 10 |
| H [9,11) | 10 | $9 < 10$ | Reject H | 10 |

The output has four activities. The optimum need not be unique. A, C, E, H is another set of size four.



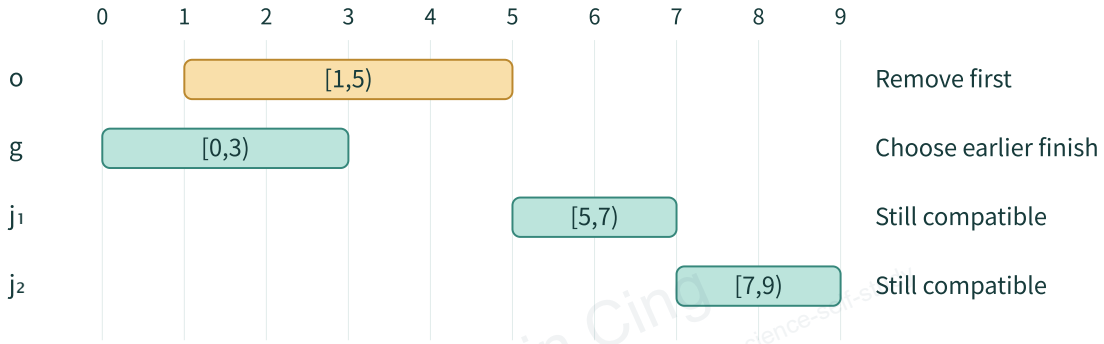
## Earliest start and shortest duration can fail

| Rule | Counterexample intervals | Rule obtains | Better solution |
|---|---|---|---|
| Earliest start | X[0,8), A[1,3), B[3,5), C[5,7) | X, size 1 | A, B, C, size 3 |
| Shortest duration | A[0,4), X[3,5), B[4,9) | X, size 1 | A, B, size 2 |

The shortest-duration rule repeatedly selects a shortest activity compatible with all previous selections. X has duration 2, strictly less than A or B, but crosses their shared endpoint. A failed earliest-start rule does not refute earliest finish.

## Exchange the first activity without losing count

### Replace o by g and keep the entire suffix




$f_g = 3 \leq f_o = 5 \leq s_{j_1} = 5$

Consider any nonempty residual problem. Every candidate starts at or after the current boundary L. Let g be a candidate with minimum finish time. Let O be an optimal solution and o its first activity. Then $f_g \leq f_o$.



## The exchanged solution stays feasible and optimal

If g=o, O already makes the greedy choice. Otherwise define $O' = (O - \{o\}) \cup \{g\}$.

1. Activity g starts at or after L, so it respects the fixed prefix.
2. Every remaining activity j in O satisfies $s_j \geq f_o \geq f_g$, so replacing o by g preserves compatibility.
3. Activity g cannot already occur later in O. Positive duration and compatibility would imply $f_g > f_o$, contradicting $f_g \leq f_o$.
4. Therefore $|O'| = |O|$. The solution $O'$ is optimal and starts with g.

The conclusion asserts that some optimal solution contains g. It does not assert that every optimal solution contains g.

## Optimal suffixes and induction prove the whole algorithm

After selecting g, retain R={i | $s_i \geq f_g$}. No candidate can precede g, because every candidate finishes at or after $f_g > s_g$.

If an optimal solution containing g had a nonoptimal suffix for R, replacing that suffix with a larger one would improve the original solution. Hence OPT(S)=1+OPT(R).

The empty set is the base case. The exchange argument makes the first choice safe. Induction on the smaller residual set proves the complete greedy solution optimal. Optimal substructure alone does not establish a safe greedy choice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Cost and assumptions of the theorem

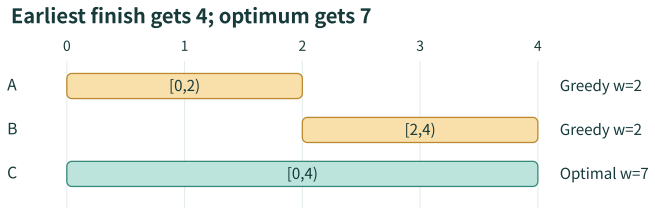
Sorting costs O(n log n). Scanning costs Θ(n). The total is O(n log n), or Θ(n) when input is already sorted by finish time.

The scan uses O(1) auxiliary space beyond sorting and output. The output contains at most n IDs.

The proof assumes one resource, fixed positive-duration intervals, and a cardinality objective. Scanning backward and selecting the latest compatible start is also correct by a symmetric exchange proof. This differs from scanning forward by earliest start.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Weights break the value-preserving exchange




A[0,2) has weight 2, B[2,4) has weight 2, and C[0,4) has weight 7. Earliest finish selects A and B for value 4. The optimum selects C for value 7.

An exchange preserves cardinality but may reduce total weight. This disproves the original rule for the weighted version. It does not prove that every conceivable greedy rule fails.



## Weighted intervals require a take-or-skip recurrence

Sort by finish time. Let p(j) be the largest i<j with $f_i \leq s_j$, or 0 if none exists. Let M(j) be the maximum weight among the first j activities.

M(0)=0, and M(j)=max(M(j−1), $w_j$+M(p(j))).

Skipping j leaves the first j−1 activities. Taking j leaves a compatible prefix among the first p(j). These cases cover every optimal solution.

For order A, B, C, p=(0,1,0) and M=(0,2,4,7). Traceback returns {C}. Binary search computes predecessors. Total time is O(n log n), with O(n) DP and traceback time and O(n) space.

## Turn symbol frequencies into a minimum bit count

Huffman optimizes binary, symbol-by-symbol, prefix-free codes over a finite alphabet. The input is a nonnegative weight $w_c$ per symbol. The output assigns a codeword to every symbol. The main proof starts with $n \geq 2$ and positive weights; boundary cases follow.

| Symbol | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| Frequency, thousands | 45 | 13 | 12 | 16 | 9 | 5 |

A fixed-length code uses 3 bits per occurrence. Huffman uses 2.24 bits on average, or 224 thousand bits, excluding the codebook and framing.

The original scan, page 20, identifies Huffman as greedy. CLRS supplies the construction and proof. The two MIT editions supply the general greedy-choice and optimal-substructure framework.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Prefix-free codes support immediate decoding

Prefix-free means that no codeword is a prefix of another codeword. In a binary tree, symbols occupy leaves only. A left edge is 0; a right edge is 1.

For example, $\{0, 10, 110, 111\}$ is prefix-free; $\{0, 01\}$ is not. On reaching a leaf, output its symbol and restart at the root. No separator is needed.

Distinct codewords may still be ambiguous. With $\{0, 01, 10\}$, the string 010 can mean 0|10 or 01|0. Prefix-free is sufficient for unique decoding, but is not necessary.

The optimization domain here is symbol-by-symbol prefix-free coding. It does not compare against block compressors that exploit dependencies between symbols.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Kraft constrains codeword lengths

For a finite binary prefix-free code with nonnegative integer lengths $\ell_i$,

$$\sum_i 2^{-\ell_i} \le 1.$$

A node at depth $\ell$ corresponds to a binary interval of length $2^{-\ell}$ in $[0, 1)$. Prefix-free words have disjoint intervals, so their total length is at most 1.

The converse also holds. Sort the lengths in increasing order and assign the leftmost unused dyadic interval at each length. Previously assigned shorter intervals occupy whole depth-$\ell$ cells. The total-length bound guarantees a free cell, giving a prefix-free code.

Lengths 1,2,3,3 sum to 1; lengths 1,1,2 sum to 1.25 and are impossible. For $n \ge 2$, lengths must be at least 1. Length 0 allows only one symbol. This criterion cannot be applied to arbitrary codes whose only requirement is distinct codewords.



## Weighted external path length is the objective

For a code tree $T$, leaf depth $d_T(c)$ equals codeword length. Minimize

$$B(T) = \sum_c w_c d_T(c).$$

With occurrence counts, $B$ is measured in bits. With probabilities, it is bits per symbol. With positive weights, deleting a unary internal node strictly lowers the cost of its descendant leaves. Thus an optimal tree must be full.

Each internal node weighs the sum of its descendant leaves. A leaf contributes once for each ancestor, hence

$$B(T) = \sum_{v \text{ internal}} w(v) = \text{sum of all merge weights}.$$

These are two ways to check the same cost. The root weight is only $\sum_c w_c$, not the total coding cost.

## Extract the two lightest trees at every round

A queue item is a whole subtree, keyed by its total weight. A merge replaces two items by one, giving exactly $n - 1$ rounds.

```
Q ← min-priority queue of symbol leaves
while |Q| > 1:
    x ← extract-min(Q)
    y ← extract-min(Q)
    z ← Node(weight=x.weight+y.weight, left=x, right=y)
    insert(Q,z)
return extract-min(Q)
```

The queue subtrees partition the original alphabet. Their weights always sum to 100. Reinsert each new subtree; scanning only the original frequencies does not implement Huffman.

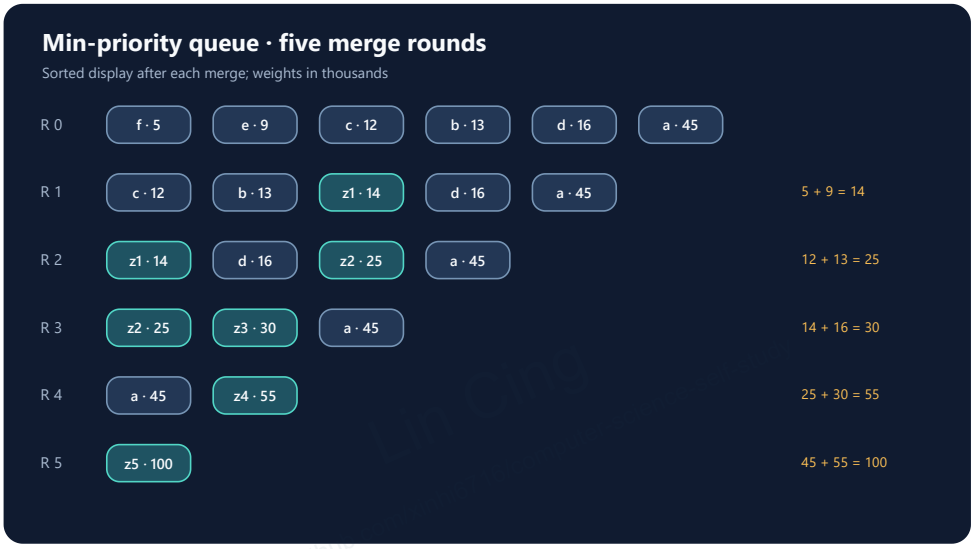


## Audit all five queue rounds

The displayed queue is sorted for readability. A heap itself need not be fully sorted. Node $z_i$ is created in round $i$.

| Round | Extract | Insert | Queue after merge, sorted by weight |
|---|---|---|---|
| 0 | | | f 5,e 9,c 12,b 13,d 16,a 45 |
| 1 | f 5,e 9 | $z_1$ 14 | c 12,b 13,$z_1$ 14,d 16,a 45 |
| 2 | c 12,b 13 | $z_2$ 25 | $z_1$ 14,d 16,$z_2$ 25,a 45 |
| 3 | $z_1$ 14,d 16 | $z_3$ 30 | $z_2$ 25,$z_3$ 30,a 45 |
| 4 | $z_2$ 25,$z_3$ 30 | $z_4$ 55 | a 45,$z_4$ 55 |
| 5 | a 45,$z_4$ 55 | $z_5$ 100 | $z_5$ 100 |

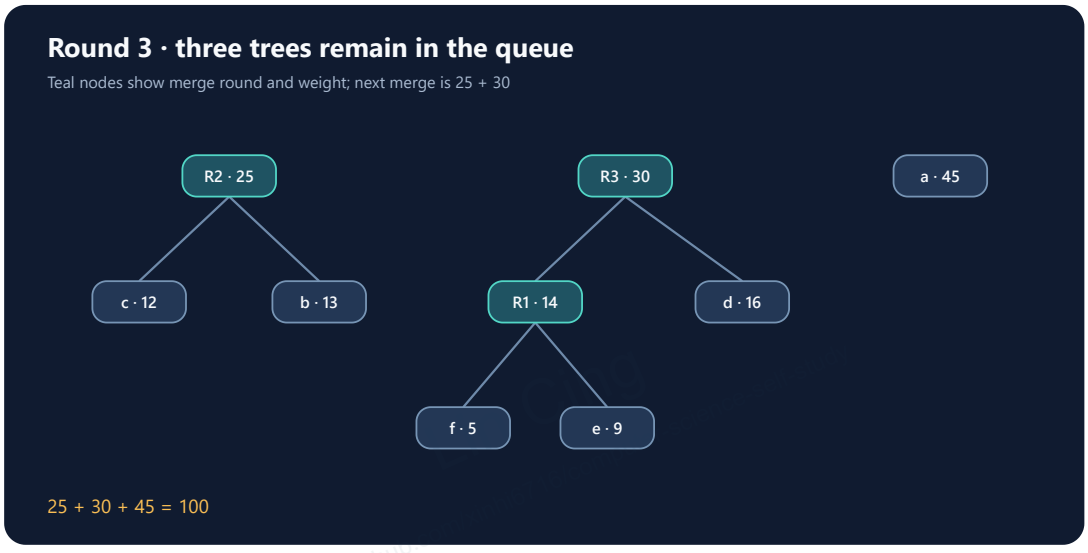
# Audit all five queue rounds (continued)



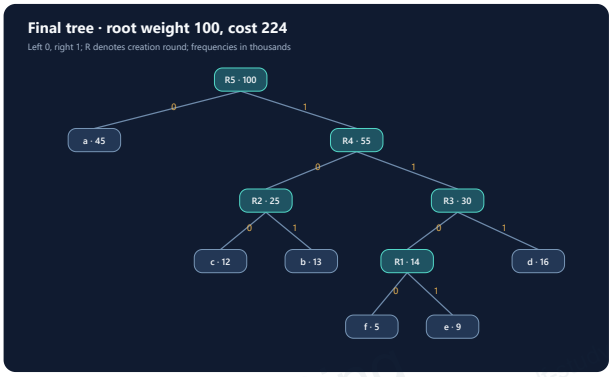

After round 3, the queue contains three trees of weights 25,30,45. The next round merges 25 and 30; it does not attach the new node 30 directly to 45.



# The intermediate forest is algorithm state



## Read codewords from the root and check both costs




| Symbol | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| Codeword | 0 | 101 | 100 | 111 | 1101 | 1100 |
| Depth | 1 | 3 | 3 | 3 | 4 | 4 |

Merge weights 14,25,30 already contribute to the final cost. Moving a subtree up by one level adds one bit to every descendant codeword, costing exactly that subtree weight.

$B = 45 + 39 + 36 + 48 + 36 + 20 = 224$ thousand bits.

The merge sum is $14 + 25 + 30 + 55 + 100 = 224$. The root weighs 100 thousand occurrences. The Kraft sum is $1/2 + 3/8 + 2/16 = 1$. The message abc encodes as 0101100.



## Exchange establishes an optimal tree compatible with the choice

Let $x, y$ be the two minimum-weight symbols. In an optimal full binary tree, choose deepest sibling leaf positions $a, b$ at depth $D$. A deepest leaf has a leaf sibling; an internal sibling would have a still deeper descendant.

First exchange $x$ with $a$. If $x$ occupies $b$, swap the two sibling positions first at no cost. The exchange changes cost by

$$B(T_{new}) - B(T) = (w_a - w_x)(d_T(x) - D) \leq 0.$$

Now exchange $y$ with the leaf in position $b$. Symbol $x$ is fixed at $a$; $y$ has minimum weight among all remaining leaves, so the same inequality applies. Skip any exchange whose symbol is already in position.

Neither exchange increases cost. Since the original tree is optimal, the new tree is optimal too. Thus some optimal tree has $x, y$ as deepest siblings. We do not claim that every optimal tree has this shape.

## Contraction and induction prove global optimality

Contract sibling leaves $x, y$ into a leaf $z$ of weight $w_z = w_x + w_y$. Other depths stay unchanged; both leaves lose one edge.

$$B(T) = B(T') + w_x + w_y.$$

If a contracted optimal tree $T'$ were suboptimal, replace it by a cheaper reduced tree and expand $z$. Expansion adds the same fixed $w_x + w_y$, contradicting optimality of the original tree.

Induct from one leaf, whose tree cost is 0. Exchange guarantees an optimal solution containing this round's minimum siblings. The induction hypothesis gives an optimal Huffman solution for the reduced $n - 1$ symbols. Expansion restores a fixed cost and therefore gives an optimal solution for the original alphabet.

A cheapest immediate merge alone is not a proof. The proof needs both the cost-preserving exchange and the optimal reduced problem.



## Ties, one symbol, and zero weights

Break equal-weight ties arbitrarily. Swapping left and right does not change lengths or cost. A fixed creation sequence can be a secondary key for reproducible codebooks.

For one symbol, the tree model assigns the empty word $\epsilon$, costing 0. A format requiring a nonempty code may instead assign 0, costing $w$. That adds a different length constraint. Decoding an empty word requires an external symbol count. Return an empty result for an empty alphabet.

Zero-weight symbols still need codewords if they belong to the alphabet. Huffman and the non-increasing exchange proof remain valid. Removing a unary node may now leave the cost unchanged, so not every optimal tree must be full. For a of weight 1 and b of weight 0, codes 00,01 cost 2 while 0,1 cost 1. If all weights are zero, every valid code costs 0.

Weights must be finite and nonnegative. Do not normalize by a zero total weight.

## Construction complexity and reproducible checks

Build a binary min-heap in $O(n)$. Each of the $n-1$ rounds uses two extract-min operations and one insertion, each $O(\log n)$. Tree construction therefore takes $O(n \log n)$ time and $O(n)$ node space, with $2n-1$ nodes.

If the weights are already sorted, keep a leaf queue and a merge-node queue and choose from their two fronts. Tree construction then takes $O(n)$; sorting is a separate cost.

These bounds cover tree construction. Materializing every codeword string also costs $\Theta(\sum_c \ell_c)$ time and output space, which can be $\Theta(n^2)$. Encoding a message additionally processes the emitted bits.

Check that no listed codeword prefixes another. Compute both costs: 45+39+36+48+36+20=224 and 14+25+30+55+100=224. The exchange and contraction proof establishes optimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## When does taking the heaviest element work?

Activity selection has its own earliest-finish rule. A matroid instead guarantees sorted greedy optimization for every nonnegative weight assignment.

We express feasibility as independent sets, prove the exchange step, and turn missed-deadline penalties into saved weight.

The main source is CLRS § § 16.4–16.5. Undirected forests illustrate set operations. MST algorithms and the cut property belong to Chapter 8.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A finite ground set and two axioms

Let M=(E,$\mathcal{I}$), where E is a finite ground set, $\mathcal{I}\subseteq 2^E$, and $\varnothing\in\mathcal{I}$.

- Heredity. If $A\in\mathcal{I}$ and $B\subseteq A$, then $B\in\mathcal{I}$.
- Exchange. If $A,B\in\mathcal{I}$ and $|A|<|B|$, some $x\in B\setminus A$ satisfies $A\cup\{x\}\in\mathcal{I}$.

These conditions define a matroid. Heredity alone defines an independence system. Exchange must hold for every eligible pair, not merely one successful example.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Maximal and maximum have different meanings

A maximal independent set admits no further element. A maximum independent set has the largest cardinality.

In a matroid, if a maximal set A were smaller than an independent set B, exchange would supply an element to add to A. This contradicts maximality. All maximal independent sets therefore have the same size. They are bases, and their size is the rank r.

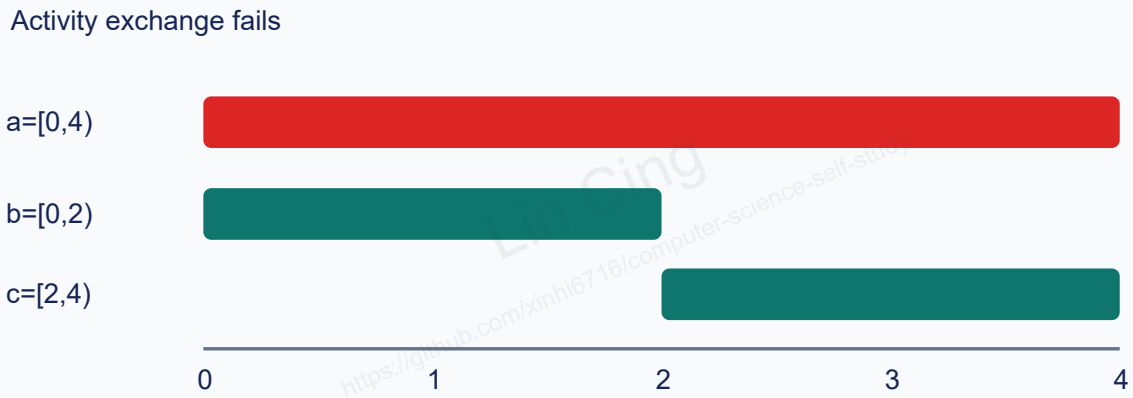
The uniform matroid $U_{2,3}$ has E={a,b,c} and $\mathcal{I}=\{X\subseteq E:|X|\leq 2\}$. Every two-element set is a basis. No singleton is maximal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Activity selection does not form a matroid

Use half-open intervals a=[0,4), b=[0,2), and c=[2,4). Independent sets contain pairwise compatible activities.

A={a} and B={b,c} are independent, with |A|=1<2=|B|. Neither A∪{b} nor A∪{c} is independent. Exchange fails.




Heredity still holds. The earliest-finish proof for activity selection does not make this independence system a matroid.



## Weighted greedy and negative weights

Maximize $\Sigma_{x \in A} w(x)$ subject to $A \in \mathcal{I}$. Sort elements by decreasing weight. Start with $A=\varnothing$ and add x exactly when $A \cup \{x\} \in \mathcal{I}$.

For $w \geq 0$, this returns a maximum-weight independent set. Zero-weight elements may be added, and the result is a basis. Sorting costs O(n log n). With n independence queries of cost T_ind, total time is O(n log n+nT_ind).

For arbitrary weights and unrestricted independent sets, skip every negative element. In $U_{2,2}$ with weights 5 and −7, filling a basis yields −2 while selecting only the first yields 5. If the objective explicitly requires a basis, negative elements cannot be skipped. The original rule still finds a maximum-weight basis.

## Exchange proof 1 | prepare an optimal basis

First assume nonnegative weights. Extend an optimal independent set to a basis without decreasing its weight. Thus an optimal basis $O$ exists.

Maintain the invariant that some optimal basis $O$ contains the current greedy set $G$. Initially $G=\varnothing$. Let $g$ be the next accepted element. If $g\in O$, retain $O$.

Otherwise start with $I_0=G\cup\{g\}$ and set $H=I_0$. While $|H|<|O|$, exchange adds an element of $O\setminus H$ to $H$. Continue until $|H|=|O|=r$. Every step preserves independence and retains $G\cup\{g\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exchange proof 2 | replace one element without loss

Now $H\subseteq O\cup\{g\}$, $|H|=|O|$, and $G\cup\{g\}\subseteq H$. Therefore $H=(O\{y\})\cup\{g\}$ for some $y\in O\setminus G$.

Because $G\cup\{y\}\subseteq O$, $y$ is feasible at the current $G$. It cannot have been rejected earlier. An earlier rejection would contradict heredity, since that earlier prefix is a subset of $G$. Thus decreasing weight order gives $w(g)\geq w(y)$.

$w(H)=w(O)-w(y)+w(g)\geq w(O)$. Optimality of $O$ makes $H$ optimal too, and $H$ contains the new $G$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Exchange proof 2 | replace one element without loss (continued)

One replacement preserves the greedy prefix

$G \subseteq O$ → $I_0 = G \cup \{g\}$ → $H = O - \{y\} + \{g\}$

$I_0 \subseteq H \subseteq O \cup \{g\}$, $|H| = |O| = r$

Extend by elements of O, preserving independence

$y \notin G$, $w(g) \geq w(y)$, $w(H) \geq w(O)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

At termination G is a basis, so an optimal basis containing G equals G. For unrestricted sets with negative weights, apply this proof to the restriction to nonnegative elements.

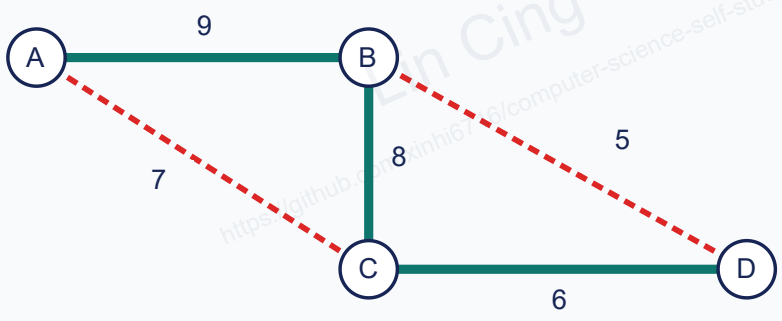


## Graphic matroid | feasible edge sets are forests

Take an undirected graph with V={A,B,C,D} and E={AB,BC,AC,CD,BD}. Let $\mathcal{I}$ contain its acyclic edge sets.

Deleting forest edges preserves acyclicity. If forests F,H satisfy |F|<|H|, some edge of H crosses two components of F. Otherwise H has at most $|V_i|-1$ edges inside each F component, totaling at most |F|. Adding a crossing edge to F creates no cycle. This proves exchange.

Accepted and rejected edges in a maximum-weight forest




AB, BC, CD

9 + 8 + 6 = 23

AC, BD rejected

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

This example maximizes forest weight. It does not develop a minimum spanning tree algorithm.

## Graphic matroid | trace every decision

Use AB=9, BC=8, AC=7, CD=6, and BD=5.

| Edge | Decision | G after scan | Weight |
|---|---|---|---|
| AB | Accept, joins A and B | {AB} | 9 |
| BC | Accept, joins C | {AB,BC} | 17 |
| AC | Reject, cycle A–B–C–A | {AB,BC} | 17 |
| CD | Accept, joins D | {AB,BC,CD} | 23 |
| BD | Reject, cycle B–C–D–B | {AB,BC,CD} | 23 |

The three accepted edges form a basis of weight 23. The independence query asks whether adding an edge preserves a forest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Unit jobs | minimize penalties by maximizing saved weight

Every job is available at time 0. One processor runs each job for exactly one time unit without preemption. Each integer deadline d_i lies in [1,n]. Completion C_i≤d_i incurs no penalty. Otherwise the nonnegative penalty p_i is paid. Every job must eventually run.

An independent set A is a job subset that can all finish on time. Define N_t(A)=|{i∈A:d_i≤t}|.

min Σ_late p_i = Σ_all p_i − max_(A∈$\mathcal{I}$) Σ_i∈A p_i.

| Job | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | $a_7$ |
|---|---|---|---|---|---|---|---|
| d_i | 4 | 2 | 4 | 3 | 1 | 4 | 6 |
| p_i | 70 | 60 | 50 | 40 | 30 | 20 | 10 |

Data follow CLRS Figure 16.7. Total penalty is 280.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Feasibility and the schedule construction

$A \in \mathcal{I}$ iff $N_t(A) \leq t$ for every integer $t \in [0,n]$, iff increasing-deadline order completes every job on time.

Necessity. If $N_t > t$, more than t unit jobs must finish in the first t slots. This is impossible.

Sufficiency. Sort deadlines as $d_{(1)} \leq \ldots \leq d_{(k)}$. If job j is late, then $d_{(j)} < j$ and $N_{d_{(j)}} \geq j > d_{(j)}$, a contradiction. Hence job j finishes at time $j \leq d_{(j)}$.

Heredity follows from $N_t(B) \leq N_t(A)$ for $B \subseteq A$. Penalties determine the greedy scan order, not feasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Job exchange | check every time prefix

Let A,B be feasible with $|A| < |B|$. Set $k = \max\{t : N_t(B) \leq N_t(A)\}$. This exists since both counts are zero at t=0. Since $N_n(B) > N_n(A)$, $k < n$.

The count difference is nonpositive at k and positive at k+1. Thus B has more jobs with deadline exactly k+1 than A. Choose $x \in B \setminus A$ with $d_x = k+1$.

- For $t \leq k$, $N_t(A \cup \{x\}) = N_t(A) \leq t$.
- For $t \geq k+1$, the integer count gap gives $N_t(A)+1 \leq N_t(B) \leq t$.

Therefore $A \cup \{x\}$ is feasible. Together with heredity and feasibility of $\varnothing$, this proves the deadline matroid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Full selection trace | show the violated prefix

Scan by decreasing penalty. Each N vector lists t=1,…,7. Update A only after acceptance.

| Candidate | Candidate $N_1,\ldots,N_7$ | Decision | Saved penalty |
|---|---|---|---|
| $a_1$ | (0,0,0,1,1,1,1) | Accept | 70 |
| $a_2$ | (0,1,1,2,2,2,2) | Accept | 130 |
| $a_3$ | (0,1,1,3,3,3,3) | Accept | 180 |
| $a_4$ | (0,1,2,4,4,4,4) | Accept | 220 |
| $a_5$ | (1,2,3,5,5,5,5) | Reject, $N_4=5>4$ | 220 |
| $a_6$ | (0,1,2,5,5,5,5) | Reject, $N_4=5>4$ | 220 |
| $a_7$ | (0,1,2,4,4,5,5) | Accept | 230 |

The on-time set is $\{a_1,a_2,a_3,a_4,a_7\}$. Minimum total penalty is $280-230=50$.



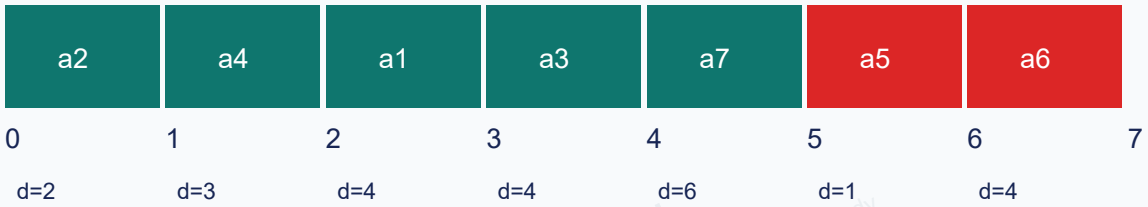
## Full schedule | selection order is not execution order

Sort the accepted jobs by increasing deadline, then append rejected jobs. Break equal-deadline ties arbitrarily.

| Slot | Job | Completion C_i | d_i | Status | Penalty |
|---|---|---|---|---|---|
| [0,1) | $a_2$ | 1 | 2 | On time | 0 |
| [1,2) | $a_4$ | 2 | 3 | On time | 0 |
| [2,3) | $a_1$ | 3 | 4 | On time | 0 |
| [3,4) | $a_3$ | 4 | 4 | On time | 0 |
| [4,5) | $a_7$ | 5 | 6 | On time | 0 |
| [5,6) | $a_5$ | 6 | 1 | Late | 30 |
| [6,7) | $a_6$ | 7 | 4 | Late | 20 |

## Full schedule | selection order is not execution order (continued)

Schedule the on-time set before the late jobs




Σ saved = 230; Σ late penalty = 30 + 20 = 50

Different processing times, release times, or multiple processors require a new check of the feasibility axioms. This proof covers only the stated unit-job model.



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 16, Greedy Algorithms. Original handwritten scanned algorithms notes (演算法_wjungle), greedy-method pages covering activity selection, Huffman coding, matroids, and scheduling. MIT 6.046J, Fall 2004, Lecture 13, Greedy Algorithms and Minimum Spanning Trees; Fall 2005, Lecture 16. These MIT lectures support the greedy and MST proof framework, not the original Huffman exposition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

# Graph traversal and connectivity

Move from BFS distance layers and DFS time intervals to dependency order, strongly connected components, and single-vertex or single-edge failures.

Study **BFS → DFS → topological sorting → SCCs → low, articulation points, bridges, and biconnected blocks**.

Prerequisites are queues, stacks, recursion, sets, and induction. Fix the graph model and adjacency order before tracing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Choose traversal information from the required output

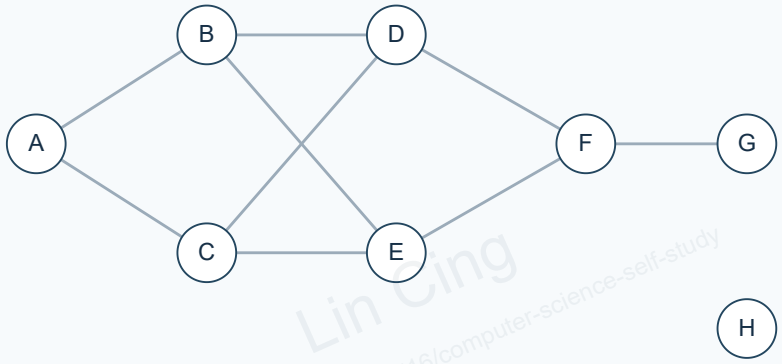
| Question | Information retained | Proof mechanism |
|---|---|---|
| Fewest edges to a vertex? | BFS distances and parents | FIFO layer invariant |
| When does a visit start and finish? | DFS colors, forest, times | Parenthesis and white-path theorems |
| Which task may run first? | DAG finish order or indegrees | Every arc respects the order |
| Which vertices reach each other? | SCCs and condensation DAG | Finish times and component boundaries |
| Does a deletion disconnect the graph? | Undirected low and edge stack | Subtree escape to an ancestor |

BFS d denotes distance; DFS d denotes discovery time. Keep SCC lowlink updates distinct from undirected low updates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The graph model determines how edges count

Fixed graph G, source A, isolated H




8 vertices, 9 undirected edges. Neighbors in alphabetical order.

Unweighted simple graph: V={A,B,C,D,E,F,G,H}; edges {AB,AC,BD,BE,CD,CE,DF,EF,FG}. From A, find each minimum edge distance and a corresponding path. H is isolated.



# Lists and a matrix store the same adjacency relation

Two representations of the same graph

Adjacency lists

A: B, C

B: A, D, E

C: A, D, E

D: B, C, F

E: B, C, F

F: D, E, G

G: F

H: ∅

| | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| D | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| E | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| F | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| G | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| H | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Lists contain 18 adjacency entries. Matrix has 8 × 8 = 64 cells and A = $A^T$.

Each undirected edge contributes two list entries, for $2|E| = 18$ total. A directed arc $u \to v$ appears only in `Adj[u]`, giving $|E|$ total entries.

## Edge lookup and neighbor enumeration have different costs

`Adj[u]` lists u's neighbors; $M[u, v] = 1$ iff uv is an edge. Here the matrix is symmetric with a zero diagonal. Let $n = |V|$ and $m = |E|$. Use ordinary unsorted lists and a directly indexed matrix.

| Operation | Adjacency lists | Adjacency matrix |
|---|---|---|
| Storage | $\Theta(n + m)$ | $\Theta(n^2)$ |
| Test $u \to v$ | Worst-case $O(1 + \deg^+(u))$ | $\Theta(1)$ |
| Enumerate neighbors of u | $\Theta(1 + \deg^+(u))$ | $\Theta(n)$ |

For undirected graphs, use $\deg(u)$ instead of outdegree. The extra 1 covers an empty list. A directed adjacency matrix need not be symmetric.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Loops and parallel edges require an explicit convention

In a directed multigraph, store one list entry for each arc, including each parallel arc and each self-loop. For an undirected multigraph, use two endpoint incidences per edge. A self-loop then contributes two entries in the same list, preserving a total of $2m$.

A binary matrix stores adjacency but loses multiplicity. A count matrix can retain multiplicity. Separate edge identities or individual weights need more storage. If an undirected loop is stored only once instead, the list total is $2m - \ell$, where $\ell$ counts loops.

BFS enqueues a vertex only on its first discovery. Loops and parallel edges add scans but do not add duplicate vertex enqueues.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Color, distance, and parent record different facts

White means undiscovered. Gray means discovered with its neighbor scan unfinished. Black means its neighbor scan has finished. The only transitions are `WHITE → GRAY → BLACK`.

$d[v]$ estimates the edge distance from A and starts at $\infty$. $\pi[v]$ records the vertex that first discovers v and starts at NIL. Set `d[A]=0`, color A gray, and enqueue it.

At each while-condition test, Q contains exactly the gray vertices. A dequeued vertex remains gray during its scan, so this equality does not hold at every instruction. In the handwritten notes p34, DFS d is a discovery timestamp. Here BFS d is a distance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Mark a vertex gray before enqueueing it

```
for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
color[A]=GRAY; d[A]=0; Q=[A]
while Q is not empty:
    u=dequeue_front(Q)
    for v in Adj[u] in alphabetical order:
        if color[v]==WHITE:
            color[v]=GRAY; d[v]=d[u]+1; π[v]=u
            enqueue_back(Q,v)
    color[u]=BLACK
```

Queue operations must take O(1) time. Removing an array's first element by shifting all remaining elements does not satisfy that assumption.

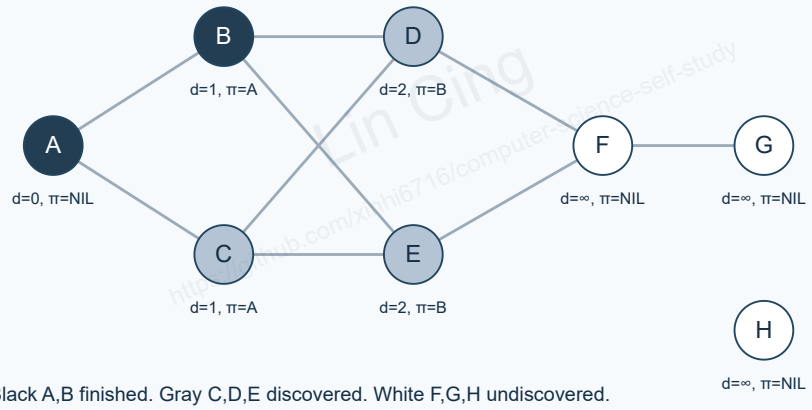
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The first two scans establish distances 1 and 2

Initially `Q=[A]`. Scanning A discovers B and C. Both turn gray, get distance 1, and get parent A. A turns black, leaving `Q=[B,C]`.

Dequeue B next. A is black, while D and E are white. Assign `d[D]=d[E]=2` and `π[D]=π[E]=B`. The completed scan leaves `Q=[C,D,E]`.

After scanning B, Q = [C, D, E]




Black A,B finished. Gray C,D,E discovered. White F,G,H undiscovered.

C is already queued and must be processed before D and E.



## Record every completed scan in the queue trace

Complete FIFO queue trace

| Scan finished | Newly discovered | Q front → back | Black vertices |
|---|---|---|---|
| init | A | [A] | ∅ |
| A | B,C | [B, C] | A |
| B | D,E | [C, D, E] | A,B |
| C | ∅ | [D, E] | A,B,C |
| D | F | [E, F] | A,B,C,D |
| E | ∅ | [F] | A,B,C,D,E |
| F | G | [G] | A,B,C,D,E,F |
| G | ∅ | [] | A,B,C,D,E,F,G |

Each row precedes the while test. Q contains exactly gray vertices. H stays white.

## Read the queue trace step by step

When C is scanned, D and E are already gray. Their distances and parents stay unchanged. D first discovers F, assigning `d[F]=3`, `π[F]=D`. E then sees F gray. F discovers G, assigning `d[G]=4`, `π[G]=F`.

After scanning G, Q is empty. Each of the seven reachable vertices was enqueued once. H was never enqueued.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

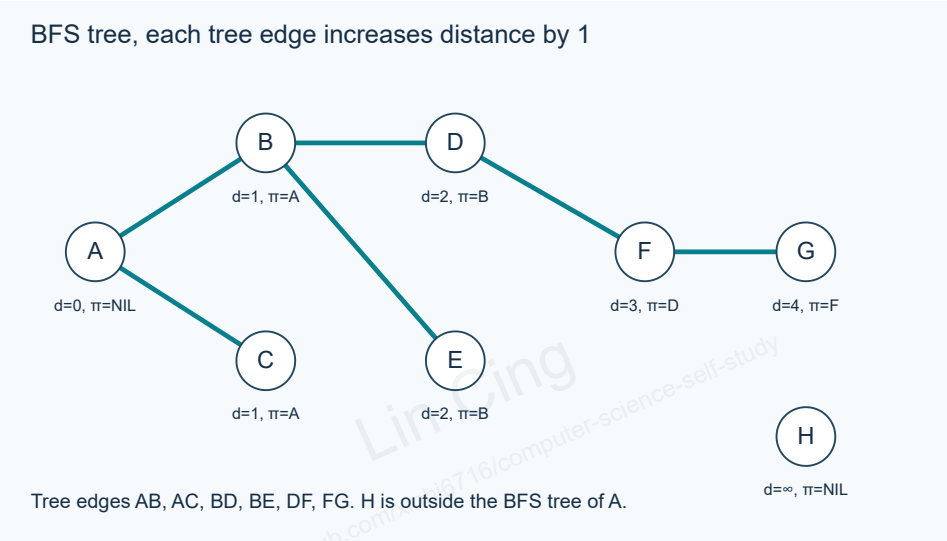


## The parent tree retains one shortest path

| v | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| d[v] | 0 | 1 | 1 | 2 | 2 | 3 | 4 | ∞ |
| π[v] | NIL | A | A | B | B | D | F | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The parent tree retains one shortest path (continued)




Following parents from G gives `G,F,D,B,A`. Reverse it to obtain `A,B,D,F,G`, a four-edge path. NIL can also mean the source. Check whether the target is A before interpreting NIL as unreachable.



## FIFO maintains nondecreasing distance order

When Q is nonempty, its distances are nondecreasing and the tail distance is at most one greater than the head distance. Only two adjacent layers can coexist in Q.

Dequeue removes the smallest value at the front. Scanning a vertex u with distance k assigns k+1 to new vertices and appends them at the back. Unprocessed vertices in the current layer stay ahead of the next layer.

After B, queued distances are `[1,2,2]`. After D, they are `[2,3]`. Every vertex of distance k is scanned before any vertex of distance k+1.

## Shortest-path correctness needs both inequalities

Let $\delta(A, v)$ be the minimum edge count in the graph. For any discovered vertex, its parent chain is an actual path of length $d[v]$. Thus $\delta(A, v) \leq d[v]$.

Now induct on true distance k. A is correct for k=0. If $\delta(A, v) = k > 0$, its predecessor u on a shortest path has distance k−1. By induction u is labeled correctly and is eventually scanned.

If v is white when u scans it, assign $d[v] = k$. If v was already discovered, FIFO ensures the earlier discovering vertex had distance at most k−1, so $d[v] \leq k$. Combining both bounds gives $d[v] = k$. Every reachable vertex is therefore discovered with its minimum edge count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Neighbor order changes the tree but not distances

Reverse every adjacency list. A discovers C before B. C discovers E and D, giving `π[D]=π[E]=C`. E is scanned before D, giving `π[F]=E`.

The dequeue order becomes `A,C,B,E,D,F,G`. The tree path to G becomes `A,C,E,F,G`. The distance vector stays `(0,1,1,2,2,3,4,∞)`.

The correctness proof applies to both trees. The graph and source determine $\delta$. The parent records a choice among equally short paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One BFS does not traverse every component

After BFS from A, H remains white with $d[H] = \infty$. No path from A reaches H.

To find connected components in an undirected graph, initialize all vertices once. Then inspect each vertex and run BFS from every white root, assigning a new component label. Here the components are `{A,B,C,D,E,F,G}` and `{H}`.

Distances in the resulting forest are relative to each tree's root. In a directed graph, this outer loop produces a reachability search forest that depends on root order. It does not compute strongly connected components.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The representation determines the scan cost

Let $V_R$ be the vertices reachable from the source. Lists require $\Theta(n)$ initialization, one dequeue per reachable vertex, and one scan per associated adjacency entry. Total time is $\Theta(n + \sum_{u \in V_R} |Adj[u]|)$, bounded by $O(n + m)$.

A matrix scans a full row for each dequeued vertex, taking $\Theta(n + n|V_R|)$ time and $\Theta(n^2)$ in the worst case. Color, distance, parent, and queue require $O(n)$ extra space in either case, excluding the graph.

Here lists scan 18 entries. The matrix scans $7 \times 8 = 56$ cells. Both initialize 8 vertices. A full forest also scans H, raising matrix probes to 64 while list scans stay at 18.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## BFS minimizes the number of edges

If all edges have the same positive weight c, a minimum-edge path also minimizes total weight, with cost $c\,d[v]$. Unequal weights invalidate that conclusion.

For example, directed edges s→t of weight 10, s→x of weight 1, and x→t of weight 1 give a one-edge BFS path of cost 10. The minimum-weight path has two edges and cost 2.

The fixed graph and five SVGs are original examples. CLRS § 22.1–22.2 supports the representations, algorithm, and proof. MIT04 L14 and MIT05 L17 pp43–46,59 explain unit weights and FIFO. The notes p40 use BFS to choose a minimum-edge augmenting path in Edmonds–Karp.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## DFS returns a forest and nested time intervals

Full DFS discovers and finishes every vertex exactly once. It records parent pointers, discovery times d, and finish times f. Tree edges give reachability witnesses; intervals encode ancestry. A DFS tree path need not be shortest.

Trace one cyclic directed graph, then delete two edges to obtain a DAG and verify topological orders.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Fix the graph and both traversal orders

Use a finite directed graph with no parallel edges or self-loops. The outer loop uses A,B,C,D,E,F. Adjacency lists have this exact order.

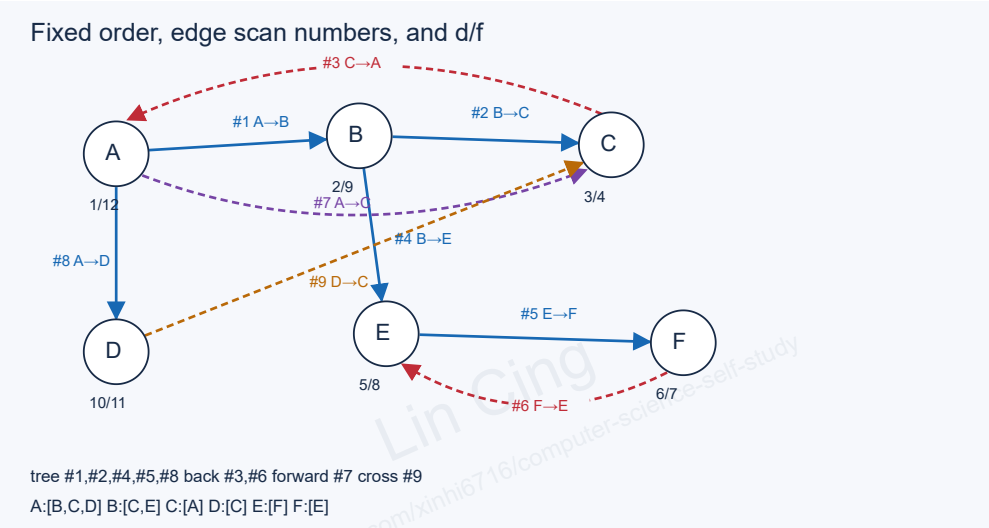
```
A:[B,C,D] B:[C,E] C:[A] D:[C] E:[F] F:[E]
```

Blue edges are tree edges, red back, purple forward, and orange cross. Types belong to this particular DFS execution; a different neighbor order may change them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fix the graph and both traversal orders (continued)




Sources • MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22.

## Three colors describe a recursive call

WHITE means undiscovered. GRAY means discovered with an active call. BLACK means every outgoing edge has been scanned. The current source of an edge scan is the deepest GRAY vertex. All GRAY vertices form the ancestor chain on the call stack.

The clock advances only on discovery and finish, not on edge scans. Each vertex has $1 \le d[u] < f[u] \le 2|V|$. The $2|V|$ event times are distinct.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Full DFS and VISIT have different jobs

```
DFS(G):
  all colors = WHITE; all parents = NIL; time = 0
  for u in fixed vertex order:
    if color[u] == WHITE: VISIT(u)
VISIT(u):
  time += 1; d[u] = time; color[u] = GRAY
  for v in Adj[u] in fixed order:
    if color[v] == WHITE:
      parent[v] = u; VISIT(v)
  color[u] = BLACK; time += 1; f[u] = time
```

VISIT(u) explores vertices that can be discovered through white vertices from u. The outer loop covers all remaining vertices. A directed DFS forest is not a partition into strongly connected components. Edges may point to BLACK vertices in previous trees.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The stack follows the event clock

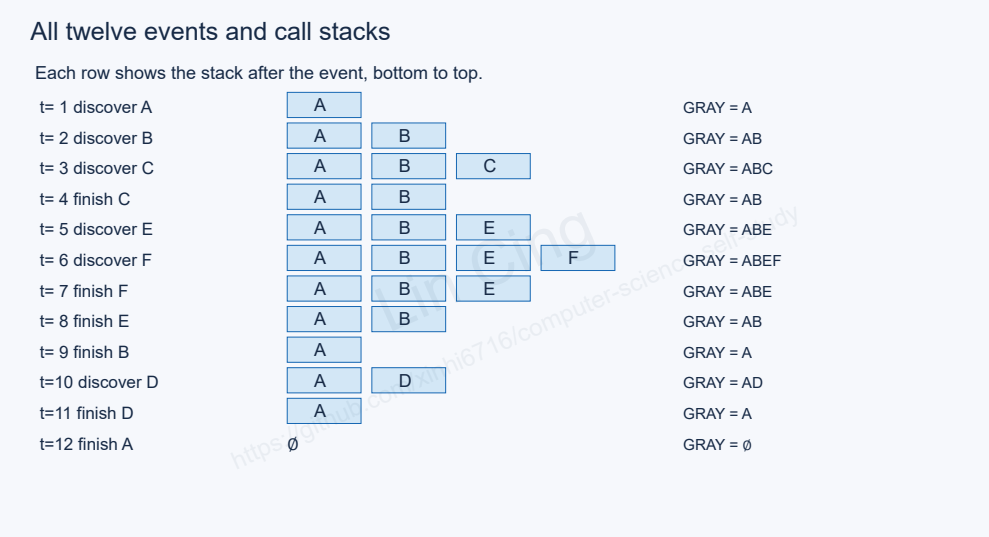
| t | event | stack after |
|---|---|---|
| 1 | discover A | A |
| 2 | discover B | AB |
| 3 | discover C | ABC |
| 4 | finish C | AB |
| 5 | discover E | ABE |
| 6 | discover F | ABEF |
| 7 | finish F | ABE |
| 8 | finish E | AB |
| 9 | finish B | A |
| 10 | discover D | AD |
| 11 | finish D | A |
| 12 | finish A | ∅ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The stack follows the event clock (continued)

Discovery pushes a call; finish pops it. At t=4, control returns to B and only then scans B→E. At t=9, control returns to A, which must still scan C and D. A finished vertex does not mean the entire algorithm has finished.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Scan every edge, including visited targets

| # | edge | target | type | stack | clock |
|---|---|---|---|---|---|
| 1 | AB | W | tree | A | 1 |
| 2 | BC | W | tree | AB | 2 |
| 3 | CA | G | back | ABC | 3 |
| 4 | BE | W | tree | AB | 4 |
| 5 | EF | W | tree | ABE | 5 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Scan the remaining edges and compare their types

| # | edge | target | type | stack | clock |
|---|---|---|---|---|---|
| 6 | FE | G | back | ABEF | 6 |
| 7 | AC | B | forward | A | 9 |
| 8 | AD | W | tree | A | 9 |
| 9 | DC | B | cross | AD | 10 |

The clock column is the last event time before the scan. W/G/B is the target color at that scan. A→C reaches a completed descendant and is forward. D→C also reaches BLACK, but C is not a descendant of D, so it is cross.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Align timestamps with the DFS tree

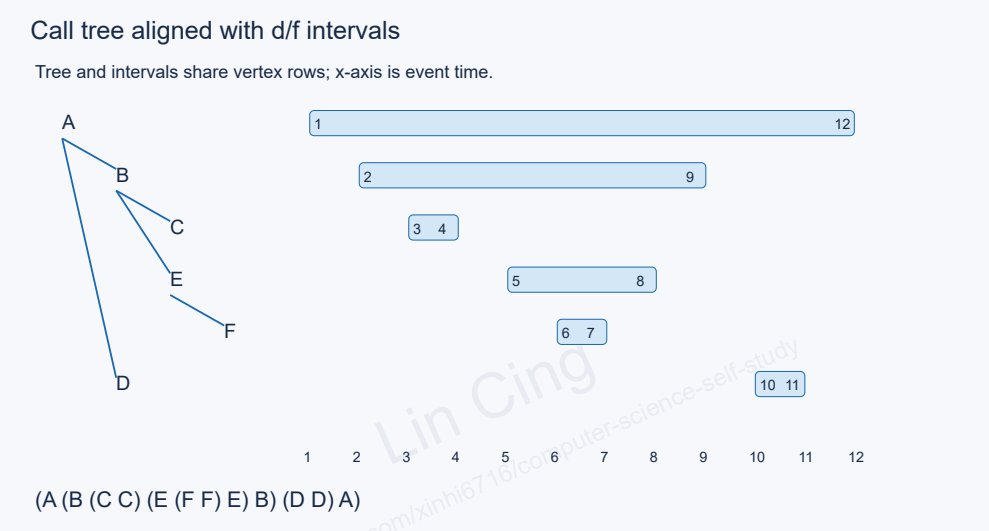
| vertex | d | f | parent |
|---|---|---|---|
| A | 1 | 12 | NIL |
| B | 2 | 9 | A |
| C | 3 | 4 | B |
| D | 10 | 11 | A |
| E | 5 | 8 | B |
| F | 6 | 7 | E |

Tree edges are A→B, B→C, B→E, E→F, A→D. Discovery order is A B C E F D. Finish order is C F E B D A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Align timestamps with the DFS tree (continued)




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The parenthesis theorem rules out crossing intervals

For distinct u,v, [d[u],f[u]] and [d[v],f[v]] are disjoint or one strictly contains the other. Strict containment holds exactly when the inner vertex is a proper descendant of the outer vertex.

Proof. Assume d[u]<d[v]. If f[u]<d[v], u has returned before v begins, so the intervals are disjoint. Otherwise v is discovered while u is GRAY. Its recursive call is nested inside u. The call for v must finish before control can finish u, giving d[u]<d[v]<f[v]<f[u]. Exchange u,v for the other discovery order.

This execution gives `(A (B (C C) (E (F F) E) B) (D D) A)`. The ordering d[u]<d[v]<f[u]<f[v] is impossible.

Sources • CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## White-path theorem requires a snapshot

At the instant u is discovered, v becomes a descendant of u if and only if a path u→…→v has every vertex after u WHITE at that instant. The start u has just become GRAY. Equivalently take the snapshot immediately before coloring u, when every path vertex is white. The zero-length path covers v=u.

Necessity follows from the tree path: every proper descendant is discovered later. For sufficiency, choose the first path vertex x that fails to become a descendant. Its predecessor w is u or a descendant. Scanning w→x discovers x immediately if white. Otherwise x was discovered after u and before w finished. In both cases x is discovered while u is active, so it is a descendant, a contradiction.

Reachability alone is insufficient. C→A→D exists, but at d[C]=3, A is GRAY; D does not become a descendant of C.

Sources • CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Four edge types in directed graphs

| Type | Test during scan | Relation after DFS |
|---|---|---|
| tree | target WHITE | parent[v]=u |
| back | target GRAY | v is ancestor of u, including a self-loop |
| forward | target BLACK and d[u]<d[v] | proper descendant, non-tree edge |
| cross | target BLACK and d[v]<d[u] | neither is ancestor of the other |

For distinct vertices, tree and forward edges share d[u]<d[v]<f[v]<f[u]. Timestamps cannot distinguish them without parent information. Back edges have d[v]<d[u]<f[u]<f[v]; cross edges have d[v]<f[v]<d[u]<f[u].

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Undirected edges are tree or back

In a simple undirected graph, each physical edge appears twice in adjacency lists. Classify it in the direction first encountered. Scanning the reverse entry of the parent edge does not create another back edge.

Proof. For {u,v}, assume d[u]<d[v]. The scan at u must reach v before u finishes, making v a descendant. If the first encounter is u to v, v is still white and the edge is tree. If first encountered from v to u, u is gray and the edge is back. No cross-subtree edge can remain.

For cycle detection skip the reverse entry of the same parent edge. In multigraphs use edge IDs: another parallel edge can make a length-2 cycle. A self-loop makes a length-1 cycle. State these conventions before applying the theorem.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## A directed cycle exists exactly when a back edge exists

Use a finite directed graph, allowing self-loops. A cycle is a nonempty directed closed path with distinct internal vertices. Run full DFS.

A back edge u→v closes the tree path v→…→u into a cycle. A self-loop is already a cycle. Conversely, choose the earliest discovered vertex v on a cycle and let u be its predecessor on that cycle. At discovery of v, every other cycle vertex is white. The cycle provides a white path from v to u. The white-path theorem makes u a descendant of v, so u→v is a back edge.

Here C→A witnesses A→B→C→A, and F→E witnesses E→F→E. An edge to BLACK alone does not prove a cycle.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

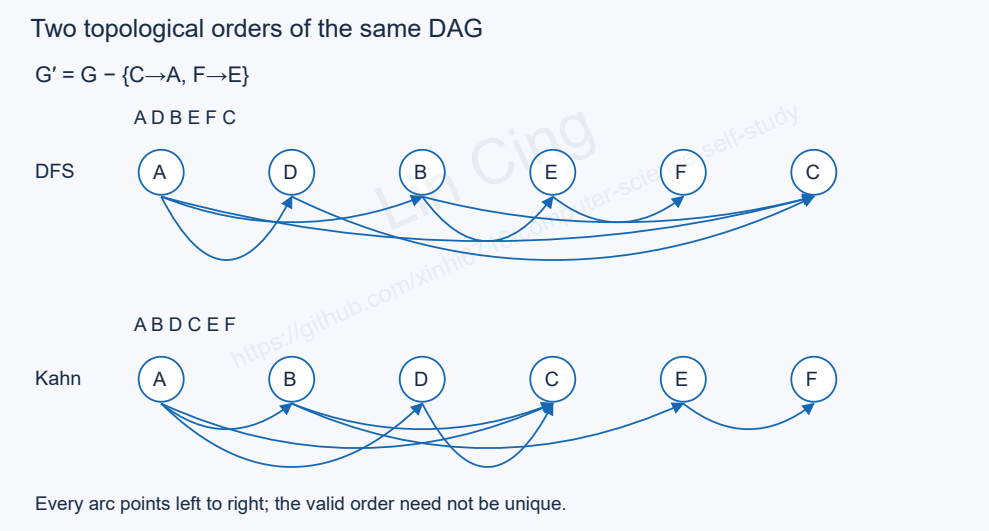
Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Delete two back edges to obtain a DAG

Let G′=G−{C→A,F→E}. Keep both traversal orders. All d/f values and tree edges stay unchanged. Reverse finish order is A D B E F C. Every retained edge points forward in that order, so it is a topological order.

The same order is invalid for G: C→A and F→E point backward. Every graph has a finish order, but only DAGs guarantee a topological order by reversing it.



## Why reverse finish order works

For any DAG edge u→v, v cannot be GRAY at the scan, because that would give a back edge and a directed cycle. If v is WHITE, recursion finishes v before u. If v is BLACK, v is already finished and u is not. Thus f[u]>f[v] for every edge. Sorting by decreasing finish time places each source before its target.

Append each vertex when it finishes and reverse the list once, or insert at the head of a linked list. Repeated insertion at the front of an array can cost $\Theta(V^2)$. Detect a back edge and reject topological sorting on a cyclic input.

Sources • CLRS3-22 § § 22.3–22.4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Kahn uses remaining indegrees on the same DAG

Repeatedly remove a vertex whose remaining indegree is zero and delete its outgoing edges. With alphabetical minimum selection, the execution is as follows.

| Removed | Available after deletion |
|---|---|
| A | B,D |
| B | D,E |
| D | C,E |
| C | E |
| E | F |
| F | ∅ |

The order A B D C E F differs from DFS but is valid. The invariant is that each selected vertex has no unprocessed predecessor. If vertices remain and every remaining indegree is positive, repeatedly following a predecessor must repeat a vertex, proving a cycle. FIFO gives $\Theta(V+E)$; alphabetical heap selection gives $O(E+V \log V)$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Cost and self-study checks

With adjacency lists, full DFS takes $\Theta(V+E)$: one call per vertex and one scan per directed edge. Two scans per undirected edge preserve this bound. An adjacency matrix requires $\Theta(V^2)$ time. Colors, parents, timestamps, and the deepest recursive chain use $\Theta(V)$ additional space.

To reproduce recursion with an explicit stack, store frames `(u,next adjacency index)` and write f[u] only after the final edge. Pushing all neighbors onto a vertex-only stack need not reproduce this tree, finish times, or classification.

Exercises A07-D01 through D06 test timestamps, classification, theorem assumptions, undirected models, cycle proofs, and topological ordering.

Sources • MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22.



## Connectivity questions have different outputs

An undirected connected component is a maximal set of mutually reachable vertices. A directed SCC requires reachability in both directions.

| Question | Output | Deletion test |
|---|---|---|
| Connected components / SCCs | Vertex partition | None |
| Articulation points | Vertex set | Removing a vertex and incident edges increases component count |
| Bridges | Edge set | Removing an edge increases component count |
| Vertex-biconnected blocks | Edge groups, with shared cut vertices | No articulation inside a block |

Undirected inputs here are finite simple graphs, without loops or parallel edges. Roots and adjacency lists use alphabetical order. Discovery numbers start at 1. Sources [S1, S2].

## A traversal forest labels undirected components

For each unlabelled vertex s, start DFS or BFS and assign one component_id to every vertex reached. If an unlabelled v were reachable from s, the first unlabelled vertex on a path would be discovered from its visited predecessor, a contradiction. Traversal cannot cross components because it follows existing edges. The outer loop must include isolated vertices. With adjacency lists, time is Θ(V+E) and auxiliary space is O(V). The two MIT BFS lectures support this traversal background; they do not supply SCC or articulation algorithms [S3, S4].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

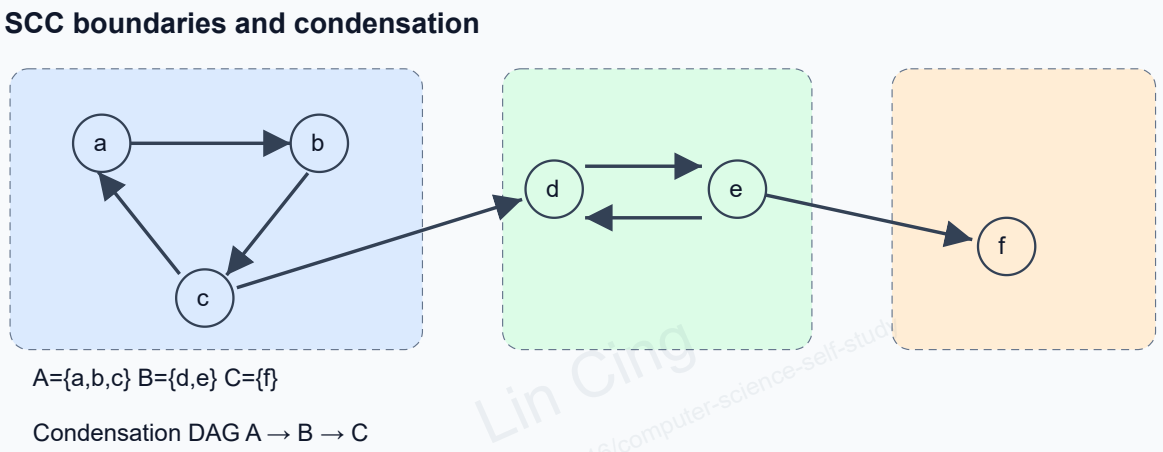


## Maximal SCCs produce a condensation DAG

Define u~v by mutual reachability. Zero-length paths give reflexivity; symmetry is explicit; path concatenation gives transitivity. SCCs are equivalence classes that partition V. Contract each SCC to a vertex, keep intercomponent arcs, discard loops, and optionally deduplicate arcs. This is the condensation graph. A directed cycle of two or more components would make all of them mutually reachable, contradicting maximality. The condensation is a DAG. A singleton SCC need not have a self-loop [S1 § 22.5].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Fix the example before tracing it




G has arcs ab, bc, ca, cd, de, ed, ef, where xy means x→y. The SCCs are A={a,b,c}, B={d,e}, C={f}; the condensation is A→B→C. One DFS from a reaches every vertex in G, so an original-graph DFS tree need not be one SCC.



## Kosaraju records finish times in the first pass

Run a complete DFS forest on G. Append each vertex to a finish list when its call returns. The finish list is f,e,d,c,b,a; reverse it for second-pass roots a,b,c,d,e,f.

| Vertex | Discovery d | Finish f | Parent |
|---|---|---|---|
| a | 1 | 12 | ∅ |
| b | 2 | 11 | a |
| c | 3 | 10 | b |
| d | 4 | 9 | c |
| e | 5 | 8 | d |
| f | 6 | 7 | e |

Increment the clock at discovery and finish. Distinguish the finish symbol f from vertex f [S1].

## Transpose preserves SCCs and reverses component arcs

$G^T$ replaces every $u \to v$ by $v \to u$. A path $u \rightsquigarrow v$ in $G$ corresponds to $v \rightsquigarrow u$ in $G^T$, so mutual-reachability classes stay unchanged. Transpose lists are a:[c], b:[a], c:[b], d:[c,e], e:[d], f:[e]. Its condensation is $C \to B \to A$. Create V empty lists, then append each reversed arc. This costs $\Theta(V+E)$. Clear visited for pass two, while retaining the first pass's finish order [S1].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The finish-time lemma has two discovery cases

For distinct SCCs with an arc $C \to D$, define $F(C)=\max\{f(v):v \in C\}$ and $d(C)=\min\{d(v):v \in C\}$. We prove $F(C)>F(D)$.

1. If $d(C)<d(D)$, the first discovered x in C has all-white paths to every vertex of C and, through the intercomponent arc, every vertex of D. They finish before x returns. Thus $F(C)=f(x)>F(D)$.
2. If $d(D)<d(C)$, the first discovered y in D reaches all of D but cannot reach C, since $D \rightsquigarrow C$ together with $C \to D$ would merge them. Every vertex of D therefore finishes before any vertex of C is discovered. Thus $F(D)<d(C)<F(C)$.

The lemma compares maximum finish times of components, not every pair of vertices across components [S1 Lemma 22.14].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why each second-pass tree is exactly one SCC

Choose the unvisited root u of greatest first-pass finish time, in SCC C. Inductively, earlier trees are complete SCCs. Thus all of C is white, and F(C) is maximal among remaining components. An arc C→D in $G^T$ implies D→C in G. The lemma gives F(D)>F(C), so D has already been visited in pass two. Traversal from C cannot enter another unvisited component. Conversely C remains strongly connected in $G^T$ and is entirely white, so DFS from u reaches all of C. The tree contains exactly C. The empty forest is the induction base; repeat until all vertices are visited. Two DFS passes and transpose construction take Θ(V+E) time and O(V+E) extra space including $G^T$ [S1 Theorem 22.16].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

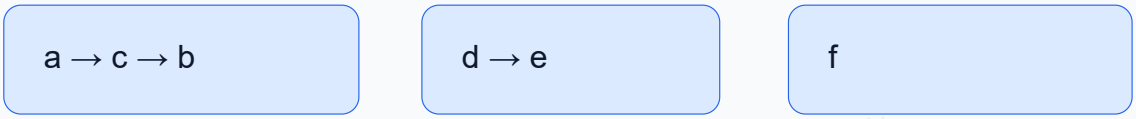


## Pass two shows roots, stacks, and skipped vertices

### Kosaraju directions and order

Pass 1 on G: finish order f,e,d,c,b,a

Reverse for root order a,b,c,d,e,f




Pass 2 on $G^T$: each box is one DFS tree and one SCC

Skip already visited root candidates b,c,e

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pass two shows roots, stacks, and skipped vertices (continued)

| Root candidate | Deepest call stack in $G^T$ | Discovery order | Tree output |
|---|---|---|---|
| a | [a,c,b] | a,c,b | {a,b,c} |
| b,c | Already visited; skip | ∅ | ∅ |
| d | [d,e]; d→c is already black | d,e | {d,e} |
| e | Already visited; skip | ∅ | ∅ |
| f | [f]; f→e is already black | f | {f} |

The call stack empties after each tree. Do not reorder output using second-pass finish times. Reusing G with decreasing f incorrectly lets a absorb all six vertices.



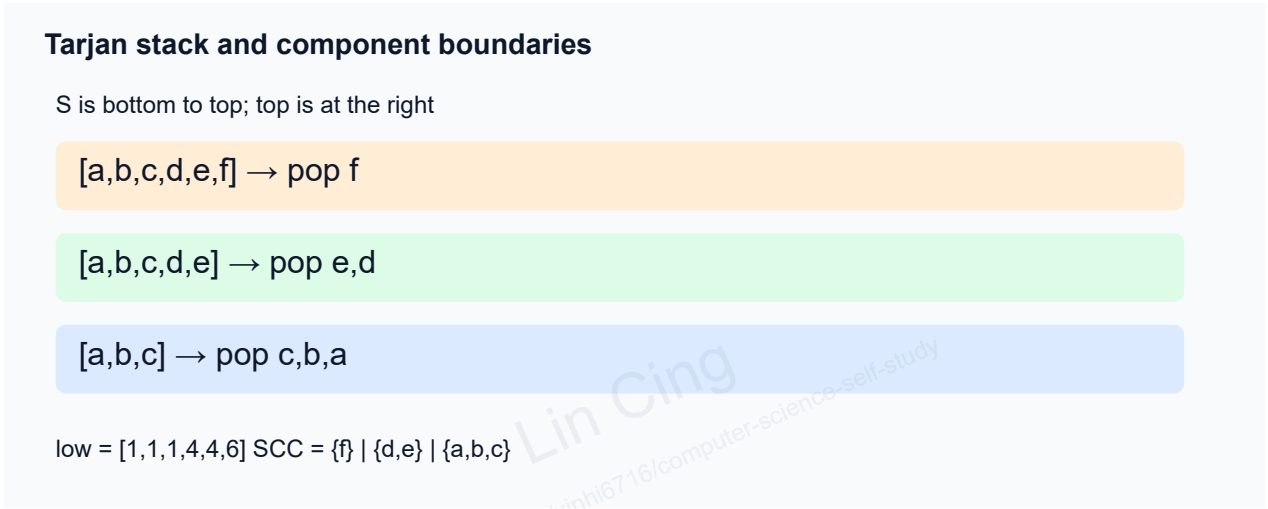
## Tarjan keeps a stack of unassigned vertices

disc[u] is the discovery index. low[u] is this recurrence's lowlink. S stores discovered vertices without an assigned SCC. onstack[u] means u∈S, and does not mean DFS-gray.

```
visit(u):
  disc[u] = low[u] = ++clock; push(S,u); onstack[u]=true
  for v in Adj[u]:
    if disc[v] is unset:
      visit(v); low[u]=min(low[u],low[v])
    else if onstack[v]: low[u]=min(low[u],disc[v])
  if low[u]==disc[u]:
    repeat: x=pop(S); onstack[x]=false; emit x
    until x==u
```

The outer loop visits every undiscovered vertex. Ignore edges to assigned SCCs. For an already discovered stack target, use disc[v], not an arbitrary substitution of low[v]. Modern notation adapted from [S5].

# A complete Tarjan return and pop trace




S runs bottom to top. Discovery indices for a,b,c,d,e,f are 1,2,3,4,5,6.



# Tarjan discoveries and low updates

| Event | Update | s |
|---|---|---|
| Enter a | disc[a]=low[a]=1 | [a] |
| Enter b | disc[b]=low[b]=2 | [a,b] |
| Enter c | disc[c]=low[c]=3 | [a,b,c] |
| c→a | low[c]=1 | [a,b,c] |
| Enter d | disc[d]=low[d]=4 | [a,b,c,d] |
| Enter e | disc[e]=low[e]=5 | [a,b,c,d,e] |
| e→d | low[e]=4 | [a,b,c,d,e] |

## Tarjan returns and emitted components

| Event | Update | s |
|---|---|---|
| Enter f | disc[f]=low[f]=6 | [a,b,c,d,e,f] |
| Return f, 6=6 | Pop f; emit {f} | [a,b,c,d,e] |
| e receives f, returns | low[e]=min(4,6)=4; no pop | [a,b,c,d,e] |
| d receives e, 4=4 | low[d]=4; pop e,d; emit {d,e} | [a,b,c] |
| c receives d, returns | low[c]=min(1,4)=1 | [a,b,c] |
| b receives c, returns | low[b]=1 | [a,b,c] |
| a receives b, 1=1 | low[a]=1; pop c,b,a; emit {a,b,c} | [] |

Final low is 1,1,1,4,4,6 and every onstack is false. After b returns it is no longer gray, but remains on S. The call stack and S differ.



## Separate directed lowlink from undirected low

| Item | Directed Tarjan SCC | Undirected cuts / bridges |
|---|---|---|
| Non-tree target | Must still be onstack | Ancestor, excluding reversed parent edge |
| Stored objects | Unassigned vertices | Edges for biconnected blocks |
| Boundary test | low[u]=disc[u] | low[v]≥disc[u] or > |
| Output | Vertex partition | Cut vertices, bridges, edge blocks |

A directed edge can reach a returned call still on S. For a→b,b→a,a→c,c→b, b's call returns before c→b is scanned; that edge must still update low[c]. Testing gray alone misclassifies c. If singleton x is emitted before visiting y with y→x, onstack[x] is false. That edge cannot merge the SCCs.

## Undirected low describes a subtree escape

For u, low[u] is the smallest discovery index reachable by descending zero or more tree edges inside u's subtree, then taking at most one back edge to an ancestor; disc[u] is included. Initialize low[u]=disc[u]. After tree child v returns, minimize with low[v]. For a nonparent edge to discovered ancestor w, minimize with disc[w]. Undirected DFS has no edges between separate DFS subtrees: the first subtree would discover the other through that edge. Thus escaping past u depends on a back edge to a proper ancestor of u [S1 Problem 22-2].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Articulation tests distinguish roots from nonroots

A nonroot u is an articulation point iff some DFS child v has low[v]≥disc[u]. Then v's subtree cannot reach a proper ancestor of u, so deleting u separates it from the parent side. If every child subtree reaches a proper ancestor, those routes keep the remaining vertices connected. A root r is an articulation point iff it has at least two DFS-tree children. Separate root subtrees have no interconnecting edge. With one child, its remaining subtree stays connected; zero children also gives no articulation. Do not apply the nonroot test to a root. In a—b, low[b]≥disc[a], but deleting a does not increase component count [S1 22-2(a,b)].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

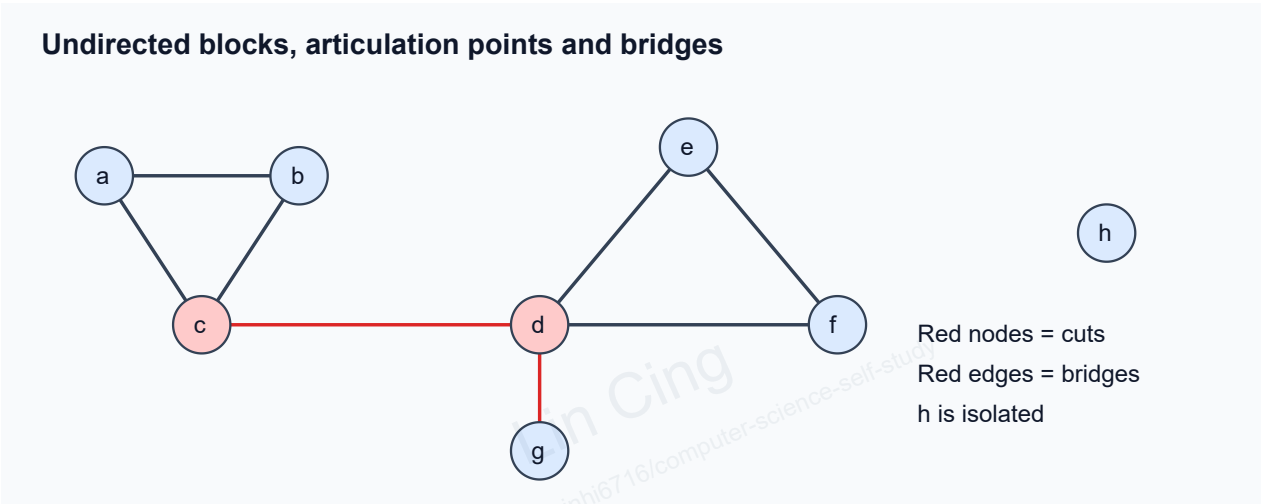
## A bridge requires a strict inequality

A tree edge (u,v) is a bridge iff low[v]>disc[u]. Its subtree has no back edge to u or any earlier ancestor, so deleting the edge separates the sides. Equality low[v]=disc[u] gives a return to u and a cycle containing (u,v), so the edge is not a bridge. Deleting u also deletes that return endpoint, so nonroot u can still be an articulation point. In a simple undirected graph, every non-tree edge lies on a cycle and is not a bridge. For multigraphs, skip exactly the parent edge_id rather than all edges to the parent vertex. Two parallel edges are not bridges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## An undirected example with complete disc / low values




Edges are ab,bc,ca,cd,de,ef,fd,dg; h is isolated. DFS uses alphabetical order.

## Undirected DFS disc / low: a through d

| Vertex | disc | low | Parent | DFS children |
|---|---|---|---|---|
| a | 1 | 1 | ∅ | b |
| b | 2 | 1 | a | c |
| c | 3 | 1 | b | d |
| d | 4 | 4 | c | e,g |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Undirected DFS disc / low: e through h and the tests

| Vertex | disc | low | Parent | DFS children |
|---|---|---|---|---|
| e | 5 | 4 | d | f |
| f | 6 | 4 | e | ∅ |
| g | 7 | 7 | d | ∅ |
| h | 8 | 8 | ∅ | ∅ |

Back edges c→a and f→d lower low[c] to 1 and low[f] to 4, then child returns propagate them. Articulation points are {c,d}; bridges are {cd,dg}. Root a has one child and is not a cut vertex. Vertex d is a nonroot despite its multiple children.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Biconnected blocks need an edge stack

Use the block convention: each bridge forms a two-vertex, one-edge block; isolated vertices receive singleton vertex blocks. Blocks with at least three vertices are maximal subgraphs without a cut vertex. This differs from the cycle-based bcc convention in CLRS 22-2, which partitions only nonbridge edges. Push each tree edge (u,v) before descent. Push each back edge to an ancestor once. When v returns with low[v]≥disc[u], repeatedly pop edges, including (u,v), then stop and emit that block. Apply this boundary to every root child independently of the root articulation test. Each edge is pushed and popped once, for Θ(V+E) time. Blocks partition edges; cut vertices can belong to several blocks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Every block boundary on the edge stack

Stacks run bottom to top. ca and fd are back edges; reversed parent edges are never pushed again.

| Event | Edge stack | Output |
|---|---|---|
| Push ab,bc,ca,cd,de,ef,fd in order | [ab,bc,ca,cd,de,ef,fd] | ∅ |
| f returns to e, 4<5 | Unchanged | ∅ |
| e returns to d, 4≥4 | [ab,bc,ca,cd] | Pop fd,ef,de |
| Push dg; g returns, 7≥4 | [ab,bc,ca,cd] | Pop dg |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Return to the root and emit blocks

| Event | Edge stack | Output |
|---|---|---|
| d returns to c, $4 \geq 3$ | [ab,bc,ca] | Pop cd |
| c returns to b, $1 < 2$ | Unchanged | ∅ |
| b returns to a, $1 \geq 1$ | [] | Pop ca,bc,ab |
| Start new root h | [] | Isolated vertex {h} |

Blocks are {de,ef,fd}, {dg}, {cd}, {ab,bc,ca}, plus {h}. Popping only one edge, or stopping before de, splits the triangle incorrectly. The edge stack must be empty after each complete DFS tree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Use the formulas to answer failure questions

The example initially has two connected components: the main graph and {h}. Deleting c leaves {a,b}, {d,e,f,g}, {h}, for three components. Deleting d leaves {a,b,c}, {e,f}, {g}, {h}, for four. Deleting bridge cd produces three; deleting de still leaves two. A cut vertex can increase component count by more than one. A bridge models one failed link; a cut vertex models removal of a node and all incident links. SCCs instead express mutual reachability under direction constraints.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Sources and their scope

S1 is CLRS third edition §22.5, printed pp. 615–620; Problem 22-2, pp. 621–622, supplies cuts and bridges. S2 is the original handwritten note, PDF pp. 34–36, covering DFS, timestamps and edge types. Those reviewed pages do not contain SCC or articulation low formulas. S3 is MIT 2004 L14, PDF pp. 45–59; S4 is MIT 2005 L17, whose same range covers BFS. They support traversal background only. S5 is Tarjan's 1972 original paper; SCC notation is modernized to disc/low/onstack.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

CLRS, Introduction to Algorithms, 3rd ed., §§22.1–22.5 and Problem 22-2. Bridges are included here as one-edge blocks. Original handwritten notes 演算法_wjungle, PDF pp. 34–36, 40, for DFS notation and BFS augmenting paths. MIT 6.046J Fall 2004, L13 pp. 2–6 and L14 pp. 43–59; Fall 2005, lec16 pp. 2–5 and lec17 pp. 43–59. Representation and BFS background. MIT 6.006 Fall 2011 Lecture 14 pp. 2–6; Spring 2020 Lecture 10 pp. 2–4. DFS and topological order. Robert Tarjan, “Depth-First Search and Linear Graph Algorithms,” SIAM J. Comput. 1(2), 146–160 (1972), doi:10.1137/0201010. SCCs.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Minimum spanning trees and shortest paths

An MST minimizes total edge weight while connecting all vertices. A shortest path minimizes the cost between specified endpoints. Their states and safe choices differ.

Study **cuts and MSTs → relaxation, DAGs, Bellman–Ford, Dijkstra → APSP, min-plus, Floyd–Warshall, Johnson**.

Prerequisites are the disjoint-sets unit in this series (original chapter 21), priority-queue contracts, and chapter 7 graphs, paths, and topological order. Original chapter 21 covers MAKE-SET, UNION, FIND, rank, path compression, and the detailed inverse-Ackermann bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Fix the problem before choosing an algorithm

| Problem and assumptions | Correctness argument |
|---|---|
| Minimum connection cost in an undirected graph | Cut property and safe-edge exchange. |
| Single-source paths in a DAG | Topological order completes predecessors first. |
| Single-source paths with negative edges | Bellman–Ford edge budgets and negative-cycle detection. |
| Single-source paths with nonnegative edges | Dijkstra's settled-vertex invariant. |
| All ordered endpoint pairs | DP states or Johnson's potential reweighting. |

Check the distance definition, graph assumptions, state trace, and cost model. Distinguish the effects of negative edges and negative cycles.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-01 | Minimize connection cost

For an undirected weighted graph G=(V,E), w:E→ℝ, an MST is a spanning tree minimizing Σe∈T w(e). A tree on all vertices has |V|−1 edges. MST paths need not be shortest paths from a root.

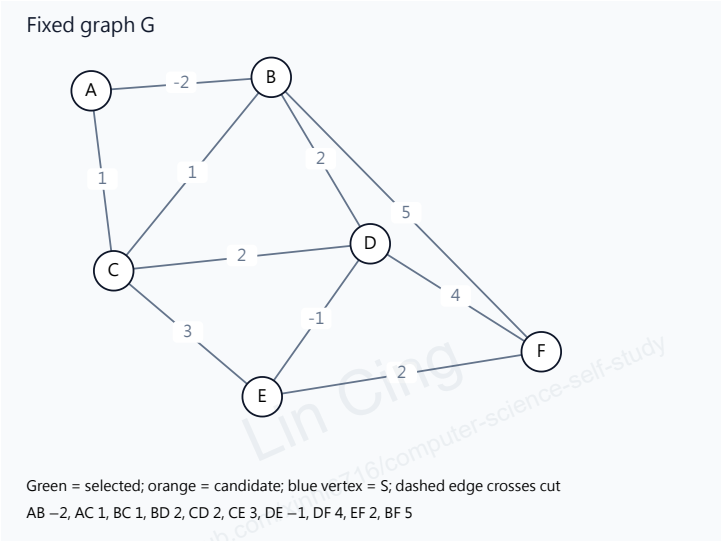
Prerequisites are MAKE-SET, UNION, FIND, rank and path compression from the disjoint-sets unit in this series (original chapter 21), heap contracts, and chapter 7 paths, cycles and cuts. The detailed DSU amortized analysis is in original chapter 21.

Sources [CLRS23 § 23.1; MIT04 L13.8–9; MIT05 L16.8–9].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# A08-MST-02 | One graph, two greedy rules




V={A,B,C,D,E,F}. AB=−2, AC=BC=1, BD=CD=EF=2, CE=3, DE=−1, DF=4, BF=5.

## A08-MST-03 | Safe preserves some optimum

Kruskal sorts by (weight,edge name). Prim starts at A and extracts by (key,vertex name). It changes a parent only for a strictly lower key. These conventions apply to every trace.

Maintain $A \subseteq$ some MST, rather than every MST. Edge e is safe for A when $A \cup \{e\}$ is contained in some MST.

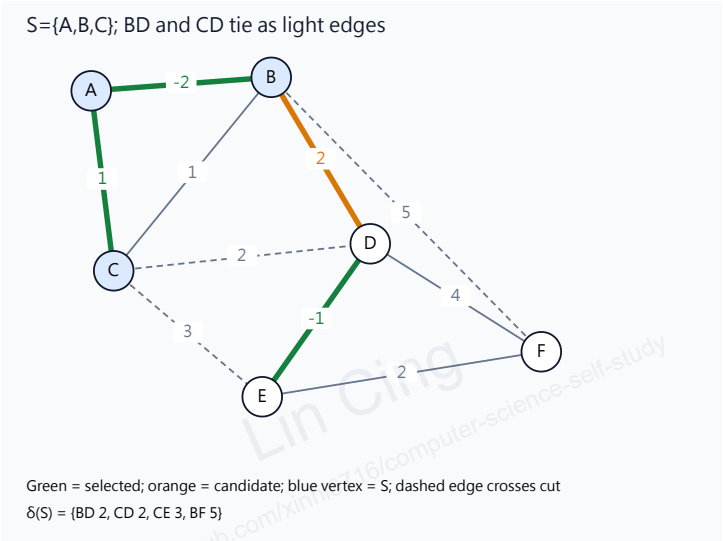
A cut (S,V−S) respects A if no edge of A crosses it. A light edge has minimum weight among edges crossing that cut. Several light edges may tie.

Source [CLRS23 Theorem 23.1].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A08-MST-04 | State the cut property with ties




If A extends to an MST and the cut respects A, any light crossing edge is safe for A. Here $\delta(ABC)=\{BD\ 2, CD\ 2, CE\ 3, BF\ 5\}$. Either BD or CD is safe.

# A08-MST-05 | Account for every exchange step

The unique light edge of a cut occurs in every MST. With ties, a light edge occurs in some MST; it need not occur in every MST.

MIT04/05 assume distinct weights for simplicity. We use the general CLRS statement.

Choose an MST T containing A. If e∈T, we are done. Otherwise T+e creates one cycle. Since e crosses the cut, the T-path between its endpoints contains another crossing edge f.

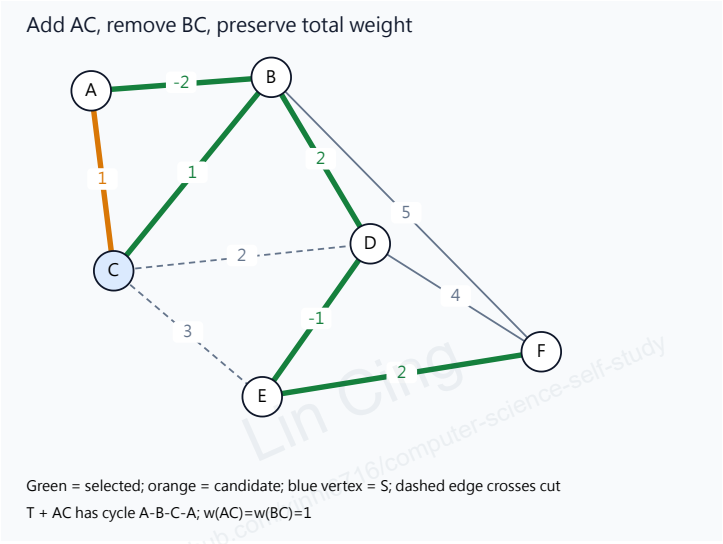
The cut respects A, so f∉A. Since e is light, w(e)≤w(f). T′=T−f+e is a tree containing A∪{e}, and w(T′)≤w(T).

Optimality of T forces equality. Ties are allowed. If e is uniquely light, excluding e would give a strictly cheaper tree and a contradiction.

Sources [CLRS23 pp627–628; MIT04 L13.22–25; MIT05 L16.22–25].



# A08-MST-06 | An exchange preserves A




Let A={AB,DE} and T={AB,BC,BD,DE,EF}. The cut S={C} respects A. AC and BC are both light, with weight 1. Adding AC creates A–B–C–A; removing BC preserves A.

## A08-MST-07 | The cycle property needs strictness

The result {AB,AC,BD,DE,EF} also weighs 2. Safe does not mean that the edge already belongs to the particular T we chose.

If e is the unique heaviest edge of a cycle, no MST contains e. If an MST T contained e, removing it would create a cut. Another cycle edge f crosses that cut and satisfies w(f)<w(e). Exchanging them would lower the weight.

If e only ties for heaviest, some MST excludes e. In a unit-weight triangle, every edge can occur in an MST and every edge can be omitted from one. Do not delete all tied edges at once.

Source [CLRS23 exercise 23.1-5, derived].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A08-MST-08 | Kruskal tests current components

Scan edges in increasing (weight,edge name) order. Accept uv and UNION only when FIND(u)≠FIND(v). Otherwise an already selected path joins u and v, so accepting uv creates a cycle.

An accepted edge is light across the cut of a current component, which respects A. DSU represents connectivity of (V,A), not connectivity of the input graph.

Source [CLRS23 § 23.2].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A08-MST-09 | All ten Kruskal rounds

| Round | Edge | Decision | DSU components after round |
|---|---|---|---|
| 0 | ∅ | initialize | A / B / C / D / E / F |
| 1 | AB, −2 | accept | AB / C / D / E / F |
| 2 | DE, −1 | accept | AB / C / DE / F |
| 3 | AC, 1 | accept | ABC / DE / F |
| 4 | BC, 1 | reject, B–A–C already connected | ABC / DE / F |
| 5 | BD, 2 | accept | ABCDE / F |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



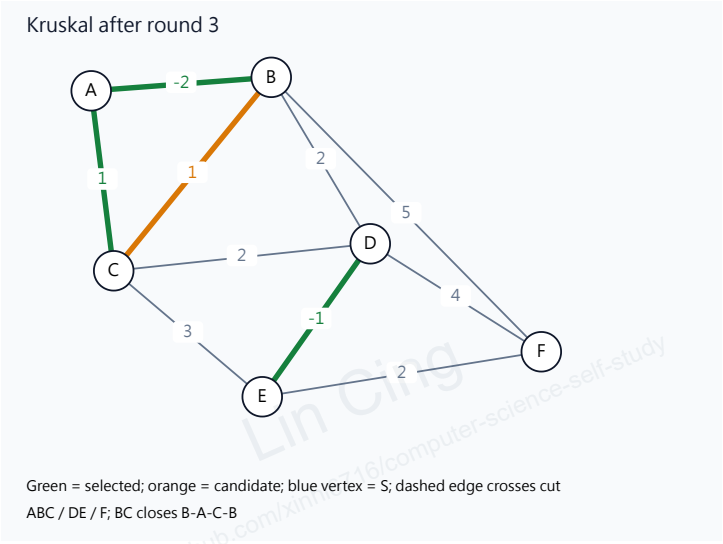
## A08-MST-09B | Kruskal: rounds 6–10

| Round | Edge | Decision | DSU components after round |
|---|---|---|---|
| 6 | CD, 2 | reject, C–A–B–D already connected | ABCDE / F |
| 7 | EF, 2 | accept | ABCDEF |
| 8 | CE, 3 | reject, C–A–B–D–E already connected | ABCDEF |
| 9 | DF, 4 | reject, D–E–F already connected | ABCDEF |
| 10 | BF, 5 | reject, B–D–E–F already connected | ABCDEF |

Round 7 reaches five accepted edges, so an implementation may stop there. This table scans all ten edges to expose every rejection. Components are semantic sets, not the internal DSU parent trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

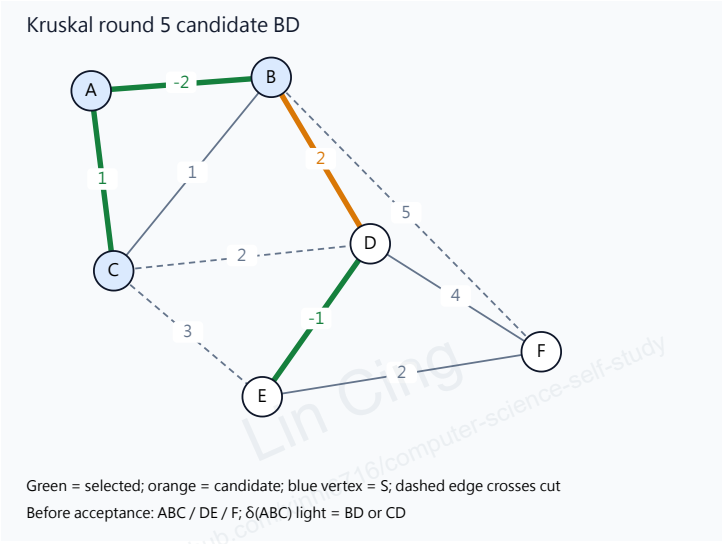
# A08-MST-10 | Reject BC, accept BD




Round 4 rejects BC because its endpoints share a component, not because it is expensive.



# A08-MST-10 | Reject BC, accept BD (continued)




The round 5 cut is the cut from card 04. CD has equal weight and could replace BD under a different tie order. It is considered later and then closes a cycle.

## A08-MST-11 | A Prim key prices one edge

S contains extracted vertices. For $v \notin S$, $key[v]=\min\{w(u,v):u\in S\}$, or $\infty$ if no such edge exists. parent[v] records a vertex attaining this minimum.

The minimum-key vertex contributes a light edge across $(S,V-S)$. Update v only while $v\in Q$ and $w(u,v)<key[v]$. Initial key[A]=0 chooses the root and is not an edge.

Sources [CLRS23 pp634–636; MIT04 L13.26; MIT05 L16.26].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



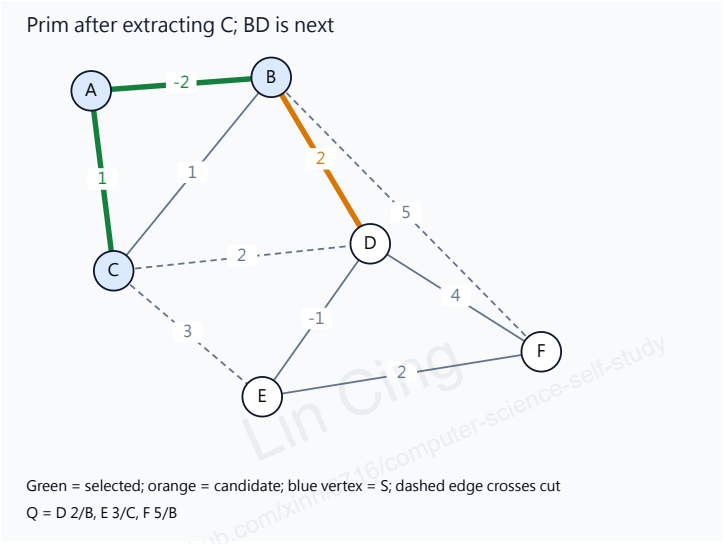
## A08-MST-12 | Every Prim key, parent and queue

| Extract | S before extraction | Add | Q after relaxation, key/parent, in priority order |
|---|---|---|---|
| Initial | ∅ | ∅ | A 0/∅, B ∞/∅, C ∞/∅, D ∞/∅, E ∞/∅, F ∞/∅ |
| A | ∅ | root | B −2/A, C 1/A, D ∞/∅, E ∞/∅, F ∞/∅ |
| B | A | AB | C 1/A, D 2/B, F 5/B, E ∞/∅ |
| C | AB | AC | D 2/B, E 3/C, F 5/B |
| D | ABC | BD | E −1/D, F 4/D |
| E | ABCD | DE | F 2/E |
| F | ABCDE | EF | ∅ |

Q is written in logical priority order, not heap-array order. Vertices absent from Q belong to S. Final parents are $B\leftarrow A, C\leftarrow A, D\leftarrow B, E\leftarrow D, F\leftarrow E$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

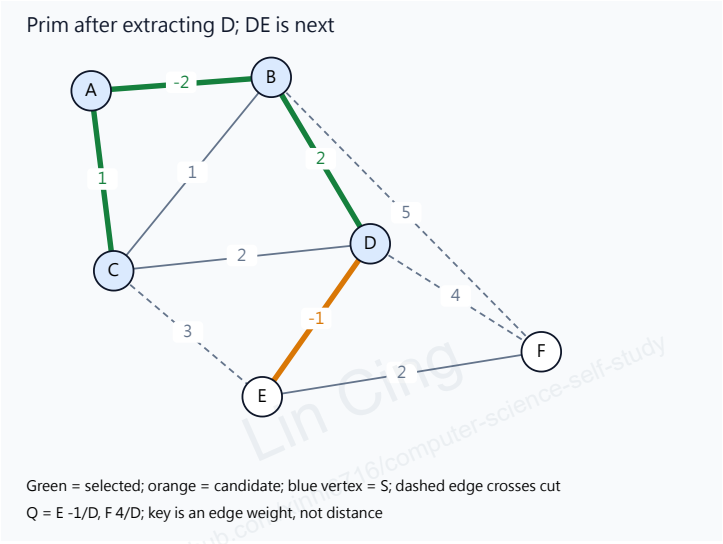
# A08-MST-13 | Each extraction has a cut




After extracting C, S=ABC and Q={D,E,F}. Its crossing edges are BD 2,CD 2,CE 3,BF 5. D has key 2 and parent B. Since CD=2 is not a strict improvement, B stays its parent.

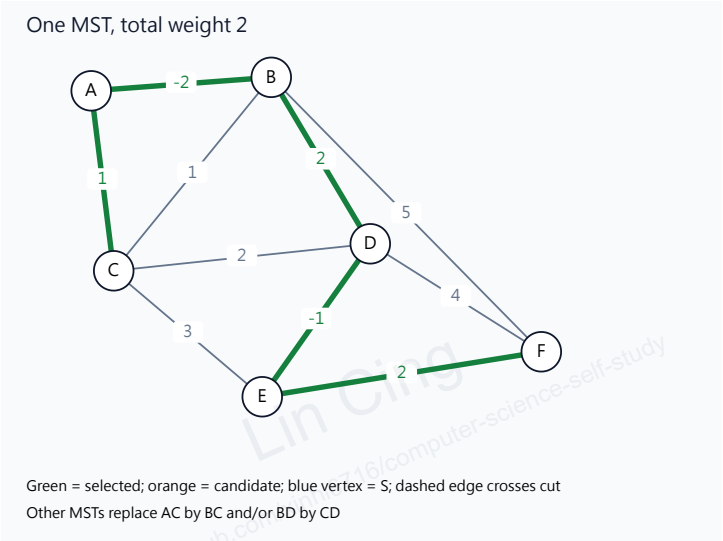


# A08-MST-14 | Negative weights are valid




D is extracted at key 2. Scanning D then decreases E to −1. The extracted key sequence 0,−2,1,2,−1,2 is not monotone.

## A08-MST-15 | Shared result, four optima




T={AB,AC,BD,DE,EF}, with weight −2+1+2−1+2=2. Independent enumeration of all C(10,5)=252 five-edge subsets, followed by the tree test, finds minimum weight 2 and exactly four MSTs.



## A08-MST-15A | Cuts, negative weights and optima

The other cuts before extraction have S={A},AB,ABCD,ABCDE. Their light candidates are AB −2, AC/BC 1, DE −1, EF 2 respectively. Root A contributes no parent edge.

The exchange proof compares edge weights and never requires nonnegative weights. Prim does not relax key[u]+w(u,v), unlike a shortest-path distance update.

All optima are {AB,DE,EF}∪{AC or BC}∪{BD or CD}. Distinct weights suffice for uniqueness. Repeated weights do not by themselves prove non-uniqueness.

## A08-MST-16 | Disconnected input yields a forest

If G has c components, no spanning tree covers all vertices. An MSF combines an MST of each component and has |V|−c edges.

Keep only {AB −2,AC 1,BC 1,DE −1,DF 4,EF 2} from our graph. Components are ABC and DEF. One MSF is {AB,AC,DE,EF}, weight 0. Replacing AC with BC gives another.

Kruskal yields an MSF by scanning all edges. In Prim, an extracted minimum key of ∞ starts a new root with parent=∅. Add no artificial infinite-weight edge, then scan its neighbors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A08-MST-17 | Name the implementation with the bound

Write n=|V| and m=|E|. Kruskal uses O(m log m) comparison sorting and O((n+m)(1+α(n))) DSU time from original chapter 21. For a simple connected graph, O(m log n) is the combined bound.

| Prim implementation | Extract minimum | Decrease key | Total time |
|---|---|---|---|
| Array scan + adjacency matrix | O(n) | O(1) | $O(n^2)$ |
| Indexed binary heap + adjacency lists | O(log n) | O(log n) | O((n+m) log n) |
| Fibonacci heap + adjacency lists | O(log n) amortized | O(1) amortized | O(m+n log n) |

An indexed heap maintains vertex positions and O(1) Q membership. A lazy heap pushes a new record on each improvement and discards stale records at pop. It can hold O(m) records, giving O(n+m log(m+1)) time and O(n+m) auxiliary space; its heap need not have only n entries.

Sources [CLRS23 pp633,636; MIT04 L13.40–50; MIT05 L16.40–50].

## A08-MST-18 | Cross-check proof, state and result

For acceptance, identify a cut respecting A and a light edge. For rejection, exhibit an already selected path. Read Prim key/parent/Q after every round and Kruskal components after every round. Ties choose between optima; negative weights do not invalidate safety.

Exercises A08-M01–08 check cuts, exchanges, cycles, DSU, PQ, MSF, uniqueness and implementation costs. `check.py` independently enumerates spanning trees and verifies both complete traces.

Original handwritten PDF pp36–37 motivates exchanges and a counterexample for the third-lightest edge. CLRS23 supplies the precise respecting-cut and tie conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Three meanings of a distance

For a finite directed graph with finite real edge weights, define $\delta(s,v)$ as the infimum of costs of all s-to-v walks. It is $\infty$ when v is unreachable, $-\infty$ when a reachable negative cycle can be visited on the way to v, and otherwise a finite minimum. The empty walk gives $\delta(s,s) \leq 0$.

We track d, $\pi$, and the queue on fixed graphs to identify when an algorithm can promise a final answer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Negative edges versus negative cycles

If v is reachable and no s-to-v walk can visit a negative cycle, delete positive cycles to improve its cost and zero cycles to preserve it. A simple shortest path with at most $|V|-1$ edges therefore exists.

For reachable v, the distance is finite exactly when there is no negative cycle reachable from s that can reach v. A negative cycle on another branch need not affect v. For example, $s \to a=1$, $a \to a=1$, $a \to v=1$ gives $\delta(s,v)=2$ despite the positive cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The shared relaxation operation

```
initialize: d[s]=0; d[v≠s]=∞; π[v]=NIL
RELAX(u,v,w):
  if d[u] ≠ ∞ and d[v] > d[u]+w:
    d[v]=d[u]+w; π[v]=u
```

Represent ∞ separately. For finite w, $\infty+w=\infty$. Check reachability before adding fixed-width numbers and prevent overflow. Never add to a maximum-integer sentinel or clamp negative values to an arbitrary floor. Update π only on strict improvement; store edge IDs for parallel arcs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Relaxation invariants and convergence

Every finite d[v] is the cost of an actual s-to-v walk, so $d[v] \geq \delta(s,v)$. Labels never increase and unreachable vertices stay at ∞.

If $s \rightsquigarrow u \to v$ is a shortest path and its prefix has converged, relaxing (u,v) makes v converge too. Relax the edges of a shortest path in path order to propagate equality, even with other relaxations interleaved. Once a finite optimum is attained, the label stays there.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

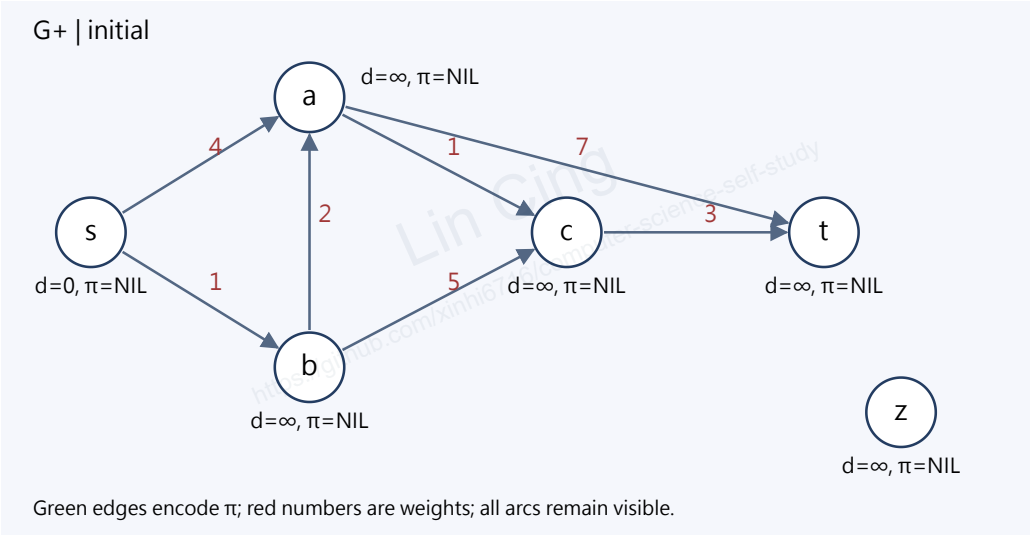


## Fixed graph G+ and initialization

Use vertex order (s,a,b,c,t,z); z is isolated. The complete edge list is

s→a:4, s→b:1, b→a:2, a→c:1, b→c:5, c→t:3, a→t:7.

Initially d=(0,∞,∞,∞,∞,∞) and every π is NIL. These graphs and numerical traces are original teaching examples.



## A DAG can have negative edges

Graph G− changes only a→c to −2. It remains a DAG. Fix topological order s,b,a,c,t,z. Before processing u, all predecessors are finished, so d[u] is final; no minimum-key choice is required.

Topological sorting and scanning all arcs take Θ(V+E). Validate acyclicity first. An arbitrary ordering of a cyclic graph cannot serve as a topological order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The DAG’ s single propagation pass

Each row follows relaxation of all outgoing arcs. Both vectors use (s,a,b,c,t,z).

| u | d | π |
|---|---|---|
| initial | (0,∞,∞,∞,∞,∞) | (NIL,NIL,NIL,NIL,NIL,NIL) |
| s | (0,4,1,∞,∞,∞) | (NIL,s,s,NIL,NIL,NIL) |
| b | (0,3,1,6,∞,∞) | (NIL,b,s,b,NIL,NIL) |
| a | (0,3,1,1,10,∞) | (NIL,b,s,a,a,NIL) |
| c | (0,3,1,1,4,∞) | (NIL,b,s,a,c,NIL) |
| t,z | (0,3,1,1,4,∞) | (NIL,b,s,a,c,NIL) |

Although a has distance 3 and c ultimately has 1, a must be processed first.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Bellman-Ford with a fixed edge order

Run on G− without using its topological order. Each pass scans exactly these seven arcs and writes labels in place.

c→t:3, a→t:7, a→c:−2, b→c:5, b→a:2, s→b:1, s→a:4.

Run |V|−1=5 passes, then check for any relaxable arc. A whole pass with no changes allows early stopping. Edge order changes intermediate states and convergence speed, but not the final answer when no reachable negative cycle exists.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Bellman-Ford’ s complete pass trace

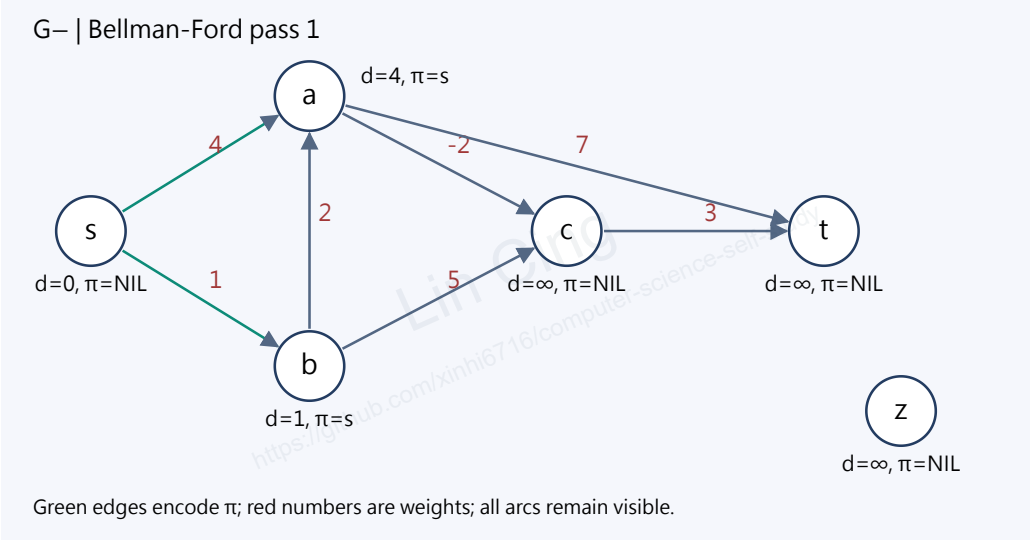
| pass | d(s,a,b,c,t,z) | π(s,a,b,c,t,z) |
|---|---|---|
| 0 | (0, ∞, ∞, ∞, ∞, ∞) | (NIL, NIL, NIL, NIL, NIL, NIL) |
| 1 | (0, 4, 1, ∞, ∞, ∞) | (NIL, s, s, NIL, NIL, NIL) |
| 2 | (0, 3, 1, 2, 11, ∞) | (NIL, b, s, a, a, NIL) |
| 3 | (0, 3, 1, 1, 5, ∞) | (NIL, b, s, a, c, NIL) |
| 4 | (0, 3, 1, 1, 4, ∞) | (NIL, b, s, a, c, NIL) |
| 5 | (0, 3, 1, 1, 4, ∞) | (NIL, b, s, a, c, NIL) |

Pass 1 reaches s’ s outgoing arcs only at its end. Pass 2 first uses a=4 to obtain c=2,t=11, then lowers a to 3. Pass 3 first lowers t to 5 and only later lowers c to 1. Pass 4 finally lowers t to 4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

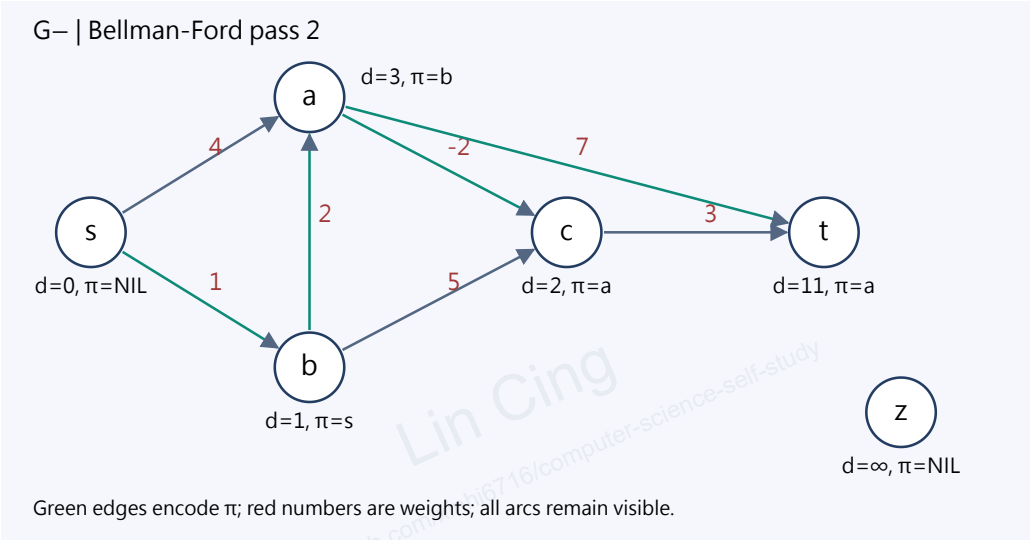
# Pass diagrams retain transient labels

These figures use the same G−, edge order, and sampling points as the table. After pass 2, π already describes s→b→a→c but d[c]=2 differs from that current parent-chain cost, 1. Parent chains and labels can be temporarily out of sync.

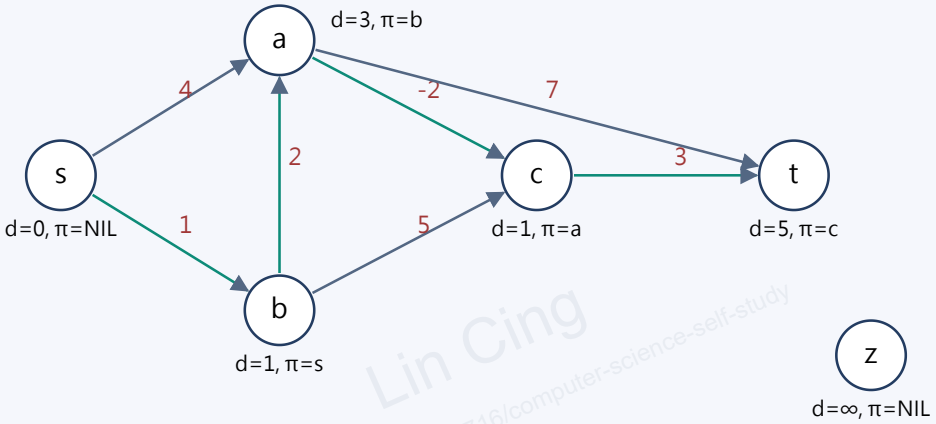





# Pass diagrams retain transient labels (continued)



## Pass diagrams retain transient labels (continued)




Green edges encode π; red numbers are weights; all arcs remain visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why V−1 passes suffice

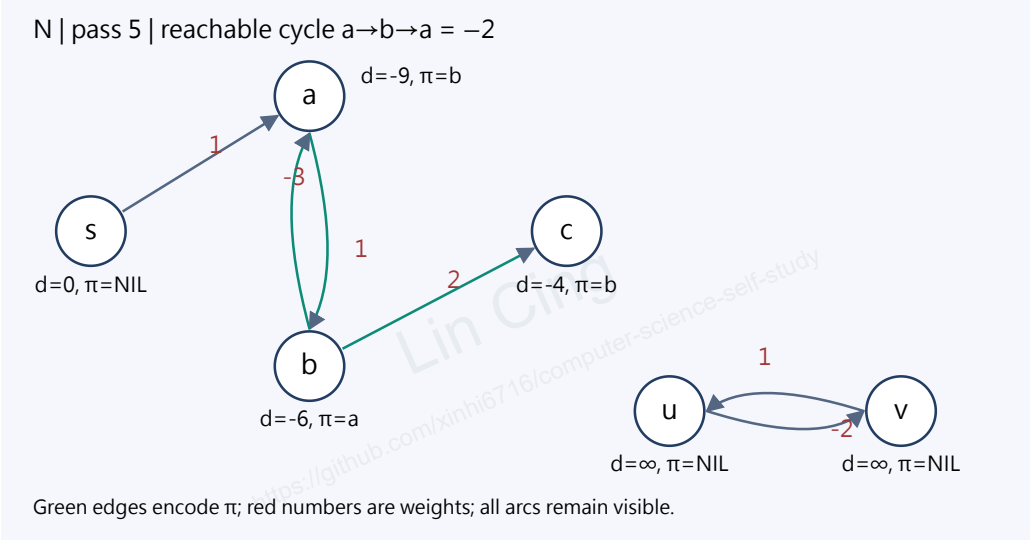
After complete pass k, every vertex with a shortest path of at most k arcs has converged. Pass k scans that path’ s kth arc, and its prefix converged by the previous pass.

An in-place pass can propagate across several arcs, so d is not necessarily the exact optimum restricted to k arcs. Without reachable negative cycles, a simple shortest path uses at most V−1 arcs. The time is O(VE); including initialization, O(V+VE) also covers E=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Reachable negative cycles and descendants

Graph N has exactly s→a:1,a→b:1,b→a:−3,b→c:2,u→v:−2,v→u:1. Vertices u,v are unreachable. Fix that edge order and vertex order (s,a,b,c,u,v).






## Detect the negative cycle and mark affected vertices

Passes 0 through 5 give d=(0,∞,∞,∞,∞,∞), (0,−1,2,4,∞,∞), (0,−3,0,2,∞,∞), (0,−5,−2,0,∞,∞), (0,−7,−4,−2,∞,∞), (0,−9,−6,−4,∞,∞).

A read-only sixth check finds a→b relaxable because −9+1<−6. Start DFS/BFS from every relaxable arc’ s destination and mark its descendants −∞. The affected set is {a,b,c}; s remains 0 and u,v stay ∞. Detection is justified by summing the supposedly nonrelaxable inequalities around a reachable negative cycle, contradicting its negative weight.

## Dijkstra’s contract

Every edge weight must be nonnegative. S contains settled vertices and Q=V−S. Extract a minimum-key u, add it to S, relax all outgoing arcs, and perform decrease-key when needed.

Our queue contains every vertex and breaks equal-key ties by vertex name. Tables display sorted logical queue contents, not heap array positions. Stop early if the minimum is ∞; all remaining vertices are unreachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Dijkstra’s full d, π, and queue trace

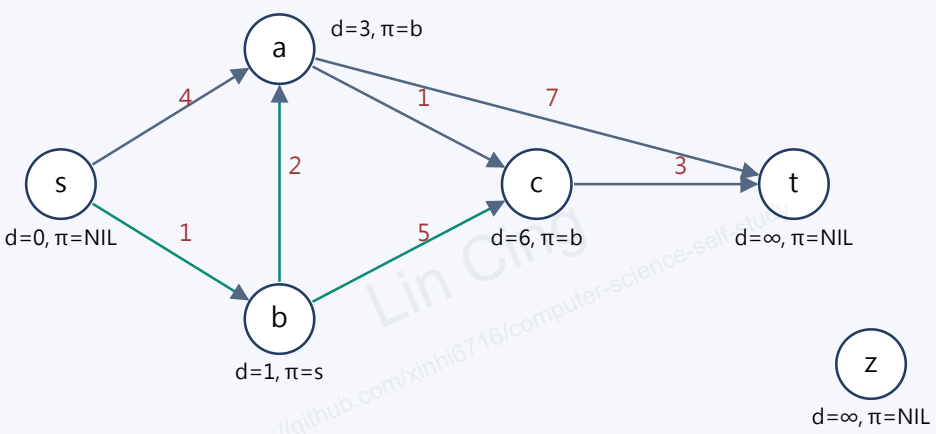
| extract | d(s,a,b,c,t,z) | π(s,a,b,c,t,z) | Q after relaxation |
|---|---|---|---|
| init | (0, ∞, ∞, ∞, ∞, ∞) | (NIL, NIL, NIL, NIL, NIL, NIL) | s:0, a:∞, b:∞, c:∞, t:∞, z:∞ |
| s | (0, 4, 1, ∞, ∞, ∞) | (NIL, s, s, NIL, NIL, NIL) | b:1, a:4, c:∞, t:∞, z:∞ |
| b | (0, 3, 1, 6, ∞, ∞) | (NIL, b, s, b, NIL, NIL) | a:3, c:6, t:∞, z:∞ |
| a | (0, 3, 1, 4, 10, ∞) | (NIL, b, s, a, a, NIL) | c:4, t:10, z:∞ |
| c | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | t:7, z:∞ |
| t | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | z:∞ |
| z | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | |

Each row samples the state after relaxing all outgoing arcs of the extracted vertex. Extracting s gives a=4,b=1; extracting b lowers a to 3 and first reaches c at 6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Two visible queue updates

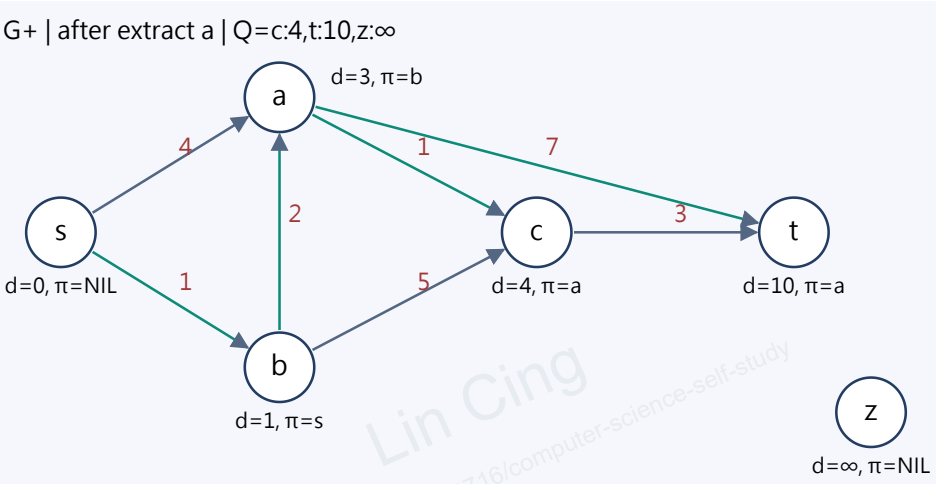
After extracting b, a=3 and c=6. After extracting a, c=4 and t=10. Thus π[c] changes from b to a and t first obtains parent a.




Green edges encode π; red numbers are weights; all arcs remain visible.



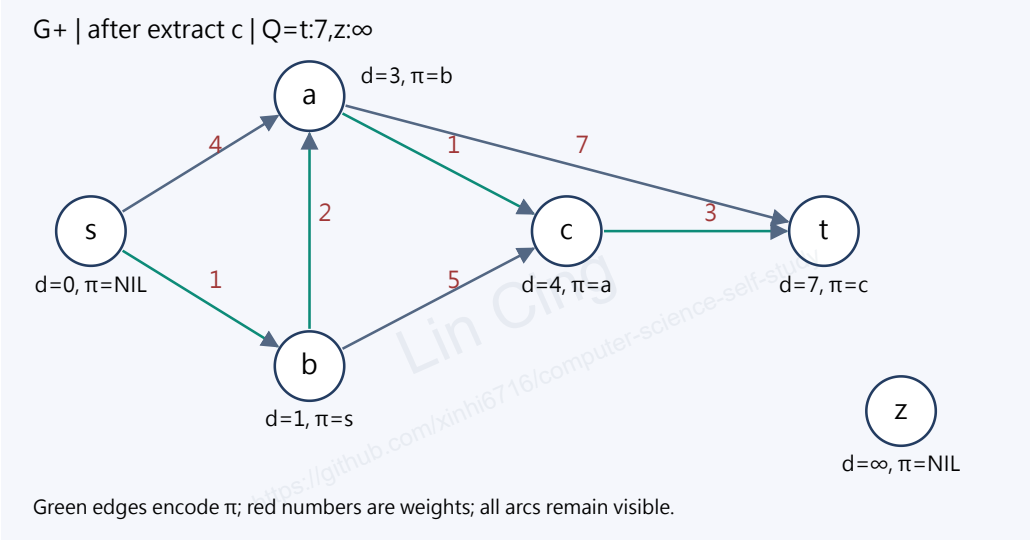
# Two visible queue updates (continued)




Green edges encode π; red numbers are weights; all arcs remain visible.

## The last improvement and unreachability

Extracting c lowers t from 10 to 7 and changes π[t] from a to c. Extracting t changes nothing. The remaining ∞ at z is the correct answer.




The shortest path s→b→a→c→t costs 1+2+1+3=7.



## Why extract-min settles a vertex

Invariant: d[x]=δ(s,x) for every x∈S. For the next reachable u, take a shortest s-to-u path. Let y be its first vertex outside S and x its predecessor. Relaxing x’ s outgoing arcs already established d[y]=δ(s,y).

The nonnegative y-to-u suffix gives δ(s,y)≤δ(s,u). Minimum extraction and the upper-bound property give δ(s,u)≤d[u]≤d[y]=δ(s,y)≤δ(s,u). Every inequality is equality. For unreachable u, use the no-path property.

## How a negative edge breaks settlement

Counterexample H has s→a:2,s→b:5,b→a:−4,a→t:2. Extraction order is s,a,t,b. Vertex a is settled at 2 although δ(s,a)=1; t is settled at 4 although its optimum is 3.

Ignoring updates to settled vertices leaves a wrong. Allowing a later update to a without rescanning its outgoing arcs still leaves t wrong. Reopening vertices requires a different proof and cost bound. The failed proof inequality is $\delta(s,b) \leq \delta(s,a)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Priority queue costs

There are |V| extract-min operations, at most |E| successful decrease-key operations, and one scan of each arc.

| Queue | Total time |
|---|---|
| Unsorted array | $O(V^2+E)$ |
| Binary heap | $O((V+E) \log V)$ |
| Fibonacci heap | $O(E+V \log V)$ amortized |

A lazy heap inserts a record on each improvement and skips stale extractions. Its bound is $O((V+E) \log(V+E))$ time and $O(V+E)$ space. Do not attribute the $O(V)$ heap space of decrease-key to this variant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Path reconstruction and algorithm choice

Reconstruct only confirmed finite distances. Follow π backward from the target to s, then reverse. Return unreachable for ∞ and no finite shortest path for −∞. A |V|-step guard reveals corrupted or unfinished parent chains.

In G−, t←c←a←b←s reverses to s→b→a→c→t of cost 1+2−2+3=4. Source s returns the empty path.

| Graph condition | Algorithm |
|---|---|
| DAG, negative arcs allowed | Topological relaxation |
| General graph, all arcs nonnegative | Dijkstra |
| General graph, negative arcs possible | Bellman-Ford and reachable-cycle check |

Sources are original notes p42, CLRS Chapter 24, MIT04 L14/L15, and MIT05 L17/L18. See sources.json for PDF and printed-page mappings.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



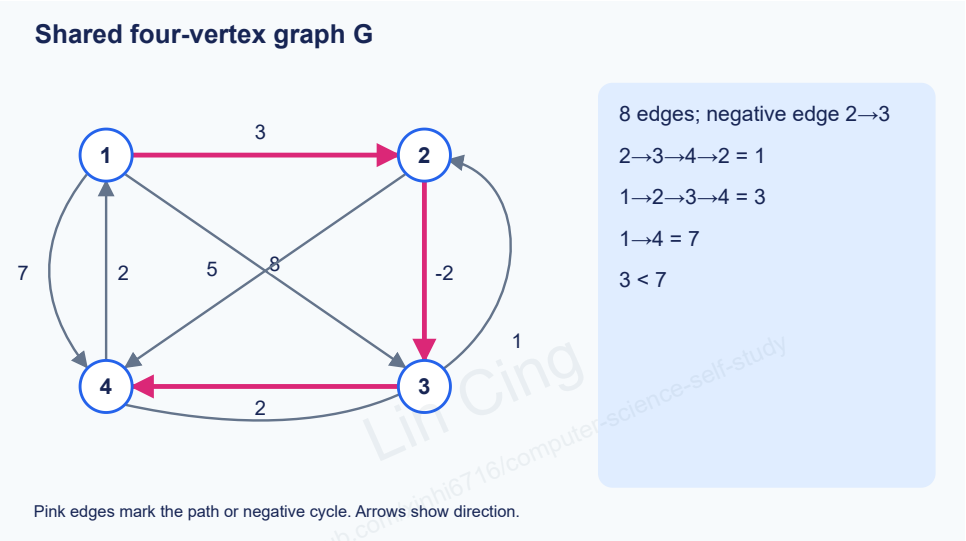
## APSP returns a distance and path for each ordered pair

Input is a directed graph G=(V,E), with n=|V| and m=|E|. Negative edges are allowed. Each row of the distance matrix fixes a source. Unreachable pairs have distance +∞. A pair that can traverse a negative cycle has distance −∞ and no finite shortest path.

First assume no negative cycle. W stores the minimum direct-edge weight, ∞ for absent edges, and min(0,self-loop weight) on the diagonal. Never erase a negative self-loop.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## APSP returns a distance and path for each ordered pair (continued)




This four-vertex example is original, not a textbook figure. Sources [S1 § 25.1–25.3], [S2 p4–23], and [S3 p4–23].



## The edge-budget state means at most t edges

$L^{(t)}[i,j]$ is the minimum weight of a walk with at most t edges. $L^{(0)}=I$, whose diagonal is 0 and other entries are $\infty$.

$$L^{(t+1)}[i,j] = \min_k \{L^{(t)}[i,k] + W[k,j]\}.$$

Split a nonempty walk before its final edge; each candidate also concatenates into a feasible walk. The diagonal W[j,j] is the minimum of 0 and any self-loop weight, so $W[j,j] \leq 0$. For a walk using at most t edges, append its chosen diagonal: the empty walk if it is 0, or a negative self-loop otherwise. The result uses at most t+1 edges and costs no more. Thus shorter walks are covered even with negative cycles; a fixed budget need not give unrestricted distances.

Without negative cycles, deleting a nonnegative cycle cannot raise weight. A shortest path can be simple, using at most n−1 edges. This justifies the stopping budget. [S1 p687–688; S2 p6–9; S3 p6–9]

## Min-plus multiplication adds edge budgets

Define $(A \otimes B)[i,j]=\min_k(A[i,k]+B[k,j])$. For $A=L^{(a)}$ and $B=L^{(b)}$, $A \otimes B=L^{(a+b)}$. Split any walk with at most a+b edges after min(a,its length) edges. Conversely, concatenate the two feasible pieces.

Associativity follows by minimizing the same expression.

$$(A \otimes B) \otimes C = A \otimes (B \otimes C), \qquad \min_{p,q}(A[i,p] + B[p,q] + C[q,j]).$$

I is the identity. Use finite numbers and $+\infty$ here, so $\infty+(-\infty)$ never arises. Replace sum and product in ordinary matrix multiplication by minimum and addition. Strassen subtraction does not transfer directly. [S1 p688–691; S2 p10–14; S3 p10–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Slow and faster change the number of stages

```
Extend(A,B):
  C = new n×n matrix filled with ∞
  for i,j,k: C[i,j] = min(C[i,j], A[i,k]+B[k,j])
  return C
Slow(W):
  L = I
  repeat n−1 times: L = Extend(L,W)
  return L
Faster(W):
  L = W; t = 1
  while t < n−1: L = Extend(L,L); t = 2*t
  return L
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Min-plus costs and negative-cycle detection

One Extend takes $\Theta(n^3)$. Slow performs $n-1$ products, taking $\Theta(n^4)$. Faster uses budgets 1,2,4,… and takes $\Theta(n^3 \log n)$. Retaining only the inputs and a fresh output uses $\Theta(n^2)$ working space. Do not overwrite an input while computing its product.

These answers assume no negative cycle. To detect one from a min-plus diagonal, use a budget of at least n. A negative simple cycle may contain n edges. The $n-1$ stopping rule applies to distances when no negative cycle exists. [S1 p689–692; S2 p13–14; S3 p13–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Compute $L^{(2)}$ and $L^{(3)}$ on the same graph

$L^{(1)}=W$. $L^{(2)}$ tries every possible final intermediate vertex.

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 7 |
| 2 | 7 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

$L^{(3)}$ already contains all distances because $n-1=3$.

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | 2 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

$L^{(2)}[1,4]=\min(7,3+5,8+2)=7$. $L^{(3)}[1,4]=1+2=3$ along $1\to2\to3\to4$. Faster computes $L^{(2)}$ and $L^{(4)}$; the latter equals $L^{(3)}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Floyd–Warshall limits the set of intermediate vertices

Every intermediate vertex of $D^{(k)}[i,j]$ belongs to $S_k=\{1,\ldots,k\}$. Endpoints i,j need not belong to $S_k$. $D^{(0)}=W$.

$$D^{(k)}[i,j] = \min\{D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}.$$

Without negative cycles, choose a simple shortest path. If it avoids k, use the first term. Otherwise split at its unique occurrence of k. Both subpaths have intermediates in $S_{k-1}$. Conversely, every candidate concatenates into an allowed walk, and removing nonnegative cycles remains feasible. Both directions prove equality.

The original notes motivate optimal substructure using shortest subpaths. The allowed set makes that observation into an executable DP. [S0 p18–19; S1 p693–695; S2 p15–17; S3 p15–17]

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## k=0 allows no intermediate vertex

$D^{(0)}$ contains only direct edges and empty paths. Π[i,j] stores the vertex immediately before j, not a split vertex. It is NIL for i=j or no path; a direct edge has predecessor i.

| $D^{(0)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 8 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | ∞ | 0 |

| $\Pi^{(0)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 1 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | NIL | NIL |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# k=1 allows intermediate vertex 1

Only 4→3 improves. Its old value ∞ becomes 2+8=10 along 4→1→3. $\Pi^{(1)}[4,3]=\Pi^{(0)}[1,3]=1$. Endpoint 3 need not belong to {1}.

| $D^{(1)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 8 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | 10 | 0 |

| $\Pi^{(1)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 1 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 1 | NIL |

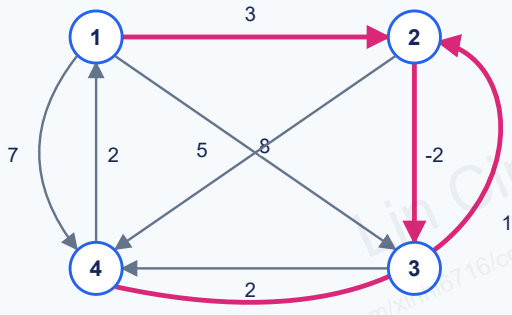
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# k=2 improves two pairs through the negative edge

1→3 falls from 8 to 1; 4→3 falls from 10 to −1. Both end with 2→3, so both predecessors become 2.

**Improvements through intermediate 2**




$S_2 = \{1, 2\}$

1→3: 8 → 3 + (−2) = 1

4→3: 10 → 1 + (−2) = −1

Endpoint 3 need not be in $S_2$

1→4 remains 7

Pink edges mark the path or negative cycle. Arrows show direction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=2 improves two pairs through the negative edge (continued)

| $D^{(2)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

| $\Pi^{(2)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

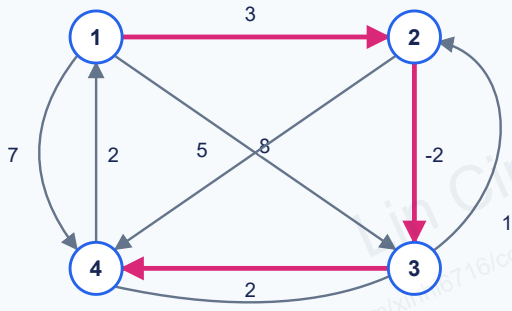
Lin Cing https://github.com/xinhi6716/computer-science-self-study



# k=3 first exposes the three-edge path

1→4 falls from 7 to 3; 2→4 falls from 5 to 0. Π[1,4]=Π[3,4]=3. Copying k itself is not a valid general predecessor rule.

**Improvements through intermediate 3**




$S_3 = \{1, 2, 3\}$

1→4: 7 → 1 + 2 = 3

2→4: 5 → −2 + 2 = 0

4→2→3→4 = 1

Nonnegative cycle keeps diagonal 0

Pink edges mark the path or negative cycle. Arrows show direction.

## k=3 first exposes the three-edge path (continued)

| $D^{(3)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | ∞ | 0 | -2 | 0 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

| $\Pi^{(3)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 3 |
| 2 | NIL | NIL | 2 | 3 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## k=4 finishes distances and reconstructs a path

New entries are 2→1=2, 3→1=4, and 3→2=3. From source 1, trace backward from 4 using Π[1,4]=3, Π[1,3]=2, and Π[1,2]=1. Reverse to obtain 1→2→3→4 of weight 3.

On strict improvement set Π[i,j]=Π[k,j]; retain the old predecessor on a tie. Check reachability and negative-cycle influence before reconstruction. Updating on ties can erase a valid predecessor through a zero-weight cycle. [S1 p695–697]

| $D^{(4)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | 2 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

| $\Pi^{(4)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 3 |
| 2 | 4 | NIL | 2 | 3 |
| 3 | 4 | 4 | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Keep k outermost to preserve the DP stage

```
D = W
for k = 1..n:
  for i = 1..n:
    for j = 1..n:
      if D[i,k]+D[k,j] < D[i,j]:
        D[i,j] = D[i,k]+D[k,j]
        Π[i,j] = Π[k,j]
```

After outer stage k, every pair has incorporated $S_k$. Without negative cycles, D[k,k]=0. Row k and column k do not change during this stage, so in-place operands retain the required previous-stage values. Swap i and j freely. Moving k inside has no such general guarantee.

Counterexample has only 1→4=1, 4→3=1, and 3→2=1. One increasing i,j,k pass examines (1,2) before D[1,3] becomes finite, leaving ∞. Later it finds D[1,3]=2 but never revisits (1,2). The true distance is 3. [S1 p695,699 ex25.2-4; S2 p17; S3 p17]



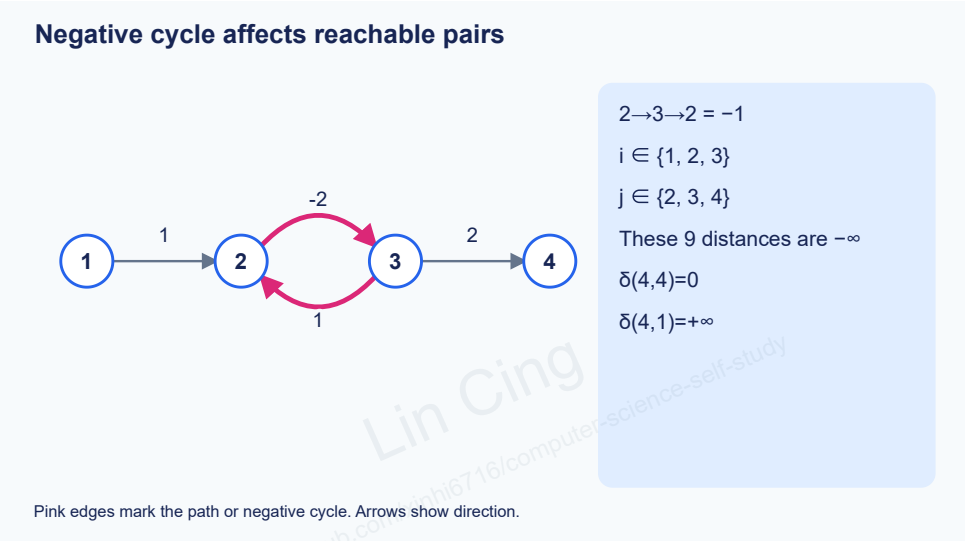
## A negative diagonal warns; mark −∞ by pair

After Floyd, some D[t,t]<0 if and only if a negative cycle exists. A negative diagonal gives a negative closed walk, which contains a negative cycle. Conversely, allowing all intermediates exposes a negative diagonal at some vertex on a negative cycle.

Use a reachability matrix R to identify affected pairs.

$$\delta(i,j) = -\infty \iff \exists t:\ R[i,t] \wedge D[t,t] < 0 \wedge R[t,j].$$

## A negative diagonal warns; mark −∞ by pair (continued)




This separate counterexample is not the shared graph G. Exactly {1,2,3}×{2,3,4}, nine pairs, are affected. Complete finite arithmetic before marking. With negative cycles, finite Floyd values are not unrestricted distances. Johnson instead reports a negative cycle and stops globally. [S1 ex25.2-6, § 25.3]



## Johnson obtains potentials from a super-source

Add s and a zero-weight edge s→v to every vertex v. No edge enters s, so it creates no cycle and cannot change paths between original vertices. All original vertices are reachable from s, so Bellman–Ford detects any negative cycle.

Only if no negative cycle exists, set h(v)=δ(s,v). On the shared graph h=(0,0,−2,0): s→2→3 gives −2; no other vertex drops below 0.

## Bellman–Ford stages establish the potentials

Synchronous Bellman–Ford stages use order (s,1,2,3,4).

| Edge budget | Distance vector |
|---|---|
| 0 | (0,∞,∞,∞,∞) |
| 1 | (0,0,0,0,0) |
| 2 | (0,0,0,−2,0) |
| 3 and 4 | (0,0,0,−2,0) |

After $|V'|-1=4$ stages, stage 5 has no improvement. With in-place edge relaxations, intermediate vectors depend on edge order, while final potentials agree. [S1 p702–704; S2 p22–23; S3 p22–23]

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## The shortest-path triangle inequality makes weights nonnegative

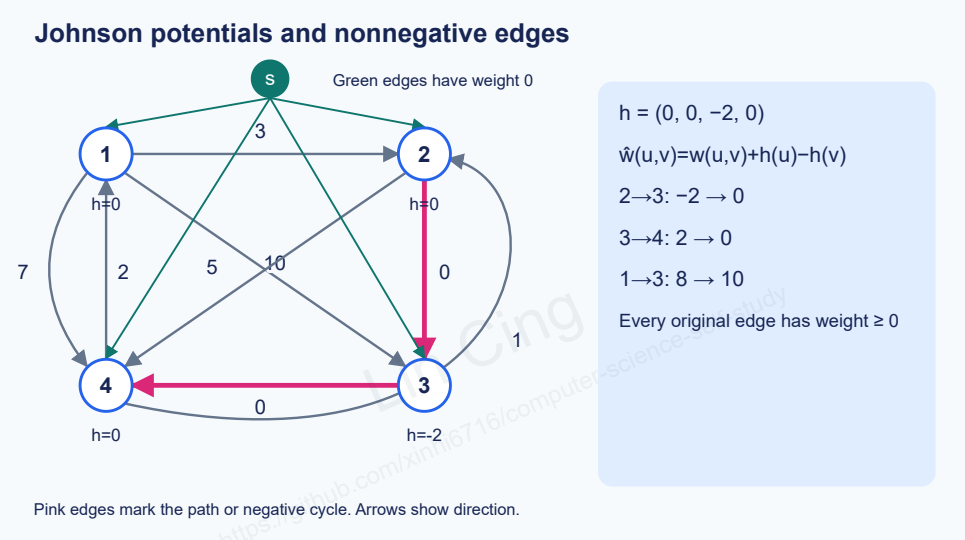
Bellman–Ford potentials satisfy $h(v) \le h(u)+w(u,v)$. Rearranging gives

$$\widehat{w}(u,v) = w(u,v) + h(u) - h(v) \ge 0.$$

| u→v | w | h(u)−h(v) | ŵ |
|---|---|---|---|
| 1→2 | 3 | 0 | 3 |
| 1→3 | 8 | 2 | 10 |
| 1→4 | 7 | 0 | 7 |
| 2→3 | -2 | 2 | 0 |
| 2→4 | 5 | 0 | 5 |
| 3→4 | 2 | -2 | 0 |
| 4→1 | 2 | 0 | 2 |
| 4→2 | 1 | 0 | 1 |

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# The shortest-path triangle inequality makes weights nonnegative (continued)




Any h preserves path order. Only h satisfying the difference constraints guarantees nonnegative edges. Adding the same constant to every edge is not a substitute, because paths can have different edge counts. [S1 lemma25.1, p702; S2 p19–23; S3 p19–23]



# Telescoping preserves the order of paths with fixed endpoints

For p=⟨v₀,…,vᵣ⟩,

$$\widehat{w}(p) = \sum_{a=1}^{r} (w(v_{a-1}, v_a) + h(v_{a-1}) - h(v_a)) = w(p) + h(v_0) - h(v_r).$$

Internal potentials cancel. Every path with the same endpoints receives the same constant, preserving strict order, ties, and the complete set of shortest paths. A cycle has equal endpoints, so its weight is unchanged. A negative cycle cannot become nonnegative.

For example, direct 1→3 has weight 8 and reweighted weight 10. Path 1→2→3 has weight 1 and reweighted weight 3. Both increase by 2. For 1→2→3→4 and direct 1→4, the endpoint potential difference is 0; weights remain 3 and 7.

Thus $\delta(u, v) = \hat{\delta}(u, v) - h(u) + h(v)$. Check reachability first. Keep ∞ unchanged rather than applying signed integer arithmetic to a sentinel. [S1 lemma25.1; S2 p19–21; S3 p19–21]

## Run Dijkstra from every source, then restore distances

From source 1, vertices settle in order 1,2,3,4. After extracting 1, estimates are (0,3,10,7). After 2, they are (0,3,3,7). After 3, they are (0,3,3,3).

All sources produce these reweighted distances.

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 3 | 3 |
| 2 | 2 | 0 | 0 | 0 |
| 3 | 2 | 1 | 0 | 0 |
| 4 | 2 | 1 | 1 | 0 |

Apply $\delta = \tilde{\delta} - h(\text{source}) + h(\text{destination})$ to each cell to recover $D^{(4)}$. For example, $\delta(1,3) = 3 - 0 - 2 = 1$ and $\delta(3,1) = 2 - (-2) + 0 = 4$. Keep the Dijkstra predecessor paths because reweighting preserves paths.

The checker also enumerates simple paths to verify all 16 entries. Agreement between two DPs is not its only evidence.



## Density and heap choice determine cost

| Method | Time | Conditions |
|---|---|---|
| Slow min-plus | $\Theta(n^4)$ | Edge-budget DP for derivation |
| Repeated squaring | $\Theta(n^3 \log n)$ | Min-plus associativity |
| Floyd–Warshall | $\Theta(n^3)$ | Matrix input and dense graphs |
| Johnson, binary heap | $O(n(m+n)\log n)$ | Sparse adjacency lists |
| Johnson, Fibonacci heap | $O(nm+n^2 \log n)$ | Amortized $O(1)$ decrease-key |

## Compare total cost and working space

With an explicit super-source, Bellman–Ford costs $O(n(m+n))$. Add n Dijkstra runs and $\Theta(n^2)$ restoration. With Fibonacci heaps, $m=\Theta(n^2)$ gives $O(n^3)$; $m=O(n)$ gives $O(n^2 \log n)$. The binary-heap dense upper bound is $O(n^3 \log n)$. A full output already needs $\Theta(n^2)$ space.

Floyd uses $\Theta(n^2)$ working space; retaining every teaching stage uses $\Theta(n^3)$. Johnson uses $O(m+n)$ for adjacency lists and one source beyond the output.

Source keys: S0=original notes p18–19; S1=CLRS 3e § 25.1–25.3, printed p684–705; S2=MIT 2004 lecture16 p1–23; S3=MIT 2005 lecture19 p1–23. See sources.json for paths, read pages, and content mappings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapters 23–25. MIT 6.046J, Fall 2004, Lectures 13–16; Fall 2005, Lectures 16–19. Original handwritten algorithms notes (演算法_wjungle). These notes provide selected proof ideas and context; complete worked traces and diagrams are original educational synthesis.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Network flows

Capacity and flow conservation describe how a network sends resources from a source to a sink. A residual network records both additional capacity and opportunities to cancel earlier decisions.

Study **augmenting paths, max-flow min-cut and Edmonds–Karp → push–relabel → matching and lower-bound models**.

Prerequisites are graphs, breadth-first search, invariants, and asymptotic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# From flow to proof and modeling

| Question | Conditions to check |
| --- | --- |
| Is a flow feasible? | Capacity constraints and conservation at intermediate vertices. |
| How can we improve it or prove optimality? | Residual paths, cut capacity, and max-flow min-cut. |
| How do local operations converge? | Preflows, heights, and push–relabel invariants. |
| How do we express new constraints? | Matching, vertex capacities, and flows with lower bounds. |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A01 | Maximum flow needs an optimality certificate

Given a directed network G=(V,E), source s, sink t, and finite nonnegative capacities c, maximize the net flow leaving s.

We rebuild the six-vertex example in [NOTE] and finish with a feasible flow of value 11 and a cut of capacity 11. Sources are [CLRS26] § § 26.1–26.2, [MIT04-20], [MIT04-21], and [NOTE] physical pages 38–40.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A02 | Capacity and conservation are separate constraints

Each original edge obeys $0 \le f(e) \le c(e)$. Every v outside {s,t} obeys Σincoming f=Σoutgoing f.

Define $|f| = \Sigma_{(s,v)} f(s,v) - \Sigma_{(v,s)} f(v,s)$. Incoming edges at s must be subtracted. Summing conservation over all internal vertices shows that net source outflow equals net sink inflow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A03 | Choose one model for antiparallel edges

Here f is nonnegative flow on each original edge. MIT04 instead uses skew-symmetric net flow. Our example has no antiparallel original edges.

If u→v and v→u both exist, give each original edge its own paired residual arcs and edge_id. Direction u→v can contain both c(u,v)−f(u,v) of forward capacity and f(v,u) of cancellation capacity. Aggregation is possible only with a defined update allocation. Alternatively subdivide one original edge as u→q→v.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A04 | Residual arcs describe allowed changes

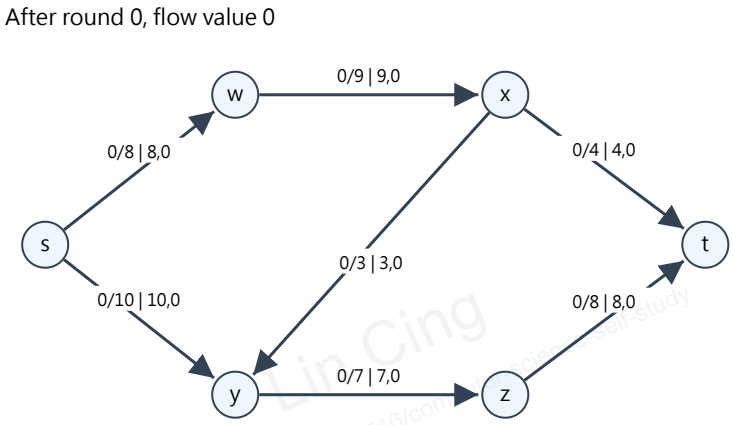
An original u→v edge contributes forward residual c−f and reverse residual f. G_f contains only positive residual arcs.

Sending δ along residual v→u reduces original f(u,v) byδ. It requires no physical reverse transport edge. A missing original v→u edge does not forbid cancellation. Preserve paired reverse arcs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A05 | The six-vertex network starts at zero flow

Capacities are sw=8, sy=10, wx=9, xy=3, yz=7, xt=4, zt=8. Initially every f is 0.




Each label is f/c | forward residual,reverse residual. Arrows show original edges; the final number states the available reverse residual.



## A09-A06 | Augmentation preserves feasibility

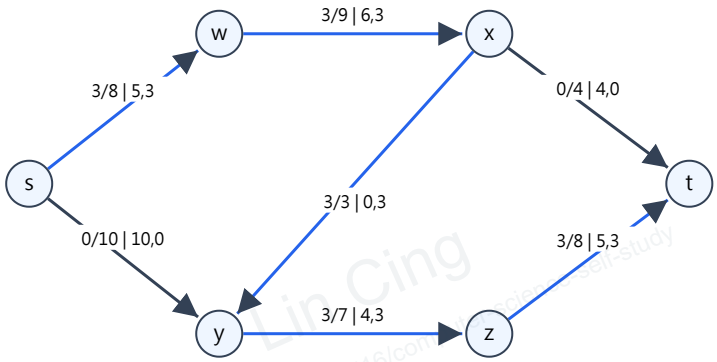
Choose a simple s→t path P in G_f and δ=min_{e∈P}r(e)>0. Addδ on original forward edges and subtractδ on reversed original edges.

The residual bound ensures capacity feasibility. Each internal path vertex receives one incoming and one outgoing flow change, so conservation remains true. Source net outflow and sink net inflow increase byδ. Thus the new value is |f|+δ, and at least one residual arc is saturated.

# A09-A07 | Round 1 deliberately chooses a long path

Choose $P_1$=s→w→x→y→z→t. Residuals are 8,9,3,7,8, so $\delta_1$=3 and the flow value is 3.

After round 1, flow value 3




Edge label f/c | forward residual,reverse residual. Blue means positive flow.

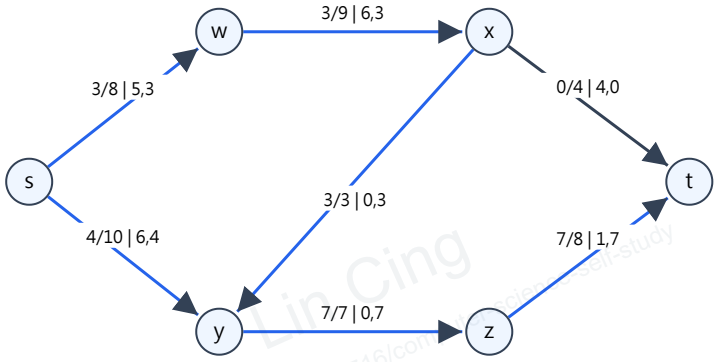
Edge xy is saturated and creates reverse residual y→x of 3. This is a valid Ford–Fulkerson choice, not an Edmonds–Karp choice.



# A09-A08 | Round 2 fills yz

Choose $P_2$=s→y→z→t. Residuals are 10,4,5, so $\delta_2$=4 and the value becomes 7.

After round 2, flow value 7

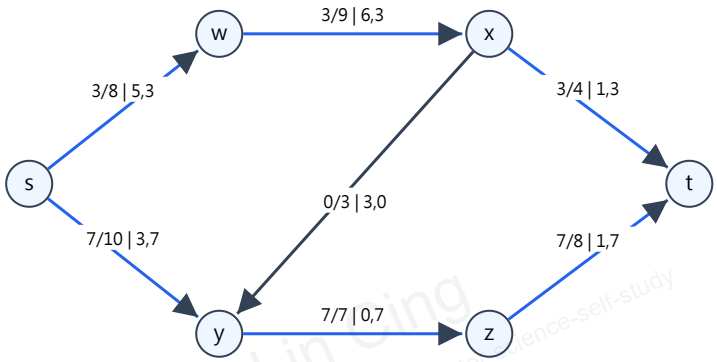



Edge label f/c | forward residual,reverse residual. Blue means positive flow.

At x, inflow 3 equals outflow 3. At y, inflow 3+4 equals outflow 7. Searching only unused original forward capacity would miss the next cancellation.

# A09-A09 | Round 3 cancels an earlier decision

After round 3, flow value 10




Edge label f/c | forward residual,reverse residual. Blue means positive flow.

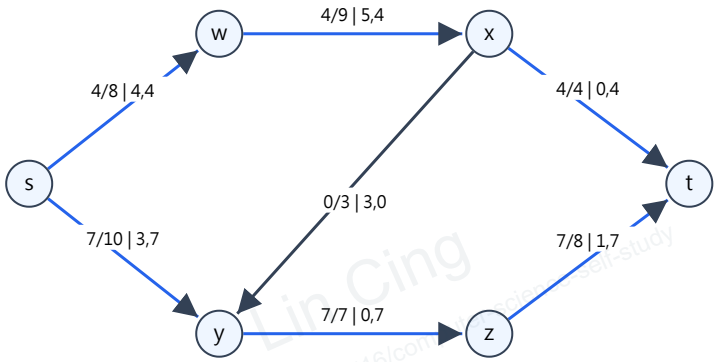
Choose $P_3$=s→y→x→t. Residuals are 6,3,4, so $\delta_3$=3. The arc y→x reverses xy and reduces f(xy) from 3 to 0. Value becomes 10. Now x has inflow 3=outflow 3 and y has inflow 7=outflow 7. Total value increases even though one original edge loses flow.



# A09-A10 | Round 4 and final conservation

Choose $P_4$=s→w→x→t. Residuals are 5,6,1, so $\delta_4$=1. The final value is 11.

After round 4, flow value 11




Edge label f/c | forward residual,reverse residual. Blue means positive flow.

At w,4 enters and 4 leaves; at x,4 enters and 4 leaves; at y and z,7 enters and 7 leaves. Source and sink totals are both 4+7=11. Final edge flows in order sw,sy,wx,xy,yz,xt,zt are (4,7,4,0,7,4,7).

## A09-A11 | Audit every round

| Round | sw | sy | wx | xy | yz | xt | zt | Value |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 3 | 0 | 3 | 3 | 3 | 0 | 3 | 3 |
| 2 | 3 | 4 | 3 | 3 | 7 | 0 | 7 | 7 |
| 3 | 3 | 7 | 3 | 0 | 7 | 3 | 7 | 10 |
| 4 | 4 | 7 | 4 | 0 | 7 | 4 | 7 | 11 |

For each row verify w:sw=wx; x:wx=xy+xt; y:sy+xy=yz; z:yz=zt. check.py audits all rounds and all 16 s–t cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A12 | Cut capacity differs from net cut flow

A cut (S,T) has $s \in S$, $t \in T$, $T = V \setminus S$.

$c(S,T) = \Sigma_{S \to T} c$ counts only original outgoing capacities. Net cut flow is $f(S,T) = \Sigma_{S \to T} f - \Sigma_{T \to S} f$.

Sum outgoing minus incoming flow over vertices in S. Internal edges cancel and conservation removes internal vertices, leaving $|f| = f(S,T)$. Capacity and nonnegativity then give $|f| \leq c(S,T)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A13 | Prove both directions of max-flow min-cut

If $|f|=c(S,T)$ for a cut, every feasible g has $|g| \leq c(S,T)=|f|$. Thus f is maximum and that cut is minimum.

Conversely, maximum f has no augmenting path, since augmentation would increase its value. Let S be the vertices reachable from s in $G_f$. Then t is outside S. Every original $S \to T$ edge is saturated, or reachability would cross the cut. Every original $T \to S$ edge carries zero flow, or its reverse residual would cross the cut. Hence $|f|=f(S,T)=c(S,T)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A14 | The stopping certificate in the example

The final residual reachable set is S={s,w,x,y}, with T={z,t}. Forward residual on sw,sy,wx,xy permits reachability within S, but yz and xt have zero forward residual.

The cut capacity is $c(yz)+c(xt)=7+4=11$. No positive flow returns from T to S. A feasible value 11 and an upper bound 11 prove maximum flow=minimum cut=11. The spare unit on zt alone cannot carry additional source flow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A15 | Integral Ford–Fulkerson terminates

```
f[e] = 0 for each original edge e
while find_path(G_f, s, t) returns P:
    delta = min(residual[arc] for arc in P)
    update each paired original edge along P by +delta or -delta
return f, reachable(G_f, s)
```

Initialize zero flow and repeatedly augment by a path bottleneck. Integer capacities keep all residuals and $\delta$ integral, so $\delta \geq 1$. A finite cut bounds the flow value, giving at most $F^* = |f^*|$ augmentations.

With O(E) search and update, time is $O(E(1+F^*))$, commonly $O(EF^*)$, plus O(V+E) initialization. This is pseudopolynomial because the numeric $F^*$ may be exponential in the number of capacity input bits.



## A09-A16 | A constant-size network can need 2 U rounds

Give su,ut,sv,vt capacity U and uv capacity 1. Alternate s→u→v→t and s→v→u→t. Each $\delta$ is 1, with the second path canceling uv. Reaching maximum 2 U takes 2 U augmentations.

Rational capacities can be scaled by a common denominator, ensuring termination without a polynomial bit-length bound. With arbitrary real capacities and arbitrary path choices, Ford–Fulkerson may not terminate. Strict increase alone proves neither finite termination nor an optimal limit.

## A09-A17 | Edmonds–Karp chooses a BFS path

Each iteration uses BFS on positive residual arcs to find an s→t path with the fewest edges. It does not maximize path capacity or bottleneck.

On this example choose s→w→x→t with δ=4, then s→y→z→t with δ=7. Two augmentations give 11. These choices differ from the earlier four-round Ford–Fulkerson trace. A general implementation still needs paired reverse arcs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A18 | Why BFS distances never decrease

Let d,d′ be source hop distances before and after an augmentation, using∞ for unreachable vertices. Assume a decrease. Along a new shortest path choose the first decreased vertex v and predecessor u. Thus d(u)≤d′(u)=d′(v)−1.

If (u,v) existed before, d(v)≤d(u)+1≤d′(v), a contradiction. Otherwise it was created as the reverse of (v,u) on the old shortest augmenting path. Then d(u)=d(v)+1, so d(v)=d(u)−1≤d′(v)−2, again a contradiction. All distances are nondecreasing; an unreachable vertex stays unreachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A09-A19 | A critical arc must rise two levels before recurring

Each augmentation saturates a critical residual arc $(u,v)$ with $r(u,v)=\delta$ and $d(v)=d(u)+1$. The arc disappears.

Before it can be critical again, an augmentation must traverse $(v,u)$ to restore capacity. At that time $d_{new}(u)=d_{new}(v)+1\geq d_{old}(v)+1=d_{old}(u)+2$. Monotonicity preserves this increase. Finite distances on augmenting paths are at most $V-1$, so each oriented edge_id has only $O(V)$ critical events.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A09-A20 | Count events to obtain the time bound

There are at most $2E$ residual arc directions. Every augmentation has a critical event, and each direction has $O(V)$ events. There are $O(VE)$ augmentations.

BFS and path updates cost $O(V+E)$ per round. Remove isolated vertices or use the relevant reachable subgraph so relevant $V=O(E)$; then each round is $O(E)$, giving $O(VE^2)$, plus $O(V+E)$ initialization. This bound does not depend on capacity magnitude and also holds for exact real arithmetic; bit-operation costs are separate.

Return both a feasible flow and its residual reachable cut. These certify optimality independently of the path history.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P01 | Store excess, then restore conservation

Ford–Fulkerson keeps a feasible flow after each augmentation. Push-relabel allows intermediate vertices to store excess. A push examines one residual edge; a relabel changes one vertex height.

The algorithm and proofs follow CLRS 3e § § 26.4–26.5. The original note scan p38 supplies residual-capacity and cancellation prerequisites, rather than the push-relabel algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P02 | Preflow and excess

For each original edge maintain $0 \le f(u,v) \le c(u,v)$. Define $e(u)=\Sigma_v f(v,u)-\Sigma_v f(u,v)$. A preflow requires $e(u) \ge 0$ for every $u \ne s$. A vertex is active exactly when $u \notin \{s,t\}$ and $e(u)>0$.

Forward residual capacity is $c-f$; reverse residual capacity is $f$. This example has no antiparallel original edges. A reverse push decreases the corresponding original flow. The source may have negative excess. The sink may have positive excess but is never discharged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P03 | Height is a certificate, not an exact distance

A valid height function satisfies $h(s)=n$, $h(t)=0$, and $h(u)\leq h(v)+1$ on every residual edge $(u,v)$. Heights need not equal current distances to $t$.

A simple residual $s$-to-$t$ path would have $k\leq n-1$ edges. Summing the inequalities gives $n=h(s)\leq h(t)+k=k$, a contradiction. Thus a valid height function always rules out such a path. A preflow may still violate conservation, so this alone does not establish completion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P04 | Initialize by saturating source edges

Set all flows, excesses, and heights to zero; set $h(s)=n$. Saturate each original edge $(s,v)$ and update $e(s)$ and $e(v)$. This is initialization, rather than an ordinary PUSH subject to the height-difference test.

The outgoing source edges are absent from the residual network. Their reverse edges $v\to s$ satisfy $0\leq n+1$. All other residual endpoints have height zero. Both the preflow and height invariants hold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P05 | Push conditions and updates

PUSH(u,v) requires an active u, positive residual capacity, and h(u)=h(v)+1. Set δ=min(e(u),c_f(u,v)). Increase original flow or cancel reverse flow, then subtract δ from e(u) and add δ to e(v).

The bounds on δ preserve capacity and nonnegative excess outside s. A newly created reverse edge v→u satisfies h(v)=h(u)−1≤h(u)+1. A saturating push removes the residual edge. A nonsaturating push empties u. If both occur, classify the push as saturating.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P06 | Relabel only when no admissible edge leaves u

RELABEL(u) requires active u and h(u)≤h(v) for all residual neighbors v, equivalently no admissible outgoing edge. Set h(u)=1+min{h(v):c_f(u,v)>0}.

Positive excess implies a positive incoming flow and hence an outgoing reverse residual edge, so the minimum is over a nonempty set. The height strictly increases. Outgoing constraints hold by construction; increasing the target height only relaxes incoming constraints.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P07 | Correctness conditional on termination

An active vertex can push if it has an admissible edge and can relabel otherwise. At termination every intermediate excess is zero, so the preflow becomes a flow. The height invariant excludes a residual s-to-t path, which certifies maximum flow.

This proves correctness if the algorithm terminates. Operation bounds below establish termination separately. Neither argument requires integral capacities or bounds iterations by the flow value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P08 | Height bounds and relabel work

Every active u has a residual path to s. Otherwise take its residual reachable set S excluding s. With no residual edge leaving S, original incoming flow across the cut is zero. Thus $\Sigma_{v \in S} e(v) \leq 0$, contradicting positive excess in S.

A simple path back to s has at most n−1 edges, so $h(u) \leq h(s)+n-1=2n-1$. Heights only increase, and only active vertices relabel. Each vertex relabels O(n) times, for $O(n^2)$ total. A relabel scans N(u) in O(deg(u)); all relabel work is $\Sigma_u O(n \deg(u))=O(nm)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P09 | O(nm) saturating pushes

After saturating u→v, another push v→u must restore residual capacity before the next saturation in the same direction. Initially h(u)=h(v)+1. At the reverse push, h′(v)=h′(u)+1≥h(u)+1.

At the next forward push, h″(u)=h″(v)+1≥h(u)+2. Repeated saturation in the same direction forces a height increase of at least two. Each direction saturates O(n) times. There are at most 2m residual directions, giving O(nm) saturating pushes. Each push costs O(1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P10 | Potential pays for nonsaturating pushes

Let Φ=Σ_{u active}h(u). Initially Φ=0 and always Φ≥0. A nonsaturating push removes u and may add v. Since h(u)=h(v)+1, Φ decreases by at least one. Neither s nor t enters the sum.

Each relabel or saturating push increases Φ by O(n). There are $O(n^2)$ relabels and O(nm) saturating pushes, so nonsaturating pushes number $O(n^3+n^2m)$. Removing isolated components irrelevant to s and t gives n=O(m), hence $O(n^2m)$. The bound does not depend on capacity magnitude.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P11 | Operation counts require an implementation

An operation count is not yet a running-time proof. Paired residual arcs support O(1) pushes. Stored excesses and an active container support O(1) active-vertex selection.

A persistent current arc scans N(u); only after reaching the end does u relabel and reset its pointer. Each vertex makes O(n) passes, for O(nm) scans and relabel work. Choosing any active vertex and discharging it is a legal generic schedule. Adding push work gives $O(n^2m)$, with O(n+m) initialization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P12 | Relabel-to-front keeps every intermediate vertex

L is a linked list of all vertices in V−{s,t}, including vertices with zero excess. N(u) lists neighbors of both incoming and outgoing original edges in a fixed order. Entries remain even when their residual capacity is zero. Each current pointer persists across calls.

DISCHARGE(u) repeats until e(u)=0. At current=NIL, relabel and reset to the first neighbor. If current is admissible, push without advancing. Otherwise advance current to the next neighbor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P13 | Move after discharge, then use the new successor

Save old=h(u). After DISCHARGE(u), move u to the front only if h(u)>old. The next vertex is u.next in the modified list. Otherwise advance in the existing order. Zero-excess vertices stay in L.

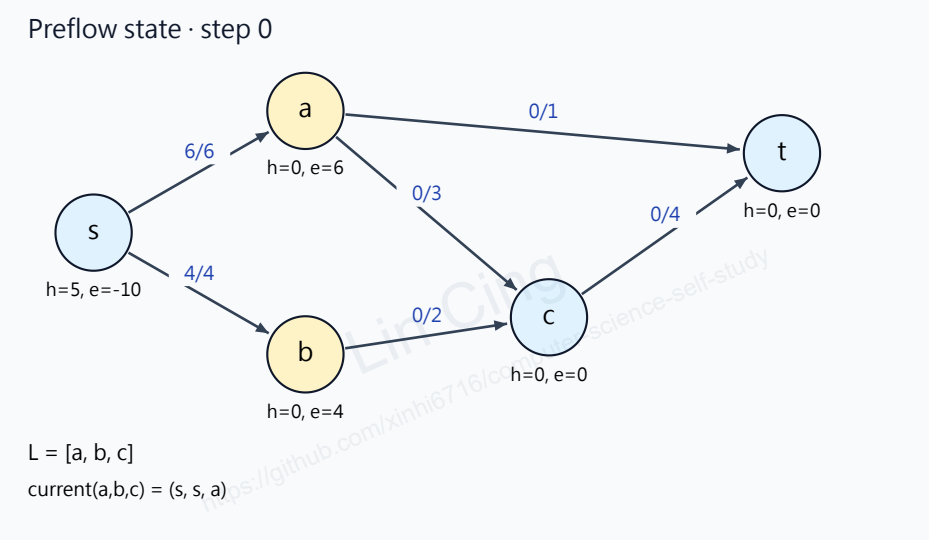
Why may current persist? Push creates no admissible edge. Relabeling another vertex creates no admissible edge into it. Therefore an outgoing edge skipped during a pass cannot become admissible before u itself relabels. Reaching NIL certifies that relabel applies.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## P14 | A fixed five-vertex example

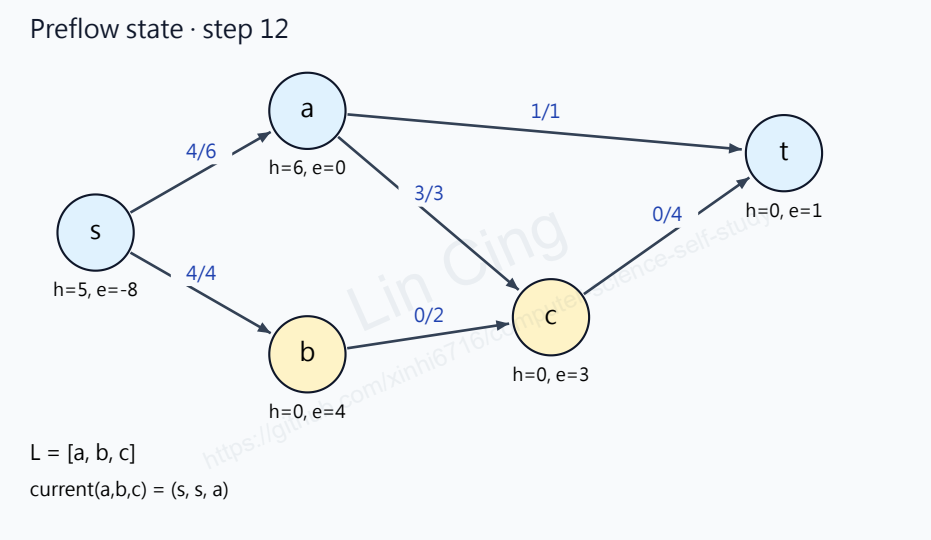
Use V=(s,a,b,c,t), n=5, with capacities sa=6, sb=4, ac=3, at=1, bc=2, ct=4. Fix N(a)=[s,c,t], N(b)=[s,c], N(c)=[a,b,t], and initial L=[a,b,c].




At step 0, h=(5,0,0,0,0), e=(-10,6,4,0,0), current=(s,s,a), and L=[a,b,c].

# P15 | a sends flow, then returns excess to s

At height 1, a sends 3 to c and 1 to t. Its remaining excess is 2. Only residual edge a→s remains, so a relabels to 6 and pushes 2 back to s. The source stays at height 5, preserving the required height difference.



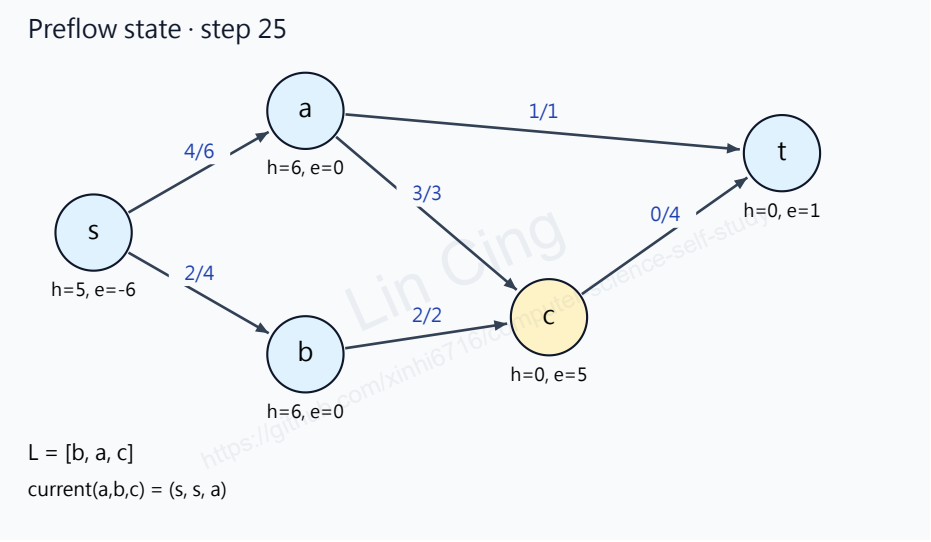



# P15 | a sends flow, then returns excess to s (continued)

| step | operation | h(s,a,b,c,t) | e(s,a,b,c,t) | current(a,b,c) | L |
|---|---|---|---|---|---|
| 5 | relabel-reset a | 5,1,0,0,0 | -10,6,4,0,0 | s,s,a | a,b,c |
| 7 | push a a→c δ=3 | 5,1,0,0,0 | -10,3,4,3,0 | c,s,a | a,b,c |
| 9 | push a a→t δ=1 | 5,1,0,0,0 | -10,2,4,3,1 | t,s,a | a,b,c |
| 11 | relabel-reset a | 5,6,0,0,0 | -10,2,4,3,1 | s,s,a | a,b,c |
| 12 | push a a→s δ=2 | 5,6,0,0,0 | -8,0,4,3,1 | s,s,a | a,b,c |
| 14 | move-front a | 5,6,0,0,0 | -8,0,4,3,1 | s,s,a | a,b,c |

# P16 | b and c move to the front

b sends 2 to c and returns 2 to s, then moves to the front, giving L=[b,a,c]. The following zero-excess discharge of a returns immediately. Then c sends 4 to t, relabels to 7, and returns 1 to a. Moving c gives L=[c,b,a].






# P16 | b and c move to the front (continued)

| step | operation | h(s,a,b,c,t) | e(s,a,b,c,t) | current(a,b,c) | L |
|---|---|---|---|---|---|
| 18 | relabel-reset b | 5,6,1,0,0 | -8,0,4,3,1 | s,s,a | a,b,c |
| 20 | push b b→c δ=2 | 5,6,1,0,0 | -8,0,2,5,1 | s,c,a | a,b,c |
| 22 | relabel-reset b | 5,6,6,0,0 | -8,0,2,5,1 | s,s,a | a,b,c |
| 23 | push b b→s δ=2 | 5,6,6,0,0 | -6,0,0,5,1 | s,s,a | a,b,c |
| 25 | move-front b | 5,6,6,0,0 | -6,0,0,5,1 | s,s,a | b,a,c |
| 32 | relabel-reset c | 5,6,6,1,0 | -6,0,0,5,1 | s,s,a | b,a,c |
| 35 | push c c→t δ=4 | 5,6,6,1,0 | -6,0,0,1,5 | s,s,t | b,a,c |
| 37 | relabel-reset c | 5,6,6,7,0 | -6,0,0,1,5 | s,s,a | b,a,c |
| 38 | push c c→a δ=1 | 5,6,6,7,0 | -6,1,0,0,5 | s,s,a | b,a,c |
| 40 | move-front c | 5,6,6,7,0 | -6,1,0,0,5 | s,s,a | c,b,a |

# P17 | Final flow, cut, and complete trace

a still has current=s. At height 6 it returns one more unit to s. Finally e=(-5,0,0,0,5), so the flow value is 5. The cut S={s,a,b,c}, T={t} has capacity at+ct=1+4=5, matching the flow.

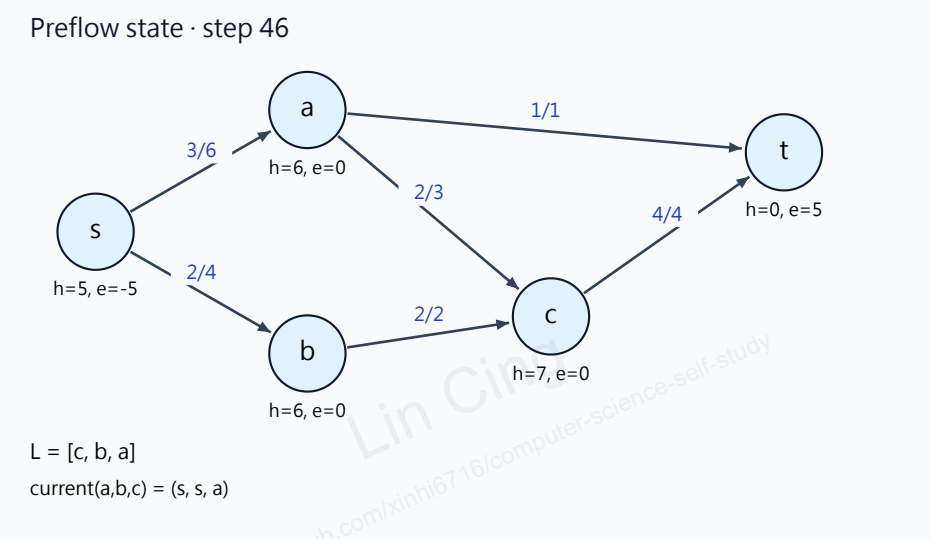
The chapter solutions give all 47 rows, including pointer advances, discharge boundaries, and list moves. Each figure has the same step number as its table row.

| step | operation | h(s,a,b,c,t) | e(s,a,b,c,t) | current(a,b,c) | L |
|---|---|---|---|---|---|
| 44 | push a a→s δ=1 | 5,6,6,7,0 | -5,0,0,0,5 | s,s,a | c,b,a |
| 46 | terminate | 5,6,6,7,0 | -5,0,0,0,5 | s,s,a | c,b,a |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# P17 | Final flow and conservation




The final edge flows are sa=3, sb=2, ac=2, at=1, bc=2, ct=4; the incoming and outgoing flow at each intermediate vertex is equal.

## P18 | Why relabel-to-front takes $O(n^3)$

At each outer-loop test, L topologically orders the admissible network on intermediate vertices, and vertices before current u have zero excess. Admissible edges decrease height, so the graph is acyclic. Initially intermediate heights are equal and u is first.

Without a relabel, pushes create no admissible edges and cannot send flow to a predecessor. Discharge empties u, so advancing preserves both claims. If u relabels, after its last relabel it has no admissible incoming edge, and pushes cannot create one. Moving u to the front preserves the order among other vertices; u is empty and is the only predecessor of the next vertex. Both claims hold again. At the end all intermediate excess is zero; valid heights exclude an augmenting path, proving maximum flow.

The claims can fail during discharge. At step 37, $c \to a$ is admissible although c follows a. At step 38, a has excess 1. Moving c at step 40 restores both.

Between relabels there are $O(n)$ DISCHARGE calls; $O(n^2)$ relabels give $O(n^3)$ calls. Relabels, arc advances and saturating pushes cost $O(nm)$. Each discharge has at most one nonsaturating push, which empties u. Total $O(n^3+nm)=O(n^3)$ for a simple graph, $m=O(n^2)$.



## Translate requirements into flow constraints

Use nonnegative arc flow x(e) on each original directed edge, with $0 \le x(e) \le c(e)$. If the chapter uses skew-symmetric net flow f, then $f(u,v)=x(u,v)-x(v,u)$. Lower bounds constrain individual arcs, so retain x rather than canceling obligations in opposite directions.

For every reduction, prove that an original solution yields a flow of equal value and that a flow yields an original solution. Discrete choices also require integral flows.

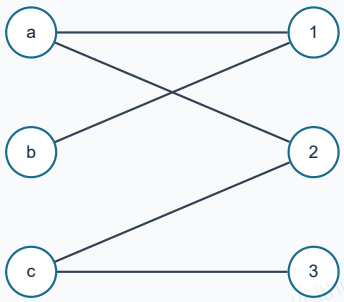
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Build a network for bipartite matching

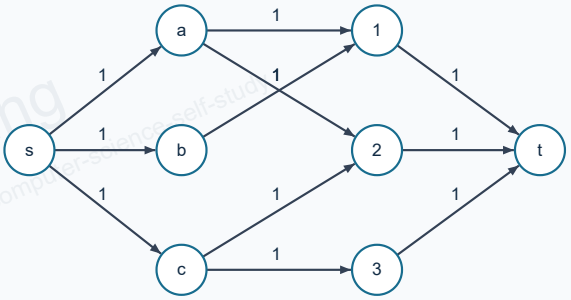
Given $G=(L \cup R, E)$, with disjoint sides and every edge crossing the partition, add s,t. Direct each original edge from L to R. Add $s \to u$ for $u \in L$ and $v \to t$ for $v \in R$. Give every arc capacity 1.

Example $L=\{a,b,c\}$, $R=\{1,2,3\}$, $E=\{a1,a2,b1,c2,c3\}$. The network has 8 vertices and 11 arcs. The left panel is the original graph and the right panel is its flow network.

Original bipartite graph




Flow network, all capacities 1




Solution {a2,b1,c3}; flow value 3; cut {s} has capacity 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Map a matching to a flow

Take $M=\{a2,b1,c3\}$. Send one unit along $s \to u \to v \to t$ for every $uv \in M$, and zero on other arcs. Matching edges share no endpoints, so no source or sink arc is used twice.

Each internal vertex receives and sends the same amount. Capacity and conservation hold. Total flow across $L \to R$ is $|M|=3$. The cut $S=\{s\}$ has capacity 3, proving both the flow and matching optimal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Map an integral flow to a matching

Given an integral flow x, choose M={uv∈E:x(u,v)=1}. Capacity 1 on s→u and conservation allow at most one outgoing arc with positive integral flow at u. Capacity 1 on v→t gives the symmetric condition at v. Thus M is a matching and |M|=|x|.

Starting Ford–Fulkerson at zero preserves integral residual capacities, bottlenecks, and flows when all capacities are integers. This proves existence of an integral maximum flow. It does not make every maximum flow integral. In $K_{2,2}$, flow 1/2 on each middle arc is a maximum flow of value 2, but its positive arcs are not a matching.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A cut explains matching failure

Let L={a,b,c}, R={1,2,3}, and E={a1,b1,c2,c3}. Both a and b can use only vertex 1. Matching {a1,c2} has size 2.

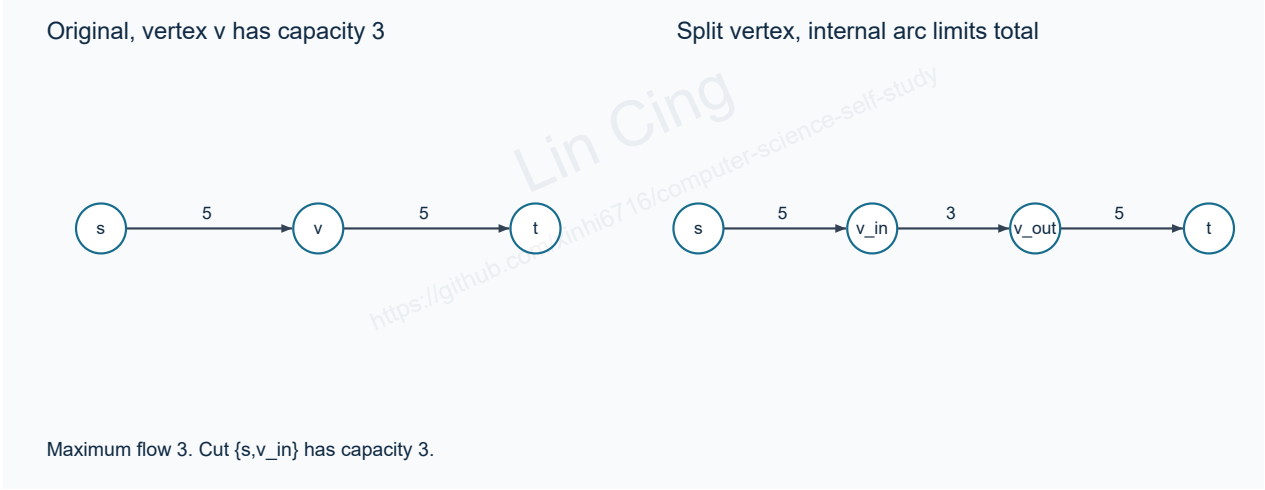
For S={s,a,b,1}, the only outgoing cut arcs are s→c and 1→t, each of capacity 1. The cut proves that the maximum matching size is 2. Set A={a,b} has |N(A)|=1<2, explaining why L cannot be fully matched. A maximal matching need not be maximum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Vertex capacity limits total throughput

The original network has s→v and v→t of capacity 5, but v can process only 3 units per time unit. Limiting every incident edge separately does not generally bound total throughput when several arcs enter v. The required constraint is $\Sigma_u x(u,v) \leq b(v)$.

Split v into v_in,v_out, joined by an internal arc of capacity b(v)=3. Redirect incoming arcs to v_in and outgoing arcs from v_out. The example has maximum flow 3, certified by cut {s,v_in}.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Prove vertex splitting in both directions

Copy each original flow to its redirected arc. Set the internal arc flow to total inflow at v. Original conservation and vertex capacity make this transformed flow feasible. Conversely, conservation at v_in and v_out equates internal arc flow with total inflow and outflow. Merging the pair recovers the original feasible flow.

Splitting all n vertices gives 2n vertices and m+n arcs. Splitting only k constrained internal vertices gives n+k vertices and m+k arcs. To limit total throughput at a source or sink, split it too and attach the external source to s_in and the external sink after t_out.

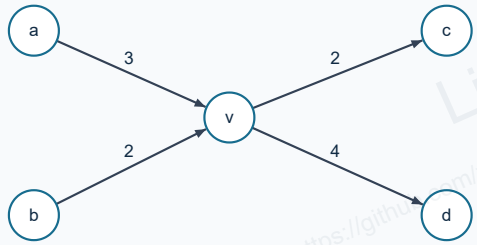
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Add a supersource and supersink

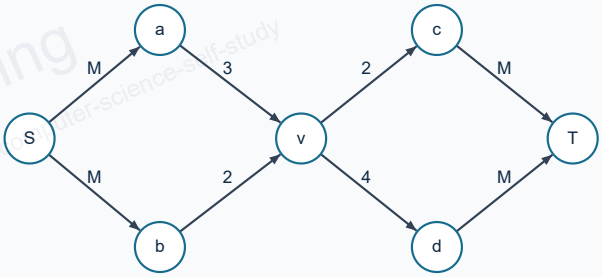
The original problem has sources a,b, sinks c,d, and transit vertex v. Capacities are a→v=3, b→v=2, v→c=2, and v→d=4. Original sources have no incoming arcs and original sinks have no outgoing arcs.

Add S→a,S→b,c→T,d→T. Use M=11, the sum of original capacities. Send 2 on a→v→c, 1 on a→v→d, and 2 on b→v→d, totaling 5. Cut {S,a,b} has capacity 3+2=5 and proves optimality.

Original multisource network




Supernodes, connector capacity M=11




Flow value 5, with 2 delivered to c and 3 to d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Maximum total flow differs from quota feasibility

Extend an original solution by setting S→a to the net outflow of a and c→T to the net inflow of c. Every original vertex then conserves flow. Removing the supernodes recovers the same total value. This proof requires nonnegative net production at sources and nonnegative net consumption at sinks.

If a,b must supply 3,2 and c,d must receive 2,3, give the new arcs capacities 3,2,2,3 and test for value P=5. Total new source and sink capacity are both 5, so value 5 means every quota arc is saturated. The previous example is feasible. If sink quotas become 3,2, arc v→c still has capacity 2. Cut {S,a,b,v,d} has capacity 2+2=4, proving quota infeasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Choose finite infinity from a proved bound

In a maximum-flow reduction, new arcs must not reduce the original optimum. With finite nonnegative original capacities, $M=\Sigma_e c(e)$ is safe, though possibly loose. A proved upper bound on original throughput also works, such as $3+2=5$ here. A guessed constant is not a general argument.

A demand-feasibility auxiliary network needs to send only $D$ units, so an artificial connector that should not restrict this routing may use capacity $D$. If a cut argument must strictly exclude artificial arcs, choose $M$ above a known candidate-cut upper bound, such as $U+1$ for an integral model. $M \geq U$ preserves the optimum but can allow a tied artificial cut.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Lower bounds create vertex imbalances

A feasible circulation conserves flow at every vertex and satisfies $\ell(e) \leq x(e) \leq u(e)$, with $0 \leq \ell \leq u$. Set $y(e)=x(e)-\ell(e)$, giving capacity $u-\ell$. Define $d(v)=\Sigma_{in}\ell-\Sigma_{out}\ell$.

Substitute into conservation at each vertex. Equation $\Sigma_{in}(y+\ell)=\Sigma_{out}(y+\ell)$ gives $\Sigma_{out}y-\Sigma_{in}y=d(v)$. Thus $d(v)>0$ requires net $y$ outflow $d(v)$, while $d(v)<0$ requires net $y$ inflow $-d(v)$. Each lower bound contributes once positively at its head and once negatively at its tail, so $\Sigma_v d(v)=0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Derive supernode directions from conservation

Add source S* and sink T*. For d(v)>0, add S*→v of capacity d(v). For d(v)<0, add v→T* of capacity −d(v). Let D=Σ_{d>0}d=Σ_{d<0}(−d).

With demand arcs saturated, a positive-d vertex satisfies Σ_in y+d=Σ_out y. A negative-d vertex satisfies Σ_in y=Σ_out y−d. A zero-d vertex has ordinary conservation. All three cases give Σ_out y−Σ_in y=d.

Feasibility is equivalent to auxiliary maximum-flow value D, or saturation of every arc leaving S*. Since total capacity into T* also equals D, those arcs are saturated too. Remove supernodes and recover x=ℓ+y.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



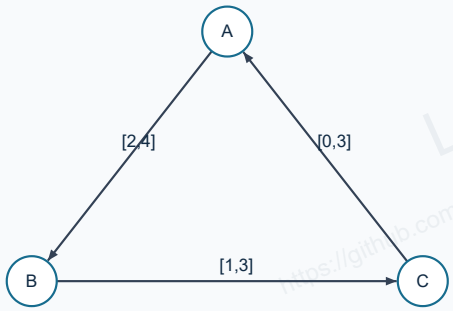
## A complete feasible lower-bound example

Original arcs are A→B:[2,4], B→C:[1,3], C→A:[0,3], with brackets denoting [ℓ,u]. Imbalances are d(A)=0−2=−2, d(B)=2−1=1, d(C)=1−0=1, and D=2.

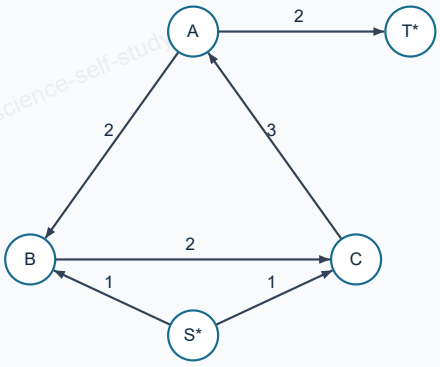
New capacities are 2,2,3. Add S*→B=1, S*→C=1, A→T*=2. Send 1 along S*→B→C→A→T*, then 1 along S*→C→A→T*. This yields y=(0,1,2), hence x=(2,2,2).

Each original vertex receives 2 and sends 2, and every lower and upper bound holds.

Original circulation with bounds




Auxiliary network, capacities u−ℓ




y=(0,1,2), x=(2,2,2); all demand arcs are saturated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## An infeasible example needs a deficient cut

Keep the previous example but change C→A to [0,1]. Imbalances and D remain unchanged. Path S*→C→A→T* carries 1 and saturates C→A.

Take auxiliary cut S={S*,B,C}, with complement {A,T*}. Its only outgoing arc is C→A, of capacity 1. Thus maximum flow is at most 1<D=2. In the original network, A must send at least 2 but can receive at most 1, contradicting conservation. Nonnegative remaining capacities alone do not prove lower-bound feasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Convert lower-bounded s-t flow to circulation

First restrict attention to s-t flows with nonnegative value. Add a return arc t→s with bounds [0,M] so every vertex conserves flow. Its flow equals the original s-t value. To test an exact value F, use [F,F] and apply the demand reduction.

Example s→a:[1,3], a→t:[1,2], with F=2. Adding t→s:[2,2] gives d(s)=1, d(a)=0, d(t)=−1. Send 1 along S*→s→a→t→T*. Recover flow 2 on both original arcs and 2 on the return arc. F=3 is infeasible because a→t has upper bound 2.

For feasibility without a fixed value, M=Σ original upper bounds is safe. A single nonnegative return arc does not represent allowed negative flow values; those require another model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Validate the recovered solution

Check original choices or quotas first, then bounds on every original arc and conservation at every original vertex. Provide an equal-value cut, saturated demand arcs, or a deficient cut as a certificate. Use integrality only when transformed capacities are integral and an integral flow is obtained.

General real capacities allow fractional flows. Even integral capacities permit fractional feasible flows and fractional maximum flows. Matching extraction requires an integral maximum flow. Integral lower and upper bounds admit an integral circulation whenever the auxiliary integral-flow construction is feasible.

Sources are CLRS 26.1 and 26.3, with Exercises 26.1-7, 26.2-5, 26.2-6, and 26.3-2. The lower-bound reduction and numerical examples are derived here. check.py verifies examples by enumeration and cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 26, Maximum Flow. Original handwritten algorithms notes (演算法_wjungle), network-flow pages. MIT 6.046J, Fall 2004, Lectures 20 and 21, Maximum Flow and Edmonds–Karp.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Computational geometry

How can exact geometric predicates support efficient and correct algorithms?

We study **orientation → segment intersection and sweep → convex hull → closest pair**. Prerequisites include sorting, balanced search trees, divide and conquer, and asymptotic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# From geometry to algorithms

| Problem | Focus |
| --- | --- |
| Orientation and intersection | Determinant signs, boundaries, and numeric precision. |
| Sweep line | Event order, status, and adjacency. |
| Convex hull | Sorting, stack invariants, and degeneracy. |
| Closest pair | Divide and conquer, the strip, and geometric packing. |

Each result depends on explicit input assumptions and a correctness proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Answer geometry questions with local signs

The input consists of planar points and closed segments. Fix coordinates and intersection semantics first. Use orientation to test a pair of segments, then use a sweep to decide whether any intersecting pair exists.

All numerical examples use exact integers or rationals. Time bounds initially count each exact arithmetic operation as constant; large-integer bit costs are additional. Sources are CLRS 33.1–33.2 and MIT 2004 L17. Original notebook PDF43–47 supplies the geometry outline and turn diagrams.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A positive left turn depends on operand order

Use an x-axis pointing right and a y-axis pointing up.

$$\operatorname{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

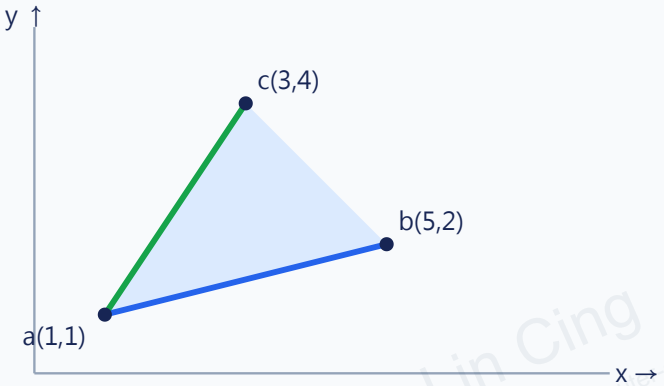
A positive value means a left turn along a,b,c; a negative value means a right turn; zero means collinearity. Its magnitude is the parallelogram area. The triangle area is half that magnitude.

CLRS `DIRECTION(a,b,c)` uses $(c - a) \times (b - a)$, the negative of our formula. Reverse its inequalities when translating code. The SVG rendering maps mathematical y upward into screen coordinates; it does not change our sign convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A positive left turn depends on operand order (continued)

### Orientation signs and coordinates




(b−a) × (c−a)

4·3 − 1·2 = +10

Left turn; triangle area 5

DIRECTION = −10

Mathematical y points up

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Evaluate one determinant

Let a=(1,1), b=(5,2), and c=(3,4). The displacement vectors are b−a=(4,1) and c−a=(2,3).

$$\operatorname{orient}(a,b,c) = 4 \cdot 3 - 1 \cdot 2 = 10 > 0.$$

Thus c lies left of the directed line ab, and the triangle area is 5. Exchanging b,c gives −10. Translating all points preserves the displacement vectors. If c=(9,3), its displacement is (8,2), so the determinant is 4 • 2−1 • 8=0. This proves collinearity, but does not prove that c belongs to closed segment ab.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Closed intersection and proper crossing

The closed segment $ab = \{a + t(b - a) : 0 \le t \le 1\}$ contains its endpoints. We also allow a=b, a point segment.

Here a proper crossing means one interior intersection of two noncollinear segments. If a specification uses “strict intersection” to include collinear interior overlap, define that separately.

| Geometric case | Closed intersection | Proper crossing |
|---|---|---|
| Two interiors cross | Yes | Yes |
| Endpoint or T contact | Yes | No |
| Collinear overlap | Yes | No |
| Point segment on another segment | Yes | No |
| Only extensions meet | No | No |

Lin Cing https://github.com/xinhi6716/computer-science-self-study



# Closed intersection and proper crossing (continued)

## Boundary cases for closed segments

Proper crossing | Endpoint contact | Collinear overlap | Point segment

Blue/red are inputs; green is overlap; gold marks contact.

All four intersect as closed segments; only the first is proper.

Endpoints, collinearity, and point segments need explicit semantics.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## on needs collinearity and both coordinate bounds

```
on(a,b,p):
    return orient(a,b,p) == 0 and
           min(ax,bx) <= px <= max(ax,bx) and
           min(ay,by) <= py <= max(ay,by)
```

With a=(2,1), b=(2,5), p=(2,8), collinearity and the x bound hold, but p is outside ab. The y bound is necessary. When a=b, the two coordinate bounds require p=a, so point segments work too.

CLRS ON-SEGMENT assumes its caller has established collinearity. Our on includes that condition. A bounding box alone cannot establish segment membership.



## Four orientations complete the test

Write o1=orient(a,b,c), o2=orient(a,b,d), o3=orient(c,d,a), and o4=orient(c,d,b).

```
opposite(u,v): return (u<0 and v>0) or (u>0 and v<0)
intersect(a,b,c,d):
    compute o1,o2,o3,o4
    if opposite(o1,o2) and opposite(o3,o4): return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

The first branch is exactly proper crossing. The second handles endpoints, collinear overlap, and point segments. Compare signs rather than multiplying o1 · o2, which would add another overflow risk. Vertical segments need no slope calculation.

## Both straddling conditions are necessary

Take ab from (0,0) to (6,0), and cd from (7,−1) to (7,2).

$$(o_1, o_2, o_3, o_4) = (-6, 12, 21, 3).$$

The endpoints c,d straddle the line through ab, but a,b lie on the same side of the line through cd. The line intersection (7,0) is beyond ab, so the segments are disjoint.

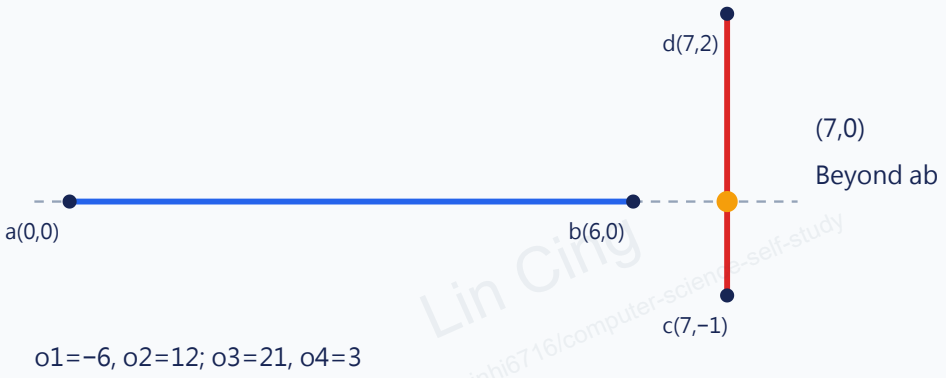
Without zero orientations, the first sign pair puts the line intersection inside cd; the second puts that same point inside ab. Both are necessary. For collinear closed intervals, overlap implies that an endpoint of one belongs to the other. The four on tests therefore cover the remaining cases.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Both straddling conditions are necessary (continued)

### Straddling an extension is insufficient




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Exact signs need enough bits

If each coordinate has absolute value at most B, differences are at most 2B. A conservative bound is $|\operatorname{orient}| \leq 8B^2$. Widen coordinates before subtraction and multiplication; casting the result is too late.

For a=(0,0), b=(100000,0), c=(0,100000), orientation is $10^{10}$, beyond signed 32-bit range. Full signed 32-bit coordinates also do not guarantee safe signed 64-bit intermediate results.

Floating subtraction can lose significant digits. Scaling coordinates by λ scales the determinant by $\lambda^2$, so a fixed epsilon has no scale-independent guarantee. Treating values within epsilon as equal can also violate comparator transitivity. Use exact arithmetic or error-bounded filters with increased precision when inconclusive. An arbitrary epsilon is not a proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Replace all pairs with events

Testing every pair of n segments uses $\Theta(n^2)$ predicates. Detection needs one witness. Reporting must output all k intersecting pairs. Since k can be $\Theta(n^2)$, the $O(n \log n)$ detection bound cannot describe reporting all intersections.

A vertical sweep uses x as time. Its event schedule stores endpoints. Its status T stores segments crossing the sweep line, ordered bottom to top.

Our initial simplified version assumes nonvertical, nonzero segments, distinct endpoint x-coordinates, no three segments through one point, and no collinear overlap. The pair predicate has none of these restrictions. They are assumptions for the sweep status and proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Both insertion and deletion create neighbors

```
T = empty balanced search tree
sort the 2n endpoints by x
for endpoint p:
    if p is the left endpoint of s:
        insert s in T
        test s against predecessor and successor; if any intersects: return true
    else:
        test predecessor(s) against successor(s); if they intersect: return true
        delete s from T
return false
```

Each predicate tests the complete segments, not only the current sweep position. Detection can therefore return before the actual crossing. At deletion, test the two segments formerly separated by s. Omitting this test can miss a newly adjacent pair.



## Trace numerical coordinates at every event

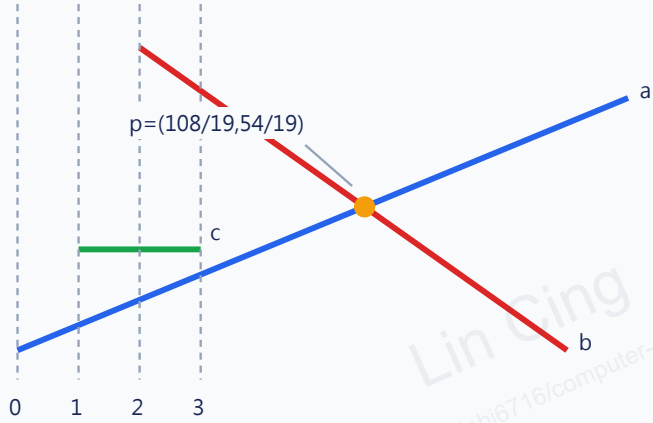
a:(0,0)→(10,5), c:(1,2)→(3,2), b:(2,6)→(9,0). Status is listed bottom to top.

| Event | Heights and status | New tests |
|---|---|---|
| x=0 insert a | [a] | None |
| x=1 insert c | a=1/2,c=2; [a,c] | a,c disjoint |
| x=2 insert b | a=1,c=2,b=6; [a,c,b] | c,b disjoint |
| x=3 before deleting c | a=3/2,c=2,b=36/7; [a,c,b] | a,b intersect; return |

Segment a reaches y=2 at x=4, beyond c. Segment b reaches y=2 at x=20/3, also beyond c. Solving $x/2 = (54 - 6x)/7$ gives the a,b intersection $(108/19, 54/19)$. Deleting c occurs before this crossing.

## Trace numerical coordinates at every event (continued)

### Deleting c detects a future intersection




| Event x | T bottom→top |
|---|---|
| 0 | [a] |
| 1 | [a,c] |
| 2 | [a,c,b] |
| 3 | a,b → true |

x=3 < 108/19

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why neighbor tests decide existence

Suppose the detector has not returned. Consider segments a,b at the leftmost intersection p. No crossing precedes p, so the status order cannot change on its own to the left of p. With no triple intersection, a,b must become adjacent before or at p.

Their first adjacency arises either when one is inserted or when the last intervening segment is deleted. The algorithm tests complete segments at both kinds of event, so it returns before or at p. If it returned earlier on another pair, the existence answer is already correct.

A balanced tree performs a constant number of O(log n) operations per event. There are 2n endpoints, sorting costs O(n log n), and total space is O(n). The proof relies on stopping after one witness; it does not prove reporting correctness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Equal x-coordinates need an event policy

To allow equal endpoint x-coordinates in the nonvertical version, sort by (x,e,y,id), with e=0 for a left endpoint and e=1 for a right endpoint. Process all starts before ends at the same x, and increasing y within each kind. The id makes identical events reproducible.

a:(0,1)→(2,1) and b:(2,1)→(4,2) meet at their shared endpoint. Deleting a before inserting b leaves them absent from T at different times and misses their contact. Inserting b first allows the test.

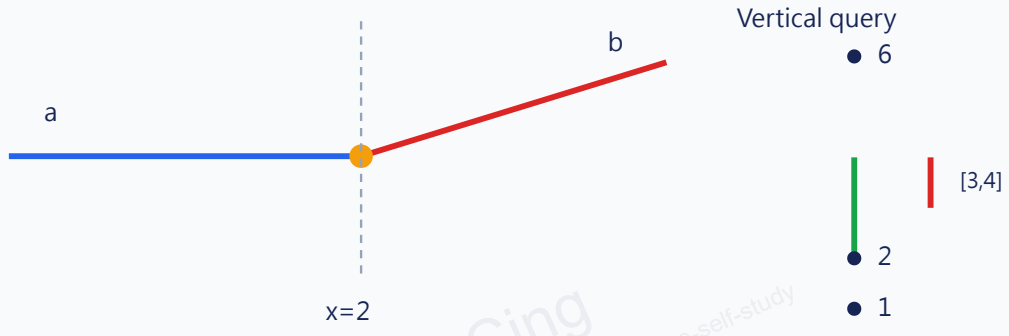
Equal height can mean an actual intersection; an arbitrary id is not a substitute for testing it. Multiple segments at one point require grouped events and equal-height checks, or the simplifying assumption must remain. CLRS 33.2 excludes triple intersections in its initial proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Equal x-coordinates need an event policy (continued)

### Same-x ordering and closed intervals




Insert b before deleting a to detect endpoint contact.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A vertical segment is an interval query

A vertical segment v:(x,ylo)→(x,yhi) cannot use one ordinary height key. It may meet any active nonvertical segment in that interval.

At the same x, insert nonvertical starts, query closed vertical intervals against status heights, then delete nonvertical ends. For detection, find the first height ≥ylo and test whether it is ≤yhi. Sort vertical intervals at that x by ylo to check their overlap separately. A zero-length segment is a point interval. Shared nonvertical endpoints also need checking.

At x=2 with active heights 1,2,6, interval [3,4] has no hit; [2,4] hits the boundary at y=2. These are required operations for a generalized implementation. The earlier simplified pseudocode does not implement all grouped-event and overlap cases, so it does not claim universal degeneracy support.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Printing and continuing breaks the status

a:(0,0)→(4,4), b:(0,4)→(4,0), c:(0,5)→(4,5). Before x=2 the order is [a,b,c]; after x=2 it is [b,a,c].

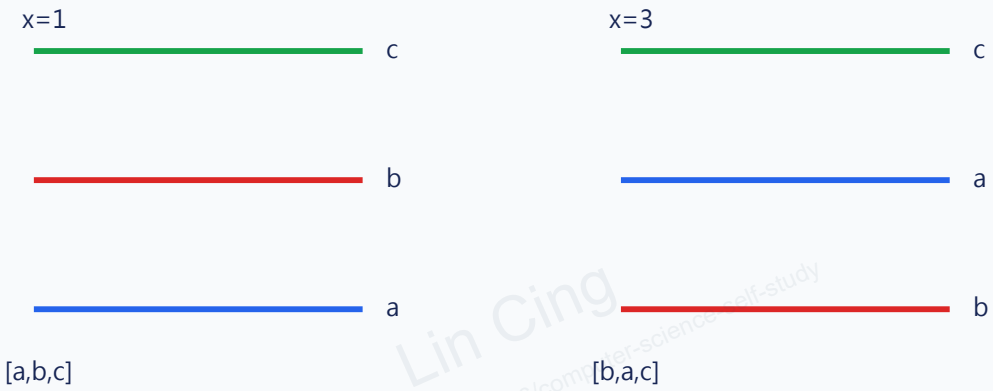
If insertion discovers a,b and merely prints them, an endpoint-only algorithm does not swap their order between endpoint events. Its tree stops representing geometric neighbors. The first tested intersection also need not be the leftmost one, since predicates test entire segments.

The MIT04 reporting version additionally schedules predicted intersection events, swaps neighbors at each crossing, and updates events with duplicate and stale-event handling. In general position its time is O((n+k) log n). With collinear overlap, define whether the output consists of intersecting pairs, points, or overlapping segments; there may be infinitely many shared points.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Printing and continuing breaks the status (continued)

### Geometric order changes after a crossing




An intersection event must exchange order



## Align predicate semantics and data structures

| Layer | Required specification | Checked example |
|---|---|---|
| Orientation | Axes, operand order, arithmetic | Determinants 10 and −10 |
| Pair intersection | Closed or proper, point segments | Overlap, endpoints, extensions |
| Sweep detection | Events, neighbors, stopping rule | Deleting c exposes a,b |
| Generalization | Shared points, verticals, collinearity | Insert, query, then delete |
| Reporting | Intersection events and output type | Update order after a crossing |

Exercises A10-P01–P06 check numerical signs, boundary cases, numerical reliability, event traces, tied coordinates, and detection versus reporting. Orientation will also determine which vertices a convex-hull scan keeps or pops.

## Hull vertices and closest pairs need different certificates

A hull scan removes points that cannot remain extreme vertices. Closest pair removes pairs that cannot improve a distance bound. Both reductions require geometric invariants. The fixed inputs and executable checks are in data.json and check.py.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Specify the output policy

Return extreme hull vertices counterclockwise, without repeating the first vertex. Omit intermediate collinear boundary points. Return endpoints for an all-collinear set, the point for a singleton, and an empty list for an empty input. Deduplicate hull coordinates. Keep distinct point ids for closest pair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Use one fixed hull input

A(0,0), B(4,0), C(3,1), D(4,3), E(2,2), F(2,4), G(0,3), H(1,1), I(2,0), J(1,2). A has minimum y, breaking ties by minimum x. orient(a,b,c)=(b−a)×(c−a) is positive for a left turn.

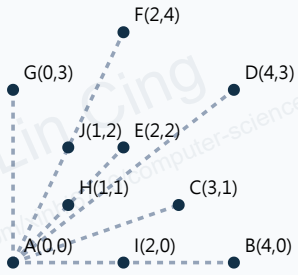
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Resolve equal polar angles

Raw polar order about A is I,B,C,D,H,E,J,F,G. Keep only the farthest point on each ray. This removes I, H, and J, leaving B,C,D,E,F,G. Exact comparisons use cross products and squared distances rather than equality of floating atan2 values.

Polar order about A; remove I, H, J




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pop on every nonleft turn

For each point p in polar order, while $|S| \geq 2$ and $\text{orient}(S[-2], S[-1], p) \leq 0$, pop. Then push p. The nonstrict test implements the extreme-vertex policy. Changing it to $<0$ alone does not provide a complete boundary-point policy; angle ties and the closing edge also need attention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Replay every stack update

| Point | Popped | Stack bottom to top |
|---|---|---|
| B | - | A,B |
| C | - | A,B,C |
| D | C | A,B,D |
| E | - | A,B,D,E |
| F | E | A,B,D,F |
| G | - | A,B,D,F,G |

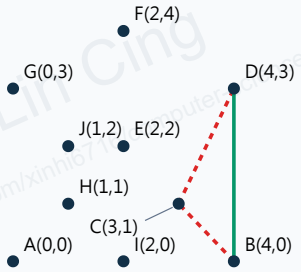
Start with S=[A]. Each row lists all pops for that insertion. On this nondegenerate input this equals the standard version initialized with the first three retained points.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Explain why a pop is safe

At D, orient(B,C,D)=−3, so pop C and obtain A,B,D. At F, orient(D,E,F)=−4, so pop E and obtain A,B,D,F. At G no point is popped. A popped point lies inside or on the triangle formed by A, its retained predecessor, and the incoming point, and is not an extreme vertex of the processed prefix.

Read D, pop C, retain A B D

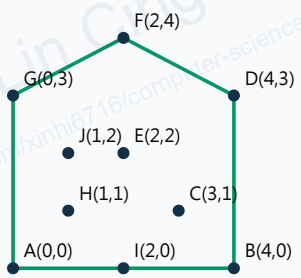





## Prove the prefix invariant

After each insertion, S contains exactly the extreme vertices of the processed prefix together with A, in counterclockwise order. Initialization holds. Pops preserve its convex hull; the next polar point extends its outside chain. At termination every input point has been considered. The answer is A,B,D,F,G, and every input point lies on or to the left of each directed hull edge.

Extreme hull vertices A B D F G



## Compare Graham scan with Jarvis march

Graham uses O(n log n) sorting and O(n) scanning because each retained point is pushed once and popped at most once. Space is O(n). Jarvis scans all n points for each next supporting edge and takes O(nh) for h hull vertices. Choose the farthest collinear candidate and stop on returning to the start. If h=n, its cost is $O(n^2)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fix the closest-pair input

A(−4,0), B(−3,4), C(−1,1), D(0,5), E(0,1.5), F(2,3), G(4,0), H(5,5). Compare squared distances and take a square root only for presentation. Distinct ids at the same coordinates immediately certify a distance of zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Presort both coordinate orders

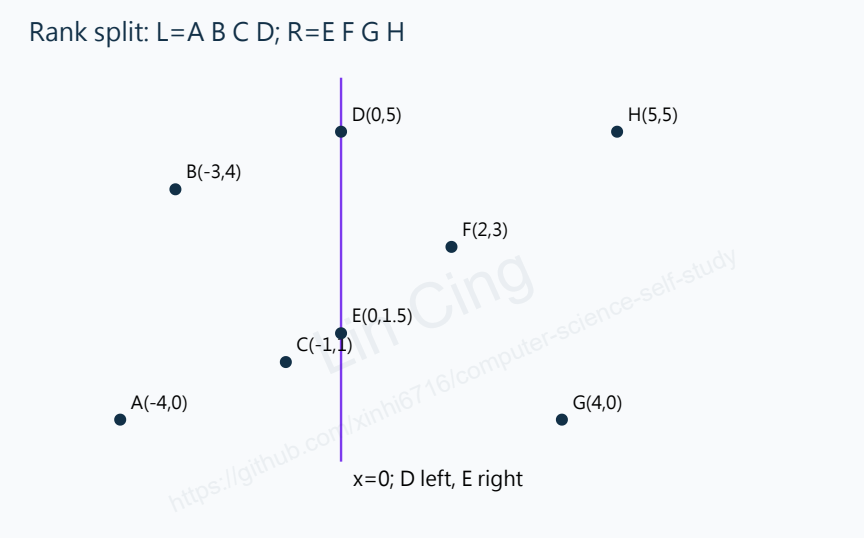
Sort X by (x,id) and Y by (y,id) once. Each recursive call receives the same ids in both arrays. Split X by rank, construct a set of left ids, and stably filter Y by id membership. Do not sort Y again at every recursive level.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Partition repeated x values by identity

X=A,B,C,D,E,F,G,H. L=A,B,C,D and R=E,F,G,H with dividing line x=0. D and E share x=0 but belong to different subproblems. Assigning every x≤0 point to the left can destroy balance.



## Separate recursive and crossing answers

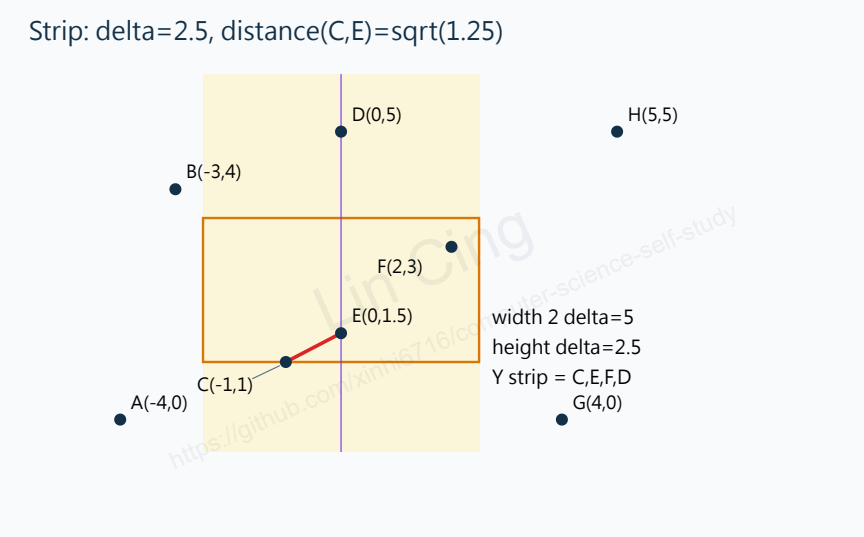
Brute-force subproblems of size at most 3. The minimum left squared distance is 10, attained by A-C and B-D. On the right it is 6.25 for E-F. Thus $\delta=\min(\sqrt{10},2.5)=2.5$. Any improving crossing pair has both $|x-0|<\delta$ and vertical separation $<\delta$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Scan the strip in y order

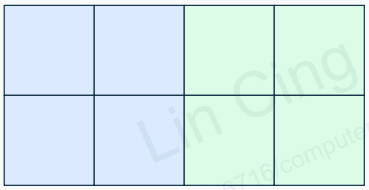
Stably filter Y by $|x-\text{line}|<\delta$. The strip order is C(−1,1), E(0,1.5), F(2,3), D(0,5). C-E has squared distance 1.25 and distance about 1.118. Other pairs passing the original vertical test are C-F with 13, E-F with 6.25, and F-D with 8. Only C-E improves δ.



## Derive the packing bound

Assume $\delta > 0$ and that each recursive side has pairwise distances at least $\delta$. The later points within vertical distance $\delta$ of p lie in a $2\delta$-wide, $\delta$-high rectangle. Split each side into four $\delta/2$ by $\delta/2$ cells. A cell holds at most one point of that side since its diagonal is $\delta/\sqrt{2} < \delta$. Points on the dividing line retain their assigned side.

Four delta/2 cells per side



2 delta by delta; cell diagonal = delta/sqrt(2)
At most 4 left + 4 right = 8 including p



## Seven successors has explicit assumptions

There are at most four left points and four right points in the rectangle, including p. At most seven points follow p. Y uses a consistent (y,id) order and a half-open cell ownership rule resolves boundaries. The proof requires separation within each side, not between the two sides. If $\delta = 0$, return before using the positive-width packing argument.

## Keep the combine step linear

Presorting takes O(n log n). Splitting X, filtering Y, building the strip, and making constantly many comparisons per point take O(n) per call. $T(n)=2T(n/2)+O(n)=O(n \log n)$. Sorting strips in each call instead gives $O(n \log^2 n)$. Depth is O(log n); sequential recursion with released temporary arrays needs O(n) extra space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Read sources and check the actual input

CLRS third edition § § 33.3–33.4, printed pages 1029–1043, PDF pages 1050–1064. Original scanned note PDF pages 46–47 support Graham scan and hull applications. MIT 2004 L18 PDF pages 3–11 cover closest pair and packing. The local 2005 inventory contains no geometry lecture. check.py enumerates all 45 hull-input and 28 closest-input pairs and checks support edges, the stack trace, and the divide-and-conquer result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 33. MIT 6.046J Fall 2004 computational geometry lecture slides, segment intersection and closest pair. Original handwritten algorithms notes (演算法_wjungle), computational geometry pages.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Complexity and approximation

How does problem size affect the cost of solving a problem? Measure time using the input encoding length, then study decision problems, verification, and polynomial-time reductions.

Next study approximation algorithms for optimization problems, using feasible solutions, bounds, and counterexamples to understand their guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Chapter roadmap

Decision problems → P and NP → reduction direction → NP-completeness → approximation ratios and algorithms.

Distinguish a reduction proving hardness from an algorithm producing an approximate solution. NP-completeness alone proves neither $P \neq NP$ nor an exponential lower bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Write the problem as a language

We start with input bit length and define P, NP, and polynomial-time many-one reduction. The goal is to reconstruct 3SAT $\leq$p CLIQUE, including both proof directions.

Prerequisites are Boolean formulas, undirected simple graphs, asymptotic bounds, and universal/existential quantifiers.

You should distinguish finding a certificate from checking one and identify the two obligations in an NP-completeness proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Separate decision, search, and optimization

| Form | CLIQUE example | Output |
|---|---|---|
| Decision | Is there a clique of at least k vertices? | yes/no |
| Search | Find a k-clique if one exists | A vertex set or failure |
| Optimization | Compute the maximum clique size ω(G) | An integer |

P and NP here are classes of decision languages. A polynomial-time optimization algorithm decides the threshold version by comparing ω(G) with k. The converse requires an argument rather than a change of terminology.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Input length is not numerical value

Let x be the entire encoding and N = |x| its bit length. For $t \geq 1$, the binary encoding has $\lfloor \log_2 t \rfloor + 1$ bits. The graph representation and threshold k both count toward N.

For t = 2^m, the encoding needs m+1 bits. A loop with t iterations takes Θ(2^m) steps. Writing O(t) does not make this polynomial in the binary input length. A unary representation has t symbols and changes the encoding.

An adjacency matrix and an explicit adjacency list are polynomially interconvertible. This does not allow expanding binary integers into unary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Input length is not numerical value (continued)

**Same value, different input lengths**

t = 2^m

| Binary | Unary |
| --- | --- |
| 1000...000 | 1111...111 |
| m + 1 bits | 2^m symbols |
| t loop steps = Θ(2^m) | t steps = Θ(input length) |

Polynomial time is measured against complete encoding length N = |x|.

O(t) does not mean O(poly(log t)).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Define P using encodings

A decision language L ⊆ {0,1}* contains the encodings of all yes instances. We reject malformed encodings in polynomial time.

$$L \in P \iff \exists M\ \exists p \text{ polynomial } \forall x:\ M(x) \text{ halts within } p(|x|) \text{ and } [M(x) = 1 \iff x \in L].$$

M is a deterministic decider. The bound covers every input, including no instances. PATH = {⟨G,s,t,k⟩ | an s-to-t path with at most k edges exists} is in P. Run BFS and compare its shortest distance with k.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## NP bounds certificate length

There are a deterministic verifier V and polynomials p and q such that

$$x \in L \iff \exists c,\ |c| \le p(|x|) : V(x, c) = 1.$$

V halts in O(q(|x|+|c|)) time. The bound on certificate length makes verification polynomial in |x| for all permitted certificates. Reject overlong certificates first.

NP means nondeterministic polynomial time. It does not mean non-polynomial. The definition supplies a short certificate for a yes instance, without supplying a polynomial-time method to find it.

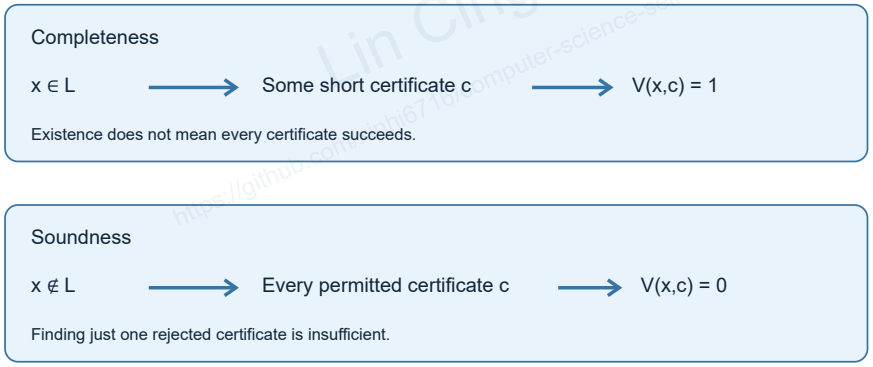
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Completeness and soundness use different quantifiers

| Obligation | Correct statement | Insufficient replacement |
|---|---|---|
| Completeness | $x \in L$ implies some short c is accepted | Every certificate is accepted |
| Soundness | $x \notin L$ implies every permitted c is rejected | Some certificate is rejected |

V must halt on every input and permitted certificate. A validator that checks only syntax is insufficient. An NP verifier must check that the certificate establishes the yes answer.

**Completeness and soundness of a verifier**




$|c| \le p(|x|)$; time $\le q(|x| + |c|)$, for polynomials p and q.



## Make the CLIQUE verifier executable

The input is a simple undirected graph G and a binary nonnegative k. Reject immediately if $k > n$. Accept an empty certificate for $k = 0$. Otherwise a certificate lists exactly k distinct vertex identifiers.

1. Check the encoding, certificate length, identifier range, and distinctness.
2. For every $i < j$, check $\{c_i,c_j\} \in E$.
3. Accept only if every check succeeds.

Certificate length is $O(k \log(n+1)) \le O(n \log(n+1))$. With an adjacency matrix, $O(n^2+k^2)$ basic graph operations suffice; bit cost is also polynomial in input length. A clique supplies a valid list. Acceptance proves that every selected pair is adjacent.

## P ⊆ NP is known; equality is unresolved

If L ∈ P, define V(x,c) to run the decider for L and ignore c. The empty certificate works for every yes instance, and no certificate makes a no instance pass. Hence P ⊆ NP.

Whether NP ⊆ P, equivalently P = NP, remains unresolved. A diagram separating P from NP-complete languages requires an explicit P ≠ NP assumption.

Known NP-completeness theorems do not prove P ≠ NP. An exponential brute-force algorithm does not prove an exponential lower bound for every algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A many-one reduction preserves yes and no

A ≤p B means that a total function f is computable in time polynomial in input length and, for every x,

$$x \in A \iff f(x) \in B.$$

The function must run in polynomial time; solving A cannot be assumed free. If A∈P and B is nonempty and not universal, decide A and output a fixed yes or no string of B accordingly. A many-one reduction outputs one string, rather than making multiple oracle queries.

The equivalence must hold for every string. If the source rejects malformed encodings and the target has a fixed no string, map them to it. A target need not have a no string: for example, B={0,1}*.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

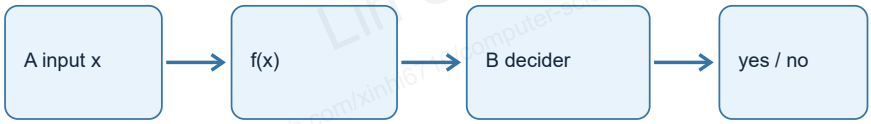
## Read the direction through the solver

If A ≤p B, a solver for B yields a solver for A. Compute x → f(x) → B-decider → the answer for A.

Thus B ∈ P implies A ∈ P. Its contrapositive is that A ∉ P implies B ∉ P. Neither A ∈ P implies B ∈ P nor hardness of B implies hardness of A.

To prove a target B hard, reduce a known hard A to B. Proving only B ≤p A shows that B can use a solver for A.

**A ≤p B lets A use a solver for B**




x ∈ A ⇔ f(x) ∈ B

B ∈ P ⇒ A ∈ P

To prove B hard, reduce a known hard A to B.

f must construct the target instance in polynomial time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Track both time and output length

Suppose f runs in O(N^a) time and |f(x)| ≤ C N^a. If a B-decider takes O(M^b) time on an input of length M, the composite takes

$$O(N^a) + O((CN^a)^b) = O(N^a + N^{ab}).$$

a, b, and C are fixed constants, so this is polynomial. In the standard bit-output model, writing the output takes at least its length in steps.

A compressed description of an exponentially large B instance is not automatically an input to the original B-decider. Its original time bound cannot simply be reused.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Prove transitivity of reductions

Suppose $A \leq_p B$ through f and $B \leq_p C$ through g. Define $h = g \circ f$.

Semantically, $x \in A$ iff $f(x) \in B$ iff $g(f(x)) \in C$. Each equivalence matters.

If f takes O(N^a) time and g takes O(M^b), then $|f(x)| = O(N^a)$ and h takes $O(N^a + N^{ab})$ time. Therefore $A \leq_p C$.

This proves closure under a fixed number of compositions. If a transformation squares length at every one of N calls, the final length may be $N^{(2^N)}$. A polynomial number of calls alone is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## NP-hard and NP-complete have different conditions

| Term | Condition |
| --- | --- |
| B is NP-hard | For every $A \in NP$, $A \leq_p B$ |
| B is NP-complete | B is NP-hard and $B \in NP$ |

If a known NP-hard A satisfies $A \leq_p B$, transitivity proves B NP-hard. A separate short-certificate verifier proves $B \in NP$ and completes the NP-completeness argument.

NP-hardness alone guarantees neither NP membership nor decidability. These definitions concern decision languages. For an optimization problem, specify the associated decision problem or the kind of solver reduction being used.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Construct 3SAT to CLIQUE

Let $F = C_1 \wedge \ldots \wedge C_m$, with three literal occurrences in each clause, allowing repetition here. The distinct-literal variant in CLRS is a restricted subset, so its NP-hardness also applies to this variant; assignment verification remains polynomial. We use the known NP-completeness of 3SAT from CLRS34 and prove the result for CLIQUE.

- Create one vertex $v_{ij}$ for each occurrence, retaining its clause i and literal $\ell_{ij}$.
- Add no edges within a clause.
- Between different clauses, add an edge exactly when the literals are not complements x and ¬x.
- Output $\langle G,k=m \rangle$.

Repeated copies of a literal remain distinct occurrence vertices. Equal literals in different clauses are consistent and must be adjacent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Draw the complete graph of a concrete formula

Use $F = (x \vee \neg y \vee z) \wedge (\neg x \vee y \vee z) \wedge (x \vee y \vee \neg z)$.

Number occurrences by clause as 1:x, 2:¬y, 3:z; 4:¬x, 5:y, 6:z; 7:x, 8:y, 9:¬z. Each gray or green line in the figure is an actual edge.

The complete edge set is $E = \{15,16,24,26,34,35,36,17,18,19,27,29,37,38,48,49,57,58,59,67,68\}$, with 21 edges. Here ij denotes {i,j}, rather than multiplication.

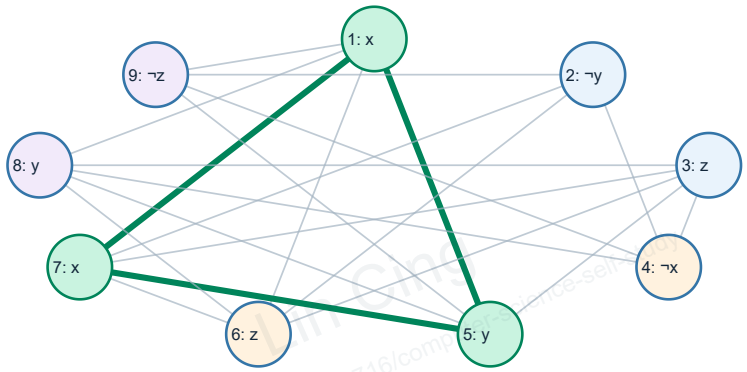
For x=y=z=true, choose {1,5,7}. Edges 15, 17, and 57 all exist, giving a 3-clique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Draw the complete graph of a concrete formula (continued)

**3SAT → CLIQUE, all 9 vertices and 21 edges**

F = (x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ z) ∧ (x ∨ y ∨ ¬z)
C1 = {1,2,3} C2 = {4,5,6} C3 = {7,8,9}




Green {1,5,7} is a 3-clique. No internal or complementary edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Forward proof, assignment to clique

Assume F is satisfiable and fix a satisfying assignment. Choose one true literal occurrence from every clause. These choices give a set S of m vertices.

Any two chosen vertices belong to different clauses. Their literals are both true, so they cannot be complements x and ¬x. The construction therefore joins them by an edge.

Every selected pair is adjacent. Thus $|S|=m$ and S is an m-clique. This proves $F \in 3SAT$ implies $\langle G,m\rangle \in CLIQUE$. The construction does not use the assignment; the correctness proof does.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Reverse proof, clique to assignment

Assume G has a clique S of size m. With no edges within a clause group, S contains at most one vertex from each group. There are m vertices and m groups, so S contains exactly one from each.

Every selected pair is adjacent, so selected literals contain no complementary pair. Set variables of selected positive literals to true and those of selected negative literals to false. No variable receives conflicting requirements. Assign all remaining variables arbitrarily.

Each clause contains its selected true literal, so F is satisfiable. This proves $\langle G,m\rangle \in$ CLIQUE implies $F \in$ 3SAT and completes the equivalence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Keep cost and boundary conventions consistent

Input length N is at least proportional to the number of occurrences. For m three-literal clauses, the graph has 3m vertices and at most $9 \cdot m(m-1)/2$ edges. Pair comparisons, including bit comparisons of variable identifiers, and writing the edge identifiers take $O(N^2 \log(N+1))$ bit operations; an explicit edge list has $O(m^2 \log(m+1))$ bits, also polynomial in N.

| Boundary | Convention and reason |
|---|---|
| m=0 | The empty conjunction is true; the empty graph has a 0-clique |
| Opposite literals across clauses | Omit the edge to exclude conflicting assignments |
| A clause contains x and ¬x | Still no internal edges; select only one true occurrence |
| Repeated literal | Keep separate occurrence vertices |
| Malformed 3CNF | Output the fixed no instance ⟨empty graph,1⟩ |

If CLIQUE permits only $k \geq 1$, map m=0 to the fixed yes instance $\langle K_1,1\rangle$ instead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Now conclude CLIQUE is NP-complete

1. Use the known NP-completeness of 3SAT.
2. Construct a polynomial-time total f and prove F satisfiable iff f(F) has an m-clique.
3. Thus 3SAT $\leq$p CLIQUE. Transitivity makes CLIQUE NP-hard.
4. The vertex-list certificate and verifier establish CLIQUE $\in$ NP.
5. Together these prove that CLIQUE is NP-complete.

If a polynomial-time CLIQUE algorithm were found, reductions would put every NP language in P. Together with P $\subseteq$ NP this would give P=NP. This conditional theorem does not prove P$\neq$NP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Use a decision oracle to recover value and certificate

For a fixed graph G, the predicate that a clique of at least k vertices exists is monotone in k. Binary search on [0,n] finds $\omega(G)$ with O(log(n+1)) decision calls. This is a multiple-call solver procedure, rather than the single many-one mapping above.

Given a graph with a k-clique, try deleting each vertex v. Delete v when G−v still has a k-clique; otherwise retain it. After at most n calls the remaining graph is an inclusion-minimal yes instance and has exactly k vertices, which form a certificate.

If a vertex outside any chosen k-clique remained, it could still be deleted. A vertex that was indispensable cannot become dispensable after other vertices are deleted. Subgraph encodings do not grow. Thus a polynomial-time decision solver gives polynomial-time certificate recovery here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## From a verifier to the first NP-complete problem

Given a Boolean circuit $C$, CIRCUIT-SAT asks whether some input $y$ makes $C(y) = 1$. It is in NP: use the input assignment as a witness and evaluate gates in topological order in polynomial time.

For NP-hardness, fix an arbitrary language $L \in \mathrm{NP}$ and a deterministic polynomial-time verifier $V$ with polynomial bounds $p, T$ such that $x \in L \iff \exists y \in \{0,1\}^{p(|x|)} : V(x, y) = 1$. A length field and padding encode witnesses of shorter lengths.

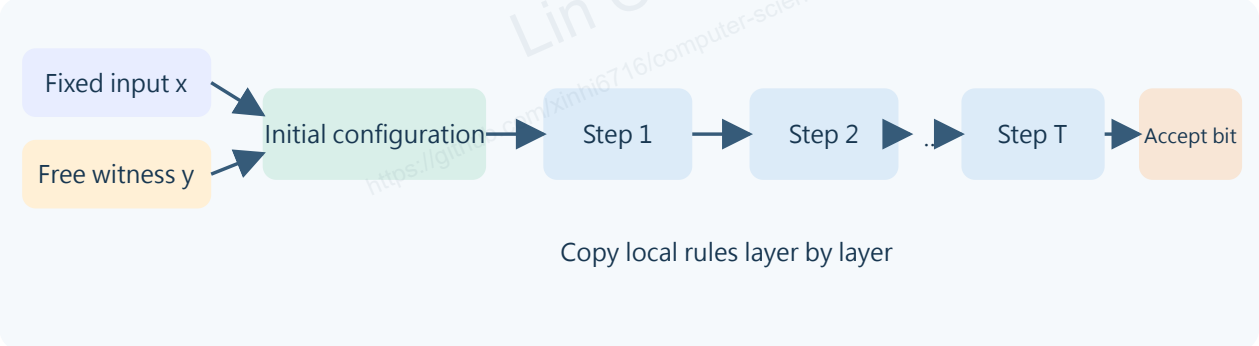
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



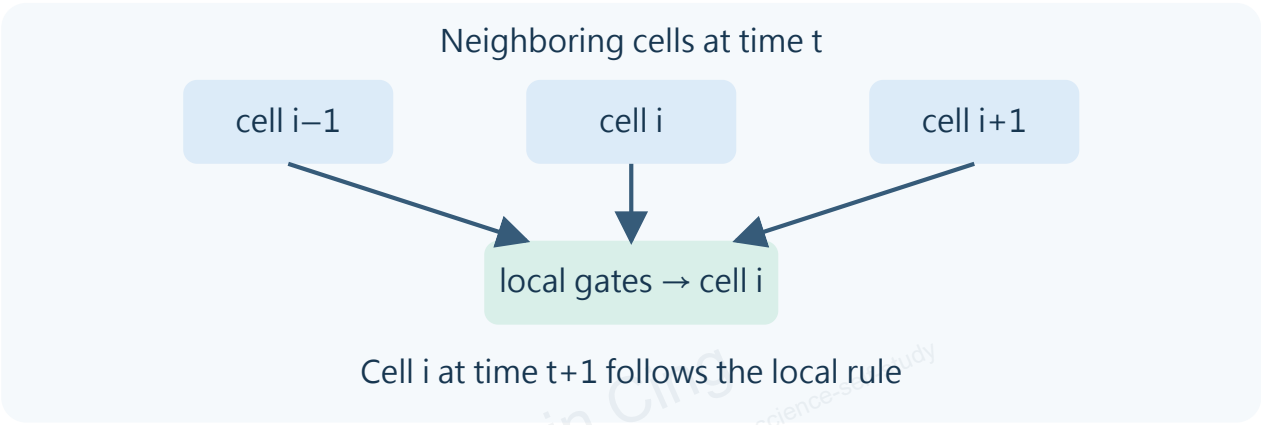
## Unroll one execution into a circuit

Fix $x$ and let $n = |x|$. Use one fixed encoding for all inputs. A configuration stores memory, head positions, and control state in $s(n) = \mathrm{poly}(n)$ bits. The verifier runs at most $T(n) = \mathrm{poly}(n)$ steps. Make the accepting state absorbing, so running exactly $T(n)$ steps preserves the answer.

In a single-tape Turing-machine model, each cell's next state depends only on itself and a fixed neighborhood. The finite local transition table has a constant-size Boolean gate implementation. Copy this circuit for every cell and every step, obtaining $O(T(n)s(n))$ gates. Other reasonable machine models must first be simulated with polynomial overhead; an arbitrary RAM operation is not automatically a constant-size gate.



## How local updates form the next layer




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Correctness by induction over configurations

The initial layer hardwires every bit of $x$, leaves the $p(n)$ witness bits as free inputs, and supplies blank cells and the initial control state. When witness lengths vary, the fixed-length encoding contains a length field and rejects invalid encodings.

For any free input $y$, layer 0 equals the initial configuration of $V(x, y)$. If layer $t$ equals its actual configuration after $t$ steps, the local gates implement the same transition and produce the configuration at step $t + 1$. By induction, the final output bit satisfies $C_x(y) = 1 \iff V(x, y) = 1$. Thus a satisfying circuit input exists exactly when $x \in L$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why the reduction takes polynomial time

Once $L$ is fixed, its verifier, local rules, constants, and polynomial bounds are fixed too; the constructor may encode them. Given $x$, it computes $T(n), s(n)$, emits each layer of gates, and hardwires $x$. Its running time and output length are polynomial in $n$. This is a uniform construction, not merely a claim that a small circuit exists for each input length.

Therefore every $L \in NP$ has $L \leq_p \text{CIRCUIT-SAT}$. Together with membership, CIRCUIT-SAT is NP-complete. Wires may be shared; an exponentially large formula tree need not be produced. The reduction outputs the circuit, not a precomputed witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Circuit-to-SAT gate constraints

Give each wire value a variable, sharing the same gate-output variable where needed. Require output $z_{out}$ by a unit clause. Encode every gate equivalently:

| Relation | CNF constraints |
|---|---|
| z=u AND v | $(\neg z \vee u) \wedge (\neg z \vee v) \wedge (z \vee \neg u \vee \neg v)$ |
| z=u OR v | $(z \vee \neg u) \wedge (z \vee \neg v) \wedge (\neg z \vee u \vee v)$ |
| z=NOT u | $(z \vee u) \wedge (\neg z \vee \neg u)$ |

For AND, the first two clauses force both inputs true if z is true; the last forces z true if both inputs are true. OR and NOT follow by the same two-direction truth cases. Fix constant wires with unit clauses.

An accepting circuit input extends to all computed wire values and satisfies the constraints. Conversely, gate equivalences force every wire to equal its actual value in topological order, and the output unit clause forces acceptance. At most three clauses per gate give linear total size in gates and wires. Repeatedly expanding shared subcircuits into formula trees could instead cause exponential growth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## SAT to three-variable clauses

Parse a formula into unary NOT and binary AND/OR nodes. Replace implication by $\neg a \vee b$. Other fixed-arity connectives have fixed-size truth-table gadgets. Give each internal node a new variable, apply the gate clauses, and require the root true. Remove repeated literals within a clause. Delete clauses containing both a literal and its negation, since they are tautologies.

Clauses now have at most three literals. To require exactly three distinct variables per clause, use fresh padding variables for each clause:

- Replace $(a \vee b)$ by $(a \vee b \vee p) \wedge (a \vee b \vee \neg p)$. If the original is true, any p works. If false, the pair forces both p and its negation.
- Replace unit a by the four clauses $(a \vee p \vee q), (a \vee p \vee \neg q), (a \vee \neg p \vee q), (a \vee \neg p \vee \neg q)$. If a is false, every p,q assignment falsifies one clause. If true, all pass.
- An empty clause is false, so output the padded version of $z \wedge \neg z$. An empty conjunction is true, so output one fixed satisfiable three-variable clause.

Original assignments extend through parse-node values. Conversely, the tree constraints recover node semantics. Padding preserves each clause requirement. This is equisatisfiability, not literal equality of functions after adding variables. Constant clauses per node and at most four per padding step give polynomial size and construction.



## Splitting a long clause

For $l_1 \vee \cdots \vee l_r$, $r > 3$, a chain also works:

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

If all original literals are false, the first clause forces y1 true, each middle clause forces the next y true, and the last forces the final y false, a contradiction. If some literal is true, let yj indicate that the first j+1 literals are all false. Every chain clause then holds. This uses r-3 new variables and r-2 clauses. Unrestricted distributive expansion of a formula into CNF may be exponentially large.

For $F = (a \vee b) \wedge \neg a$, set $u = a \vee b, v = \neg a, z = u \wedge v$, add their gate clauses, and require z. Input a=false,b=true extends to u=v=z=true. If a=true, the NOT gadget forces v=false while z and AND require v=true, excluding a false acceptance.

The book's three-CNF definition uses three distinct literals. The main reductions here can further normalize to three distinct variables. Deduplication and padding also handle repeated literals before the occurrence construction.

## Clique, independent set, and vertex cover

An independent set has no edge between its members. A vertex cover touches every edge. For a loopless undirected n-vertex graph, its complement has edge uv exactly when u≠v and G lacks that edge. Thus a set is a clique in G iff it is independent in its complement.

If S is independent in H, every H edge has an endpoint outside S, so $V \setminus S$ is a vertex cover. Conversely, a cover $V \setminus S$ forbids any edge with both endpoints in S, so S is independent. For $0 \le k \le n$,

$$\mathrm{CLIQUE}(G, k) \iff \mathrm{VC}(\overline{G}, n - k).$$

The reductions $(G, k) \mapsto (\overline{G}, n - k)$ and $(H, \ell) \mapsto (\overline{H}, n - \ell)$ both take $O(n^2)$ pair checks. A clique of at least k contains one of exactly k. A cover of at most n-k can be enlarged to exactly n-k. Handle out-of-range thresholds with fixed yes/no instances rather than negative certificate sizes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A complement example

For vertices {1,2,3} and edges {12,23}, the set {1,2} is a size-2 clique. The complement has only edge 13. The same set is independent there, and its complement {3} covers the only edge, with size 3-2=1.

The transformation is to VC on the complement, with threshold n-k. Complementation is polynomial and reversible, giving reductions in both directions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Vertex cover to Hamiltonian cycle

A Hamiltonian-cycle certificate is a permutation of all vertices with valid consecutive edges and closure. This proves NP membership. For hardness, build an undirected gadget for every original edge {u,v}. Crossings in its drawing are not vertices, and no planar crossing gadget is required.

Create twelve vertices a1,...,a6 and b1,...,b6, where ai=[u,v,i] and bi=[v,u,i]. Add fourteen internal edges. Ten form the two chains a1-a2-a3-a4-a5-a6 and b1-b2-b3-b4-b5-b6. Four cross links are a1-b3, a3-b1, a4-b6, a6-b4.

Only a1,a6,b1,b6 may connect outside the gadget. The other eight vertices need two internal edges in any final cycle. In particular a2,a5,b2,b5 already have degree two in the gadget, forcing their eight adjacent chain edges.

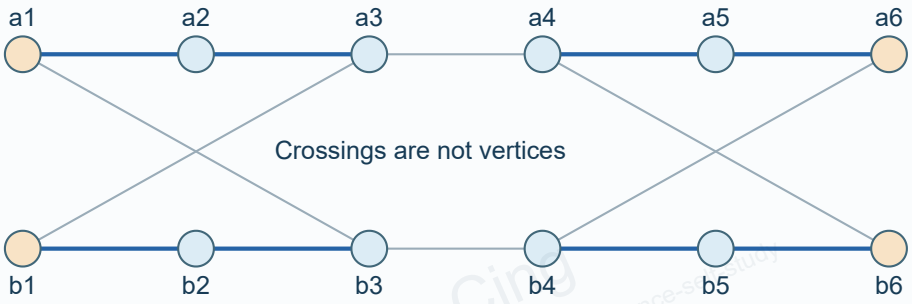
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Vertex cover to Hamiltonian cycle (continued)

HC gadget: only a1,a6,b1,b6 connect outside

$a_i=[u,v,i]$; $b_i=[v,u,i]$




Eight blue edges are forced by degree 2 at a2,a5,b2,b5. Six gray edges remain undecided. Middle edges a3−a4 and b3−b4 force the four cross links. Orange marks the four ports.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three legal gadget covers

The remaining choices are the middle edges a3-a4 and b3-b4. Once these are selected or omitted, degree two at internal vertices forces every cross link.

| Middle edges selected | Cover of all twelve vertices |
|---|---|
| Both | Two disjoint six-vertex paths, a1 through a6 and b1 through b6 |
| Only b3-b4 | One path a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6 |
| Only a3-a4 | The symmetric full path from b1 to b6 |
| Neither | Two closed six-vertex cycles with no way to join the external cycle, forbidden |

These cases exhaust all internal degree configurations. A legal Hamiltonian cycle uses one of the first three. It cannot enter an a port and leave through a b external chain. It must cover every gadget vertex. Locally, at least one endpoint of the original edge must be selected.



## Middle edges force external port use

Let x indicate selection of a3-a4 and y selection of b3-b4. Vertex a3 already uses forced edge a2-a3, so a3-b1 has indicator 1-x. Similarly a4-b6 has 1-x. Edges a1-b3 and a6-b4 each have 1-y.

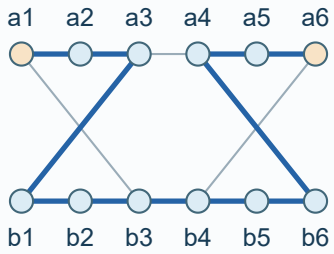
Thus a1,a6 have internal degree 2-y and external degree y. Ports b1,b6 have external degree x. Both ends on one original-vertex side connect externally together or neither does.

For x=y=0, the cycles are a1-a2-a3-b1-b2-b3-a1 and a4-a5-a6-b4-b5-b6-a4. Every vertex already has degree two, so neither cycle connects to a selector. They cannot form the whole graph's single Hamiltonian cycle. The three remaining full path covers are shown below.

## Middle edges force external port use (continued)

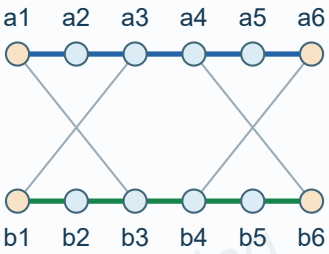
All three legal modes preserve the original-vertex side

u side only: 12 vertices



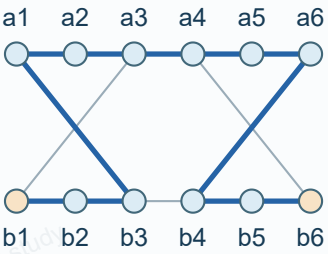

x=0, y=1

6 vertices on each side




x=1, y=1

v side only: 12 vertices




x=1, y=0

Gray edges are unused. Blue/green show paths. Each orange port needs one external edge for degree 2.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Incident chains and selectors

VERTEX-COVER asks for size at most k. Remove isolated vertices without changing the minimum cover. If k<0, or k=0 with an edge present, output a fixed no Hamiltonian-cycle instance. If no edges remain, or k is at least the remaining vertex count, output a fixed yes triangle. The main case has no isolated vertices and $1 \leq k < |V|$. A smaller cover can be enlarged to exactly k nonisolated vertices.

For each original u, order its neighbors u(1),...,u(d) arbitrarily. Connect its gadget sides by edges $[u, u(i), 6] - [u, u(i+1), 1]$, i=1,...,d-1. An external path selecting u must follow all of u's incident gadgets from one chain endpoint to the other.

Add k selectors s1,...,sk. Each selector connects to both ends of every u chain, with no other external edges. There are $12|E| + k$ vertices, $14|E|$ internal edges, at most $2|E|$ chain links, and $2k|V|$ selector links. The size is polynomial. Both source and target graphs are undirected.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Why activation propagates along the entire chain

Call a u-side **active** when both of its ports use an external edge. Each interior port of an ordered u-chain has exactly one external edge available: the link to the adjacent u-side. If side i is active, it uses that link; the adjacent side's port then uses an external edge, so its other port must also do so because the two ports on one side are used together. Induction in both directions activates every incident gadget of u, including both chain endpoints. If one side is inactive, the same argument rules out any active side on that chain.

The two chain endpoints have only selector edges outside their gadgets. An active chain must therefore use one selector edge at each end. Its local paths connect its two u-ports, so following the cycle cannot switch to another original vertex before reaching a selector. An inactive side is nevertheless visited: its edge gadget has at least one active side, and when just the other side is active, that side's 12-vertex path covers every vertex, including the inactive side.



## Why activation propagates along the entire chain (continued)

Example: u–v–w, k=1, cover={v}

In this cover, v is active on both sides; U.b6 → V.a1 connects them

Unused u and w sides are visited inside the full 12-vertex paths

## Both directions of vertex cover to cycle

Given a cover $C = \{u_1, \ldots, u_k\}$, make one path through each selected vertex's incident chain. If only one endpoint of an original edge is selected, use that side's full twelve-vertex path. If both endpoints are selected, use the two disjoint six-vertex paths. Every edge has a selected endpoint, so every gadget vertex is visited exactly once. Join s1, the u1 chain, s2, the u2 chain, ..., sk, the uk chain, and back to s1. This visits every selector once and gives one Hamiltonian cycle.

Conversely, propagation along incident links makes each active u-side activate the entire u-chain, which connects two selectors and stays on the same original vertex. Select every active u. Each selector has cycle degree two and every selected chain consumes two selector incidences. Hence exactly k chains are active, including k=1 where both chain ends connect to the same selector by distinct edges. A chain cannot occur twice because it has only one pair of endpoints. In each edge gadget the zero-active case forms two isolated cycles, so at least one endpoint chain is active. The selected k vertices therefore cover every original edge.

Therefore VC(G,k) holds iff the constructed graph has a Hamiltonian cycle. Construction, decoding, and size are polynomial. Fixed yes/no instances handle the preliminary cases without inventing empty chains for selectors.



## A full twenty-five-vertex cycle

For original path u-v-w with k=1, choose cover {v}. Build U=$W_{uv}$ and V=$W_{vw}$, each with twelve vertices, and one selector. The b side of U and a side of V represent v. Connect U.b6 to V.a1 and the selector to the two ends of the v chain.

Use these full paths, with every point prefixed by its gadget:

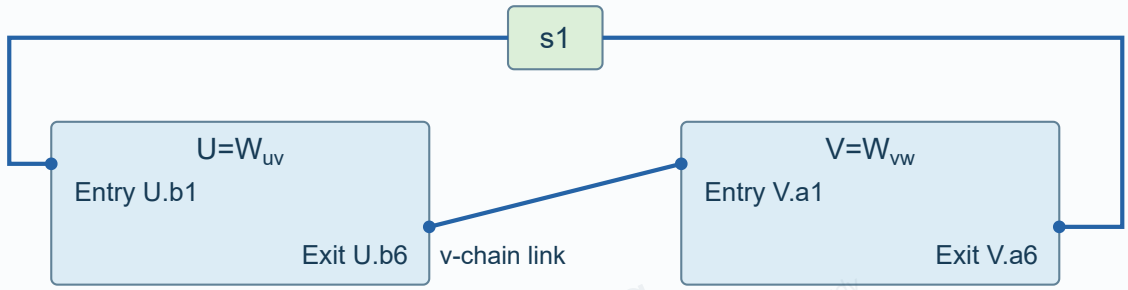
- U: b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6.
- V: a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6.

The complete cycle starts s1→U.b1, follows all of U, takes U.b6→V.a1, follows all of V, and returns V.a6→s1. Same-named vertices in different gadgets are distinct. Exactly 24 gadget vertices plus one selector are visited once. Every adjacency is an internal edge, chain link, or selector edge.

Both original edges have only v selected. In U, v is the b-side (x=1,y=0); in V, v is the a-side (x=0,y=1). The link U.b6–V.a1 forces both sides active; the other u and w sides are inactive, but their vertices appear inside these two full paths. Choosing only u would leave $W_{vw}$ uncovered and cannot create a cycle.

## A full twenty-five-vertex cycle (continued)

Original path u−v−w, cover={v}: one selector closes a 25-vertex cycle




Each box represents its full 12-vertex path. U and V vertices are distinct.

This shows the selected HC. All external edges follow the selector and incident-chain rules.



## Hamiltonian cycle to TSP decision

Given an undirected graph with $n \geq 3$, build complete graph $K_n$. Give original edges weight 1 and missing edges weight 2. Set threshold B=n. Construction takes $O(n^2)$.

A Hamiltonian cycle uses n original edges and becomes a tour of cost n. Conversely, every tour has n edges of weight at least 1. Total cost at most n forces every edge to have weight 1, so every tour edge belongs to the original graph. It is a Hamiltonian cycle there.

CLRS uses weights 0 for original edges and 1 for missing edges with threshold 0. This teaching variant adds 1 to every edge: every n-edge tour gains exactly n, so its threshold becomes n and the yes/no answer is unchanged.

Thus HC(G) iff TSP(Kn,w,n). TSP decision is in NP. Its certificate is a vertex permutation, checked for uniqueness, closure, and total cost.

## TSP trace and small graphs

For a triangle with edges 12,23,31, all three weights are 1. Tour 1→2→3→1 costs 3 and meets threshold B=3. Removing original edge 31 leaves that edge in the complete target graph with weight 2. Every three-point tour then costs 1+1+2=4>3, matching the absence of an original Hamiltonian cycle.

Simple undirected graphs with n<3 have no Hamiltonian cycle under this chapter's convention. Map them to a fixed no TSP instance. The n-edge argument above assumes n≥3.

Optimization TSP returns a minimum cost and, when requested, a tour. A polynomial exact optimizer could decide the threshold problem by comparison, so hardness of decision also makes exact optimization hard unless P=NP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Subset-sum membership and dynamic programming

Decision SUBSET-SUM asks whether some subset of given positive integers sums exactly to target t. A certificate lists selected indices. Check bounds, nonrepetition, and exact sum in polynomial time, proving membership in NP.

Let dp[i,s] mean that a subset of the first i integers reaches s. Then

$$dp[i, s] = dp[i - 1, s] \vee (a_i \leq s \wedge dp[i - 1, s - a_i]).$$

The $O(nt)$ algorithm is pseudopolynomial with binary t. The reduction below uses large values with polynomial bit length, consistently with that DP limitation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three-CNF-SAT to a digit table

Normalize to n appearing variables and k clauses, each with three distinct variables and no complementary tautology. Deduplication and padding above achieve this. Map constant true or false formulas separately to fixed yes/no integer instances.

Create n+k decimal columns, first for variables and then clauses. These columns compactly define integers. The final delivered encoding is binary.

- For each xi, create Ti and Fi. Both have 1 in variable column i and 0 in other variable columns. Ti has 1 in each clause containing xi; Fi has 1 in each clause containing ¬xi. Other clause digits are zero.
- For each clause j, add two slack numbers, with digits 1 and 2 respectively in that clause column and zero elsewhere.
- The target has 1 in each variable column and 4 in each clause column.

There are 2n+2k positive numbers, each with n+k decimal digits and $O(n + k)$ binary bits. Total output length is $O((n + k)^2)$, and construction is polynomial. If the SUBSET-SUM input format requires a set rather than a multiset, the nondegenerate premises ensure Ti and Fi differ: each variable appears, and positive/negative occurrences cannot coincide in a clause. Different variables have different high columns, and slack columns also distinguish the slack numbers.



## No carry for every candidate subset

An arbitrary subset sums each variable column to at most 2. A clause has three distinct literals, so all truth-value numbers together contribute at most three 1s in its column. Slack adds at most 1+2=3. Thus every clause sum is at most 6<10. **No subset can carry**, not only a correctly constructed witness.

If the formula is satisfiable, choose exactly Ti or Fi according to each variable's value. Variable columns become 1. Each clause has r true literals, r=1,2,3. Fill its deficit 4-r by slack 1+2, 2, or 1, respectively. All columns reach the target.

Conversely, any subset reaching the target has exactly one of Ti,Fi per variable, because no carries can alter its variable column. These choices define a consistent assignment. Each clause target is 4 but its slack contributes at most 3, so some chosen truth-value number contributes 1 there. The corresponding literal is true. Every clause is satisfied.

The two implications and polynomial size prove NP-hardness. Together with NP membership, SUBSET-SUM is NP-complete.

## A hand-checkable no-carry sum

Use $F = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Column order is x1,x2,x3,C1,C2.

| Numbers | Decimal column representation |
|---|---|
| T1,F1 | 10010, 10001 |
| T2,F2 | 01011, 01000 |
| T3,F3 | 00110, 00101 |
| S1,S1′ | 00010, 00020 |
| S2,S2′ | 00001, 00002 |
| Target | 11144 |

Assignment (true,true,false) chooses T1+T2+F3=11122. Each clause has two true literals. Add S1′=20 and S2′=2 to reach 11144. No carries occur. Each high digit 1 confirms one truth-value number per variable.

Values can be of order $10^{n+k}$, while binary length is only $O(n+k)$. The reduction has polynomial output even though exact $O(nt)$ DP is not polynomial in bit length.



## A hand-checkable no-carry sum (continued)

T1+T2+F3=11122; add slack 00022 to reach target 11144

| | x1 | x2 | x3 | C1 | C2 | |
|---|---|---|---|---|---|---|
| T1 | 1 | 0 | 0 | 1 | 0 | Each variable digit is 1 |
| T2 | 0 | 1 | 0 | 1 | 1 | Each clause has 2 true literals |
| F3 | 0 | 0 | 1 | 0 | 1 | |
| slack | 0 | 0 | 0 | 2 | 2 | Slack 2 fills each to 4 |
| target | 1 | 1 | 1 | 4 | 4 | |

## Why universal no-carry matters

Proving no carry only for satisfying-assignment selections would not establish the reverse direction. A wrong subset might otherwise forge target digits through carries. Here every subset has variable digits at most 2 and clause sums at most 6, so no carry can occur for any candidate.

Converting an n+k digit decimal number to binary by repeated multiplication by 10 and digit addition has a simple $O((n+k)^2)$ bit implementation. Performing it for all 2n+2k numbers is still polynomial. Verification sums at most $O(n+k)$ numbers of $O(n+k)$ bits, adding only $O(\log(n+k))$ bits to the sum.

For assignment (false,false,false) in the example, the two clauses have 0 and 2 true literals. Even both slack numbers give only 3 in the first clause column, below 4. No carry from another column can conceal the failure.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A guarantee quantifies over every input

An NP-hard optimization problem can still admit a polynomial-time approximation. Each algorithm must return a feasible solution and prove its relation to an unknown OPT on every allowed input. Good performance on one instance is not a ratio proof.

We use matching and MST lower bounds, element charges, and trimmed dynamic programming. Prerequisites are greedy algorithms, MSTs, induction, and binary input length.

Sources are CLRS 3e § § 35.1–35.3 and 35.5, MIT 2004 lecture 24, and handwritten notes PDF 54–56.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Minimization and maximization reverse the bound

Let A be a feasible value, OPT the optimum, and $\rho \geq 1$. For positive objectives, $\max(A/OPT, OPT/A) \leq \rho$ unifies both directions.

| Problem | Feasibility | Guarantee |
|---|---|---|
| Minimize | $OPT \leq A$ | $A \leq \rho OPT$ |
| Maximize | $A \leq OPT$ | $A \geq OPT/\rho$ |

Minimization values OPT=10, A=15 and maximization values OPT=15, A=10 both give ratio 1.5. For maximization, $A \leq \rho OPT$ says almost nothing. Handle OPT=0 with multiplicative inequalities rather than division. Subset sum returns 0 if no positive item fits.

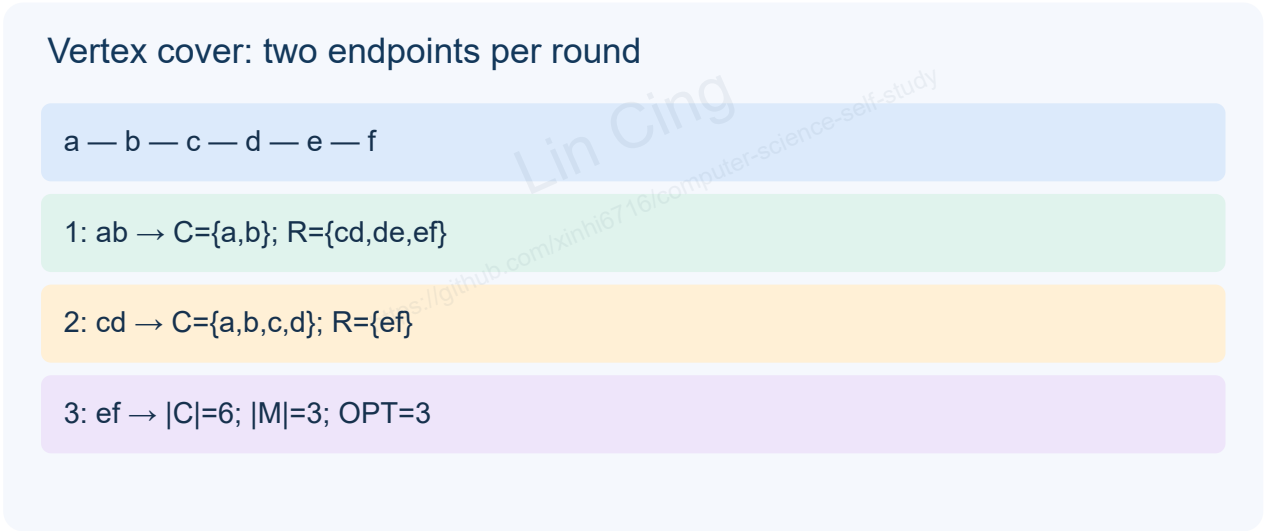
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Vertex cover selects both endpoints

For an undirected graph $G=(V,E)$, a vertex cover C touches every edge; minimize $|C|$. Initialize $C=\varnothing$ and $R=E$. While R is nonempty, choose $uv \in R$, add u and v to C, and delete every edge incident to either endpoint.

Every deleted original edge has an endpoint in C, so the final C is feasible. A single edge scan can build a maximal matching and return all its endpoints in $O(|V|+|E|)$ time. This algorithm does not directly solve weighted vertex cover.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A round trace exposes the certificate

Use E={ab,bc,cd,de,ef} and choose ab, cd, ef in this order.

| Round | Edge | Added vertices | Remaining edges deleted | Edges left |
|---|---|---|---|---|
| 1 | ab | a,b | ab,bc | cd,de,ef |
| 2 | cd | c,d | cd,de | ef |
| 3 | ef | e,f | ef | ∅ |

M={ab,cd,ef} is a matching and C contains all six vertices. The cover {b,d,f} has size 3; the three disjoint edges force any cover to use at least 3 vertices. Thus OPT=3 and A/OPT=6/3=2, attaining the guarantee.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The factor 2 comes from a matching lower bound

Let M contain the selected edges. Selecting uv deletes every edge incident to u or v, so future selected edges share neither endpoint. Thus M is a matching.

Any cover C* must cover each edge of M. These edges have disjoint endpoints, so they require distinct cover vertices. Hence $|C^*| \geq |M|$. The algorithm takes exactly two new endpoints per selected edge, giving $|C| = 2|M| \leq 2|C^*|$.

M is maximal because no edge can be added at termination; it need not be maximum. On a-b-c-d, {bc} is maximal of size 1, while {ab,cd} is maximum of size 2. Taking only one arbitrary endpoint per matching edge can leave other edges uncovered.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Metric TSP specifies the graph and costs

The input is a complete undirected graph with at least three vertices and nonnegative symmetric costs satisfying $c(u,w) \le c(u,v)+c(v,w)$ for every triple. A feasible Hamiltonian tour visits each vertex once and returns to its start.

Compute an MST T and root it anywhere. A full DFS walk W traverses each tree edge once in each direction. Keep the order of first visits and close the sequence to obtain H.

W revisits the root and branch vertices, so it is a closed walk rather than a feasible TSP tour. Completeness ensures shortcut edges exist.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## From a tree walk to a numerical tour

Use Euclidean distances on A=(0,0), B=(1,0), C=(1,1), D=(0,1). An MST uses AB, BC, CD and costs 3.

The full walk A,B,C,D,C,B,A costs 6. Keeping first visits and closing gives A,B,C,D,A of cost 4. Every Hamiltonian cycle has four edges of length at least 1, so $OPT \ge 4$; the displayed tour attains 4, proving OPT=4.

### Unit square: walk and tour

A(0,0), B(1,0), C(1,1), D(0,1)

T={AB,BC,CD}; c(T)=3

W=A,B,C,D,C,B,A; c(W)=6

H=A,B,C,D,A; c(H)=4=OPT

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The three steps of the double-tree proof

Delete any edge from an optimal tour $H^*$ to obtain a spanning tree. Nonnegative costs prevent this deletion from increasing cost. MST optimality gives $c(T) \leq c(H^*)$.

$W$ traverses each tree edge twice, so $c(W)=2c(T)$. Between consecutive retained vertices, let the original walk segment be $u=v_0,v_1,\ldots,v_k=w$. Repeated triangle inequalities give $c(u,w) \leq \sum_j c(v_{j-1},v_j)$. These segments partition $W$, so $c(H) \leq c(W)$.

Therefore $c(H) \leq c(W)=2c(T) \leq 2\mathrm{OPT}$. Dense Prim takes $O(n^2)$; walking and retaining first visits take $O(n)$, for $O(n^2)$ overall.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A shortcut can be more expensive without metric costs

Let $AB=BC=CD=DA=1$ and $AC=BD=M>4$ in a complete four-vertex graph. An MST can use $AB$, $AD$, $BC$ rooted at $A$. Visiting $D$ before $B$ gives preorder $A,D,B,C$ and tour cost $2+2M$. The perimeter tour is optimal with cost 4.

The full walk $A,D,A,B,C,B,A$ costs 6. Replacing $D \to A \to B$ by $D \to B$ raises cost from 2 to $M$; replacing $C \to B \to A$ by $C \to A$ also raises cost. The ratio $(2+2M)/4$ is unbounded. At $M=10$ it is $22/4=5.5$.

This defeats the double-tree guarantee. The stronger impossibility of every constant approximation unless P=NP requires a separate gap reduction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Set cover chooses the most newly covered elements

Given universe U and family F with $\cup F=U$, choose as few sets as possible to cover U. Repeatedly select S maximizing $|S\cap R|$, where R is uncovered, until $R=\varnothing$. Break ties alphabetically.

Let U={1,…,6}, A={1,2,3,4}, B={1,2,5}, C={3,4,6}. Greedy selects A for four new elements, B for element 5, and C for element 6. It uses 3 sets. $B\cup C=U$ and no set covers U alone, so OPT=2.

### Charges at first coverage

A={1,2,3,4}; B={1,2,5}; C={3,4,6}

A → 1,2,3,4: each 1/4

B → 5: 1; C → 6: 1

Total=3; OPT=2; H4=25/12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Split each unit cost among newly covered elements

If a round covers k new elements, charge each $c_x=1/k$. Each element is charged once, so the greedy set count is $|G|=\Sigma_x c_x$.

In the example, $c_1=c_2=c_3=c_4=1/4$ and $c_5=c_6=1$. The total is $4\times 1/4+1+1=3$. Set B carries charge 3/2, as does C.

For any optimal cover F*, each element occurs in at least one chosen set. Nonnegative charges imply $\sum_x c_x \le \sum_{S\in F^*}\sum_{x\in S} c_x$. Counting overlap twice only increases the right side. It remains to bound the charge inside any one set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The complete local harmonic charging argument

Fix any $S \in F$ with $s = |S|$. Order its elements $x_1, \ldots, x_s$ by first coverage, breaking ties within a round arbitrarily.

At the start of the round covering $x_j$, at least $s - j + 1$ elements of $S$ remain uncovered. Earlier elements from the same round only make this lower bound weaker. The greedy set covers $k \geq |S \cap R| \geq s - j + 1$ new elements, since otherwise $S$ would be a better choice.

Thus $c_{x_j} = 1/k \leq 1/(s - j + 1)$. Summing gives $\sum_{x \in S} c_x \leq \sum_{j=1}^{s} 1/(s - j + 1) = H_s$. This applies to every candidate set, including the sets in the optimal cover.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Local charges yield a global approximation ratio

If $U = \varnothing$, greedy and optimal covers are empty and OPT=0. The cost inequality holds; do not evaluate 0/0, a maximum over an empty family, or ln 0. For $U \neq \varnothing$, set $H_0 = 0$ and $d = \max_{S \in F} |S| \geq 1$. Combining local charges with the optimal cover gives

$|G| = \sum_x c_x \leq \sum_{S \in F^*} \sum_{x \in S} c_x \leq \sum_{S \in F^*} H_{|S|} \leq H_d |F^*|$.

Since $H_d = 1 + \sum_{j=2}^{d} 1/j \leq 1 + \int_1^d dx/x = 1 + \ln d \leq 1 + \ln |U|$, greedy is an $H_d$ approximation. Here d=4 and $H_4 = 25/12$; the bound is $3 \leq (25/12) \times 2 = 25/6$, while the actual ratio is 3/2.

A simple rescan takes $O(mN)$ per round and at most $N$ rounds, hence $O(mN^2)$ for $m = |F|$ and $N = |U|$. This proves polynomial time. Do not drop the additive 1 in the logarithmic bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why exact subset-sum lists grow exponentially

Given positive integers $x_1,\dots,x_n$ and integer target $t\geq 1$, maximize a subset sum at most $t$. The empty subset is allowed. Exact states satisfy $P_0=\{0\}$, $P_i=P_{i-1}\cup(P_{i-1}+x_i)$, discarding values above $t$.

Every state corresponds to an actual subset, but there can be $2^n$ distinct states. A Boolean $O(nt)$ DP is only pseudo-polynomial because binary $t$ has length $\Theta(\log t)$.

For $0<\varepsilon<1$, set $\delta=\varepsilon/(2n)$. Approximate lists retain separated representatives and aim for $\mathrm{OPT}/(1+\varepsilon)\leq \max L_n\leq \mathrm{OPT}$. Return 0 directly if $n=0$ or no item fits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Trimming keeps smaller feasible representatives

In a sorted deduplicated list, keep the smallest value. Keep a later $y$ only when $y>(1+\delta)\mathrm{last}$, then update last; otherwise last represents $y$. Every original $y$ has a retained $z$ with $y/(1+\delta)\leq z\leq y$.

At round $i$, merge $L_{i-1}$ and $L_{i-1}+x_i$, trim, then discard values above $t$. Always retain 0. Treat zero separately when bounding positive values with logarithms.

A smaller representative remains no larger than the corresponding true sum after adding the same nonnegative item. Retained values are actual subset sums. Store predecessor states and item indices if the subset itself is required.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Trimming keeps smaller feasible representatives (continued)

Trimmed lists: $\delta=0.05$, $t=308$

$L_1=\{0,104\}$

$L_2=\{0,102,206\}$

$L_3=\{0,102,201,303\}$

$L_4=\{0,101,201,302\}$; OPT=307

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## A round-by-round trimming calculation

Take $x=(104,102,201,101)$, $t=308$, $\varepsilon=0.4$, $n=4$, $\delta=0.05$. Use exact comparisons to avoid floating-point boundary errors.

| i | Merged list | After trimming and removing >308 |
|---|---|---|
| 1 | 0,104 | 0,104 |
| 2 | 0,102,104,206 | 0,102,206 |
| 3 | 0,102,201,206,303,407 | 0,102,201,303 |
| 4 | 0,101,102,201,203,302,303,404 | 0,101,201,302 |

Round 2 removes $104 \le 102 \times 1.05 = 107.1$. Round 3 removes $206 \le 201 \times 1.05 = 211.05$. Round 4 removes $303 \le 302 \times 1.05 = 317.1$. The answer is $302 = 201 + 101$; $OPT = 307 = 104 + 102 + 101$. Thus $307/302 \approx 1.01656 \le 1.4$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## An induction tracks multiplicative error across rounds

Claim that each $y \in P_i$ with $y \le t$ has $z \in L_i$ satisfying $y/(1+\delta)^i \le z \le y$. The base case $i=0$ contains only 0.

If $y$ omits $x_i$, use its previous representative $q$. If $y=p+x_i$, a representative $q$ of $p$ satisfies $q+x_i \ge p/(1+\delta)^{i-1}+x_i \ge (p+x_i)/(1+\delta)^{i-1}$ because $x_i \ge 0$. In both cases the merged list contains $w \in [y/(1+\delta)^{i-1}, y]$.

Trimming retains $z \in [w/(1+\delta), w]$, proving the claim. Since $z \le y \le t$, capacity deletion does not remove $z$. Applying the claim to OPT yields $A \ge \mathrm{OPT}/(1+\delta)^n$.

Using $\ln(1+\delta) \le \delta$ gives $(1+\delta)^n \le e^{\{\varepsilon/2\}} \le 1+\varepsilon$. For the last inequality, $f(\varepsilon)=\ln(1+\varepsilon)-\varepsilon/2$ has $f(0)=0$ and $f'(\varepsilon)=1/(1+\varepsilon)-1/2 \ge 0$ for $0 \le \varepsilon \le 1$. Thus the multiplicative guarantee $A \ge \mathrm{OPT}/(1+\varepsilon)$ holds in all cases. If $\mathrm{OPT}=0$, feasibility gives $A=0$ and no ratio is taken. If $\mathrm{OPT}>0$, the bound implies $A>0$, so division yields $\mathrm{OPT}/A \le 1+\varepsilon$.



## An FPTAS is polynomial in bit length and 1/ε

After trimming, consecutive positive values have ratio $>1+\delta$ and lie in $[1,t]$. Therefore $|L_i| \le 2+\ln t/\ln(1+\delta)$. Since $\ln(1+\delta) \ge \delta/(1+\delta)$ and $\delta=\varepsilon/(2n)<1/2$, $|L_i| \le 2+3n \ln t/\varepsilon$.

Write $K=O(1+n \log t/\varepsilon)$. Linear merging of sorted lists takes $O(K)$ arithmetic operations per round, or $O(nK)$ overall. General sorting adds $O(\log K)$ but remains polynomial.

Binary input length $B$ includes every $x_i$, $t$, and the rational encoding of $\varepsilon$. Retained integers use $O(\log t)$ bits; merged values are at most $t+\max x_i$. Rational threshold comparisons use cross multiplication, so bit complexity is polynomial in $B$ and $1/\varepsilon$. Do not treat $\varepsilon$ as a free infinite-precision real.

A PTAS needs polynomial time in $B$ for fixed $\varepsilon$, such as $B^{\{1/\varepsilon\}}$; an FPTAS also needs polynomial dependence on $1/\varepsilon$. Trimming meets the latter requirement.

## Weighted set cover: price per new element

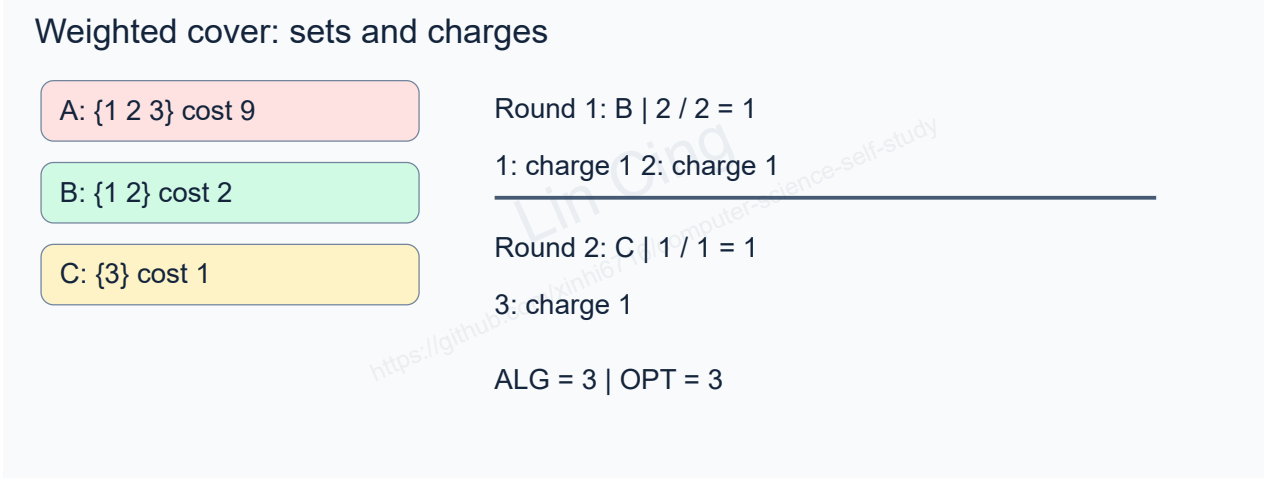
A finite universe $U$ is covered by sets with positive rational costs. Let $R$ be the uncovered elements. Repeatedly choose a set with minimum $w(S)/|S \cap R|$ among sets with a nonzero denominator. Return the empty solution for empty $U$; report infeasibility if the union misses an element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Trace and counterexample

Let $U = \{1, 2, 3\}$, $A = U$, $B = \{1, 2\}$, $C = \{3\}$, with costs 9, 2, 1. Their prices are 3, 1, 1. If $B$ is chosen first, charge each of its elements 1, then choose $C$ and charge element 3 one unit. Total cost is 3. Sets $B, C$ cost 3; any solution containing $A$ costs at least 9, so OPT=3. Choosing most new elements selects $A$ first. Replacing its cost by arbitrary $M$ makes that rule's ratio unbounded.



## Charges give a local bound

When $S$ newly covers $k$ elements, charge each $w(S)/k$. Every element is charged exactly on first coverage, so charges sum to ALG. Fix a set $A$ and order its elements $e_1, \ldots, e_t$ by first coverage. At the beginning of the round covering $e_i$, at least $t - i + 1$ elements of $A$ remain uncovered. This also holds for any ordering of elements covered in the same round.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Sum the local bound over an optimal cover

Greedy could choose $A$ in that round, hence $charge(e_i) \le w(A)/(t - i + 1)$. Summing gives $\sum_{e \in A} charge(e) \le w(A) H_{|A|}$. Let $d = \max_S |S|$. An optimal cover contains each element at least once. Charges are nonnegative, so counting overlaps again only increases the right side:

$ALG \le \sum_{A \in OPT} \sum_{e \in A} charge(e) \le H_d OPT$, where $H_d = \sum_{i=1}^{d} 1/i \le 1 + \ln d$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## MAX-3-CNF: an expected guarantee

Each clause has three distinct variables, each independently assigned a fair bit. Only one of eight assignments falsifies a clause, so it is satisfied with probability $7/8$. Let $Y$ count satisfied clauses and $m$ be the total. Linearity yields $E[Y] = 7m/8 \geq 7OPT/8$; clauses need not be independent. A single clause still fails with probability $1/8$, so this is not a guarantee for every run. Repeated literals $(a \vee a \vee a)$ have satisfaction probability only $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



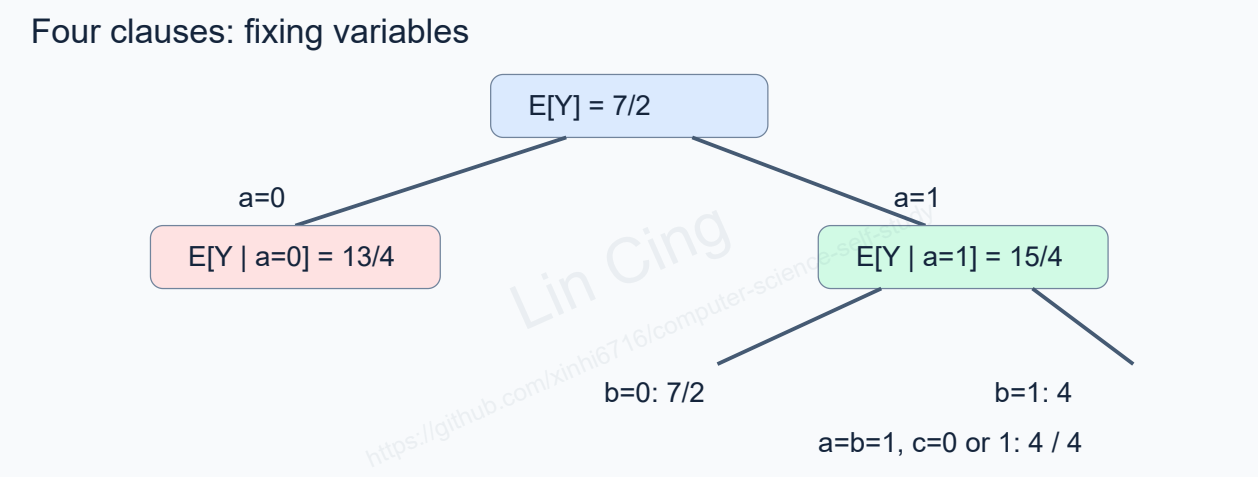
## Fix variables without decreasing conditional expectation

Under a partial assignment, an already satisfied clause contributes 1, a completely assigned false clause contributes 0, and otherwise a clause with $r$ unassigned variables contributes $1 - 2^{-r}$. The two conditional expectations after fixing the next variable to 0 or 1 average to the current expectation. Choose the larger and repeat. Once all variables are fixed, the expectation equals the actual satisfied count, at least $7m/8 \geq 7OPT/8$. Direct recomputation takes $O(nm)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Four-clause conditional-expectation trace

The clauses are $(a \vee b \vee c)$, $(\neg a \vee b \vee c)$, $(a \vee \neg b \vee c)$, $(a \vee b \vee \neg c)$. Initial expectation is $7/2$. Fixing $a = 0$ or $1$ yields $13/4$ or $15/4$, so choose 1. Next $b = 0$ or $1$ yields $7/2$ or 4, so choose 1. Either value of $c$ satisfies all four.






## Weighted vertex cover: integer model and LP

An undirected graph has nonnegative rational vertex weights. Let $x_v \in \{0, 1\}$ denote selection. Minimize $\sum_v w_v x_v$ with constraint $x_u + x_v \geq 1$ for every edge $uv$. Relax to $0 \leq x_v \leq 1$. Every integer solution remains feasible, so $LP^* \leq OPT$. A polynomial-time rational LP algorithm gives an exact optimum.

## Threshold rounding is feasible and at most twice optimal

Select every vertex with $x_v \geq 1/2$. If both endpoints of an edge were omitted, their values would sum to less than 1, contradicting its constraint. Pointwise $1_{v \in C} \leq 2x_v$; multiplying by nonnegative weights and summing gives $w(C) \leq 2LP^* \leq 2OPT$. For one edge with endpoint weights 1 and $M$, taking both matching endpoints costs $1 + M$ while OPT=1. The unweighted rule cannot be reused unchanged.

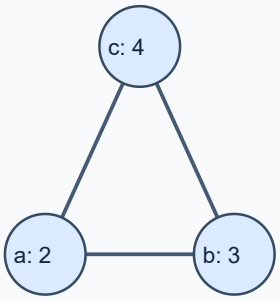
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Triangle: a lower-bound certificate and rounding

A triangle $a, b, c$ has weights 2, 3, 4. The half-valued solution costs $9/2$. Multiply the constraints for $ab, ac, bc$ by $1/2, 3/2, 5/2$ and add: every feasible LP costs at least $9/2$, so the half solution is optimal. Threshold rounding chooses all three at cost 9; the integer optimum selects $a, b$ at cost 5, for ratio $9/5$.

Triangle LP and certificate




$x_a = x_b = x_c = 1/2$

ab * 1/2 + ac * 3/2 + bc * 5/2

$2x_a + 3x_b + 4x_c \geq 9/2$

LP* = 9/2 → threshold 1/2

{a,b,c}: cost 9 | {a,b}: OPT 5

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Randomized rounding: uncovered elements

Let $N = |U| \geq 2$, costs be nonnegative, and the optimal LP satisfy $\sum_{S \ni e} x_S \geq 1$ for every element. Set $k = 3\lceil \log_2 N \rceil$ and independently choose each set with $p_S = \min(1, kx_S)$. If some set containing $e$ has $p_S = 1$, it is certainly covered; otherwise $\Pr(e \text{ uncovered}) = \prod_{S \ni e}(1 - p_S) \leq e^{-k} \leq N^{-3}$. A union bound gives probability at most $N^{-2} \leq 1/4$ that any element is uncovered.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



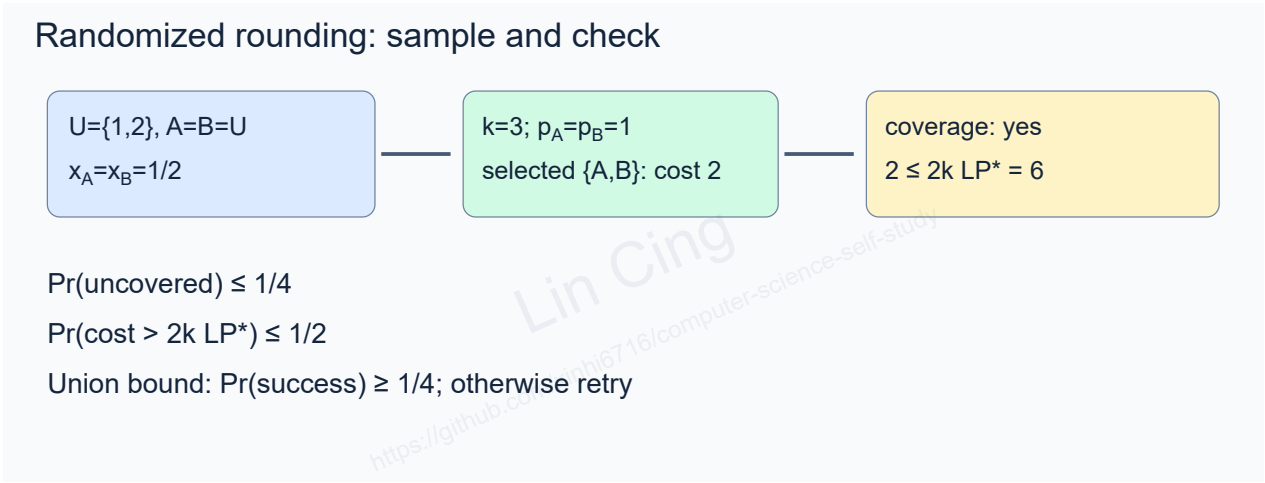
## Check cost and coverage together

Expected cost $\sum_S w_S p_S \leq kLP^*$. If $LP^* > 0$, Markov bounds the probability that cost exceeds $2kLP^*$ by $1/2$. Adding the coverage failure bound gives probability at most $3/4$ that either condition fails, so success probability is at least $1/4$. Check both and retry independently after failure. Expected trials are at most 4; successful cost is $\leq 2kOPT$. If $LP^* = 0$, cost is necessarily zero and only coverage requires retrying. For $N = 0$ return empty; for $N = 1$ choose the cheapest covering set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One sampling step

Let $U = \{1, 2\}$, $A = B = U$, and each set cost 1. The optimal LP solution $x_A = x_B = 1/2$ has $LP^* = 1$. With $k = 3$, $p_A = p_B = 1$. Both are chosen, costing 2 and covering everything, with $2 \leq 2kLP^* = 6$. This illustrates clipping at 1; it is not tight for the failure bound.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Nonmetric TSP: construct a gap

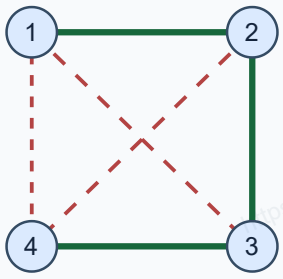
Assume a deterministic polynomial-time $\rho$-approximation for fixed $\rho \geq 1$, and choose a fixed integer $R \geq \rho$. Convert an $n$-vertex Hamiltonian-cycle input $G$ into a complete graph: original edges cost 1, nonedges cost $M = Rn + 1$. If $G$ has a Hamiltonian cycle, $OPT = n$ and the output costs at most $Rn$. Otherwise every tour uses a nonedge and costs at least $M + n - 1 > Rn$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The cost threshold decides Hamiltonian cycle

Test whether the approximation output costs at most $Rn$. This decides Hamiltonian cycle, so existence of the approximation algorithm implies $P = NP$. The binary length of $M$ is $O(\log n)$ and the complete graph has polynomial output size. For $n = 4, R = 2, M = 9$. If the original graph is a four-vertex path, each tour uses a nonedge and costs at least 12, above threshold 8. Triangle inequality can fail; this argument also does not directly rule out randomized algorithms with only expected guarantees.

Path becomes a complete graph: cost gap




n=4, R=2, M=Rn+1=9

green original edge: 1

red dashed nonedge: 9

1-2-3-4-1: cost 12

no HC: every tour ≥ 12 > 8

HC: approximation output ≤ 8



## Three-literal clauses: expectation and success probability

Let $m \geq 1$ and let each clause contain exactly three distinct variables, possibly negated. Assign variables independently and uniformly. Exactly one of eight local assignments falsifies each clause, so $\mathbb{E}[X] = 7m/8$, where $X$ counts satisfied clauses. Clauses may share variables: linearity of expectation does not require their satisfaction events to be independent.

Set $q = \Pr(X \geq 3m/4)$. Since $0 \leq X \leq m$, bound $X$ by $m$ on success and by $3m/4$ on failure:

$$\frac{7m}{8} = \mathbb{E}[X] \leq qm + (1-q)\frac{3m}{4}, \qquad q \geq \frac{1}{2}.$$

Since $OPT \leq m$, success also implies $X \geq 3OPT/4$. This is a probability lower bound, not a guarantee for every trial. The $7/8$ premise may fail for unit clauses or repeated variables.

## Checkable retries and a small example

Run $r$ independent trials and retain the assignment satisfying the most clauses. The chance of failing to reach $3m/4$ is at most $2^{-r}$. Each trial can be checked by counting, without knowing OPT. For the single clause $(a \vee b \vee c)$, seven of eight assignments satisfy it, so $\Pr(X \geq 3/4) = 7/8$, above the general lower bound $1/2$. For $m = 0$, return any assignment directly.

This is a high-probability $3OPT/4$ guarantee. Expected approximation, single-trial success probability, and a guarantee for every run differ. Conditional expectations provide a separate deterministic method.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Christofides: objective and assumptions

The input is a complete undirected graph with $n \geq 3$, nonnegative symmetric edge costs, and triangle inequality $c(u, w) \leq c(u, v) + c(v, w)$ for all vertices. Distinct vertices may have zero distance. The goal is a minimum-cost Hamiltonian tour, with cost OPT. The algorithm returns a feasible tour costing at most $3OPT/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## From a tree to a connected even-degree multigraph

1. Find a minimum spanning tree $T$ and its odd-degree vertex set $O$.
2. Find a **minimum-weight perfect matching** $M$ in the complete graph induced by $O$.
3. Combine $T$ and $M$ as a multigraph, keeping both copies of any shared edge.
4. Find an Euler closed walk using every edge once. Skip repeated vertices and return to the start.

The handshake lemma makes $|O|$ even. The matching adds exactly one edge to each vertex in $O$, making its degree even; other degrees do not change. The graph stays connected because it contains $T$, and therefore has an Euler closed walk. Minimum-weight perfect matching in a general graph is polynomial-time solvable; an arbitrary matching does not guarantee the bound below.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## First lower bound: the tree costs at most the best tour

Removing any edge from an optimal tour gives a spanning tree. Nonnegative costs ensure this removal cannot increase cost, and the minimum spanning tree is no more expensive than that tree. Thus $c(T) \leq OPT$. This argument does not work unchanged with negative edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Second lower bound: extract the odd vertices from the best tour

Visit $O$ in the order in which its vertices occur on an optimal tour, shortcutting every other vertex. Completeness supplies each shortcut edge, and repeated triangle inequalities yield a closed cycle on $O$ costing at most OPT. A tree with $n \geq 3$ has at least two odd vertices. When $|O| = 2$, regard the cycle as two copies of the edge, one for each direction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Alternate the cycle edges into two perfect matchings

The number of odd vertices is even. Alternately color the edges of the cycle in two groups. Each group meets each odd vertex once and is a perfect matching. Their costs sum to at most OPT, so the cheaper costs at most $OPT/2$. The minimum matching $M$ is no more expensive, giving $c(M) \leq OPT/2$. For two vertices, each of the two traversals gives the same single-edge matching, so the argument still holds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Shortcutting preserves feasibility and does not increase cost

The Euler closed walk costs exactly $c(T) + c(M)$. Keep each vertex on its first occurrence and replace skipped stretches by direct edges. Triangle inequality ensures each shortcut does not increase cost. Return to the starting vertex to obtain a tour visiting every vertex once. Consequently

$c(\text{tour}) \leq c(T) + c(M) \leq OPT + OPT/2 = 3OPT/2.$

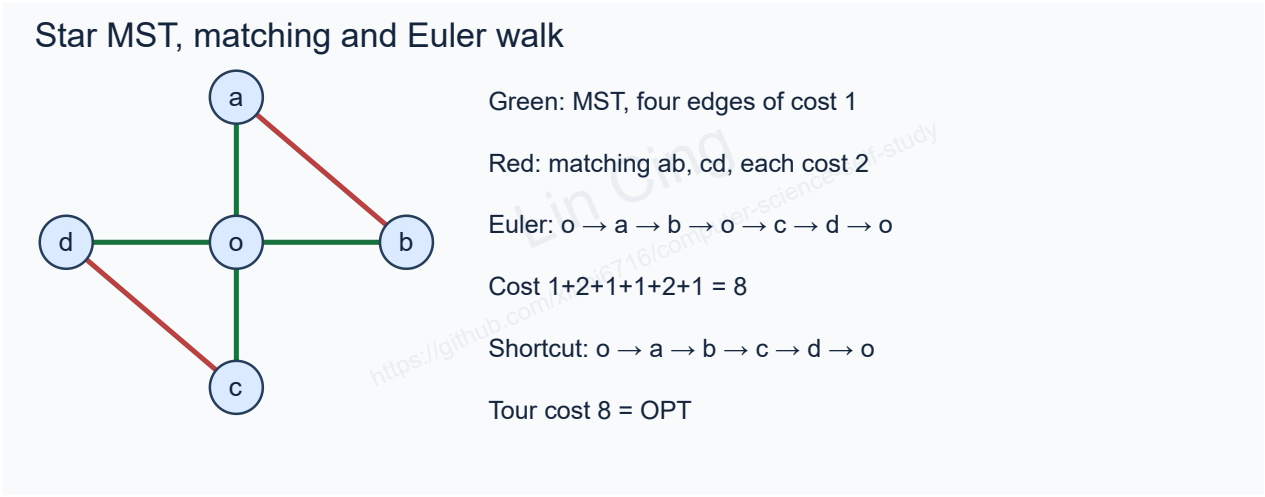
If OPT is zero, this multiplicative inequality still holds without dividing by OPT.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Four-leaf star metric: tree, matching and tour

There are five vertices $o, a, b, c, d$: center-to-leaf distance is 1 and every leaf-to-leaf distance is 2. These are tree-path distances and satisfy triangle inequality. The four star edges cost 4 and form an MST: any spanning tree has four edges, each costing at least 1. The center has degree 4 and each leaf degree 1, so $O = \{a, b, c, d\}$. All leaf pairs cost 2; choose $M = \{ab, cd\}$ for cost 4.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The Euler walk and OPT in the example

One Euler walk is $o \to a \to b \to o \to c \to d \to o$, costing $1 + 2 + 1 + 1 + 2 + 1 = 8$. Skip the second occurrence of $o$ to obtain $o \to a \to b \to c \to d \to o$, costing $1 + 2 + 2 + 2 + 1 = 8$. Every five-vertex tour uses two center edges of cost 1 and three leaf-to-leaf edges of cost 2, so OPT=8. This output is optimal; $3/2$ is a guarantee for all inputs satisfying the assumptions, not the actual ratio here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, chapters 34 and 35. MIT 6.046J Fall 2004, Lecture 24, approximation algorithms. Original handwritten algorithms notes, complexity and approximation pages.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# String matching

Find every occurrence of a pattern in a text, including overlaps. Compare direct search, rolling hashes, finite automata and KMP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# A roadmap for matching

Define the output and edge cases first. Then trace filtering, state transitions and prefix fallback; distinguish correctness from running time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## String matching

Text T has length n and pattern P has length m. Report every zero-based s with T[s:s+m]=P, including overlapping matches.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Boundary contract

The empty pattern matches all n+1 boundaries 0..n. A nonempty pattern longer than the text has no matches. The remaining algorithms first assume $1 \le m \le n$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Four methods

Naive compares directly. Rabin–Karp filters by hash. A DFA precomputes transitions. KMP uses prefix links to fall back on demand.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Naive matching

For every s=0..n−m compare P[j] with T[s+j] in order. Report s if all match. Increment s by one to retain overlaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Naive trace

T=ababa, P=aba. At s=0 compare a,b,a and match; s=1 fails at b≠a; s=2 compares a,b,a and matches. Output 0,2.

**Naive matching**

```
T   a b a b a
P   a b a     → 0
P     a b a   → fail
P       a b a → 2
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Naive cost

At most (n−m+1)m comparisons give worst-case O((n−m+1)m) time and O(1) extra space. Repeated characters can approach the worst case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rabin–Karp fingerprints

For symbols in 0..d−1, hash a length-m string as Σ x[j]d^(m−1−j) mod q. Equal strings have equal hashes; the converse is false.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rolling hash

Let h=d^(m−1) mod q. Remove a and append b using t′=(d(t−a h)+b) mod q. Normalize negative remainders into 0..q−1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Deriving the rolling update

Let W=T[s:s+m], and let c=T[s+m] be the next symbol. Treat symbols as values and write the unreduced polynomial H(W)=Σ(j=0..m−1) W[j]d^(m−1−j).

Remove the high term W[0]d^(m−1), multiply by d, and add c:

H(T[s+1:s+m+1]) = d(H(W)−W[0]d^(m−1))+c.

Each remaining term shifts one position. Reduce modulo q; with t_s=H(W) mod q and h=d^(m−1) mod q, t_(s+1)=(d(t_s−W[0]h)+c) mod q. Any q≥2 works; primality is unnecessary here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

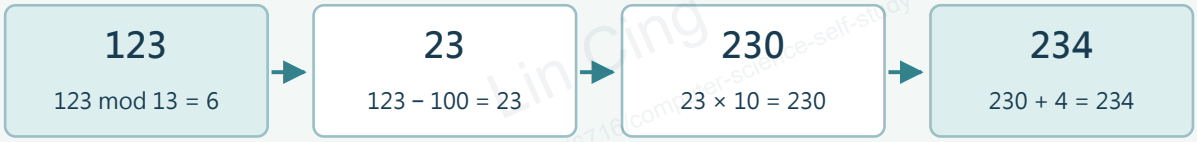


## Rolling-scan loop invariant

Before processing shift s, t_s is the hash of the current window T[s:s+m], and p is the pattern hash. Horner's rule initializes p and $t_0$. If s<n−m, the identity on the preceding slide establishes the invariant for s+1; the last window does not read T[n].

Equal strings have equal hashes, so no true match is filtered out. Verify characters on equal hash, so collisions cannot cause false reports. An intermediate difference may be negative: normalize the final remainder to 0..q−1.

**d=10, q=13, m=3: window 123 → 234**




h=100 mod 13=9; (10·(6−1·9)+4) mod 13=0.
Normalize negative remainders; 234 mod 13=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rolling hash trace

With d=10, q=13, m=2, T=1234 and P=34, window hashes for 12,23,34 are 12,10,8. The pattern hash is 8; confirm the final window character by character.

### Slide the window one position

remove 1 → ×10 → append 3
(10·(12 − 1·10) + 3) mod 13 = 10

remove 2 → ×10 → append 4
(10·(10 − 2·10) + 4) mod 13 = 8

h = 10; reducing an intermediate value modulo 13 preserves the result.



## Collisions and cost

For $1 \le m \le n$ and constant-time modular arithmetic, let H count all equal-hash windows, including true matches. Verification gives $O(n+m+mH)$, with worst case $\Theta(n+m+m(n-m+1))$.

If $T=a^n$ and $P=a^{\lfloor n/2 \rfloor}$, every window is a true match and verification costs $\Theta(nm)$, even with no false hits.

If a randomized hash gives each unequal window collision probability at most $1/q$, and V is the number of true matches, expected time is $O(n+m+mV+mn/q)$. Near-linear time also requires bounded true-match verification, for example $V=O(1)$ and $q \ge m$.

## Random-base hashing: assumptions

Fix the input first, then choose b uniformly from the finite field F_q for prime q. Distinct symbols have distinct encodings in F_q. For a length-m string X, H_b(X)=Σ(j=0..m−1) X[j]・b^(m−1−j); use b⁰=1 even when b=0.

For fixed distinct X,Y, D(b)=H_b(X)−H_b(Y) is a nonzero polynomial of degree at most m−1. A nonzero field polynomial has at most its degree many roots, hence collision probability ≤min(1,(m−1)/q), not unconditionally 1/q.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Root bound and expected running time

If r is a root, factor D(x)=(x−r)Q(x). Every other distinct root is a root of Q; repeating the argument bounds the number of roots by the degree.

Let N=n−m+1 and z be the number of true matches. Linearity of expectation over the nonmatching-window indicators needs no independence between collisions: E[f]≤(N−z)min(1,(m−1)/q). With character-by-character verification, expected time is O(n+m+mz+m(N−z)min(1,(m−1)/q)), assuming O(1) field operations. The mz verification term for true matches remains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One false hit and two true matches

Use digit encodings, q=11, b=10, m=2, P=12, T=122312. H_b(xy)=10x+y mod 11; the pattern hash is 1.

The five windows 12, 22, 23, 31, 12 have hashes 1, 0, 1, 9, 1. Window 23 has the same hash as P but differs; character verification rejects it. Positions 0 and 4 are true matches.

### q=11, b=10: equal hashes still require verification

| s = 0 | s = 1 | s = 2 | s = 3 | s = 4 |
|---|---|---|---|---|
| 12 → 1 | 22 → 0 | 23 → 1 | 31 → 9 | 12 → 1 |
| match | skip | false hit | skip | match |

P=12 hashes to 1; verify, output 0 and 4, reject 23.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exact collision probability in the example

Now choose b uniformly from $F_{11}$ rather than fixing b=10. The collision equations for nonmatching windows 22, 23, 31 against P=12 have the respective solutions b=0, b=10, b=6; each collides with probability 1/11.

With collision indicators $I_1, I_2, I_3$, $E[f]=E[I_1]+E[I_2]+E[I_3]=3/11$. No independence is needed. At b=10, only 23 is a false hit, and both true matches still require verification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Automaton state

State q is the length of the longest suffix of the scanned text that is also a prefix of P, with $0 \le q \le m$. After $\delta(q,c)$, report end−m+1 when q=m.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



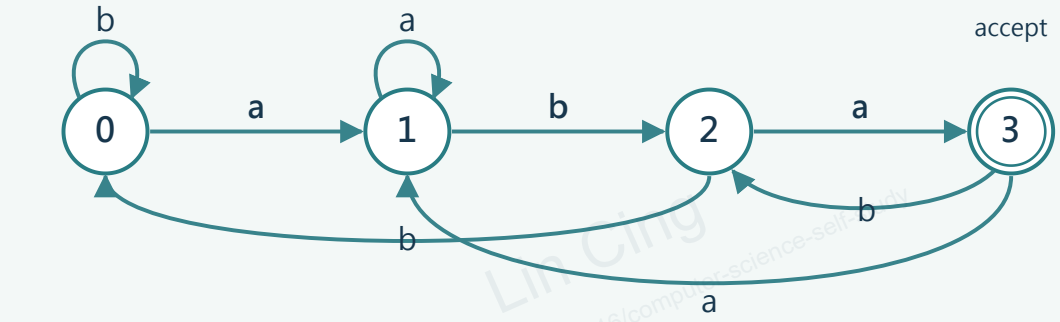
## DFA transition definition

$\delta(q,c)=\max\{k \le m: P[:k] \text{ is a suffix of } P[:q]+c\}$. Include transitions out of accepting state m so overlapping matches are found.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## DFA transition table

For P=aba and alphabet {a,b}, transitions (a,b) are 0:(1,0), 1:(1,2), 2:(3,0), 3:(1,2). Scanning takes O(n); the table uses O(m|Σ|) space.






## Automaton state as a longest suffix

Let P⟨k⟩ be the pattern prefix of length k, with P⟨0⟩=ε; P[i] denotes the zero-based character at index i. σ(x) is the length of the longest pattern prefix that is a suffix of x. State q=σ(x), and δ(q,a)=σ(P⟨q⟩a).

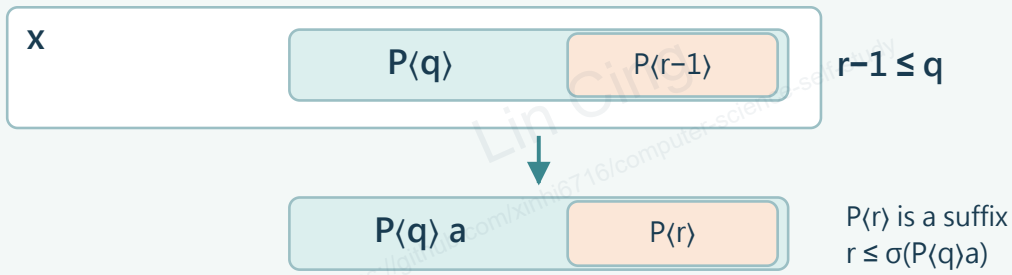
The key identity is δ(σ(x),a)=σ(xa): the state suffices to update the match without storing the entire text prefix x.

## State identity: the first inequality

Let $q=\sigma(x)$, $r=\sigma(xa)$. If $r>0$, removing the last character of $P\langle r\rangle$ shows that $P\langle r-1\rangle$ is a suffix of $x$, so $r-1\le q$.

Both $P\langle q\rangle$ and $P\langle r-1\rangle$ are suffixes of $x$; the shorter is a suffix of the longer. Thus $P\langle r-1\rangle$ is a suffix of $P\langle q\rangle$, and after appending $a$, $P\langle r\rangle$ is a suffix of $P\langle q\rangle a$. Hence $r\le\sigma(P\langle q\rangle a)$. For $r=0$ the inequality is immediate.

**Compare suffixes of x, then append the same symbol a**






## State identity: converse and correctness

Since $P\langle q\rangle$ is a suffix of $x$, $P\langle q\rangle a$ is a suffix of $xa$; every suffix of $P\langle q\rangle a$ is also a suffix of $xa$. Thus $\sigma(P\langle q\rangle a)\le\sigma(xa)=r$. Together the inequalities give equality.

Initially $\sigma(\varepsilon)=0$. Induction on the input establishes that the state is $\sigma$(the text read so far). State $m$ occurs exactly when $P$ is a suffix of the scanned text, which is exactly a match. Continue transitions after acceptance to detect overlap.

## Build the DFA using π

Let π[q−1] be the length of the longest proper border of P⟨q⟩. For finite alphabet Σ and m≥1:

• q=0: δ(0,a)=1 if a=P[0], and 0 otherwise. • 1≤q≤m: if q<m and a=P[q], set δ(q,a)=q+1; otherwise set δ(q,a)=δ(π[q−1],a).

At q=m do not read P[m]: use the second branch. Since π[q−1]<q, the required row has already been computed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why border fallback builds the table

When direct extension fails, a match of length q+1 is impossible. For any shorter nonempty match P⟨k⟩, dropping its last character leaves a proper border P⟨k−1⟩ of P⟨q⟩. All proper borders occur on the π chain, so retrying the same a from the longest proper border loses no candidate; k=0 remains possible.

Computing π uses O(m) time and space. There are m+1 rows and |Σ| constant-time cells per row: table time and space O(m|Σ|), assuming O(1) alphabet and table access. Scanning costs O(n) plus output.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Prefix function

π[i] is the length of the longest proper prefix of P[:i+1] that is also its suffix, with $0 \leq \pi[i] \leq i$. Proper excludes the whole string.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Prefix function example

For P=abababaca at indices 0..6, π=[0,0,1,2,3,0,1]. Symbols a b a b a c a correspond to 0 0 1 2 3 0 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Build the prefix function token by token

i=0:a→0; 1:b differs from a→0; 2:a=a→1; 3:b=b→2; 4:a=a→3; 5:c≠b, 3→1, c≠b, 1→0, c≠a→0; 6:a=a→1.

### i = 5: repeated fallback while comparing c

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | |
|---|---|---|---|---|---|---|---|---|
| | a | b | a | b | a | c | a | |
| k = 3 | | | a | b | a | b ≠ c | | 3 → 1 |
| k = 1 | | | | | a | b ≠ c | | 1 → 0 |
| k = 0 | | | | | | a ≠ c | | π[5] = 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why fallback works

If k characters match but the next does not, a shorter candidate must be a border of P[:k]. The longest candidate is π[k−1]. Repeat until a match or k=0. For P=ababaca, k=5 and next symbol x, fallback is 5→3→1→0: one fallback is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why the border chain is complete

Let B be the longest proper border of X. Any shorter proper border C is both a prefix and a suffix of X, hence a prefix and a suffix of B, and therefore a border of B. Conversely, any border of B is a border of X.

Repeatedly apply π[length−1], beginning at |X|. Length strictly decreases to 0; the two containments prove that the chain lists exactly all proper border lengths, without omissions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

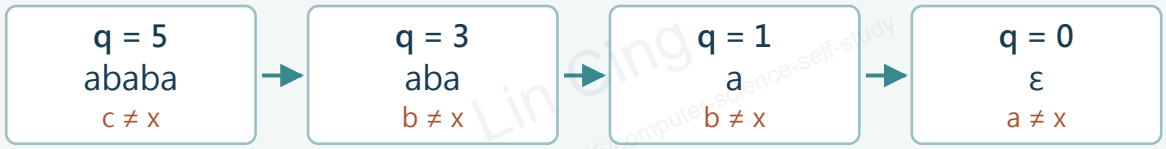


## Prefix function: invariant and proof

If m=0, π is empty. Otherwise set π[0]=0: one symbol has no nonempty proper border. For i=1,…,m−1, known earlier entries give k=π[i−1]. While k>0 and P[k]≠P[i], set k←π[k−1], keeping P[i] fixed.

Candidates are visited from longest to shortest. On equality set π[i]=k+1; if k=0 and P[0] differs, set π[i]=0. This completes the induction.

### P = ababaca: retry the same symbol x




5 → 3 → 1 → 0; at q=0 compare P[0], then stay at 0.

Every shorter border is a border of the longest one.

## Why π takes linear time

Use the nonnegative potential k. Each i can increase k at most once, by at most one; each while fallback lowers k by at least one. Starting at zero, total fallbacks cannot exceed total increases $\leq m-1$.

Each i also performs at most one terminating comparison and constant work. Thus preprocessing is O(m); the same argument gives O(n) for the KMP scan. This bounds the entire run, although one input character can trigger multiple fallbacks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## KMP preprocessing

Set π[0]=0. For i=1..m−1, start k=π[i−1]; while k>0 and characters differ set k=π[k−1]; increment k on equality; set π[i]=k.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## KMP scan

Start $q=0$. For each $c$, while $q>0$ and $P[q] \neq c$ set $q=\pi[q-1]$. Increment $q$ on equality. At $q=m$ report $i-m+1$, then set $q=\pi[m-1]$ to preserve overlaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Two KMP scan states

Assume $m \geq 1$. Let $\sigma(x)$ be the longest pattern-prefix suffix of $x$, allowing length $m$; let $\rho(x)$ be the longest such prefix with length strictly less than $m$.

At the start of each iteration $q=\rho(x)$, where $x$ is the text read so far; initially $q=\rho(\varepsilon)=0$. If $\sigma(x) <m$, then $\rho(x)=\sigma(x)$. If $\sigma(x)=m$, then $\rho(x)=\pi[m-1]$. Thus the stored post-match state is the longest proper border, not $m$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A scan step simulates the DFA

For the same new symbol a, the while loop tries the border chain from q in decreasing order. The first extendable candidate is longest; if none extends, the state is 0. Thus the temporary state r=δ(q,a).

If σ(x)<m, q=σ(x); if σ(x)=m, q=π[m−1] and δ(m,a)=δ(π[m−1],a). By δ(σ(x),a)=σ(xa), in both cases r=σ(xa).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why post-match fallback preserves overlaps

r=m exactly when the pattern is a suffix of the scanned text xa, so output i−m+1 is correct. Then q←π[m−1]=ρ(xa). If r<m, set q←r=ρ(xa). Either way the next iteration's invariant is restored.

For P=aba and T=ababa, positions 2 and 4 produce outputs 0 and 2. After either acceptance, retain q=1; extending that border preserves the overlapping occurrence.

**P=aba, T=ababa: accept first, retain the border**

| i | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| T[i] | a | b | a | b | a |
| q before | 0 | 1 | 2 | 1 | 2 |
| r = σ | 1 | 2 | 3 | 2 | 3 |
| q after | 1 | 2 | 1 | 2 | 1 |
| output | – | – | 0 | – | 2 |

When r=3 report, then set q=π[2]=1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## KMP token trace

T=ababa, P=aba, π=0,0,1. Each i, symbol, q: 0 a→1; 1 b→2; 2 a→3, report 0 and fall back to 1; 3 b→2; 4 a→3, report 2 and fall back to 1.

### P = aba: retain border length 1 after a match

| T index | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| input | a | b | a | b | a |
| state q | 1 | 2 | 3 | 2 | 3 |
| match at | | | 0<br>3 → 1 | | 2<br>3 → 1 |

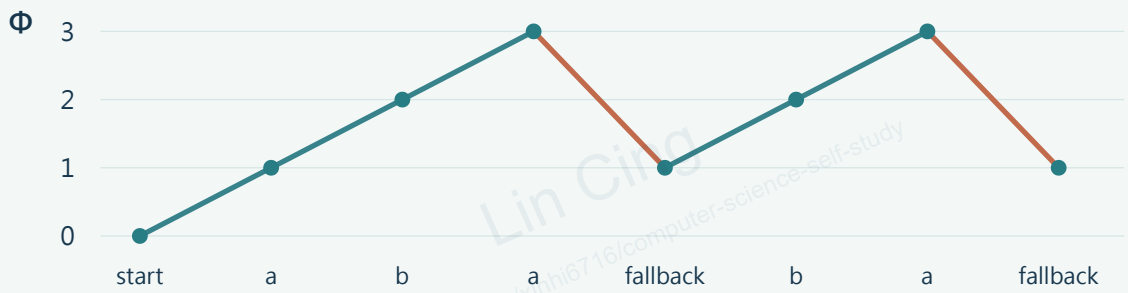
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Amortized linear time

Use q or k as potential in each preprocessing or scan pass. Each input position increases it by at most 1; every nonzero fallback decreases it by at least 1. Total fallbacks cannot exceed total increases. Other comparisons cost O(1) per position, so total time is O(m+n).

### Potential Φ = q: each input raises it at most once




Every fallback drops Φ by at least 1; total drops ≤ total rises.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## KMP time and memory

Preprocessing costs O(m), scanning O(n), and the π table O(m). Storing z outputs costs an additional O(z); streaming output avoids that storage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Four methods compared

For $1 \le m \le n$, naive takes worst-case O(nm) time and O(1) space; Rabin–Karp worst-case O(nm) and O(1) workspace; DFA O(n) scan and O(m|Σ|) table; KMP O(n+m) and O(m).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Implementation pitfalls

Specify the symbol and index unit: bytes, Unicode code points, and grapheme clusters differ. Handle the empty pattern first; verify hash hits; do not reset to zero after a match.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Verification strategy

Use direct enumeration as an oracle and exhaust all short texts and patterns over a small alphabet. Compare complete outputs of Rabin–Karp, DFA and KMP, including collisions, empty patterns, m>n, and overlaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Takeaway

Borders of the matched prefix form the viable fallback chain. KMP follows it on demand; the DFA precomputes state updates; hashing requires verification to guarantee correctness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

CLRS, Introduction to Algorithms, 3rd ed., chapter 32, pp. 990–1006. MIT 6.046J Fall 2004, Lecture 22.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Matrix operations and FFT

Begin with block matrices and multiplication, proceed to elimination, PA=LU and least squares, then use the DFT and FFT for polynomial convolution.

Matrix methods organize computation through blocks and factorizations; FFT reuses roots of unity through even-odd splitting. Track dimensions, pivots, transform conventions and padding.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Reuse the same coefficient matrix

In $Ax = b$, separate the coefficients from the right-hand side. Reuse a factorization when $A$ stays fixed and $b$ changes. Use real matrices, column vectors, $PA = LU$, and unit-diagonal $L$. Define the least-squares residual as $r = b - A\hat{x}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Match the inner dimensions

For $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, $C = AB \in \mathbb{R}^{m \times n}$ with $c_{ij} = \sum_{t=1}^{k} a_{it} b_{tj}$. The direct algorithm uses $mkn$ multiplications and $mn(k-1)$ additions. A defined $AB$ does not imply a defined $BA$.

### Matrix dimensions and one dot product

A: m × k

B: k × n

AB: m × n

Shared index t = 1...k

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Compute entries with their indices

Let $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ and $B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$. Then $c_{12} = 1(0) + 2(1) + 3(1) = 5$ and $AB = \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix}$.

Here $BA$ is $3 \times 3$, so it cannot equal $AB$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Check dimensions inside every block

Split the inner dimension as $k = k_1 + k_2$. For $A = [A_1\ A_2]$ and $B = [B_1; B_2]$, let $A_s$ have size $m \times k_s$ and $B_s$ have size $k_s \times n$. Then $AB = A_1B_1 + A_2B_2$. In a two-by-two partition, $C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}$. Blocks need not commute.

### Split the inner dimension into k₁ + k₂

$A_1$: m × $k_1$

$B_1$: $k_1$ × n

$A_1B_1 + A_2B_2$: m × n

$A_2$: m × $k_2$; $B_2$: $k_2$ × n

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fewer recursive products change the exponent

Ordinary square blocking gives $8T(n/2) + \Theta(n^2) = \Theta(n^3)$. Strassen reconstructs four output blocks from seven products, giving $7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7})$. Pad with zeros to a power of two and crop the result. Arithmetic complexity does not guarantee faster small instances.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strassen: form seven products

Partition square matrices into equal blocks $A = [A_{11}, A_{12}; A_{21}, A_{22}]$ and $B = [B_{11}, B_{12}; B_{21}, B_{22}]$. Form seven block products:

$P_1 = (A_{11} + A_{22})(B_{11} + B_{22})$; $P_2 = (A_{21} + A_{22})B_{11}$; $P_3 = A_{11}(B_{12} - B_{22})$; $P_4 = A_{22}(B_{21} - B_{11})$.

$P_5 = (A_{11} + A_{12})B_{22}$; $P_6 = (A_{21} - A_{11})(B_{11} + B_{12})$; $P_7 = (A_{12} - A_{22})(B_{21} + B_{22})$. Block multiplication is not commutative.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Strassen: recombination and proof

$C_{11} = P_1 + P_4 - P_5 + P_7$, $C_{12} = P_3 + P_5$; $C_{21} = P_2 + P_4$, $C_{22} = P_1 - P_2 + P_3 + P_6$.

For instance, expanding $C_{11}$ cancels the cross terms and leaves $A_{11}B_{11} + A_{12}B_{21}$. Expanding the other blocks gives ordinary block multiplication. Seven half-size products and $\Theta(n^2)$ additions per level yield $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7})$; zero-pad to a power of two if needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strassen: a 2×2 trace

Let $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. The seven scalar products are $P_1 = 65$, $P_2 = 35$, $P_3 = -2$, $P_4 = 8$, $P_5 = 24$, $P_6 = 22$, $P_7 = -30$.

Recombination gives $C_{11} = 65 + 8 - 24 - 30 = 19$, $C_{12} = -2 + 24 = 22$, $C_{21} = 35 + 8 = 43$, and $C_{22} = 65 - 35 - 2 + 22 = 50$, matching direct multiplication. Arithmetic counts alone do not capture finite precision or small-size constants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Elimination stores multipliers and updates the tail

At step $k$, use a nonzero pivot $u_{kk}$, set $l_{ik} = a_{ik}/u_{kk}$, and apply $R_i \leftarrow R_i - l_{ik}R_k$. The surviving upper triangle is $U$; the multipliers form $L$. The first block update is the Schur complement $A_{22} - a_{21}a_{12}/a_{11}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## An invertible matrix can have a zero pivot

The matrix $A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ has determinant $-1$, but elimination without a row exchange divides by zero immediately. With $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $PA = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = IU$. A zero chosen pivot alone does not prove that the original matrix is singular.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## First-column row exchange and elimination

Use the CLRS28.1 example $A = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$. Partial pivoting chooses the largest available absolute entry. Swap $R_1 \leftrightarrow R_3$ to obtain $[5, 6, 3; 3, 4, 4; 1, 2, 0]$. With multipliers $3/5$ and $1/5$, eliminate to obtain $[5, 6, 3; 0, 2/5, 11/5; 0, 4/5, -3/5]$.

Round 1: compare, swap, eliminate

A

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$$

P₁A

$$\begin{bmatrix} 5 & 6 & 3 \\ 3 & 4 & 4 \\ 1 & 2 & 0 \end{bmatrix}$$

U after round 1

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

|1| < |3| < |5| → R₁ ↔ R₃

l₂₁ = 3/5, l₃₁ = 1/5; R₂ ← R₂ − (3/5)R₁; R₃ ← R₃ − (1/5)R₁

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A later swap must move stored multipliers

In column two, $4/5 > 2/5$, so exchange working rows two and three. Also exchange the already stored first-column multipliers in $L$, giving $l_{21} = 1/5, l_{31} = 3/5$. The working matrix is $[5, 6, 3; 0, 4/5, -3/5; 0, 2/5, 11/5]$. With $l_{32} = 1/2$, the final pivot is $11/5 - (1/2)(-3/5) = 5/2$.

Round 2: swap the working matrix, P and stored L

U₁

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

P₁

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

L: stored column 1

$$\begin{bmatrix} 1 & 0 & 0 \\ 3/5 & 1 & 0 \\ 1/5 & 0 & 1 \end{bmatrix}$$

|4/5| > |2/5| → R₂ ↔ R₃

Before the swap; exchange only the completed column of L.



## Verify the convention with PA=LU

$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$, $L = \begin{bmatrix} 1 & 0 & 0 \\ 1/5 & 1 & 0 \\ 3/5 & 1/2 & 1 \end{bmatrix}$, and $U = \begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$. Both $PA$ and $LU$ equal $[5, 6, 3; 1, 2, 0; 3, 4, 4]$. $P$ reorders equations, leaving the unknown ordering unchanged.

Eliminate and verify every entry of PA = LU

U after R₂ ↔ R₃

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 2/5 & 11/5 \end{bmatrix}$$

U

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$$

PA = LU

$$\begin{bmatrix} 5 & 6 & 3 \\ 1 & 2 & 0 \\ 3 & 4 & 4 \end{bmatrix}$$

l₃₂ = (2/5)/(4/5) = 1/2; R₃ ← R₃ − (1/2)R₂

LU rows: (5, 6, 3); (1, 6/5+4/5, 3/5−3/5);

(3, 18/5+2/5, 9/5−3/10+5/2) = (3, 4, 4)

## Why can LUP keep finding a pivot?

Consider an invertible current trailing block $K$. Its first column contains a nonzero entry; otherwise $K$ would have a zero column and be singular. Swap rows within the trailing block to move such an entry to the pivot $a$. The swap preserves invertibility.

Write $K' = PK$ for the block after the swap. Block elimination gives $\det K' = a \det S$, while $\det K' = \pm \det K$. Since $\det K \neq 0$ and $a \neq 0$, the new Schur complement $S$ is invertible, and induction continues to the last pivot. Later row swaps also exchange corresponding multipliers already stored in $L$, producing $PA = LU$. Selecting the largest absolute pivot controls local multipliers but does not guarantee numerical stability for every problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## In-place LUP: inputs, outputs and storage

Input is a real $n \times n$ matrix $A$. Start with working array $W = A$ and permutation vector $p = [1, \ldots, n]$. On success, the diagonal and upper triangle of $W$ store $U$, its strict lower triangle stores the multipliers of $L$, and the unit diagonal of $L$ is implicit. The convention is $(Pb)_i = b_{p_i}$ and $PA = LU$.

The algorithm below uses exact arithmetic and one-based indices; for $n = 0$ one may return the empty factorization. “In place” means overwriting the working matrix; preserving the original $A$ requires another $\Theta(n^2)$ storage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## In-place LUP: pivot, swap the whole row, update

For $k = 1, \dots, n$, choose $q$ maximizing $|W_{ik}|$ among $i = k, \dots, n$. If $W_{qk} = 0$, report a singular matrix and stop; otherwise exchange entire rows $k, q$ of $W$, and exchange $p_k, p_q$.

For each $i = k+1, \dots, n$, first store $W_{ik} \leftarrow W_{ik}/W_{kk}$; then for $j = k+1, \dots, n$ update $W_{ij} \leftarrow W_{ij} - W_{ik}W_{kj}$. Swapping whole rows includes earlier multipliers in columns $j < k$. Swapping only the trailing block would break $PA = LU$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Invariant for the completed blocks

After the first $k$ steps, the currently permuted original matrix satisfies

$PA = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & S \end{bmatrix}$, with $k \times k$ leading triangular blocks and remaining Schur complement $S$. Completed columns $L_{11}, L_{21}$ and completed rows $U_{11}, U_{12}$ are stored in $W$.

The next row swap also reorders the already-stored multipliers in $L_{21}$. Dividing below the new pivot gives the new multipliers, and the outer-product update modifies $S$ exactly as block multiplication requires. Initially $k = 0$ and $S = A$; at $k = n$ the trailing block is empty and $PA = LU$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## What a zero pivot reports

Under exact arithmetic, if all remaining entries in the current pivot column are zero, the trailing block has a zero column and is singular. Earlier pivots were nonzero and all previous permutations and elimination operations were invertible, so the original $A$ is singular. Conversely, an invertible $A$ has an invertible trailing block at each step and hence a nonzero candidate pivot.

In floating point, treating a small pivot as zero under a chosen scale and tolerance only reports failure under that rule; it does not prove exact mathematical singularity. Rounding, scaling and conditioning matter. Numerical rank requires an appropriate rank-revealing method and an explicit tolerance.

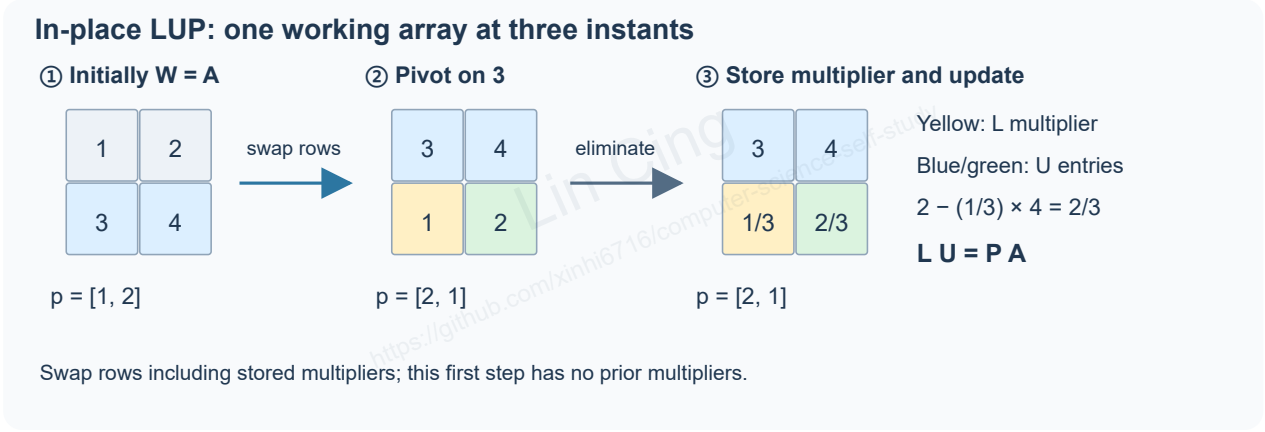
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Trace the in-place array

Take $A = [1, 2; 3, 4]$ and $b = (5, 11)^T$. The largest absolute value in column 1 is 3. Swap rows and set $p = (2, 1)$: the working matrix becomes $[3, 4; 1, 2]$.

Compute $l_{21} = 1/3$ and update the lower-right entry to $2 - (1/3)4 = 2/3$. Thus $W = [3, 4; 1/3, 2/3]$ stores $L = [1, 0; 1/3, 1]$ and $U = [3, 4; 0, 2/3]$. Check $LU = [3, 4; 1, 2] = PA$ and $Pb = (11, 5)^T$.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Forward substitution computes the intermediate vector

For $b = (3, 7, 8)^T$, permute to $Pb = (8, 3, 7)^T$. Solve $Ly = Pb$. Then $y_1 = 8$, $y_2 = 3 - (1/5)8 = 7/5$, and $y_3 = 7 - (3/5)8 - (1/2)(7/5) = 3/2$. In general, $y_i = (Pb)_i - \sum_{j<i} l_{ij} y_j$. One forward solve takes $\Theta(n^2)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Back substitution starts at the last row

Solve $Ux = y$. Obtain $x_3 = (3/2)/(5/2) = 3/5$, $x_2 = [7/5 + (3/5)(3/5)]/(4/5) = 11/5$, and $x_1 = [8 - 6(11/5) - 3(3/5)]/5 = -7/5$. Substitution gives $Ax = (3, 7, 8)^T$. In general, $x_i = (y_i - \sum_{j>i} u_{ij} x_j)/u_{ii}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Factor once for many right-hand sides

Dense elimination updates $\sum_{k=1}^{n-1}(n-k)^2 = \Theta(n^3)$ trailing entries. With the factors available, each new right-hand side needs two triangular solves in $\Theta(n^2)$. For $q$ right-hand sides, the total is $\Theta(n^3 + qn^2)$. Partial pivoting can still suffer element growth; it does not guarantee accuracy for every problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Derive determinants and inverses from the factors

After $s$ row exchanges, $\det A = (-1)^s \prod_i u_{ii}$. Two exchanges give $\det A = 5(4/5)(5/2) = 10$. To invert, solve $Az_j = e_j$; $z_j$ is inverse column $j$. After factorization, $n$ solves cost $\Theta(n^3)$. For a single $Ax = b$, use the factors directly instead of computing the whole inverse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Forward and back substitution invariants

For unit lower-triangular $L$, solve $Ly = Pb$ by $y_i = (Pb)_i - \sum_{j<i} l_{ij} y_j$. Before step $i$, rows $1, \ldots, i-1$ hold and those $y_j$ stay fixed. The formula makes row $i$ hold; induction gives $Ly = Pb$.

For nonsingular upper-triangular $U$, solve $Ux = y$ from $i = n$ down to 1 using $x_i = (y_i - \sum_{j>i} u_{ij} x_j)/u_{ii}$. Before step $i$, rows $i+1, \ldots, n$ hold; their equations do not depend on $x_i$, and the step establishes row $i$. Nonzero pivots make division valid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Both triangular solves on the same example

Forward substitution gives $y_1 = 11$ and $y_2 = 5 - (1/3)11 = 4/3$. Back substitution gives $x_2 = (4/3)/(2/3) = 2$ and $x_1 = (11 - 4 \cdot 2)/3 = 1$. Finally $A(1, 2)^T = (5, 11)^T = b$.

Row 2 of the upper-triangular system holds first. Computing $x_1$ leaves $x_2$ unchanged, so row 2 still holds: this is the back-substitution invariant in a concrete calculation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

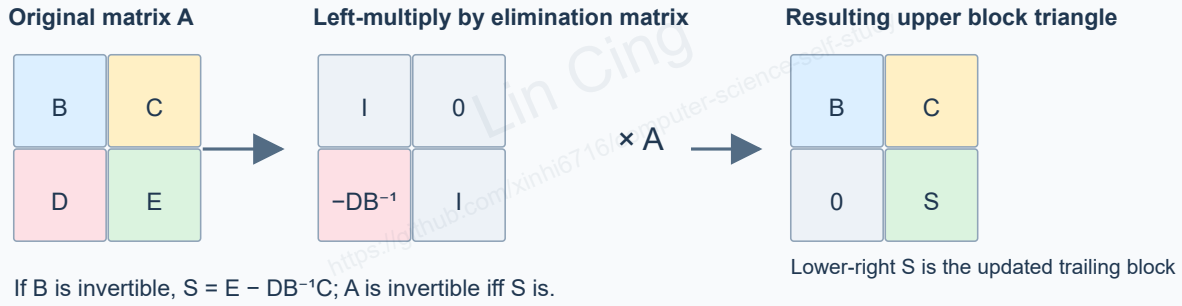
## Block inversion starts with elimination

Let $A = \begin{bmatrix} B & C \\ D & E \end{bmatrix}$, with square, invertible $B$ and compatible dimensions. Its Schur complement is $S = E - DB^{-1}C$. Block elimination gives

$$\begin{bmatrix} I & 0 \\ -DB^{-1} & I \end{bmatrix} A = \begin{bmatrix} B & C \\ 0 & S \end{bmatrix}.$$

Eliminate the lower-left block first, then work on $S$. Block products cannot be reordered.

**Schur complement: eliminate lower left and update lower right**




If B is invertible, S = E − DB⁻¹C; A is invertible iff S is.



## Block inverse and its hypotheses

If both $B$ and $S$ are invertible, reversing the block elimination factors yields

$$A^{-1} = \begin{bmatrix} B^{-1} + B^{-1}CS^{-1}DB^{-1} & -B^{-1}CS^{-1} \\ -S^{-1}DB^{-1} & S^{-1} \end{bmatrix}.$$

Check the result by multiplying $AA^{-1}$. When $B$ is invertible, $A$ is invertible exactly when $S$ is. If $B$ is singular this formula does not apply, even though $A$ itself may be invertible.

## A block inverse: compute the Schur complement

Take $A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{bmatrix}$ and partition after the first two rows and columns. Then $B = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}$, $C = (2,1)^T$, $D = C^T$, $E = 3$.

$B^{-1} = \frac{1}{16}\begin{bmatrix} 5 & -2 \\ -2 & 4 \end{bmatrix}$, $B^{-1}C = (1/2, 0)^T$, and $S = 3 - (2,1)(1/2,0)^T = 2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Assemble the inverse block by block

$S^{-1} = 1/2$ and the upper-right block is $-B^{-1}CS^{-1} = (-1/4, 0)^T$; the lower-left block is its transpose.

The upper-left block is $B^{-1} + (1/2,0)^T(1/2)(1/2,0) = \begin{bmatrix} 7/16 & -1/8 \\ -1/8 & 1/4 \end{bmatrix}$.

Thus $A^{-1} = \begin{bmatrix} 7/16 & -1/8 & -1/4 \\ -1/8 & 1/4 & 0 \\ -1/4 & 0 & 1/2 \end{bmatrix}$. For example, the first row times the first column gives $4(7/16) + 2(-1/8) + 2(-1/4) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Positive definiteness protects recursive pivots

A real symmetric matrix is positive definite (SPD) when $z^T Az > 0$ for every nonzero $z$. With $A = \begin{bmatrix} B & C \\ C^T & D \end{bmatrix}$, the principal block $B$ is SPD: choose $z = (x, 0)$ for nonzero $x$.

The Schur complement $S = D - C^T B^{-1} C$ is also SPD: for nonzero $y$ choose $z = (-B^{-1}Cy, y)$, so $y^T Sy = z^T Az > 0$. Consequently $B$ and $S$ are invertible and the block formula can recurse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Cholesky: square root and trailing update

Every real symmetric positive-definite matrix has a unique factorization $A = LL^T$, where $L$ is lower triangular with positive diagonal. Partition $A = \begin{bmatrix} a & r^T \\ r & D \end{bmatrix}$. Since $a > 0$, set $\ell_{11} = \sqrt{a}$ and $v = r/\sqrt{a}$; the remaining matrix $S = D - vv^T$ is SPD.

Recursively factor $S = L_2 L_2^T$ to obtain $L = \begin{bmatrix} \sqrt{a} & 0 \\ v & L_2 \end{bmatrix}$. Each square root is positive. The first column is forced by the positive diagonal convention, and induction proves uniqueness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Elementwise Cholesky recurrence

For real symmetric positive-definite $A = LL^T$ with positive-diagonal lower-triangular $L$, the identity $a_{ij} = \sum_{k=1}^{\min(i,j)} l_{ik} l_{jk}$ gives the column-by-column recurrence for $j = 1, \ldots, n$:

$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$; for $i > j$, $l_{ij} = \dfrac{a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk}}{l_{jj}}$.

An empty sum is zero. Compute the diagonal first, then the entries below it; all columns $k < j$ have already been computed. Entries above the diagonal are zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why are the square roots and divisions valid?

At step $j$, the radicand is the leading diagonal entry of the Schur complement remaining after eliminating columns $1, \ldots, j-1$. Principal blocks and Schur complements of an SPD matrix are SPD, so this entry is strictly positive. Thus $l_{jj} > 0$ and the division below it is defined.

Substituting the computed entries into $a_{ij} = \sum_{k \le \min(i,j)} l_{ik} l_{jk}$ shows, one column at a time, that $A = LL^T$. The positive diagonal forces each square root and each entry below it uniquely. Without real symmetry and positive definiteness, the radicand or divisor need not be positive.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Compute every entry in the 3×3 example

For $A = [4, 2, 2; 2, 5, 1; 2, 1, 3]$, the first column is $l_{11} = \sqrt{4} = 2$, $l_{21} = 2/2 = 1$, $l_{31} = 2/2 = 1$.

The second column is $l_{22} = \sqrt{5 - l_{21}^2} = 2$, $l_{32} = (1 - l_{31}l_{21})/l_{22} = 0$. Finally $l_{33} = \sqrt{3 - l_{31}^2 - l_{32}^2} = \sqrt{2}$.

Thus $L = [2, 0, 0; 1, 2, 0; 1, 0, \sqrt{2}]$. Check $l_{31}l_{21} + l_{32}l_{22} = 1 = a_{32}$ and verify every entry with $LL^T = A$.

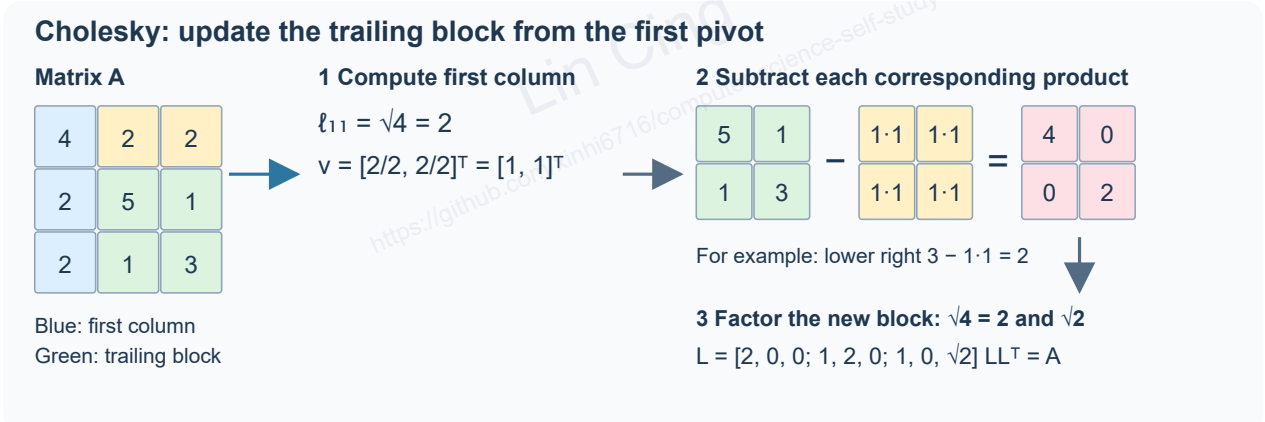
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Cholesky example: follow the update

For $A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{bmatrix}$, the first pivot is $\ell_{11} = 2$, and both entries below it equal $1$. The trailing update is $\begin{bmatrix} 5 & 1 \\ 1 & 3 \end{bmatrix} - \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$.

Hence $L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \sqrt{2} \end{bmatrix}$. Multiplication gives $LL^T = A$. Also $x^T Ax = \|L^T x\|^2 > 0$ for nonzero $x$, verifying that this example is SPD.



## Solve using a Cholesky factor

Solve $LL^Tx = b$ by forward substitution $Ly = b$ and back substitution $L^Tx = y$. For $b = (8, 8, 6)^T$, forward substitution gives $y = (4, 2, \sqrt{2})^T$. Back substitution yields $x_3 = 1$, $x_2 = 1$, $x_1 = 1$; check $A(1, 1, 1)^T = b$.

Dense Cholesky factorization costs $\Theta(n^3)$ arithmetic operations; each subsequent right-hand side costs $\Theta(n^2)$ for both triangular solves. The SPD hypothesis matters; near-positive-definite floating-point inputs also need rounding and failure handling.

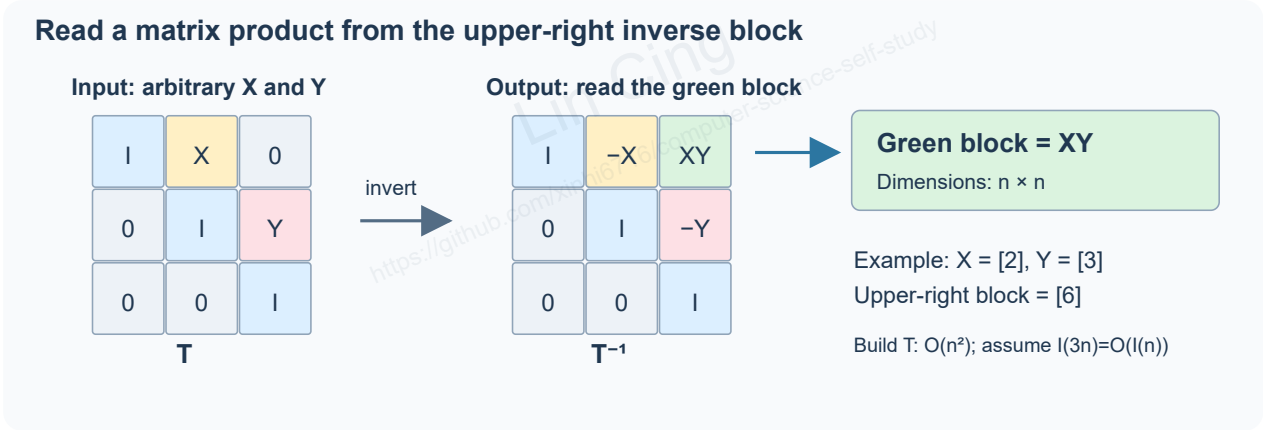
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Matrix inversion can compute a product

For two $n \times n$ matrices $X, Y$, form $T = \begin{bmatrix} I & X & 0 \\ 0 & I & Y \\ 0 & 0 & I \end{bmatrix}$.

$T^{-1} = \begin{bmatrix} I & -X & XY \\ 0 & I & -Y \\ 0 & 0 & I \end{bmatrix}$. Block multiplication checks the formula: the upper-right block is $XY - XY = 0$ and the other off-diagonal blocks cancel. Read $XY$ from the upper-right block; $T$ is invertible for every $X, Y$.



## The cost of extracting a product

Let $I(n)$ bound the cost of inverting an invertible $n \times n$ matrix. If $I(n) = \Omega(n^2)$ and $I(3n) = O(I(n))$, forming the $3n \times 3n$ matrix $T$ and extracting its upper-right block takes $O(n^2)$, so matrix multiplication costs $M(n) = O(I(n))$.

This reduction counts arithmetic operations. The scaling assumption matters: a bound for one dimension alone does not justify replacing $I(3n)$ by $I(n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fast multiplication gives fast inversion

For an SPD matrix, the block formula recursively inverts $B$ and $S$, and uses a constant number of half-size products and $O(n^2)$ additions. Thus $I_{SPD}(n) \leq 2I_{SPD}(n/2) + O(M(n) + n^2)$.

If $M(n) = \Omega(n^2)$ and $M(n/2) \leq cM(n)$ for a fixed $c < 1/2$, successive recursion levels cost at most a ratio $2c < 1$ of the preceding bound. Summing yields $I_{SPD}(n) = O(M(n))$. The conclusion needs these cost assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## General invertible matrices and the arithmetic model

For real nonsingular $A$, $A^TA$ is SPD since $x^TA^TAx = \|Ax\|^2 > 0$ for nonzero $x$. Form and invert $A^TA$, then multiply by $A^T$: $A^{-1} = (A^TA)^{-1}A^T$. To pad to a power of two, use $\operatorname{diag}(A, I)$ and assume $M(n+k) = O(M(n))$ for $0 \le k \le n$.

Under the stated regularity conditions and exact arithmetic model, multiplication and inversion reduce to one another. This does not guarantee floating-point stability: forming $A^TA$ squares the condition number, so practical solves typically use pivoted elimination or a suitable factorization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The relation between LDLᵀ and Cholesky

For SPD matrices we may retain the elimination pivots in a diagonal matrix: $A = LDL^T$, where $L$ is unit lower triangular and $D$ is positive diagonal. Each pivot is the top-left entry of the current SPD Schur complement and is therefore positive.

For $D = \operatorname{diag}(d_1, \dots, d_n)$, set $D^{1/2} = \operatorname{diag}(\sqrt{d_i})$. Then $A = (LD^{1/2})(LD^{1/2})^T$, giving the positive-diagonal Cholesky factor. In the previous example $L = [1, 0, 0; 1/2, 1, 0; 1/2, 0, 1]$, $D = \operatorname{diag}(4, 4, 2)$; multiplying by $D^{1/2}$ gives $[2, 0, 0; 1, 2, 0; 1, 0, \sqrt{2}]$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Leading arithmetic cost of dense factorizations

Assume dense $n \times n$ input, count each scalar addition, subtraction, multiplication and division as one flop, and no early singular return. LUP trailing updates contribute about $2\sum_{s=1}^{n-1} s^2 = \frac{2}{3}n^3 + O(n^2)$ flops; multiplier divisions add $O(n^2)$. The two triangular solves for one right-hand side take about $2n^2 + O(n)$ flops together.

For Cholesky, elementwise multiply-add terms take $\frac{1}{3}n^3 + O(n^2)$ flops; divisions and square roots occur $O(n^2)$ and $n$ times respectively. Square roots may have a different cost and should be reported separately. These leading coefficients depend on the stated flop convention and are not wall-clock-time ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Account separately for pivots and memory traffic

Partial pivot selection scans absolute values and makes $O(n^2)$ comparisons. At most $n$ full-row swaps move $O(n^2)$ matrix entries and $O(n)$ permutation indices. These comparisons, memory accesses and square roots are not normally included in the multiply-add flop totals; actual time can be dominated by memory behavior or implementation choices.

$W$ occupies $\Theta(n^2)$ storage and the permutation needs $\Theta(n)$ extra space. Whether the original matrix is retained changes the additional storage. Sparse matrices, low-rank shortcuts, parallelism and early exits are outside the direct dense full-run time accounting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three noncollinear points require a fit

Fit $b \approx \alpha + \beta t$ to $(t, b) = (0, 1), (1, 2), (2, 2)$. Let $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \in \mathbb{R}^{3 \times 2}$, $x = (\alpha, \beta)^T$, and $b = (1, 2, 2)^T$. Consecutive slopes are 1 and 0, so no line passes through all three points. Minimize $\|b - Ax\|_2^2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Set the gradient to zero for the normal equations

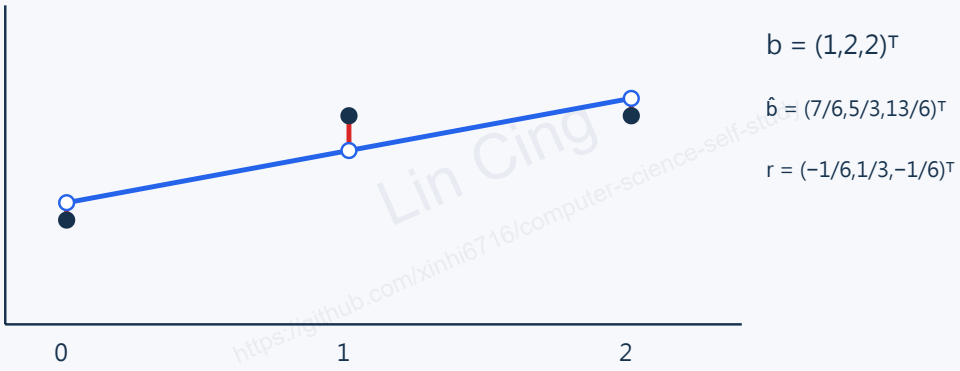
The gradient is $-2A^T(b - Ax)$, so $A^T A\hat{x} = A^T b$. Here $A^T A = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$ and $A^T b = (5, 6)^T$. Full column rank gives $z^T A^T Az = \|Az\|^2 > 0$ for every nonzero $z$, hence a unique solution. Dependent columns invalidate that uniqueness argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Compute coefficients, predictions, and residuals

The equations are $3\alpha + 3\beta = 5$ and $3\alpha + 5\beta = 6$. Subtraction gives $\beta = 1/2$, then $\alpha = 7/6$. Predictions are $A\hat{x} = (7/6, 5/3, 13/6)^T$ and residuals are $r = (-1/6, 1/3, -1/6)^T$. The squared error is $1/36 + 1/9 + 1/36 = 1/6$.

### Vertical residuals, not shortest point-line distances



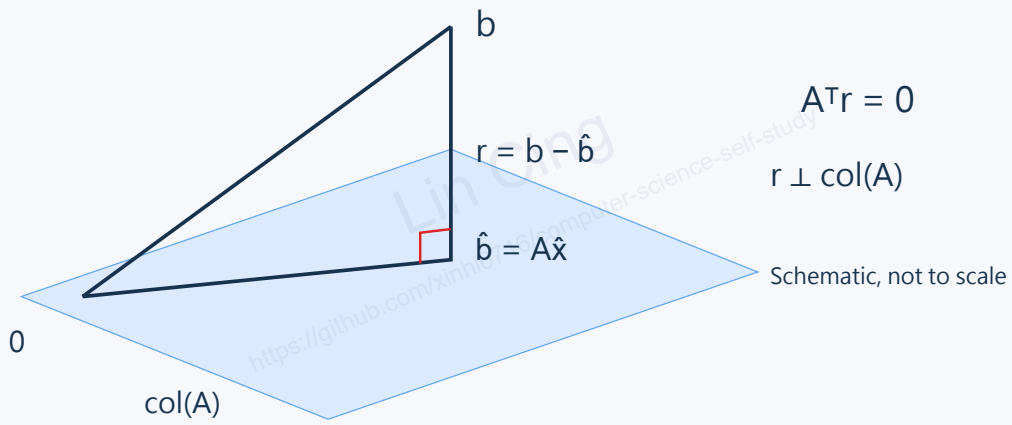

Red segments fix t; blue line b̂ = 7/6 + t/2



## Vertical residuals and data-space orthogonality

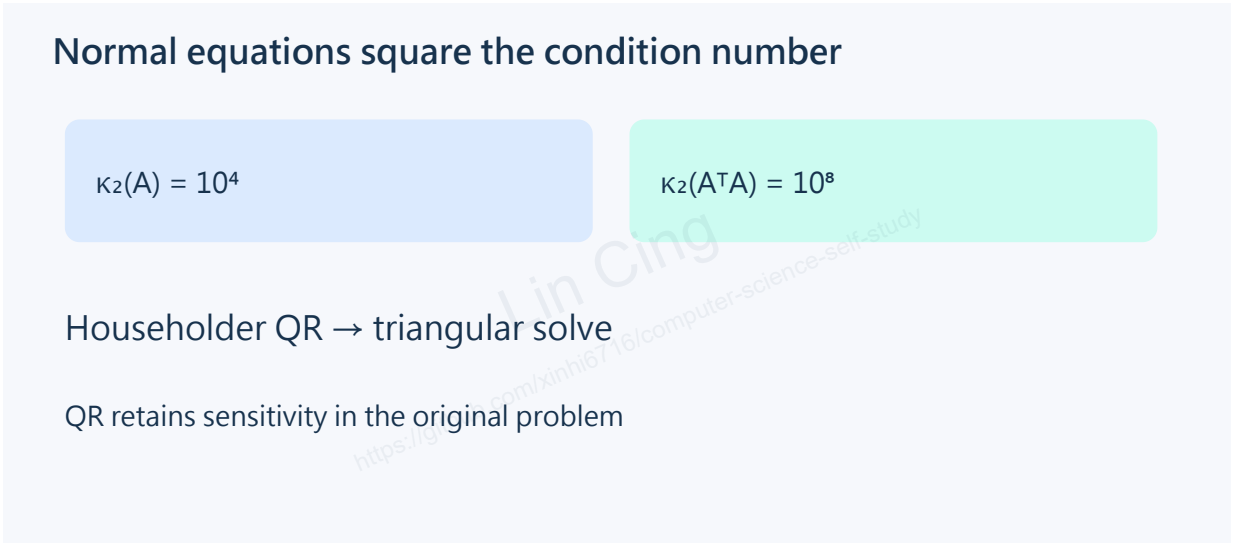
In the scatterplot, draw vertical segments at fixed $t_i$ with length $|b_i - \hat{b}_i|$. These are not shortest distances to the fitted line. In data space $\mathbb{R}^3$, $\hat{b} = A\hat{x} \in \mathrm{col}(A)$ and $r \perp \mathrm{col}(A)$. Check $A^T r = (-1/6 + 1/3 - 1/6,\ 1/3 - 2/6)^T = (0, 0)^T$.

### Orthogonal projection in data space R³



## Normal equations square the condition number

For full column rank, $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$. Since $A^TA$ has eigenvalues $\sigma_i^2$, $\kappa_2(A^TA) = \kappa_2(A)^2$. For $A = \mathrm{diag}(1, 10^{-4})$, the numbers are $10^4$ and $10^8$. Forming $A^TA$ in floating point makes weak directions harder to retain; there is no universal fixed digit loss.






## QR avoids forming the squared-condition matrix

With thin QR, $A = QR$ and $Q^TQ = I$. Solve $R\hat{x} = Q^Tb$. Here $q_1 = (1,1,1)^T/\sqrt{3}$, $q_2 = (-1,0,1)^T/\sqrt{2}$, $R = [\sqrt{3}, \sqrt{3}; 0, \sqrt{2}]$, and $Q^Tb = (5/\sqrt{3}, 1/\sqrt{2})^T$, giving $(7/6, 1/2)^T$ again. Use Householder QR in practice; use pivoted QR or SVD when rank is uncertain. QR does not remove ill-conditioning in the data.

## Any rank: the normal equation is necessary

Let $f(x) = \|b - Ax\|_2^2$. If $\hat{x}$ is a global minimizer, then for every direction $z$, the scalar function $g(t) = f(\hat{x} + tz)$ has a minimum at $t = 0$. Thus $g'(0) = -2z^T A^T (b - A\hat{x}) = 0$.

Because this holds for every $z$, $A^T(b - A\hat{x}) = 0$, or $A^T A\hat{x} = A^T b$. No full-column-rank or invertibility assumption is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Any rank: the normal equation is sufficient

Conversely, suppose $A^T(b - A\hat{x}) = 0$ and write $r = b - A\hat{x}$. For any $z$, expand:

$\|b - A(\hat{x} + z)\|^2 = \|r - Az\|^2 = \|r\|^2 - 2z^T A^T r + \|Az\|^2 = \|r\|^2 + \|Az\|^2 \geq \|r\|^2$.

Therefore $\hat{x}$ is a global minimizer. Equality holds exactly when $Az = 0$, so all minimizers are $\hat{x} + \ker(A)$. Moreover $A\hat{x} \in \mathrm{col}(A)$ and $r \perp \mathrm{col}(A)$, making it the unique orthogonal projection of $b$. The projection onto a finite-dimensional column space exists, hence at least one such $\hat{x}$ exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The projection matrix in the full-column-rank case

When $A \in \mathbb{R}^{m \times n}$ has full column rank, $A^T A$ is SPD and $\hat{x} = (A^T A)^{-1} A^T b$ is the unique least-squares solution. Define $P = A(A^T A)^{-1} A^T$; the fitted vector is $p = Pb$.

$P^T = P$ and $P^2 = P$, so $P$ is the orthogonal projector onto $\mathrm{col}(A)$. Equivalently, $A^T(b - Pb) = 0$ proves the residual is orthogonal. Without full column rank $(A^T A)^{-1}$ does not exist; use the SVD/pseudoinverse formulation instead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rank deficiency: the fitted vector is unique

For any real $A$ and $b$, $\min_x \|Ax - b\|_2^2$ has a solution, and all solutions satisfy $A^T Ax = A^T b$. The orthogonal projection $p$ of $b$ onto $\mathrm{col}(A)$ exists and is unique; every $x$ satisfying $Ax = p$ is a minimizer.

If the columns are dependent, $\ker(A)$ contains a nonzero vector. For one solution $x_0$, all solutions are precisely $x_0 + \ker(A)$: the fit $Ax = p$ stays unique while the coefficients need not. If $A = 0$, every $x$ gives the same residual.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A rank-deficient example

Take $A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 0 & 0 \end{bmatrix}$, $b = (2, 3, 1)^T$, and $u = (1, 2, 0)^T$. Then $p = u(u^T b)/(u^T u) = (8/5, 16/5, 0)^T$ and $r = b - p = (2/5, -1/5, 1)^T$; check $A^T r = 0$.

$Ax = p$ means $x_1 + x_2 = 8/5$, so all solutions are $(4/5, 4/5)^T + t(1, -1)^T$. All give the same fit. The minimum-norm coefficient vector occurs at $t = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## SVD selects the unique minimum-norm solution

Let $A = U\Sigma V^T$ have nonzero singular values $\sigma_1, \ldots, \sigma_r$ and define $A^+ = \sum_{i=1}^{r} \sigma_i^{-1} v_i u_i^T$. Then $x_* = A^+ b$ is the unique minimum-norm least-squares solution; all least-squares solutions are $x_* + z$ for $z \in \ker(A)$.

Since $x_* \in \ker(A)^\perp$, $\|x_* + z\|^2 = \|x_*\|^2 + \|z\|^2$; only $z = 0$ minimizes the norm. At rank zero the empty sum gives $A^+ = 0$. Numerical implementations need a scale-aware rank cutoff.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why evaluate polynomials?

For coefficient vectors of lengths $m$ and $n$, direct multiplication takes $O(mn)$ arithmetic operations. Evaluation at shared points turns multiplication into pointwise multiplication; interpolation returns coefficients.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Can point values determine the coefficients?

For $n$ points $x_0, \ldots, x_{n-1}$ in a field $F$, coefficient vector $a$ and value vector $y$ satisfy $y = Va$, where $V_{ij} = x_i^j$ for $0 \le i, j < n$. A square matrix alone does not guarantee a unique solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The Vandermonde determinant

$\det V = \prod_{0 \le i < j < n} (x_j - x_i)$. Thus $V$ is invertible over a field exactly when the points are pairwise distinct. Repeated points give equal rows.

For $1, 2, 4$, the determinant is $(2-1)(4-1)(4-2) = 6$. It is nonzero over the reals but zero over $\mathbb{F}_3$, where $1$ and $4$ coincide.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Proof: treat the last point as a variable

Proceed by induction. For $n = 1$, the determinant and empty product both equal 1. Treat $x_0, \ldots, x_{n-2}$ as distinct indeterminates, and let $D(t)$ be the determinant whose last row is $[1, t, \ldots, t^{n-1}]$.

$D$ has degree at most $n - 1$. Each substitution $t = x_i$ duplicates a row, so $D(x_i) = 0$. In $F[x_0, \ldots, x_{n-2}, t]$, these distinct monic linear factors are irreducible, so their product divides $D(t)$ and $D(t) = c \prod_{i=0}^{n-2} (t - x_i)$. The resulting polynomial identity can then be specialized to any field elements, including repeated points.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Proof: identify the leading coefficient

Expand along the last row. The only term containing $t^{n-1}$ has positive cofactor equal to the Vandermonde determinant for the first $n-1$ points.

Thus $c = \det V_{n-1} = \prod_{0 \le i < j < n-1} (x_j - x_i)$. Substitute $t = x_{n-1}$ to complete the induction. No distinctness assumption was needed in this identity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Lagrange interpolation constructs the solution

For pairwise distinct $x_0, \dots, x_{n-1}$ define $L_i(z) = \prod_{j \ne i} \frac{z - x_j}{x_i - x_j}$. Each denominator is nonzero and invertible in the field. Thus $L_i(x_k) = 1$ when $i = k$ and $0$ otherwise.

For arbitrary values $y_i$, the polynomial $P(z) = \sum_i y_i L_i(z)$ has degree less than $n$ and $P(x_i) = y_i$. If another degree-less-than-$n$ polynomial $Q$ has the same values, $P - Q$ has $n$ distinct roots and must be zero. The interpolant is unique, in agreement with invertibility of the Vandermonde matrix.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Computing interpolating coefficients in $O(n^2)$

Construct $G(z) = \prod_{j=0}^{n-1}(z - x_j)$ by multiplying one linear factor at a time. Updating the coefficient array of length $k+1$ at step $k$ takes $O(k)$, hence $O(n^2)$ field operations overall.

For each $i$, synthetic division computes $Q_i(z) = G(z)/(z - x_i)$ in $O(n)$. Horner evaluation gives $d_i = Q_i(x_i) = \prod_{j \neq i}(x_i - x_j) \neq 0$. Add the $n$ coefficients of $y_i d_i^{-1} Q_i$ to the answer. The $n$ iterations each cost $O(n)$, so the total is $O(n^2)$ field operations, under the unit-cost field-arithmetic model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fewer than $n$ points do not ensure uniqueness

Given $m < n$ distinct points $x_0, \ldots, x_{m-1}$, both $P(z) = 0$ and $Q(z) = \prod_{i=0}^{m-1}(z - x_i)$ vanish at all the points. They are different polynomials, both of degree less than $n$.

For example, to determine a polynomial of degree at most two, knowing only $P(0) = P(1) = 0$ admits both $P(z) = 0$ and $P(z) = z(z-1)$. A value at a third distinct point permits the earlier $n$-point uniqueness theorem to apply.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Horner evaluation and its loop invariant

For $A(z) = \sum_{j=0}^{n-1} a_j z^j$, fix an evaluation point $x$, set $b = 0$, and update $b \leftarrow a_k + xb$ for $k = n-1, n-2, \ldots, 0$. After processing $k$, the invariant is $b = \sum_{j=k}^{n-1} a_j x^{j-k}$. Initialization is the empty sum; each update multiplies the prior sum by $x$ and adds $a_k$. At termination $k = 0$, so $b = A(x)$.

For $A(z) = 2 + 3z + 4z^2$ and $x = 2$, the successive values are $4$, $3 + 2 \cdot 4 = 11$, and $2 + 2 \cdot 11 = 24$. This implementation takes $\Theta(n)$ field operations for $n$ coefficients. Repeating it for $n$ points costs $\Theta(n^2)$ for this method; this is not a lower bound for all multipoint evaluation methods.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Set the transform convention

For $n = 2^s$, let $\omega_n = e^{2\pi i/n}$ and $X_k = \sum_{j=0}^{n-1} x_j \omega_n^{jk}$. This is the positive-exponent DFT; indices range from $0$ to $n-1$.

$\omega_4 = i$

k: 0 1 2 3

$\omega_4^k$: 1 i −1 −i

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Roots and periodicity

$\omega_n^n = 1$, $\omega_n^{k+n} = \omega_n^k$, and $\omega_n^{k+n/2} = -\omega_n^k$ for even $n$. For $n = 4$, the roots are $1, i, -1, -i$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Split even and odd coefficients

Write $A(z) = E(z^2) + zO(z^2)$ where $E(t) = \sum_r x_{2r} t^r$ and $O(t) = \sum_r x_{2r+1} t^r$. The two half-size DFTs reuse the same squared roots.

A(z) = E(z²) + z O(z²)

E: x₀, x₂, x₄, …

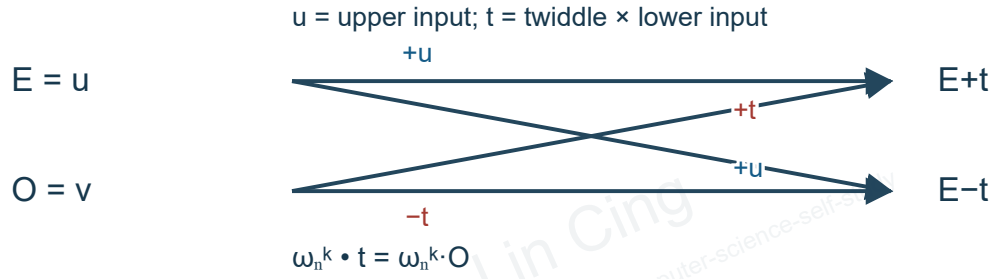
O: x₁, x₃, x₅, …

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Combine a pair

For $0 \le k < n/2$, set $t = \omega_n^k O_k$. Then $X_k = E_k + t$ and $X_{k+n/2} = E_k - t$. The pair is a butterfly; the multiplier is the twiddle factor.

### One butterfly




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why the cost is $O(n \log n)$

The recurrence $T(n) = 2T(n/2) + O(n)$ has $\log_2 n$ levels and $O(n)$ work per level. The iterative in-place version has $n/2$ butterflies per stage and $\log_2 n$ stages.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Bit-reverse the input

For $n = 4$, reverse the two-bit indices: $00, 01, 10, 11 \mapsto 00, 10, 01, 11$. Thus $[1, 2, 0, -1]$ becomes $[1, 0, 2, -1]$. The permutation prepares the iterative decimation-in-time stages.

index: 00 01 10 11

reverse: 00 10 01 11

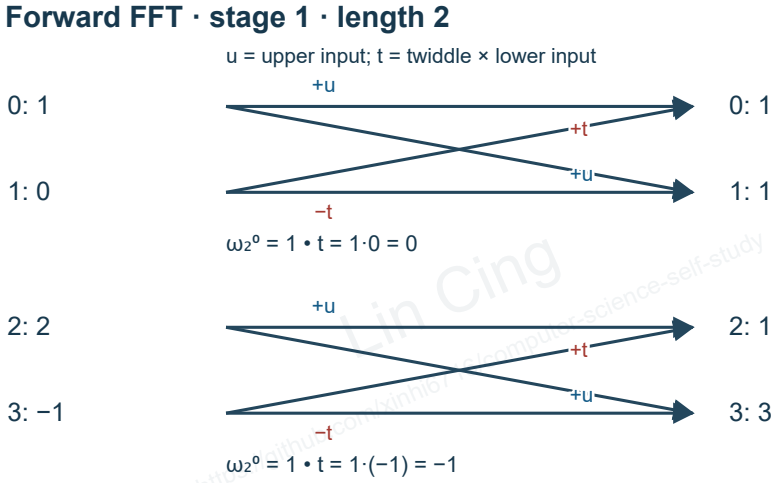
values: 1 0 2 −1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## First stage, length 2

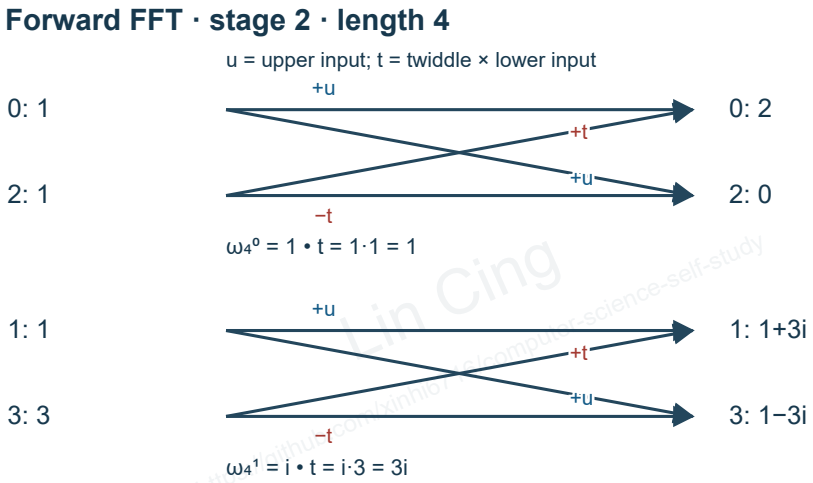
For $[1, 0, 2, -1]$, both twiddles are $\omega_2^0 = 1$. Indices $(0, 1)$ give $(1 + 0, 1 - 0) = (1, 1)$; indices $(2, 3)$ give $(2 - 1, 2 - (-1)) = (1, 3)$. State: $[1, 1, 1, 3]$.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Second stage, length 4

Input $[1, 1, 1, 3]$. Pair $(0, 2)$ uses $\omega_4^0 = 1$: $(1 + 1, 1 - 1) = (2, 0)$. Pair $(1, 3)$ uses $\omega_4^1 = i$: $(1 + 3i, 1 - 3i)$. In index order $X = [2, 1 + 3i, 0, 1 - 3i]$.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



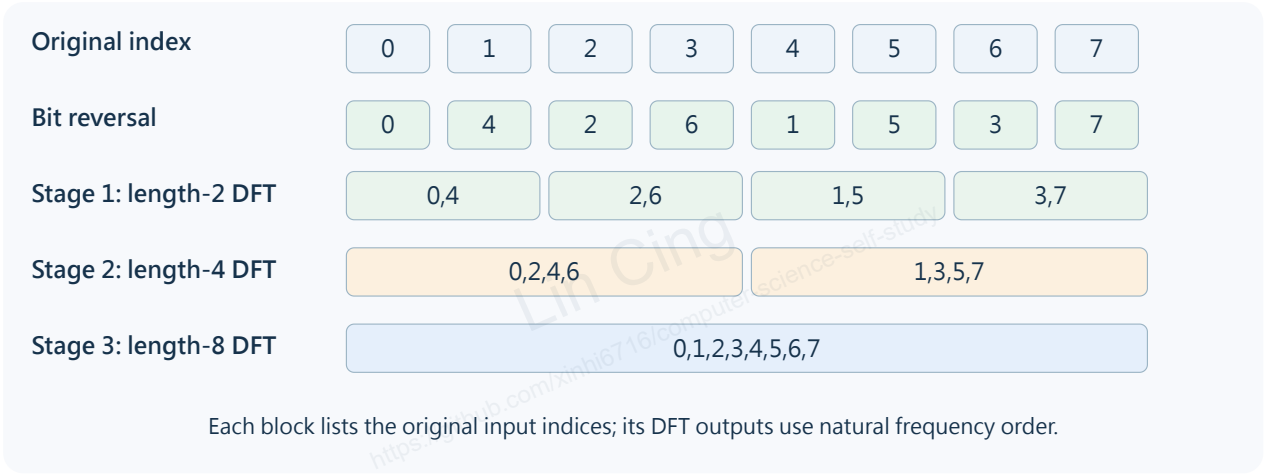
## Check with the definition

Direct sums give $X_0 = 1 + 2 + 0 - 1 = 2$, $X_1 = 1 + 2i + (-1)(-i) = 1 + 3i$, $X_2 = 1 - 2 + 0 + 1 = 0$, and $X_3 = 1 - 2i + (-1)i = 1 - 3i$. This independently checks indexing and signs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The iterative butterfly invariant

Let $N = 2^\ell$. After stage $s$ ($0 \le s \le \ell$), each contiguous block of length $2^s$ stores a length-$2^s$ DFT in natural frequency order. Block $b$ is the DFT of the original subsequence $x_{r+tN/2^s}$, $0 \le t < 2^s$, where $r$ is the reversal of the $\ell - s$ bits of $b$.




Each block lists the original input indices; its DFT outputs use natural frequency order.



## Why each butterfly preserves the invariant

At $s = 0$, each singleton is in bit-reversed order and the claim holds. Assume it holds at stage $s$: the two halves of each next-stage block are DFTs of the even and odd positions of one larger subsequence. For each $k$, combine them as $E_k \pm \omega_{2^{s+1}}^k O_k$ in positions $k$ and $k + 2^s$; the result is the DFT of that larger subsequence.

The old subsequences have stride $N/2^s$ and their starting indices differ by $N/2^{s+1}$, exactly the even/odd split of the new subsequence. Induction to $s = \ell$ gives the entire input DFT in natural frequency order.

## Bit reversal has a cost

For $N = 2^\ell$, let $\mathrm{rev}_\ell(i)$ reverse the $\ell$ bits of $i$. Scanning all $\ell$ bits independently for every index takes $O(N \log N)$ work. A reversed counter updates only the bits affected by a carry; over all $N$ updates these flips total $O(N)$, so all indices can be generated in $O(N)$ work.

For in-place permutation, swap only when $i < \mathrm{rev}_\ell(i)$ so each pair is swapped once. The earlier parallel FFT assumes the map is supplied when stating $O(1)$ permutation depth; generating it must otherwise be charged separately. Word size and instruction model affect the bit-operation cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The iterative algorithm

Bit-reverse $x$. For $L = 2, 4, \ldots, n$, for each block beginning at $b$ and $0 \le j < L/2$, set $u = a_{b+j}$ and $t = \omega_L^j a_{b+j+L/2}$; assign $(a_{b+j}, a_{b+j+L/2}) = (u + t, u - t)$. Save $u$ and $t$ before writing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Invert the DFT

The inverse is $x_j = \frac{1}{n}\sum_{k=0}^{n-1} X_k \omega_n^{-jk}$. Apply the same FFT with inverse roots, then divide each result by $n$.

bit reverse → inverse-root FFT → divide by n

[2, 1+3i, 0, 1−3i] → [1, 2, 0, −1]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



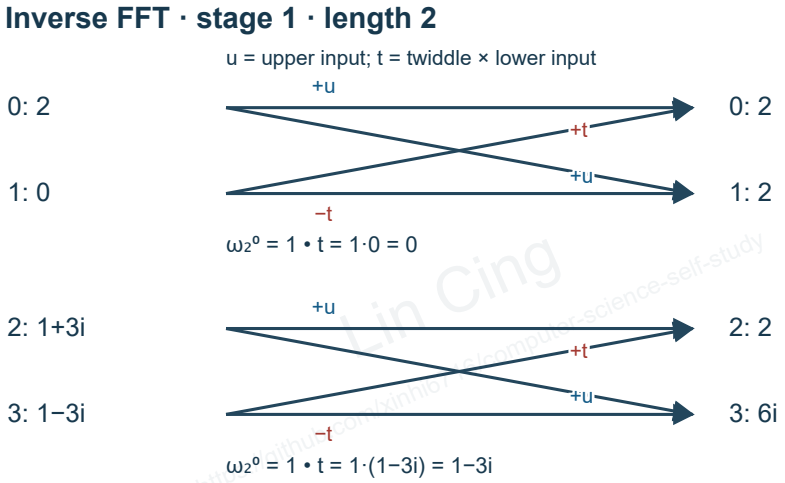
## Why division by $n$ works

Substitute $X_k$: $\frac{1}{n}\sum_r x_r \sum_k \omega_n^{k(r-j)}$. The inner geometric sum is $n$ when $r = j$ and $0$ otherwise because $0 \le r, j < n$. Hence only $x_j$ remains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Inverse example, first stage

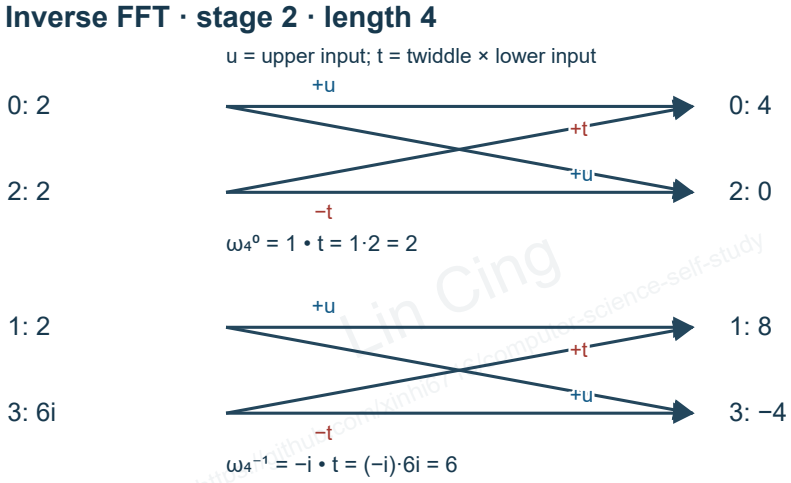
Bit-reverse $[2, 1+3i, 0, 1-3i]$ to $[2, 0, 1+3i, 1-3i]$. With length-2 twiddle $1$, the pairs produce $[2, 2, 2, 6i]$.






# Inverse example, second stage

Use $\omega_4^{-1} = -i$. Pair $(0, 2)$ yields $(4, 0)$ with twiddle $1$. Pair $(1, 3)$ has $(-i)(6i) = 6$ and yields $(8, -4)$. Divide $[4, 8, 0, -4]$ by $4$ to recover $[1, 2, 0, -1]$.



## Recursive FFT space: count what is live at once

In a sequential depth-first implementation, suppose each level allocates temporary arrays proportional to its subproblem and releases them when finished. Along one recursion path the live capacity is at most $N + N/2 + N/4 + \cdots < 2N$; including input/output arrays and an $O(\log N)$ call stack gives $O(N)$ peak extra space.

If instead all arrays at every level are retained for tracing or reuse, each level holds $N$ elements and there are $\log_2 N + 1$ levels, so storage is $\Theta(N \log N)$. Both approaches may perform $\Theta(N \log N)$ work; retention policy, not recursion depth alone, accounts for the space difference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why applying the forward DFT twice is not inversion

Under $X_k = \sum_j x_j \omega^{jk}$, applying the same forward transform again and dividing by $N$ gives $\frac{1}{N} \sum_k X_k \omega^{k\ell} = \frac{1}{N} \sum_j x_j \sum_k \omega^{k(j+\ell)} = x_{(-\ell) \bmod N}$.

The inner sum is $N$ only when $j + \ell \equiv 0 \pmod N$ and is zero otherwise. Thus the result reverses cyclic indices. The inverse instead uses $\omega^{-1}$ and divides by $N$, or follows the second forward transform with index reversal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A concrete counterexample

Let $N = 4, \omega = i$ and $x = [1, 2, 0, 0]$. The first forward DFT is $[3, 1 + 2i, -1, 1 - 2i]$. Applying the same forward root a second time and dividing by 4 produces $[1, 0, 0, 2]$, not the input $[1, 2, 0, 0]$. The issue is the direction of the exponent, not a missing factor of $N$; using inverse root $-i$ restores the input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Work and depth of parallel FFT

Use a synchronous random-access EREW PRAM, with $N = 2^s \geq 2$ and a given bit-reversal address map. Assign $N/2$ processors per stage. Each processor reads one disjoint input pair, computes one butterfly and writes the pair back; distinct butterflies access disjoint locations. A barrier separates stages. Assume twiddle factors are precomputed.

Bit-reversal copying takes $O(N)$ work and $O(1)$ time; each of $\log_2 N$ stages takes $\Theta(N)$ work and $\Theta(1)$ time. Therefore $W = \Theta(N \log N)$, depth $D = \Theta(\log N)$ and extra storage $O(N)$. This is a model result, not a promise for every machine. Charge index or twiddle generation separately if they are not supplied.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Linear versus circular convolution

Pointwise multiplying length-$N$ transforms gives circular convolution $c_k = \sum_{j=0}^{N-1} a_j b_{(k-j) \bmod N}$. Without enough padding, coefficients wrap and add to earlier indices.

m + n − 1 ≤ N

[1, 2, 0, 0] × [3, 4, 0, 0]

linear output: [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Pad before multiplying

For lengths $m, n \geq 1$, choose a power of two $N \geq m + n - 1$, append zeros to both inputs, FFT, multiply pointwise, and inverse FFT. The first $m + n - 1$ outputs are the linear convolution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Convolution example

$[1,2] * [3,4] = [3,10,8]$. With $N = 4$, pad to $[1,2,0,0]$ and $[3,4,0,0]$. The inverse pointwise product gives $[3,10,8,0]$; $N = 2$ would instead wrap $8$ into the constant term.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Complex FFT convolution: two forward transforms

For $A(z) = 1 + 2z$, $B(z) = 3 + 4z$, the linear convolution has length $2 + 2 - 1 = 3$, so pad to $N = 4$. Use the positive-exponent convention $\omega = i$: $A = [1,2,0,0]$, $B = [3,4,0,0]$.

Evaluate $X_k = \sum_j x_j i^{jk}$ at $1, i, -1, -i$: $\widehat{A} = [3, 1+2i, -1, 1-2i]$ and $\widehat{B} = [7, 3+4i, -1, 3-4i]$. For example, $A(i) = 1 + 2i$ and $B(-1) = 3 - 4 = -1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Multiply matching frequencies and invert

The pointwise products are $C = [21, -5+10i, 1, -5-10i]$; for example $(1+2i)(3+4i) = -5+10i$. The inverse uses $i^{-1} = -i$: $c_j = \frac{1}{4}\sum_{k=0}^{3} C_k(-i)^{jk}$.

The four unscaled sums are $[12, 40, 32, 0]$, so $c = [3, 10, 8, 0]$. Direct multiplication confirms $(1+2z)(3+4z) = 3+10z+8z^2$. Pair values at the same frequency, not across frequencies.

| Frequency | Value of A | Value of B | | Pointwise product |
|---|---|---|---|---|
| 1 | 3 | 7 | × | 21 |
| i | 1+2i | 3+4i | × | −5+10i |
| −1 | −1 | −1 | × | 1 |
| −i | 1−2i | 3−4i | × | −5−10i |

Inverse: [21, −5+10i, 1, −5−10i] → [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fourier transforms over a finite field

Choose a prime $p$, let $N = 2^s$ and $N \mid (p-1)$. Choose $\omega \in \mathbb{F}_p$ of multiplicative order exactly $N$: $\omega^N = 1$ and $\omega^k \neq 1$ for $1 \le k < N$. Such a root exists because the nonzero elements form a cyclic group of order $p-1$.

Define $X_k = \sum_{j=0}^{N-1} x_j \omega^{jk} \pmod{p}$; all arithmetic takes place in $\mathbb{F}_p$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why the inverse works

Since $N \mid p-1$, $p \nmid N$, so $N^{-1}$ exists. If $0 < r < N$, then $\omega^r \neq 1$ and $1-\omega^r$ is invertible; consequently $\sum_{k=0}^{N-1} \omega^{kr} = 0$. For $r = 0$, the sum is $N$.

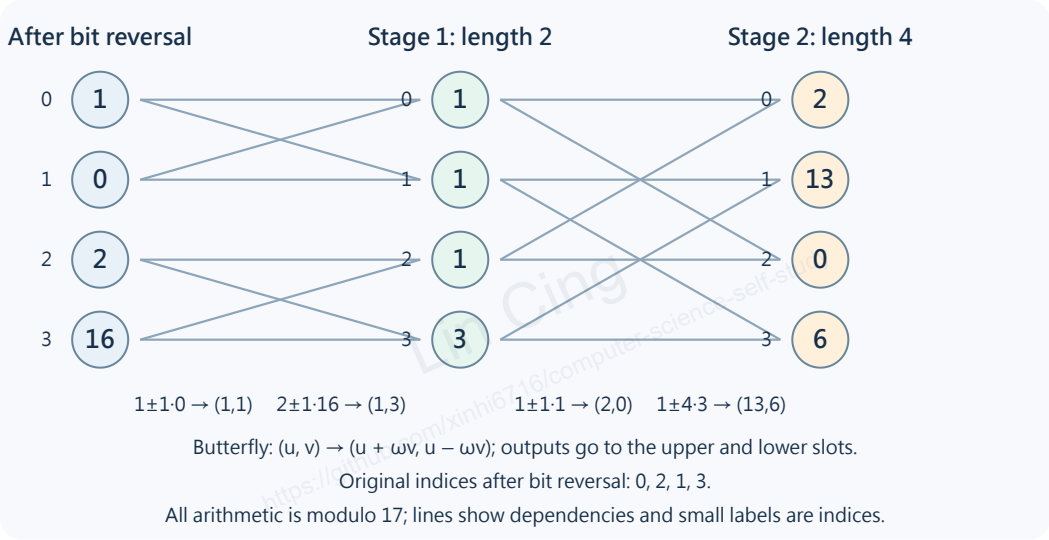
Substitution and exchange of sums give $x_j = N^{-1} \sum_{k=0}^{N-1} X_k \omega^{-jk} \pmod{p}$. The proof uses invertibility of nonzero field elements; it does not automatically extend to composite moduli.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## NTT example: two butterfly stages modulo 17

Take $N = 4$, $p = 17$, $\omega = 4$. Since $4^2 = 16 = -1$ and $4^4 = 1$, its order is exactly 4; $4^{-1} = 13$. Input $[1, 2, 0, -1]$ is $[1, 2, 0, 16]$ modulo 17.




Direct evaluation also gives $[2, 13, 0, 6]$. Interpret every negative value modulo 17.

## Pointwise product and integer recovery

Pad $A = [1, 2]$ and $B = [3, 4]$ to $N = 4$. Their NTTs are $[3, 9, 16, 10]$ and $[7, 2, 16, 4]$; their pointwise product is $[4, 1, 1, 6]$. The inverse is $[3, 10, 8, 0]$, giving $3 + 10z + 8z^2$.

Linear rather than cyclic convolution requires $N \geq r + s - 1$. A modular result identifies only a residue. If every integer coefficient has known absolute bound $M$ and $p > 2M$, its symmetric representative is unique. Here $r = s = 2$, $|a_i|, |b_i| \leq 4$ gives the coarse bound $M = 2 \cdot 4^2 = 32$, but $17 \not> 64$; the residue $10$ also represents $-7$, so without a sufficiently tight range bound the integer coefficient cannot be identified uniquely.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Signed integer convolution with a sufficient bound

Take $A = [1, 2]$ and $B = [1, -1]$, again with $p = 17$, $N = 4$, $\omega = 4$. Their padded transforms are $[3, 9, 16, 10]$ and $[0, 14, 2, 5]$. The pointwise product $[0, 7, 15, 16]$ inverses to $[1, 1, 15, 0]$.

Both inputs have coefficient magnitude at most $B = 2$; each convolution coefficient contains at most $\min(2, 2) = 2$ products, so $M = 2B^2 = 8$. Since $p = 17 > 2M = 16$, the symmetric representatives in $[-8, 8]$ are unique: residue $15$ decodes to $-2$, giving $[1, 1, -2, 0]$.

A length-$N$ FFT uses $\Theta(N \log N)$ arithmetic operations modulo $p$. If $p$ grows with the input, the bit cost of modular additions and multiplications depends on the arithmetic method for $\log p$-bit integers; it is not automatically constant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Precision and cost

Three transforms and $N$ pointwise products take $O(N \log N)$ arithmetic and $O(N)$ storage. This is an arithmetic-operation bound, not a bit-complexity guarantee. Floating-point roundoff accumulates; rounding to integer coefficients is valid only when every absolute error is proved below $1/2$. Otherwise use higher precision or an exact method.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Choose the right method

For small inputs direct multiplication may be simpler. FFT requires suitable padding and a numerical error budget; for exact integer results, consider number-theoretic transforms with suitable moduli and reconstruction bounds. Check convention, indices, and normalization at interfaces.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References and attribution

CLRS, Introduction to Algorithms, third edition, chapters 28 and 30. Matrix: PDF pages 834–861. FFT: PDF pages 919, 921, 923, 928–936, 939–941, 944–945 (PDF page numbering, third edition). MIT 6.046J Fall 2004 lecture 3 slides, PDF pages 27 and 35–43, and Fall 2005 lecture 3 notes support matrix multiplication and Strassen, not FFT.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Number-theoretic algorithms

Build gcd, modular equations, exponentiation, primality testing, and RSA from divisibility and invariants.

## Divisibility and congruence

For $n>0$, $a \equiv b \pmod{n}$ means $n$ divides $a-b$. For example $29 \equiv 5 \pmod{12}$, since $29-5=2 \cdot 12$. Reduce sums and products term by term.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Euclid’s algorithm

For gcd(252,198): 252=1 • 198+54; 198=3 • 54+36; 54=1 • 36+18; 36=2 • 18+0. The gcd is 18.

$$252 = 1 \times 198 + 54$$

$$198 = 3 \times 54 + 36$$

$$54 = 1 \times 36 + 18$$

$$36 = 2 \times 18 + 0$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The gcd invariant

If a=qb+r, the common divisors of a,b are exactly those of b,r: r=a−qb and a=qb+r. For inputs not both zero, normalized to absolute values, the last nonzero remainder is the gcd.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Extended Euclid table

Maintain r=252s+198t. Rows (r,s,t): (252,1,0), (198,0,1), (54,1,−1), (36,−3,4), (18,4,−5), (0,−11,14). Thus 18=4 • 252−5 • 198.

- $252 = 1\times252 + 0\times198$
- $198 = 0\times252 + 1\times198$
- $54 = 1\times252 - 1\times198$
- $36 = -3\times252 + 4\times198$
- $18 = 4\times252 - 5\times198$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Coefficient update and termination

From rows $(r_0,s_0,t_0)$, $(r_1,s_1,t_1)$, set $q=\lfloor r_0/r_1 \rfloor$. The next row is $(r_0-qr_1,s_0-qs_1,t_0-qt_1)$. Use this update only while $r_1>0$. Then $0\le r_0-qr_1<r_1$; the linear combination persists and the positive remainders decrease.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Integer inputs and Bézout coefficients

For $(a,b)=(0,0)$, define gcd=0 and choose $x=y=0$. Otherwise start with $r_0=|a|$, $r_1=|b|$ and coefficient rows $(1,0)$, $(0,1)$ for $|a|$ and $|b|$. Stop when $r_1=0$, before division. The last nonzero $r_0$ is gcd(a,b).

If $r_0=s|a|+t|b|$, return $x=s\cdot\operatorname{sign}(a)$, $y=t\cdot\operatorname{sign}(b)$, with $\operatorname{sign}(0)=0$. Then $ax+by=\gcd(a,b)$, including either zero input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Signed input examples

For (7,−3), use (7,3): 7=2 • 3+1; 3=3 • 1+0. Back substitute 1=7−2 • 3=1 • 7+2 • (−3), so (x,y)=(1,2).

For (−12,18), use (12,18): 12=0 • 18+12; 18=1 • 12+6; 12=2 • 6+0. Back substitute 6=18−12=1 • (−12)+1 • 18, so (x,y)=(1,1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Zero input examples

For (0,−5), use (0,5): 0=0 • 5+0, then stop. The gcd is 5=0 • 0+(−1) • (−5), so (x,y)=(0,−1).

For (0,0), no division occurs. By convention gcd(0,0)=0=0 • 0+0 • 0, with (x,y)=(0,0).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Euclid: worst-case divisions

For $a \geq b > 0$, suppose Euclid produces $k$ nonzero remainders. Reverse the recurrence: the smallest possible inputs are consecutive Fibonacci numbers, so $b \geq F_{k+1}$ and $F_j \geq \phi^{j-2}$ for $j \geq 2$, where $\phi = (1+\sqrt{5})/2$. Thus there are $O(\log(b+1))$ divisions, including $b=1$. As $b$ grows, consecutive Fibonacci inputs attain $\Theta(\log b)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why Fibonacci numbers appear

Let $g = \gcd(a,b) > 0$ and divide every remainder by $g$. The final normalized remainders are 1,0; the unnormalized final nonzero remainder is $g$. Working backward through the normalized sequence, $r_{i-1} = q_i r_i + r_{i+1}$ with $q_i \geq 1$, so $r_{i-1} \geq r_i + r_{i+1}$. Taking quotient 1 yields the Fibonacci lower bound. For example, $34 = 21+13$, $21 = 13+8$, $13 = 8+5$, $8 = 5+3$, $5 = 3+2$, $3 = 2+1$, $2 = 2 \cdot 1$.

### Reconstruct the smallest remainders

Each value is at least the sum of the next two

34 ▶ 21 ▶ 13 ▶ 8 ▶ 5 ▶ 3 ▶ 2 ▶ 1

Equality occurs when the quotient is 1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Division count versus bit cost

If both inputs have at most B bits, classical long division costs $O(B^2)$ bit operations per division. Combining that with O(B) divisions yields the simple $O(B^3)$ upper bound. This bound is deliberately conservative, not a claim of optimal implementation; O(B) arithmetic divisions is not the same as O(B) bit operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Euclid halves within two divisions

For $a \geq b > 0$ let $r = a \bmod b$. If $r \leq b/2$, the second argument has at least halved in one division. If $r > b/2$, then $b/2 < r < b$, so the next quotient is 1 and $b \bmod r = b - r < b/2$. Thus within at most two divisions the new second argument is at most half the old one, or the algorithm terminates. This gives $O(\log(b+1))$ divisions without treating large-integer division as constant bit cost.

For example $391 = 299 + 92$, $299 = 3 \cdot 92 + 23$ and $92 = 4 \cdot 23 + 0$, giving gcd=23. Back-substitution yields $23 = 299 - 3(391 - 299) = 4 \cdot 299 - 3 \cdot 391$. Because $\gcd(299, 391) = 23 \neq 1$, 299 has no inverse modulo 391: an inverse would imply $299u - 391v = 1$ although the left side is divisible by 23.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Solvability of linear congruences

ax≡b (mod n) iff there is an integer y with ax+ny=b. Let d=gcd(a,n). For n>0, a solution exists iff d|b, and when it does there are exactly d distinct residues modulo n: necessity follows from divisibility; sufficiency follows by multiplying Bézout’ s identity by b/d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## All solutions of a congruence

Solve 14x≡30 (mod 100). Divide by d=2: 7x≡15 (mod 50). Since $7^{-1}\equiv 43$, x≡45 (mod 50). Exactly two residues modulo 100, 45 and 95, both yield remainder 30.

- 14x ≡ 30 (mod 100)
- 7x ≡ 15 (mod 50)
- x ≡ 45 (mod 50)
- x = 45, 95 (mod 100)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Modular inverses

a has an inverse modulo n iff gcd(a,n)=1. For example $7 \cdot 43 = 301 \equiv 1 \pmod{50}$. Extended Euclid supplies the Bézout coefficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Linear congruences: compact solution sets

Solve $ax \equiv b \pmod{n}$, $n \geq 1$. Let $g = \gcd(a,n)$. If g does not divide b there is no solution. If g divides b, divide the equation and modulus by g. When $step = n/g > 1$, $a/g$ is coprime to step and its inverse gives one $x_0$. When step=1, take $x_0 = 0$ directly without invoking an inverse modulo 1. The complete set modulo n is $x_0 + j \cdot step$ for $0 \leq j < g$, where $step = n/g$ and $0 \leq x_0 < step$. The tuple $(x_0, step, g)$ encodes the full set.

Example: $24x \equiv 36 \pmod{60}$ has $g = 12$ and reduces to $2x \equiv 3 \pmod{5}$. Thus $x_0 = 4$, step=5, and (4,5,12) represents the 12 solutions 4,9,…,59. Every residue solves $0x \equiv 0 \pmod{n}$, represented by (0,1,n), including n=1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Compact output versus enumeration

After reducing a,b modulo n, each tuple component is at most n, so the representation uses O(log(n+1)) bits. That is output size alone, not the computation cost for gcd or an inverse.

Listing all g solutions requires at least $\Omega(g)$ output operations. For $0x \equiv 0 \pmod{n}$, $g=n$; writing all representatives in ordinary binary takes $\Theta(n \log(n+1))$ output bits. Given n in $\beta=\lceil \log_2(n+1) \rceil$ bits, n can grow exponentially in β. A compact representation and explicit enumeration therefore have different costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Modular division means multiplying an inverse

Only residues coprime to n have multiplicative inverses modulo n. If $b \in U(n)$, a/b denotes $a \cdot b^{-1} \pmod{n}$, not integer division followed by reduction. For example $7 \cdot 13=91 \equiv 1 \pmod{15}$, so $4/7 \equiv 4 \cdot 13 \equiv 7 \pmod{15}$. A non-unit denominator cannot be cancelled directly; solve the linear congruence and check its gcd condition instead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Chinese remainder theorem

For pairwise coprime moduli the prescribed residues have one solution modulo their product. For $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, candidates 2,5,8 show $x \equiv 8 \pmod{15}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Non-coprime compatibility

$x \equiv a \pmod m$, $x \equiv b \pmod n$ is solvable iff $\gcd(m,n) \mid (b-a)$. The solution is unique modulo $\operatorname{lcm}(m,n)$. Substitute $x = a + mt$ to obtain $mt \equiv b - a \pmod n$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Non-coprime example

For $x \equiv 2 \pmod 6$, $x \equiv 5 \pmod 9$, gcd=3 divides the difference 3. Solve $6t \equiv 3 \pmod 9$, obtaining $t \equiv 2 \pmod 3$, hence $x \equiv 14 \pmod{18}$. Replacing 5 by 4 makes the system impossible.

$$x = 2 + 6t$$

$$6t \equiv 3 \pmod 9$$

$$t \equiv 2 \pmod 3$$

$$x \equiv 14 \pmod{18}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## CRT construction for coprime moduli

For pairwise coprime $m_1,\ldots,m_k>1$ and arbitrary residues $a_i$, set $M=\prod m_i$ and $M_i=M/m_i$. Choose $y_i=M_i^{-1} \bmod m_i$; then $x \equiv \Sigma a_i M_i y_i \pmod M$.

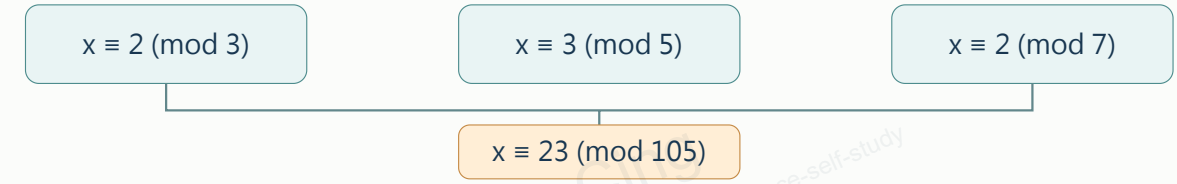
Modulo $m_i$, only the i-th term survives, and it equals $a_i$. If x and x′ are solutions, every $m_i$ divides $x-x'$, so pairwise coprimality implies M divides the difference. The solution is unique modulo M.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## CRT example

For $x\equiv 2 \pmod 3$, $x\equiv 3 \pmod 5$, $x\equiv 2 \pmod 7$, $M=105$ and $(M_1,M_2,M_3)=(35,21,15)$. Their respective inverses are $(2,1,1)$, so $x\equiv 2\cdot 35\cdot 2+3\cdot 21+2\cdot 15=233\equiv 23 \pmod{105}$. Check the three remainders: 2, 3, 2.

**One integer satisfies all three constraints**




23 has remainders 2, 3, 2 modulo 3, 5, 7

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## CRT for non-coprime moduli

The congruences $x\equiv a \pmod m$, $x\equiv b \pmod n$ are solvable exactly when $d=\gcd(m,n)$ divides $b-a$. Necessity follows by subtracting the congruences. For sufficiency put $x=a+mt$ and solve $(m/d)t\equiv(b-a)/d \pmod{n/d}$. Since $m/d$ and $n/d$ are coprime, $t$ is unique modulo $n/d$ and $x$ is unique modulo $\operatorname{lcm}(m,n)$. Merge several constraints successively; stop if a pair is incompatible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Modular exponentiation

For $m>1$, $e\geq 0$, initialize accumulator=1 and scan all bits of e from most to least significant. Square acc modulo m, then multiply by a modulo m if the bit is 1. For e=0 the result is 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Bit-by-bit exponentiation

Compute $7^{13} \bmod 23$, with $13=(1101)_2$. Starting from 1, the first bit gives $1^2 \cdot 7 \bmod 23=7$. The second gives $7^2 \bmod 23=3$, then $3 \cdot 7=21$. After each bit the accumulator is 7, 21, 4, 20. The answer is 20.

bits: 1 1 0 1

acc: 1 → 7 → 21 → 4 → 20

$7^{13} \bmod 23 = 20$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Exponentiation invariant

After a prefix representing k, acc$\equiv a^k \pmod m$. Appending bit b forms 2k+b; squaring and multiplying by $a^b$ establishes the next row. O(log(e+1)) counts modular multiplications, not bit operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Right-to-left modular exponentiation

Inputs are $A \in \mathbb{Z}$, $B \geq 0$, $n \geq 1$. Initialize r=1 mod n, a=A mod n and b=B. While b>0, if b is odd set r←ra mod n, then set a←$a^2$ mod n and b←$\lfloor b/2 \rfloor$. The invariant is $r \cdot a^b \equiv A^B \pmod n$.

For b=2q, the updated product is $r(a^2)^q = r \cdot a^b$. For b=2q+1 it is $(ra)(a^2)^q = r \cdot a^b$. Each round halves b; at b=0 the invariant yields $r \equiv A^B$. If B=0 return 1 mod n immediately; in particular n=1 yields 0. Non-coprime bases work because no inverse is used. Negative exponents are outside this algorithm; they require a separately defined inverse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Right-to-left modular exponentiation trace

For $7^{19} \bmod 23$ start at (r,a,b)=(1,7,19). Each row preserves the invariant; the final result is 11.

| Before (r,a,b) | After (r,a,b) |
|---|---|
| (1,7,19) | (7,3,9) |
| (7,3,9) | (21,9,4) |
| (21,9,4) | (21,12,2) |
| (21,12,2) | (21,6,1) |
| (21,6,1) | (11,13,0) |

For example, in row 3, b=4 is even: r remains 21, a becomes $9^2 \equiv 12$ and b becomes 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Finite closed subsets and element order

A nonempty subset S of a finite group G that is closed under the group operation is a subgroup. For $a \in S$, all $a, a^2, \ldots$ lie in S; finiteness gives $a^i = a^j$ for some $1 \le i < j$. Cancellation in G gives $a^{j-i} = e \in S$. If $j - i > 1$, $a^{j-i-1} \in S$ is the inverse of a; if $j - i = 1$ then $a = e$, whose inverse is also in S. Finiteness is essential.

The order of a is the least positive t with $a^t = e$. The elements $e, a, \ldots, a^{t-1}$ are distinct and all powers repeat modulo t, so $|\langle a \rangle| = t$. Lagrange gives $t \mid |G|$ and hence $a^{|G|} = e$. In additive $Z_8$, 2 generates {0,2,4,6} and has order 4; in multiplicative U(7), 2 generates {1,2,4} and has order 3. The operation determines the meaning of $\langle 2 \rangle$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Lagrange’ s theorem: partition into cosets

Let G be a finite group and H a subgroup. A left coset is $aH=\{ah:h\in H\}$. The map $h\mapsto ah$ is a bijection: left multiplication by $a^{-1}$ recovers h. Thus every coset has $|H|$ elements.

Two cosets are either identical or disjoint: if $ah_1=bh_2$, then $b^{-1}a=h_2h_1^{-1}\in H$ and consequently $aH=bH$. Every a lies in aH because H contains the identity. Thus the cosets partition G into equal-sized blocks and $|G|=[G:H]|H|$. In particular $|H|$ divides $|G|$. For $H=\langle g\rangle$, $\mathrm{ord}(g)$ divides $|G|$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Euler ϕ: prime powers and multiplicativity

For prime p and integer $e\geq 1$, precisely $p^{e-1}$ of $1,\ldots,p^e$ are divisible by p; all other entries are coprime to $p^e$. Therefore $\phi(p^e)=p^e-p^{e-1}=p^{e-1}(p-1)$.

For $\gcd(m,n)=1$, CRT gives a bijection of residue classes. A residue is a unit modulo mn exactly when its components are units modulo m and n. Therefore $U(mn)\cong U(m)\times U(n)$, and counting gives $\phi(mn)=\phi(m)\phi(n)$. Factor a general modulus into pairwise-coprime prime powers and multiply. Example: $72=8\cdot 9$, so $\phi(72)=\phi(8)\phi(9)=4\cdot 6=24$; set $\phi(1)=1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Finite groups, Euler’ s theorem, and discrete logs

The invertible residues U(n) form a multiplicative group of size ϕ(n). Multiplication by a∈U(n) permutes U(n); multiplying all residues and cancelling invertible factors gives $a^{\varphi(n)} \equiv 1 \pmod{n}$. More precisely, the order $\mathrm{ord}_n(a)$ divides ϕ(n), since its generated subgroup has order dividing the group order.

If $\mathrm{ord}_n(g)=\phi(n)$, g is a primitive root and U(n)=⟨g⟩. For h∈⟨g⟩ a discrete logarithm is x satisfying $g^x=h$, unique modulo ord(g); outside the subgroup no solution exists. Not every modulus has a primitive root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Group and discrete-log example

In U(7)={1,2,3,4,5,6}, successive powers of 3 are 3,2,6,4,5,1. Thus $\mathrm{ord}_7(3)=6$, 3 is primitive, and $\log_3 5 \equiv 5 \pmod 6$. Powers of 2 are 2,4,1, generating {1,2,4}; hence $2^x \equiv 3 \pmod 7$ has no solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why a generator exists modulo a prime

Let G be the nonzero multiplicative group of $F_p$. Let L be the least common multiple of the orders of all elements of G. For each maximal prime-power factor $\ell^a$ of L, choose an element whose order contains that factor and take an appropriate power to obtain an element of order exactly $\ell^a$.

These prime-power orders are pairwise coprime. Since G is abelian, their product has order L: if its k-th power is 1, each factor's k-th power lies both in its own subgroup and in the subgroup generated by all other factors. These subgroup orders are coprime, so their intersection is the identity and each factor order divides k; conversely the L-th power is 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Finish using the polynomial root bound

Every $x \in G$ satisfies $x^L=1$. A nonzero polynomial $X^L-1$ over a field has at most L roots, so $p-1 \le L$. Lagrange's theorem says every element order divides $p-1$, hence their least common multiple L divides $p-1$ and $L \le p-1$. Thus $L=p-1$ and the element constructed above generates G.

The root bound depends on working over a field; it cannot be applied unchanged to an arbitrary composite-modulus residue ring.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Lift a generator from p to $p^2$

Let p be an odd prime and g a primitive root modulo p. Compare g and g+p. The binomial theorem gives $(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p \pmod{p^2}$. The second term is a nonzero multiple of p modulo $p^2$; therefore at least one choice h satisfies $h^{p-1} \not\equiv 1 \pmod{p^2}$. As $h^{p-1} \equiv 1 \pmod{p}$, write it as 1+cp with p∤c.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Lift to every odd prime power

For odd p and $v_p(z-1)=j \geq 1$, expand $z^p-1=(1+(z-1))^p-1$. The leading term has p-adic valuation j+1 and every other term has greater valuation. Hence $v_p(z^p-1)=j+1$. Induction gives $v_p(h^{(p-1)p^k}-1) = k+1$.

The order modulo $p^e$ is a multiple of p−1 and divides $\phi(p^e)=(p-1)p^{e-1}$. The valuation excludes every smaller power of p, so the order is $(p-1)p^{e-1}$ and $U(p^e)$ is cyclic. For p=3, g=2 gives $2^2=4 \not\equiv 1 \pmod 9$, so 2 generates $U(3^e)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Which moduli have primitive roots?

For $n>1$, $U(n)$ is cyclic exactly for $n=2$, $4$, $p^e$ or $2p^e$, where $p$ is an odd prime and $e\geq 1$. $U(2)$ and $U(4)$ can be listed directly; $U(p^e)$ was proved cyclic above. CRT gives $U(2p^e)\cong U(2)\times U(p^e)$, and the first component has one element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exclude the remaining moduli

If $8|n$, write $n=2^a m$ with $a\geq 3$ and $m$ odd. For any $b\in U(8)$, choose its odd integer representative as a unit modulo $2^a$. When $m>1$, apply CRT to the coprime moduli $2^a$ and $m$ to obtain $x\equiv b \pmod{2^a}$, $x\equiv 1 \pmod{m}$; when $m=1$, take $x=b$. Then $x$ is a unit modulo $n$ and $x\equiv b \pmod 8$, proving the natural projection $U(n)\to U(8)$ is surjective. Every element of the four-element group $U(8)=\{1,3,5,7\}$ squares to $1$, so it is not cyclic. A homomorphic image of a cyclic group is cyclic, a contradiction.

If $n$ has two distinct odd prime factors, CRT allows independent choices of $\pm 1$ and yields at least four unit solutions of $x^2=1$. The same holds if $4|n$ and $n$ has an odd prime factor, using $U(4)$ and the odd component. A finite cyclic group has at most two solutions to $x^2=1$. These exclusions cover every other $n$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A primitive root modulo 121

The order of 2 modulo 11 is 10. To lift it to $11^2=121$, check $2^{10}=1024\equiv 56\not\equiv 1 \pmod{121}$. Its order modulo 121 is a multiple of its order 10 modulo 11 and divides $\phi(121)=110$, so it is either 10 or 110. The first case is excluded, giving order 110: 2 generates U(121).

Only after the non-1 check modulo $p^2$ can the valuation argument above lift the same representative to higher odd prime powers. A generator modulo p does not automatically make every chosen integer representative a generator modulo $p^2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The only square roots of 1 modulo an odd prime power

Let p be an odd prime and $e\geq 1$. If $x^2\equiv 1 \pmod{p^e}$, then $p^e \mid (x-1)(x+1)$. As $\gcd(x-1,x+1)$ divides 2, p cannot divide both factors. Thus all e copies of p divide one side and $x\equiv 1$ or $-1 \pmod{p^e}$. Both residues are roots and they are distinct because p is odd.

For example modulo 25 the only roots are 1 and 24. A composite modulus can have more: 1,4,11,14 are all roots modulo 15, arising from independent sign choices in the CRT components.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A nontrivial square root exposes factors

Let odd $n>1$ and $x^2 \equiv 1 \pmod{n}$, with $x \not\equiv 1, -1 \pmod{n}$. Set $d_- = \gcd(x-1, n)$, $d_+ = \gcd(x+1, n)$. Both are less than $n$, otherwise $x \equiv \pm 1$. If $d_- = 1$, cancellation in $n \mid (x-1)(x+1)$ forces $n \mid (x+1)$, a contradiction; similarly $d_+ \neq 1$. Both are nontrivial factors.

For $n=77$ and $x=34$, $34^2 = 1156 \equiv 1 \pmod{77}$, while $\gcd(33,77)=11$ and $\gcd(35,77)=7$. In CRT coordinates, $34 \equiv 1 \pmod{11}$ and $34 \equiv -1 \pmod{7}$: the components chose different signs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Primes and composites

Trial division up to $\sqrt{n}$ decides $n \geq 2$ but can take many steps for long inputs. For example $91 = 7 \cdot 13$ is composite; For 97, test the primes 2, 3, 5, and 7 up to $\sqrt{97}$; none divides it, so 97 is prime.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Miller–Rabin preconditions

For $n<2$ return not prime; 2 and 3 are prime; even $n>2$ is composite. For odd $n\geq 5$ write $n-1=2^s d$ with $d$ odd, and choose $a\in\{2,\ldots,n-2\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Witness test

Compute $x=a^d \bmod n$. Pass if $x=1$ or $n-1$; otherwise square at most $s-1$ times and pass if $n-1$ appears. Otherwise $a$ witnesses compositeness. For $n=15$, $a=2$, $d=7$, $s=1$, $x=8$, so the test fails.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why primes always pass

For odd prime p, $x^2 \equiv 1 \pmod p$ implies $x \equiv \pm 1$ because p divides (x−1)(x+1). Together with Fermat’ s theorem $a^{p-1} \equiv 1$, this forces every legal base to pass.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Random error bound and scope

For a fixed odd composite $n \geq 5$, choose each base independently and uniformly from 2… n−2. The probability that all t rounds pass is at most $4^{-t}$. This is conditional on a fixed composite, not a posterior probability that an arbitrary input is composite.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Fermat pseudoprime: 341

341=11 · 31 is composite, yet $2^{10}=1024\equiv 1 \pmod{341}$, hence $2^{340}=(2^{10})^{34}\equiv 1$. A base-2 Fermat test accepts 341; that does not make it prime.

For Miller–Rabin, $340=2^2 \cdot 85$. We have $2^{85}=(2^{10})^8 \cdot 2^5\equiv 32$; the next square is $32^2=1024\equiv 1 \pmod{341}$. The initial value is neither 1 nor 340, and −1 did not appear before reaching 1, so the test rejects 341. Here 32 is a nontrivial square root of 1: gcd(32−1,341)=31 and gcd(32+1,341)=11.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Carmichael number: 561

561=3 · 11 · 17 is composite, and 560 is a multiple of 3−1=2, 11−1=10 and 17−1=16. For gcd(a,561)=1, Fermat’s theorem gives $a^{560}\equiv 1$ modulo each prime factor, so CRT gives $a^{560}\equiv 1 \pmod{561}$. Every base coprime to 561 passes its Fermat test; the claim does not apply to non-coprime bases.

For Miller–Rabin, $560=2^4 \cdot 35$. Base 2 yields $2^{35}\equiv 263$ followed by squares 166,67,1 modulo 561. The initial value is not 1 and none of the checked values is 560, so base 2 exposes compositeness. All coprime bases fool Fermat here; they do not all fool the strong test.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Strong pseudoprime: 2047

$2047=23\cdot 89$ is composite, and $2046=2\cdot 1023$, so $s=1$ and $d=1023$. Since $2^{11}=2048\equiv 1 \pmod{2047}$ and $1023=11\cdot 93$, $2^{1023}\equiv 1$. The base-2 Miller–Rabin round therefore accepts 2047; 2 is a strong liar for this composite.

The comparison concerns one round with a fixed base. The randomized Miller–Rabin error bound requires independently and uniformly sampling bases as specified; always using base 2 does not provide that probability guarantee.

### Base 2: results for three composite numbers

| Number | Fermat | Miller–Rabin | Evidence |
|---|---|---|---|
| $341 = 11\cdot 31$ | passes | rejects | $32 \to 1$ |
| $561 = 3\cdot 11\cdot 17$ | passes | rejects | $263 \to 166 \to 67 \to 1$ |
| $2047 = 23\cdot 89$ | passes | passes | $2^{1023} \equiv 1$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exact strong liars for 65

$65=5\cdot 13$ and $64=2^6\cdot 1$, so $s=6$ and $d=1$. Since $\phi(5)=2^2\cdot 1$ and $\phi(13)=2^2\cdot 3$, both components have $\gcd(1,u_i)=1$ and $t_i=2$. The count is $L=1+1+4=6$.

Initial value 1 gives $a=1$; initial value $-1$ gives $a=64$. The next layer requires $a^2\equiv -1 \pmod{65}$. Its roots modulo 5 are 2,3 and modulo 13 are 5,8; CRT gives 8,18,47,57. The full liar set is $\{1,8,18,47,57,64\}$; sampling only from $2,\ldots,63$ retains four liars, with acceptance probability $4/62=2/31$. Base 2 has the six checked values 2,4,16,61,16,61: its initial value is not 1 and none equals 64, so it is a witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Miller–Rabin: setting up the 1/4 bound

Let odd composite $n = \prod_i p_i^{e_i}$ and n−1=2ˢd with d odd. Write $\varphi(p_i^{e_i}) = 2^{t_i} u_i$ with $u_i$ odd, set A=∏gcd(d,$u_i$) and t=min(s,$t_1$,…,$t_k$). The multiplicative group modulo an odd prime power is cyclic. In a cyclic group of order $2^t u_i$, $z^d = 1$ has gcd(d,$u_i$) solutions.

Also, $z^{2^r d} = -1$ has $2^r$gcd(d,$u_i$) solutions for $0 \le r < t_i$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exact count of strong liars

A non-unit is necessarily a witness. CRT separates a unit into its prime-power coordinates. The strong test accepts exactly when all d-th powers are 1, or there is one common r∈[0,s) such that every $2^r$d-th power is −1. These cases are disjoint, giving $L = A\left(1 + \sum_{r=0}^{t-1} 2^{kr}\right)$, where k is the number of distinct prime factors. This is a count, not yet a probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Counting bound: at least three prime factors

For $k \geq 3$, $\phi(n) = \prod 2^{t_i} u_i \geq 2^{kt} A$. Also $\left(1 + \sum_{r=0}^{t-1} 2^{kr}\right) / 2^{kt} \leq 1/4$: at $t=1$ this is $2/2^k \leq 1/4$; at $t>1$ the geometric sum gives a smaller ratio. Thus $L \leq \phi(n)/4 \leq (n-1)/4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Counting bound: two prime factors

If the two $t_i$ differ, $\phi(n) \geq 2^{2t+1} A$ and $1 + \sum_{r=0}^{t-1} 4^r \leq 4^t/2$, so $L \leq \phi(n)/4$. If they are equal but some $\gcd(d, u_i) < u_i$ or some $e_i > 1$, the lost odd factor is at least 3. Hence $\phi(n) \geq 3 \cdot 2^{2t} A$ and $L \leq \phi(n)/4$.

The remaining case has distinct odd primes $p, q$ to the first power, equal $t$ and $u_1, u_2$ dividing $d$. Since $pq-1$ is divisible by $2^t$, $s \geq t$; hence $p-1$ and $q-1$ both divide $pq-1$. Reducing modulo $p-1$ and $q-1$ gives $p-1 \mid q-1$ and $q-1 \mid p-1$, forcing $p=q$, a contradiction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Counting bound: odd prime powers

For $n=p^e$ with $e\geq 2$, $p$ does not divide $d$. Thus $A=\gcd(d, \text{the odd part of } p^{e-1}(p-1))$ is at most the odd part of $p-1$. Since $t\leq v_2(p-1)$, $L=A\cdot 2^t\leq p-1\leq (p^e-1)/4$ for odd $p\geq 3$ and $e\geq 2$.

Combining the cases gives $L\leq (n-1)/4$. A uniformly random base in $1,\ldots,n-1$ therefore has error probability at most $1/4$. The same holds for $2,\ldots,n-2$ after removing the two guaranteed liars $1,n-1$. Independent rounds give $\leq 4^{-r}$; a prime is never incorrectly rejected.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fixed-composite error versus posterior probability

For each fixed odd composite, $k$ independent rounds with uniformly sampled legal bases all pass with probability at most $\varepsilon=4^{-k}$. To make $\varepsilon\leq 10^{-9}$, note $4^{14}=268435456<10^9\leq 4^{15}=1073741824$, so 15 is the smallest choice certified by this universal bound. This does not claim that every input needs 15 rounds to meet the target in actual error probability.

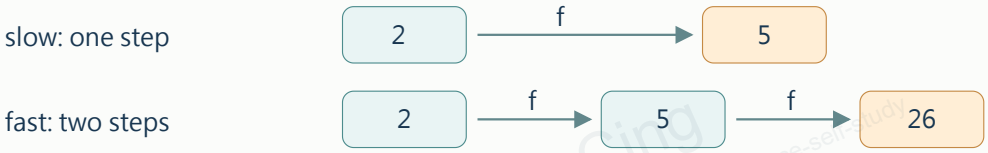
Assume the candidate distribution has prior prime probability $\pi=1/100$, primes always pass, and every composite passes with probability at most $\varepsilon$. Bayes' rule gives $P(\text{composite}|\text{pass})\leq (1-\pi)\varepsilon/[\pi+(1-\pi)\varepsilon]$. At $k=15$ this is $99/(1073741824+99)\approx 9.22\times 10^{-8}$. This posterior upper bound depends on the stated prior; it is not the fixed-composite false-positive probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pollard rho and Floyd collisions

Choose $f(x)=(x^2+1) \bmod n$. Each round update slow=f(slow), fast=f(f(fast)), then d=gcd(|slow−fast|,n). A value $1<d<n$ is a nontrivial factor. If d=n, this run failed to separate a factor; retry with a new seed or function. If d=1, continue. A discovered factor need not be prime, so factor recursively or test it.

**Floyd's two speeds: n=91, $f(x)=x^2+1 \bmod 91$**




gcd(|5−26|,91)=7 yields factor 7

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Pollard rho example and heuristic

Let n=91 and $x_0=2$. In the first round slow=f(2)=5, fast=f(f(2))=f(5)=26, and gcd(|5−26|,91)=gcd(21,91)=7. Thus 91=7 · 13. This fixed seed and function happen to succeed; other runs need not.

If the sequence modulo the unknown smallest prime factor p behaves like a random mapping, the birthday-collision heuristic suggests approximately $O(\sqrt{p})$ iterations. That model is not a worst-case guarantee and excludes the bit cost of modular multiplication and gcd. A bounded run can be restarted; do not present the heuristic as a guaranteed termination theorem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three Floyd rounds for 8051

Fix n=8051, $x_0$=2 and f(x)=$x^2$+1 mod 8051. Each round advances slow once and fast twice; compute d=gcd(|slow−fast|,8051).

| Round | slow | fast | Absolute difference | d |
|---|---|---|---|---|
| 1 | 5 | 26 | 21 | 1 |
| 2 | 26 | 7474 | 7448 | 1 |
| 3 | 677 | 871 | 194 | 97 |

In round 3, both 677 and 871 are 95 modulo 97; modulo 83 their residues are 13 and 41. Thus gcd(194,8051)=97 and 8051=97 · 83. Trial division through each square root confirms the two factors are prime.

### Third round compared modulo each factor

| slow = 677 | | fast = 871 |
|---|---|---|
| 677 mod 97 = 95 | = | 871 mod 97 = 95 |
| 677 mod 83 = 13 | ≠ | 871 mod 83 = 41 |

Equal modulo 97, unequal modulo 83; hence gcd = 97



## A different schedule: checkpoints

Define $x_{i+1}$=f($x_i$). For j=0,1,2,… save y=$x_{2^j}$, and advance x through $x_{2^j+1},\ldots,x_{2^{j+1}}$, computing gcd(|x−y|,n) at every step. Compare before updating the next checkpoint. Return when 1<d<n; if d=n, retry with a new function or seed. Exhausting the work budget only means that this run did not find a factor.

A trajectory in a finite function graph eventually enters a cycle of length λ. Once y is in that cycle, an interval of at least λ subsequent steps reaches the same component value as y. The doubling schedule uses a constant number of integer states. It is not the same trace as Floyd’ s slow/fast schedule.

## Checkpoint trace

Use n=8051 and $x_0$=2 again. The sequence is $x_1$=5, $x_2$=26, $x_3$=677, $x_4$=7474, $x_5$=2839, $x_6$=871, $x_7$=1848.

| checkpoint y | successive values of x | gcd results |
|---|---|---|
| $x_1$=5 | $x_2$=26 | 1 |
| $x_2$=26 | $x_3$=677, $x_4$=7474 | 1, 1 |
| $x_4$=7474 | $x_5$=2839, $x_6$=871, $x_7$=1848 | 1, 1, 97 |

At $x_7$ the gcd is 97, a proper factor of 8051, so stop. This comparison order differs from Floyd’s although the underlying sequence $x_i$ is the same.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Deriving the birthday-collision bound

Model k independent uniform draws from a set of size p. For k≤p, after j distinct draws the next avoids them with probability 1−j/p. Multiplication gives P(no collision)=$\prod_{j=0}^{k-1}(1-j/p)$. Using $1-u\le e^{-u}$, P(no collision)≤exp(−k(k−1)/(2p)); for k>p the probability is zero.

With $k\approx c\sqrt{p}$, the exponent is approximately $-c^2/2$, so $\sqrt{p}$ is the typical collision scale. This is exact for the independent-uniform model; a random mapping’s first-collision trajectory has the same product. A fixed polynomial $f(x)=x^2+c$ need not satisfy the model, so $\sqrt{p}$ is a heuristic rather than an unconditional worst-case or per-restart success guarantee. For balanced semiprimes $p\approx\sqrt{n}$ the heuristic is $n^{1/4}$ iterations; bit costs of modular operations and gcd are separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Independence and Floyd’ s exact meeting bound

Repeating a fixed base does not create independent events: 2047 passes base 2 on every repetition. The per-round bounds multiply only when the legal bases are sampled independently as specified.

A trajectory in a finite function graph has tail length $\mu \geq 0$ and cycle length $\lambda \geq 1$. After k Floyd rounds the positions are $x_k$ and $x_{2k}$. Choose k as the least multiple of λ at least $\max(\mu,1)$. Both positions lie on the cycle and their index difference k is divisible by λ, so they meet, with $k \leq \mu+\lambda$. This meeting bound is deterministic, but equality of positions need not produce a proper factor. An $O(\sqrt{p})$ estimate additionally assumes behavior resembling a random mapping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A run that reaches gcd=n

Let n=15, $x_0=0$ and $f(x)=x^2+2 \bmod 15$. Round 1 gives slow=2, fast=6 and $\gcd(|2-6|,15)=1$. Round 2 gives slow=6 and fast=f(f(6))=f(8)=6, hence $\gcd(0,15)=15$. The two states meet modulo all of n; returning n is not a successful nontrivial factor.

Retry with a new seed or function, or report that this run found no factor when its work budget is exhausted. Failure does not prove n prime and 15 is not a nontrivial factorization of 15.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## RSA key construction

Choose distinct primes $p=11$, $q=13$, $n=143$ and $\phi(n)=120$. Choose $e=7$ with $\gcd(7,120)=1$; $d=103$ since $7 \cdot 103=721 \equiv 1 \pmod{120}$. Publish $(n,e)$; keep $d,p,q$ secret.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## RSA encryption and decryption

Represent a message by $0 \le M < n$. For $M=9$, $C=9^7 \bmod 143=48$; decrypt $48^{103} \bmod 143=9$. These small numbers are only for teaching.

- $p=11$, $q=13$, $n=143$
- $e=7$, $d=103$
- $9 \to 48 \to 9$
- encrypt: $9^7 \bmod 143 = 48$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## RSA correctness

ed=1+kϕ(n). Modulo p, if p divides M both sides are 0; otherwise Fermat gives $M^{ed} \equiv M$. The same holds modulo q. CRT gives $M^{ed} \equiv M \pmod{n}$, even for non-coprime messages.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## RSA also recovers non-units

Choose distinct primes p=11, q=17; n=187, ϕ(n)=160, e=7 and d=23, since 7 · 23=161≡1 (mod 160). The message m=11 has gcd(11,187)=11 but is a valid residue modulo n.

Encryption: $11^2 \equiv 121$ and $11^4 \equiv 55$, so $11^7 \equiv 55 \cdot 121 \cdot 11 \equiv 88 \pmod{187}$. Decryption: $88^2 \equiv 77$, $88^4 \equiv 132$, $88^8 \equiv 33$, $88^{16} \equiv 154$. Since 23=16+4+2+1, 154 · 132≡132, 132 · 77≡66 and 66 · 88≡11. Hence $88^{23} \equiv 11 \pmod{187}$.

### Recover a non-unit message in each prime component

Both components recover the message; CRT gives 11 mod 187

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why RSA works for non-units

Let $ed \equiv 1 \pmod{(p-1)(q-1)}$. For each $r \in \{p,q\}$, if $r \mid m$ then $m^{ed} \equiv m \equiv 0 \pmod{r}$; otherwise Fermat's theorem and $r-1 \mid ed-1$ give $m^{ed} \equiv m \pmod{r}$. CRT then gives $m^{ed} \equiv m \pmod{pq}$.

In the example, modulo 11 the components are $11 \to 0$ and $88 \to 0$; modulo 17 they are $11 \to 11$ and $88 \to 3$, with $3^{23} \equiv 11$. CRT uniquely reconstructs 11 modulo 187. The proof assumes distinct primes p,q, positive e,d and the stated congruence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Malleability of textbook RSA

Unencoded $E(m)=m^e \bmod n$ is deterministic, and $E(m_1)E(m_2) \equiv E(m_1 m_2) \pmod{n}$. Given $c=E(m)$, anyone can choose $r \in U(n)$ and compute $c' = c \cdot r^e \bmod n$; it decrypts to $mr \bmod n$. This algebraic malleability does not require knowing m or the private key.

For $n=187$, $e=7$, $c=88$ and $r=2$, $2^7 \equiv 128$ and $c' \equiv 88 \cdot 128 \equiv 44$. Decrypting gives $44^{23} \equiv 22 = 2 \cdot 11$. This does not claim every real protocol reveals plaintext, but shows that the bare exponentiation is insufficient for the desired security; practical encryption requires standardized randomized encoding and an appropriate security model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A separate flaw in textbook RSA signatures

Suppose signing is $\sigma = m^d \bmod n$ and verification only checks $\sigma^e \equiv m \pmod n$. An attacker may first choose $\sigma$ and then set $m = \sigma^e \bmod n$, producing a verifying message/signature pair without the private key. The attacker chooses $\sigma$ and in general cannot also prescribe an arbitrary target message; the demonstrated claim is one valid unsigned pair under this message space and bare verification rule.

For $n = 55$ and $e = 3$ choose $\sigma = 12$. Then $m = 12^3 \bmod 55 = 23$, so $12^3 \equiv 23$ passes verification. This is a signature-verification flaw, distinct from the ciphertext malleability discussed earlier. Practical signatures require suitable hashing, encoding, and strict verification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Scope and practice

This chapter gives mathematical algorithms. Practical RSA requires standardized encoding and randomized padding, secure parameters, and side-channel resistant implementation. The primality bound relies on the stated sampling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 31. MIT 6.046J Fall 2004 Lecture 7, p. 33, and Fall 2005 Lecture 8, p. 13: inverses modulo a prime.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Linear programming: models and certificates

Describe decisions with linear objectives and constraints. Distinguish optimality, infeasibility, and an unbounded objective.

Start with equivalent formulations and models, then use simplex, duality, and Phase I to build checkable certificates.

## Learning route

| Question | Method |
| --- | --- |
| How do we preserve the original solutions? | Convert to standard and slack forms. |
| How do we move while staying feasible? | Bases, pivots, and the ratio test. |
| How do we certify the result? | Dual certificates, infeasibility certificates, or improving rays. |

Prerequisites include linear algebra, sets, and basic optimization concepts.



# Standard and slack forms

Our standard form is $\max\ c^{\top}x,\ Ax \le b,\ x \ge 0$, with $A \in R^{(m \times n)}$. Constraints are non-strict and variables continuous real numbers. Finite binary-input bit complexity assumes rational coefficients. Other texts call equality form standard; check conventions first.

A feasible point satisfies all constraints; an optimum maximizes the objective over feasible points. Infeasible means no feasible point. Unbounded means feasible with arbitrarily large objective, not merely an unbounded feasible region. For example, $\max(-x)$ over $x \ge 0$ has an unbounded region and optimum $0$.

Intersections of finitely many linear half-spaces are convex polyhedra:$Ax \le b$, $Ay \le b$ imply $A(\lambda x+(1-\lambda)y) \le b$ for $0 \le \lambda \le 1$. Convexity neither guarantees integer solutions nor makes nonlinear products linear constraints.

## Four conversions must recover original variables

Convert minimizing f to maximizing $-f$ and negate the returned objective. Multiply $a^T x \geq b$ by $-1$. Split equality into opposite inequalities. Replace unrestricted u by $u^+ - u^-$ with both nonnegative. Every u has representation$(\max(u,0), \max(-u,0))$; every such pair recovers an original value by subtraction. Although not one-to-one, the optimum values correspond.

For `min 2u-v; u+v=3; u-2v≥-3; v≥0; u free`, maximize $-2u^+ + 2u^- + v$ subject to

$u^+ - u^- + v \leq 3;\ -u^+ + u^- - v \leq -3;\ -u^+ + u^- + 2v \leq 3$.

Substituting $u = 3 - v$ gives $v \leq 2$ and original objective $6 - 3v$, so $v = 2$, $u = 1$ attains minimum 0, corresponding to maximum 0. Starting with n variables, m constraints, sign splitting uses at most $2n$ variables and equality splitting at most $2m$ constraints, adding only constant coefficient bit length.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Slack form, bases, and basic feasibility

Introduce $s = b - Ax \geq 0$. Let $M = [A\ I_m]$, $q = (x, s) \geq 0$, so $Mq = b$. Slacks are uniquely determined, not freely allocated independent decisions.

Choose m independent columns as basis B and let N be the others. Invertibility gives

$$q_B = M_B^{-1} b - M_B^{-1} M_N q_N, \qquad z = v + \rho_N^T q_N.$$

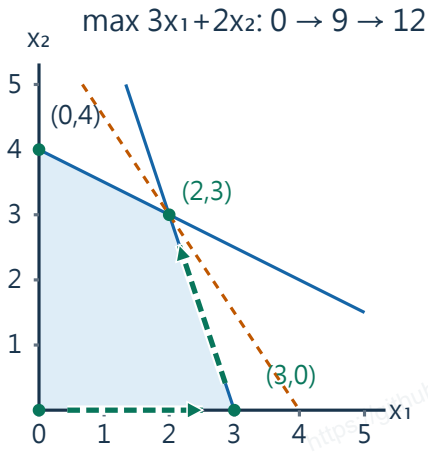
Set all nonbasic variables to 0 for a basic solution. It is feasible exactly when $M_B^{-1} b \geq 0$ componentwise. Initial slack basis I is feasible exactly when $b \geq 0$. Basic values need not be positive; zero basic variables cause degeneracy. A fixed basis determines a unique dictionary because the inverse is unique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Geometry, level lines, and basis on one example

Use $\max\ 3x_1+2x_2;\ x_1+2x_2\le 8;\ 3x_1+x_2\le 9;\ x\ge 0$. Feasible vertices are(0,0),(3,0),(2,3),(0,4). The vertex path improves objective 0→9→12, ending at(2,3).

Initially $x_3=8-x_1-2x_2;\ x_4=9-3x_1-x_2;\ z=3x_1+2x_2$, with basis{$x_3$, $x_4$} and slacks(8,9).




blue: resource constraints

$x_1+2x_2=8$

$3x_1+x_2=9$

dashed arrows: two simplex moves

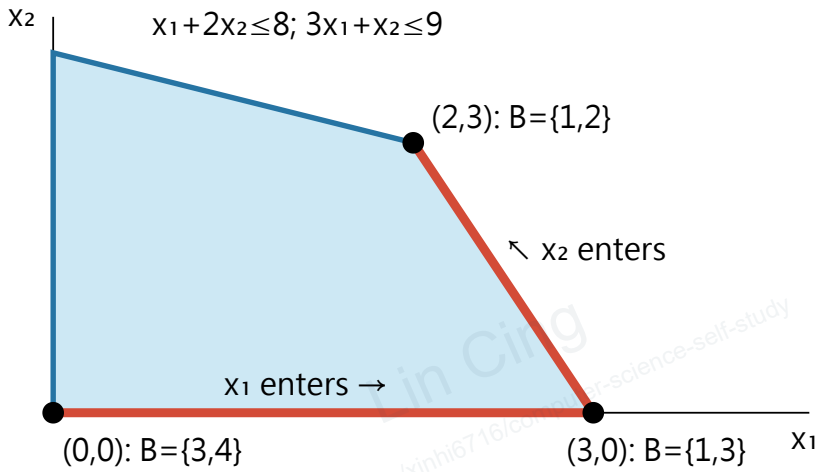
orange dashed: $3x_1+2x_2=12$

intersection (2,3), objective 12

Lin Cing https://github.com/xinhi6716/computer-science-self-study



# Geometry, level lines, and basis on one example (continued)




An entire edge can be optimal. For objective $x_1+2x_2$, the segment(0,4) to(2,3) has value 8. A vertex method returns one optimal basic solution without asserting uniqueness.

## Formulating linear programs

Specify each variable's meaning, unit, and domain. Map every model-feasible solution to an original plan and every original feasible plan back. Matching objectives while omitting capacity or conservation does not give equivalent models.

For shortest paths, use potentials $d_v$, fix $d_s$=0, and constrain $d_v - d_u \leq w_{uv}$ per edge. Maximize $d_t$. Potentials are unrestricted; negative paths can require $d_t$<0.

First assume all vertices are source-reachable, no negative cycle exists, and t is reachable. Summing inequalities along any s-to-t path gives $d_t \leq$ its length. Every feasible potential is therefore a lower bound on the shortest distance. Actual distances δ(s, v) satisfy all triangle inequalities and attain δ(s, t). Maximization, not minimization, finds the shortest distance.



## Potential boundaries are not program Infinity values

Restricting to the s-reachable subgraph gives finite true distances when no negative cycle exists. Report unreachable t separately;∞ is not a real LP variable. Keeping an unreachable subgraph with a negative cycle instead makes the whole potential LP infeasible even if the source-to-target distance is ordinary. That assumption cannot be omitted.

If the entire graph has no negative cycle, a zero-edge super-source gives globally feasible potentials. For t unreachable from s, increase every unreachable vertex's potential by arbitrary K>0. No reachable-to-unreachable edge exists, so all differences stay feasible. Then $d_t$ is unbounded, corresponding to unreachability, not a negative-length path.

For s→a:2, a→t:−1, s→t:4, constraints are $d_a \leq 2$, $d_t \leq d_a - 1$, $d_t \leq 4$. Values(0,2,1) are feasible and the two-edge path bounds $d_t \leq 1$, so optimum 1. Erroneously requiring all d≥0 would exclude correct solutions on other negative-total-path examples.

## Flow models use nonnegative variables per original arc

Use arc flow $g_{uv}$ rather than the preceding chapter's antisymmetric net flow. Constrain $0 \leq g_{uv} \leq c_{uv}$, internal outflow minus inflow 0, and maximize net source outflow. Antiparallel original arcs each retain their own variable and capacity. Do not also impose $g_{uv} = -g_{vu}$.

Every original flow satisfies the model, and capacity/conservation make every model solution a flow. There are m variables and O(n+m) constraints, without $n^2$ nonedge variables.

Minimum-cost flow additionally fixes source net outflow D and sink net inflow D, and minimizes $\sum_{(u,v)} a_{uv} g_{uv}$. Demand is exact; maximizing flow then reporting cost does not solve the same objective. Finite original capacities bound negative-cost circulation, which need not make the LP unbounded. Prohibiting circulation requires an additional model restriction.



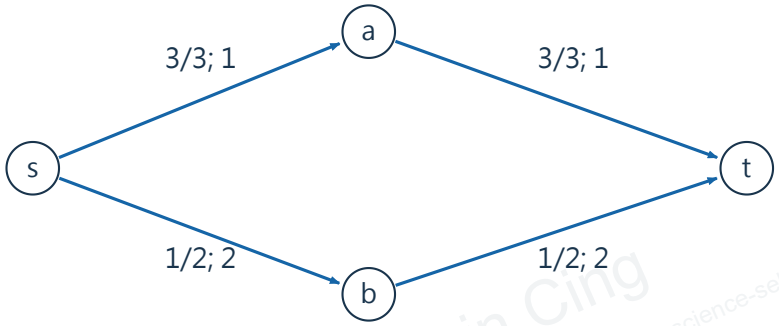
## Complete minimum-cost-flow example

Arcs s→a, a→t each have capacity 3, unit cost 1. Arcs s→b, b→t each have capacity 2, unit cost 2. Demand D=4. Conservation makes route flows u, v, with $u+v=4, 0 \leq u \leq 3, 0 \leq v \leq 2$, and objective min 2u+4v.

Substitute v=4−u to get cost 16−2u and feasible $2 \leq u \leq 3$. Thus u=3, v=1 costs 10. This specifies every arc flow, demand, feasible interval, and optimum.

## Complete minimum-cost-flow example (continued)

Demand D=4; edge label: flow / capacity; unit cost




u+v=4

2≤u≤3

cost = 16−2u

optimal: u=3, v=1

upper: 3×(1+1)=6; lower: 1×(2+2)=4; total 10

Using u+v≤4 would incorrectly make zero flow at cost 0 optimal. Omitting u≤3 would incorrectly select u=4 at cost 8.



## Multicommodity flow shares capacities

Commodity k has$(s_k, t_k, D_k)$ and per-arc $g_{uv}^k \geq 0$. Each commodity separately conserves flow and meets endpoint net supplies$\pm D_k$. Shared capacity is $\sum_k g_{uv}^k \leq c_{uv}$. A feasibility objective may be0; costs can also be included.

Two commodities each uniquely requiring a capacity 1 bottleneck u→v with demand 3/4 are individually feasible but jointly need 3/2>1. Independent max-flow solutions miss this shared resource.

With k commodities, m arcs, n vertices, there are km variables and O(kn+m) constraints, a polynomial-size LP for rational data. This models splittable flow. One-path-per-commodity or integral-flow requirements create different discrete problems, without the same general LP guarantees.

## Simplex dictionaries and the ratio test

Fix the sign convention below: $a_{ij}$ is the coefficient **being subtracted** in a dictionary row.

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j, \qquad z = v + \sum_{j \in N} c_j x_j.$$

From a feasible dictionary with $b_i \geq 0$, choose $c_e > 0$ to enter, set $x_e = \theta \geq 0$, and keep the other nonbasic variables zero. Each basic value becomes $x_i = b_i - a_{ie}\theta$.

If $a_{ie} > 0$, feasibility requires $\theta \leq b_i / a_{ie}$; if $a_{ie} \leq 0$, the row imposes no upper bound. Choose the **minimum nonnegative ratio, including zero**, among positive denominators.

If there is no positive denominator, set $q_e = 1$, $q_i = -a_{ie} \geq 0$, and the other $q_j = 0$. The ray $x(\theta) = x(0) + \theta q$ is feasible and its objective $v + c_e\theta \to \infty$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Pivot: solve for the entering variable

Let $e$ enter and $l$ leave, with $a_{le} \neq 0$. Solve row $l$ exactly:

$$x_e = \frac{b_l}{a_{le}} - \frac{x_l}{a_{le}} - \sum_{j \in N \setminus \{e\}} \frac{a_{lj}}{a_{le}} x_j.$$

Substitute this row into the other rows and the objective, then exchange the basic set $B$ and nonbasic set $N$. Variable $x_l$ becomes nonbasic but remains in the dictionary. Omitting $-x_l / a_{le}$ changes the feasible set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pivot: update the coefficients

For the new entering row,

$$a'_{el} = \frac{1}{a_{le}}, \quad b'_e = \frac{b_l}{a_{le}}, \quad a'_{ej} = \frac{a_{lj}}{a_{le}}.$$

For the remaining $i \neq l$ and $j \in N \setminus \{e\}$,

$$b'_i = b_i - a_{ie}b'_e, \quad a'_{il} = -\frac{a_{ie}}{a_{le}}, \quad a'_{ij} = a_{ij} - \frac{a_{ie}a_{lj}}{a_{le}}.$$

The objective row gives

$$v' = v + c_e b'_e, \quad c'_l = -\frac{c_e}{a_{le}}, \quad c'_j = c_j - \frac{c_e a_{lj}}{a_{le}}.$$

This invertible substitution preserves all points satisfying the equations and their objective values. Adding nonnegativity yields the same LP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why the ratio test preserves feasibility

The new entering value is $b'_e = b_l/a_{le} = \theta \geq 0$. The other basic values are

$$b'_i = b_i - a_{ie}\theta.$$

For $a_{ie} > 0$, the minimum ratio ensures $\theta \leq b_i/a_{ie}$, so $b'_i \geq 0$. For $a_{ie} \leq 0$, we have $b'_i \geq b_i \geq 0$. The new nonbasic variables are zero, so the new basic solution is feasible.

The objective changes by $c_e\theta \geq 0$. It strictly improves if $\theta > 0$; at $\theta = 0$, the point and objective may remain unchanged while the basis changes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Simplex control flow

```
while some reduced cost c[e] > 0:
    choose entering e
    candidates = {i in B : a[i,e] > 0}
    if candidates empty: return feasible point + improving ray
    l = minimum-ratio row, using an anti-cycling rule
    pivot(l,e)
return current basic solution
```

A zero ratio must remain eligible. If there is no leaving candidate, the improving ray certifies unboundedness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Two pivots on the same two-dimensional example

Initially B={3,4}. Enter $x_1$:ratios 8/1=8,9/3=3 make $x_4$ leave with $\theta=3$:

$x_1=3-x_2/3-x_4/3;\ x_3=5-5x_2/3+x_4/3;\ z=9+x_2-x_4.$

Enter $x_2$:ratios 3/(1/3)=9,5/(5/3)=3 make $x_3$ leave:

$x_2=3-3x_3/5+x_4/5;\ x_1=2+x_3/5-2x_4/5;\ z=12-3x_3/5-4x_4/5.$

Nonbasic $x_3=x_4=0$ gives(2,3), tight in both constraints. Every reduced cost is nonpositive, so every feasible point has objective$\le 12$. No further numerical search of the polygon is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Zero steps are legal pivots

Use $\max z=x_1;\ x_3=-x_1;\ x_4=1-x_1$, all variables nonnegative. Initially B={3,4}, N={1}, basic values(0,1).

| Basic row | Positive denominator | Ratio | At θ=0 | Incorrect θ=1 |
|---|---|---|---|---|
| $x_3$ | 1 | 0 | 0 | −1, infeasible |
| $x_4$ | 1 | 1 | 1 | 0 |

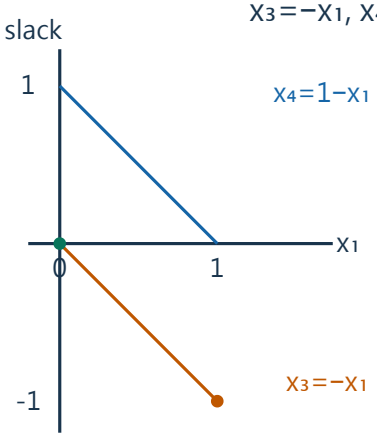
Let $x_3$ leave. New $x_1=-x_3$, $x_4=1+x_3$, $z=-x_3$ has the same point(0,0,1), objective 0, but now nonpositive reduced costs prove optimality. Ignoring zero ratios immediately violates feasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Zero steps are legal pivots (continued)

$x_3=-x_1$, $x_4=1-x_1$: do not skip a zero ratio




$x_1=0$: $x_3=0$, $x_4=1$, feasible

ratios: 0/1=0 and 1/1=1

feasible step θ=0; $x_3$ leaves

wrong step θ=1: $x_3=-1$, infeasible

only $x_1=0$ is feasible

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Finitely many bases alone do not prevent cycling

Matrix M=[A I] has m rows, and a basis selects m independent columns. There are at most C(n+m, m) possibilities. A fixed basis determines one dictionary, so an infinite run repeats a basis. Degenerate steps may preserve objective, so mere nondecrease does not forbid repetition.

Use exact lexicographic perturbation for a self-contained anti-cycling proof. From an initial feasible dictionary $q_B = \beta - Dq_N$, replace its RHS by $\beta_i+\varepsilon^i$ for i=1,…, m. Here ε is a positive infinitesimal,**not an arbitrary floating decimal**. Nonnegative constants and a positive first ε coefficient make the initial dictionary strictly feasible in the ordered field.

Represent comparisons by coefficient vectors:constant first, then ε,$\varepsilon^2$,…; the first nonzero coefficient determines sign. Continue minimum-ratio tests with positive denominators, using later coefficients to resolve equal real ratios.

Bland’ s rule fixes variable indices: choose the smallest index with positive reduced cost, and among minimum nonnegative-ratio ties choose the smallest-index leaving variable. The finite-termination theorem assumes exact arithmetic and both choices; tie-breaking alone is insufficient.



## Why the lexicographic rule terminates

Let the initial equation matrix be $M_0$=[I D], with RHS $\beta + \sum_i e_i\epsilon^i$. For any invertible basis K, its basic values are

$$M_{0,K}^{-1}\beta + \sum_{i=1}^{m}(M_{0,K}^{-1}e_i)\epsilon^i.$$

No inverse row is entirely zero, so no basic value is the identically-zero polynomial. Feasibility therefore makes every basic value strictly positive. Two positive-denominator ratios cannot coincide identically, or two inverse rows would be proportional, contradicting invertibility. Every legal θ is strictly positive, and $c_e$>0 makes the lexicographic objective strictly increase.

A fixed basis has one dictionary/objective, so it cannot repeat. Finite bases imply termination. Taking constant coefficients returns an original-LP feasible basic point. Reduced costs do not depend on RHS; nonpositive costs establish its optimality. An improving ray likewise has ε-independent direction and certifies original unboundedness from the constant feasible point. No guessed sufficiently-small floating ε is required.

## A reproducible lexicographic tie

Use $x_3=2-x_1;\ x_4=2-x_1-x_2;\ \max\ x_1+x_2$, with RHS $2+\varepsilon, 2+\varepsilon^2$. Entering $x_1$ compares $2+\varepsilon$ against $2+\varepsilon^2$; the latter is smaller, so $x_4$ leaves.

New rows are $x_1=2+\varepsilon^2-x_2-x_4;\ x_3=\varepsilon-\varepsilon^2+x_2+x_4;\ z=2+\varepsilon^2-x_4$. No reduced cost is positive. The constant solution is $x_1=2$, other variables 0, objective 2. Although $x_3$ has zero constant, $\varepsilon-\varepsilon^2$ is strictly positive and separates degeneracy symbolically.

Another common anti-cycling choice is Bland's rule:smallest-index positive reduced-cost entering variable, and smallest-index leaving variable among minimum-ratio ties. Both parts matter. The complete termination proof here uses lexicographic perturbation and does not claim arbitrary choices terminate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Per-pivot cost is not total simplex cost

A dense ordinary dictionary updates O(mn) coefficients per pivot. This does not prove polynomial total time; basis count C(n+m, m) can be exponential. Lexicographic RHS adds m+1 coefficients per row and a polynomial update/comparison factor, but retains only a finite/exponential general worst-case pivot guarantee.

Rational exact pivots use fractions. Basis solutions are ratios of subdeterminants. After clearing original denominators, a determinant has at most m! terms, giving bit length bounded polynomially by dimension and input bit length. A single basis thus needs no unrestricted representation growth, but pivot count need not be polynomial.

Floating implementations need separate feasibility/reduced-cost tolerances, near-zero ratio handling, and residual recomputation. Clamping negative b to 0 can change the problem. Ellipsoid and interior-point methods are different algorithms; their polynomial theory does not follow from these pivot counts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Duality

Primal `max cᵀx, Ax≤b, x≥0` has dual `min bᵀy, Aᵀy≥c, y≥0`. Its m resource rows give m prices and n primal variables give n price inequalities. Transpose A; merely exchanging max/min is insufficient.

For feasible x, y, nonnegativity and both constraint systems give

$$c^T x \leq x^T A^T y = y^T A x \leq y^T b.$$

This is weak duality. A dual feasible point bounds every primal objective above. Equality certifies optimality on both sides. Equal objective numbers without checking feasibility are not a certificate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Constructing a dual from the terminal basis

Let M=[A I], d=(c,0), and terminal basis matrix $M_B$ be invertible. Define row $y^T = d_B^T M_B^{-1}$. Substituting basic variables gives

$$z = y^T b + \sum_{j \in N} (d_j - y^T M_j) q_j.$$

For every column, reduced cost $\rho_j = d_j - y^T M_j$, and basic costs are 0. Terminal $\rho \leq 0$ gives $-y_i \leq 0$ from slack columns $e_i$, hence $y \geq 0$. Decision columns give $c_j - y^T A_j \leq 0$, hence $A^T y \geq c$.

The current feasible basic objective is $y^T b$, attaining weak-duality equality. A finite optimal primal therefore has a same-value dual. Read $y_i$ as the negative reduced cost of original slack i, or0 if that slack is basic. This follows from the full dictionary identity, not coefficient matching at only one point.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The same example's dual and complementary slackness

The two-dimensional example has dual `min 8y₁+9y₂; y₁+3y₂≥3; 2y₁+y₂≥2; y≥0`. Terminal `z=12−3x₃/5−4x₄/5` gives y=(3/5,4/5). Both dual rows are tight, and 24/5+36/5=12 matches x=(2,3).

The general gap decomposes exactly as

$$b^T y - c^T x = y^T(b - Ax) + x^T(A^T y - c).$$

For feasible sides, every summand is nonnegative. Gap 0 is therefore equivalent to all products $y_i(b_i - A_i x) = 0$ and $x_j((A^T y)_j - c_j) = 0$. This proves necessity and sufficiency of complementary slackness. Positive primal slack forces its price 0; positive primal variable forces its dual row tight.

For `max 2x₁+x₂; x₁+x₂≤4; x₁≤2; x≥0`, dual y=(1,1) gives upper bound 6. Point(2,2) has value 6 and all complementary products 0. Point(1,3) has value 5, with the sole positive product $y_2(2-x_1)=1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The same example's dual and complementary slackness (continued)

$$\text{gap} = y^T(b - Ax) + x^T(A^T y - c) \geq 0$$

| $x=(2,2)$ | $x'=(1,3)$ |
|---|---|
| $y=(1,1)$, dual value 6 | $y=(1,1)$, dual value 6 |
| primal 6 | primal 5 |
| $y_2(2-x_1) = 0$ | $y_2(2-x_1) = 1$ |
| other three products are 0 | other three products are 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Unboundedness and infeasibility are not interchangeable

An unbounded primal has no feasible dual, or one fixed dual value would bound all primal values. A downward-unbounded dual similarly makes primal infeasible. The converse need not hold:an infeasible primal need not have an unbounded dual, because both can be infeasible.

Primal `max -x; 0x≤-1; x≥0` is infeasible; dual `min -y; 0y≥-1; y≥0` is feasible and downward-unbounded. Changing primal objective to max x gives dual constraint $0 \geq 1$, making both sides infeasible.

For other sign conventions, convert to standard first. A primal equality has a free dual multiplier; an unrestricted primal variable gives a dual equality. These rules follow by combining split nonnegative variables or opposite inequalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The initial basic feasible solution

Negative entries in initial b prevent starting a ratio loop that assumes feasibility. For $Ax \leq b$, $x \geq 0$, add $a \geq 0$ and solve auxiliary `max -a; Ax-a1≤b`. It is always feasible:take x=0, a=max(0,$-\min_i b_i$). Its objective is at most 0 and cannot be upward-unbounded.

Original feasibility allows the same x with a=0, attaining auxiliary optimum 0. Conversely, auxiliary optimum 0 gives a=0 and an original feasible x. A strictly negative optimum proves infeasibility. A currently positive artificial variable alone is insufficient; first establish auxiliary optimality.

When m=0, use the origin directly. A positive $c_j$ gives an unbounded coordinate ray; otherwise the origin is optimal. No minimum over an empty row set is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A special pivot initializes auxiliary feasibility

Write $s_i$=$b_i$−$A_i x$+a and choose r with minimum $b_r$<0. Let a enter, $s_r$ leave. The pivot coefficient is−1; this is **not** the improving ratio step from an already-feasible basis.

Solving gives $a = -b_r + A_r x + s_r$; other rows become $s_i = b_i - b_r + (A_r - A_i)x + s_r$. Set new nonbasic x=$s_r$=0. Then a=−$b_r$>0 and $s_i$=$b_i$−$b_r$≥0, establishing a feasible basis. Only now use simplex and lexicographic anti-cycling.

Start symbolic RHS perturbation at this feasible dictionary and take constant parts afterward. Reduced costs ignore RHS, so the result is the original auxiliary optimum and feasible point, rather than a false infeasibility judgment from an ε objective.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Zero artificial value still requires basis cleanup

At auxiliary optimum 0, delete a's column if it is nonbasic. If a is basic with value 0, its row is $a = -\sum_j a_{aj} x_j$. A nonzero coefficient permits that $x_j$ to enter and a to leave. This zero-step algebraic pivot leaves the point unchanged; its coefficient may have either sign and it is not an improving ratio step.

If every coefficient is 0, the equation a=0 is redundant. Delete its row and a. General equality or multi-artificial implementations need this branch. Full independent slack rows can sometimes exclude it by rank, but never justify division by 0.

Finally substitute every currently basic variable appearing in the **original objective** to recover a nonbasic-only objective. Changing a title or retaining auxiliary−a does not restore the original problem. This completes Phase I; Phase II begins with that feasible basis and restored objective.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Complete Phase I and original-objective example

Original $\min\ z=2x_1+x_2;\ x_1+x_2\geq 5;\ x\geq 0$ uses surplus $x_3\geq 0$ with $x_1+x_2-x_3=5$. Add artificial $a=5-x_1-x_2+x_3$, minimizing $w=a$ in Phase I, equivalent to maximizing $-a$.

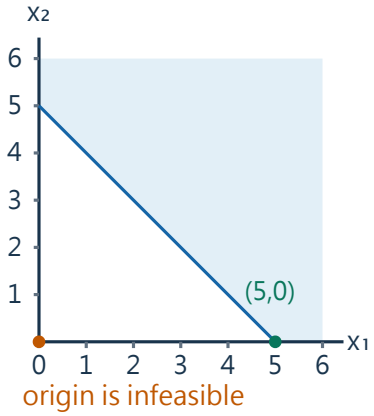
| State | Basic row | At nonbasics 0 | Objective |
|---|---|---|---|
| Initial | $a=5-x_1-x_2+x_3$ | $a=5$ | $w=5$ |
| $x_1$ enters, a leaves | $x_1=5-x_2+x_3-a$ | $x_1=5$ | $w=a$, min 0 |
| Delete a; substitute original objective | $x_1=5-x_2+x_3$ | $x_1=5$ | $z=10-x_2+2x_3$ |
| $x_2$ enters, $x_1$ leaves | $x_2=5-x_1+x_3$ | $x_2=5$ | $z=5+x_1+x_3$ |

Nonnegative final minimization coefficients prove optimum 5. Original origin(0,0) is infeasible; adding $a=5$ makes it feasible only for the auxiliary problem. This one-equation example uses one artificial column; the shared-$a$ construction above covers arbitrary $Ax\leq b$ systems.

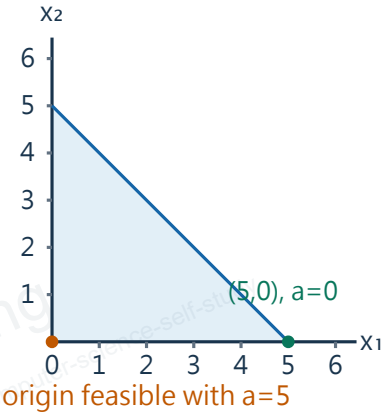


# Complete Phase I and original-objective example (continued)

Original: $x_1+x_2\geq 5$




Auxiliary slice $x_3=0$: $a=5-x_1-x_2$




The auxiliary problem returns to the original constraint only when $a=0$

## Infeasibility needs a completed certificate

Original $x\le 1$, $x\ge 2$, $x\ge 0$ becomes $x-a\le 1$, $-x-a\le -2$. Summing gives $-2a\le -1$, so $a\ge 1/2$. Values $x=3/2$, $a=1/2$ satisfy both tightly, proving auxiliary max $-a=-1/2<0$ and original infeasibility.

The complete dictionary starts $s_1=1-x+a$, $s_2=-2+x+a$. Special a-entry, $s_2$-exit gives $a=2-x+s_2$, $s_1=3-2x+s_2$, $w=-2+x-s_2$. Enter x:ratios 2,3/2 make $s_1$ leave, producing

$x=3/2-s_1/2+s_2/2;\ a=1/2+s_1/2+s_2/2;\ w=-1/2-s_1/2-s_2/2.$

Both reduced costs are nonpositive, so this is truly optimal. A positive artificial amount alone would not suffice. The original inequalities also sum to $0\le -1$, an independent contradiction certificate. More generally, $y\ge 0$, $A^Ty\ge 0$ and $b^Ty<0$ certify infeasibility: any feasible x would imply $0\le x^TA^Ty=y^TAx\le y^Tb<0$. Here $y=(1/2,1/2)$, $A^Ty=0$ and $b^Ty=-1/2$.



## Three outcomes and the proof dependencies

Phase I starts from a feasible auxiliary basis. Lexicographic simplex terminates finitely, and auxiliary objective is bounded above. A negative optimum means infeasible; a zero optimum permits artificial cleanup and objective restoration.

Phase II also starts feasible and terminates under the lexicographic rule. No positive ratio denominator produces a feasible point plus improving ray, certifying unboundedness. Otherwise nonpositive terminal reduced costs produce a feasible basic point and same-value dual, certifying a finite attained optimum.

Thus every LP in this standard form is exactly one of infeasible, feasible with unbounded objective, or feasible with a finite attained optimum. Finite-dimensional structure and non-strict constraints matter. Problem $\max\ x;\ 0\le x<1$ has unattained supremum 1 and lies outside this trichotomy's assumptions.

## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, Chapter 29. MIT 6.046J Fall 2004, Lecture 15; Fall 2005, Lecture 18.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Multithreaded algorithms

Dynamic multithreading uses a dependency DAG to separate total work, critical path and scheduling. Starting from spawn, sync and race avoidance, this chapter derives matrix multiplication and parallel merge/sort, with correctness proofs and space assumptions.

Prerequisites are DAGs, recurrence analysis and parallel cost models. We also distinguish a parallelism bound from the actual benefit of another processor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# The basics of dynamic multithreading

Programs specify computations allowed to run concurrently; the scheduler decides when and on which processor they run. A logical task is not a permanently bound OS thread, and spawning does not promise an immediate second core.

Assume P identical ideal processors and sequentially consistent shared memory. Instructions interleave in an order respecting dependencies; each load/store is one step, but a combined read-add-write is not automatically atomic. Arithmetic, indexing, and spawn/sync have constant unit work. Scheduler overhead, bandwidth, caches, and communication are outside this ideal bound.

A nonempty finite computation is a DAG. Nodes can be strands, sequential instruction sequences without parallel control, with positive execution time. Remove empty strands; split positive integral strands into unit-instruction chains for unit-node proofs. Edges express control and data dependencies. Nodes merely drawn at the same level need not be independent.

## Exact semantics of spawn, call, and sync

`x = spawn F(...)` allows F to run with the caller's continuation. An ordinary `y = G(...)` requires the caller to await G's return. A `sync` waits for all children previously spawned by this procedure; it is not a system-wide barrier.

Spawn arguments are evaluated in the parent first. Each child owns its call-local variables, but shared-array references can still lead several tasks to the same cell. Different variable names do not prove independence. Reading a spawned result requires a dependency, usually an explicit sync.

Procedure return has an implicit sync here. However, if `return x+y` evaluates its expression before returning, that implicit wait does not permit an earlier read of x. Explicitly sync before computing the return expression. Removing spawn/sync and replacing parallel for by ordinary for gives a serialization. Correct serialization alone does not prove every parallel interleaving correct.



## Fibonacci code and correctness

```
P-FIB(n):                // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

Children read their own n and return separate results, with no shared writes. Decreasing n ensures termination. Bases F0=0, F1=1 and induction, followed by synchronized addition, prove Fn. Spawning both children still requires waiting before addition and changes constants rather than work/span orders.

This version deliberately omits memoization and recomputes parameters. High parallelism does not imply work efficiency: sequential iteration uses O(n) arithmetic operations, whereas occupying more cores does not excuse exponential work.

## Work and span in one explicit strand model

Split each nonbase call into three unit strands:pre through spawning, cont before the second ordinary call, and join for addition/return after sync. A base call is one unit strand. Pre precedes the left child and cont; cont precedes the right child; both child finishes precede join.

With total work W and longest dependency-path cost D,

$$W_n = W_{n-1} + W_{n-2} + 3, \qquad W_0 = W_1 = 1;$$
$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \qquad D_0 = D_1 = 1.$$

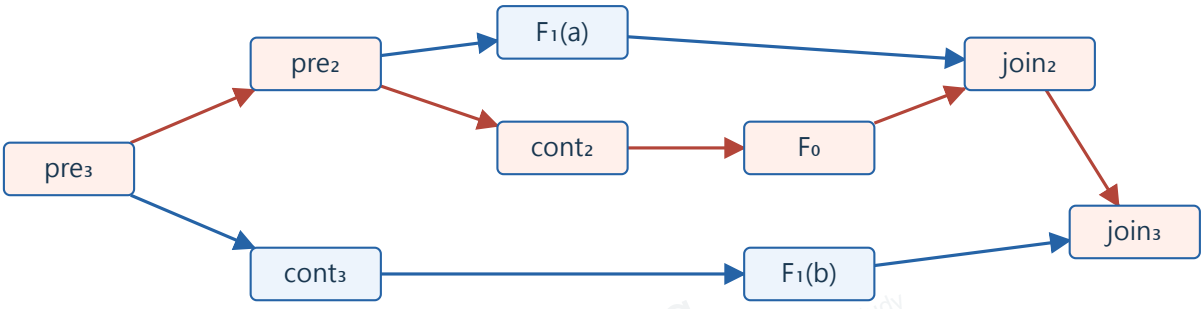
| n | Fn | Wn | Dn |
|---|---|---|---|
| 2 | 1 | 5 | 4 |
| 3 | 2 | 9 | 6 |
| 4 | 3 | 17 | 8 |

Induction verifies Wn=4F(n+1)−3 and Dn=2n for n≥2. Let $\varphi = (1+\sqrt{5})/2$. Since ϕ²=ϕ+1, induction gives ϕ^(n−2)≤Fn≤ϕ^(n−1) for n≥2:check2,3, and add adjacent bounds to obtain the next power. Thus W=Θ(ϕ^n), D=Θ(n). A complete recursive call is not one O(1) node in this model.



## Trace all dependencies among nine strands

All nine strands of P-FIB(3): arrows express dependencies




Each box is one unit strand: W=9. The red six-box critical path gives D=6.

Pre3 forks to pre2 and cont3; cont3 calls the right F1. Pre2 forks to its F1 and cont2; cont2 calls F0. Both bases precede join2, and join2 plus the other F1 precede join3.

The red path pre3→pre2→cont2→F0→join2→join3 has 6 nodes, so W=9, D=6. Path pre3→cont3→F1→join3 has only 4. Arrows express dependencies, not a single sequential chain through every node of a call.

## Work law, span law, and achievable speedup

Write $W = T_1$,$D = T_\infty$, and actual P-processor time $T_P$. At most P units of work finish per time unit, and a critical path cannot parallelize, so

$$T_P \geq \max\{W/P, D\}.$$

The discrete unit-node model also gives $T_P \geq \lceil W/P \rceil$. Speedup $S_P = W/T_P \leq \min(P, W/D)$, and efficiency S_P/P≤1. Average parallelism A=W/D limits speedup. P>A precludes perfect P-fold acceleration, but does not imply processor P+1 has no benefit.

Sequential composition adds both W and D. Independent fork/join branches add work and take the maximum branch span, plus explicit fork/join costs. An empty computation has W=D=0 and time 0;0/0 is not defined parallelism.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Complete greedy-scheduling upper-bound proof

At each step execute P ready nodes if possible, otherwise execute all ready nodes. Let C be complete steps and I incomplete steps, T=C+I. Every incomplete step before completion executes at least one node, so W≥PC+I.

An incomplete step removes all sources of the remaining DAG. Any longest path remaining afterward can be extended backward in the previous DAG to at least one removed source. If nothing remains, the earlier span was at least 1. Thus each incomplete step decreases remaining span by at least 1, and I≤D. Consequently

$$T \leq (W - I)/P + I \leq (W - D)/P + D \leq W/P + D.$$

The substitution uses coefficient1−1/P≥0. Positive integral strand durations can be split into chains for this proof. It does not give the same guarantee in a different model with arbitrary indivisible tasks of unknown lengths.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Two-approximation and quantified slackness

An optimal scheduler also obeys $T_P^* \geq \max(W/P, D)$, so greedy T≤W/P+D≤2T*. The sharper bound is(2−1/P)T*. Constant-factor performance requires no future-DAG knowledge.

Define slackness $\sigma = W/(PD) = A/P$. For D>0, the bounds give

$$1 \leq T_P/(W/P) \leq 1 + (1 - 1/P)/\sigma \leq 1 + 1/\sigma.$$

Large σ brings ideal greedy time near W/P. This is a mathematical guarantee, not measured bandwidth or scheduler overhead. An optimal schedule can ignore excess processors, so its time cannot increase with P. Measured implementations also depend on overhead and scheduling choices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Parallelism 30 still benefits from processor 31

Complete 697 independent unit tasks before starting a 23-unit dependency chain. Then W=697+23=720, D=1+23=24, A=30.

| P | Independent-stage allocation | Stage time | Chain time | Exact optimum |
|---|---|---|---|---|
| 30 | 23 rounds of 30, then7 | 24 | 23 | 47 |
| 31 | 22 rounds of 31, then 15 | 23 | 23 | 46 |

The chain cannot start until every predecessor finishes. Thus ceil(697/P)+23 is both an attainable schedule and a lower bound. Speedups 720/47≈15.32 and 720/46≈15.65 both stay below 30, yet the added processor helps. Only P≥697 makes the first stage one round and attains span 24.

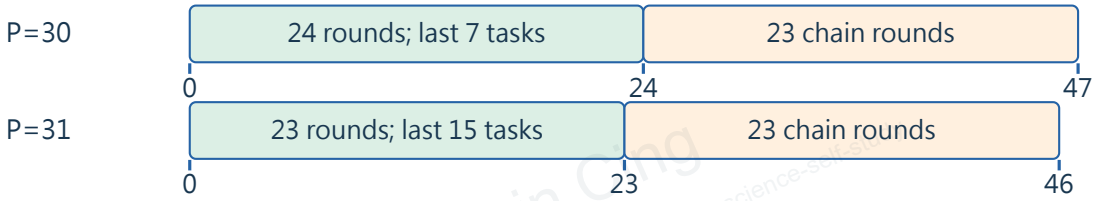
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Parallelism 30 still benefits from processor 31 (continued)

Same DAG: processor 31 still improves on 30

W=697+23=720
D=1+23=24

Average parallelism W/D=30. Both rows use the same time scale.




$T_{30}=47$, $T_{31}=46$. Speedup≤30 does not preclude benefit from processor 31.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Less work but greater span can reverse the winner

Two correct versions have(W, D)=(1024,4) and(768,16), both with ideal greedy scheduling.

At P=8, the first takes at least 1024/8=128; the second takes at most 768/8+16=112, so the second is definitely faster. At P=128, the first takes at most 1024/128+4=12; the second takes at least 16, so the first is faster.

Disjoint upper/lower bounds justify these conclusions. W/P+D is an upper bound, not exact time. Overlapping feasible-time intervals may leave the ordering unresolved from W, D alone. A single small-core measurement also cannot identify separate work and span changes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Implementing parallel for by balanced fork/join

Use zero-based half-open intervals[l, r), with l=r empty, here and in merge algorithms.

```
PAR-FOR(l, r, body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l, m, body)
    PAR-FOR(m, r, body)
    sync
```

The n nonempty iterations give n leaves and n−1 internal nodes. Constant control work per internal node adds O(n), not n log n. The longest call/join chain has depth Θ(log n), so span is Θ(log n) plus the largest iteration span, provided iterations are independent.

An ordinary loop spawning n tasks serially has a Θ(n) issuing chain in its parent and cannot claim the same logarithmic span. The final sync also makes all results ready before the next phase reads them.



## Matrix-vector multiplication safely parallelizes rows

```
MAT-VEC(A, x):
    n=A.rows
    allocate y[n]
    parallel for i=0 to n-1:
        y[i]=0
        for j=0 to n-1: y[i]+=A[i, j]*x[j]
    return y
```

Iteration i exclusively owns y[i], with A, x read-only. After inner step j, the entry is the sum of the first j+1 products, proving correctness. Work is $\Theta(n^2)$, span Θ(log n)+Θ(n)=Θ(n), parallelism Θ(n). For n=0 return an empty vector.

Making the inner loop parallel as well causes unsynchronized read-modify-writes to one y[i], a determinacy race. Two logically parallel accesses conflict when they touch one cell and at least one writes. Even equal written values or one successful run does not prove independence.

## Sequential consistency still permits lost updates

Lost updates: loads, local registers, and final stores

Initially x=0. Each task preserves load → add → store.

| Step | Task 1 | Task 2 | Shared x |
|---|---|---|---|
| 1 | load $r_1 \leftarrow x = 0$ | | 0 |
| 2 | | load $r_2 \leftarrow x = 0$ | 0 |
| 3 | add $r_1 \leftarrow r_1 + 1 = 1$ | | 0 |
| 4 | store $x \leftarrow r_1$ | | 1 |
| 5 | | add $r_2 \leftarrow r_2 + 1 = 1$ | 1 |
| 6 | | store $x \leftarrow r_2$ | 1 |

After sync both read 1. Waiting cannot make three instructions one atomic operation.

The interleaving respects each task's load→add→store but ends at x=1. Running one task completely before the other gives 2. Sequential consistency orders individual instructions; it does not make the three-instruction compound operation atomic.

Use disjoint ownership or reduction to remove the race. Atomic additions or locks can impose dependencies, but their serialization, work, and span need separate accounting. They do not retain an analysis assuming all additions complete simultaneously.



## A reduction parallelizes the dot product correctly

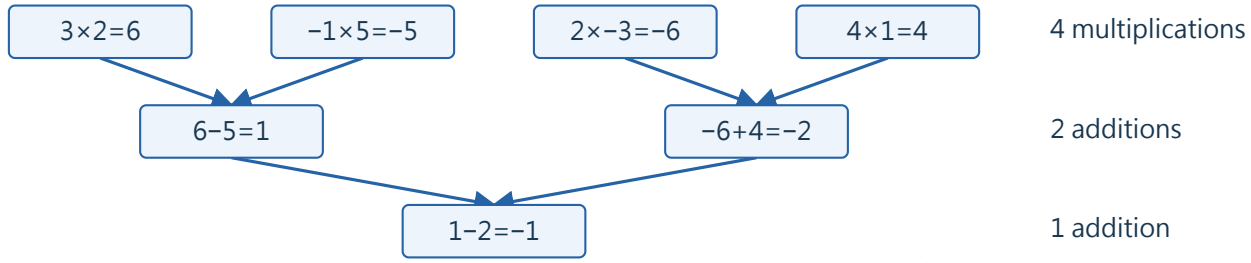
```
DOT(Arow, x, l, r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m=floor((l+r)/2)
    left=spawn DOT(Arow, x, l, m)
    right=DOT(Arow, x, m, r)
    sync
    return left+right
```

Children return separate values rather than sharing an accumulator. Associativity and induction give the interval dot product. Work is $\Theta(n)$, span $\Theta(\log(n+1))$. Replacing each MAT-VEC row by DOT gives $\Theta(n^2)$ work and $\Theta(\log n)$ span for $n \geq 2$.

For row=[3,−1,2,4], x=[2,5,−3,1], products are[6,−5,−6,4], first-level sums[1,−2], and result−1.

## A reduction parallelizes the dot product correctly (continued)

Dot product: four private products combine in a tree




Arithmetic DAG: work=7, span=3. Fork/join adds Θ(log n) control depth.

Floating-point tree addition may round differently from sequential accumulation. The proof assumes exact associative arithmetic and does not promise bitwise-identical floating results.



## Granularity and the allocation model

Group n constant-cost iterations into batches of at most g,$1 \le g \le n$, and issue batches with balanced PAR-FOR. Work is Θ(n), span $\Theta(\log\lceil n/g \rceil + g)$. Larger g reduces task overhead constants but increases leaf serial time; the tradeoff depends on P and slackness.

Here allocate obtains a descriptor for uninitialized, nonoverlapping storage at constant abstract span. If actual allocation initializes each cell, count that work and parallelize initialization to retain logarithmic span. Serial zeroing of $n^2$ cells is not hidden inside O(1).

Matrix blocks use views(row origin, column origin, stride, order), without hidden full-block copies. Task descriptors, allocators, caches, and memory bandwidth have implementation costs beyond the stated ideal model.

## Multithreaded matrix multiplication

```
P-MULTIPLY(A, B):
    n=A.rows                // both n by n
    allocate C[n, n]
    parallel for i=0 to n-1:
        parallel for j=0 to n-1:
            C[i, j]=0
            for k=0 to n-1: C[i, j]+=A[i, k]*B[k, j]
    return C
```

Each(i, j) owns one output cell and accumulates its inner loop sequentially. Work is $\Theta(n^3)$. The two issuing trees contribute $\Theta(\log n)+\Theta(\log n)$, then the inner loop contributes $\Theta(n)$, so span $\Theta(n)$, parallelism $\Theta(n^2)$. Two parallel keywords do not remove each cell's n-term dependency.

Using a DOT-style reduction for each Cij preserves $\Theta(n^3)$ work but reduces span to $\Theta(\log n)$, giving $\Theta(n^3/\log n)$ parallelism. Concurrent writes to Cij are not this reduction algorithm.

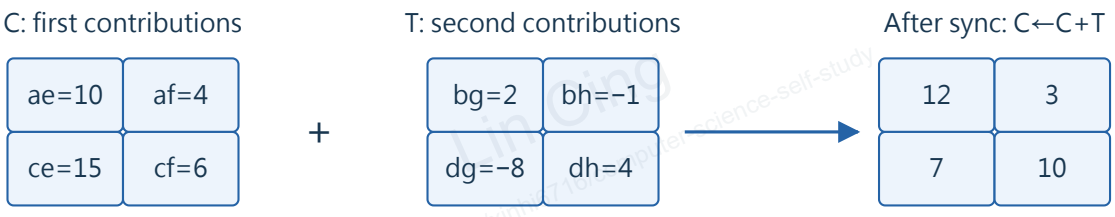


## Eight subproducts need eight distinct destinations

Write A=[[a, b],[c, d]], B=[[e, f],[g, h]], where letters are equal-order blocks. Compute C=[[ae, af],[ce, cf]] and temporary T=[[bg, bh],[dg, dh]]. Every one of eight products has its own destination. Sync before C=C+T.

For A=[[2,−1],[3,4]], B=[[5,2],[−2,1]], temporary C=[[10,4],[15,6]], T=[[2,−1],[−8,4]], giving[[12,3],[7,10]].

Eight independent products: separate destinations before addition




Each product writes one C or T cell. Await all products, then add cellwise.
A=[[2,−1],[3,4]], B=[[5,2],[−2,1]]. No simultaneous += contributions to one cell.

Two tasks executing C11+=ae and C11+=bg conflict. The temporary storage removes a shared-write dependency; spawning alone cannot do that.

## Complete recursive matrix-product procedure

```
P-REC(C, A, B):
    n=A.rows
    if n == 1: C[0,0]=A[0,0]*B[0,0]; return
    allocate T[n, n]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    view C, T as four n/2 by n/2 blocks
    spawn P-REC(C11, a, e); spawn P-REC(C12, a, f)
    spawn P-REC(C21, c, e); spawn P-REC(C22, c, f)
    spawn P-REC(T11, b, g); spawn P-REC(T12, b, h)
    spawn P-REC(T21, d, g); P-REC(T22, d, h)
    sync
    parallel for i=0 to n-1:
        parallel for j=0 to n-1: C[i, j]+=T[i, j]
```

Inputs are equal-order square matrices of power-of-two order, A, B read-only, and C disjoint from them. Pad any other positive order with zeros to the next power of two and extract the original result. Return an empty matrix for n=0. T and child workspaces are independent. Eight spawn/call operations are a fixed constant, not an n-long issuing chain.



## Inductive correctness and eight-way costs

The base writes the correct scalar product. Inductively, eight half-size products are correct and race-free through distinct destinations. Sync makes them available before cellwise addition. Block multiplication identities then yield AB. Shared reads of A, B are harmless.

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

Span follows only the longest child, not the sum of eight. Each level's nested parallel combination has Θ(log n) issuing/join depth, not O(1). For n=2^L, the sum1+…+L=Θ($L^2$) gives both bounds. Order4 has $8^2$=64 scalar-product leaves; parallelization has not removed multiplications.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Three dependency stages for parallel Strassen

Here $M_i$ denotes the same product as $P_i$ in Chapter 13. First prepare block sums/differences(a+d),(e+h),(c+d),(f−h),(g−e),(a+b),(c−a),(e+f),(b−d),(g+h) with cellwise parallel operations. Once inputs are ready, run seven products:

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Sync after all seven, then combine in parallel: C11=M1+M4−M5+M7, C12=M3+M5, C21=M2+M4, C22=M1−M2+M3+M6. Products cannot consume unfinished sums, and combination cannot consume unfinished products.



## Strassen's work, span, and parallel workspace

Seven half-size children and a constant number of matrix sums give

$$W_S(n) = 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}),$$
$$D_S(n) = D_S(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

Correctness follows from Chapter 13's expanded block identities and recursive induction, with the three additional parallel dependency stages. Parallelism Θ(n^(log₂7)/log²n) is smaller than for eight-way multiplication but work is lower. Maximizing a parallelism ratio alone is not the objective.

Sequential depth-first O(n²) temporaries do not transfer to simultaneous full child expansion. If each call keeps its own Θ(n²) workspace through join, eight-way space satisfies Q(n)=8Q(n/2)+Θ(n²)=Θ(n³); seven-way space is Θ(n^(log₂7)). Finite P and reuse can lower peaks, but require a consistent schedule/span analysis rather than an unimplemented space claim.

## Executable specification of parallel Strassen

PADD(X, Y, s) allocates independent R and uses nested parallel loops to write R[i, j]=X[i, j]+s · Y[i, j], waiting before return. For $s=\pm 1$, work is $\Theta(n^2)$, span $\Theta(\log n)$. Pad equal-order A, B to powers of two; the outer wrapper handles n=0.

```
P-STRASSEN(A, B):
    n=A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    S1=PADD(a, d,+1); S2=PADD(e, h,+1)
    S3=PADD(c, d,+1); S4=PADD(f, h,-1)
    S5=PADD(g, e,-1); S6=PADD(a, b,+1)
    S7=PADD(c, a,-1); S8=PADD(e, f,+1)
    S9=PADD(b, d,-1); S10=PADD(g, h,+1)
    M1=spawn P-STRASSEN(S1, S2)
    M2=spawn P-STRASSEN(S3, e)
    M3=spawn P-STRASSEN(a, S4)
    M4=spawn P-STRASSEN(d, S5)
```



## Executable specification of parallel Strassen (continued)

```
    M5=spawn P-STRASSEN(S6, h)
    M6=spawn P-STRASSEN(S7, S8)
    M7=P-STRASSEN(S9, S10)
    sync
    allocate C[n, n], viewed as four blocks
    parallel for i=0 to n/2-1:
        parallel for j=0 to n/2-1:
            C11[i, j]=M1[i, j]+M4[i, j]-M5[i, j]+M7[i, j]
            C12[i, j]=M3[i, j]+M5[i, j]
            C21[i, j]=M2[i, j]+M4[i, j]
            C22[i, j]=M1[i, j]-M2[i, j]+M3[i, j]+M6[i, j]
    return C
```

Ten PADD calls are sequential here, but their fixed count retains $\Theta(\log n)$ preparation span. Finish all before spawning products, consistent with the three-stage contract. No arbitrarily long serial issuing chain is ignored.

## Multithreaded merge sort

Parallel recursive sorts with a serial merge are insufficient:

```
PAR-SORT-SERIAL-MERGE(A, l, r):
    if r-l <= 1: return
    m=floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A, l, m)
    PAR-SORT-SERIAL-MERGE(A, m, r)
    sync
    serial MERGE the sorted halves using an auxiliary buffer
```

Work satisfies2W(n/2)+Θ(n)=Θ(n log n), but span D(n/2)+Θ(n)=Θ(n), giving only Θ(log n) parallelism. Sync before merge prevents reading unfinished runs. Reducing span further requires parallel merge itself.



## Define half-open cuts and tie order first

Read-only T contains runs[l1, r1),[l2, r2). Output O[o, o+N), where N is total length, must not overwrite T. Compare records by(key, original position), a total order with unique original positions, so equal keys retain original order.

```
LOWER-BOUND(T, l, r, x):
    while l < r:
        m=floor((l+r)/2)
        if T[m] < x: l=m+1
        else: r=m
    return l
```

The original run's boundary between values<x and≥x remains in current[l, r]. Positions before current l are<x; positions at and beyond current r within the original run are≥x. Each comparison preserves a possible cut and strictly reduces interval length. An empty run returns l without reading nonexistent T[l]. Work and span are O(1+log(n2+1)), including constant empty handling.

## Complete parallel merge, including empty runs

```
P-MERGE(T, l1, r1, l2, r2, O, o):
    n1=r1-l1; n2=r2-l2
    if n1 < n2: swap (l1, r1),(l2, r2); swap n1, n2
    if n1 == 0: return
    m=floor((l1+r1)/2)
    j=LOWER-BOUND(T, l2, r2, T[m])
    q=o+(m-l1)+(j-l2)
    O[q]=T[m]
    spawn P-MERGE(T, l1, m, l2, j, O, o)
    P-MERGE(T, m+1, r1, j, r2, O, q+1)
    sync
```

The longer run occupies the first descriptor. Swapping views neither moves nor relabels records. Empty first means both empty. Empty second still permits recursive median placement from the first without reading an empty interval. Each call removes one pivot, leaving total child size N−1, so recursion terminates.



## General proof of position, correctness, and stability

There are m−l1 first-run and j−l2 second-run records smaller than the pivot, total q−o. Every remaining record is larger in the total order. The pivot therefore belongs at q. Child output ranges[o, q),[q+1, o+N) are disjoint from one another and from the pivot.

Induct on N. Empty merge is correct. Both children are smaller and correctly merge their designated records by induction. Concatenating left result, pivot, right result is sorted and contains every input record exactly once. T is read-only and output ranges have exclusive writers, so no determinacy race occurs. Sync makes the complete answer available.

Original positions order equal keys, proving stability. A key-only comparison with the same lower_bound rule after swapping longer-run roles can place an equal right-run record first. Correct key sorting alone does not prove stability.

## Root cut and binary-search steps for eight records

Use T=[1a,4b,6c,9d | 2e,3f,6g,8h], where letters a..h identify original positions0..7. Compare keys first, then positions. Set l1=0, r1=4, l2=4, r2=8, o=0. The first length 4 run has m=2, pivot6c.

| Search[l, r) | mid | Comparison with 6c | Update |
|---|---|---|---|
| [4,8) | 6 | 6g>6c: equal key, later position | r=6 |
| [4,6) | 5 | 3f<6c | l=6 |
| [6,6) | none | empty interval | return j=6 |

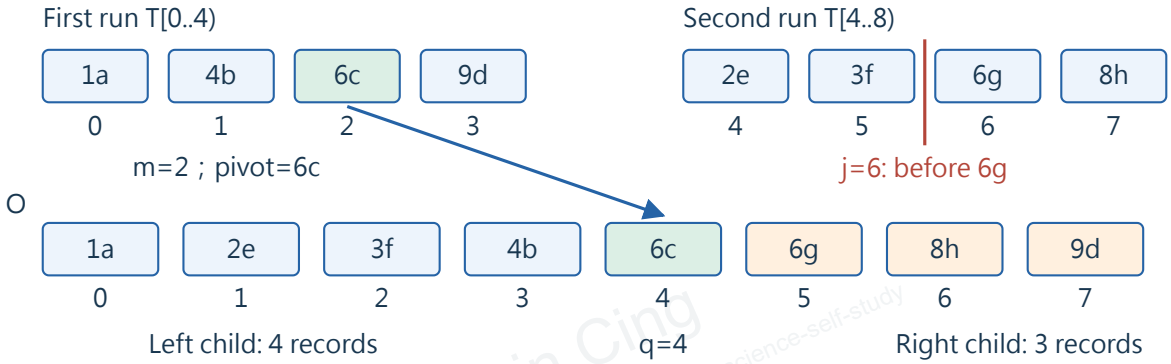
Position q=0+(2−0)+(6−4)=4. Left child contains 4 records, right child3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Root cut and binary-search steps for eight records (continued)

Median pivot rank: $q=o+(m-l_1)+(j-l_2)=4$



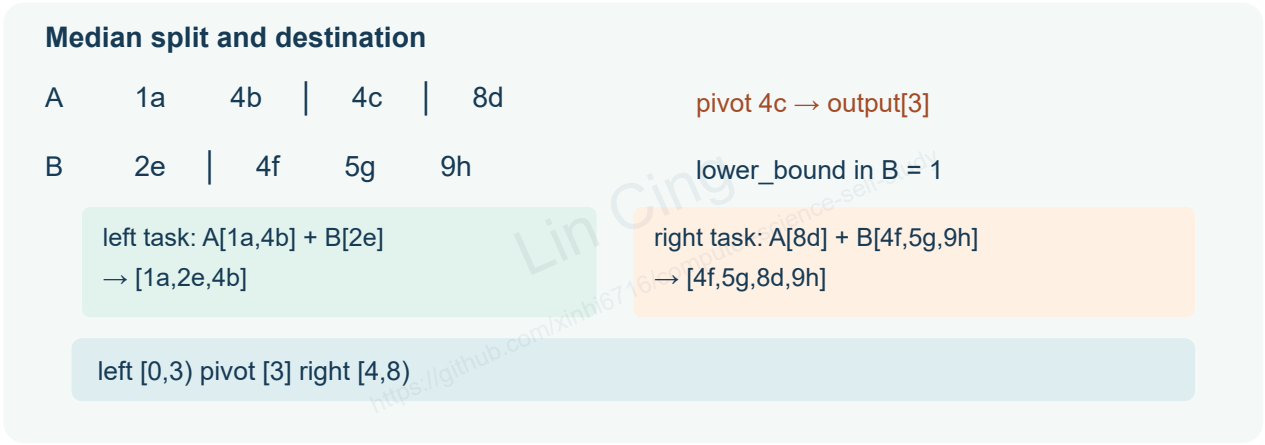

Compare(key, original position): 6c<6g. Left, pivot, right outputs are disjoint.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A duplicate-key trace

Let `A=[1a,4b,4c,8d]`, `B=[2e,4f,5g,9h]`, with original positions b < c < f. Select `x=4c`. Two items precede it in A, and B's lower_bound position is 1, so `q=2+1=3`.

The left task merges `[1a,4b]` with `[2e]`; the right merges `[8d]` with `[4f,5g,9h]`. The combined output is `[1a,2e,4b,4c,4f,5g,8d,9h]`.






## Every output write on the same example

One legal depth-first serialization writes pivots in the following order. Parallel interleavings may differ, but destinations stay fixed and each is written once.

| Write | Pivot | q, zero-based | Meaning |
|---|---|---|---|
| 1 | 6c | 4 | Root division |
| 2 | 4b | 3 | Last record of root's left region |
| 3 | 3f | 2 | Median after swapping longer run |
| 4 | 2e | 1 | Small-run cut |
| 5 | 1a | 0 | Other run empty |
| 6 | 8h | 6 | Median after right-region swap |
| 7 | 6g | 5 | Right region's left child |
| 8 | 9d | 7 | Right region's right child |

Final O=[1a,2e,3f,4b,6c,6g,8h,9d], with c still before g. Running the right child first gives the same answer because T is read-only and every destination unique. Sortedness does not rely on a global sequential write order.

## Upper and lower bounds on worst-case merge span

For n1≥n2, either child has size at most⌊n1/2⌋+n2≤(N+n2)/2≤3N/4, even if the other cut lands at an endpoint. Binary search costs O(log(N+1)), so

$$D_M(N) \le D_M(3N/4) + O(\log(N+1)) = O(\log^2(N+1)).$$

Constant-factor shrinkage gives O(log N) levels with logarithmic sizes O(log N−cr); their arithmetic sum gives the square.

For a lower bound, merge interleaved runs[0,2,…,2m−2] and[1,3,…,2m−1], with power-of-two m. The left child retains the same equal-length interleaving structure, halving length each level. Its other-run median search takes Θ(log m) comparisons. Summing log m,(log m−1),… gives Ω(log²m), hence worst-case Θ(log²N). If the other run stays empty, search costs constant and span is only Θ(log N). Worst-case order is not exact cost for every input.



## Writing each record once does not pay for searches

Every nonempty call removes a distinct pivot, so there are N nonempty calls. Each creates two children, for 2N+1 total calls including empty returns. The N output writes give Ω(N), but an O(N) upper bound must also pay each binary search.

A size-m node has nonrecursive cost at most c log(m+1). Divide it among its m input records, charging each c log(m+1)/m. A fixed record x is charged along one ancestor chain until chosen as pivot.

Count r levels upward from the last call containing x. The 3/4 child bound makes that ancestor's size at least(4/3)^r. Sum these charges below, including empty children, without taking log0 or falsely assuming both small children have size at least N/4.

## Each record's total search charge is bounded

Function $g(u) = \log_2(u+1)/u$ decreases for u>0. The sign of its derivative numerator is u/(u+1)−ln(u+1)<0, using $\int_0^u (1+t)^{-1}dt > u/(1+u)$. Thus the charge r levels upward is at most

$$c\frac{\log_2((4/3)^r+1)}{(4/3)^r} \le c[1 + r\log_2(4/3)](3/4)^r.$$

Both Σq^r=1/(1−q) and Σr q^r=q/(1−q)² converge at q=3/4; the latter follows by differentiating finite sums and taking limits. Each record pays O(1), for O(N) total. The N+1 empty calls add constant work each. With the output lower bound, W_M(N)=Θ(N).

This proof covers arbitrary legal cuts, empty runs, and unbalanced small children. A finite example is not the upper-bound proof.



## Complete two-buffer parallel merge sort

Attach original positions to records and initialize separate arrays X, Y of length n to the same input, using one parallel copy. Call SORT(X, Y,0, n), then read Y.

```
SORT(src, dst, l, r):
    if l == r: return
    if r-l == 1: dst[l]=src[l]; return
    m=floor((l+r)/2)
    spawn SORT(dst, src, l, m)
    SORT(dst, src, m, r)
    sync
    P-MERGE(src, l, m, m, r, dst, l)
```

Children swap buffer roles, leaving their sorted runs in the parent's src. Sync before merging into dst. P-MERGE input and output must be different physical buffers. Two global buffers avoid allocating a new length-n temporary per call and avoid overwriting unread input.

## Allocation and practical cutoffs

The bound assumes constant-time reservation of uninitialized buffers, with actual reads and writes charged separately. If allocation serially initializes n entries, that initialization alone can add Θ(n) span, invalidating the bound above.

The simple recursive version may hold Θ(n) buffer locations at each of O(log n) levels, for O(n log n) extra space. Two preallocated n-element buffers alternating roles between levels can reduce extra space to O(n), after separately proving that concurrent accesses do not conflict.

For small subproblems, a serial merge/sort cutoff reduces scheduling overhead. It changes constants and span and must be analyzed for the chosen cutoff; theoretical bounds are not benchmark results.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Buffer-swap correctness and race checks

At each SORT entry, both buffers' current intervals still contain that interval's original input. The root copy establishes this. Before launching children, the parent has not altered its interval, and each child touches only its disjoint half, so the entry property holds recursively.

A singleton copies to dst. By induction, children leave two sorted unchanged-multiset runs in parent src. After sync, P-MERGE stably merges them to dst. The parent need not promise src equals dst on exit; the equality condition is needed only on entry.

Even with role swaps, children write disjoint index ranges of the same two arrays. Parent merging waits for both. Data workspace is O(n), peak task descriptors O(n), and each call chain O(log n). Initial copying takes Θ(n) work,Θ(log n) span, without changing sorting orders.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A tight sort bound needs one compatible worst-case input

Every sort level has Θ(n) merge work and there are Θ(log n) levels, so W_S(n)=Θ(n log n). Span obeys

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\log^2 n) = O(\log^3 n).$$

For the lower bound, the same input must force expensive merges at all relevant levels. For n=2^L, place keys in L-bit-reversed index order; for n=8, use[0,4,2,6,1,5,3,7]. Root halves sort into interleaved even and odd runs. Each child's relative ranks have the same construction. Every level therefore has an Ω(log²(size)) merge, and parent merging starts only after both children finish.

Along a child chain,Σ_(r=1..L)r²=Θ(L³), giving worst-case Θ(log³n) for powers of two. Average parallelism is Θ(n/log²n). Merge's squared-log span is not the sort's span.



## Compatible expensive levels for nonpowers of two

For any n, recursively assign relative ranks. If n is even, put even ranks in the first half and odd in the second. If odd, put odd ranks in the shorter first half and even in the longer second. Recursively arrange each half's ordered key set the same way. Every sorted pair of runs interleaves with length difference at most1. For powers of two, this is bit reversal.

At each such merge, the longer-run median has m same-run records on its left, and m or m+1 other-run records. The left child remains interleaved with length difference at most1 and total size between N/2−1 and N/2+1. For N≥8, both runs have Θ(N) size. The selected path has Θ(log N) levels and logarithmic search cost at each shrinking scale, giving Θ(log²N) span.

The same recursive rank arrangement gives this property at every sort node. Follow the larger child, whose size roughly halves. Its completion precedes parent merge, so critical paths concatenate and give Ω(log³n). This supplies the tight lower bound for arbitrary n, without choosing incompatible worst inputs separately at different levels.

## Compare work and worst span in one model

| Algorithm | Work | Worst span | Average parallelism |
|---|---|---|---|
| Recursive P-FIB | $\Theta(\varphi^n)$ | $\Theta(n)$ | $\Theta(\varphi^n/n)$ |
| Row-parallel matrix-vector | $\Theta(n^2)$ | $\Theta(n)$ | $\Theta(n)$ |
| Reduction matrix-vector | $\Theta(n^2)$ | $\Theta(\log n)$ | $\Theta(n^2/\log n)$ |
| Two outer parallel loops for matrix product | $\Theta(n^3)$ | $\Theta(n)$ | $\Theta(n^2)$ |
| Eight-way recursive matrix product | $\Theta(n^3)$ | $\Theta(\log^2 n)$ | $\Theta(n^3/\log^2 n)$ |
| Parallel Strassen | $\Theta(n^{\log_2 7})$ | $\Theta(\log^2 n)$ | $\Theta(n^{\log_2 7}/\log^2 n)$ |
| DOT-reduction matrix product | $\Theta(n^3)$ | $\Theta(\log n)$ | $\Theta(n^3/\log n)$ |
| Parallel merge | $\Theta(n)$ | $\Theta(\log^2 n)$ | $\Theta(n/\log^2 n)$ |
| Parallel merge sort | $\Theta(n \log n)$ | $\Theta(\log^3 n)$ | $\Theta(n/\log^2 n)$ |

Asymptotic entries assume sufficiently large n; empty and singleton cases use explicit bases. Work efficiency, small span, and low space are distinct objectives. For fixed P, compare W/P, D, and memory together rather than substituting one parallelism ratio for the full analysis.



## References and attribution

Cormen, Leiserson, Rivest, Stein, Introduction to Algorithms, third edition, Chapter 27. MIT 6.046J Fall 2004, Lecture 3, matrix multiplication and Strassen. MIT 6.046J Fall 2005, A Minicourse on Dynamic Multithreaded Algorithms, Handout 29.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Summations

*Part VIII Appendix*

## Scope

Derive arithmetic, power, geometric, telescoping, and product identities from finite sums. Analyze loops with fixed-constant induction, tail ratios, dyadic blocks, and integral bounds. State convergence assumptions before using infinite series or changing summation order.

## Prerequisites

Arithmetic and geometric sequences, logarithms, and integral estimates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Appendix route

*Learning objectives*

- Use summation formulas and linearity.
- Bound sums by integration, splitting, and geometric series.
- Distinguish finite sums from infinite series.
- A.1 Summation formulas and properties
- A.2 Bounding summations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Summation formulas and properties

The finite sum $\sum_{k=1}^{n} a_k$ equals $0$ when $n = 0$. The finite product $\prod_{k=1}^{n} a_k$ equals $1$ when $n = 0$. These empty expressions use the additive and multiplicative identities, so formulas also cover loops with zero iterations. Finite terms may be reordered arbitrarily, and

$$\sum_{k=1}^{n}(ca_k + b_k) = c\sum_{k=1}^{n} a_k + \sum_{k=1}^{n} b_k.$$

An infinite series $\sum_{k=1}^{\infty} a_k$ is the limit of its partial sums $S_n$. It is not obtained by substituting $\infty$ into a finite formula. If the partial sums have no finite limit, the series diverges. Convergence requires $a_k \to 0$, because $a_k = S_k - S_{k-1}$. That condition is insufficient. The terms $1/k$ tend to zero, but the harmonic series diverges.

Absolute convergence means that $\sum_{k=1}^{\infty} |a_k|$ converges. It permits arbitrary reordering without changing the sum. A conditionally convergent series, such as $\sum_{k=1}^{\infty} (-1)^{k+1}/k$, need not preserve its sum under reordering, which changes partial-sum cancellation. Termwise calculus below requires a convergence interval. Finite linearity does not justify arbitrary infinite operations.



## Arithmetic sums by pairing and induction

Write $S_n = 1 + 2 + \cdots + n$ and reverse the order to obtain $n + (n-1) + \cdots + 1$. Adding the two expressions gives $2S_n = n(n+1)$, so

$$S_n = \frac{n(n+1)}{2}.$$

The formula includes $n = 0$. Induction gives a second verification. The base case is $S_0 = 0$, and the induction step is

$$S_{n+1} = \frac{n(n+1)}{2} + (n+1) = \frac{(n+1)(n+2)}{2}.$$

The induction coefficient stays fixed as $n$ changes. More generally,

$$\sum_{j=0}^{n-1}(a + jd) = na + \frac{dn(n-1)}{2}.$$

A loop with $j = 3, \ldots, 7$ that performs $2j - 1$ units of work has total cost

## Arithmetic sums by pairing and induction (continued)

$$\begin{aligned}\sum_{j=3}^{7}(2j-1) &= \left(2\sum_{j=1}^{7} j - 7\right) - \left(2\sum_{j=1}^{2} j - 2\right) \\ &= 49 - 4 = 45 \\ &= 5 + 7 + 9 + 11 + 13.\end{aligned}$$

Changing the starting index requires subtracting the correct prefix. The upper index $7$ is not the number of terms, which is $5$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Square and cube sums from telescoping

Sum $(k+1)^3 - k^3 = 3k^2 + 3k + 1$ for $k = 0, \ldots, n$. The left side telescopes, giving

$$(n+1)^3 = 3\sum_{k=0}^{n} k^2 + \frac{3n(n+1)}{2} + (n+1).$$

Solving for the sum gives

$$\sum_{k=0}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}.$$

Similarly, sum $(k+1)^4 - k^4 = 4k^3 + 6k^2 + 4k + 1$ and substitute the preceding square and arithmetic sums:

$$\begin{aligned}(n+1)^4 &= 4\sum_{k=0}^{n} k^3 + 6\sum_{k=0}^{n} k^2 + 4\sum_{k=0}^{n} k + (n+1), \\ \sum_{k=0}^{n} k^3 &= \frac{n^2(n+1)^2}{4} = \left(\frac{n(n+1)}{2}\right)^2.\end{aligned}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Square and cube sums from telescoping (continued)

Both formulas hold at $n = 0$. At $n = 4$, the checks are

$$0 + 1 + 4 + 9 + 16 = 30 = \frac{4 \cdot 5 \cdot 9}{6},$$
$$0 + 1 + 8 + 27 + 64 = 100 = \left(\frac{4 \cdot 5}{2}\right)^2.$$

Telescoping proves the formulas for general $n$. The small cases check arithmetic and transcription.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Finite, infinite, and boundary geometric sums

Let $G_n(x) = \sum_{k=0}^{n} x^k$. Subtract $G_n$ from $xG_n$. Interior terms cancel, so

$$(x - 1)G_n(x) = x^{n+1} - 1.$$

For $x \neq 1$, this gives $G_n(x) = (x^{n+1} - 1)/(x - 1)$. At $x = 1$, evaluate separately to obtain $G_n(1) = n + 1$ ; division by zero is invalid.

If $|x| < 1$, then $x^{n+1} \to 0$, and

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}.$$

At $x = 1$, partial sums are unbounded. At $x = -1$, they alternate between $1$ and $0$. If $|x| > 1$, the terms do not tend to zero.

For $x = -1/2$ and $n = 3$, the finite sum is $1 - 1/2 + 1/4 - 1/8 = 5/8$, whereas the infinite sum is $2/3$. When $|x| < 1$, the exact tail and its absolute bound are

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Finite, infinite, and boundary geometric sums (continued)

$$\sum_{k=n+1}^{\infty} x^k = \frac{x^{n+1}}{1-x}, \qquad \left|\frac{x^{n+1}}{1-x}\right| \le \frac{|x|^{n+1}}{1-|x|}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Weighted geometric sums and valid calculus

Finite polynomials may be differentiated term by term. A direct derivation starts with $W_n = \sum_{k=0}^{n} kx^k$. Subtracting $xW_n$ gives

$$(1-x)W_n = x + x^2 + \cdots + x^n - nx^{n+1}.$$

Substitute the finite geometric formula and rearrange:

$$W_n = \frac{x - (n+1)x^{n+1} + nx^{n+2}}{(1-x)^2}, \qquad x \neq 1.$$

At $x = 1$, $W_n = n(n+1)/2$. For $|x| < 1$, $n|x|^n \to 0$, because an exponential with a fixed base dominates a polynomial. Taking the limit gives

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}.$$

This derivation does not interchange differentiation with an infinite sum without proof. Power series permit termwise differentiation and integration inside their radius of convergence, which is $1$ here. Endpoints require separate checks.

## Weighted geometric sums and valid calculus (continued)

Apply $x\,d/dx$ once more inside that radius:

$$\sum_{k=0}^{\infty} k^2 x^k = \frac{x(1+x)}{(1-x)^3}.$$

Thus $x = 1/2$ gives $\sum_{k=0}^{\infty} k/2^k = 2$ and $\sum_{k=0}^{\infty} k^2/2^k = 6$. Terms tending to zero alone would not establish these convergent sums.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Telescoping leaves boundary terms

Each interior term occurs once with each sign, so

$$\sum_{k=1}^{n} (a_k - a_{k-1}) = a_n - a_0.$$

Align indices and endpoints before cancellation. For example,

$$\frac{1}{k(k+2)} = \frac{1}{2}\left(\frac{1}{k} - \frac{1}{k+2}\right),$$

which gives

$$\sum_{k=1}^{n} \frac{1}{k(k+2)} = \frac{1}{2}\left(1 + \frac{1}{2} - \frac{1}{n+1} - \frac{1}{n+2}\right).$$

At $n = 3$, $1/3 + 1/8 + 1/15 = 21/40$, matching $\frac{1}{2}(3/2 - 1/4 - 1/5)$. Letting $n \to \infty$ gives $3/4$, since the terminal denominators tend to infinity.

Products telescope as well. For $n \geq 2$,

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Telescoping leaves boundary terms (continued)

$$\prod_{k=2}^{n}\left(1-\frac{1}{k^2}\right)=\left(\prod_{k=2}^{n}\frac{k-1}{k}\right)\left(\prod_{k=2}^{n}\frac{k+1}{k}\right)$$
$$=\frac{1}{n}\frac{n+1}{2}=\frac{n+1}{2n}.$$

At $n=1$, the empty product is $1$, as the formula also predicts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Products as sums and indispensable premises

If every $a_k>0$, then

$$\ln\left(\prod_{k=1}^{n}a_k\right)=\sum_{k=1}^{n}\ln a_k.$$

A zero or negative factor has no real logarithm. For example,

$$\prod_{k=1}^{n}2^{3k+1}=2^{\sum_{k=1}^{n}(3k+1)}=2^{3n(n+1)/2+n}.$$

For $0\le u_k<1$, logarithm bounds give

$$-\frac{u_k}{1-u_k}\le\ln(1-u_k)\le-u_k.$$

These bounds control products through sums. As $u_k$ approaches $1$, the lower-bound denominator cannot be treated as a fixed constant.

If fixed constants satisfy $0<c_1f(k)\le h(k)\le c_2f(k)$ for every $k\ge1$, summing gives

## Products as sums and indispensable premises (continued)

$$c_1 \sum_{k=1}^{n} f(k) \le \sum_{k=1}^{n} h(k) \le c_2 \sum_{k=1}^{n} f(k).$$

If the inequalities hold only on a tail, retain the finite prefix separately. When $\sum_{k=1}^{n} f(k) \to \infty$ and the prefix is finite, the prefix does not change the $\Theta$ order. Constants must be uniform over $k$. An unbounded choice of a different constant at every index proves no common $\Theta$ bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Bounding summations

Decide whether an exact value, an upper bound, or a two-sided order is needed. Induction, termwise bounds, fixed ratios, splitting, and integration serve different purposes. A rough upper bound alone does not establish tightness.

For $S_n = \sum_{k=0}^{n} 4^k$, seek $S_n \le c4^n$. The base case $n = 0$ requires $c \ge 1$. The induction step gives

$$S_{n+1} \le c4^n + 4^{n+1} = \left(\frac{c}{4} + 1\right) 4^{n+1}.$$

It closes when $c/4 + 1 \le c$, or $c \ge 4/3$. The fixed choice $c = 4/3$ works for all $n$. The last term gives $S_n \ge 4^n$, so $S_n = \Theta(4^n)$.

The false induction claiming that $\sum_{k=1}^{n} k = O(n)$ remains linear after adding $n + 1$ hides a growing constant. An assumption $\sum_{k=1}^{n} k \le cn$ would require

$$cn + n + 1 \le c(n + 1), \qquad \text{or} \qquad n + 1 \le c.$$

No fixed $c$ satisfies this for every $n$.

## Maximum-term upper bounds and tail lower bounds

For nonnegative $a_k$ and any subset $J \subseteq \{1, \dots, n\}$,

$$\sum_{k=1}^{n} a_k \le n \max_{1 \le k \le n} a_k, \qquad \sum_{k=1}^{n} a_k \ge \sum_{k \in J} a_k.$$

Upper bounds may use all terms. Lower bounds often retain enough large terms. For example, $\sum_{k=1}^{n} k \le n^2$. The indices $\lceil n/2 \rceil, \dots, n$ contain at least $n/2$ terms, each at least $n/2$. Thus $\sum_{k=1}^{n} k \ge n^2/4$ for both odd and even $n$, proving $\Theta(n^2)$ without an exact formula.

For a fixed real $r \ge 0$, the same argument gives

$$2^{-r-1} n^{r+1} \le \sum_{k=1}^{n} k^r \le n^{r+1}.$$

Hence the sum is $\Theta(n^{r+1})$. Fixing $r$ is essential. If $r = n$, the lower coefficient $2^{-r-1}$ depends on $n$, and this proof gives no uniform order.



## A geometric tail requires a fixed ratio

Suppose $a_k \ge 0$ and a fixed $0 < \rho < 1$ satisfies $a_{k+1} \le \rho a_k$ for every $k \ge K$. Induction gives $a_{K+j} \le a_K \rho^j$, so

$$\sum_{k=K}^{\infty} a_k \le \frac{a_K}{1 - \rho}.$$

The multiplicative inequality also covers zero terms without division by zero.

For $a_k = k^2/3^k$ and $k \ge 2$,

$$\frac{a_{k+1}}{a_k} = \frac{1}{3}\left(1 + \frac{1}{k}\right)^2 \le \frac{3}{4}.$$

The prefix at $k = 0, 1$ contributes $1/3$. The tail starts at $a_2 = 4/9$, giving

$$\sum_{k=2}^{\infty} a_k \le \frac{4/9}{1/4} = \frac{16}{9}, \qquad \sum_{k=0}^{\infty} a_k \le \frac{19}{9}.$$

## A geometric tail requires a fixed ratio (continued)

The exact weighted geometric formula gives $3/2$, a sharper finite bound. Harmonic ratios $k/(k+1) < 1$ tend to $1$, so no fixed $\rho < 1$ bounds the whole tail. A prefix that depends on $n$, such as two terms each equal to $n^2$, is not $O(1)$ merely because it contains only two terms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Dyadic blocks reveal harmonic layers

Let $H_n = \sum_{k=1}^{n} 1/k$. The block $k = 2^j, \ldots, 2^{j+1} - 1$ has $2^j$ terms, each between $1/2^{j+1}$ and $1/2^j$. Each complete block contributes between $1/2$ and $1$.

Complete the last partial block for an upper bound. There are $\lfloor \log_2 n \rfloor + 1$ blocks, each contributing at most $1$. The first $\lfloor \log_2 n \rfloor$ complete blocks give a lower bound. For $n \geq 2$,

$$\frac{\lfloor \log_2 n \rfloor}{2} \leq H_n \leq \lfloor \log_2 n \rfloor + 1.$$

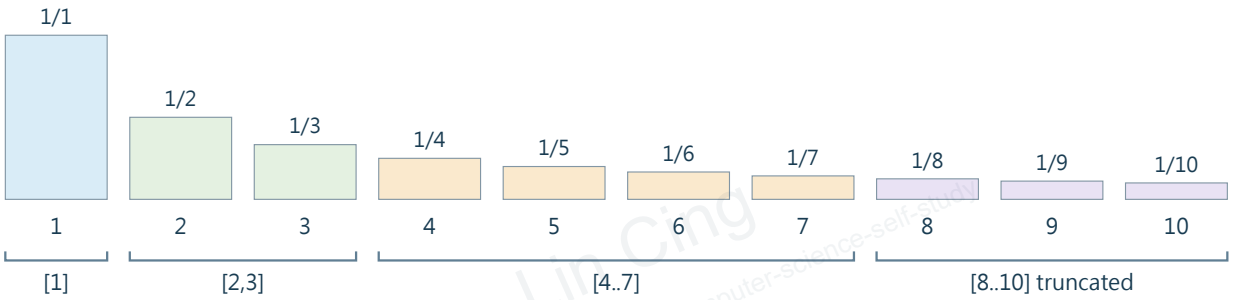
The lower bound is $\Omega(\log n)$, so $H_n = \Theta(\log n)$, also proving that the infinite series diverges.

At $n = 10$, the blocks $[1]$, $[2, 3]$, $[4, 5, 6, 7]$, and $[8, 9, 10]$ contribute $1$, $5/6$, $319/420$, and $121/360$. Their total is $H_{10} = 7381/2520 \approx 2.928968$. The last block has three terms, not eight.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Dyadic blocks reveal harmonic layers (continued)

n=10: each full dyadic block sums between1/2 and1




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Integral bounds follow from unit intervals

If $f$ is integrable and nondecreasing on $[m-1, n+1]$, each integer $k$ satisfies

$$\int_{k-1}^{k} f(x)\,dx \le f(k) \le \int_{k}^{k+1} f(x)\,dx.$$

Summing gives

$$\int_{m-1}^{n} f(x)\,dx \le \sum_{k=m}^{n} f(k) \le \int_{m}^{n+1} f(x)\,dx.$$

For a nonincreasing $f$, the inequalities reverse:

$$\int_{m}^{n+1} f(x)\,dx \le \sum_{k=m}^{n} f(k) \le \int_{m-1}^{n} f(x)\,dx.$$

The endpoint shifts control the error. A sum is not simply replaced by an integral. Since $1/x$ decreases for $x > 0$,

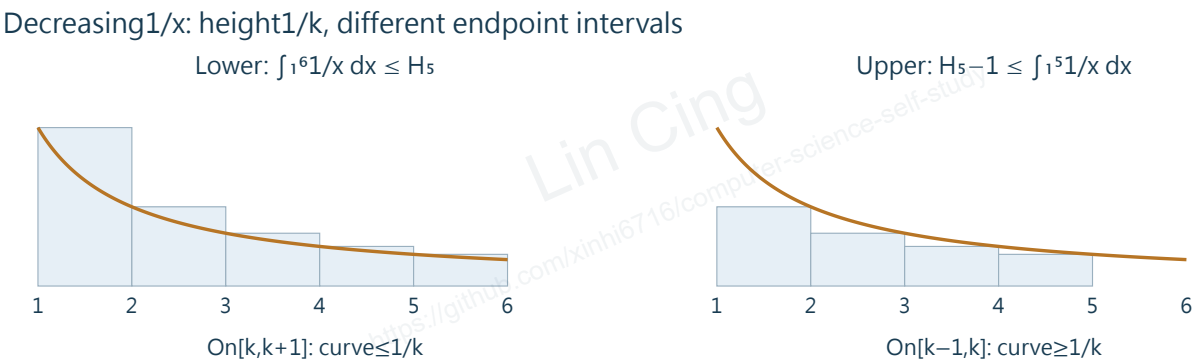
$$H_n \ge \int_{1}^{n+1} \frac{dx}{x} = \ln(n+1).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Integral bounds follow from unit intervals (continued)

Integrating from zero for an upper bound encounters a singularity. Remove the first term, then bound the terms $k = 2, \ldots, n$:

$$H_n \le 1 + \int_1^n \frac{dx}{x} = 1 + \ln n.$$






## Recomputable two-sided integral bounds

At $n = 5$, $H_5 = 137/60 \approx 2.283333$ lies between $\ln 6 \approx 1.791759$ and $1 + \ln 5 \approx 2.609438$. This bracket has an explicit width; it does not assert equality with $\ln n$. In general,

$$\ln(n+1) \le H_n \le \ln n + 1, \qquad H_n = \ln n + O(1).$$

For a fixed $p > 1$,

$$\sum_{k=1}^{n} \frac{1}{k^p} \le 1 + \int_1^n x^{-p}\,dx \le 1 + \frac{1}{p-1}.$$

The nonnegative partial sums increase and are bounded, so they converge. The case $p = 1$ diverges. For $0 \le p < 1$, the integral lower bound grows as $n^{1-p}$, and the sum is $\Theta(n^{1-p})$. Vanishing terms alone do not imply convergence.

For a fixed $r \ge 0$, integration also gives

$$\frac{n^{r+1}}{r+1} \le \sum_{k=1}^{n} k^r \le \frac{(n+1)^{r+1} - 1}{r+1}.$$

## Recomputable two-sided integral bounds (continued)

This again proves $\Theta(n^{r+1})$, with constants allowed to depend on the fixed parameter $r$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Nested loops and changing summation order

Loops with $i = 1, \dots, n$ and $j = i, \dots, n$ perform constant work

$$\sum_{i=1}^{n} (n - i + 1) = \frac{n(n+1)}{2}$$

times. Counting by $j$ instead gives $i = 1, \dots, j$, hence $\sum_{j=1}^{n} j$. Both count the index set $\{(i, j) : 1 \leq i \leq j \leq n\}$.

If an iteration costs $j$, then

$$\sum_{i=1}^{n} \sum_{j=i}^{n} j = \sum_{j=1}^{n} \sum_{i=1}^{j} j = \sum_{j=1}^{n} j^2 = \frac{n(n+1)(2n+1)}{6}.$$

At $n = 3$, the row sums are $6, 5, 3$, totaling $14$. The column count $1 \cdot 1 + 2 \cdot 2 + 3 \cdot 3$ also totals $14$.

Finite reordering is safe. An infinite double sum with signed terms requires absolute convergence before arbitrary interchange of order. Finite identities alone do not justify that operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Sources

*Bibliography*

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Sets, relations, functions, graphs, and trees

*Part VIII Appendix*

### Scope

Use elementwise logic and general proofs to connect sets, equivalence partitions, functions, graphs, and trees. State graph, path, direction, empty-set, and binary-slot conventions explicitly. Prove the six equivalent tree conditions and the bipartite criterion.

### Prerequisites

Basic discrete-mathematics notation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Appendix route

*Learning objectives*

- Apply definitions of sets, relations, and functions precisely.
- Recognize graph and tree structure.
- Connect Cartesian products, equivalence relations, and recursive structures.
- B.1 Sets
- B.2 Relations
- B.3 Functions
- B.4 Graphs
- B.5 Trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Sets

A set is unordered and has no repetition, so $\{u, v, u\} = \{v, u\}$. A multiset retains multiplicity, while a sequence retains positions. These are different objects. Membership is written $x \in A$. The condition $A \subseteq B$ means that every $x \in A$ also lies in $B$; $A \subsetneq B$ additionally requires $A \neq B$.

The empty set is a subset of every set $A$, but need not be an element of $A$. The set $\{\varnothing\}$ has one element and is not empty. Equality requires both inclusions, $A \subseteq B$ and $B \subseteq A$.

Here $\mathbb{N} = \{0, 1, \ldots\}$, $\mathbb{Z}$ denotes the integers, and $\mathbb{R}$ denotes the reals. The expression $\{x \in \mathbb{Z} : 3 \mid x\}$ specifies a subset by a property, rather than an ordered list.

Set operations express elementwise logic. Membership in $A \cup B$ uses "or," membership in $A \cap B$ uses "and," and $x \in A \setminus B$ means $x \in A$ and $x \notin B$. A complement requires a specified universe $U$, with $A^c = U \setminus A$. Changing $U$ changes the complement.



## Set identities quantify over every element

| Property | Union form | Intersection form |
|---|---|---|
| Identity and zero | $A \cup \varnothing = A$ | $A \cap \varnothing = \varnothing$ |
| Idempotence | $A \cup A = A$ | $A \cap A = A$ |
| Commutativity | $A \cup B = B \cup A$ | $A \cap B = B \cap A$ |
| Associativity | $A \cup (B \cup C) = (A \cup B) \cup C$ | $A \cap (B \cap C) = (A \cap B) \cap C$ |
| Distributivity | $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ | $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ |
| Absorption | $A \cup (A \cap B) = A$ | $A \cap (A \cup B) = A$ |
| De Morgan | $(B \cup C)^c = B^c \cap C^c$ | $(B \cap C)^c = B^c \cup C^c$ |

For an arbitrary element $x$,

## Set identities quantify over every element (continued)

$$\begin{aligned} x \in A \setminus (B \cap C) &\iff x \in A \ \wedge\ \neg(x \in B \wedge x \in C) \\ &\iff (x \in A \wedge x \notin B) \ \vee\ (x \in A \wedge x \notin C) \\ &\iff x \in (A \setminus B) \cup (A \setminus C). \end{aligned}$$

This two-way equivalence proves the identity for every $x$. A finite Venn example cannot replace that universal quantifier.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Partitions, cardinality, and overlapping counts

A partition of $S$ is a collection of nonempty, pairwise disjoint blocks whose union is $S$. Every $x \in S$ belongs to exactly one block. For $S = \varnothing$, the empty collection of blocks is allowed. Congruence modulo $3$ partitions $\{0, \dots, 7\}$ into $\{0, 3, 6\}$, $\{1, 4, 7\}$, and $\{2, 5\}$.

For finite $A, B$, split their union into the disjoint parts $A \setminus B$, $A \cap B$, and $B \setminus A$. This gives

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

For $A = \{0, 2, 4, 6\}$ and $B = \{2, 3, 6\}$, the union has size $4 + 3 - 2 = 5$. Simply adding cardinalities counts the intersection twice.

Three-set inclusion-exclusion adds the three singleton sizes, subtracts the three pair intersections, and adds the triple intersection. In general, an element that appears in $r \geq 1$ sets receives coefficient

$$\sum_{j=1}^{r} (-1)^{j+1} \binom{r}{j} = 1,$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Partitions, cardinality, and overlapping counts (continued)

because $(1-1)^r = 0$. An element in no set receives coefficient $0$. This proves inclusion-exclusion element by element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Power sets versus Cartesian products

The power set is $\mathcal{P}(S) = \{T : T \subseteq S\}$. For $n$ distinguishable elements, each subset chooses whether each element is included. This is a bijection with the $n$-bit strings, so $|\mathcal{P}(S)| = 2^n$. In particular, $\mathcal{P}(\varnothing) = \{\varnothing\}$ has one element.

An ordered pair $(a, b)$ retains positions and generally differs from $(b, a)$. Define

$$A \times B = \{(a, b) : a \in A,\ b \in B\}.$$

For finite sets, each fixed $a$ gives $|B|$ pairs. The $|A|$ disjoint rows give $|A \times B| = |A||B|$. An $n$-fold product counts ordered tuples and multiplies the finite factor sizes. The set $A^0$ contains the unique empty tuple, so it has size $1$.

For $A = \{p, q\}$ and $B = \{0, 1, 2\}$, $A \times B$ has $6$ elements. By contrast, $\mathcal{P}(A)$ has the four elements $\varnothing$, $\{p\}$, $\{q\}$, and $\{p, q\}$. A sequence is a function from positions to values. Equal values at distinct positions remain distinct entries.

## Infinite cardinality requires bijections

Two sets have equal cardinality when a bijection exists. Being infinite alone does not establish equal cardinality. Define a map from $\mathbb{N}$ to $\mathbb{Z}$ by

$$f(0) = 0, \qquad f(2k-1) = k, \qquad f(2k) = -k \quad (k \geq 1).$$

Every integer has exactly one preimage, so $\mathbb{Z}$ is countable. The map $n \mapsto 2n$ is a bijection onto the even natural numbers. An infinite set can have the same cardinality as a proper subset.

To prove that $\mathbb{R}$ is uncountable, suppose $(0, 1)$ has a list $x_1, x_2, \ldots$. Choose decimal representations that do not end in infinitely many $9$s. Define the $i$th digit of $y$ to be $1$ unless the $i$th digit of $x_i$ is $1$, in which case choose $2$. Then $y \in (0, 1)$, its representation is unambiguous, and it differs from every $x_i$ at digit $i$. This contradicts the claimed complete list.

For finite sets of equal size, injection implies surjection. For infinite sets, $f(n) = n + 1$ on $\mathbb{N}$ is injective but misses $0$. The finite premise is essential.



## Relations

A binary relation is a subset $R \subseteq A \times B$. A relation on one set has $R \subseteq A \times A$. Write $aRb$ when $(a, b) \in R$. A relation need not assign a partner to every input, and a partner need not be unique.

| Property | Requirement for all $a, b, c \in A$ |
|---|---|
| Reflexive | $aRa$ |
| Symmetric | $aRb \implies bRa$ |
| Antisymmetric | $(aRb \land bRa) \implies a = b$ |
| Transitive | $(aRb \land bRc) \implies aRc$ |

Symmetry and antisymmetry are not negations. Equality satisfies both. For distinct $a, b, c$, a relation containing $a \leftrightarrow b$ and $b \to c$, but lacking $c \to b$, can satisfy neither. On a nonempty $A$, the empty relation is vacuously symmetric, antisymmetric, and transitive, but is not reflexive. Symmetry and transitivity do not guarantee the existence of any related pair.

## Equivalence relations induce partitions

An equivalence relation is reflexive, symmetric, and transitive. Define $[a] = \{b \in A : aRb\}$. Reflexivity makes each class nonempty and ensures that every $a$ is covered.

Suppose $c \in [a] \cap [b]$. Then $aRc$ and $bRc$ imply $aRb$ by symmetry and transitivity. For any $x \in [a]$, symmetry gives $bRa$, and transitivity with $aRx$ gives $bRx$. Thus $[a] \subseteq [b]$. Exchanging $a$ and $b$ gives the reverse inclusion. Classes coincide or are disjoint, and removing duplicates yields a partition.

Conversely, given a partition, define $aRb$ exactly when $a$ and $b$ share a block. Every element lies in one block, proving reflexivity. Sharing a block is symmetric. If $a, b$ share one block and $b, c$ share another, uniqueness of the block containing $b$ makes all three share a block. This proves transitivity. The block containing $a$ is exactly $[a]$, so the constructions correspond.

Congruence modulo $3$ illustrates the axioms. The difference $a - b$ is a multiple of $3$. Zero proves reflexivity, negation proves symmetry, and addition proves transitivity. The integer classes have residues $0, 1, 2$; restricting them to the earlier finite set gives its three blocks.



## Partial orders, total orders, and extrema

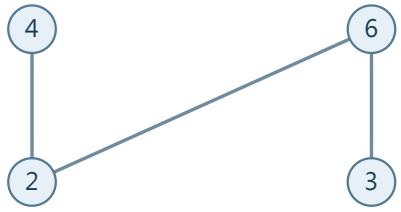
A partial order is reflexive, antisymmetric, and transitive. A total order also compares every pair. A total preorder requires reflexivity, transitivity, and comparability, but permits distinct elements tied in both directions.

A maximum $M$ satisfies $x \leq M$ for every $x$. A maximal element $m$ merely has no strictly larger element. Minimum and minimal are the dual notions. If $M, N$ are both maxima, then $M \leq N$ and $N \leq M$, so antisymmetry gives $M = N$. A maximum is maximal, but a maximal element need not be a maximum.

On $S = \{2, 3, 4, 6\}$ ordered by divisibility, the minimal elements are $2, 3$ and the maximal elements are $4, 6$. Neither a minimum nor a maximum exists. A Hasse edge is a cover $x < y$ with no $z$ satisfying $x < z < y$. Draw larger elements above smaller ones and omit reflexive and transitive edges.

## Partial orders, total orders, and extrema (continued)

S={2,3,4,6}: divisibility Hasse diagram




Upward: larger divisible element

covers：2–4、2–6、3–6

minimal：2、3

maximal：4、6

No minimum or maximum

Lines are covers, not numerical distances. Reflexive and transitive edges are omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Functions

A function $f : A \to B$ is a relation assigning exactly one $b \in B$ to every $a \in A$. The set $A$ is the domain, $B$ is the codomain, and the range $f(A)$ may be smaller than $B$. A partial function undefined at some $a$ is not a total function under this definition.

An injection satisfies $f(a) = f(a') \implies a = a'$. A surjection gives every $b \in B$ at least one preimage. A bijection satisfies both properties. The map $f(n) = 2n$ from $\mathbb{N}$ to $\mathbb{N}$ is injective but not onto. Changing its codomain to the nonnegative even integers makes it bijective. The map $n \mapsto \lfloor n/2 \rfloor$ from $\mathbb{N}$ to $\mathbb{N}$ is onto but not injective, since both $0$ and $1$ map to $0$.

For finite sets, an injection uses distinct codomain elements, so $|A| \leq |B|$. A surjection must supply every codomain element, so $|A| \geq |B|$. With equal finite sizes, either property implies the other. The infinite counterexample above prevents extending that inference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Inverses, preimages, and composition

For a bijection, $f^{-1}(b)$ is the unique $a$ with $f(a) = b$, and

$$f^{-1} \circ f = \mathrm{id}_A, \qquad f \circ f^{-1} = \mathrm{id}_B .$$

An injection alone has an inverse only on its range. Without injection, the inverse relation generally is not a function.

For any function and $T \subseteq B$, the set preimage is

$$f^{-1}(T) = \{a \in A : f(a) \in T\}.$$

It is defined without invertibility. It shares notation with an inverse function but is a different object.

For $f : A \to B$ and $g : B \to C$, composition satisfies $(g \circ f)(a) = g(f(a))$. If both maps are injective, equality of composite values lets us cancel $g$, then $f$, proving equality of inputs. If both are onto, given $c \in C$, choose $b \in B$ with $g(b) = c$, then $a \in A$ with $f(a) = b$. This proves surjectivity of the composite. For bijections,

$$(g \circ f)^{-1} = f^{-1} \circ g^{-1},$$



## Inverses, preimages, and composition (continued)

with the order reversed.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Graph conventions and degree sums

A finite digraph $G = (V, E)$ has $E \subseteq V \times V$, permitting self-loops but no parallel edges. A simple undirected graph uses unordered pairs of distinct vertices and has no loops. A multigraph must preserve edge identities and specify how loops contribute to degree.

A directed edge $(u, v)$ leaves $u$ and enters $v$. A self-loop contributes one to each of indegree and outdegree. Undirected degree counts incident edges. Each undirected edge has two endpoints, while each directed edge has one tail and one head. Thus

$$\sum_{v \in V} \deg(v) = 2|E|,$$

$$\sum_{v \in V} \operatorname{outdeg}(v) = \sum_{v \in V} \operatorname{indeg}(v) = |E|.$$

The first identity implies that the number of odd-degree vertices is even. Moving vertices or bending edges does not change $E$. Crossing lines are not junctions unless marked as vertices. Legends or edge lists must determine directions, identities, and values.



## Walks, simple paths, and cycles

A walk $v_0, \ldots, v_k$ follows an edge between each adjacent pair. Its length is $k$ edges, rather than $k + 1$ vertices. CLRS permits repetition in a path; that object is called a walk here. A simple path has distinct vertices. Every vertex has a length-zero path to itself.

For undirected acyclicity, a cycle has length at least $3$ and repeats only the start and end vertex. Traversing one edge from $u$ to $v$ and back is not a cycle. Directed simple cycles may have length $1$ for a loop or $2$ for opposite arcs, and must follow every direction. A general closed walk may repeat other vertices and differs from a simple cycle.

If a walk from $u$ to $v$ repeats $v_i = v_j$ with $i < j$, remove the closed segment between those occurrences. The remaining sequence is a shorter walk with the same endpoints. Repeated removal strictly decreases a finite length and eventually leaves a simple path. This proves equivalence of reachability by walks and by simple paths without listing infinitely many walks.

## Components, SCCs, and direction

Undirected reachability is reflexive by zero-length paths, symmetric by reversal, and transitive by concatenation. Components are its equivalence classes. Directed reachability is reflexive and transitive, but need not be symmetric. Mutual directed reachability has all three properties; its classes are strongly connected components, or SCCs.

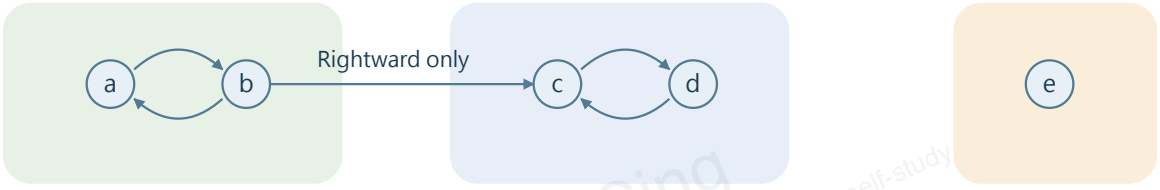
Take the arcs $a \to b$, $b \to a$, $b \to c$, $c \to d$, $d \to c$, and an isolated vertex $e$. The SCCs are $\{a, b\}$, $\{c, d\}$, and $\{e\}$. Vertex $a$ reaches $d$, but $d$ cannot reach $a$, so the first two SCCs remain distinct. Ignoring directions gives only the components $\{a, b, c, d\}$ and $\{e\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Components, SCCs, and direction (continued)

Reachability is directional; mutual reachability differs




a reaches d; d cannot reach a. Ignore direction: a/b/c/d together, e still isolated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Subgraphs, isomorphism, and contraction

A subgraph selects vertices and some original edges, keeping both endpoints of every selected edge. An induced subgraph $G[W]$ keeps all original edges with both endpoints in $W$. For the preceding example, $W = \{a, b, c\}$ induces exactly $a \to b$, $b \to a$, and $b \to c$.

An isomorphism is a vertex bijection $f : V \to V'$ satisfying

$$(u, v) \in E \iff (f(u), f(v)) \in E'.$$

Preservation in one direction alone is not the general definition. Equal vertex counts, edge counts, and degree multisets are necessary but insufficient. The cycle $C_6$ and two disjoint triangles both have $6$ vertices, $6$ edges, and degree $2$ everywhere, but only the cycle is connected.

Contracting an undirected edge $\{u, v\}$ merges its endpoints into $x$, redirects other incident edges, and removes the contracted edge. A simple representation merges parallel copies to a common neighbor and discards loops. Algorithms that need parallel edges, including some randomized contractions, must explicitly use a multigraph instead of silently deduplicating.



## Graph classes and the bipartite criterion

The complete graph $K_n$ joins every distinct pair and has $n(n-1)/2$ edges. A bipartite graph has a partition $V = L \sqcup R$ with every edge crossing the two sides, equivalently a proper two-coloring. A DAG is directed and acyclic; trees and forests here are undirected. Absence of directed cycles alone does not make a tree.

A bipartite graph has no odd cycle. Every edge switches sides, and an odd number of switches cannot return to the original side. Conversely, suppose there is no odd simple cycle. Root each component and color vertices by the parity of their shortest undirected distance from the root. An edge joining equal parities would create an odd closed walk by following a root-to-$u$ path, the edge, and a reversed root-to-$v$ path.

If that odd closed walk repeats an internal vertex, split there into two shorter closed walks. Their lengths sum to an odd number, so one is odd. Repeatedly keep the odd part until no internal repetition remains. This is an odd simple cycle, a contradiction. Hence every edge crosses colors, proving bipartiteness.

## Graph classes and the bipartite criterion (continued)

A hyperedge may contain many vertices. Its incidence bipartite graph puts original vertices on one side and hyperedges on the other, with an edge for each membership. These are different node types; a hyperedge is not an ordinary vertex pair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Trees and forests

A free tree here is a nonempty finite simple undirected graph that is connected and has no simple cycle. A forest is acyclic and may contain multiple components or isolated vertices. Empty forests are allowed. Empty graphs are outside the six-equivalence theorem below.

Every tree with $n \geq 2$ has a leaf. Take an endpoint $v$ of a longest simple path. A neighbor outside the path would extend it. An additional neighbor elsewhere on the path would create a cycle. Thus $v$ has only its adjacent path neighbor and has degree $1$.

Removing a leaf and its edge preserves connectivity because simple paths between other vertices cannot use a leaf internally. It also preserves acyclicity. Starting with one vertex and zero edges, induction gives $n - 1$ edges in a tree. For a forest with $c$ components, sum the component tree counts:

$$|E| = \sum_{i=1}^{c} (n_i - 1) = n - c.$$

The identity includes the empty forest, where $n = c = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Six equivalent tree conditions

For a nonempty finite simple undirected graph with $n$ vertices and $m$ edges, these conditions are equivalent:

1. The graph is a tree.
2. Each pair of vertices has exactly one simple path between them.
3. The graph is connected, and deleting any edge disconnects it.
4. The graph is connected, with $m = n - 1$.
5. The graph is acyclic, with $m = n - 1$.
6. The graph is acyclic, and adding any missing edge creates a cycle.

$(1) \implies (2)$**.** Connectivity gives a path. If two distinct simple paths existed, their first divergence and subsequent first meeting would delimit internally disjoint segments forming a cycle. This contradicts acyclicity.

$(2) \implies (3)$**.** An edge $\{u, v\}$ is itself the unique simple $u$-to-$v$ path. Deleting it destroys that reachability.



## Six equivalent tree conditions (continued)

$(3) \implies (1)$**.** If a cycle existed, deleting one of its edges would leave the other arc as a replacement in any path using that edge. The graph would remain connected, a contradiction. The leaf-induction count gives $(1) \implies (4)$ and $(1) \implies (5)$. Existence of a path and uniqueness of that path are separate assertions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Remaining directions and a spanning-tree construction

Every connected graph has a spanning tree. Start with one visited vertex, forming a set $S$. While $S \neq V$, connectivity supplies an edge from $S$ to its complement. Add that edge and its one new endpoint. Each step attaches one new vertex and creates no cycle. After $n-1$ steps, the selected edges form a spanning tree with $n-1$ edges. In particular, every connected graph has $m \geq n-1$.

$(4) \implies (1)$**.** The constructed spanning tree uses all $n-1$ graph edges. There are no extra edges, so the graph equals that tree.

$(5) \implies (1)$**.** The forest identity $m = n - c$, together with $m = n-1$, gives $c = 1$, proving connectivity.

$(1) \implies (6)$**.** Every new edge $\{u, v\}$ closes the tree's unique $u$-to-$v$ path into a cycle.

$(6) \implies (1)$**.** If two components existed, adding an edge between them could not create a cycle. This contradicts condition $(6)$, so the graph is connected.



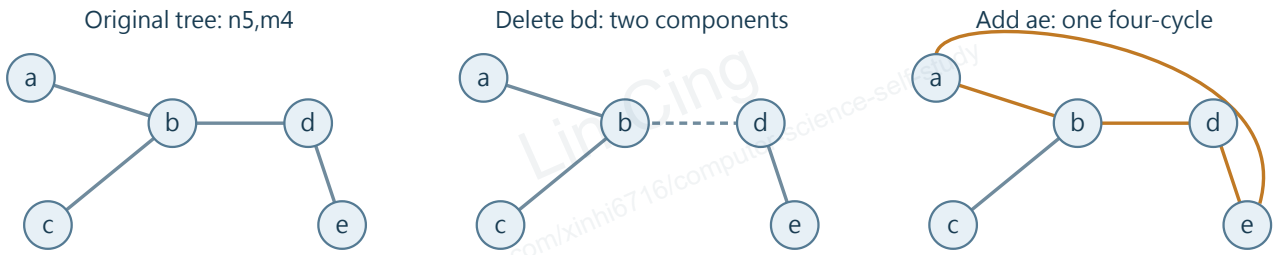
## Remaining directions and a spanning-tree construction (continued)

These implications establish all six equivalences. The equality $m = n-1$ alone is insufficient. A triangle plus an isolated vertex has $n = 4, m = 3$ but is not a tree. Acyclicity alone also does not ensure connectivity.

## The same tree under deletion and cycle creation

Let $V = \{a, b, c, d, e\}$ and $E = \{ab, bc, bd, de\}$, so $n = 5, m = 4$. The unique simple path from $a$ to $e$ is $a, b, d, e$, of length $3$. Deleting $bd$ gives components $\{a, b, c\}$ and $\{d, e\}$. Adding $ae$ creates the four-cycle $a, b, d, e, a$.

Same vertices: remove an edge to disconnect, add one to cycle




Middle dashed edge is deleted. Right orange cycle follows a–b–d–e–a via new edge ea.

A graph with a cycle can still have a spanning tree, although the graph itself is not a tree. Minimum-spanning-tree optimization adds a weight objective that is absent from the definition of a tree.



## Rooted and ordered trees

Choose a root $r$ in a free tree. Every nonroot vertex $x$ has as its parent the preceding vertex on its unique simple path from $r$. Its other, downward neighbors are its children. Here ancestors and descendants include the node itself; proper ancestors and descendants exclude it. The subtree rooted at $x$ contains all descendants of $x$.

Depth counts edges from the root to a node. A node's height counts edges to its deepest leaf. Leaves have height $0$, the root has depth $0$, and tree height is the maximum depth. Rooted degree counts children. Graph degree also counts the parent, so it is larger by $1$ for each nonroot node.

Root the preceding example at $b$. Its children are $a, c, d$, and $d$ has child $e$. Vertices $a, c, d$ have depth $1$, and $e$ has depth $2$. The heights are $2$ for $b$, $1$ for $d$, and $0$ for the others. Vertex $b$ has both graph degree and child count $3$. Vertex $d$ has graph degree $2$ and one child.

An ordered tree also orders each child list. Replacing the order $(a, c, d)$ at $b$ by $(c, a, d)$ keeps the rooted relationships but changes the ordered tree.

## Binary and positional trees retain empty slots

A binary tree is recursively either the empty tree `NIL` or a root with disjoint left and right binary subtrees. A single child in the left slot differs from a single child in the right slot. An ordered tree with one child has no such slot distinction, so an ordered tree of degree at most $2$ does not fully specify a binary tree.

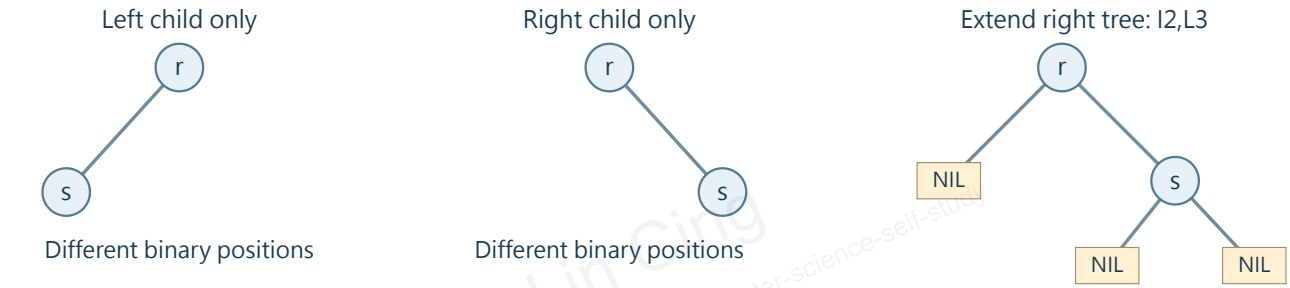
A positional $k$-ary tree has fixed slots $1, \ldots, k$, including meaningful empty positions. Replacing every missing binary child by an external `NIL` leaf gives a full binary tree. Each internal node has two children, and the left and right positions are preserved.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Binary and positional trees retain empty slots (continued)

Empty slots retain position: left and right differ




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For a nonempty binary tree, $n$ real nodes have $2n$ child slots. The $n - 1$ real parent-child edges occupy that many slots, leaving

$$2n - (n - 1) = n + 1$$

missing positions. The extended tree has $n$ internal nodes and $n + 1$ external leaves. Sharing one `NIL` sentinel object does not change the number of logical positions or require $n + 1$ actual allocations.

## Full, complete, and height counts

A full $k$-ary tree gives each internal node exactly $k$ children. Leaves may have different depths. With $I$ internal nodes and $L$ leaves, counting edges gives

$$kI = I + L - 1, \qquad L = (k-1)I + 1.$$

For binary trees, $L = I + 1$.

CLRS's appendix uses "complete $k$-ary" to require that all leaves also have depth $h$; this is often called "perfect" elsewhere. For $k \geq 2$, level $d$ has $k^d$ nodes, and

$$L = k^h, \qquad I = \sum_{d=0}^{h-1} k^d = \frac{k^h - 1}{k - 1}.$$

At $k = 3, h = 2$, the level sizes are $1, 3, 9$, so $I = 4, L = 9$, the total is $13$, and there are $12 = 3I$ edges. For $k = 1$, a chain has $I = h, L = 1$; do not divide by $k - 1 = 0$.



## Full, complete, and height counts (continued)

Heap chapters often use "complete binary" for a possibly partial final level filled from the left. Check which convention applies. A general nonempty binary tree of height $h$ has at most $2^{h+1} - 1$ nodes, giving

$$h \geq \lceil \log_2(n+1) \rceil - 1.$$

A chain has height $n - 1$. Without balance, binary-tree height need not be $\Theta(\log n)$.

## Sources

*Bibliography*

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Counting and probability

*Part VIII Appendix*

### Scope

Define sample objects and common denominators before using counting, conditional probability, Bayes' rule, and indicators. Derive geometric and binomial moments and tail bounds. Distinguish levels of independence, zero-probability and support boundaries, a centered deviation $r$, and a total threshold $t$.

### Prerequisites

Permutations and combinations, probability axioms, and random variables.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Appendix route

*Learning objectives*

- Count by products, disjoint sums, and complements.
- Derive expectation and variance for geometric and binomial distributions.
- Bound binomial tails using exponential moments.
- C.1 Counting
- C.2 Probability
- C.3 Discrete random variables
- C.4 The geometric and binomial distributions
- C.5 The tails of the binomial distribution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Counting starts with the sample object

Define what one outcome records. The sum rule requires disjoint cases. The product rule requires a known branch count after each prefix. The available choices may depend on that prefix, provided each level's count is fixed. Variable branching requires summing branches rather than multiplying unspecified averages.

From six students, two distinct office holders give $6 \cdot 5 = 30$ ordered assignments. An unordered two-person committee is counted twice by those assignments, so there are $15$ committees. Allowing one person to hold both offices gives $6^2 = 36$ assignments. These count three different objects.

There are $q^m$ length-$m$ strings over $q$ symbols, retaining positions and allowing repetition. Substrings are contiguous; subsequences may skip positions. Specify whether identical contents at distinct positions count separately. For an $n$-character string and $k \geq 1$, the number of positional length-$k$ substrings is $\max(0, n - k + 1)$. This need not equal the number of distinct contents.



## Permutations, combinations, and uniform overcounting

There are $n!$ full permutations of $n$ distinct records. Ordered selection of $k$ records without replacement has

$$P(n, k) = \frac{n!}{(n-k)!}, \qquad 0 \leq k \leq n.$$

Every unordered $k$-subset has exactly $k!$ orderings, giving

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

The case $k = 0$ gives one empty choice. Extend the coefficient to $0$ for $k < 0$ or $k > n$.

Distinct record identities still give $n!$ permutations when keys repeat. If only key strings matter and the multiplicities are $m_1, \ldots, m_s$, each string has exactly $\prod_{i=1}^{s} m_i!$ identity permutations. Thus the number of distinct strings is $n! / \prod_{i=1}^{s} m_i!$. Prove equal overcounting before dividing.

Complementation gives $\binom{n}{k} = \binom{n}{n-k}$. Partitioning subsets by whether they include a specified element gives Pascal's identity:

## Permutations, combinations, and uniform overcounting (continued)

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Binomial expansion and entropy bounds

In $(x+y)^n$, there are $\binom{n}{k}$ ways to choose the $k$ factor positions contributing $x$. Hence

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

Setting $x = y = 1$ gives $\sum_{k=0}^{n} \binom{n}{k} = 2^n$, also counting subsets by size. For $1 \le k \le n$,

$$\binom{n}{k} = \prod_{j=0}^{k-1} \frac{n-j}{k-j} \ge \left(\frac{n}{k}\right)^k,$$

because each factor is at least $n/k$. Also,

$$\binom{n}{k} \le \frac{n^k}{k!} \le \left(\frac{en}{k}\right)^k,$$

using $k! \ge (k/e)^k$.

Let $\alpha = k/n \in (0,1)$. The nonnegative $k$th term in the expansion of $(\alpha + 1 - \alpha)^n = 1$ is at most 1. Therefore

## Binomial expansion and entropy bounds (continued)

$$\binom{n}{k} \le \alpha^{-k}(1-\alpha)^{-(n-k)} = 2^{nH(\alpha)},$$

where $H(\alpha) = -\alpha \log_2 \alpha - (1-\alpha)\log_2(1-\alpha)$. Extend to the endpoints with $0 \log_2 0 = 0$, giving coefficient $1$. Since $H'(\alpha) = \log_2((1-\alpha)/\alpha)$, the maximum is $H(1/2) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Probability axioms and uniform outcomes

An elementary outcome records one complete distinguishable result. An event is a measurable subset of the sample space $S$. On the finite or countable discrete spaces here, every subset is an event. An uncountable space requires a specified event family closed under complements and countable unions; arbitrary subsets are not automatically measurable.

Probability is nonnegative, satisfies $P(S) = 1$, and is finitely or countably additive on pairwise disjoint events. It follows that

$$P(\varnothing) = 0, \qquad P(A^c) = 1 - P(A), \qquad A \subseteq B \implies P(A) \le P(B).$$

In a discrete space, $P(A) = \sum_{s \in A} P(\{s\})$.

The formula $P(A) = |A|/|S|$ requires finitely many equally likely elementary outcomes. Two independent fair bits have four ordered outcomes, each with probability $1/4$. Compressing them to the number of ones gives values $0, 1, 2$ with probabilities $1/4, 1/2, 1/4$, respectively. Those three values are not uniform.

## Union bounds need no independence

Disjoint decomposition into the $A$-only part, the intersection, and the $B$-only part gives

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \leq P(A) + P(B).$$

For many events, define

$$B_1 = A_1, \qquad B_i = A_i \setminus \bigcup_{j<i} A_j.$$

The $B_i$ are disjoint, have the same union as the $A_i$, and satisfy $B_i \subseteq A_i$. Additivity and monotonicity give

$$P\left(\bigcup_i A_i\right) = \sum_i P(B_i) \leq \sum_i P(A_i).$$

This also holds for countably many events and needs no independence. If the right side exceeds $1$, cap it at $1$. By contrast, factoring a general collection of intersection probabilities requires mutual independence.



## Union bounds need no independence (continued)

If three failure events each have probability at most $0.01$, their union has probability at most $0.03$, regardless of dependence. If the three events are identical and each has probability $0.01$, their union has probability $0.01$. The expression $1 - 0.99^3$ requires independence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Conditional probability and the chain rule

For $P(B) > 0$, conditional probability renormalizes the outcomes retained in $B$:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(A \cap B) = P(B)P(A \mid B).$$

Repeated application yields

$$P\left(\bigcap_{i=1}^{n} A_i\right) = P(A_1) \prod_{i=2}^{n} P\left(A_i \,\middle|\, \bigcap_{j=1}^{i-1} A_j\right).$$

Every conditioning prefix must have positive probability. A zero-probability prefix already makes the total intersection zero; subsequent undefined ratios must not be multiplied.

A bag contains three blue and two red balls. Drawing twice without replacement gives $P(\text{both blue}) = (3/5)(2/4) = 3/10$. The second draw's blue marginal remains $3/5$, but conditional on a first blue it is $1/2$. Thus the draws are dependent. Replacement with fresh uniform selection gives $(3/5)^2$ instead.



## Conditional probability and the chain rule (continued)

Knowing that at least one occurrence happened usually differs from knowing that a specified occurrence happened. Observation and selection rules belong to the model. Two remaining verbal cases need not each have probability $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Independence, disjointness, and pairwise independence

Events $A, B$ are independent when $P(A \cap B) = P(A)P(B)$. This includes probability-zero events. The equivalent expression $P(A \mid B) = P(A)$ additionally needs $P(B) > 0$. Disjoint events with positive probabilities are dependent. The empty event is independent of every event.

Mutual independence requires the intersection probability of every finite subcollection to factor into the product of its marginals. Pairwise independence checks only pairs and cannot justify factoring three or more events.

Let $U, V$ be independent fair bits, and define $W = U \operatorname{XOR} V$. The outcomes $000, 011, 101, 110$ each have probability $1/4$. Each of the pairs $(U, V)$, $(U, W)$, and $(V, W)$ has all four combinations equally likely, so every pair is independent. Yet

$$P(U = V = W = 1) = 0 \neq \frac{1}{8} = P(U = 1)P(V = 1)P(W = 1).$$

The three bits are not mutually independent.



## Independence, disjointness, and pairwise independence (continued)

Pairwise independent, but not mutually independent

| U | V | W=U XOR V | P(outcome) |
|---|---|---|---|
| 0 | 0 | 0 | 1/4 |
| 0 | 1 | 1 | 1/4 |
| 1 | 0 | 1 | 1/4 |
| 1 | 1 | 0 | 1/4 |

Each pair:00/01/10/11, each1/4

All three P(=1) marginals are1/2

But111 is absent from sample space

$0 \neq (1/2)^3 = 1/8$

Pair checks do not factor triple intersections. Variances add here; triple products do not.

## Total probability and Bayes denominators

Let the $B_i$ form a countable partition of $S$, with $P(B_i) > 0$. The event $A$ is the disjoint union of $A \cap B_i$, so

$$P(A) = \sum_i P(B_i) P(A \mid B_i).$$

When $P(A) > 0$, Bayes' rule gives

$$P(B_j \mid A) = \frac{P(B_j) P(A \mid B_j)}{P(A)}.$$

A monitor has fault prior $0.02$, sensitivity $0.9$, and false-positive rate $0.1$. Its alarm probability and posterior fault probability are

$$P(\text{alarm}) = 0.02 \cdot 0.9 + 0.98 \cdot 0.1 = 0.116,$$

$$P(\text{fault} \mid \text{alarm}) = \frac{0.018}{0.116} = \frac{9}{58} \approx 0.155172.$$

Sensitivity $0.9$ is not posterior probability $0.9$. Normal cases contribute $0.098$ false alarms to the denominator. If two tests are independent conditional on fault status, then



## Total probability and Bayes denominators (continued)

$$P(\text{two alarms}) = 0.02 \cdot 0.9^2 + 0.98 \cdot 0.1^2 = 0.026,$$

$$P(\text{fault} \mid \text{two alarms}) = \frac{0.0162}{0.026} = \frac{81}{130} \approx 0.623077.$$

Squaring the likelihoods requires that conditional independence assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Continuous uniform probabilities

Let $X$ be continuous uniform on $[a, b]$, where $a < b$. For $a \le c \le d \le b$,

$$P(c \le X \le d) = \frac{d - c}{b - a}.$$

Each singleton has probability $0$, while the whole interval has probability $1$. Countable additivity does not call for summing uncountably many singleton masses.

On $[2, 8]$, $P(3 \le X \le 5) = 2/6 = 1/3$, and removing endpoints changes nothing. The variable $Z = \lfloor X \rfloor$ has values $2, \ldots, 7$, each with probability $1/6$. The value $Z = 8$ occurs only when $X = 8$, with probability $0$. Do not assign equal weights to seven integer labels.

The following sections concern discrete random variables and probability mass. A continuous density may exceed $1$ even though every singleton has probability zero. Integrate a density over intervals rather than treating it as point probability.



## Discrete random variables and joint masses

A random variable $X : S \to \mathbb{R}$ assigns a numerical value to an outcome. The event $\{X = x\}$ is a preimage. Its probability mass function is $p_X(x) = P(X = x)$, with nonnegative values satisfying $\sum_x p_X(x) = 1$. The word “mass” distinguishes discrete probabilities from continuous density.

The joint mass $p(x, y) = P(X = x, Y = y)$ gives marginals

$$p_X(x) = \sum_y p(x, y), \qquad p_Y(y) = \sum_x p(x, y).$$

For $p_Y(y) > 0$, the conditional mass is $p(x, y)/p_Y(y)$. Independence means $p(x, y) = p_X(x) p_Y(y)$ for every pair $(x, y)$.

Let $A, B$ be independent fair four-sided dice, and let $M = \max(A, B)$. Then

$$P(M \le m) = \frac{m^2}{16}, \qquad P(M = m) = \frac{m^2 - (m - 1)^2}{16} = \frac{2m - 1}{16}.$$

For $m = 1, 2, 3, 4$, the masses are $1, 3, 5, 7$ divided by $16$. They are not uniform.

## Expectation need not be a possible outcome

Expectation is $E[X] = \sum_x x p_X(x)$. Finite support always permits evaluation. On countable signed support, $E[|X|] < \infty$ ensures a finite expectation that may be reordered. A nonnegative $X$ may have infinite expectation. The expression $\infty - \infty$ is not valid arithmetic.

For the dice maximum,

$$E[M] = \frac{1 + 6 + 15 + 28}{16} = \frac{50}{16} = \frac{25}{8},$$
$$E[M^2] = \frac{1 + 12 + 45 + 112}{16} = \frac{170}{16} = \frac{85}{8}.$$

The mean $3.125$ is outside the integer support and is still correct.

For any $g$ satisfying $E[|g(X)|] < \infty$, grouping outcomes by their $X$-preimages gives

$$E[g(X)] = \sum_x g(x) p_X(x).$$

There is no need to list the distinct values of $g(X)$ first. In general, $E[g(X)] \neq g(E[X])$; for the square function, the difference is variance.



## Linearity and indicators

For integrable $X, Y$, expand using their joint mass:

$$\begin{aligned} E[X + Y] &= \sum_x \sum_y (x + y) p(x, y) \\ &= \sum_x x \sum_y p(x, y) + \sum_y y \sum_x p(x, y) \\ &= E[X] + E[Y]. \end{aligned}$$

Absolute integrability justifies splitting and reordering. Independence was not used. Also $E[aX + b] = aE[X] + b$, and repeated application gives linearity for finite sums.

The indicator $I_A$ equals $1$ on $A$ and $0$ otherwise, so $E[I_A] = P(A)$. If $X$ counts events, then $X = \sum_i I_i$ gives $E[X] = \sum_i P(A_i)$, even with dependence. In a uniform permutation of $n$ distinct keys, each inversion-pair indicator has expectation $1/2$, giving total expected inversions $n(n-1)/4$.

For independent integrable $X, Y$, joint factorization gives

## Linearity and indicators (continued)

$$E[XY] = \left(\sum_x x p_X(x)\right)\left(\sum_y y p_Y(y)\right) = E[X]E[Y].$$

This step uses independence. If $X = Y$ is one fair bit, $E[XY] = 1/2 \neq 1/4$. Products of many variables generally require mutual independence, rather than pairwise independence alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Tail sums and Jensen

A nonnegative integer variable satisfies, outcome by outcome,

$$X = \sum_{i \geq 1} I_{\{X \geq i\}}.$$

Take expectations of finite truncations, then use monotone limits of nonnegative terms. This gives

$$E[X] = \sum_{i \geq 1} P(X \geq i),$$

possibly with value $\infty$. A signed variable cannot use this formula directly.

A convex function satisfies

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y), \qquad 0 \leq \lambda \leq 1.$$

For a finite support $x_1, \ldots, x_m$ with weights $w_i$, let $s = \sum_{i<m} w_i$. When $0 < s < 1$, combine the first $m-1$ points into $z = \sum_{i<m} w_i x_i / s$. Two-point convexity and induction give

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Tail sums and Jensen (continued)

$$\begin{aligned} f\left(\sum_{i=1}^{m} w_i x_i\right) &= f(sz + w_m x_m) \\ &\le s f(z) + w_m f(x_m) \\ &\le \sum_{i=1}^{m} w_i f(x_i). \end{aligned}$$

Zero weights may be omitted, and the single-point case is equality. Thus $f(E[X]) \le E[f(X)]$.

For the general integrable case, assume $f$ is convex on an interval containing the support and $\mu = E[X]$. A supporting line at an interior $\mu$ has

$$f(x) \ge f(\mu) + a(x - \mu).$$

Taking expectations gives $E[f(X)] \ge f(\mu) + a(E[X] - \mu) = f(\mu)$. At a degenerate endpoint, $X$ is concentrated there. The relevant expectations must be defined. Jensen is not permission to move an arbitrary function through expectation.



## Variance, covariance, and dependence

For $E[X^2] < \infty$ and $\mu = E[X]$,

$$\operatorname{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2 \ge 0.$$

The standard deviation is $\sigma = \sqrt{\operatorname{Var}(X)}$. Scaling gives $\operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X)$. For the dice maximum,

$$\operatorname{Var}(M) = \frac{85}{8} - \left(\frac{25}{8}\right)^2 = \frac{55}{64}, \qquad \sigma = \frac{\sqrt{55}}{8}.$$

Define $\operatorname{Cov}(X, Y) = E[XY] - E[X]E[Y]$. Expanding squares gives

$$\operatorname{Var}(X + Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X, Y),$$

$$\operatorname{Var}\left(\sum_i X_i\right) = \sum_i \operatorname{Var}(X_i) + 2\sum_{i<j} \operatorname{Cov}(X_i, X_j).$$

Pairwise independence and finite second moments suffice to make every covariance zero. Mutual independence is unnecessary for variance addition.

## Variance, covariance, and dependence (continued)

Identical marginals alone do not suffice. If all $n$ variables are the same fair bit, their sum equals $0$ or $n$ with equal probability and has variance $n^2/4$. Adding individual variances would give only $n/4$. Omitting covariance underestimates uncertainty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Markov and Chebyshev assumptions

Let $X \geq 0$ have finite mean, and let $t > 0$. The pointwise inequality

$$X \geq tI_{\{X \geq t\}}$$

gives $E[X] \geq tP(X \geq t)$ after taking expectations. Thus Markov's inequality is

$$P(X \geq t) \leq \frac{E[X]}{t}.$$

It requires no independence. It cannot be applied directly to signed variables, where positive and negative cancellation may hide a large positive tail.

For $Y$ with finite second moment, set $X = (Y - \mu)^2$ and use the threshold $a^2$, where $a > 0$. Markov gives Chebyshev's inequality:

$$P(|Y - \mu| \geq a) = P((Y - \mu)^2 \geq a^2) \leq \frac{\operatorname{Var}(Y)}{a^2}.$$

This is a two-sided bound. It also bounds either one-sided event, although it may be loose.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Markov and Chebyshev assumptions (continued)

For the maximum $M$, use $\mu = 25/8$, variance $55/64$, and $a = 2$. The bound is $55/256 \approx 0.214844$. The actual event contains only $M = 1$, with probability $1/16$. A bound is not an equality; values above $1$ may be capped at $1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Geometric and binomial distributions

A Bernoulli variable with parameter $p$ equals $1$ with probability $p$ and $0$ with probability $q = 1 - p$. Since $X^2 = X$, it has $E[X] = p$ and $\mathrm{Var}(X) = p - p^2 = pq$. The trial sequences below additionally assume mutual independence and a common $p$. Equal marginals alone are insufficient.

Here the geometric variable $G$ counts trials including the first success. Its support is $1, 2, \ldots$, with $0 < p \leq 1$. The event $G = k$ requires $k - 1$ failures followed by success, so

$$P(G = k) = q^{k-1}p.$$

At $p = 0$, success never arrives and $G = \infty$, outside this finite-valued distribution.

Counting earlier failures gives $F = G - 1$, with support $0, 1, \ldots$, mass $P(F = f) = q^f p$, and mean $q/p$. Do not use the trial mean $1/p$ for both conventions. At $p = 1$, $G \equiv 1$, $F \equiv 0$, and both variances are zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Geometric normalization and moments

The geometric masses normalize because

$$\sum_{k\ge 1} q^{k-1}p = \frac{p}{1-q} = 1.$$

For an integer $t \ge 0$, $P(G > t) = q^t$, hence $P(G \ge i) = q^{i-1}$. Tail summation gives

$$E[G] = \sum_{i\ge 1} q^{i-1} = \frac{1}{1-q} = \frac{1}{p}.$$

The weighted geometric identity yields

$$\sum_{k\ge 1} k^2 q^{k-1} = \frac{1+q}{(1-q)^3}, \qquad E[G^2] = \frac{1+q}{p^2}.$$

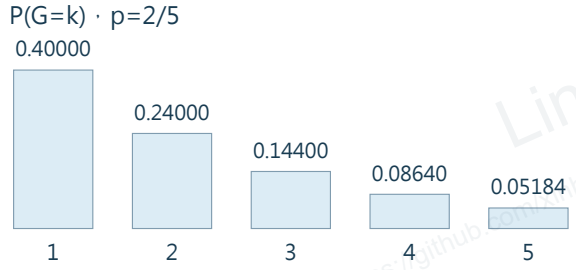
Subtracting the squared mean gives $\operatorname{Var}(G) = q/p^2$. Handle $q = 0$ directly as $G \equiv 1$, without dividing by $q$.



## Geometric normalization and moments (continued)

At $p = 2/5$, the first five masses are $2/5$, $6/25$, $18/125$, $54/625$, and $162/3125$. Their sum is $2882/3125$, and the remaining probability is $P(G > 5) = 243/3125$. The mean is $5/2$ and the variance is $15/4$. A mean of $2.5$ averages repeated complete experiments; it does not place a success on a fractional trial.

Fix the counting convention before masses and tails




One outcome: F → F → F → S

G=4: includes the success trial

F=G−1=3: failures only

No success in five: q⁵=243/3125

This tail remains beyond the first five bars

Axis counts trials including first success. E[G]=2.5,E[F]=1.5; the counts differ.

## Memorylessness and retry-model boundaries

For $0 < p < 1$ and integers $s, t \geq 0$,

$$P(G > s + t \mid G > s) = \frac{q^{s+t}}{q^s} = q^t = P(G > t).$$

Past failures do not change the next success rate because the trials are independent with a common $p$. A long failure streak does not make success due.

Changing retry probabilities, persistent shared faults, and sampling without replacement violate that model. Independent trials with unequal $p_i$ instead have tail $\prod_{i=1}^{t}(1 - p_i)$. Dependence may require conditional probabilities rather than $q^t$.

A fixed trial cost $c$ gives expected total cost $c/p$. Different costs for failure and success give

$$\text{total cost} = c_{\text{fail}}(G - 1) + c_{\text{success}},$$
$$E[\text{total cost}] = c_{\text{fail}}\frac{q}{p} + c_{\text{success}}.$$

Specify how costs relate to trials before applying linearity.



## Binomial mass, mean, and variance

For $n$ mutually independent trials with a common success probability $p$, let $X = \sum_{i=1}^{n} I_i \in \{0, \ldots, n\}$. A specified set of $k$ success positions has probability $p^k q^{n-k}$. There are $\binom{n}{k}$ disjoint such position sets, so

$$b(k; n, p) = \binom{n}{k} p^k q^{n-k}.$$

Normalization follows from $(p + q)^n = 1$.

Linearity gives $\mu = np$ without independence. Variance addition requires zero pairwise covariances, giving $\operatorname{Var}(X) = npq$. The complete binomial mass formula used mutual independence. Pairwise independence suffices for the mean and variance but does not imply the distribution.

At $n = 0$ or $p = 0$, $X$ is concentrated at $0$; at $p = 1$, it is concentrated at $n$. The endpoint convention $0^0 = 1$ represents an empty product.

## Binomial mass, mean, and variance (continued)

Mutually independent trials with unequal $p_i$ instead have a Poisson-binomial distribution, with mean $\sum_i p_i$ and variance $\sum_i p_i(1-p_i)$. Substituting the average $p$ into a binomial mass formula does not make the distributions equal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A fully checkable binomial table

For $n=8, p=1/4, q=3/4$, the common denominator is $4^8=65536$. The numerators are $\binom{8}{k}3^{8-k}$.

| $k$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| Numerator | 6561 | 17496 | 20412 | 13608 | 5670 | 1512 | 252 | 24 | 1 |

The numerators sum to $65536$. Their $k$-weighted sum is $131072$, giving $E[X]=2$. Their $k^2$-weighted sum is $360448$, giving $E[X^2]=11/2$ and variance $11/2-4=3/2$.

The exact tail is

$$P(X \ge 5)=\frac{1512+252+24+1}{65536}=\frac{1789}{65536}\approx 0.027298.$$

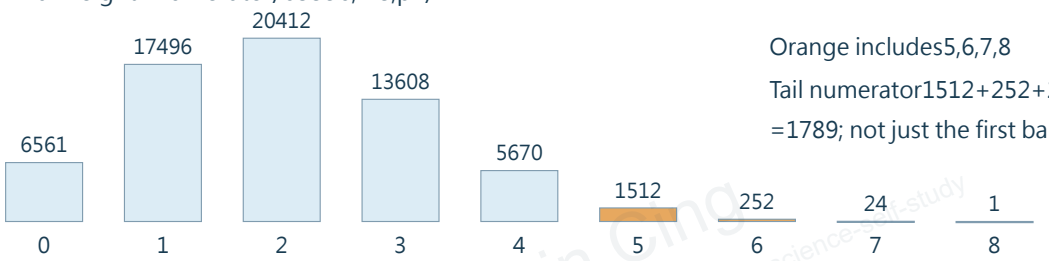
It includes every term through $k=8$, rather than only $b(5)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A fully checkable binomial table (continued)

A single mass differs from a tail; threshold5 includes5

Bar height=numerator/65536; n8,p1/4




Orange includes5,6,7,8

Tail numerator1512+252+24+1 =1789; not just the first bar

Numerators sum65536; mean2,variance1.5; P(X≥5)=1789/65536≈.027298.



## Modes follow adjacent ratios

For $0 < p < 1$ and $1 \le k \le n$,

$$\frac{b(k)}{b(k-1)} = \frac{(n-k+1)p}{kq}.$$

The ratio exceeds $1$ exactly when $k < (n+1)p$, and equals $1$ when $k = (n+1)p$.

If $(n+1)p$ is not an integer, the unique mode is $\lfloor (n+1)p \rfloor$. If it is an integer $m$, both $m-1$ and $m$ are modes. For $n = 8, p = 1/4$, the mode is $2$ and coincidentally equals the mean $2$. For $n = 4, p = 1/2$, $(n+1)p = 2.5$ gives mode $2$. For $n = 3, p = 1/2$, the modes are $1, 2$, while the mean $1.5$ has no integer probability bar.

The coefficient and entropy bound gives

$$b(k) \le \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.$$

Interpret endpoints by continuous extension or the $0^0$ convention. This bounds one mass, rather than a complete tail.

## Binomial tails and union bounds

For an integer-valued $X$, $P(X \ge a)$ sums from $\lceil a \rceil$, whereas $P(X > a)$ starts at $\lfloor a \rfloor + 1$. For an integer $k$, $P(X < k)$ ends at $k - 1$, unlike $P(X \le k)$.

For $X \sim \mathrm{Bin}(n, p)$, the event $X \ge k$ implies that some specified set of $k$ trials all succeed. Each such set has probability $p^k$. Applying the union bound over the $\binom{n}{k}$ sets gives

$$P(X \ge k) \le \binom{n}{k} p^k, \qquad 0 \le k \le n.$$

The events need not be disjoint. At $k = 0$, the bound is $1$; at $k > n$, the actual event is empty. Apply the same argument to failures $Y = n - X \sim \mathrm{Bin}(n, q)$ to get

$$P(X \le k) \le \binom{n}{k} q^{n-k}, \qquad 0 \le k \le n.$$

At $n = 8, p = 1/4, k = 5$, the upper bound is $56/1024 = 7/128 \approx 0.054688$, compared with the exact tail $1789/65536 \approx 0.027298$. Outcomes with six or more successes are counted repeatedly, which explains why the bound is usually strict.



## Deriving a geometric left-tail ratio bound

Assume $0 < p < 1$ and an integer $0 < k < np$. For $1 \le i \le k$,

$$\frac{b(i-1)}{b(i)} = \frac{iq}{(n-i+1)p} < \frac{kq}{(n-k)p} = \rho < 1.$$

Working backward gives $b(k-j) < \rho^j b(k)$ for $j = 1, \dots, k$. Consequently,

$$\begin{aligned} P(X < k) &= \sum_{i=0}^{k-1} b(i) < b(k) \sum_{j \ge 1} \rho^j \\ &= \frac{\rho}{1-\rho} b(k) = \frac{kq}{np - k} b(k). \end{aligned}$$

The denominator is positive by assumption. Substituting $k = np$ is invalid.

If $0 < k \le np/2$, the coefficient is at most $q \le 1$, so $L_k = P(X < k) < b(k)$. Since $L_{k+1} = L_k + b(k)$,

$$\frac{L_k}{L_{k+1}} = \frac{L_k}{L_k + b(k)} < \frac{1}{2}.$$

Moving one integer threshold farther into the left tail at least halves its probability.

## Right tails change both variables and thresholds

Let $Y = n - X \sim \mathrm{Bin}(n, q)$. For an integer $np < k < n$, we have $n - k < nq$. Apply the left-tail bound at threshold $n - k$:

$$\begin{aligned} P(X > k) &= P(Y < n - k) \\ &< \frac{(n-k)p}{nq - (n-k)} b(n-k; n, q) \\ &= \frac{(n-k)p}{k - np} b(k; n, p). \end{aligned}$$

If $k > (np + n)/2$, the left-tail halving result gives

$$P(X > k) < \frac{1}{2} P(X > k - 1).$$

This exchanges the roles of success and failure, including their thresholds.

At $n = 8, p = 1/4, k = 4$, the coefficient is

$$\frac{(8-4)(1/4)}{4-2} = \frac{1}{2}.$$



## Right tails change both variables and thresholds (continued)

Thus $P(X > 4) < 2835/65536 \approx 0.043259$, compared with the exact tail $1789/65536$. This improves the union bound in this example, but need not do so for every parameter choice.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Exponential moments turn tails into nonnegative quantities

Let $I_i$ be mutually independent Bernoulli indicators with parameters $p_i$. Write $X = \sum_i I_i$ and $\mu = \sum_i p_i$. For $\lambda > 0$, monotonicity and Markov give

$$P(X \geq t) \leq e^{-\lambda t} E[e^{\lambda X}].$$

Independence permits factorization. Using $1 + z \leq e^z$,

$$\begin{aligned} E[e^{\lambda X}] &= \prod_i E[e^{\lambda I_i}] \\ &= \prod_i [1 + p_i(e^\lambda - 1)] \\ &\leq \exp[\mu(e^\lambda - 1)]. \end{aligned}$$

Therefore $P(X \geq t) \leq \exp[\mu(e^\lambda - 1) - \lambda t]$. For $\mu > 0, t > \mu$, differentiating the exponent gives $\mu e^\lambda - t = 0$. The minimum occurs at $\lambda = \ln(t/\mu) > 0$, because the second derivative is $\mu e^\lambda > 0$. Substitution gives

$$P(X \geq t) \leq e^{-\mu} \left(\frac{e\mu}{t}\right)^t.$$



## Exponential moments turn tails into nonnegative quantities (continued)

Unequal $p_i$ are allowed; mutual independence is required. If $\mu = 0$, every indicator is zero almost surely. All positive-threshold tails are zero directly, without evaluating $\ln(t/0)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Centered r and total t are different thresholds

The looser CLRS form is

$$P(X-\mu \geq r) \leq \left(\frac{e\mu}{r}\right)^r, \qquad r > \mu > 0.$$

To derive it directly, for $\lambda > 0$ bound each centered exponential moment:

$$\begin{aligned} E[e^{\lambda(I_i - p_i)}] &= p_i e^{\lambda(1-p_i)} + (1-p_i)e^{-\lambda p_i} \\ &\leq p_i e^{\lambda} + 1 \leq \exp(p_i e^{\lambda}). \end{aligned}$$

Factor the moments using mutual independence and apply Markov:

$$\begin{aligned} P(X-\mu \geq r) &\leq e^{-\lambda r} \prod_i E[e^{\lambda(I_i - p_i)}] \\ &\leq \exp(\mu e^{\lambda} - \lambda r). \end{aligned}$$

The exponent is minimized at $\lambda = \ln(r/\mu) > 0$, giving the stated bound.

Here $r$ is the deviation above the mean, so the total threshold is $t = \mu + r$. In the preceding formula, $t$ counts total successes. With $\mu = 10, r = 40$, the event is $X \geq 50$. The two bounds are



## Centered r and total t are different thresholds (continued)

$$\left(\frac{e}{4}\right)^{40} \approx 1.947 \times 10^{-7},$$

$$e^{-10}\left(\frac{e}{5}\right)^{50} \approx 2.650 \times 10^{-18}.$$

Exchanging $40$ and $50$ changes the derivation. When $r$ only slightly exceeds $\mu$, the centered bound may exceed $1$; cap it at $1$. For an integer binomial variable, the exact tail starts at $\lceil \mu + r \rceil$, never a rounded-down threshold.

## Compare guarantees on one binomial tail

Let $X \sim \mathrm{Bin}(1000, 0.01)$, so $\mu = 10$, and consider $X \geq 50$.

| Bound | Calculation | Upper bound |
|---|---|---|
| Markov | $10/50$ | $0.2$ |
| Chebyshev | $9.9/40^2$ | $0.0061875$ |
| Centered, $r = 40$ | $(e/4)^{40}$ | $\approx 1.947 \times 10^{-7}$ |
| Total threshold, $t = 50$ | $e^{-10}(e/5)^{50}$ | $\approx 2.650 \times 10^{-18}$ |

The exact value remains

$$\sum_{k=50}^{1000} \binom{1000}{k} 0.01^k 0.99^{1000-k}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Compare guarantees on one binomial tail (continued)

An adjacent-ratio recurrence or log masses avoid floating-point overflow from large factorials. Markov requires nonnegativity and the mean. Chebyshev requires the true variance, which is $9.9$ here. Both exponential bounds require mutually independent indicators.

If all indicators instead share the same Bernoulli variable with parameter $0.01$, then $X$ is $0$ or $1000$, and $P(X \geq 50) = 0.01$. Markov still applies, but the binomial variance and exponential-product factorization do not. Equal marginals do not justify the binomial or Chernoff model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Sources

*Bibliography*

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix C.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Matrices

*Part VIII Appendix*

## Scope

Entries, dimensions, and field assumptions define matrix arithmetic. Complete elimination, rank factorizations, and inverse traces support proofs of equal row and column rank, determinant laws, and Gram positive definiteness. Real, complex, and finite fields require different inner-product conventions.

## Prerequisites

Vector spaces, linear maps, and matrix multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Appendix route

*Learning objectives*

- Apply matrix arithmetic and transposition.
- Relate rank, determinant, and inverse.
- State the conditions for symmetric positive definiteness and spectral reasoning.
- D.1 Matrices and matrix operations
- D.2 Basic matrix properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Matrices and matrix operations

Unless stated otherwise, dimensions are positive integers, entries are real, and vectors are columns. A rank-zero factorization allows zero intermediate columns and rows, with empty sums defined as $0$. A matrix $A \in \mathbb{R}^{m \times n}$ has $m$ rows and $n$ columns. The entry $a_{ij}$ uses the row index first and the column index second. Equality requires equal dimensions and equality of every corresponding entry. Addition and scalar multiplication are defined entrywise.

For example,

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \qquad A^T = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}$$

have dimensions $2 \times 3$ and $3 \times 2$, respectively. Transposition means $(A^T)_{ji} = a_{ij}$. Row-major and column-major storage are memory conventions; the mathematical transpose is defined by this index correspondence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Matrices and matrix operations (continued)

The vector $x = (1, -1, 2)^T$ is $3 \times 1$, while $x^T$ is $1 \times 3$. The standard basis vector $e_i$ has a $1$ in position $i$ and zeros elsewhere, with dimension specified by context. Zero matrices also require dimensions. The scalar $0$ and a $2 \times 3$ zero matrix are different types of object.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Exact zero positions in common square structures

| Type | Entry condition for an $n \times n$ matrix |
|---|---|
| Diagonal | $i \neq j \Rightarrow a_{ij} = 0$ |
| Identity $I_n$ | Diagonal entries are $1$; all others are $0$. |
| Tridiagonal | $\lvert i - j \rvert > 1 \Rightarrow a_{ij} = 0$ |
| Upper triangular | $i > j \Rightarrow a_{ij} = 0$ |
| Lower triangular | $i < j \Rightarrow a_{ij} = 0$ |
| Unit triangular | Triangular, with every diagonal entry equal to $1$. |
| Permutation | Exactly one $1$ in each row and each column; all other entries are $0$. |
| Real symmetric | $A = A^T$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exact zero positions in common square structures (continued)

These classes can overlap. The identity is diagonal, upper and lower triangular, symmetric, and a permutation matrix. A tridiagonal matrix need not be symmetric because opposite diagonals may differ. Unit triangular does not mean that every entry is $1$. A unit vector likewise need not be an all-ones vector.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Check dimensions and the contracted index

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the product $C = AB$ belongs to $\mathbb{R}^{m \times p}$, with

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}, \qquad 1 \le i \le m, \quad 1 \le j \le p.$$

The shared index $k$ is summed out; the outer indices $i, j$ remain. This is not entrywise multiplication.

Using the earlier $A$ and $B = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 4 & 0 \end{bmatrix}$ gives

| Entry | Calculation |
|---|---|
| $(AB)_{11}$ | $1 \cdot 2 + 2(-1) + 0 \cdot 4 = 0$ |
| $(AB)_{12}$ | $1 \cdot 1 + 2 \cdot 3 + 0 \cdot 0 = 7$ |
| $(AB)_{21}$ | $0 \cdot 2 + 1(-1) + 1 \cdot 4 = 3$ |
| $(AB)_{22}$ | $0 \cdot 1 + 1 \cdot 3 + 1 \cdot 0 = 3$ |



## Check dimensions and the contracted index (continued)

Thus $AB = \begin{bmatrix} 0 & 7 \\ 3 & 3 \end{bmatrix}$. The reverse product is also defined, but has size $3 \times 3$:

$$BA = \begin{bmatrix} 2 & 5 & 1 \\ -1 & 1 & 3 \\ 4 & 8 & 0 \end{bmatrix}.$$

The products differ even in size. Square products also need not commute; scalar commutativity does not transfer to matrix multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Check dimensions and the contracted index (continued)

Check sizes, then sum three aligned k terms

A : 2 rows × 3 columns

| | | |
|---|---|---|
| 1 | 2 | 0 |
| 0 | 1 | 1 |

×

B : 3 rows × 2 columns

| | |
|---|---|
| 2 | 1 |
| -1 | 3 |
| 4 | 0 |

=

C=AB : 2×2

| | |
|---|---|
| 0 | 7 |
| 3 | 3 |

C[1,2] = row1 of A · column2 of B = 1×1+2×3+0×0 = 7

Yellow row/column lengths3 share k. Output retains row1,column2; not entrywise multiplication.



## Implementable multiplication and exact cost

```
MULTIPLY(A[m,n], B[n,p])
    allocate independent C[m,p]
    for i=1..m
        for j=1..p
            s = A[i,1] * B[1,j]
            for k=2..n
                s = s + A[i,k] * B[k,j]
            C[i,j] = s
    return C
```

For $n \geq 1$, this uses $mnp$ scalar multiplications, $mp(n-1)$ additions, and $mp$ output writes, plus loop control. Constant-cost scalar arithmetic in the RAM model gives $\Theta(mnp)$ time, or $\Theta(n^3)$ for square matrices. Initializing $s = 0$ and doing $n$ additions instead adds $mp$ additions. That is a different exact count with the same asymptotic order.

Output entries must not alias input entries that will be read later, since writes could corrupt those reads. Arbitrary-precision integers and field elements require separate bit costs. Performing $n^3$ scalar multiplications does not guarantee $n^3$ time for arbitrary bit lengths.

## Associativity, distributivity, and compatible dimensions

Let $A$ be $m \times n$, $B$ be $n \times p$, and $C$ be $p \times q$. For every valid output index $(i, j)$,

$$\begin{aligned}((AB)C)_{ij} &= \sum_{k=1}^{p} \sum_{\ell=1}^{n} a_{i\ell} b_{\ell k} c_{kj} \\ &= \sum_{\ell=1}^{n} a_{i\ell} \left( \sum_{k=1}^{p} b_{\ell k} c_{kj} \right) = (A(BC))_{ij}.\end{aligned}$$

Reordering is valid because the sums are finite. Distributivity follows from scalar distributivity at each entry. Identity dimensions must match: $I_m A = A I_n = A$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Transposition reverses factor order

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, compare position $(j, i)$ on both sides of $(AB)^T = B^T A^T$:

$$\begin{aligned}((AB)^T)_{ji} &= \sum_{k=1}^{n} a_{ik} b_{kj}, \\ (B^T A^T)_{ji} &= \sum_{k=1}^{n} (B^T)_{jk} (A^T)_{ki} = \sum_{k=1}^{n} b_{kj} a_{ik}.\end{aligned}$$

The entries agree by **scalar** commutativity. The matrix product itself need not commute; transposition reverses the factor order.

For real symmetric matrices $A, B$ of the same size, their sum and difference remain symmetric. Their product is symmetric if and only if $AB = BA$, since $(AB)^T = BA$. Two symmetric inputs alone do not imply a symmetric product.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Matrix-vector, inner, and outer products

Write the columns of $A \in \mathbb{R}^{m \times n}$ as $a_1, \dots, a_n$. For $x \in \mathbb{R}^n$, the product $Ax = \sum_{j=1}^{n} x_j a_j$ is an $m$-vector. The earlier example gives

$$Ax = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix},$$

which matches the row dot products.

For equal-length real vectors $u, v$, the inner product $u^T v$ is a scalar and the outer product $uv^T$ is a matrix. With $u = (1, 2)^T$ and $v = (3, -1)^T$,

$$u^T v = 1, \qquad uv^T = \begin{bmatrix} 3 & -1 \\ 6 & -2 \end{bmatrix}.$$

Column $j$ is $v_j u$, so the rank is at most one. If both vectors are nonzero, at least one column is a nonzero multiple of $u$ and the rank is one. If either vector is zero, the rank is zero.

The real norm is $\|x\|_2 = \sqrt{x^T x}$. Complex vectors require $x^* x = \sum_i |x_i|^2$, where $*$ denotes conjugate transpose. For $x = [i]$, ordinary transpose gives $x^T x = -1$, which is not squared length.



## Permutations act differently on rows and columns

For

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix},$$

one has $Px = (x_3, x_1, x_2)^T$. If $A$ has three rows, $PA$ selects its old rows in the order $3, 1, 2$. In $(PA)_{ij} = \sum_{k=1}^{3} p_{ik} a_{kj}$, each row of $P$ selects one term.

For a matrix $M$ with three columns, $MP$ selects its old columns in the order $2, 3, 1$, because the columns of $P$ are $e_2, e_3, e_1$. Derive the direction from entries or basis columns, rather than memorizing one order for both sides.

Distinct basis columns give $P^T P = I_3 = P P^T$, so $P^{-1} = P^T$. Products of permutation matrices remain permutation matrices because composing two basis reorderings is a bijection.

## Rank and null vectors

Vectors $v_1, \ldots, v_n$ are linearly independent if $\sum_{j=1}^{n} c_j v_j = 0$ forces every coefficient $c_j = 0$; otherwise they are dependent. Column rank is the largest number of independent columns, and row rank is defined analogously. Zero matrices have rank zero.

Since $Ax = \sum_j x_j a_j$, column independence is equivalent to $Ax = 0$ having only the solution $x = 0$. A nonzero solution is a nonzero null vector. The null space always contains $0$; absence of a nonzero null vector does not mean the null space contains no vectors.

The matrix

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$$

has eight nonzero entries but rank two. Its third column is the sum of the first two, which are independent. A null vector is $(-1, -1, 1)^T$. Rank is neither the nonzero-entry count nor the number of nonzero entries on the original diagonal.



## Elimination operations and intermediate states

Row swaps, nonzero row scaling, and adding a multiple of another row are invertible operations. Each is left multiplication by an invertible elementary matrix. They preserve column-dependence coefficients and row span.

For $M$, the complete elimination trace is

$$M \xrightarrow{R_2 \leftarrow R_2 - 2R_1} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} \xrightarrow{R_2 \leftrightarrow R_3} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{R_1 \leftarrow R_1 - 2R_2} R = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

Pivot columns $1, 2$ leave $x_3$ free, with $x_1 = x_2 = -x_3$. Hence the null space is $\{t(-1, -1, 1)^T : t \in \mathbb{R}\}$. The two nonzero rows have distinct leading positions and are independent. The nonpivot column is the sum of the pivot columns.

## Elimination operations and intermediate states (continued)

M=BC: dimension2 from original pivot columns

Original M:3×3,rank2

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 4 | 6 |
| 0 | 1 | 1 |

=

B: original pivot columns

| | |
|---|---|
| 1 | 2 |
| 2 | 4 |
| 0 | 1 |

×

C: column coordinates

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 1 |

M.column3=column1+column2, so C.column3=$(1,1)^{T}$.

B retains original vectors. Reduced unit columns give coordinates and cannot replace B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Gauss–Jordan without assuming diagonal pivots

```
R = copy(A[m,n]); r = 0; pivots = empty list
for j=1..n
    find t in r+1..m with R[t,j] != 0
    if none: continue
    r = r+1; swap rows r,t
    divide row r by R[r,j]
    for i=1..m with i != r
        a = R[i,j]
        row i = row i - a * row r
    append j to pivots
return R, pivots
```

Before each step, the first $r$ pivot columns are distinct unit columns, and unused rows are zero in those columns. A new pivot comes from a remaining row. Normalizing it and eliminating elsewhere preserves old pivot columns because its old pivot coordinates are zero. Skip a column only when every remaining row is zero there. The finite loops produce $r$ pivots and zero rows in $O(mn\min(m,n))$ field operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Gauss–Jordan without assuming diagonal pivots (continued)

Exact comparison with zero follows the field arithmetic. Floating-point arithmetic needs pivoting, scale, and tolerance analysis. A fixed rule such as $|\text{pivot}| < 10^{-12}$ is not a universally correct mathematical definition of rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why row rank equals column rank

Let $R = EA$ be the reduced row echelon form, with $E$ invertible. Applying the same reversible map to every column preserves exactly their dependence coefficients. The $r$ pivot columns of $R$ are independent, and every other column is their combination. Thus the column rank of $A$ is $r$.

Invertible row operations preserve row span. The $r$ nonzero rows of $R$ are independent: their successive pivot coordinates force each coefficient in a dependence relation to vanish. Thus the row rank of $A$ is also $r$. Write $\text{rank}(A)$ for this common value, between $0$ and $\min(m, n)$.

The nullity is $n - r$. Free coordinates uniquely determine the pivot coordinates. Set one free variable at a time to $1$ and the others to $0$ to construct a null-space basis. These vectors are independent by their free coordinates and span every solution. This proves the dimension statement generally, beyond the example $M$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rank factorization and product bounds

Choose the $r$ original pivot columns of $A$ to form $B \in \mathbb{R}^{m \times r}$. Put the coordinates of every original column relative to $B$ in $C \in \mathbb{R}^{r \times n}$. Then $A = BC$, the columns of $B$ are independent, and $C$ contains identity columns. Conversely, if $A = UV$ with $s$ intermediate columns, every column of $A$ lies in the span of $U$, so $\operatorname{rank}(A) \leq s$. Therefore $r$ is the minimum possible intermediate dimension.

For $M$,

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \qquad M = BC.$$

The pivot vectors of $R$ are not the original vectors of $M$; row operations changed their entries.

For compatible $A, D$, factor $A = B_r C_r$ to get $AD = B_r(C_r D)$, so $\operatorname{rank}(AD) \leq \operatorname{rank}(A)$. Factoring $D = U_s V_s$ similarly gives $AD = (AU_s)V_s$ and the bound by $\operatorname{rank}(D)$. Thus

$$\operatorname{rank}(AD) \leq \min\{\operatorname{rank}(A), \operatorname{rank}(D)\}.$$



## Rank factorization and product bounds (continued)

For an invertible left factor $P$, the identity $A = P^{-1}(PA)$ proves the reverse inequality, giving $\operatorname{rank}(PA) = \operatorname{rank}(A)$. Invertible right multiplication is analogous.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Inverse existence and uniqueness

An inverse $B$ of a square $n \times n$ matrix $A$ satisfies $AB = I_n = BA$. If $B, C$ are both inverses, $B = B(AC) = (BA)C = C$. For invertible $A, B$, the product inverse is $(AB)^{-1} = B^{-1}A^{-1}$, as multiplication on both sides verifies. Transposing the two inverse equations gives $(A^{-1})^T = (A^T)^{-1}$.

Square full rank is equivalent to no nonzero null vector, hence to injectivity of $x \mapsto Ax$. The $n$ independent columns form a basis of $n$-space. Each $e_j$ therefore has a unique solution $Ab_j = e_j$. Assemble these columns into $B$ to obtain $AB = I_n$. Then $A(BA - I_n) = 0$ and injectivity imply $BA - I_n = 0$ columnwise. Thus full rank is equivalent to invertibility. Conversely, multiplying $Ax = 0$ by the inverse gives $x = 0$.

A rectangular one-sided inverse need not be two-sided. For $A = \begin{bmatrix} 1 & 0 \end{bmatrix}$ and $B = (1, 0)^T$, one has $AB = I_1$ but $BA = \operatorname{diag}(1, 0) \neq I_2$. The square premise cannot be dropped.



## A fully checkable inverse trace

Let $H = \begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix}$. Apply each row operation to both halves of $[H \mid I_2]$:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 3 & 2 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - (3/2)R_1} \left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1/2 & -3/2 & 1 \end{array}\right].$$

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1/2 & -3/2 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow 2R_2} \left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

## Finish and verify the inverse

Subtract row two from row one, then normalize row one:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1 & -3 & 2 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1 - R_2} \left[\begin{array}{cc|cc} 2 & 0 & 4 & -2 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

$$\left[\begin{array}{cc|cc} 2 & 0 & 4 & -2 \\ 0 & 1 & -3 & 2 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1/2} \left[\begin{array}{cc|cc} 1 & 0 & 2 & -1 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

The right half is $H^{-1} = \begin{bmatrix} 2 & -1 \\ -3 & 2 \end{bmatrix}$. Both checks give the identity:

$$HH^{-1} = \begin{bmatrix} 4-3 & -2+2 \\ 6-6 & -3+4 \end{bmatrix} = I_2, \qquad H^{-1}H = \begin{bmatrix} 4-3 & 2-2 \\ -6+6 & -3+4 \end{bmatrix} = I_2.$$

Elimination applies $E$ to the left half while turning the right half into $E$. When $EH = I_2$, $E$ is the inverse. Without all $n$ left-side pivots for an $n \times n$ matrix, the right half cannot simply be declared an inverse.



## Determinants, minors, and cofactors

For a square matrix $A$, the minor matrix $A[ij]$ removes row $i$ and column $j$. The cofactor is $C_{ij} = (-1)^{i+j} \det A[ij]$. For $n = 1$, define $\det[a] = a$. The optional $0 \times 0$ convention $\det(\varnothing) = 1$ unifies cofactor formulas. Expansion along row $i$ gives $\det A = \sum_{j=1}^{n} a_{ij} C_{ij}$.

For $N = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 2 \\ 0 & -1 & 1 \end{bmatrix}$, first-row signs $+, -, +$ give

$$\det N = 2\big(3 \cdot 1 - 2(-1)\big) - 1(1 \cdot 1 - 2 \cdot 0) + 0 = 10 - 1 = 9.$$

A minor is a matrix, its determinant is a scalar, and a cofactor adds a sign. These are distinct objects.

In general, with $S_n$ the permutations of $\{1, \ldots, n\}$,

$$\det A = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} a_{i,\sigma(i)}.$$

Each term selects one entry per row and per column, with a permutation-parity sign. Grouping terms by the selected column in a fixed row gives cofactor expansion.

## Reasons for determinant laws

The permutation formula is linear separately in every column. With two equal columns, pair permutations that exchange those columns; the terms cancel, giving determinant zero. Swapping columns changes the sign. The same arguments apply to rows, and the bijection $\sigma \mapsto \sigma^{-1}$ gives $\det(A^T) = \det A$. Scaling a row by $\alpha$ scales the determinant by $\alpha$. Adding a multiple of another row leaves it unchanged because the extra term from multilinearity has a repeated row.

For any alternating multilinear scalar function $F$ of the columns, expand each column in the standard basis. Repeated basis indices vanish. The remaining permutation terms satisfy

$$F(e_{\sigma(1)}, \dots, e_{\sigma(n)}) = \operatorname{sgn}(\sigma) F(I_n).$$

Thus $F(B) = F(I_n) \det B$. Fix $A$ and take $F(B) = \det(AB)$. Each product column is $A$ times a column of $B$, so $F$ is alternating multilinear and $F(I_n) = \det A$. This proves $\det(AB) = \det A \det B$ without assuming $A$ invertible. If $\det A = 0$, the function is identically zero.



## Determinants and singularity beyond the $2 \times 2$ formula

For an upper-triangular matrix, a nonzero permutation term requires $\sigma(i) \geq i$ for every $i$. Since $\sum_i \sigma(i) = \sum_i i$, every inequality is an equality. Only the diagonal term remains, so the determinant is the diagonal product. The lower-triangular case follows symmetrically or by transpose. A square echelon matrix with a zero row has determinant zero; with all $n$ pivots nonzero, it has a nonzero diagonal product.

Swaps and nonzero scalings multiply the determinant by nonzero factors; row addition preserves it. Therefore

$$\begin{aligned} \det A = 0 &\iff \text{a pivot is missing} \\ &\iff \operatorname{rank}(A) < n \\ &\iff A \text{ is singular.} \end{aligned}$$

A nonzero determinant proves invertibility, without guaranteeing good numerical conditioning. For $A = \operatorname{diag}(1, \varepsilon)$ with $\varepsilon \neq 0$, exact full rank holds but the inverse contains $1/\varepsilon$. Small $\varepsilon$ can amplify errors. Stable factorizations usually solve $Ax = b$ without explicitly forming the inverse. Mathematical existence and numerical reliability are different issues.

## Real symmetric positive definiteness and Gram matrices

Here positive definite means a **real symmetric** matrix $S$ with $x^T S x > 0$ for every nonzero real $x$. Positive semidefinite replaces $>$ with $\geq$. Without symmetry, a quadratic form sees only $(S + S^T)/2$; positivity alone does not imply $S = S^T$.

For real $A \in \mathbb{R}^{m \times n}$, the Gram matrix $G = A^T A$ is symmetric and

$$x^T G x = (Ax)^T (Ax) = \|Ax\|_2^2 \geq 0.$$

Equality holds exactly when $Ax = 0$. Thus $G$ is positive definite exactly when $A$ has no nonzero null vector, or equivalently has full column rank. This is impossible when $n > m$. A merely nonzero $A$ is insufficient.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Check Gram positivity on all vectors

For

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad X^T X = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

the vector $x = (s, t)^T$ gives $x^T X^T X x = s^2 + t^2 + (s + t)^2 > 0$ unless $s = t = 0$.

For

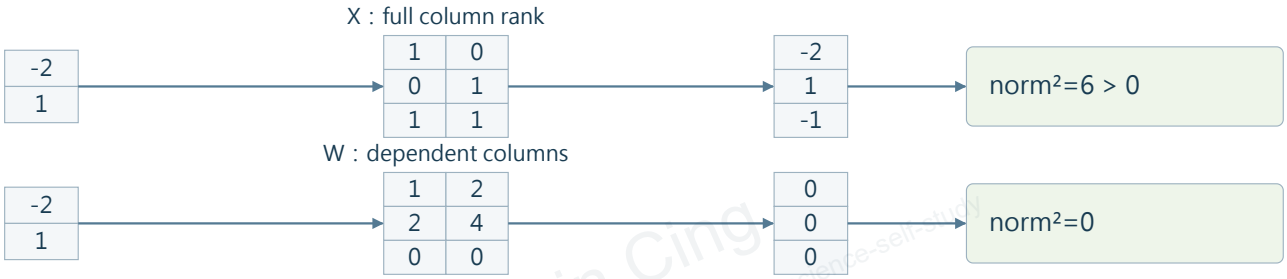
$$W = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 0 \end{bmatrix}, \qquad W^T W = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix},$$

the quadratic form is $5(s + 2t)^2$. The nonzero vector $(-2, 1)^T$ has $Wx = 0$, so this Gram matrix is only positive semidefinite.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Check Gram positivity on all vectors (continued)

Compare $x^TA^TAx$=norm(Ax)² at two ranks




Same nonzero $x=(-2,1)^T$. Positive definiteness needs all nonzero x; one null vector refutes W.



## Complex and finite fields change some premises

Field arguments for multiplication, inverses, rank, and determinants extend to $\mathbb{C}$ and prime fields $\mathbb{F}_p$. Real order and Euclidean inner products do not transfer unchanged. Arithmetic modulo a composite number forms a ring in which nonzero elements may be noninvertible. For example, $2$ modulo $6$ has no inverse, so pivot normalization and field-rank arguments require reconsideration.

The complex conjugate transpose is $A^*$. A Hermitian matrix satisfies $S^* = S$. The Gram matrix $A^*A$ is positive semidefinite and is positive definite for full column rank because

$$x^*A^*Ax = \sum_i |(Ax)_i|^2.$$

Ordinary $A^TA$ need not be positive. For $A = [i]$, one has $A^TA = [-1]$ but $A^*A = [1]$.

For real or complex square $A$, the algebraic transpose identity is $\det(A^TA) = (\det A)^2$. The conjugate version is $\det(A^*A) = |\det A|^2 \geq 0$. The first expression implies nonnegativity only for real entries. Distinguish algebraic transposition from conjugation in inner products.

## Sources

*Bibliography*

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix D.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study